

TAG

ĐỊNH VỊ BẰNG DẤU HIỆU NHÂN TẠO

CÂY TRI THỨC MARKER

Trình bày theo mạch

vấn đề → giải pháp → giả định → giá phải trả

*“Mục tiêu 5 mm / 0,5° không bị giới hạn bởi thuật toán,
mà bởi ngân sách sai số.”*

luận điểm trung tâm của cây — Phần C, chương 15

Năm phần A · Nguyên lý và toán học B · Giải thuật và phương pháp
C · Ngân sách sai số D · Framework và công cụ E · Lộ trình

Bản gộp toàn cây — Ngày 14 tháng 9 năm 2026

Bản đồ cây

TÓM TẮT

Đây là bản đồ điều hướng của cả cây: nó nói cây gồm gì, vì sao được chia thành năm phần theo thứ tự đó, và bốn luận điểm sẽ gặp lại ở gần như mọi chương. Đọc xong bạn biết nên bắt đầu từ đâu, chương nào là xương sống phải tự dẫn lại công thức, chương nào chỉ cần biết là có. Nếu chỉ nhớ một điều: *mục tiêu 5 mm / 0,5° không bị giới hạn bởi thuật toán mà bởi ngân sách sai số* — chủ yếu là hình học của constellation, độ chính xác cơ khí của tấm marker, và calibration. Toàn bộ cây được dựng để làm rõ câu đó, để chỉ ra chỗ nó dẫn tới quyết định thiết kế cụ thể, và để chỉ ra khi nào nó hỏng.

Cây này được dựng từ một lộ trình tự soạn, với một mục tiêu hẹp và một bài toán cụ thể: *đưa một robot đi động cập vào dock với sai lệch dưới 5 mm và 0,5°, dùng marker thị giác*. Nó không nhằm liệt kê cho đủ các họ marker đã có, cũng không nhằm là một khảo sát về fiducial marker nói chung. Sự khác biệt giữa hai mục tiêu ấy quyết định toàn bộ cách trình bày phía sau, nên đáng nói rõ ngay từ đầu: mọi chương ở đây đều được viết để trả lời câu hỏi “*cái này ăn bao nhiêu milimét trong ngân sách của tôi*”, chứ không phải câu hỏi “*phương pháp này có gì hay*”.

Luận điểm trung tâm: ngân sách sai số, không phải thuật toán (The Error Budget, Not the Algorithm)

Người mới vào bài toán docking bằng marker gần như luôn đi theo cùng một con đường. Chọn một họ tag — thường là AprilTag vì nó nổi tiếng và có mã nguồn tốt. Dán một tag lên dock. Gọi solvePnP. Nhìn con số pose trả về, thấy nó nhảy, rồi bắt đầu một chuỗi dài những việc mà ai cũng làm: thêm bộ lọc trung bình, thử tag lớn hơn, thử detector khác, thử một thuật toán PnP khác, đọc về lọc Kalman. Chuỗi ấy tiêu tốn nhiều tuần và nó *cải thiện* kết quả, đủ để duy trì cảm giác đang đi đúng hướng. Nhưng nó thường dừng lại ở quanh một, hai độ sai lệch góc, và không ai hiểu vì sao không xuống thấp hơn được.

Cách nhìn đó sai, và nó sai theo một kiểu tổn kém đặc biệt: nó tốn tiền gia công. Bài toán ở đây không phải bài toán thuật toán mà là bài toán **ngân sách sai số** (*error budget*) — cùng loại bài toán mà kỹ sư đo lường làm khi thiết kế một hệ thống đo, và có chuẩn quốc tế riêng cho nó [1]. Sai lệch cuối cùng ở mũi robot là tổng hợp của một chuỗi: nhiều định vị góc trên ảnh, hình học của tập marker, sai số cơ khí của tấm in marker, sai số intrinsic của camera, sai số hand-eye giữa camera và thân robot, và cuối cùng là sai số bám của bộ điều khiển. Mỗi khâu đóng góp một lượng, và *lượng nhỏ nhất trong số đó không quyết định gì cả* — khâu lớn nhất mới quyết định. Thay một detector tốt hơn để giảm nhiễu góc từ 0,2 px xuống 0,1 px là vô nghĩa nếu tấm marker của bạn cong 1 mm.

Điều này không chỉ là một nhận xét đẹp. Nó có một hệ quả thực dụng ngay, và hệ quả ấy đảo ngược thứ tự công việc: *phép tính đầu tiên phải làm không phải là viết code mà là lập ngân sách*. Với các con số hình học của hệ thống dự kiến, ta tính được ngay sai số vị trí và sai số góc kỳ vọng, trước khi mua một con camera nào. Chương 15 làm đúng phép tính đó và nó cho ra một kết luận mà tôi tin là bất ngờ với phần lớn người đọc: *sai số tịnh tiến gần như luôn đạt, còn sai số xoay mới là nút thắt*, và cách chữa nút thắt ấy là hình học — tăng baseline, phá tính đồng phẳng — chứ không phải thuật toán tinh vi hơn.

TRỰC GIÁC

Hình dung chuỗi đo như một sợi dây xích treo vật. Sức chịu của xích bằng sức chịu của mắt yếu nhất, không phải trung bình của các mắt. Bạn có thể thay năm mắt xích bằng thép tốt nhất thế giới, và nếu mắt thứ sáu vẫn là dây thép buộc thì tải trọng cho phép không đổi một gram.

Ngân sách sai số là bản kê khai sức chịu của từng mắt. Việc đầu tiên không phải là cải thiện mắt nào cả, mà là *cân từng mắt* để biết mắt nào yếu nhất. Gần như mọi lần, mắt yếu nhất không nằm ở chỗ người ta nghĩ: nó không phải thuật toán PnP, mà là độ phẳng của tấm nhôm dán tag, hoặc là chuyện constellation của bạn đồng phẳng nên góc gần như không quan sát được.

Và có một mắt xích mà ta hay quên hẳn: pose bạn *thực sự* cần là $T_{\text{dock}}^{\text{base}}$ — từ dock sang thân robot. Marker chỉ là một mắt trung gian. Mỗi phép biến đổi trên chuỗi từ tag tới dock frame, rồi từ camera tới base frame, đều là một mắt xích có sai số riêng, và không mắt nào trong số đó được cải thiện bởi một detector tốt hơn.

Bốn hệ quả kéo theo, và chúng chi phối cả cây (Four Consequences)

Luận điểm trung tâm sinh ra bốn hệ quả cụ thể. Chúng xuất hiện lại ở những chỗ tưởng như không liên quan, và nhận ra chúng là dấu hiệu đã đọc đúng.

Thứ nhất, **translation dễ, rotation khó**. Với các con số điển hình của một bài toán docking, sai số vị trí ngang cỡ phần mười milimét và sai số dọc trục cỡ một milimét — thoải mái trong ngân sách. Còn sai số xoay ước lượng từ *một tag phẳng nhìn gần vuông góc* dễ dàng vượt một độ. Chương 5 dẫn ra vì sao: độ nhạy của phép đo theo góc nghiêng tỉ lệ với chính sin của góc nghiêng ấy, nên ở tư thế fronto-parallel — đúng tư thế mà một robot tiếp cận dock thẳng trục sẽ ở — thông tin về góc gần như biến mất. Đây là một bài toán *observability*, không phải bài toán nhiễu.

Thứ hai, **hình học thắng thuật toán**. Cách chữa nút thắt xoay là dùng nhiều marker đặt xa nhau và *không đồng phẳng*. Sai số góc từ một constellation có baseline B xấp xỉ sai số vị trí ngang chia cho B , nên nó cải thiện tuyến tính theo baseline — một đại lượng bạn mua được bằng cách đặt hai tấm tag cách nhau nửa mét, không phải bằng cách viết thêm code. Phá tính đồng phẳng thì còn làm một việc thứ hai, quan trọng không kém: nó khử tận gốc hiện tượng hai nghiệm của target phẳng (chương 4).

Thứ ba, **cơ khí là sàן sai số**. Nếu vị trí tương đối giữa các tag trong constellation sai 1 mm so với mô hình bạn nạp vào PnP, thì không phép xử lý nào lấy lại được milimét ấy. Tag in giấy dán lên bề mặt cong sai hơn thế. Hệ quả thực hành rất cụ thể và nó nằm ở chương 12: *đừng tin bản vẽ CAD, hãy tự đo lại constellation bằng bundle adjustment* từ nhiều ảnh chụp ở nhiều góc [2]. Đây là bước rẻ và nó thường là bước quyết định giữa đạt và không đạt.

Thứ tư, và đây là mero lớn nhất của cả cây: **docking cần repeatability, không cần accuracy**. Robot không cần biết nó đang ở đâu trong hệ toạ độ tuyệt đối của thế giới; nó cần trở về *đúng chỗ nó đã từng ở*. Hai yêu cầu ấy khác nhau về bản chất và tiêu chuẩn robot công nghiệp phân biệt rõ chúng [3]. Nếu ta ghi lại pose (hoặc thẳng luôn là toạ độ góc trên ảnh) tại vị trí dock đúng, rồi điều khiển để triệt tiêu *sai lệch tương đối* so với bản ghi đó, thì mọi sai số hệ thống — intrinsic sai, kích thước tag sai, hand-eye sai — xuất hiện ở cả pose hiện tại lẫn pose mục tiêu và triệt tiêu lẫn nhau. Đây là nội dung của chương 13, và nếu bài toán của bạn đúng là “lập lại một vị trí”, thì đó là con đường ngắn nhất tới 5 mm.

Vì sao chia thành năm phần (The Five Parts)

Năm phần không phải năm chủ đề song song mà là năm tầng câu hỏi nối tiếp. Phần A trả lời *sai số sinh ra ở đâu và lan truyền thế nào*, và nó là phần nền: không có nó thì mọi con số ở Phần C là phép thuật. Phần B trả lời *có những cách nào để giảm từng nguồn sai số, và mỗi cách trả giá bằng gì*. Phần C là chỗ cả cây hội tụ: lập ngân sách bằng số cho một cấu hình cụ thể, rồi kiểm nó bằng Monte Carlo trước khi tốn tiền gia công. Phần D là tầng công cụ — cố ý đặt gần cuối vì nó hết hạn nhanh nhất. Phần E là lộ trình rút gọn.

Bảng 1. Năm phần, câu hỏi mỗi phần trả lời, và mức độ ưu tiên.

	Phần	Câu hỏi nó trả lời	Mức
A	Nguyên lý và toán học	Sai số sinh ra ở đâu, và một pixel nhiễu biến thành bao nhiêu milimét?	cốt lõi
B	Giải thuật và phương pháp	Có những cách nào giảm từng nguồn, và mỗi cách trả giá bằng gì?	cốt lõi
C	Ngân sách sai số	Cấu hình cụ thể của tôi có đạt 5 mm / 0,5° không, và biết trước bằng cách nào?	cốt lõi
D	Framework và công cụ	Ai đã cài sẵn cái gì, và đọc mã nguồn nào để học được nhiều nhất?	công cụ
E	Lộ trình	Nếu chỉ có thời gian cho phần xương sống thì đọc gì, theo thứ tự nào?	điều hướng

Bảng 1 cũng là thứ tự đọc đề xuất cho lần đầu, với một ngoại lệ đáng kể và tôi khuyên dùng nó: *nếu bạn sắp phải quyết định bố trí marker hoặc sắp đặt gia công, hãy đọc chương 15 trước tiên*, ngay cả khi chưa hiểu hết các công thức trong đó. Chương 15 chỉ cần bốn quan hệ tỉ lệ từ chương 5, và nó cho bạn biết ngay bố trí nào có cơ hội đạt mục tiêu. Một buổi chiều ở chương 15 tiết kiệm được vài tuần và vài triệu tiền gia công tấm marker sai.

Ba mức ưu tiên, đánh dấu ngay ở tiêu đề chương (Three Priority Levels)

Không phải chương nào cũng đáng đầu tư như nhau. Cây dùng đúng ba mức, lấy từ quy ước sao trong lộ trình gốc (*** cốt lõi, ** quan trọng, * bổ trợ), và mức xuất hiện ngay ở dòng đầu mỗi chương.

Bảng 2. Ba mức ưu tiên và cách đọc tương ứng.

Mức	Nghĩa	Cách đọc tương ứng
● ● ● xương sống	Không hiểu thì không đạt được mục tiêu 5 mm	Đọc kỹ, tự dẫn lại công thức trên giấy , làm ví dụ nhỏ tính tay, trả lời hết câu hỏi tự kiểm
● ● quan trọng	Cần hiểu ý tưởng và biết khi nào dùng	Đọc hiểu cơ chế và điều kiện áp dụng; không cần tự dẫn lại toàn bộ
● bổ trợ	Biết là có, tra khi gặp vấn đề cụ thể	Đọc lướt một lượt để biết trong đó có gì, quay lại khi cần

Phân bố không đều là có chủ ý. Xương sống của cây gồm bảy chương: chương 4 (pose ambiguity), chương 5 (lan truyền sai số), chương 9 (constellation), chương 10 (xử lý ambiguity), chương 12 (calibration), chương 13 (docking control) và chương 15 (ngân sách sai số). Đó là bảy chương

chiếm phần lớn giá trị của cả cây, và chúng cũng chính là lộ trình rút gọn ở Phần E. Đáng chú ý là trong bảy chương ấy, chỉ có hai chương nói về thuật toán xử lý ảnh — điều đó không phải ngẫu nhiên mà là hệ quả trực tiếp của luận điểm trung tâm.

Ba điều mà cây này cố ý nói ngược với thói quen phổ biến (Three Contrarian Claims)

Đáng nêu sớm ba chỗ mà kết luận của cây khác với thứ người ta hay làm, để người đọc biết mình sắp gặp gì.

Một. Không nên trung bình pose từ từng tag. Với nhiều tag nhìn thấy cùng lúc, cách đúng là chạy một bài toán PnP duy nhất trên toàn bộ tập góc, với tọa độ 3D trong hệ dock. Trung bình pose là sai về mặt thống kê (pose sống trên đa tạp, và mỗi pose thành phần có hiệp phương sai rất dị hướng), và nó vứt bỏ đúng thứ ta cần: ràng buộc hình học giữa các tag. Chương 9 giải thích và chương 3 cho nền.

Hai. AprilTag không phải lựa chọn chính xác nhất cho khâu đo tinh. Nó xuất sắc ở khâu *nhận diện và bắt bám từ xa* — bền với che khuất, decode tin cậy, tỉ lệ báo sai cực thấp. Nhưng góc của nó đến từ giao hai đường thẳng khớp trên cạnh, còn góc chessboard đến từ một điểm yên ngựa có cấu trúc đối xứng hai chiều, và cái sau định vị chính xác hơn. Chương 8 bàn chuyện này, và kết luận thực hành là một target lai: AprilTag để bắt bám, vùng chessboard/ChArUco để đo tinh ở cự ly gần.

Ba. Côn dẫn hướng cơ khí ở dock thường là giải pháp rẻ nhất, và nó nên được xét *trước* khi ép thị giác xuống 5 mm. Một chamfer dẫn hướng biến yêu cầu “vào đúng 5 mm” thành “vào trong 15 mm rồi cơ cấu tự dẫn nốt” — một bài toán dễ hơn hẳn một bậc, và lý thuyết của nó đã có từ lâu trong ngành lắp ráp tự động [4]. Chương 13 nói rõ, và đây là chỗ mà một cây tri thức về thị giác phải trung thực nói rằng đôi khi câu trả lời không nằm ở thị giác.

Cách dùng cây (How to Use This Tree)

Mỗi thư mục có một tệp `00-map.tex` nói nhánh đó gồm gì và vì sao chia như vậy; đó là chỗ nên đọc trước khi vào nội dung. Mỗi tài liệu mở đầu bằng một dòng *Phụ thuộc* cho biết cần đọc gì trước, và kết bằng một mục *Kết nối* trỏ sang các nhánh liên quan cùng một câu giải thích quan hệ. Cuối mỗi tài liệu là ba tới bảy câu hỏi tự kiểm; theo lịch ôn của kho, hãy trả lời chúng *trước* khi mở lại bài, sau vài ngày, hai tuần, hai tháng, rồi nửa năm.

Tệp `99-chưa-biết.md` ở gốc cây là sổ những điều chưa biết theo tinh thần của Feynman: mỗi mục là một câu hỏi cụ thể mà tôi biết là mình chưa trả lời được. Nó chỉ có ích khi được thu ngắn dần, nên mỗi lần giải quyết xong một mục thì chuyển nó thành một đoạn trong bài chính và ghi ngày.

CẠM BÃY

Cây này hiện ở trạng thái **bản nháp có kiểm soát**, và cần biết rõ điều đó trước khi dựa vào nó.

`refs.bib` được soạn từ trí nhớ về các công trình gốc: tên tác giả, năm và venue đủ chính xác để tra cứu, nhưng *chưa* được đối chiếu với trang gốc của nhà xuất bản. Theo `learning-rules.md` mục 5, như vậy là chưa qua cửa kiểm chứng nào. Trước khi trích dẫn ra ngoài cây hoặc dùng để ra quyết định thật, phải mở bản gốc.

Quan trọng hơn với cây này: **các con số về độ chính xác góc** — kiểu “góc AprilTag khoảng 0,1–0,3 px, góc ChArUco sau tinh chỉnh khoảng 0,03–0,1 px” — là *bậc độ lớn mang tính kinh*

nghiệm, không phải kết quả đo của một thí nghiệm cụ thể có điều kiện xác định. Chúng đủ tốt để lập ngân sách sơ bộ và để so sánh tương đối giữa các phương pháp, và *không* đủ tốt để nghiệm thu. Con số σ của *hệ thống của bạn* phải do chính bạn đo, và chương 16 nói cách đo. Mọi chỗ trong cây dùng các con số này đều ghi rõ chúng là bậc độ lớn.

Câu hỏi còn mở (Open Questions)

Bốn điều cần xử lý khi quay lại cây này. Thứ nhất, toàn bộ refs.bib cần một lượt đối chiếu với nguồn gốc, và mỗi entry cần được dán mức tin A/B/C *sau khi* đã mở được ít nhất abstract. Thứ hai, script Monte Carlo ở C-ngan-sach-sai-so/16-danh-gia-va-nghiem-thu/code/ cần được chạy trên cấu hình thật và kết quả của nó phải được đối chiếu với công thức giải tích ở chương 5 — đây là cửa kiểm chứng (b) cho toàn bộ Phần C, và cho tới khi làm xong thì các công thức ở chương 5 mới chỉ qua cửa (a). Thứ ba, các con số σ góc cần được thay bằng số đo thật trên phần cứng cụ thể, kèm điều kiện đo. Thứ tư, Phần D chốt tại tháng 9/2026 và là phần hết hạn nhanh nhất; rà lại mỗi sáu tháng.

MỤC LỤC

Bản đồ cây	i
A NGUYÊN LÝ VÀ TOÁN HỌC	1
1 Mô hình camera và méo ảnh (Camera Model and Distortion)	4
1.1 Đặt vấn đề	5
1.2 Mô hình pinhole và ma trận K	6
1.3 Các mô hình méo	7
1.4 Vì sao méo phá hoại detector marker một cách đặc biệt	8
1.5 Điểm chính sai: nguồn thiên lệch xoay không có triệu chứng	9
1.6 Sai số mô hình so với sai số tham số	10
1.7 Khử méo: toàn ảnh hay từng điểm	11
1.8 Giới hạn và chế độ hỏng	12
1.9 Thực hành	12
1.10 Điều gì chứng minh cách hiểu này sai	15
1.11 Kết nối	16
2 Homography và hình học phẳng (Homography and Planar Geometry)	17
2.1 Đặt vấn đề	18
2.2 Homography của một mặt phẳng	19
2.3 Phân rã H thành pose	20
2.4 Điều kiện số: vì sao tag nhỏ tệ hơn công thức dự đoán	20
2.5 Bất biến của phép chiếu: chỉ có tỉ số kép	21
2.6 Bias tâm ellipse: tâm chiếu không bằng chiếu của tâm	22
2.7 Giới hạn và chế độ hỏng	23
2.8 Thực hành	23
2.9 Điều gì chứng minh cách hiểu này sai	26
2.10 Kết nối	27
3 PnP và ước lượng tư thế (Perspective-n-Point)	28
3.1 Đặt vấn đề	29
3.2 P3P: ba điểm và bốn nghiệm	30

3.3	Các lời giải dạng đóng cho n điểm	31
3.4	Tinh chỉnh: nghiệm thật sự	32
3.5	PnP hợp nhất trên nhiều tag	33
3.6	Trọng số: không phải góc nào cũng như nhau	34
3.7	Loại ngoại lai	35
3.8	Giới hạn và chế độ hỏng	36
3.9	Thực hành	36
3.10	Điều gì chứng minh cách hiểu này sai	39
3.11	Kết nối	40
4	Pose ambiguity của target phẳng (Planar Pose Ambiguity)	42
4.1	Đặt vấn đề	43
4.2	Vì sao đúng hai nghiệm: hình học của phép lật	44
4.3	Vì sao thêm điểm đồng phẳng không cứu được	45
4.4	Khi nào ambiguity tệ nhất	45
4.5	Đo ambiguity: tỉ số sai số tái chiếu	46
4.6	Lối thoát triệt để: phá tính đồng phẳng	47
4.7	Giới hạn và chế độ hỏng	48
4.8	Thực hành	48
4.9	Điều gì chứng minh cách hiểu này sai	51
4.10	Kết nối	52
5	Lan truyền sai số từ pixel sang pose (Error Propagation)	54
5.1	Đặt vấn đề	55
5.2	Công thức gốc: nghịch đảo của ma trận thông tin	56
5.3	Hai quan hệ tịnh tiến: chia cho f , chia cho w	57
5.4	Ví dụ số chuẩn của cây	58
5.5	Sai số xoay từ một tag phẳng: quy luật hoàn toàn khác	59
5.6	Sai số xoay từ constellation: chia cho baseline	60
5.7	Vì sao trung bình pose từ từng tag là sai	61
5.8	Cộng các nguồn sai số: khi nào bình phương, khi nào cộng thẳng	62
5.9	Giới hạn và chế độ hỏng	63
5.10	Thực hành: dùng bốn quan hệ này trong một buổi chiều	64
5.11	Điều gì chứng minh cách hiểu này sai	67
5.12	Kết nối	68

6	Hiệu ứng vật lý và quang học (Physical and Optical Effects)	70
6.1	Đặt vấn đề	71
6.2	Rolling shutter: tag bị xé, không chỉ bị mờ	72
6.3	Motion blur: nhòe có hướng, và nó gây thiên lệch	73
6.4	Lấy mẫu, răng cưa, và ngưỡng kích thước tag	73
6.5	Quang học: MTF, nhiễu xạ, và độ sâu trường ảnh	74
6.6	Vật liệu và bề mặt marker	75
6.7	Tổng kết: nhiễu hay thiên lệch, và cái gì biến mất khi dừng	75
6.8	Giới hạn và chế độ hỏng	76
6.9	Thực hành	76
6.10	Điều gì chứng minh cách hiểu này sai	80
6.11	Kết nối	80
B	GIẢI THUẬT VÀ PHƯƠNG PHÁP	82
7	Detection pipeline (Marker Detection)	85
7.1	Đặt vấn đề	86
7.2	AprilTag: bốn bước và chỗ σ sinh ra	87
7.3	ArUco và ChArUco	88
7.4	Các họ marker khác	88
7.5	Mã hoá, khoảng cách Hamming, và tỉ lệ báo sai	89
7.6	Giới hạn và chế độ hỏng	90
7.7	Thực hành	91
7.8	Điều gì chứng minh cách hiểu này sai	94
7.9	Kết nối	95
8	Corner refinement (Subpixel Corner Refinement)	96
8.1	Đặt vấn đề	97
8.2	Trục giao gradient: cornerSubPix	98
8.3	Điểm yên ngựa: vì sao chessboard thắng	99
8.4	Các phương pháp còn lại	100
8.5	Trung bình nhiều khung: cái rẻ nhất	100
8.6	Các con số: một cảnh báo cần lặp lại	102
8.7	Hiệp phương sai của góc: sản phẩm phụ đáng lấy	102
8.8	Giới hạn và chế độ hỏng	103
8.9	Thực hành	103

8.10	Điều gì chứng minh cách hiểu này sai	106
8.11	Kết nối	107
9	Multi-marker constellation (Marker Constellation Design)	109
9.1	Đặt vấn đề	110
9.2	Nguyên tắc một: PnP hợp nhất, và điều nó đòi hỏi	111
9.3	Nguyên tắc hai: baseline có hướng	112
9.4	Nguyên tắc ba: phá tính đồng phẳng	112
9.5	Nguyên tắc bốn: đo constellation bằng BA, đừng tin CAD	113
9.6	Nguyên tắc năm: nested multi-scale	114
9.7	Trọng số và loại ngoại lai	115
9.8	Tổng hợp: một bản thiết kế mẫu	116
9.9	Giới hạn và chế độ hỏng	116
9.10	Thực hành	117
9.11	Điều gì chứng minh cách hiểu này sai	120
9.12	Kết nối	121
10	Xử lý ambiguity (Handling Pose Ambiguity)	123
10.1	Đặt vấn đề	124
10.2	Thang bốn bậc	124
10.3	Bậc 2: đo ρ và từ chối	125
10.4	Bậc 3: nhất quán thời gian, và chế độ khoá cứng	126
10.5	Bậc 4: fuse với cảm biến khác	126
10.6	Giới hạn và chế độ hỏng	127
10.7	Thực hành	127
10.8	Điều gì chứng minh cách hiểu này sai	131
10.9	Kết nối	131
11	Ước lượng pose theo thời gian (Temporal Pose Estimation)	132
11.1	Đặt vấn đề	133
11.2	Trường hợp đơn giản nhất: camera đứng yên	134
11.3	Trung bình trên đa tạp: vì sao quaternion thô là sai	135
11.4	Lọc: phép đo phải là reprojection	136
11.5	Fixed-lag smoothing và đồ thị nhân tử	136
11.6	Điều chương này không làm được	137
11.7	Giới hạn và chế độ hỏng	138
11.8	Thực hành	138

11.9	Điều gì chứng minh cách hiểu này sai	141
11.10	Kết nối	142
12	Calibration cho bài toán docking (Calibration for Docking)	143
12.1	Đặt vấn đề	144
12.2	Thứ tự bắt buộc	145
12.3	Hiệu chuẩn intrinsic chất lượng cao	145
12.4	Đo constellation	147
12.5	Hand-eye	147
12.6	Trôi theo thời gian	148
12.7	Nếu dùng teach-by-showing thì khác gì	149
12.8	Giới hạn và chế độ hỏng	149
12.9	Thực hành	150
12.10	Điều gì chứng minh cách hiểu này sai	153
12.11	Kết nối	154
13	Docking control (Docking Control)	155
13.1	Đặt vấn đề	156
13.2	Hai họ phương pháp: PBVS và IBVS	157
13.3	Teach-by-showing: mẹo lớn nhất	158
13.4	Stop-and-go	159
13.5	Coarse-to-fine	160
13.6	Ràng buộc nonholonomic	160
13.7	Dung sai cơ khí: côn dẫn hướng	161
13.8	Tiêu chí hội tụ và cơ chế retry	162
13.9	Giới hạn và chế độ hỏng	162
13.10	Thực hành	163
13.11	Điều gì chứng minh cách hiểu này sai	167
13.12	Kết nối	167
14	Điều kiện quan sát và ánh sáng (Observation Conditions)	169
14.1	Đặt vấn đề	170
14.2	Ba cơ chế làm điều kiện thay đổi	171
14.3	Chống loá	171
14.4	Chiếu sáng chủ động	172
14.5	Khoá mọi thứ khoá được	173
14.6	Chọn cảm biến	174

14.7	Chống báo sai và xử lý che khuất	174
14.8	Giới hạn và chế độ hỏng	175
14.9	Thực hành	175
14.10	Điều gì chứng minh cách hiểu này sai	179
14.11	Kết nối	179

C NGÂN SÁCH SAI SỐ 181

15	Ngân sách sai số (The Error Budget)	183
15.1	Đặt vấn đề	184
15.2	Cấu trúc của bảng	185
15.3	Bảng ngân sách: ví dụ số đầy đủ	186
15.4	Cánh tay đòn: vì sao góc và vị trí không độc lập	187
15.5	Hai chế độ, hai bảng	188
15.6	Ambiguity và lỗi thô: vì sao chúng không nằm trong bảng	189
15.7	Sáu kết luận thiết kế	189
15.8	Giới hạn và chế độ hỏng	190
15.9	Thực hành	190
15.10	Điều gì chứng minh cách hiểu này sai	194
15.11	Kết nối	194
16	Đánh giá và nghiệm thu (Evaluation and Acceptance)	196
16.1	Đặt vấn đề	197
16.2	Mô phỏng Monte Carlo: công cụ và cách dùng	198
16.3	Kết quả đo được	198
16.4	Nghiệm thu trên phần cứng	202
16.5	Kiểm tính nhất quán của hiệp phương sai	203
16.6	Giới hạn của mô phỏng	203
16.7	Thực hành: quy trình sáu bước	204
16.8	Điều gì chứng minh cách hiểu này sai	208
16.9	Kết nối	208

D FRAMEWORK VÀ CÔNG CỤ 210

17	Thư viện và detector (Libraries and Detectors)	212
17.1	Đặt vấn đề	213
17.2	Đại lượng phải lấy ra khỏi pipeline	213

17.3	OpenCV: các API và một phép chạy thật	214
17.4	Ba tệp mã đáng đọc	215
17.5	Các họ tag khác và detector học sâu	215
17.6	Wrapper ROS	216
17.7	Giới hạn	216
17.8	Thực hành	216
17.9	Điều gì chứng minh cách hiểu này sai	219
17.10	Kết nối	220
18	Công cụ hiệu chuẩn, servoing và đo đạc (One-Time Tools)	221
18.1	Đặt vấn đề	222
18.2	Hiệu chuẩn intrinsic	222
18.3	Đo constellation bằng photogrammetry	222
18.4	Hand-eye	223
18.5	Đánh giá: chỗ nên tự viết	224
18.6	Giới hạn	224
18.7	Thực hành	225
18.8	Điều gì chứng minh cách hiểu này sai	228
18.9	Kết nối	229
E	LỘ TRÌNH	230
19	Lộ trình tối thiểu (The Minimum Path)	232
19.1	Đặt vấn đề: ba thứ tự sai	232
19.2	Nguyên tắc sắp xếp	233
19.3	Tám bước	233
19.4	Dấu hiệu đã xong mỗi bước	235
19.5	Ba lộ trình cho ba tình huống	236
19.6	Lộ trình này có ý bỏ gì	236
19.7	Điều gì chứng minh lộ trình này sai	236
19.8	Kết nối	237
	Số những điều chưa biết	239

NGUYÊN LÝ VÀ TOÁN HỌC



TÓM TẮT

Phần A dựng đúng lượng nền cần thiết cho bốn phần sau, và nó được tổ chức theo *đường đi của một photon tới một con số*: camera biến điểm 3D thành pixel thế nào và méo ảnh phá hỏng gì, hình học phẳng cho ta công cụ gì, PnP biến tập góc thành pose ra sao, vì sao target phẳng luôn có hai nghiệm, một pixel nhiều biến thành bao nhiêu milimét, và cuối cùng là những hiệu ứng vật lý mà mô hình toán bỏ qua nhưng thực tế thì không. Nếu chỉ nhớ một điều: bốn chương đầu là công cụ, chương 5 là chương định giá — nó biến mọi lựa chọn thiết kế thành một con số milimét, và nó là chương mà cả Phần C dựa vào.

Phần này tồn tại để phục vụ Phần C, và nguyên tắc chi phối nó là *học theo nhu cầu của ngân sách*: mỗi chương chỉ đi tới mức đủ để ta biết khâu đó đóng góp bao nhiêu vào sai số cuối cùng, và đủ để ta tự dẫn lại được quan hệ tỉ lệ khi cần. Đây là lựa chọn có chủ ý và nó khác với một giáo trình thị giác máy tính. Một giáo trình sẽ dành nhiều trang cho việc phân rã homography thành pose vì đó là một kết quả đẹp; ở đây kết quả ấy được trình bày ngắn, vì trong thực hành nó chỉ dùng làm khởi tạo cho một phép tinh chỉnh phi tuyến, và chính phép tinh chỉnh mới quyết định sai số. Ngược lại, quan hệ $\delta Z \approx Z\sigma/w$ — một dòng trong hầu hết sách — được dành cả một mục ở chương 5, vì nó là dòng quyết định bạn có đạt 5 mm hay không.

Sáu chương có vai trò rất khác nhau, và mức đầu tư nên khác nhau theo. Chương 1 là mô hình camera: nó tồn tại chủ yếu để giải thích vì sao méo ảnh chưa khử sạch làm *cong* cạnh tag và vì thế làm sai chỗ giao của hai đường thẳng, và vì sao principal point sai gây ra một thiên lệch xoay có hệ thống — nguồn lỗi xoay lớn nhất trong thực tế mà hầu như không ai ngờ tới. Chương 2 cho hình học phẳng, và nó chứa một sự thật nhỏ mà hậu quả lớn: tâm của hình chiếu không bằng hình chiếu của tâm. Chương 3 là PnP, tức cỗ máy biến góc thành pose. Chương 4 là chương bệnh lý: với target phẳng luôn tồn tại đúng hai nghiệm, và đây là nguyên nhân số một của hiện tượng pose nhảy. Chương 5 là chương định giá. Chương 6 gom những hiệu ứng vật lý mà năm chương trước giả định là không có.

Thứ tự đọc đề xuất là tuần tự, với hai lưu ý. Chương 2 có thể đọc lướt nếu bạn đã quen hình học đa khung nhìn — chỉ cần dừng kỹ ở mục về cross-ratio và bias tâm ellipse. Chương 5 thì ngược lại: nên đọc kỹ ngay cả khi chưa hiểu hết chi tiết Jacobian, vì bốn quan hệ tỉ lệ trong đó là thứ bạn sẽ dùng hằng ngày, và chúng đủ đơn giản để thuộc lòng.

Bảng 3. Bản đồ Phần A: các chương, nội dung, và mức đầu tư đề xuất.

Ch.	Thư mục	Nội dung	Mức
1	01-mo-hinh-camera-va-d:	Pinhole, ma trận K , tọa độ chuẩn hoá; méo radial/tangential/rational/KB; vì sao méo làm cong cạnh tag; sai số mô hình so với sai số tham số; principal point sai gây thiên lệch xoay	● ● ●
2	02-homography-va-hinh-t	$H = K[r_1 \ r_2 \ t]$ và phân rã; điều kiện suy biến và số điều kiện theo diện tích tag; cross-ratio; vì sao tâm chiếu $\neq$ chiếu của tâm	● ●
3	03-pnp-va-uoc-luong-tu	P3P và bốn nghiệm; EPnP, DLT, IPPE cho target phẳng; tinh chỉnh bằng sai số tái chiếu; PnP hợp nhất trên nhiều tag; PnP có trọng số	● ● ●
4	04-pose-ambiguity-targ	Hai nghiệm của target phẳng; khi nào tệ nhất; biểu hiện nhảy/flicker; vì sao không đồng phẳng khử được triệt để	● ● ●
5	05-lan-truyen-sai-so-p	Jacobian và $(J^T \Sigma^{-1} J)^{-1}$; bốn quan hệ tỉ lệ phải thuộc; vì sao xoay từ tag phẳng gần như không quan sát được; cải thiện tuyến tính theo baseline	● ● ●
6	06-hieu-ung-vat-ly-va-c	Rolling shutter, motion blur, lấy mẫu và răng cưa, độ sâu trường ảnh và MTF, bias tâm ellipse, đặc trưng quang học của vật liệu	● ●

Chương 5 là chương có giá trị cao nhất của cả phần và nên đọc kỹ nhất; bốn quan hệ tỉ lệ trong đó đủ gần để thuộc lòng và chúng là công cụ chẩn đoán hàng ngày. Chương 1 và 3 là nền bắt buộc. Chương 4 gần nhưng đừng bỏ: nó giải thích hiện tượng mà gần như ai làm marker cũng gặp và ít người gọi đúng tên. Chương 2 đọc lướt được nếu đã quen hình học đa khung nhìn, trừ mục cross-ratio. Chương 6 đọc một lượt rồi quay lại khi gặp triệu chứng cụ thể.

Mô hình camera và méo ảnh (Camera Model and Distortion)

PHỤ THUỘC Không có — đây là chương mở đầu của cây. Nó chỉ giả định bạn biết tọa độ thuần nhất và phép nhân ma trận. Mọi chương còn lại của Phần A đều gọi về đây, và A/05 lấy từ đây đại lượng f mà mọi công thức sai số của nó đều chia cho.

TÓM TẮT

Chương này dựng mô hình biến một điểm trong không gian thành một pixel, và nó tồn tại vì hai lý do rất cụ thể chứ không phải vì tính đầy đủ. Lý do thứ nhất: nó cấp *tiêu cự tính bằng pixel* f , đại lượng camera duy nhất xuất hiện trong mọi công thức sai số của cây. Lý do thứ hai, và đây mới là lý do chính: nó giải thích hai cơ chế hỏng mà người làm marker gặp thường xuyên và hầu như không bao giờ chẩn đoán đúng. Cơ chế một — méo ảnh chưa khử làm *cong* cạnh tag, mà detector thì khớp *đường thẳng* lên cạnh, nên giao điểm hai đường thẳng sai, nên góc sai, nên pose sai. Cơ chế hai — điểm chính (principal point) sai gây ra một thiên lệch xoay *có hệ thống* phụ thuộc vị trí tag trên ảnh. Nếu chỉ nhớ một điều: *méo ảnh không gây nhiễu, nó gây thiên lệch* — và thiên lệch thì không trung bình mất đi dù lấy bao nhiêu khung.

KIẾN THỨC CẦN CÓ TRƯỚC

- **tọa độ thuần nhất (homogeneous coordinates)** — cách viết một điểm 2D bằng ba số (x, y, w) với quy ước $(x, y, w) \sim (\lambda x, \lambda y, \lambda w)$ cho mọi $\lambda \neq 0$; điểm Euclid tương ứng là $(x/w, y/w)$. Nó tồn tại để phép chiếu phối cảnh — vốn là một phép chia — viết được thành một phép nhân ma trận.
- **tọa độ chuẩn hoá (normalized coordinates)** — tọa độ của điểm sau khi chia cho chiều sâu nhưng *trước khi* nhân với K : $(x_n, y_n) = (X/Z, Y/Z)$. Đây là hệ tọa độ mà mọi mô hình méo được định nghĩa trên đó, và nhầm lẫn giữa nó với tọa độ pixel là lỗi cài đặt phổ biến nhất khi tự viết code undistort.
- **tiêu cự tính bằng pixel** — $f_x = F/s_x$ với F là tiêu cự vật lý của ống kính và s_x là kích thước một pixel. Nó *không* phải một chiều dài; nó là một tỉ số không thứ nguyên, và đó là lý do nó xuất hiện được ở mẫu số của các công thức sai số.
- **điểm chính (principal point)** — giao điểm của trục quang với mặt phẳng ảnh, ký hiệu (c_x, c_y) . Trục giác nói nó nằm ở giữa ảnh; thực tế nó lệch vài pixel tới vài chục pixel do sai lệch lắp ráp, và mục 5 giải thích vì sao sai lệch ấy nguy hiểm hơn vẻ ngoài.
- **méo xuyên tâm (radial distortion)** — sai lệch mà độ lớn chỉ phụ thuộc khoảng cách từ điểm chính, theo hướng ra xa hoặc vào gần tâm. Đây là thành phần méo lớn nhất trong hầu hết ống kính.
- **méo tiếp tuyến (tangential distortion)** — sai lệch theo hướng vuông góc với bán kính, sinh ra khi mặt phẳng cảm biến không song song hoàn hảo với mặt phẳng thấu kính. Thường nhỏ hơn méo xuyên tâm một bậc, nhưng nó là thành phần *bất đối xứng* nên nó không tự triệt tiêu.
- **sai số mô hình và sai số tham số** — sai số tham số là khi bạn dùng đúng dạng hàm nhưng ước lượng sai hệ số; sai số mô hình là khi dạng hàm bản thân nó không đủ để mô tả hiện thực. Loại thứ hai nguy hiểm hơn vì thêm dữ liệu không chứa được nó, và mục 6 dành riêng cho việc phân biệt hai loại.

CÂU HỎI MỞ ĐẦU

- Q1. Bạn hiệu chuẩn camera, sai số tái chiếu trung bình là 0,25 px — một con số nghe rất tốt. Vì sao con số ấy có thể đang che giấu một lỗi mô hình nghiêm trọng, và bạn nhìn vào đâu để phát hiện?
- Q2. Detector AprilTag khớp một đường *thẳng* lên mỗi cạnh tag rồi lấy giao điểm làm góc. Nếu ảnh còn méo 3 px ở rìa, sai số góc mà bạn nhận được là bao nhiêu — và vì sao câu trả lời không phải “3 px”?
- Q3. Điểm chính của bạn bị sai 10 px so với giá trị thật. Sai số ấy gây ra hậu quả gì cho góc của pose, và vì sao hậu quả ấy thay đổi khi tag di chuyển trong khung hình?
- Q4. Có hai cách xử lý méo: khử méo toàn ảnh rồi chạy detector, hoặc chạy detector trên ảnh gốc rồi khử méo cho từng toạ độ góc. Cách nào chính xác hơn, và cách nào nhanh hơn — và vì sao câu trả lời cho hai câu ấy không giống nhau?
- Q5. Bạn dùng mô hình hai hệ số k_1, k_2 cho một ống kính góc rộng 120° . Hiệu chuẩn hội tụ, residual nhỏ. Vì sao điều đó *không* chứng minh mô hình của bạn đủ, và dấu hiệu nào sẽ lộ ra sự thiếu?

1.1 Đặt vấn đề

Mọi thứ trong cây này đều bắt đầu từ một phép biến đổi: một điểm ở đâu đó trong không gian trở thành một toạ độ pixel trên một tấm ảnh. Nếu phép biến đổi ấy được mô tả đúng, mọi công thức phía sau có nghĩa. Nếu nó được mô tả sai — kể cả sai một chút — thì mọi thứ phía sau là chính xác quanh một giá trị sai.

Bài toán gốc thuộc về ngành trắc lượng ảnh (photogrammetry) chứ không phải thị giác máy tính, và nó đã có từ trước khi có máy tính: người ta cần đo đặc địa hình từ ảnh chụp trên không, và để làm được thì phải biết chính xác ống kính đã biến dạng ảnh như thế nào. Mô hình méo vẫn đang dùng ngày nay mang tên Brown–Conrady và được công bố ở dạng hiện đại năm 1971 [5]; phương pháp hiệu chuẩn từ một target phẳng mà mọi hộp công cụ đang dùng thì đến sau đó gần ba mươi năm [6].

Điều làm chương này đáng đọc với người làm marker — thay vì chỉ tra một công thức khi cần — nằm ở một quan sát mà tôi cho là chưa được nói đủ rõ ở đâu cả. Méo ảnh ảnh hưởng tới các bài toán thị giác theo những cách rất khác nhau, và nó ảnh hưởng tới *phát hiện marker* theo một cách đặc biệt xấu. Lý do là thế này: hầu hết thuật toán thị giác dùng đặc trưng điểm, và một điểm bị méo vẫn là một điểm — nó chỉ nằm sai chỗ, và khử méo là đủ. Nhưng detector marker không làm việc với điểm; nó làm việc với *cạnh thẳng*. AprilTag khớp một đường thẳng lên mỗi cạnh của tứ giác rồi lấy giao của bốn đường thẳng ấy làm bốn góc [7]. Và phép méo xuyên tâm biến một đường thẳng thành một *đường cong*. Khớp một đường thẳng lên một đường cong thì được một đường thẳng sai, và giao của hai đường thẳng sai thì sai nhiều hơn cả hai — mục 4 định lượng chuyện này.

Cách hiển nhiên để xử lý là khử méo toàn ảnh trước khi chạy detector, và cách ấy đúng. Chỗ nó không đủ nằm ở hai điểm mà mục 6 và mục 7 bàn tới: khử méo chỉ tốt đúng bằng mô hình méo bạn đã ước lượng, và nếu mô hình *sai dạng* — chẳng hạn dùng hai hệ số cho một ống kính góc rộng — thì phần méo còn sót lại không phải nhiễu ngẫu nhiên mà là một mẫu hình có cấu trúc, lớn nhất ở đúng chỗ tệ nhất là rìa ảnh.

Còn một lý do thứ hai để chương này không phải một phụ lục, và nó là lý do làm tôi xếp chương này ở mức xương sống thay vì mức tham khảo. Trong tất cả các tham số nội của camera, có một tham số mà sai lệch của nó gây hậu quả không tương xứng với vẻ ngoài của nó: điểm chính. Sai điểm chính không làm ảnh trông khác đi chút nào — không ai nhìn ảnh mà phát hiện được — nhưng nó tạo ra một thiên lệch trong *hướng* của pose, và thiên lệch ấy thay đổi theo vị trí tag trong khung hình nên nó trông hết như một sai số phụ thuộc tư thế. Mục 5 dành riêng cho nó.

TRỰC GIÁC

Hình dung camera như một cái thấu.

Đỉnh thấu là tâm quang học — một điểm. Mọi tia sáng đi tới ảnh đều phải chui qua đúng điểm ấy. Đáy thấu là mặt phẳng ảnh, đặt cách đỉnh một khoảng f . Một điểm trong không gian được chiếu lên ảnh bằng cách kẻ một đường thẳng từ nó tới đỉnh thấu và xem đường ấy cắt mặt phẳng ảnh ở đâu.

Mô hình pinhole nói rằng thấu là hoàn hảo: mọi tia đi thẳng, đỉnh thấu là một điểm toán học, mặt phẳng ảnh vuông góc với trục. Ống kính thật không phải thế. Thấu kính bẻ tia sáng, và nó bẻ các tia ở rìa nhiều hơn các tia ở giữa — đó là méo xuyên tâm. Cảm biến không lắp vuông góc tuyệt đối — đó là méo tiếp tuyến. Trục quang không đi qua đúng giữa cảm biến — đó là điểm chính lệch.

Ba sai lệch ấy khác nhau về bản chất theo một cách quan trọng. Hai cái đầu làm ảnh *trông* khác: đường thẳng cong đi, và mắt người phát hiện được nếu chụp một cái cửa sổ. Cái thứ ba thì không: dịch cả ảnh sang trái ba pixel thì không ai nhìn ra, vì không có gì trong ảnh để so sánh. Đó là lý do cái thứ ba nguy hiểm nhất — nó là sai lệch duy nhất không có triệu chứng thị giác.

1.2 Mô hình pinhole và ma trận K

Dựng mô hình theo ba bước, mỗi bước một phép biến đổi, vì việc tách bạch ba bước ấy là điều kiện để biết mình đang làm việc trong hệ tọa độ nào.

Bước một, từ thế giới sang camera. Một điểm $\mathbf{X}_w$ trong hệ thế giới được đưa sang hệ camera bằng một phép biến đổi cứng:

$$\mathbf{X}_c = R\mathbf{X}_w + \mathbf{t},$$

với $R \in SO(3)$ và $\mathbf{t} \in \mathbb{R}^3$. Cặp $(R, \mathbf{t})$ chính là *pose* mà toàn bộ cây này tìm cách ước lượng; sáu tham số của nó là sáu ẩn của bài toán PnP ở A/03.

Bước hai, chiếu phối cảnh. Chia cho chiều sâu:

$$\begin{pmatrix} x_n \\ y_n \end{pmatrix} = \begin{pmatrix} X_c/Z_c \\ Y_c/Z_c \end{pmatrix}.$$

Đây là tọa độ chuẩn hoá. Chúng không thứ nguyên, chúng là tang của góc nhìn theo hai hướng, và — quan trọng nhất cho mục đích của ta — *mọi mô hình méo được định nghĩa trên hệ tọa độ này, không phải trên pixel*. Đây là bước duy nhất trong ba bước mang tính phi tuyến, và toàn bộ độ khó của bài toán pose nằm ở phép chia này.

Bước ba, từ tọa độ chuẩn hoá sang pixel. Nhân với ma trận nội:

$$\begin{pmatrix} u \\ v \\ 1 \end{pmatrix} = K \begin{pmatrix} x_d \\ y_d \\ 1 \end{pmatrix}, \quad K = \begin{pmatrix} f_x & s & c_x \\ 0 & f_y & c_y \\ 0 & 0 & 1 \end{pmatrix},$$

với (x_d, y_d) là tọa độ chuẩn hoá đã méo (mục 3). Bốn tham số thật sự quan trọng là f_x, f_y, c_x, c_y ; hệ số trượt s mô tả trường hợp trục pixel không vuông góc và với cảm biến số hiện đại thì nó bằng không tới mức không đo được — đừng ước lượng nó trừ khi có lý do cụ thể, vì một tham số dư luôn hút một phần sai số của các tham số khác.

Hai điều đáng dừng lại về f_x và f_y . Thứ nhất, chúng là *tỉ số không thứ nguyên*, không phải chiều dài — $f_x = F/s_x$ với F là tiêu cự vật lý và s_x là bề rộng một pixel. Đây là lý do f đứng được ở

mẫu số trong $\delta x \approx Z\sigma/f$ mà vẫn ra đúng thứ nguyên chiều dài. Thứ hai, có một cách rất nhanh để ước lượng f mà không cần hiệu chuẩn, dùng khi thiết kế: với trường nhìn ngang θ_{FOV} và độ phân giải ngang N_x pixel,

$$f_x \approx \frac{N_x/2}{\tan(\theta_{\text{FOV}}/2)}.$$

Ví dụ: camera 1280 px ngang, FOV 60°. Ta có $f_x \approx 640/\tan 30^\circ = 640/0,577 = 1109$ px. Con số ấy đủ chính xác để lập ngân sách sai số ở C/15 trước khi có camera trong tay — và đó chính là cách dùng nó.

1.3 Các mô hình méo

Méo được mô hình hoá như một phép biến đổi phi tuyến *trên toạ độ chuẩn hoá*, chen vào giữa bước hai và bước ba: $(x_n, y_n) \mapsto (x_d, y_d)$.

A. Brown–Conrady: radial cộng tangential

Mô hình phổ biến nhất, và là mặc định của OpenCV. Đặt $r^2 = x_n^2 + y_n^2$:

$$\begin{aligned} x_d &= x_n \underbrace{(1 + k_1 r^2 + k_2 r^4 + k_3 r^6)}_{\text{xuên tâm}} + \underbrace{2p_1 x_n y_n + p_2 (r^2 + 2x_n^2)}_{\text{tiếp tuyến}}, \\ y_d &= y_n (1 + k_1 r^2 + k_2 r^4 + k_3 r^6) + p_1 (r^2 + 2y_n^2) + 2p_2 x_n y_n. \end{aligned} \quad (1.1)$$

Đây là dạng chuẩn [5], viết theo quy ước của [6].

Ba điều cần đọc ra. Thứ nhất, thành phần xuyên tâm chỉ chứa *luỹ thừa chẵn* của r — đó không phải tùy tiện mà là hệ quả của tính đối xứng quay của thấu kính: nếu phép méo đối xứng quanh trục thì nó phải là hàm chẵn. Thứ hai, thành phần xuyên tâm *giữ nguyên hướng* của điểm so với tâm và chỉ đổi bán kính; hệ quả là một đường thẳng đi *qua* điểm chính vẫn thẳng sau khi méo, còn mọi đường thẳng khác thì cong. Ghi nhớ điều này — nó là chìa khoá của mục 4. Thứ ba, $k_1 < 0$ cho méo thùng (barrel, điển hình của ống kính góc rộng: các điểm ở rìa bị kéo vào trong) và $k_1 > 0$ cho méo gối (pincushion).

B. Mô hình hữu tỉ (rational)

Với ống kính góc rất rộng, đa thức bậc sáu không đủ. Mô hình hữu tỉ thay đa thức bằng một phân thức:

$$x_d = x_n \cdot \frac{1 + k_1 r^2 + k_2 r^4 + k_3 r^6}{1 + k_4 r^2 + k_5 r^4 + k_6 r^6} + (\text{tangential}).$$

Nó linh hoạt hơn nhiều nhưng nó có nhiều tham số hơn, và nhiều tham số hơn nghĩa là dễ khớp quá mức (overfitting) nếu dữ liệu hiệu chuẩn không phủ kín trường nhìn — một cạm bẫy mà B/12 bàn kỹ.

C. Mô hình fisheye Kannala–Brandt (KB) và các mô hình góc

Với ống kính trên khoảng 120°, toàn bộ họ mô hình trên gặp một vấn đề cấu trúc chứ không phải vấn đề độ chính xác: phép chiếu pinhole $r_n = \tan \theta$ phân kỳ khi $\theta \rightarrow 90^\circ$, nên không đa thức nào sửa được. Mô hình KB bỏ hẳn pinhole và mô hình hoá trực tiếp quan hệ giữa *góc tới* θ và bán kính ảnh [8]:

$$r_d = \theta + k_1 \theta^3 + k_2 \theta^5 + k_3 \theta^7 + k_4 \theta^9.$$

Đây là mô hình mặc định cho fisheye trong hầu hết hộp công cụ hiện đại, và có các mô hình mới hơn với ít tham số hơn cho cùng độ chính xác, chẳng hạn Double Sphere [9].

Bảng 1.1. Bốn họ mô hình méo, phạm vi áp dụng, và chế độ hỏng.

Mô hình	Số tham số	Dùng cho	Hỏng khi
Radial 2 hệ số (k_1, k_2)	2	Ống kính hẹp, dưới $\sim 50^\circ$	FOV rộng: residual có cấu trúc ở rìa, lớn dần theo r
Brown–Conrady đầy đủ	5	Phổ thông, $50\text{--}90^\circ$	Fisheye: dạng hàm không đủ dù thêm bao nhiêu ảnh
Hữu tỉ (rational)	8	Góc rộng $90\text{--}120^\circ$	Dữ liệu hiệu chuẩn không phủ rìa: khớp quá mức, ngoại suy sai
KB / fisheye	4	Trên 120° , tới $180^\circ+$	Dùng nhầm cho ống kính hẹp: tham số hoá kém điều kiện

1.4 Vì sao méo phá hoại detector marker một cách đặc biệt

Đây là mục quan trọng nhất của chương, và nó là chỗ chương này khác một chương về mô hình camera trong sách giáo khoa.

A. Cơ chế: khớp thẳng lên đường cong

AprilTag định vị góc theo một quy trình rất cụ thể: sau khi tìm được tứ giác ứng viên, nó khớp một *đường thẳng* lên mỗi cạnh, dùng trọng số theo độ lớn gradient, rồi lấy giao của các cặp đường thẳng kề nhau làm bốn góc [7]. Cách làm ấy chính xác hơn nhiều so với việc lấy trực tiếp điểm cực trị của contour, vì nó dùng *toàn bộ* chiều dài cạnh — hàng chục tới hàng trăm pixel — để xác định một đường, thay vì dựa vào vài pixel quanh góc.

Nhưng nó có một giả định ngầm: *cạnh của tag là một đường thẳng trên ảnh*. Giả định ấy đúng với mô hình pinhole (ảnh phối cảnh của một đường thẳng là một đường thẳng) và *sai* khi ảnh còn méo. Như mục 3A đã chỉ ra, méo xuyên tâm giữ nguyên hướng của điểm so với tâm nhưng đổi bán kính theo một hàm phi tuyến, nên một đường thẳng không đi qua điểm chính trở thành một đường cong.

Hậu quả có ba tầng, và cả ba đều đi theo chiều xấu.

Tầng một, đường thẳng khớp bị lệch. Khớp bình phương tối thiểu một đường thẳng lên một cung cong cho ra một đường thẳng đi qua giữa cung — nó cắt cung ở hai chỗ và lệch khỏi cung ở hai đầu và ở giữa, theo hướng ngược nhau.

Tầng hai, và đây là tầng quyết định: sai số được khuếch đại tại giao điểm. Hai đường thẳng gần vuông góc giao nhau thì sai số góc nhỏ của mỗi đường chuyển thành sai số vị trí ở giao điểm theo một hệ số phụ thuộc góc giao. Với hai cạnh của một tag nhìn chếch, góc giao có thể khá nhỏ, và khi đó hệ số khuếch đại lớn. Đây là câu trả lời cho Q2: sai lệch của góc *không* bằng sai lệch của cạnh — nó có thể lớn hơn nhiều.

Tầng ba, sai số là thiên lệch chứ không phải nhiễu. Với một tag ở một vị trí cố định trên ảnh, méo luôn đẩy cạnh theo cùng một hướng, mỗi khung hình như nhau. Lấy trung bình một nghìn khung không giúp gì — đây chính là loại nguồn “hệ thống” ở A/05 mục 8, loại không chia cho $\sqrt{N}$.

B. Ví dụ số: méo lớn cỡ nào ở rìa ảnh

Để thấy bậc độ lớn, lấy một ống kính góc rộng phổ thông. Giả sử $f = 600$ px, ảnh 1280×960 , và $k_1 = -0,30$ (giá trị điển hình cho góc rộng; *không phải số đo của một ống kính cụ thể*). Xét một điểm ở rìa ngang, cách tâm $r_{\text{px}} = 400$ px.

Toạ độ chuẩn hoá: $r_n = 400/600 = 0,667$, nên $r_n^2 = 0,444$.

Hệ số méo xuyên tâm: $1 + k_1 r_n^2 = 1 - 0,30 \times 0,444 = 1 - 0,133 = 0,867$.

Điểm bị kéo vào trong: bán kính méo là $0,667 \times 0,867 = 0,578$ trong toạ độ chuẩn hoá, tức 347 px.
Độ lệch: $400 - 347 = 53$ px.

Năm mươi ba pixel. Đặt cạnh nhiều định vị góc $\sigma = 0,2$ px, méo lớn hơn nhiều hơn 250 lần. Đây là lý do khử méo không phải một bước tinh chỉnh mà là một bước bắt buộc: bỏ qua nó thì mọi thứ khác trong cây là vô nghĩa.

Con số quan trọng hơn cho mục đích của ta là *méo còn sót lại sau khi khử*. Nếu mô hình đúng dạng và hiệu chuẩn tốt, phần sót cỡ 0,1–0,3 px — cùng bậc với σ , chấp nhận được. Nếu mô hình *sai dạng* — hai hệ số cho ống kính góc rộng — phần sót có thể vẫn là vài pixel ở rìa, và vì nó có cấu trúc chứ không ngẫu nhiên nên nó thành một thiên lệch phụ thuộc vị trí. Mục 6 bàn cách phát hiện.

CẠM BÃY

Có một lỗi cài đặt cực kỳ phổ biến khi tự viết code khử méo, và nó gây ra triệu chứng rất khó truy: *áp công thức méo lên toạ độ pixel thay vì lên toạ độ chuẩn hoá*.

Công thức (1.1) định nghĩa trên (x_n, y_n) , tức sau khi chia cho Z nhưng *trước* khi nhân K . Nếu bạn tính $r^2 = u^2 + v^2$ bằng pixel thay vì $r^2 = x_n^2 + y_n^2$, kết quả sai theo hệ số f^2 — tức sai hàng trăm nghìn lần ở số hạng $k_1 r^2$.

Triệu chứng: hiệu chuẩn vẫn “hội tụ” vì bộ tối ưu sẽ tìm ra một k_1 nhỏ bù lại tỉ lệ, và residual trông không tệ. Nhưng các hệ số ra sẽ khác vài bậc độ lớn so với giá trị điển hình, và mô hình sẽ ngoại suy sai kinh khủng ở những vùng ảnh không có dữ liệu hiệu chuẩn.

Phép kiểm rẻ: hệ số k_1 của ống kính thường nằm trong khoảng -1 tới $+1$. Nếu bạn thấy $k_1 \sim 10^{-7}$ thì gần như chắc chắn bạn đang tính r bằng pixel.

1.5 Điểm chính sai: nguồn thiên lệch xoay không có triệu chứng

Mục này bàn một cơ chế mà tôi cho là bị đánh giá thấp nhất trong toàn bộ chuỗi đo, và nó xứng đáng được nhìn kỹ.

Điểm chính (c_x, c_y) là giao của trục quang với mặt phẳng ảnh. Trục giác nói nó ở giữa ảnh, và hầu hết người ta khởi tạo nó bằng $(N_x/2, N_y/2)$. Thực tế nó lệch: sai lệch lắp ráp cảm biến so với ống kính cỡ vài chục micromet là bình thường, và với pixel 3 μm thì vài chục micromet là *cả chục pixel*.

Vì sao sai lệch ấy nguy hiểm? Hãy xem nó làm gì với phép chiếu. Giả sử giá trị thật là c_x nhưng ta dùng $\hat{c}_x = c_x + \Delta$. Toạ độ chuẩn hoá mà ta tính ra từ một pixel đo được u là

$$\hat{x}_n = \frac{u - \hat{c}_x}{f} = \frac{u - c_x}{f} - \frac{\Delta}{f} = x_n - \frac{\Delta}{f}.$$

Nghĩa là: *mọi* tia nhìn đều bị dịch đi cùng một lượng góc Δ/f radian. Đây không phải nhiều; đây là một phép quay hệ thống của toàn bộ chùm tia.

Với $\Delta = 10$ px và $f = 600$ px: $\Delta/f = 0,0167$ rad = $0,95^\circ$. Gần một độ — và ngân sách góc của ta là $0,5^\circ$.

Bây giờ đến phần tinh tế, và nó là phần giải thích vì sao triệu chứng khó truy. Nếu sai lệch ấy *chỉ* là một phép quay hệ thống của cả camera, thì nó vô hại trong bài toán docking theo teach-by-showing: nó xuất hiện ở cả pose hiện tại lẫn pose mục tiêu và triệt tiêu (A/05 mục 8). Nhưng nó không chỉ là một phép quay, vì x_n vào công thức chiếu *cùng* với phép chia cho Z . Hậu quả là hiệu

ứng của Δ lên pose ước lượng phụ thuộc vào *tag* đang nằm ở đâu trong khung hình và ở khoảng cách nào — một tag ở giữa ảnh chịu ảnh hưởng khác một tag ở rìa.

Triệu chứng thực tế theo đó: pose sai một lượng thay đổi trơn theo tư thế, không nhảy, không nhiễu, và không có gì trong ảnh trông bất thường. Nó trông hết như một sai số hand-eye, hoặc như một constellation sai — và đó là lý do người ta hay đi tìm nhầm chỗ.

Ghi chú trung thực về mức kiểm chứng. Tôi dẫn cơ chế này bằng cách viết lại phép chiếu (cửa a) và tôi tin nó đúng về cấu trúc. Tôi *chưa* chạy mô phỏng để đo biên độ thiên lệch pose theo Δ (cửa b), và tôi *chưa* tìm được một nguồn phát biểu tường minh đúng dạng này để đối chiếu (cửa c). Vì vậy phần định lượng của mục này nên được coi là ước lượng bậc độ lớn của tôi chứ không phải kết quả đã kiểm. Đây là mục đứng đầu danh sách nợ kiểm chứng của chương trong 99-chua-biet.md.

CẦN NHỚ

Trong bốn tham số nội, f_x và f_y được ước lượng rất tốt bởi mọi quy trình hiệu chuẩn, còn c_x và c_y thì *không*: chúng tương quan mạnh với tư thế của target, và nếu dữ liệu hiệu chuẩn chỉ gồm các ảnh chụp gần vuông góc thì bộ tối ưu không tách được “điểm chính lệch sang trái” khỏi “target nghiêng sang phải”.

Hệ quả thực hành trực tiếp, và nó sẽ được nhắc lại ở B/12: dữ liệu hiệu chuẩn *phải* gồm các ảnh ở tư thế nghiêng mạnh và target phải phủ tới sát bốn góc khung hình. Một bộ ảnh hiệu chuẩn “đẹp” — target luôn ở giữa, luôn vuông góc — cho residual nhỏ và điểm chính sai.

Đây là một trường hợp cụ thể của nguyên tắc chung: residual nhỏ không chứng minh tham số đúng; nó chỉ chứng minh mô hình khớp được dữ liệu bạn đã cho.

1.6 Sai số mô hình so với sai số tham số

Hai loại sai lệch rất khác nhau thường bị gộp làm một, và phân biệt chúng là kỹ năng chẩn đoán quan trọng nhất của chương.

Sai số tham số (*parameter error*) là khi dạng hàm đúng nhưng hệ số ước lượng chưa chính xác — chẳng hạn mô hình Brown–Conrady đúng cho ống kính của bạn nhưng k_1 lệch 2% vì dữ liệu hiệu chuẩn ít. Loại này *chữa được bằng thêm dữ liệu*: chụp thêm ảnh, phủ rộng hơn, và hệ số hội tụ về giá trị đúng.

Sai số mô hình (*model error*) là khi dạng hàm bản thân nó không đủ để mô tả ống kính — chẳng hạn dùng hai hệ số radial cho một ống kính 120°. Loại này *không* chữa được bằng thêm dữ liệu. Bộ tối ưu sẽ tìm ra bộ hệ số tốt nhất trong họ hàm bạn đã chọn, và nếu họ hàm ấy không chứa hàm đúng thì “tốt nhất” vẫn sai. Thêm một nghìn ảnh cho ra cùng một kết quả sai, chỉ với khoảng tin cậy hẹp hơn — một tình huống đặc biệt nguy hiểm vì nó làm bạn tin hơn vào một con số sai.

Phép thử phân biệt hai loại rất đơn giản và nó là kỹ thuật chẩn đoán giá trị nhất trong chương: *vẽ sai số tái chiếu theo vị trí trên ảnh*.

Nếu chỉ có sai số tham số, các vectơ residual trông như nhiễu trắng — hướng ngẫu nhiên, độ lớn đều, không có cấu trúc không gian. Nếu có sai số mô hình, chúng tạo thành một *mẫu hình*: thường là các vectơ hướng xuyên tâm với độ lớn tăng dần theo bán kính, hoặc các xoáy có tổ chức ở bốn góc ảnh. Kỹ thuật này đã được dùng trong hiệu chuẩn camera chính xác từ lâu [10] và nó rẻ đến mức không có lý do gì để bỏ qua.

CẠM BẮY

Sai số tái chiếu trung bình (RMS reprojection error) là con số mà mọi công cụ hiệu chuẩn in ra, và nó là con số *tệ nhất* để đánh giá chất lượng hiệu chuẩn khi dùng một mình. Ba lý do.

Thứ nhất, nó là một trung bình trên toàn ảnh, nên một vùng rìa sai 2 px chìm mất trong hàng nghìn điểm ở giữa sai 0,1 px.

Thứ hai, nó *giảm* khi bạn thêm tham số, kể cả tham số vô nghĩa — đó là bản chất của khớp bình phương tối thiểu. Một mô hình hữu tỉ 8 tham số luôn cho RMS nhỏ hơn Brown-Conrady 5 tham số trên cùng dữ liệu, và điều đó không nói gì về việc mô hình nào đúng hơn.

Thứ ba, và nghiêm trọng nhất: nó đo độ khớp trên *dữ liệu hiệu chuẩn*, không đo độ đúng khi ngoại suy sang vùng không có dữ liệu. Nếu target của bạn không bao giờ chạm tới góc khung hình thì RMS không biết gì về góc khung hình, và tag của robot lúc tiếp cận thì rất có thể nằm đúng ở đó.

Thay vào đó hãy nhìn: bản đồ residual theo vị trí (có cấu trúc hay không), độ phủ của dữ liệu hiệu chuẩn (có chạm bốn góc không), và độ ổn định của tham số qua nhiều lần hiệu chuẩn độc lập.

1.7 Khử méo: toàn ảnh hay từng điểm

Hai cách, và chúng không tương đương — đây là câu trả lời cho Q4.

Cách một, khử méo toàn ảnh rồi chạy detector. Tạo một ảnh mới trong đó mọi pixel đã được đặt lại đúng vị trí pinhole, rồi chạy detector trên ảnh ấy. Ưu điểm: mọi giả định của detector về đường thẳng trở thành đúng, nên khớp cạnh hợp lệ. Nhược điểm: phép khử méo đòi *nội suy* giá trị pixel ở các vị trí không nguyên, và nội suy làm nhoè ảnh — tức làm tăng σ . Với nội suy song tuyến tính (bilinear), mức nhoè đáng kể; với nội suy bậc cao hơn thì đỡ hơn nhưng chậm hơn.

Cách hai, chạy detector trên ảnh gốc rồi khử méo cho bốn toạ độ góc. Ưu điểm: không nội suy, nên không mất độ sắc nét, nên σ giữ nguyên; và rẻ hơn nhiều vì chỉ xử lý bốn điểm thay vì một triệu. Nhược điểm: detector đang khớp đường thẳng lên cạnh cong, nên bốn góc nó trả về đã sai *trước khi* bạn khử méo — và khử méo cho một điểm sai chỉ cho ra một điểm sai ở chỗ khác.

Vậy cách nào đúng? Câu trả lời phụ thuộc vào độ lớn của méo trong vùng tag chiếm, và nó dẫn tới một quy tắc thực hành khá rõ. Nếu tag nhỏ và nằm ở vùng ảnh có méo gần tuyến tính cục bộ — tức gần tâm, hoặc ống kính ít méo — thì cách hai tốt hơn, vì phần cong trên chiều dài một cạnh tag là không đáng kể còn lợi ích về độ sắc nét thì thật. Nếu tag lớn hoặc nằm ở rìa ảnh của một ống kính góc rộng, cách một bắt buộc.

Có một cách thứ ba, tốt hơn cả hai nhưng ít được dùng: *đưa mô hình méo vào ngay trong hàm sai số tái chiếu* của bước tinh chỉnh PnP. Tức là không khử méo gì cả; thay vào đó, khi dự đoán vị trí của một góc, ta chiếu điểm 3D qua pinhole rồi *áp méo*, và so với toạ độ đo được trên ảnh gốc. Cách này đúng về mặt mô hình và nó không mất độ sắc nét; giá phải trả là bước phát hiện ban đầu vẫn dùng ảnh gốc nên vẫn có vấn đề khớp thẳng-lên-cong ở tầng detector. Nó là cách mà các hộp công cụ hiệu chuẩn chất lượng cao dùng, và nó là lựa chọn đúng khi bạn kiểm soát được toàn bộ chuỗi.

1.8 Giới hạn và chế độ hồng

Giả thiết tâm chiếu là một điểm. Ống kính thật có *đồng tử vào* (entrance pupil) với kích thước hữu hạn, và vị trí hiệu dụng của nó dịch chuyển theo góc tới. Với chụp gần — và docking là chụp gần — hiệu ứng này không hoàn toàn bỏ qua được. Nó thường bị hấp thụ vào các hệ số méo trong quá trình hiệu chuẩn, nghĩa là các hệ số ấy trở thành phụ thuộc khoảng cách. Hệ quả thực hành: *hiệu chuẩn ở khoảng cách gần với khoảng cách vận hành*.

Giả thiết mô hình méo không đổi. Ống kính trôi theo nhiệt độ: giãn nở nhiệt làm tiêu cự đổi và làm điểm chính dịch. Với ống kính nhựa và chênh lệch nhiệt độ vài chục độ, hiệu ứng có thể ở mức pixel. B/12 bàn chuyện này; ở đây chỉ cần ghi nhận rằng “đã hiệu chuẩn” không phải một trạng thái vĩnh viễn.

Giả thiết lấy nét cố định. Đổi tiêu điểm làm đổi cả f lẫn hệ số méo. Với ống kính lấy nét tự động, mọi tham số nội của bạn chỉ đúng ở đúng vị trí lấy nét lúc hiệu chuẩn. Đây là lý do hệ thống đo lường dùng ống kính khoá nét, và là lý do nên dán keo vào vòng lấy nét sau khi hiệu chuẩn.

Giả thiết cảm biến phẳng và lưới pixel đều. Đúng ở mức rất tốt với cảm biến hiện đại, nên đây hiếm khi là vấn đề — ghi ra để hoàn chỉnh danh sách giả thiết chứ không phải để lo lắng.

1.9 Thực hành

Bốn việc, theo thứ tự.

Một, ước lượng f từ FOV ngay khi thiết kế, bằng công thức ở mục 2. Bạn cần nó cho ngân sách sai số ở C/15 và bạn cần nó *trước* khi mua camera.

Hai, chọn mô hình méo theo FOV chứ không theo mặc định của thư viện, dùng bảng 1.1. Dùng mô hình quá nghèo là sai số mô hình và nó không chứa được bằng dữ liệu; dùng mô hình quá giàu là rủi ro khớp quá mức và nó chứa được bằng dữ liệu phủ rộng.

Ba, sau mỗi lần hiệu chuẩn, vẽ bản đồ residual theo vị trí ảnh trước khi nhìn con số RMS. Nếu thấy cấu trúc xuyên tâm hoặc xoáy ở góc thì mô hình sai dạng, và thêm ảnh không cứu được.

Bốn, kiểm c_x, c_y bằng cách hiệu chuẩn lại vài lần với các bộ ảnh khác nhau và xem chúng có ổn định không. Nếu điểm chính nhảy hàng chục pixel giữa các lần thì dữ liệu của bạn không đủ đa dạng về tư thế, và con số bạn đang dùng là ngẫu nhiên.

Active recall (che cột phải, tự trả lời, rồi mở ra đối chiếu)

Thuật ngữ / câu hỏi	Trả lời
● ● ● Ba bước của mô hình chiếu	Thế giới sang camera (R, t) ; chia cho Z ra toạ độ chuẩn hoá (bước phi tuyến duy nhất); áp méo rồi nhân K ra pixel. Méo định nghĩa trên toạ độ <i>chuẩn hoá</i> , không phải pixel.
● ● ● Tiêu cự tính bằng pixel	$f_x = F/s_x$, một tỉ số <i>không thứ nguyên</i> . Ước lượng nhanh từ FOV: $f_x \approx (N_x/2)/\tan(\theta_{\text{FOV}}/2)$. Ví dụ 1280 px, $60^\circ \Rightarrow 1109$ px.
● ● ● Vì sao méo phá detector marker đặc biệt	Detector khớp <i>đường thẳng</i> lên cạnh tag rồi lấy giao [7], nhưng méo xuyên tâm biến đường thẳng thành đường cong. Khớp thẳng lên cong cho đường sai, và giao hai đường sai bị khuếch đại.
● ● ● Méo là nhiều hay thiên lệch?	<i>Thiên lệch</i> . Với tag ở một vị trí cố định, méo đẩy cạnh theo cùng hướng ở mọi khung. Không chia cho $\sqrt{N}$ — đây là nguồn “hệ thống” ở A/05 mục 8.
● ● ● Độ lớn méo ở rìa	Ví dụ: $f = 600$, $k_1 = -0,3$, điểm cách tâm 400 px $\Rightarrow$ lệch 53 px. Lớn hơn $\sigma = 0,2$ px hơn 250 lần. Nên khử méo là bắt buộc, không phải tinh chỉnh.

Thuật ngữ / câu hỏi	Trả lời (tiếp)
●●● Điểm chính sai gây gì	Dịch mọi tia nhìn đi Δ/f radian. Với $\Delta = 10$ px, $f = 600$: $0,95^\circ$ — gần gấp đôi ngân sách góc. Không có triệu chứng thị giác nào.
●●● Vì sao c_x, c_y khó hiệu chuẩn	Chúng tương quan mạnh với tư thế target. Ảnh chỉ chụp gần vuông góc thì không tách được “điểm chính lệch trái” khỏi “target nghiêng phải”. Phải có tư thế nghiêng mạnh và phủ tới bốn góc.
●●● Sai số mô hình so với sai số tham số	Tham số: dạng hàm đúng, hệ số lệch — chứa được bằng thêm dữ liệu. Mô hình: dạng hàm không đủ — <i>không</i> chứa được bằng dữ liệu, thêm nghìn ảnh vẫn sai với khoảng tin cậy hẹp hơn.
●●● Phép thử phân biệt hai loại	Vẽ residual theo vị trí ảnh [10]. Nhiều trắng $\Rightarrow$ sai số tham số. Mẫu hình xuyên tâm hoặc xoáy ở góc $\Rightarrow$ sai số mô hình.
●● Vì sao RMS reprojection là chỉ số tệ	Trung bình che vùng rìa; nó giảm khi thêm tham số bất kể tham số có nghĩa hay không; và nó đo độ khớp trên dữ liệu hiệu chuẩn chứ không đo độ đúng khi ngoại suy sang vùng chưa có dữ liệu.
●● Khử méo toàn ảnh hay từng điểm	Toàn ảnh: giả định detector thành đúng, nhưng nội suy làm nhoè $\Rightarrow$ tăng σ . Từng điểm: giữ độ sắc nét, nhưng góc đã sai từ khâu khớp cạnh. Tốt hơn cả: đưa méo vào hàm sai số tái chiếu của PnP.
●● Chọn mô hình méo theo FOV	Dưới 50° : 2 hệ số radial. $50\text{--}90^\circ$: Brown–Conrady 5 tham số. $90\text{--}120^\circ$: hữu tỉ. Trên 120° : KB/fisheye [8]. Dùng nhằm theo hướng nghèo là sai số mô hình.
● Lỗi cài đặt phổ biến nhất	Tính r^2 bằng pixel thay vì tọa độ chuẩn hoá — sai theo hệ số f^2 . Triệu chứng: hiệu chuẩn vẫn hội tụ nhưng $k_1 \sim 10^{-7}$ thay vì cỡ -1 tới $+1$.
● Ba giả thiết hay bị quên	Tâm chiếu là điểm (sai khi chụp gần — nên hiệu chuẩn ở khoảng cách vận hành); mô hình không đổi (sai vì trôi nhiệt); lấy nét cố định (đổi nét là đổi cả f lẫn hệ số méo — nên khoá nét).

●●● phải thuộc ●● nên nắm ● biết là có

I Kiến thức này đang sống ở đâu (công trình đang dùng chính ý tưởng của chương; chốt 9/2026)

Công trình / hệ thống	Dùng ý tưởng nào của chương	Ghi chú
OpenCV <code>calibrateCamera</code> , <code>undistortPoints</code>	Cài Brown–Conrady và hữu tỉ; <code>undistortPoints</code> chính là “cách hai” ở mục 7	Đọc kỹ thứ tự tham số <code>distCoeffs</code> : k_1, k_2, p_1, p_2, k_3 — không phải thứ tự trong công thức
Kalibr [11]	Hỗ trợ nhiều mô hình méo và báo cáo residual theo vị trí ảnh	Công cụ duy nhất trong danh sách mặc định cho bạn bản đồ residual ở mục 6
deltille [12]	Target lưới tam giác cho định vị góc chính xác hơn, dùng khi cần intrinsic chất lượng cao	Nổi sang B/08; target tốt hơn cho cùng thuật toán
Hiệu chuẩn từ target phẳng [6]	Phương pháp nền của mọi hộp công cụ; giải thích vì sao cần nhiều tư thế nghiêng	Đọc §3 để thấy vì sao tư thế fronto-parallel không cho thông tin về c_x, c_y
Double Sphere [9]	Mô hình ít tham số cho FOV rất rộng	Lựa chọn hiện đại thay cho KB khi cần ít tham số

MINI PROJECT · Đo méo còn sót lại sau khi khử, và vẽ bản đồ residual · 1–2 ngày

Mục tiêu	biến câu “hiệu chuẩn của tôi có tốt không” thành một bản đồ hai chiều, và tự tay nhìn thấy sự khác nhau giữa sai số mô hình và sai số tham số.
Dữ liệu	một camera thật và một target ChArUco hoặc chessboard in trên nền cứng phẳng. Nếu chưa có phần cứng, mô phỏng: sinh ảnh bằng một mô hình méo “thật” (chẳng hạn hữu tỉ 8 tham số) rồi hiệu chuẩn bằng một mô hình nghèo hơn (2 hệ số) — cách này còn rõ hơn vì bạn biết chân trị.
Các bước	chụp ít nhất 30 ảnh phủ kín khung hình, <i>bắt buộc</i> có ảnh mà target chạm sát bốn góc và có ảnh nghiêng trên 45° ; hiệu chuẩn với ba mô hình khác nhau (2 hệ số, Brown–Conrady đầy đủ, hữu tỉ); với mỗi mô hình, lấy vectơ residual của từng điểm và vẽ chúng như một trường vectơ trên mặt phẳng ảnh, phóng đại 50 lần; so sánh cả ba bản đồ; đồng thời ghi lại c_x, c_y của ba lần và của các tập con ảnh khác nhau.
Xong khi	bạn có ba bản đồ residual đặt cạnh nhau và phát biểu được một câu kiểu “với mô hình 2 hệ số, residual có cấu trúc xuyên tâm rõ với độ lớn tới X px ở bán kính $r > Y$; với Brown–Conrady đầy đủ thì cấu trúc biến mất”. Kiểm tra chéo bắt buộc: nếu c_x, c_y nhảy hơn 5 px giữa các tập con ảnh, thì dữ liệu của bạn chưa đủ đa dạng tư thế và mọi kết luận về mô hình méo đều chưa đáng tin — chụp thêm ảnh nghiêng trước đã.
Nối với	B/12 cho quy trình hiệu chuẩn đầy đủ; A/05 dùng f mà bạn vừa đo.

TRẢ LỜI CÂU HỎI MỞ ĐẦU

- A1. Vì RMS là một trung bình trên toàn ảnh và trên toàn bộ dữ liệu hiệu chuẩn, nên nó có ba điểm mù. Nó che vùng rìa: hàng nghìn điểm giữa ảnh sai 0,1 px chìm chết vài chục điểm rìa sai 2 px. Nó luôn giảm khi thêm tham số, kể cả tham số vô nghĩa — nên so RMS giữa hai mô hình có số tham số khác nhau là vô nghĩa. Và quan trọng nhất, nó đo độ khớp *trên dữ liệu bạn đã cho* chứ không đo độ đúng ở vùng không có dữ liệu, mà tag của robot lúc tiếp cận rất có thể nằm đúng ở vùng ấy. Chỗ cần nhìn là bản đồ residual theo vị trí ảnh: nhiều trắng thì ổn, còn thấy cấu trúc xuyên tâm hay xoáy ở bốn góc thì mô hình sai dạng và thêm ảnh không cứu được.
- A2. Không phải 3 px, và thường là lớn hơn. Lý do là detector không lấy trực tiếp vị trí góc mà khớp một đường thẳng lên mỗi cạnh rồi lấy *giao* của hai đường. Méo làm cạnh cong, nên đường thẳng khớp lên nó bị lệch cả về vị trí lẫn về *hướng*; và một sai lệch hướng nhỏ của hai đường giao nhau sẽ chuyển thành sai lệch vị trí lớn ở giao điểm, với hệ số khuếch đại phụ thuộc góc giao — càng chệch thì càng khuếch đại. Cần thêm một điều: sai lệch này là thiên lệch chứ không phải nhiễu, nên lấy trung bình nghìn khung không giúp gì.
- A3. Nó dịch *mọi* tia nhìn đi một góc Δ/f radian — với $\Delta = 10$ px và $f = 600$ px thì là $0,95^\circ$, tức gần gấp đôi ngân sách góc của cây. Hậu quả thay đổi theo vị trí tag trong khung hình vì x_n vào phép chiếu cùng với phép chia cho Z , nên hiệu ứng của Δ lên sáu tham số pose không phải một phép quay thuần mà phụ thuộc cả vị trí lẫn khoảng cách. Điều làm nó nguy hiểm đặc biệt: nó không có triệu chứng thị giác nào — không ai nhìn ảnh mà phát hiện được điểm chính sai — và triệu chứng ở đầu ra thì trông hết như sai số hand-eye hoặc constellation sai, nên người ta hay đi tìm nhầm chỗ.
- A4. Khử méo toàn ảnh chính xác hơn về mặt mô hình, vì nó làm giả định “cạnh là đường thẳng” của detector trở thành đúng; nhưng nó đòi nội suy pixel, mà nội suy làm nhoè, mà nhoè làm tăng σ . Khử méo từng điểm nhanh hơn nhiều và giữ nguyên độ sắc nét, nhưng nó không sửa được việc bốn góc đã bị định vị sai từ khâu khớp cạnh trên ảnh cong. Hai câu trả lời khác nhau vì hai câu hỏi đo hai thứ khác nhau: một câu hỏi về tính đúng của mô hình, một câu hỏi về chi phí tính toán và về σ . Quy tắc thực hành: tag nhỏ và gần tâm thì dùng cách hai; tag lớn hoặc ở rìa ống kính góc rộng thì bắt buộc cách một; và tốt hơn cả hai là đưa mô hình méo thẳng vào hàm sai số tái chiếu của bước tinh chỉnh PnP.
- A5. Vì residual nhỏ chỉ chứng minh mô hình khớp được *dữ liệu bạn đã cho*, không chứng minh nó mô tả đúng ống kính. Với ống kính 120° , phép chiếu pinhole $r_n = \tan \theta$ đã gần phân kỳ, và không đa thức bậc bốn nào theo dõi kịp — nhưng bộ tối ưu vẫn tìm ra bộ (k_1, k_2) tốt nhất trong họ hàm nghèo ấy và báo về một con số RMS nghe được. Đây là sai số mô hình, và thêm một nghìn ảnh chỉ làm khoảng tin cậy của các hệ số hẹp lại quanh một giá trị vẫn sai — tệ hơn, vì bạn sẽ tin nó hơn. Dấu hiệu lộ ra: bản đồ residual có cấu trúc xuyên tâm với độ lớn tăng theo

bán kính; và nếu bạn hiệu chuẩn hai lần với hai tập ảnh phủ vùng khác nhau, các hệ số sẽ khác nhau đáng kể vì mỗi lần mô hình đang “cố” khớp một vùng khác.

NÓI LẠI THẬT ĐƠN GIẢN

Một cái camera về bản chất là một cái hộp có một lỗ nhỏ. Ánh sáng từ mọi vật chui qua cái lỗ ấy rồi đập vào thành sau của hộp, và ảnh hiện ra. Nếu cái lỗ thật sự nhỏ như một điểm và ánh sáng đi thẳng, thì mọi thứ rất dễ tính: cứ kẻ một đường từ vật qua cái lỗ, nó đập vào đâu thì ảnh ở đó. Vấn đề là camera thật không có lỗ nhỏ, nó có thấu kính — vì lỗ nhỏ cho quá ít ánh sáng. Thấu kính thu được nhiều sáng hơn nhưng nó bẻ cong tia sáng, và nó bẻ tia ở rìa nhiều hơn tia ở giữa. Kết quả là một cái cửa sổ hình chữ nhật chụp bằng ống kính góc rộng trông phình ra như quả trứng.

Với phần lớn bài toán thị giác thì chuyện đó không quá tệ, vì người ta chỉ cần biết từng điểm nằm đâu, và có công thức để trả điểm về đúng chỗ. Nhưng cái tag thì khác, và đây là chỗ cần chú ý. Máy tính tìm góc của tag bằng cách nhìn bốn cạnh, kẻ bốn đường thẳng dọc theo chúng, rồi lấy chỗ các đường cắt nhau. Cách ấy rất khôn vì nó dùng cả trăm pixel dọc cạnh chứ không chỉ vài pixel ở góc. Nhưng nó chỉ đúng nếu cạnh thật sự thẳng — mà ống kính vừa làm chúng cong.

Kẻ một đường thẳng dọc theo một đường cong thì được một đường lệch. Lấy chỗ hai đường lệch cắt nhau thì được một điểm lệch nhiều hơn nữa — giống như hai cây gậy dài chạm nhau ở đầu: nghiêng mỗi cây một chút thôi, đầu gậy đã trượt đi khá xa.

Còn một thứ nữa, kín đáo hơn và vì thế phiền hơn. Cái lỗ của camera đáng lẽ nằm đúng giữa thành sau, nhưng khi lắp ráp thì nó lệch đi một chút. Chuyện lệch ấy không làm ảnh trông khác đi chút nào — bạn không có cách nào nhìn một tấm ảnh rồi bảo “lỗ này lệch sang trái mười pixel”, vì trong ảnh không có gì để so. Nhưng máy tính lại dùng vị trí của cái lỗ để tính hướng nhìn, nên lệch mười pixel tương đương với tính sai hướng gần một độ. Và một độ thì đã quá ngân sách của chúng ta rồi.

Bài học chung của cả chương: có những sai lệch làm ảnh trông xấu, và có những sai lệch không làm ảnh trông khác gì cả. Loại thứ hai mới là loại phải cẩn thận, vì không có gì nhắc bạn rằng chúng đang ở đó.

Chỗ không rút gọn được: việc chọn mô hình méo nào cho ống kính nào thì không có quy tắc đơn giản đúng cho mọi trường hợp. Bảng trong chương là hướng dẫn theo góc nhìn, và nó đúng phần lớn thời gian, nhưng cách chắc chắn duy nhất là hiệu chuẩn với vài mô hình rồi nhìn bản đồ residual xem cái nào hết cấu trúc.

Một câu: Ống kính bẻ cong đường thẳng, còn máy tính thì cứ tưởng chúng thẳng — và cái lỗ lệch mười pixel thì không ai nhìn thấy nhưng nó ăn gần một độ.

1.10 Điều gì chứng minh cách hiểu này sai

Mệnh đề chính — *méo chưa khử làm cong cạnh tag, và việc khớp đường thẳng lên cạnh cong khuếch đại sai số tại giao điểm* — bị bác bỏ nếu một mô phỏng cho thấy sai số góc do méo gây ra nhỏ hơn hoặc bằng độ lớn của méo tại vị trí góc đó. Tôi dẫn mệnh đề bằng hình học (cửa a) và bằng ví dụ số cho độ lớn méo (cửa b cho phần độ lớn), nhưng phần *khuếch đại tại giao điểm* thì tôi chưa đo bằng mô phỏng — đó là một nợ kiểm chứng.

Mệnh đề thứ hai — *điểm chính sai Δ px gây thiên lệch hướng cỡ Δ/f radian* — là mệnh đề yếu nhất về mặt kiểm chứng trong chương, như đã ghi thẳng ở mục 5. Nó bị bác bỏ nếu mô phỏng cho thấy sai số pose do Δ gây ra được hấp thụ gần hết vào các tham số khác và biên độ còn lại nhỏ hơn nhiều so với Δ/f . Đây là mệnh đề tôi khuyên kiểm sớm nhất, vì nếu nó sai thì mức ưu tiên tôi gán cho việc hiệu chuẩn điểm chính ở B/12 là quá cao.

Mệnh đề thứ ba — *sai số mô hình không chữa được bằng thêm dữ liệu* — là mệnh đề chắc chắn nhất, vì nó là một mệnh đề về bản chất của khớp trong một họ hàm hạn chế, không phải một mệnh

để thực nghiệm. Nó “bị bác bỏ” chỉ nếu ai đó chỉ ra rằng họ hàm bạn dùng thật ra có chứa hàm đúng — tức là chỉ ra rằng bạn chẩn đoán nhầm, chứ không phải mệnh đề sai.

Cuối cùng, một mệnh đề mềm: chương này khẳng định rằng *trong thực tế, điểm chính là tham số nội gây hại nhất*. Điều đó bị bác bỏ nếu trong một hệ thực, sau khi đã hiệu chuẩn điểm chính cẩn thận, sai số xoay không cải thiện đáng kể so với khi dùng giá trị mặc định giữa ảnh. (Tôi phát biểu dựa trên cấu trúc của phép chiếu và trên sự thiếu triệu chứng của lỗi này, không dựa trên thống kê cao hồng thực tế.)

1.11 Kết nối

Chương này là gốc của Phần A và nó cấp vật liệu cho gần như mọi chương sau.

Nối xuống dưới trong Phần A. [A/02-homography-va-hinh-hoc-phang](#) dùng ma trận K trực tiếp trong phân rã $H = K[r_1 \ r_2 \ t]$, và nó giả định ảnh đã được khử méo — một giả định mà chương này vừa cho thấy không hiển nhiên. [A/03-pnp-va-uoc-luong-tu-the](#) dùng mô hình chiếu ở mục 2 làm hàm $\hat{u}(\xi)$ trong sai số tái chiếu, và mục 7 ở đây bàn chỗ đưa méo vào hàm ấy. [A/05-lan-truyen-sai-so-pixel-sang-pose](#) lấy f làm mẫu số của công thức sai số ngang, và nó lấy từ mục 6 ở đây luận điểm quan trọng rằng “mô hình đúng” là một giả thiết chứ không phải một sự thật — khi giả thiết ấy vi phạm, hiệp phương sai mà chương 5 tính ra là sai theo chiều nguy hiểm.

Nối sang Phần B. [B/07-detection-pipeline](#) là chương mô tả chi tiết cơ chế khớp cạnh mà mục 4 ở đây dựa vào; đọc hai mục ấy cùng nhau thì thấy rõ vì sao méo và detector tương tác xấu. [B/08-corner-refinement](#) quyết định σ , và mục 7 ở đây cảnh báo rằng khử méo toàn ảnh làm tăng chính σ ấy — một đánh đổi phải cân. [B/12-calibration-cho-docking](#) là chương nhận trực tiếp và đầy đủ nhất: toàn bộ mục 5 và mục 6 ở đây trở thành quy trình cụ thể ở đó, gồm cả yêu cầu về độ phủ dữ liệu và phép thử residual có cấu trúc.

Nối sang Phần C. [C/15-ngan-sach-sai-so](#) có một dòng riêng cho “méo còn sót lại” và một dòng riêng cho “sai số intrinsic”, và cả hai dòng ấy được định giá bằng các cơ chế ở chương này. Đáng chú ý là chúng thuộc loại *hệ thống* theo phân loại ở [A/05](#) mục 8, nên chúng không giảm khi lấy trung bình nhiều khung — và đó là lý do chúng thường chiếm phần lớn ngân sách trong một hệ thống đã tối ưu tốt phần nhiều.

Câu hỏi tự kiểm (trả lời không nhìn bài)

1. Vì sao phép méo xuyên tâm biến một đường thẳng thành đường cong, nhưng có đúng một họ đường thẳng vẫn thẳng sau khi méo — họ nào, và vì sao?
2. Detector khớp đường thẳng lên cạnh tag. Nếu chính xác chuỗi ba bước biến “ảnh còn méo 3 px” thành “pose sai”, và chỉ ra bước nào khuếch đại sai số.
3. Bạn hiệu chuẩn hai lần với hai tập ảnh khác nhau và được hai giá trị c_x cách nhau 15 px. Điều đó nói gì về dữ liệu của bạn, và vì sao f_x của hai lần lại có thể gần như trùng nhau?
4. Vì sao thêm tham số vào mô hình méo luôn làm RMS reprojection giảm, và vì sao điều đó không chứng minh mô hình tốt hơn?
5. Một đồng nghiệp khử méo toàn ảnh bằng nội suy song tuyến tính rồi thấy σ tăng. Giải thích cơ chế, và nêu hai cách giữ được cả tính đúng của mô hình lẫn độ sắc nét.
6. Với $\Delta c_x = 10$ px và $f = 600$ px, thiên lệch hướng là bao nhiêu độ? Vì sao thiên lệch ấy phần lớn triệt tiêu trong sơ đồ teach-by-showing nhưng *không* triệt tiêu hoàn toàn?
7. Nêu ba giả thiết ngầm của mô hình pinhole cộng méo mà chương này liệt kê, và với mỗi giả thiết, một tình huống docking cụ thể làm nó bị vi phạm.

Homography và hình học phẳng (Homography and Planar Geometry)

PHỤ THUỘC A/01-mo-hinh-camera-va-distortion (ma trận K , tọa độ chuẩn hoá, và — quan trọng — cảnh báo rằng mọi thứ trong chương này giả định ảnh đã được khử méo; nếu chưa thì không có homography nào khớp được).

TÓM TẮT

Chương này dựng công cụ cho trường hợp riêng mà toàn bộ bài toán marker rơi vào: mọi điểm điều khiển nằm trên một mặt phẳng. Trường hợp riêng ấy đơn giản hơn trường hợp tổng quát một cách đáng kể — quan hệ giữa mặt phẳng vật và mặt phẳng ảnh là một phép biến đổi tuyến tính trong toạ độ thuần nhất, gọi là homography, chỉ có tám bậc tự do và giải được bằng bốn điểm. Ba điều đáng mang ra khỏi chương. Một, công thức $H = K[r_1 \ r_2 \ t]$ và phép phân rã ngược để lấy pose — nó là cách khởi tạo chuẩn cho PnP. Hai, số điều kiện của H xấu đi nhanh khi tag nhỏ trên ảnh, và đó là lý do định lượng cho việc đặt ngưỡng kích thước tối thiểu. Ba, và đây là mục ít người biết nhất: *tâm của hình chiếu không bằng hình chiếu của tâm*. Với marker tròn, sự thật ấy tạo ra một thiên lệch có hệ thống phụ thuộc tư thế, và nó là lý do marker tròn cần hiệu chỉnh conic chứ không dùng thẳng tâm ellipse.

KIẾN THỨC CẦN CÓ TRƯỚC

- **homography** — phép biến đổi tuyến tính khả nghịch trên toạ độ thuần nhất 2D, biểu diễn bởi một ma trận 3×3 xác định sai khác một hệ số. Nó là quan hệ chính xác giữa hai ảnh của cùng một mặt phẳng, và cũng là quan hệ giữa mặt phẳng vật và ảnh của nó.
- **bậc tự do của H** — tám, không phải chín: ma trận có chín phần tử nhưng nhân toàn bộ với một hằng số cho cùng một phép biến đổi, nên một bậc tự do là dư. Hệ quả thực hành: cần bốn cặp điểm tương ứng (mỗi cặp cho hai phương trình) để xác định H .
- **DLT** — *Direct Linear Transform*, phương pháp giải H bằng cách viết các ràng buộc thành một hệ tuyến tính thuần nhất $Ah = 0$ rồi lấy vectơ kỳ dị ứng với giá trị kỳ dị nhỏ nhất.
- **chuẩn hoá Hartley** — phép biến đổi tiền xử lý đưa tập điểm về trung bình không và khoảng cách trung bình tới gốc bằng $\sqrt{2}$, trước khi chạy DLT. Nó không phải một mẹo làm đẹp: nó thay đổi số điều kiện của A vài bậc độ lớn, và bỏ nó là lỗi cài đặt kinh điển [13, §4.4].
- **số điều kiện (condition number)** — tỉ số giữa giá trị kỳ dị lớn nhất và nhỏ nhất của một ma trận. Nó đo mức khuếch đại sai số: số điều kiện 10^6 nghĩa là nhiễu đầu vào có thể bị nhân lên triệu lần ở một hướng nào đó của nghiệm.
- **tỉ số kép (cross-ratio)** — đại lượng gồm bốn điểm thẳng hàng, $(AC/BC)/(AD/BD)$, và là bất biến *duy nhất* của phép chiếu phối cảnh trên một đường thẳng. Mọi thứ khác — khoảng cách, tỉ số hai đoạn, trung điểm — đều không bảo toàn.
- **conic** — đường cong bậc hai trên mặt phẳng, biểu diễn bởi ma trận đối xứng 3×3 : $\mathbf{x}^T C \mathbf{x} = 0$. Đường tròn và ellipse đều là conic, và ảnh phối cảnh của một conic vẫn là một conic — đây là tính chất làm marker tròn xử lý được bằng đại số tuyến tính.

CÂU HỎI MỞ ĐẦU

- Q1. Homography có tám bậc tự do còn pose chỉ có sáu. Hai bậc dư ấy chứa gì, và tại sao sự dư thừa ấy lại là một công cụ chẩn đoán miễn phí?
- Q2. Bạn vẽ một vòng tròn lên tám marker rồi tìm tâm của ellipse mà nó tạo ra trên ảnh. Vì sao điểm bạn vừa tìm *không* phải hình chiếu của tâm vòng tròn, và sai lệch ấy phụ thuộc vào cái gì?
- Q3. Tag của bạn co lại từ 200 px xuống 40 px trên ảnh khi robot lùi ra xa. Nhiều góc σ không đổi. Pose lại xấu đi nhanh hơn nhiều so với tỉ lệ $1/w$ mà A/05 dự đoán. Cơ chế nào trong chương này giải thích phần vượt trội đó?
- Q4. Một đồng nghiệp cài DLT để giải homography, không chuẩn hoá điểm trước, và kết quả “chạy được” trên dữ liệu thử. Vì sao nó sẽ hỏng trên dữ liệu thật, và hỏng theo kiểu nào?
- Q5. Trong bốn bất biến sau, cái nào được phép chiếu phối cảnh bảo toàn: khoảng cách, tỉ số hai đoạn thẳng hàng, trung điểm, tỉ số kép? Và vì sao câu trả lời quyết định cách bạn thiết kế marker?

2.1 Đặt vấn đề

Bài toán ước lượng pose từ ảnh nói chung là một bài toán khó: sáu ẩn, quan hệ phi tuyến, nhiều nghiệm, cần lập. Nhưng bài toán marker rơi vào một trường hợp riêng mà hình học ưu ái đặc biệt — *mọi điểm điều khiển nằm trên một mặt phẳng* — và trường hợp riêng ấy có một cấu trúc đẹp đến mức đáng dành một chương.

Cấu trúc ấy là thế này. Khi vật là một mặt phẳng, quan hệ giữa toạ độ trên mặt phẳng vật và toạ độ trên ảnh là *tuyến tính* trong toạ độ thuần nhất. Không có phép chia phi tuyến còn lại, không cần lập, không có nhiều nghiệm ở tầng này: bốn điểm là đủ và lời giải là một bài toán trị riêng. Đây là lý do lịch sử vì sao hiệu chuẩn camera bằng target phẳng — phương pháp Zhang [6] — lại thay thế được các phương pháp cũ dùng vật hiệu chuẩn ba chiều đắt tiền: mặt phẳng dễ chế tạo chính xác, và toán học của nó dễ hơn.

Cấu trúc ấy cũng là lý do vì sao mọi detector marker đều dùng homography ở giữa quy trình. ArUco rectify vùng tag về hình vuông chuẩn bằng homography rồi mới lấy mẫu các ô bit [14]; không có bước ấy thì việc đọc bit từ một tứ giác méo là một cơn ác mộng. AprilTag dùng homography để khởi tạo pose trước khi tinh chỉnh. ChArUco dùng nó để dự đoán vị trí các góc chessboard sau khi đã định danh bằng các ô ArUco.

Nhưng chương này không tồn tại chỉ để trình bày một công cụ tiện lợi. Nó tồn tại vì hình học phẳng mang theo *hai cái bẫy* mà người dùng công cụ không thấy.

Cái bẫy thứ nhất là về điều kiện số. Homography luôn giải được về mặt đại số, kể cả khi bốn điểm gần trùng nhau hoặc gần thẳng hàng — bộ giải trả về một ma trận và không báo lỗi. Nhưng chất lượng của ma trận ấy phụ thuộc dữ dội vào hình học của bốn điểm, và sự phụ thuộc ấy không xuất hiện trong bất kỳ thông báo nào. Mục 4 định lượng chuyện này, và nó giải thích một hiện tượng mà A/05 một mình không giải thích được: vì sao pose xấu đi *nhANH* hơn tỉ lệ tuyến tính khi tag co lại.

Cái bẫy thứ hai tinh vi hơn nhiều và nó là lý do chính tôi xếp chương này ở mức quan trọng thay vì tham khảo. Phép chiếu phối cảnh *không bảo toàn trung điểm*. Nghe thì vô hại, nhưng nó có một hệ quả rất cụ thể: nếu bạn vẽ một hình tròn và tìm tâm của ellipse mà nó tạo trên ảnh, điểm bạn tìm được *không* phải hình chiếu của tâm hình tròn. Sai lệch ấy là một thiên lệch có hệ thống, nó phụ thuộc tư thế, và nó là nguồn sai số chính khi dùng marker tròn nếu không hiệu chỉnh. Kết quả này đã được biết rõ trong cộng đồng hiệu chuẩn camera từ hai thập niên [15], [16] và nó vẫn tiếp tục làm người mới bất ngờ.

TRỰC GIÁC

Hình dung việc chiếu bóng một tấm bìa lên tường bằng một cái đèn pin.

Nếu tấm bìa song song với tường, bóng của nó là một bản phóng to trung thực: hình vuông vẫn vuông, trung điểm vẫn ở giữa, hai đoạn bằng nhau vẫn bằng nhau. Đây là phép đồng dạng, và mọi trực giác hình học quen thuộc đều còn đúng.

Nghiêng tấm bìa đi, và mọi thứ bắt đầu lệch. Hình vuông thành hình thang. Cạnh xa đèn trông ngắn hơn cạnh gần. Và — đây là chỗ đáng chú ý — *trung điểm của cạnh không còn chiếu thành trung điểm của bóng*: vì phần gần đèn bị phóng to nhiều hơn phần xa, điểm giữa bị đẩy về phía xa.

Cái mà phép chiếu *vẫn* bảo toàn không phải là khoảng cách hay tỉ lệ mà là một đại lượng phức tạp hơn gồm bốn điểm, gọi là tỉ số kép. Bốn điểm thẳng hàng có một con số gắn với chúng, và con số ấy giống hệt nhau ở vật và ở bóng, dù nghiêng thế nào. Nó là bất biến *duy nhất*, và mọi thứ đơn giản hơn nó đều không bảo toàn.

Giờ áp vào marker. Một cái tag vuông có bốn góc, và bốn góc ấy là bốn điểm rời rạc xác định rõ — phép chiếu đưa chúng đi đâu thì đi, ta vẫn biết điểm nào là điểm nào. Nhưng một cái marker tròn thì không có góc: cái ta đo được là *tâm của đường cong*, mà tâm là một khái niệm dựa trên trung điểm, mà trung điểm thì không bảo toàn. Ta đang đo một thứ không phải thứ ta nghĩ mình đang đo.

2.2 Homography của một mặt phẳng

Dẫn công thức, vì nó ngắn và vì mỗi bước đều nói một điều.

Chọn hệ toạ độ trên mặt phẳng marker sao cho mặt phẳng ấy là $Z = 0$. Một điểm trên marker có toạ độ $(X, Y, 0)$. Áp mô hình chiếu của A/01:

$$\begin{pmatrix} u \\ v \\ 1 \end{pmatrix} \sim K [\mathbf{r}_1 \ \mathbf{r}_2 \ \mathbf{r}_3 \ \mathbf{t}] \begin{pmatrix} X \\ Y \\ 0 \\ 1 \end{pmatrix} = K [\mathbf{r}_1 \ \mathbf{r}_2 \ \mathbf{t}] \begin{pmatrix} X \\ Y \\ 1 \end{pmatrix},$$

trong đó $\mathbf{r}_i$ là các cột của R , và dấu $\sim$ nghĩa là bằng nhau sai khác một hệ số khác không.

Cột thứ ba $\mathbf{r}_3$ biến mất vì nó nhân với $Z = 0$. Đó là toàn bộ mẹo, và nó đáng dừng lại một câu: chính việc *mất* một cột làm cho quan hệ trở thành một ma trận 3×3 — tức một phép biến đổi tuyến tính giữa hai mặt phẳng — thay vì một phép chiếu từ ba chiều xuống hai chiều. Ta định nghĩa

$$H = K [\mathbf{r}_1 \ \mathbf{r}_2 \ \mathbf{t}]. \quad (2.1)$$

Đây cũng là chỗ trả lời Q1 về hai bậc tự do dư. H có tám bậc tự do (chín phần tử trừ một hệ số tỉ lệ). Pose có sáu. Vậy hai bậc dư nằm ở đâu? Chúng nằm ở chỗ một ma trận 3×3 bất kỳ *không* nhất thiết có dạng $K[\mathbf{r}_1 \ \mathbf{r}_2 \ \mathbf{t}]$ với $\mathbf{r}_1, \mathbf{r}_2$ trực chuẩn. Hai ràng buộc $\|\mathbf{r}_1\| = \|\mathbf{r}_2\|$ và $\mathbf{r}_1^\top \mathbf{r}_2 = 0$ chính là hai bậc tự do bị loại đi khi ta đi từ H tổng quát về một pose hợp lệ.

Và đây là chỗ sự dư thừa trở thành công cụ chẩn đoán miễn phí — phần thứ hai của Q1. Sau khi tính H từ dữ liệu bằng DLT, ta có thể kiểm xem $K^{-1}H$ có *thật sự* thoả hai ràng buộc trực chuẩn không. Nếu hai cột đầu của $K^{-1}H$ có độ dài lệch nhau 20% hoặc tích vô hướng của chúng khác không đáng kể, thì một trong ba điều đã sai: các điểm không thực sự đồng phẳng, ảnh chưa khử méo đúng, hoặc K sai. Phép kiểm ấy tốn vài dòng code và nó bắt được cả ba lỗi phổ biến nhất — đáng ngạc nhiên là rất ít cài đặt làm nó.

2.3 Phân rã H thành pose

Đi ngược từ H về $(R, \mathbf{t})$. Từ (2.1), đặt $M = K^{-1}H = [\mathbf{m}_1 \ \mathbf{m}_2 \ \mathbf{m}_3]$. Về nguyên tắc $\mathbf{m}_1 = \lambda \mathbf{r}_1$, $\mathbf{m}_2 = \lambda \mathbf{r}_2$, $\mathbf{m}_3 = \lambda \mathbf{t}$ với một hệ số λ chưa biết.

Vì $\|\mathbf{r}_1\| = 1$, ta có $\lambda = \|\mathbf{m}_1\|$. Trong thực tế, do nhiễu, $\|\mathbf{m}_1\| \neq \|\mathbf{m}_2\|$, nên cách dùng phổ biến là lấy trung bình:

$$\lambda = \frac{1}{2}(\|\mathbf{m}_1\| + \|\mathbf{m}_2\|).$$

Rồi

$$\mathbf{r}_1 = \mathbf{m}_1/\lambda, \quad \mathbf{r}_2 = \mathbf{m}_2/\lambda, \quad \mathbf{r}_3 = \mathbf{r}_1 \times \mathbf{r}_2, \quad \mathbf{t} = \mathbf{m}_3/\lambda.$$

Cột thứ ba lấy bằng tích có hướng vì R trực giao thuận. Đây là quy trình chuẩn, chép ý từ [6, §3.1] với ký hiệu đổi sang của cây.

Ma trận R thu được theo cách này *không* trực giao chính xác vì nhiễu, nên phải chiếu nó về $SO(3)$ gần nhất. Cách đúng là qua phân tích SVD: nếu $R = U\Sigma V^T$ thì ma trận trực giao gần nhất theo chuẩn Frobenius là UV^T — một kết quả cổ điển gọi là bài toán Procrustes trực giao. Cần chú ý một chi tiết cài đặt: phải kiểm $\det(UV^T) = +1$; nếu ra -1 thì đã thu được một phép phản xạ chứ không phải phép quay, và phải đổi dấu cột cuối của V .

Cũng có một tình huống dấu cần xử lý: H xác định sai khác một hệ số, kể cả dấu, nên có thể ra $\mathbf{t}$ với $t_z < 0$ — tức marker nằm *sau* camera. Vô lý về vật lý, nên nếu gặp thì đổi dấu toàn bộ H rồi làm lại.

CẠM BẮY

Phép phân rã ở mục này cho ra một pose *hợp lệ*, không phải pose *tốt nhất*. Ba lý do, và cả ba đều dẫn tới cùng một kết luận thực hành.

Thứ nhất, DLT cực tiểu hoá một sai số đại số ($\|A\mathbf{h}\|$) chứ không phải sai số hình học có ý nghĩa. Đại lượng ta thật sự quan tâm là sai số tái chiếu tính bằng pixel, và hai thứ đó không tỉ lệ với nhau.

Thứ hai, phép trung bình λ và phép chiếu về $SO(3)$ đều là những cách sửa *sau khi* đã có nghiệm, không phải cách tìm nghiệm tốt hơn. Chúng đảm bảo kết quả là một pose hợp lệ nhưng không đảm bảo nó tối ưu theo tiêu chí nào cả.

Thứ ba, phép phân rã không nói gì về bài toán hai nghiệm của A/04. Nó cho ra một nghiệm, và nghiệm ấy có thể là nghiệm sai.

Kết luận: *luôn coi kết quả phân rã H là điểm khởi đầu, không phải kết quả*. Chạy tinh chỉnh phi tuyến trên sai số tái chiếu sau đó (A/03), và với target phẳng thì dùng IPPE để lấy *cả hai* nghiệm thay vì phân rã H — IPPE làm đúng việc này và làm tốt hơn [17].

2.4 Điều kiện số: vì sao tag nhỏ tệ hơn công thức dự đoán

Đây là mục có giá trị thực hành cao nhất trong chương, và nó trả lời Q3.

A/05 cho quan hệ $\delta Z \approx Z\sigma/w$ — sai số dọc trực tỉ lệ nghịch với kích thước tag trên ảnh. Đó là một quan hệ tuyến tính và nó đúng trong chế độ hoạt động tốt. Nhưng người ta quan sát được rằng khi tag co xuống dưới một ngưỡng nào đó, chất lượng pose xuống dốc nhanh hơn nhiều so với $1/w$. Cơ chế của phần vượt trội ấy nằm ở chương này.

Bài toán DLT là giải hệ thuần nhất $A\mathbf{h} = 0$, với A dựng từ toạ độ các điểm tương ứng. Chất lượng nghiệm phụ thuộc vào *số điều kiện* của A , tức tỉ số giữa giá trị kỳ dị lớn nhất và nhỏ nhất. Số điều kiện càng lớn thì nhiễu đầu vào càng bị khuếch đại.

Có hai cơ chế làm số điều kiện xấu đi.

Cơ chế thứ nhất, và nó khắc phục được: thang đo không đồng nhất. Các phần tử của A gồm các tích như uX , u , X , và 1 . Nếu u cỡ 10^3 (toạ độ pixel) còn X cỡ 10^{-1} (mét), thì các phần tử của A trải qua nhiều bậc độ lớn, và số điều kiện tệ ngay từ đầu bất kể hình học có tốt hay không. Cách khắc phục là *chuẩn hoá Hartley*: đưa cả hai tập điểm về trung bình không và khoảng cách trung bình $\sqrt{2}$ trước khi dựng A , rồi biến đổi ngược H sau khi giải [13, §4.4]. Đây không phải tùy chọn — nó cải thiện số điều kiện vài bậc độ lớn, và bỏ nó là lỗi cài đặt kinh điển. Đây cũng là câu trả lời cho Q4: code không chuẩn hoá “chạy được” trên dữ liệu thử vì dữ liệu thử thường có thang đo hiện, và nó hỏng trên dữ liệu thật theo kiểu khó truy nhất — kết quả không sai hẵn mà chỉ kém chính xác một cách thất thường, tệ hơn ở một số tư thế và ổn ở tư thế khác.

Cơ chế thứ hai, và nó không khắc phục được bằng code: hình học của bốn điểm. Kể cả sau khi chuẩn hoá, số điều kiện vẫn phụ thuộc vào việc bốn điểm phân bố thế nào. Bốn điểm trải rộng thành một tứ giác gần vuông cho điều kiện tốt; bốn điểm gần trùng nhau, gần thẳng hàng, hoặc tạo thành một tứ giác rất dẹt cho điều kiện xấu. Và khi tag co nhỏ trên ảnh, bốn góc của nó gần trùng nhau theo nghĩa toạ độ pixel của chúng chênh nhau ít — nên điều kiện xấu đi.

Vì sao điều đó gây tệ hơn tuyến tính? Vì hai hiệu ứng chồng lên nhau. Sai số góc σ cố định chiếm một *tỉ lệ* ngày càng lớn trong kích thước tag khi tag co lại (hiệu ứng thứ nhất, tuyến tính, đã có trong A/05), và đồng thời phép giải trở nên kém điều kiện nên khuếch đại phần sai số ấy nhiều hơn (hiệu ứng thứ hai). Tích của hai hiệu ứng tệ hơn mỗi cái riêng lẻ.

Ghi chú trung thực: tôi dẫn cơ chế này bằng lập luận về số điều kiện (cửa a) nhưng tôi chưa có ví dụ số đo mức độ vượt trội so với $1/w$ (cửa b chưa qua). Vì vậy tôi phát biểu nó ở dạng định tính — “tệ hơn tuyến tính” — chứ không gán một số mũ. Đây là một mục trong 99-chua-biet.md, và nó đo được dễ dàng bằng script Monte Carlo của C/16.

CẦN NHỚ

Hai quy tắc thực hành rút ra từ mục này, và cả hai đều rẻ.

Một, *luôn chuẩn hoá điểm trước DLT*. Nếu bạn dùng findHomography của OpenCV thì nó đã làm sẵn; nếu bạn tự cài thì đừng bỏ qua.

Hai, *đặt một ngưỡng kích thước tag tối thiểu và từ chối phát hiện dưới ngưỡng ấy*. Ngưỡng thường dùng là khoảng 40–60 px cho cạnh tag, nhưng con số đúng cho hệ của bạn phải đo: chạy Monte Carlo với w giảm dần và tìm chỗ sai số bắt đầu chệch khỏi đường $1/w$. Đó là ngưỡng của bạn.

Hệ quả thiết kế quan trọng hơn cả hai: đây chính là lý do của chiến lược *nested multi-scale tags* ở B/09 — tag lớn cho cự ly xa, tag nhỏ cho cự ly gần, sao cho luôn có ít nhất một tag nằm trong dải kích thước tốt trên ảnh.

2.5 Bất biến của phép chiếu: chỉ có tỉ số kép

Mục này ngắn nhưng nó là nền của mục 6, nên đừng lướt.

Câu hỏi: phép chiếu phối cảnh bảo toàn cái gì? Danh sách ngắn hơn trực giác gợi ý nhiều.

Không bảo toàn: khoảng cách (hiển nhiên); tỉ số hai đoạn thẳng hàng (cạnh xa trông ngắn hơn); trung điểm (hệ quả của điều trước); song song (hai đường ray gặp nhau ở đường chân trời); góc; diện tích.

Có bảo toàn: tính thẳng hàng (đường thẳng vẫn là đường thẳng); tính đồng quy; bậc của đường cong đại số (conic vẫn là conic); và *tỉ số kép*.

Tỉ số kép của bốn điểm thẳng hàng A, B, C, D là

$$\text{cr}(A, B; C, D) = \frac{\overline{AC}/\overline{BC}}{\overline{AD}/\overline{BD}},$$

và nó giống hệt nhau ở vật và ở ảnh. Đây là câu trả lời cho Q5, và điều đáng chú ý không phải bản thân công thức mà là *tính duy nhất* của nó: tỉ số kép là bất biến độc lập duy nhất của phép chiếu trên một đường thẳng, nên bất kỳ đại lượng nào bạn muốn đo mà bảo toàn qua phép chiếu thì phải viết được theo nó.

Hệ quả thiết kế cho marker, và đây là chỗ mục này trả giá trị. Một marker mã hoá thông tin bằng *khoảng cách* hoặc bằng *tỉ lệ hai đoạn* sẽ không đọc được tin cậy dưới góc nghiêng, vì những đại lượng ấy không bảo toàn. Một marker mã hoá bằng *cấu trúc rời rạc* — các ô đen trắng trong một lưới, như ArUco và AprilTag — thì đọc được, vì sau khi rectify bằng homography thì lưới trở lại đều. Và một marker mã hoá bằng tỉ số kép trực tiếp thì đọc được mà *không cần* rectify — một ý tưởng đẹp đã được dùng trong một số thiết kế marker nhưng nó không thắng được cách tiếp cận lưới về mặt thực dụng, vì đo bốn điểm chính xác khó hơn đọc một ô sáng hay tối.

2.6 Bias tâm ellipse: tâm chiếu không bằng chiếu của tâm

Đây là mục ít được biết nhất và có giá trị cao nhất trong chương, và nó là câu trả lời cho Q2.

Vẽ một hình tròn bán kính R lên tấm marker. Ảnh của nó dưới phép chiếu phối cảnh là một ellipse — điều này đúng và nó là hệ quả của việc conic chiếu thành conic. Bây giờ câu hỏi: tâm của ellipse ấy có phải là ảnh của tâm hình tròn không?

Không. Và lý do nằm đúng ở mục 5: tâm của hình tròn là trung điểm của mọi dây cung đi qua nó, mà trung điểm *không* bảo toàn qua phép chiếu. Cụ thể hơn: xét một đường kính của hình tròn nằm theo hướng nghiêng ra xa camera. Nửa gần camera được phóng to nhiều hơn nửa xa, nên trên ảnh, ảnh của tâm bị đẩy về phía *xa*, trong khi tâm của ellipse — theo định nghĩa là trung điểm của các dây cung của chính ellipse — nằm ở chỗ khác.

Kết quả này là kiến thức chuẩn trong hiệu chuẩn camera chính xác [15], và Mallon cùng Whelan đã so sánh định lượng mức thiên lệch của các loại target khác nhau, cho thấy target tròn chịu thiên lệch mà target chessboard thì không [16]. Đây là một trong số ít chỗ trong cây mà tôi có được hai nguồn độc lập khẳng định cùng một điều, nên mức tin cậy cao.

Thiên lệch phụ thuộc vào ba thứ, và biết chúng là biết khi nào lo. Nó tăng theo *kích thước hình tròn so với khoảng cách* — hình tròn nhỏ ở xa thì thiên lệch nhỏ. Nó tăng theo *góc nghiêng* — fronto-parallel thì thiên lệch bằng không (một trong số ít chỗ mà fronto-parallel là tư thế tốt). Và nó tăng theo *độ lệch khỏi tâm ảnh*, vì tia nhìn tới một hình tròn ở rìa ảnh chệch nhiều hơn.

Cách chữa là dùng *đại số conic* thay vì lấy tâm ellipse. Ảnh của hình tròn là một conic C mà ta khớp được từ các điểm biên; từ C và K đã biết, có thể giải ra mặt phẳng chứa hình tròn và vị trí tâm thật của nó bằng cách phân tích trị riêng của $K^T C K$. Chi tiết nằm ngoài phạm vi chương này — B/08 bàn trong bối cảnh tinh chỉnh — nhưng điều cần nhớ ở đây là: *tồn tại cách chữa đúng, và nó bắt buộc nếu dùng marker tròn cho phép đo chính xác*.

CẦN NHỚ

Bias tâm ellipse là một *thiên lệch có hệ thống phụ thuộc tư thế*, không phải nhiễu. Ba hệ quả. Một, nó không giảm khi lấy trung bình nhiều khung. Nó thuộc loại nguồn “hệ thống” ở A/05 mục 8.

Hai, nó thay đổi khi robot di chuyển, vì góc nghiêng và vị trí trên ảnh đều đổi. Nghĩa là nó *không* triệt tiêu hoàn toàn trong sơ đồ teach-by-showing của B/13 — nó chỉ triệt tiêu ở đúng tư thế đã ghi, và sai lệch tăng dần khi robot lệch khỏi tư thế ấy.

Ba, và đây là điều đáng nói nhất: nó là một lý do thực chất để *ưu tiên góc chessboard* hơn *marker tròn* cho khâu đo tinh, dù marker tròn có độ chính xác định vị tâm rất cao. Một phép đo chính xác quanh một giá trị thiên lệch thì không bằng một phép đo kém chính xác hơn mà không thiên lệch — nhất là khi ngân sách của bạn là 5 mm và các nguồn hệ thống là những nguồn chiếm phần lớn ngân sách ấy.

2.7 Giới hạn và chế độ hỏng

Giả thiết ảnh đã khử méo. Toàn bộ chương giả định phép chiếu là pinhole thuần. Nếu còn méo, không tồn tại homography nào khớp được các điểm, và DLT sẽ trả về một ma trận “tốt nhất” trong nghĩa bình phương tối thiểu — tức một ma trận sai với residual không giải thích được. Dấu hiệu: residual của H có cấu trúc không gian (A/01 mục 6), và phép kiểm trực chuẩn ở mục 2 thất bại.

Giả thiết các điểm thực sự đồng phẳng. Nếu bốn góc tag không đồng phẳng do tấm marker cong, thì cũng không có homography nào khớp. Điều này quan trọng hơn vẻ ngoài: một tờ giấy in tag dán lên bề mặt cong lệch vài milimét khỏi mặt phẳng, và với tag 100 mm thì vài milimét là một sai lệch tương đối đáng kể. Đây là lý do 00-map nhấn mạnh nền cứng phẳng.

Giả thiết tương ứng điểm đúng. DLT giả định bạn biết điểm nào tương ứng điểm nào. Với tag có mã thì đúng — decode cho biết hướng. Nhưng nếu decode sai hướng (xoay 90 độ) thì bạn có bốn tương ứng sai và H sẽ là một phép quay 90 độ so với đúng, mà residual thì vẫn nhỏ vì hình vuông đối xứng. Đây là một chế độ hỏng thầm lặng và nó thuộc về B/07.

Homography chỉ áp dụng cho một mặt phẳng. Nếu constellation của bạn không đồng phẳng — và A/04 vừa khuyên mạnh rằng nó *nên* không đồng phẳng — thì không có một homography chung cho cả tập. Bạn có một homography cho mỗi mặt phẳng, và việc gộp chúng lại phải làm ở tầng PnP chứ không ở tầng homography. Đây là một giới hạn có cấu trúc, không phải một bất tiện: nó nói rằng homography là công cụ cho *từng tag*, còn công cụ cho *constellation* là PnP hợp nhất (B/09).

2.8 Thực hành

Ba việc.

Một, thêm phép kiểm trực chuẩn vào pipeline. Sau khi có H , tính $M = K^{-1}H$ và kiểm hai đại lượng: tỉ số $\|\mathbf{m}_1\|/\|\mathbf{m}_2\|$ (nên gần 1) và $|\mathbf{m}_1^T \mathbf{m}_2|/(\|\mathbf{m}_1\|\|\mathbf{m}_2\|)$ (nên gần 0). Ghi cả hai vào log. Chúng bắt được méo chưa khử, K sai, và marker không phẳng — ba lỗi mà không có chỉ số nào khác trong pipeline phát hiện được.

Hai, đặt ngưỡng kích thước tag tối thiểu và ghi nhận khi bị từ chối, thay vì âm thầm trả về một pose kém.

Ba, nếu dùng marker tròn, hiệu chỉnh conic hoặc chấp nhận thiên lệch một cách có ý thức. “Có ý thức” nghĩa là: ước lượng biên độ thiên lệch cho dải tư thế vận hành của bạn, đưa nó vào bảng ngân sách ở C/15 như một dòng hệ thống, và kiểm rằng tổng vẫn trong ngưỡng. Cách tệ nhất là không biết nó tồn tại.

Active recall (che cột phải, tự trả lời, rồi mở ra đối chiếu)

Thuật ngữ / câu hỏi	Trả lời
••• Homography của mặt phẳng	$H = K[\mathbf{r}_1 \ \mathbf{r}_2 \ \mathbf{t}]$; có được vì đặt $Z = 0$ làm cột $\mathbf{r}_3$ biến mất, biến phép chiếu 3D-2D thành biến đổi tuyến tính 2D-2D.
••• Bậc tự do của H	Tám (9 phần tử trừ 1 hệ số tỉ lệ). Pose có sáu. Hai bậc dư chính là hai ràng buộc trực chuẩn $\ \mathbf{r}_1\ = \ \mathbf{r}_2\ $ và $\mathbf{r}_1^\top \mathbf{r}_2 = 0$.
••• Dùng hai bậc dư làm gì	Phép kiểm miễn phí: tính $M = K^{-1}H$ và kiểm hai cột đầu có trực chuẩn không. Thất bại $\Rightarrow$ méo chưa khử, hoặc K sai, hoặc điểm không đồng phẳng.
•• Phân rã H thành pose	$\lambda = \frac{1}{2}(\ \mathbf{m}_1\ + \ \mathbf{m}_2\)$; $\mathbf{r}_i = \mathbf{m}_i/\lambda$; $\mathbf{r}_3 = \mathbf{r}_1 \times \mathbf{r}_2$; $\mathbf{t} = \mathbf{m}_3/\lambda$; rồi chiếu về $SO(3)$ bằng SVD, nhớ kiểm $\det = +1$ [6].
••• Phân rã H là khởi tạo, không phải kết quả	DLT cực tiểu sai số <i>đại số</i> , không phải sai số tái chiếu; các phép sửa λ và chiếu $SO(3)$ đảm bảo hợp lệ chứ không tối ưu; và nó không xử lý bài toán hai nghiệm. Luôn tinh chỉnh sau đó.
••• Chuẩn hoá Hartley	Đưa điểm về trung bình không, khoảng cách trung bình $\sqrt{2}$ trước DLT [13, §4.4]. Cải thiện số điều kiện vài bậc. Bỏ nó là lỗi cài đặt kinh điển, và triệu chứng là kém chính xác thất thường chứ không sai hẳn.
••• Vì sao tag nhỏ tệ hơn $1/w$	Hai hiệu ứng chồng nhau: σ chiếm tỉ lệ lớn hơn trong w (tuyến tính, đã có ở A/05), và bốn góc gần nhau làm DLT kém điều kiện nên khuếch đại thêm. Tích của hai hiệu ứng.
•• Ngưỡng kích thước tag	Thường 40–60 px cho cạnh, nhưng phải đo cho hệ mình: chạy Monte Carlo với w giảm dần, tìm chỗ sai số chệch khỏi đường $1/w$. Đây là lý do của chiến lược nested multi-scale ở B/09.
••• Bất biến duy nhất của phép chiếu	Tỉ số kép của bốn điểm thẳng hàng. <i>Không</i> bảo toàn: khoảng cách, tỉ số hai đoạn, trung điểm, song song, góc, diện tích. Có bảo toàn: thẳng hàng, đồng quy, bậc của conic.
••• Tâm chiếu $\neq$ chiếu của tâm	Hệ quả trực tiếp của việc trung điểm không bảo toàn. Với hình tròn, tâm của ellipse ảnh không phải ảnh của tâm hình tròn [15], [16].
••• Bias tâm ellipse phụ thuộc gì	Tăng theo kích thước hình tròn so với khoảng cách; tăng theo góc nghiêng (bằng 0 ở fronto-parallel); tăng theo độ lệch khỏi tâm ảnh.
••• Vì sao bias ellipse nguy hiểm	Nó là <i>thiên lệch hệ thống phụ thuộc tư thế</i> : không giảm theo $\sqrt{N}$, và không triệt tiêu hoàn toàn trong teach-by-showing vì nó đổi khi tư thế đổi. Đây là lý do thực chất để ưu tiên góc chessboard cho khâu đo tính.
•• Hệ quả thiết kế của tỉ số kép	Marker mã hoá bằng khoảng cách hoặc tỉ lệ hai đoạn không đọc được dưới góc nghiêng. Marker mã hoá bằng lưới ô rời rạc thì đọc được sau khi rectify — đó là lý do ArUco và AprilTag đều dùng lưới.
• Homography không dùng cho constellation không đồng phẳng	Mỗi mặt phẳng một H riêng; gộp phải làm ở tầng PnP. Giới hạn có cấu trúc: H là công cụ cho từng tag, PnP hợp nhất là công cụ cho constellation.

••• phải thuộc •• nên nắm • biết là có

Kiến thức này đang sống ở đâu (công trình đang dùng chính ý tưởng của chương; chốt 9/2026)

Công trình / hệ thống	Dùng ý tưởng nào của chương	Ghi chú
OpenCV findHomography	DLT có chuẩn hoá Hartley sẵn, kèm RANSAC tùy chọn	Đã làm đúng phần chuẩn hoá — đừng tự cài lại trừ khi có lý do
ArUco [14]	Rectify vùng tag về hình vuông chuẩn bằng H rồi lấy mẫu ô bit	Ứng dụng trực tiếp nhất của homography trong pipeline marker

Công trình / hệ thống	Dùng ý tưởng nào của chương	Ghi chú
Hiệu chuẩn Zhang [6]	Lấy H từ mỗi ảnh target phẳng, rồi từ nhiều H giải ra K qua ràng buộc trục chuẩn	Dùng đúng hai bậc tự do dư ở mục 2 làm phương trình cho K
Heikkilä [15]	Hiệu chỉnh bias tâm bằng đại số conic cho target điểm tròn	Nguồn chuẩn cho mục 6; đọc nếu dùng marker tròn
CCTag [18], WhyCon [19]	Marker tròn đồng tâm; độ chính xác định vị tâm rất cao, nhưng phải xử lý bias	Minh hoạ đánh đổi ở mục 6: precision cao, bias khác không

MINI PROJECT · Đo bias tâm ellipse bằng mô phỏng, và so với góc chessboard · 1 ngày

Mục tiêu	biến “tâm chiếu khác chiều của tâm” từ một mệnh đề thành một con số milimét cho dải tư thế của bạn, để quyết định có dùng được marker tròn hay không.
Dữ liệu	mô phỏng thuần. Sinh một hình tròn bán kính R trên mặt phẳng marker bằng cách lấy mẫu dày các điểm trên biên; chiếu chúng bằng K và một pose đã biết.
Các bước	với mỗi tư thế, chiếu tập điểm biên ra ảnh; khớp một ellipse lên tập điểm chiếu; lấy tâm ellipse; so sánh với hình chiếu của tâm hình tròn thật (tính trực tiếp); ghi lại độ lệch tính bằng pixel; quét góc nghiêng từ 0° tới 60° , quét tỉ số R/Z , quét vị trí trên ảnh từ tâm ra rìa; cuối cùng quy độ lệch pixel thành milimét bằng hệ số Z/f của A/05.
Xong khi	bạn có một bảng ba chiều và phát biểu được một câu kiểu “với $R/Z < 0,05$ và góc nghiêng dưới 20° , bias tâm dưới X px tức Y mm, chấp nhận được”. Kiểm tra chéo bất buộc: ở 0° nghiêng, bias phải bằng không tới sai số số học — nếu không thì code khớp ellipse của bạn có lỗi, không phải bias có thật.
Nối với	B/08 cho cách hiệu chỉnh bằng conic; C/15 nhận con số milimét này như một dòng hệ thống trong ngân sách.

TRẢ LỜI CÂU HỎI MỞ ĐẦU

- A1. Hai bậc dư chứa hai ràng buộc trục chuẩn: một ma trận 3×3 bất kỳ không nhất thiết có dạng $K[\mathbf{r}_1 \ \mathbf{r}_2 \ \mathbf{t}]$ với $\mathbf{r}_1, \mathbf{r}_2$ là hai cột của một ma trận quay, và hai điều kiện $\|\mathbf{r}_1\| = \|\mathbf{r}_2\|$ cùng $\mathbf{r}_1^\top \mathbf{r}_2 = 0$ chính là hai bậc bị loại khi đi từ H tổng quát về pose. Sự dư thừa ấy thành công cụ chẩn đoán vì nó cho ta một phép kiểm không tốn thêm dữ liệu: tính $M = K^{-1}H$ và xem hai cột đầu có thật sự trục chuẩn không. Nếu không, thì hoặc ảnh chưa khử méo đúng, hoặc K sai, hoặc các điểm không thực sự đồng phẳng — ba lỗi phổ biến nhất, và không có chỉ số nào khác trong pipeline bắt được cả ba.
- A2. Vì tâm hình tròn được định nghĩa qua trung điểm của các dây cung, mà phép chiếu phối cảnh không bảo toàn trung điểm: nửa đường kính gần camera được phóng to nhiều hơn nửa xa, nên ảnh của tâm bị đẩy lệch so với tâm hình học của ellipse ảnh [15]. Sai lệch phụ thuộc ba thứ: kích thước hình tròn so với khoảng cách (tỉ số R/Z càng lớn càng lệch), góc nghiêng (bằng không ở fronto-parallel, tăng theo góc), và độ lệch khỏi tâm ảnh. Điểm quan trọng: đây là *thiên lệch có hệ thống phụ thuộc tư thế*, nên nó không giảm khi lấy trung bình nhiều khung và nó không triệt tiêu hoàn toàn trong teach-by-showing.
- A3. Số điều kiện của bài toán DLT. Sai số của A/05 giả định bài toán được giải tốt và chỉ tính việc σ chiếm tỉ lệ ngày càng lớn trong w — một hiệu ứng tuyến tính. Nhưng khi bốn góc tag gần nhau trên ảnh, ma trận A trong $A\mathbf{h} = 0$ trở nên kém điều kiện, nên nó *khuếch đại* phần sai số ấy thêm một hệ số nữa. Hai hiệu ứng nhân nhau chứ không cộng, nên tổng thể tệ hơn $1/w$. Cách chữa không phải thuật toán tốt hơn mà là đảm bảo luôn có tag đủ lớn trên ảnh — tức chiến lược nested multi-scale.
- A4. Vì dữ liệu thử thường có thang đo hiện: toạ độ nhỏ, tag ở giữa ảnh, tư thế đơn giản. Trên dữ liệu thật, các phần tử của ma trận A trải qua nhiều bậc độ lớn (tích uX với $u \sim 10^3$ và $X \sim 10^{-1}$), nên số điều kiện tệ ngay từ cách dựng bài toán. Kiểu hỏng đặc biệt khó chịu: nó *không* sai hẳn — kết quả vẫn gần đúng — mà chỉ kém chính xác một cách thất thường, tốt ở tư thế này và tệ ở tư thế khác, không lặp lại được. Người ta sẽ đi tìm nguyên nhân ở detector, ở ánh sáng, ở camera, và không tìm ra, vì nguyên nhân nằm ở một dòng code tiền xử lý bị thiếu.

A5. Chỉ tỉ số kép. Ba cái kia đều không bảo toàn, và trung điểm không bảo toàn là hệ quả của việc tỉ số hai đoạn không bảo toàn. Câu trả lời quyết định thiết kế marker theo một cách rất cụ thể: bất kỳ sơ đồ mã hoá nào dựa trên khoảng cách hoặc trên tỉ lệ hai đoạn đều không đọc được tin cậy dưới góc nghiêng, còn sơ đồ dựa trên *cấu trúc rời rạc* — các ô sáng tối trong một lưới — thì đọc được, vì homography đưa lưới méo trở lại lưới đều và việc còn lại chỉ là hỏi mỗi ô sáng hay tối. Đó chính là lý do cả ArUco lẫn AprilTag đều chọn lưới ô, và đó cũng là lý do bước rectify bằng homography xuất hiện trong mọi pipeline marker.

NÓI LẠI THẬT ĐƠN GIẢN

Chiếu bóng một tấm bia lên tường bằng đèn pin. Nếu tấm bia song song với tường thì bóng là bản phóng to trung thực — hình vuông vẫn vuông, điểm giữa vẫn ở giữa. Nghiêng tấm bia đi thì bóng thành hình thang, và ở đây bắt đầu những chuyện bất ngờ.

Bất ngờ thứ nhất, và nó là tin tốt: dù nghiêng thế nào, có một công thức đơn giản biến toạ độ trên tấm bia thành toạ độ trên tường. Chỉ cần biết bốn điểm đi đâu là suy ra được mọi điểm khác. Đây là lý do máy tính đọc được cái tag dù nó nghiêng: nó “nắn” cái hình thang trở lại thành hình vuông rồi mới đọc các ô đen trắng.

Bất ngờ thứ hai là tin xấu, và ít người để ý. Điểm giữa của một cạnh trên tấm bia *không* chiếu thành điểm giữa của cạnh bóng. Nghe thì nhỏ nhặt, nhưng nó phá hỏng một thứ ta hay tin tưởng: cái tâm. Nếu bạn vẽ một vòng tròn lên tấm bia, bóng của nó là một hình bầu dục — và tâm của hình bầu dục ấy không phải bóng của tâm vòng tròn. Chúng lệch nhau. Càng nghiêng thì càng lệch.

Điều này quan trọng vì có những loại marker dùng vòng tròn thay vì hình vuông, và cách đo của chúng là tìm tâm cái hình bầu dục. Cách ấy đo được rất chính xác — tâm một hình bầu dục thì xác định sắc nét hơn nhiều so với góc một hình vuông. Nhưng nó chính xác quanh một chỗ hơi lệch, và chỗ lệch ấy thay đổi khi robot di chuyển. Đo chính xác quanh một chỗ sai thì không bằng đo hơi kém quanh chỗ đúng — nhất là khi cái sai ấy không lộ ra ở bất cứ con số nào.

Còn một chuyện nữa về kích thước. Khi cái tag còn to trên ảnh, việc tính toán rất ổn định. Khi nó co nhỏ lại, bốn góc xúm gần nhau, và việc suy ra hướng nghiêng từ bốn điểm sát nhau giống như đoán hướng của một cây kim từ việc nhìn hai đầu của nó — hai đầu càng gần thì đoán càng tệ, và tệ nhanh hơn bạn nghĩ. Đó là lý do người ta hay dán một cái tag to và một cái tag nhỏ cạnh nhau: xa thì đọc cái to, gần thì đọc cái nhỏ, và lúc nào cũng có một cái ở kích thước vừa phải.

Chỗ không rút gọn được: vì sao đúng *tỉ số kép* chứ không phải một đại lượng nào khác là bất biến duy nhất — chuyện này có chứng minh, nhưng nó thuộc hình học xạ ảnh và không rút gọn thành một câu trực quan được. Điều dùng được thì đơn giản: đừng tin bất cứ phép đo nào dựa trên khoảng cách hay điểm giữa, trừ khi đã nắn ảnh về đúng dạng trước.

Một câu: *Nắn được hình thang về hình vuông thì đọc được mọi thứ, nhưng cái tâm thì đừng tin — nó không nằm ở chỗ bạn tưởng.*

2.9 Điều gì chứng minh cách hiểu này sai

Mệnh đề chính — *tâm của ellipse ảnh không phải ảnh của tâm hình tròn, và sai lệch tăng theo góc nghiêng* — là mệnh đề tôi tin ở mức cao vì nó qua cửa (c) với hai nguồn độc lập [15], [16] và qua cửa (a) bằng lập luận về trung điểm. Nó bị bác bỏ nếu mô phỏng của mini project cho ra bias bằng không ở mọi góc nghiêng — và nếu bạn thấy thế thì gần như chắc chắn code khớp ellipse của bạn đang có lỗi chứ không phải mệnh đề sai.

Mệnh đề thứ hai — *tag nhỏ làm pose xấu nhanh hơn $1/w$ do số điều kiện* — là mệnh đề yếu nhất về kiểm chứng trong chương, như đã ghi ở mục 4. Nó bị bác bỏ nếu Monte Carlo cho thấy sai số pose bám sát đường $1/w$ xuống tới kích thước tag rất nhỏ, chỉ xấu đi khi decode bắt đầu thất bại. Nếu vậy thì ngưỡng kích thước tối thiểu nên được đặt theo tiêu chí decode chứ không theo tiêu chí điều kiện số, và lời khuyên ở mục 4 cần sửa.

Mệnh đề thứ ba — *phép kiểm trực chuẩn bất được méo chưa khử, K sai, và điểm không đồng phẳng* — bị bác bỏ nếu có một trường hợp trong ba trường hợp ấy mà hai chỉ số trực chuẩn vẫn nằm trong ngưỡng bình thường. Tôi dẫn nó từ cấu trúc của công thức (của a) và chưa kiểm bằng mô phỏng cả ba trường hợp; đây là một phép kiểm rẻ và nên làm.

2.10 Kết nối

Chương này là chương công cụ, và nó cấp ba thứ cho phần còn lại của cây.

Nối lên trên. [A/01-mo-hinh-camera-va-distortion](#) là điều kiện tiên quyết theo nghĩa mạnh: nếu ảnh còn méo thì không tồn tại homography nào khớp, và mọi kết quả của chương này vô nghĩa. Phép kiểm trực chuẩn ở mục 2 chính là cách phát hiện tình trạng ấy, nên hai chương gắn với nhau theo cả hai chiều.

Nối ngang trong Phần A. [A/03-pnp-va-uoc-luong-tu-the](#) nhận phép phân rã H làm một trong các cách khởi tạo, nhưng nó cũng giải thích vì sao IPPE là lựa chọn tốt hơn cho target phẳng. [A/04-pose-ambiguity-target-phang](#) có gốc rễ ở đây: phép phân rã H vốn đã cho nhiều nghiệm, và chương 4 giải thích ý nghĩa hình học của chúng. [A/05-lan-truyen-sai-so-pixel-sang-pose](#) nhận từ mục 4 phần bổ sung cho công thức $1/w$ của nó — chương 5 cho quan hệ tiệm cận, chương này cho biết quan hệ ấy hỏng ở đâu.

Nối xuống Phần B. [B/07-detection-pipeline](#) dùng homography ở khâu rectify, và mục 5 ở đây giải thích vì sao khâu ấy cần thiết: không rectify thì lưới ô không đều và việc đọc bit không tin cậy. [B/08-corner-refinement](#) nhận mục 6 làm cơ sở cho phần hiệu chỉnh conic của marker tròn, và nó cũng nhận luận điểm rằng góc chessboard không chịu thiên lệch này — một lý do thực chất cho lựa chọn target lai ở [00-map](#). [B/09-multi-marker-constellation](#) nhận từ mục 4 lý do định lượng cho chiến lược nested multi-scale, và từ mục 7 lý do vì sao constellation không đồng phẳng phải xử lý ở tầng PnP chứ không ở tầng homography.

Cuối cùng, nối sang [C/15-ngan-sach-sai-so](#): bias tâm ellipse ở mục 6 là một dòng *hệ thống* trong bảng ngân sách, và mini project của chương này là cách định giá dòng ấy bằng số.

Câu hỏi tự kiểm (trả lời không nhìn bài)

- Homography có tám bậc tự do, pose có sáu. Chỉ ra chính xác hai ràng buộc chiếm phần dư, và mô tả một phép kiểm dùng chúng bất được ba lỗi khác nhau.
- Vì sao đặt $Z = 0$ lại biến một phép chiếu 3D–2D thành một biến đổi 2D–2D, và điều đó ngừng đúng khi nào?
- Một đồng nghiệp bỏ bước chuẩn hoá Hartley. Mô tả kiểu hỏng — cụ thể là vì sao nó khó truy hơn một lỗi làm kết quả sai hẳn.
- Vì sao sai số pose xấu đi nhanh hơn $1/w$ khi tag co nhỏ, và hai hiệu ứng nào đang nhân với nhau?
- Phép chiếu phối cảnh bảo toàn đúng một bất biến trên đường thẳng. Nêu nó, và giải thích vì sao điều đó buộc mọi họ tag hiện đại phải mã hoá bằng lưới ô rời rạc.
- Bạn dùng marker tròn vì nó cho σ nhỏ nhất trong mọi loại target. Nêu lý do có thể làm lựa chọn ấy vẫn sai cho bài toán 5 mm, và nêu điều kiện mà dưới đó nó lại đúng.
- Constellation của bạn gồm ba tag trên ba mặt phẳng khác nhau. Bạn tính được mấy homography, và vì sao không gộp chúng lại ở tầng homography được?

PnP và ước lượng tư thế (Perspective-n-Point)

PHỤ THUỘC A/01-mô-hình-camera-va-distortion (mô hình chiếu và ma trận K — PnP giả định cả hai đã biết chính xác);
A/02-homography-va-hình-học-phang (phân rã homography, một trong các cách khởi tạo).

TÓM TẮT

Chương này dựng cỗ máy trung tâm của cả cây: cho một tập điểm 3D đã biết toạ độ trong hệ vật và hình chiếu của chúng trên ảnh, tìm pose của camera. Bức tranh gồm hai tầng và tách bạch chúng là điều quan trọng nhất cần mang ra khỏi chương. Tầng thứ nhất là các lời giải *dạng đóng* — P3P, EPnP, DLT, IPPE — chúng nhanh, không cần khởi tạo, và *không* tối ưu theo tiêu chí có ý nghĩa. Tầng thứ hai là *tinh chỉnh phi tuyến* cực tiểu hoá sai số tái chiếu, và đây mới là nghiệm cuối cùng. Ba điều thực hành: với target phẳng hãy dùng IPPE vì nó cho cả hai nghiệm; với nhiều tag hãy chạy *một* PnP trên toàn bộ tập góc chứ đừng trung bình pose; và hãy trọng số hoá theo chất lượng từng góc thay vì coi mọi góc như nhau. Nếu chỉ nhớ một điều: *lời giải dạng đóng là để khởi tạo, tinh chỉnh mới là kết quả* — và Jacobian của bước tinh chỉnh chính là ma trận mà A/05 đảo để ra mọi công thức sai số của cây.

KIẾN THỨC CẦN CÓ TRƯỚC

- **PnP — Perspective-n-Point**: bài toán tìm pose camera từ n điểm 3D đã biết và hình chiếu 2D của chúng, với K đã biết. Ba điểm là số tối thiểu cho một tập nghiệm hữu hạn; bốn điểm thường đủ để chọn ra một nghiệm.
- **sai số tái chiếu (reprojection error)** — với một pose giả định ξ , chiếu điểm 3D ra ảnh được $\hat{u}_i(\xi)$, rồi lấy hiệu với góc đo được u_i . Đây là hàm mục tiêu có ý nghĩa vật lý vì nó đo đúng đại lượng mà nhiễu tác động lên — toạ độ pixel.
- **sai số đại số (algebraic error)** — đại lượng mà các lời giải tuyến tính như DLT cực tiểu hoá, thường có dạng $\|Ah\|$. Nó dễ giải vì tuyến tính, và nó *không* tỉ lệ với sai số tái chiếu — đó là toàn bộ lý do cần bước tinh chỉnh.
- **Gauss–Newton** — phương pháp lặp cho bình phương tối thiểu phi tuyến: tuyến tính hoá phần dư quanh ước lượng hiện tại, giải một hệ tuyến tính cho bước cập nhật, lặp lại. Nó xấp xỉ Hessian bằng $J^T J$, bỏ số hạng chứa đạo hàm bậc hai của mô hình.
- **Levenberg–Marquardt (LM)** — Gauss–Newton có thêm số hạng chính quy hoá λI trên đường chéo, nội suy giữa Gauss–Newton (khi λ nhỏ) và bước giảm gradient (khi λ lớn). Nó bền hơn khi khởi tạo xa hoặc khi $J^T J$ kém điều kiện.
- **tham số hoá tối thiểu của xoay** — pose có sáu bậc tự do nhưng R có chín phần tử với sáu ràng buộc. Để tối ưu trên $SO(3)$ cần một tham số hoá cục bộ sáu tham số — thường là vectơ xoay qua ánh xạ mũ, tức $\mathfrak{se}(3)$. Dùng góc Euler thay vào là nguồn của kỳ dị gimbal lock.
- **RANSAC — Random Sample Consensus**: chọn ngẫu nhiên con tối thiểu, giải, đếm số điểm phù hợp, lặp và giữ tập con tốt nhất [20]. Nó loại điểm ngoại lai chứ không cải thiện độ chính xác của điểm nội lai.

CÂU HỎI MỞ ĐẦU

- Q1.** P3P với ba điểm cho tối đa bốn nghiệm hình học hợp lệ. Vì sao bốn mà không phải hai hay tám, và điều đó có ý nghĩa gì với việc bạn phải làm ở tầng code?

- Q2. EPnP chạy nhanh, không cần khởi tạo, xử lý được hàng trăm điểm. Vì sao nó vẫn không đủ, và cụ thể đại lượng nào nó cực tiểu hoá mà không phải đại lượng bạn quan tâm?
- Q3. Bạn có bốn tag, mỗi tag bốn góc. Có hai cách: bốn lần PnP rồi gộp, hoặc một lần PnP trên mười sáu góc. Chúng cho kết quả khác nhau đáng kể — vì sao, và cách nào đúng?
- Q4. Bộ tối ưu của bạn dùng góc Euler để tham số hoá xoay. Nó chạy tốt trong thử nghiệm rồi thỉnh thoảng phân kỳ hoàn toàn ở một số tư thế. Cơ chế gì, và cách sửa đúng là gì?
- Q5. Một tag ở xa và nghiêng mạnh, một tag ở gần và nhìn thẳng, cả hai đều nhìn thấy. Nếu bạn cho chúng trọng số như nhau trong PnP hợp nhất, bạn đang giả định gì — và giả định ấy sai theo chiều nào?

3.1 Đặt vấn đề

Toàn bộ cây này quy về một phép biến đổi: từ vài chục tọa độ pixel sang sáu con số pose. Chương này là chương mô tả phép biến đổi ấy.

Bài toán có một lịch sử dài hơn thị giác máy tính. Trường hợp ba điểm được Grunert giải năm 1841 trong bối cảnh trắc địa — xác định vị trí một trạm quan trắc từ ba mốc đã biết — và phân tích đầy đủ về số nghiệm của nó được Haralick và cộng sự tổng hợp lại một thế kỷ rưỡi sau [21]. Trường hợp n điểm tổng quát thì có nhiều lời giải hiện đại, mỗi lời giải đánh đổi khác nhau giữa tốc độ, độ bền và độ chính xác.

Điều đáng nói với người làm marker — và lý do chương này không chỉ là một danh mục thuật toán — là một sự nhầm lẫn rất phổ biến về *vai trò* của các thuật toán ấy. Người ta hay hỏi “dùng PnP nào tốt nhất”, như thể đó là một lựa chọn duy nhất. Câu hỏi ấy đặt sai, vì bức tranh đúng có hai tầng và chúng phục vụ hai mục đích khác nhau.

Tầng thứ nhất gồm các lời giải *dạng đóng*: P3P, EPnP, DLT, IPPE, SQPnP. Chúng giải một hệ đại số và cho ra nghiệm trong một số hữu hạn phép tính, không cần điểm khởi đầu. Ưu điểm rõ ràng; nhược điểm thì kín đáo hơn: chúng cực tiểu hoá một đại lượng đại số nào đó *không phải* sai số tái chiếu. Hệ quả là nghiệm chúng cho ra nói chung không phải nghiệm hợp lý cực đại, và độ lệch ấy đáng kể khi nhiễu không nhỏ.

Tầng thứ hai là *tinh chỉnh phi tuyến*: xuất phát từ một pose khởi tạo, lặp Gauss–Newton hoặc LM để cực tiểu hoá sai số tái chiếu. Đây là bước cho ra nghiệm hợp lý cực đại dưới giả thiết nhiễu Gauss, và đây là nghiệm duy nhất mà công thức hiệp phương sai của A/05 áp dụng được. Bỏ bước này là bỏ đi cả độ chính xác lẫn khả năng nói gì đó có cơ sở về bất định.

Cách hiển nhiên mà nhiều cài đặt thực tế làm — gọi một hàm PnP dạng đóng rồi dùng thẳng kết quả — do đó hỏng theo hai cách cùng lúc. Nó mất một phần độ chính xác, và nó mất luôn ma trận thông tin mà bước tinh chỉnh sinh ra như một sản phẩm phụ miễn phí. Cái thứ hai đắt hơn cái thứ nhất, vì không có ma trận thông tin thì không có trọng số để hợp nhất, không có cơ sở để loại ngoại lai theo chi-square, và không có gì để đưa vào ngân sách sai số.

Có một cách hiển nhiên thứ hai cũng hỏng, và nó phổ biến hơn: khi có nhiều tag, chạy PnP cho từng tag rồi gộp các pose lại. A/05 mục 7 đã nêu kết luận; mục 6 của chương này giải thích cơ chế bằng ngôn ngữ của bài toán tối ưu.

TRƯỚC GIÁC

Hình dung bài toán PnP như việc định vị bằng giác kế.

Bạn đứng ở một chỗ chưa biết và nhìn thấy vài cột mốc mà bạn biết chính xác toạ độ. Với mỗi cột mốc, bạn đo được *hướng* tới nó — không phải khoảng cách, chỉ hướng. Câu hỏi: bạn đang ở đâu và đang quay mặt về phía nào?

Mỗi cột mốc cho bạn một tia: “từ chỗ tôi đứng, cột mốc này nằm theo hướng đó”. Với một cột mốc thì vô số vị trí thoả mãn. Với hai thì bớt đi. Với ba thì — và đây là chỗ bất ngờ — vẫn còn tới bốn vị trí có thể, vì bài toán đại số ẩn sau nó là bậc bốn. Phải có cột mốc thứ tư để loại bớt.

Bây giờ thêm nhiều. Bạn đo hướng không chính xác tuyệt đối, nên các tia không giao nhau ở đúng một điểm — chúng tạo thành một chùm tia hơi lệch nhau. Không có vị trí nào thoả mãn *tất cả* các phép đo. Câu hỏi đổi thành: vị trí nào làm tổng bình phương độ lệch nhỏ nhất?

Và đây là chỗ hai tầng xuất hiện. Lời giải dạng đóng giống như việc dùng một công thức sẵn có để ra ngay một đáp án gần đúng — nhanh, và nó tối thiểu hoá một thứ gì đó tiện tính chứ không phải thứ bạn thật sự quan tâm. Tình hình giống như việc đứng ở đáp án ấy rồi dịch chuyển từng chút một theo hướng làm tổng lệch giảm, cho tới khi không giảm được nữa.

Điều đáng nói cuối: khi bạn dừng lại, độ *cong* của mặt tổng-lệch quanh chỗ bạn đứng cho bạn biết bạn chắc chắn tới mức nào. Dốc đứng theo mọi hướng thì vị trí xác định tốt; có một hướng gần phẳng thì theo hướng ấy bạn không biết gì cả. Đó chính là ma trận thông tin của [A/05](#), và nó chỉ tồn tại nếu bạn đã làm bước tinh chỉnh.

3.2 P3P: ba điểm và bốn nghiệm

Trường hợp tối thiểu, và nó đáng hiểu vì nó là hạt nhân của RANSAC.

Cho ba điểm P_1, P_2, P_3 đã biết khoảng cách đôi một d_{12}, d_{13}, d_{23} , và ba hướng tia đơn vị $\mathbf{v}_1, \mathbf{v}_2, \mathbf{v}_3$ từ tâm camera tới chúng (tính được từ toạ độ ảnh và K^{-1}). Ẩn là ba khoảng cách s_1, s_2, s_3 từ tâm camera tới ba điểm.

Định lý cosin trong ba tam giác cho ba phương trình:

$$s_i^2 + s_j^2 - 2s_i s_j \cos \theta_{ij} = d_{ij}^2, \quad \cos \theta_{ij} = \mathbf{v}_i^\top \mathbf{v}_j.$$

Ba phương trình bậc hai, ba ẩn. Khử hai ẩn cho ra một đa thức bậc bốn theo ẩn còn lại, và đó là nguồn gốc của con số bốn trong “tối đa bốn nghiệm” — đây là câu trả lời cho Q1. Không phải hai vì hệ thực sự bậc bốn; không phải tám vì các nghiệm âm (điểm sau camera) bị loại bởi vật lý.

Sau khi có s_i , toạ độ ba điểm trong hệ camera là $s_i \mathbf{v}_i$, và pose thu được bằng cách khớp hai tập ba điểm — một bài toán Procrustes giải bằng SVD.

Ý nghĩa thực hành của “bốn nghiệm” là rất cụ thể: *bạn phải có điểm thứ tư để chọn*. Quy trình chuẩn là dùng ba điểm giải P3P, được tới bốn ứng viên, rồi chiếu điểm thứ tư bằng từng ứng viên và giữ ứng viên cho sai số tái chiếu nhỏ nhất. Đây là lý do mọi cài đặt RANSAC cho PnP dùng tập con bốn điểm chứ không phải ba.

Có nhiều công thức hoá khác nhau cho P3P, khác nhau ở độ ổn định số học và tốc độ; công thức của Kneip và cộng sự tránh được bước khớp Procrustes bằng cách xây trực tiếp một hệ toạ độ trung gian [22]. Với mục đích của cây thì chi tiết ấy không quan trọng — điều quan trọng là biết P3P tồn tại, nó cho nhiều nghiệm, và vai trò của nó là *hạt nhân tối thiểu cho RANSAC*, không phải bộ giải chính.

CẠM BÃY

Đừng nhầm bốn nghiệm của P3P với hai nghiệm của target phẳng ở A/04. Chúng là hai hiện tượng khác nhau và cách xử lý khác nhau.

Bốn nghiệm của P3P là hệ quả của việc *thiếu dữ liệu*: ba điểm không đủ để xác định sáu bậc tự do một cách duy nhất. Thêm một điểm thứ tư (không suy biến) là loại được ba trong bốn, và vấn đề biến mất hoàn toàn.

Hai nghiệm của target phẳng là hệ quả của một *đối xứng hình học* và nó *không* biến mất khi thêm điểm — miễn là các điểm thêm vào vẫn đồng phẳng. Đó là một loại bệnh hoàn toàn khác, chữa bằng cách phá tính đồng phẳng chứ không bằng cách thêm dữ liệu.

Hệ quả cài đặt: một tag vuông đơn lẻ có bốn góc, đủ để loại bớt nghiệm P3P, nhưng *vẫn* có hai nghiệm phẳng. Nếu bạn thấy hai nghiệm sau khi đã dùng cả bốn góc thì đó không phải bug.

3.3 Các lời giải dạng đóng cho n điểm

Bốn họ, mỗi họ một ý tưởng.

DLT. Viết phương trình chiếu thành hệ tuyến tính theo mười hai phần tử của $[R|t]$, bỏ qua ràng buộc trực giao, giải bằng SVD, rồi chiếu về $SO(3)$ sau. Đơn giản nhất, và cũng kém chính xác nhất, vì nó “quên” sáu ràng buộc rồi sửa sau. Nó cũng không dùng được K đã biết một cách hiệu quả. Vai trò thực tế: giá trị giáo dục, và khởi tạo khi không có gì tốt hơn.

EPnP. Ý tưởng đẹp: biểu diễn n điểm 3D bằng tổ hợp affine của bốn “điểm điều khiển ảo”, rồi giải cho toạ độ camera của bốn điểm ảo ấy thay vì cho n điểm. Bài toán rút gọn về kích thước cố định bất kể n , nên độ phức tạp là $O(n)$ [23]. Đây là lựa chọn tốt cho bài toán có nhiều điểm.

IPPE. Chuyên cho target phẳng. Nó khai thác cấu trúc phẳng bằng cách phân tích phép biến đổi *vi phân* quanh mỗi điểm, và điều quan trọng nhất với ta: nó trả về *cả hai* nghiệm cùng sai số tái chiếu của từng nghiệm [17]. Với marker vuông — tức gần như toàn bộ bài toán của cây — đây là lựa chọn đúng, và biến thể cho tag vuông có trong OpenCV dưới tên SOLVEPNP_IPPE_SQUARE.

SQPnP và các phương pháp tối ưu toàn cục. Giải bài toán bình phương tối thiểu trên $SO(3)$ bằng lập trình bậc hai tuần tự, đảm bảo tìm được cực tiểu toàn cục của một hàm mục tiêu đại số [24]. Chúng khắc phục nhược điểm “không tối ưu” của các phương pháp trước ở mức đại số, nhưng vẫn không phải nghiệm tối ưu theo sai số tái chiếu.

Bảng 3.1. Bốn họ lời giải dạng đóng: ý tưởng, ưu nhược, và vai trò trong pipeline marker.

Phương pháp	Ý tưởng	Điểm yếu	Vai trò cho marker
P3P	Định lý cosin, đa thức bậc bốn	Tối đa 4 nghiệm, cần điểm thứ tư để chọn	Hạt nhân RANSAC; không dùng làm bộ giải chính
DLT	Bỏ ràng buộc trực giao, giải tuyến tính	Kém chính xác nhất; không tận dụng K	Khởi tạo dự phòng; giá trị giáo dục
EPnP [23]	Bốn điểm điều khiển ảo, $O(n)$	Cực tiểu sai số đại số, không phải tái chiếu	Tốt khi nhiều điểm và không đồng phẳng
IPPE [17]	Khai thác cấu trúc phẳng, phân tích vi phân	Chỉ cho target phẳng	Lựa chọn mặc định cho marker vuông — cho cả hai nghiệm

Ghi chú trung thực: tôi không đưa vào chương này các con số so sánh tốc độ hay độ chính xác giữa các thuật toán, dù chúng có sẵn trong các bài báo gốc. Lý do là những con số ấy được đo trên các cấu hình rất khác nhau — số điểm, mức nhiễu, phân bố hình học — và trích chúng ra khỏi ngữ cảnh thì chúng không so sánh được với nhau. Điều tôi khẳng định ở đây chỉ là các *tính chất cấu trúc*: IPPE cho cả hai nghiệm của target phẳng, EPnP là $O(n)$, P3P cho tối đa bốn nghiệm. Ba điều ấy là tính chất toán học chứ không phải kết quả đo.

3.4 Tinh chỉnh: nghiệm thật sự

Đây là bước quyết định, và nó là bước cấp Jacobian cho A/05.

Hàm mục tiêu là sai số tái chiếu có trọng số:

$$C(\xi) = \frac{1}{2} \sum_{i=1}^N \mathbf{r}_i(\xi)^\top \Sigma_i^{-1} \mathbf{r}_i(\xi), \quad \mathbf{r}_i(\xi) = u_i - \hat{u}_i(\xi). \quad (3.1)$$

Đây là hàm hợp lý âm (negative log-likelihood) dưới giả thiết nhiễu Gauss độc lập trên tọa độ pixel, nên nghiệm của nó là ước lượng hợp lý cực đại — đó là lý do nó đáng làm, chứ không phải vì nó “chính xác hơn” một cách mơ hồ.

A. Tham số hoá xoay: vì sao phải dùng $\mathfrak{se}(3)$

Trước khi lấy đạo hàm, phải quyết định biểu diễn ξ thế nào. Đây là chỗ Q4 chỉ tới và nó là một lỗi cài đặt hay gặp.

Vấn đề: R có chín phần tử nhưng chỉ ba bậc tự do. Tối ưu trực tiếp trên chín phần tử với sáu ràng buộc là phiền; tối ưu trên góc Euler thì gặp **khóa gimbal** (*gimbal lock*) — ở một số tư thế, hai trong ba trục trùng nhau và Jacobian mất hạng. Triệu chứng đúng như Q4 mô tả: chạy tốt phần lớn thời gian rồi phân kỳ ở một số tư thế cụ thể, lặp đi lặp lại ở đúng những tư thế ấy.

Cách đúng là dùng một *tham số hoá cục bộ*: giữ ước lượng hiện tại R_k dưới dạng ma trận (hoặc quaternion), và tham số hoá *số gia* bằng một vectơ ba chiều ω qua ánh xạ mũ:

$$R_{k+1} = R_k \exp(\omega^\wedge),$$

với $\cdot^\wedge$ là toán tử đưa vectơ ba chiều thành ma trận phản đối xứng. Tại mỗi bước lặp ta tối ưu trên ω quanh không, nơi tham số hoá không có kỳ dị, rồi cập nhật và đặt lại. Đây là ý tưởng chuẩn của tối ưu trên đa tạp, và một tài liệu tham chiếu gọn cho nó là [25].

B. Bước Gauss–Newton

Tuyến tính hoá $\mathbf{r}_i$ quanh ξ_k :

$$\mathbf{r}_i(\xi_k + \delta) \approx \mathbf{r}_i(\xi_k) - J_i \delta, \quad J_i = \left. \frac{\partial \hat{u}_i}{\partial \xi} \right|_{\xi_k}.$$

Thay vào (3.1), lấy đạo hàm theo δ và cho bằng không:

$$\left(\sum_i J_i^\top \Sigma_i^{-1} J_i \right) \delta = \sum_i J_i^\top \Sigma_i^{-1} \mathbf{r}_i. \quad (3.2)$$

Đây là *phương trình chuẩn*, và ma trận bên trái là chính xác ma trận thông tin $\mathcal{I}$ mà A/05 đảo để ra hiệp phương sai. Điểm đáng nhấn mạnh: $\mathcal{I}$ không phải một thứ phải tính thêm — nó *đã có sẵn* như một sản phẩm phụ của mỗi bước lặp. Bỏ qua nó là vứt đi thông tin miễn phí.

Với sáu ẩn, hệ (3.2) là 6×6 — giải trong micro giây. Đây cũng là lý do PnP tinh chỉnh rẻ đến mức không có lý do gì để bỏ qua: vài chục bước lặp trên một hệ 6×6 là không đáng kể so với chi phí chạy detector.

LM thêm số hạng $\lambda \text{diag}(\mathcal{I})$ vào về trái, điều chỉnh λ theo mức độ thành công của từng bước. Nó bền hơn khi khởi tạo xa hoặc khi $\mathcal{I}$ kém điều kiện — tức đúng các tình huống mà A/02 mục 4 cảnh báo.

C. Jacobian của phép chiếu

Để hoàn chỉnh, đây là mảnh ghép cuối. Với điểm trong hệ camera $\mathbf{X}_c = (X, Y, Z)$, phép chiếu là $\hat{u} = fX/Z + c_x$, $\hat{v} = fY/Z + c_y$. Đạo hàm theo $\mathbf{X}_c$:

$$\frac{\partial \hat{u}}{\partial \mathbf{X}_c} = \frac{f}{Z} \begin{pmatrix} 1 & 0 & -X/Z \\ 0 & 1 & -Y/Z \end{pmatrix}.$$

Nhìn ma trận này một lát thì thấy ngay nguồn gốc của mọi kết luận ở A/05: hệ số chung là f/Z , giải thích vì sao sai số ngang là $Z\sigma/f$; và cột thứ ba — đạo hàm theo chiều sâu — mang thêm hệ số X/Z và Y/Z , tức *tỉ lệ với độ lệch khỏi trục quang*. Một điểm nằm đúng trên trục quang có $X = Y = 0$ nên đạo hàm theo Z bằng không: dịch nó ra xa hay lại gần không đổi hình chiếu chút nào. Đây chính là cơ chế sâu xa vì sao chiều sâu khó đo, và vì sao nó chỉ đo được nhờ các điểm *cách xa trục quang* — tức nhờ kích thước tag.

Phần còn lại của Jacobian là đạo hàm của $\mathbf{X}_c$ theo ξ , và nó có dạng chuẩn $[I \quad -\mathbf{X}_c^\wedge]$ với tham số hoá $\mathfrak{se}(3)$. Ghép lại bằng quy tắc chuỗi.

CẦN NHỚ

Ba lý do bắt buộc phải tinh chỉnh, không phải một.

Một, độ chính xác: lời giải dạng đóng cực tiểu sai số đại số, tinh chỉnh cực tiểu sai số tái chiếu — chỉ cái sau là ước lượng hợp lý cực đại.

Hai, và đây là lý do hay bị quên: tinh chỉnh sinh ra ma trận thông tin $\mathcal{I} = \sum J^\top \Sigma^{-1} J$ miễn phí. Không có nó thì không có hiệp phương sai để hợp nhất, không có cơ sở loại ngoại lai theo chi-square, và không có gì đưa vào ngân sách sai số.

Ba, chi phí gần như bằng không: một hệ 6×6 giải vài chục lần, không đáng kể so với detector.

Kết luận thực hành: solvePnPRefineLM hoặc tương đương phải luôn có trong pipeline, ngay sau lời giải dạng đóng.

3.5 PnP hợp nhất trên nhiều tag

Đây là mục quan trọng nhất của chương với bài toán docking, và nó trả lời Q3.

Tình huống: M tag, mỗi tag bốn góc, tọa độ 3D của tất cả $4M$ góc đã biết *trong cùng một hệ tọa độ dock*. Câu hỏi: tìm pose của camera so với hệ dock.

Cách đúng là chạy (3.1) với $N = 4M$, tức một bài toán tối ưu duy nhất trên toàn bộ tập góc. Không có gì đặc biệt về mặt thuật toán — vẫn là (3.2) với tổng chạy qua nhiều số hạng hơn. Cái đặc biệt nằm ở chỗ *thông tin hình học được sử dụng*.

Để thấy rõ, so sánh hai cách bằng ngôn ngữ của ma trận thông tin.

Cách hợp nhất cộng các ma trận thông tin trong cùng một hệ tọa độ:

$$\mathcal{I}_{\text{hợp nhất}} = \sum_{m=1}^M \sum_{i \in \text{tag } m} J_i^T \Sigma_i^{-1} J_i.$$

Các J_i ở đây được tính với tọa độ 3D thật của từng góc trong hệ dock, nên chúng *biết* rằng tag 1 nằm cách tag 2 một khoảng B .

Cách trung bình pose tính M pose riêng, mỗi pose có hiệp phương sai $\Sigma_m = \mathcal{I}_m^{-1}$ tính trong hệ của riêng tag ấy, rồi gộp. Ngay cả khi gộp đúng cách (trung bình có trọng số theo nghịch đảo hiệp phương sai, trên đa tạp), kết quả vẫn khác, vì phép nghịch đảo và phép tổng không giao hoán:

$$\left(\sum_m \mathcal{I}_m \right)^{-1} \neq \left(\sum_m \mathcal{I}_m^{-1} \right)^{-1} \dots$$

và quan trọng hơn cả bất đẳng thức đại số: khi tính $\mathcal{I}_m$ riêng cho từng tag, ta đã *vứt bỏ* ràng buộc rằng các tag gắn cứng với nhau. Ràng buộc ấy chính là thứ tạo ra cơ chế baseline của A/05 mục 6.

Lưu ý điểm này là trường hợp riêng của một nguyên lý tổng quát trong hiệu chỉnh chùm tia: *cộng thông tin, đừng cộng ước lượng* [2, §2]. Nó xuất hiện ở khắp nơi trong ước lượng trạng thái, và nó luôn nói cùng một điều: gộp ở tầng phần dư thì giữ được cấu trúc, gộp ở tầng kết quả thì mất.

CẠM BẮY

Có một điều kiện tiên quyết cho PnP hợp nhất mà nếu không thoả thì nó *tệ hơn* trung bình pose: *tọa độ 3D của các tag trong hệ dock phải đúng*.

Lý do: PnP hợp nhất tin vào mô hình constellation và ép một pose duy nhất giải thích mọi góc. Nếu mô hình sai — tag 2 thực ra cách tag 1 là 398 mm chứ không phải 400 mm như CAD — thì bộ giải phải thoả hiệp, và nó phân bổ sai lệch ấy vào pose theo một cách không kiểm soát được. Trung bình pose thì “rộng lượng” hơn: mỗi pose riêng vẫn đúng trong hệ của tag nó, và sai lệch constellation chỉ làm các pose tản mát.

Hệ quả: *PnP hợp nhất chỉ tốt hơn nếu bạn đã đo constellation cẩn thận*, và đó là lý do B/12 đặt bước hiệu chuẩn constellation bằng bundle adjustment ở mức bắt buộc chứ không phải tùy chọn.

May mắn là có một chỉ báo: nếu sai số tái chiếu sau khi hợp nhất lớn hơn hẳn so với khi giải từng tag riêng, thì mô hình constellation của bạn sai. Đây là một phép thử miễn phí và nó đáng chạy định kỳ.

3.6 Trọng số: không phải góc nào cũng như nhau

Công thức (3.1) có Σ_i cho từng góc, và phần lớn cài đặt đặt $\Sigma_i = \sigma^2 I$ như nhau cho mọi góc. Mục này trả lời Q5: giả định ấy sai theo chiều nào, và sửa thế nào.

Bốn nguồn làm Σ_i khác nhau giữa các góc.

Khoảng cách và kích thước biểu kiến. Một tag ở xa có kích thước nhỏ trên ảnh; các cạnh của nó ngắn hơn nên đường thẳng khớp lên chúng kém xác định hơn, nên góc kém chính xác hơn. Đây là nguồn lớn nhất trong bài toán docking vì các tag trong constellation có thể ở các khoảng cách rất khác nhau khi robot tới gần.

Góc nghiêng. Một tag nhìn rất chếch có các cạnh bị nén, và số pixel dọc theo cạnh ít đi, nên định vị kém hơn. Thêm nữa, cạnh bị nén có gradient thay đổi theo cách khác, làm phép khớp đường kém ổn định.

Vị trí trên ảnh. Méo còn sót lại lớn hơn ở rìa (A/01), và nó thể hiện như một thành phần thiên lệch — không phải nhiễu, nhưng nếu ta không mô hình hoá được nó thì cách thực dụng là tăng Σ cho các góc ở rìa để giảm ảnh hưởng của chúng.

Điều kiện quang học cục bộ. Một góc nằm trong vùng bị loá, hoặc bị mờ do độ sâu trường ảnh, hoặc có tương phản thấp, thì kém chính xác hơn (A/06, B/14).

Hai cách hiện thực hoá. Cách đơn giản là gán Σ_i theo một quy tắc heuristic — chẳng hạn tỉ lệ nghịch với kích thước tag trên ảnh — và điều đó đã tốt hơn nhiều so với trọng số đều. Cách đúng hơn là để chính detector ước lượng bất định của mỗi góc: bước khớp đường thẳng ở B/08 sinh ra một hiệp phương sai cho mỗi góc như sản phẩm phụ, và nó mang cả thông tin về *hướng* — một góc trên một cạnh dài và sắc nét được xác định tốt theo hướng vuông góc cạnh và kém hơn theo hướng dọc cạnh, tức Σ_i dị hướng.

Khung xử lý dị hướng đúng cách đã được đưa ra trong MLPnP [26], và ý tưởng của nó áp dụng thẳng vào đây: dùng hiệp phương sai thật của từng phép đo thay vì một hằng số. Lợi ích không chỉ là độ chính xác mà còn là *tính trung thực của hiệp phương sai đầu ra* — nếu Σ_i đầu vào đúng thì T^{-1} đầu ra mới là một ước lượng bất định đáng tin, và cả chuỗi hợp nhất ở B/11 phụ thuộc vào điều đó.

3.7 Loại ngoại lai

Bình phương tối thiểu rất nhạy với điểm ngoại lai: một góc sai 50 px đóng góp vào hàm mục tiêu gấp 250^2 lần một góc sai 0,2 px, nên nó kéo nghiệm về phía nó.

Trong bài toán marker, ngoại lai đến từ đâu? Không nhiều như trong bài toán đặc trưng tự nhiên — đó là một lợi thế lớn của marker. Nhưng vẫn có: một tag bị che một phần cho góc sai; một tag bị bắn hoặc mòn cho decode đúng nhưng góc lệch; một tag decode nhầm hướng cho bốn tương ứng xoay 90 độ; và một tag trong constellation bị lắp sai vị trí so với mô hình cho một sai lệch có hệ thống trông như ngoại lai.

Ba lớp công cụ, dùng theo thứ tự.

RANSAC ở mức tag, không ở mức góc. Vì bốn góc của một tag hoặc cùng đúng hoặc cùng sai, việc lấy mẫu ngẫu nhiên ở mức góc là lãng phí. Lấy mẫu ở mức tag: chọn ngẫu nhiên hai tag, giải PnP, đếm số tag phù hợp, lặp [20]. Rẻ hơn và đúng cấu trúc hơn.

Cổng chi-square trên phần dư. Sau khi hội tụ, phần dư chuẩn hoá $\mathbf{r}_i^\top \Sigma_i^{-1} \mathbf{r}_i$ theo lý thuyết có phân bố χ^2 hai bậc tự do. Loại các góc vượt quá một phân vị — chẳng hạn 99% — rồi giải lại. Đây là phép thử có cơ sở thống kê chứ không phải một ngưỡng tùy ý, và nó đòi Σ_i đúng — một lý do nữa để làm mục 6 cho tử tế.

Hàm mất mát bền (robust loss). Thay bình phương bằng Huber hoặc Cauchy, làm giảm ảnh hưởng của phần dư lớn mà không loại hẳn. Ưu điểm so với loại cứng: liên tục, nên không gây nhảy khi một điểm dao động quanh ngưỡng. Nhược điểm: nó làm hiệp phương sai đầu ra khó diễn giải hơn.

CẦN NHỚ

Có một phép kiểm nhất quán riêng cho constellation mà không hệ nào tự làm và nó rất đáng thêm: *giải PnP riêng cho từng tag, rồi so các pose với nhau.*

Nếu constellation đúng và mọi tag lành, các pose ấy phải nhất quán trong phạm vi hiệp phương sai của chúng. Nếu pose từ tag A và tag B lệch nhau quá ngưỡng, thì một trong ba điều đã sai: mô hình constellation sai (tag bị lắp lệch, hoặc CAD sai), một tag bị che hoặc bắn, hoặc một tag đang ở nghiệm ambiguity khác.

Phép kiểm này không thay thế PnP hợp nhất — nó chạy *song song* với nó như một giám sát. Chi phí thấp, và nó bắt được đúng loại lỗi mà sai số tái chiếu của bài toán hợp nhất có thể che giấu khi số tag lớn.

3.8 Giới hạn và chế độ hỏng

Giả thiết K và hệ số méo đúng. PnP coi chúng là hằng số đã biết. Nếu chúng sai thì PnP vẫn hội tụ, sai số tái chiếu vẫn nhỏ, và pose sai — không có tín hiệu nào. Đây là lý do A/01 nằm trước chương này và B/12 nằm ở mức xương sống.

Giả thiết tọa độ 3D đúng. Như mục 5 đã nêu, PnP hợp nhất khuếch đại hậu quả của sai số constellation. Chỉ báo: so sai số tái chiếu giữa hợp nhất và từng tag.

Giả thiết tương ứng đúng. PnP giả định biết góc nào ứng với điểm 3D nào. Với tag có mã thì decode giải quyết, nhưng decode sai hướng cho một chế độ hỏng thâm lặn: pose sai đúng 90 độ và sai số tái chiếu vẫn nhỏ vì hình vuông đối xứng bốn lần. Phép bắt: kiểm tính nhất quán với tag khác trong constellation.

Tối ưu cục bộ. Tinh chỉnh hội tụ về cực tiểu gần nhất, không phải cực tiểu toàn cục. Với target phẳng thì có đúng hai cực tiểu (A/04), và khởi tạo quyết định bạn rơi vào cái nào. Đây là lý do khởi tạo bằng IPPE — vốn tìm cả hai — tốt hơn khởi tạo bằng bất cứ thứ gì khác.

Giả thiết nhiễu Gauss. Hàm mục tiêu bình phương là hợp lý cực đại chỉ dưới giả thiết này. Với nhiễu có đuôi nặng (ngoại lai), nó không còn tối ưu — đó là lý do mục 7 tồn tại.

3.9 Thực hành

Pipeline khuyến nghị, năm bước, và nó là công thức tôi khuyên dùng làm mặc định.

Một, khử méo tọa độ góc (hoặc đưa méo vào hàm chiếu — A/01 mục 7).

Hai, khởi tạo bằng IPPE cho target phẳng, lấy cả hai nghiệm và tỉ số sai số tái chiếu ρ của chúng. Với constellation không đồng phẳng, dùng EPnP hoặc SQPnP.

Ba, tinh chỉnh LM trên toàn bộ tập góc của mọi tag nhìn thấy, với tọa độ 3D trong hệ dock và trọng số theo Σ_i từ mục 6.

Bốn, công chi-square trên phần dư, loại góc ngoại lai, giải lại nếu có loại.

Năm, xuất ra ba thứ chứ không phải một: pose, hiệp phương sai $\mathcal{I}^{-1}$, và ρ . Ba con số ấy là tối thiểu để tăng trên (B/11, B/13) làm việc có cơ sở.

Trong OpenCV, chuỗi ấy là `undistortPoints` $\rightarrow$ `solvePnPGeneric` với `SOLVEPNP_IPPE_SQUARE` $\rightarrow$ `solvePnPRefineLM`, cộng phần tính $\mathcal{I}$ tự viết (OpenCV không trả về nó, và đây là một thiếu sót đáng tiếc của API).

Active recall (che cột phải, tự trả lời, rồi mở ra đối chiếu)

Thuật ngữ / câu hỏi	Trả lời
●●● Hai tầng của PnP	Dạng đóng (P3P, EPnP, IPPE, DLT) cực tiểu sai số <i>đại số</i> , không cần khởi tạo, nhanh. Tinh chỉnh phi tuyến cực tiểu sai số <i>tái chiếu</i> — đây mới là ước lượng hợp lý cực đại và là nghiệm cuối.

Thuật ngữ / câu hỏi	Trả lời (tiếp)
●●● Vì sao P3P có tối đa 4 nghiệm	Khử hai trong ba phương trình định lý cosin cho một đa thức bậc bốn [21]. Cần điểm thứ tư để chọn — đó là lý do RANSAC cho PnP dùng tập con 4 điểm.
●●● 4 nghiệm P3P vs 2 nghiệm phẳng	Khác loại. 4 nghiệm P3P là do <i>thiếu dữ liệu</i> , biến mất khi thêm điểm. 2 nghiệm phẳng là do <i>đối xứng hình học</i> , không biến mất khi thêm điểm đồng phẳng (A/04).
●●● Ba lý do phải tinh chỉnh	Độ chính xác (hợp lý cực đại); ma trận thông tin $\mathcal{I} = \sum J^T \Sigma^{-1} J$ sinh ra <i>miễn phí</i> ; và chi phí gần bằng không (hệ 6×6).
●●● Phương trình chuẩn	$(\sum_i J_i^T \Sigma_i^{-1} J_i) \delta = \sum_i J_i^T \Sigma_i^{-1} \mathbf{r}_i$. Ma trận về trái chính là $\mathcal{I}$ mà A/05 đảo.
●●● Vì sao không dùng góc Euler	Khoá gimbal: ở một số tư thế hai trục trùng nhau, Jacobian mất hạng, bộ giải phân kỳ. Triệu chứng đặc trưng: hỏng lặp lại ở đúng vài tư thế cụ thể. Cách đúng: tham số hoá cục bộ $R_{k+1} = R_k \exp(\omega^\wedge)$ [25].
●●● Jacobian của phép chiếu	$\partial \hat{\mathbf{u}} / \partial \mathbf{X}_c = (f/Z) \begin{pmatrix} 1 & 0 & -X/Z \\ 0 & 1 & -Y/Z \end{pmatrix}$. Hệ số chung f/Z giải thích $\delta x = Z \sigma / f$; cột thứ ba tỉ lệ với X/Z — điểm trên trục quang không cho thông tin chiều sâu nào.
●●● PnP hợp nhất vs trung bình pose	Hợp nhất cộng <i>thông tin</i> trong cùng hệ toạ độ, nên giữ được ràng buộc rằng các tag gắn cứng với nhau — tức giữ được cơ chế baseline. Trung bình pose cộng <i>ước lượng</i> , vứt bỏ ràng buộc ấy [2, §2].
●●● Điều kiện để hợp nhất tốt hơn	Toạ độ 3D constellation phải đúng. Nếu sai, hợp nhất <i>tệ hơn</i> trung bình vì nó ép một pose duy nhất giải thích mô hình sai. Chỉ báo: số sai số tái chiếu hợp nhất với từng tag riêng.
●● Bốn nguồn làm Σ_i khác nhau	Khoảng cách và kích thước biểu kiến (lớn nhất trong docking); góc nghiêng; vị trí trên ảnh (méo còn sót ở rìa); điều kiện quang học cục bộ.
●● Vì sao trọng số đúng quan trọng hơn độ chính xác	Vì $\mathcal{I}^{-1}$ chỉ là ước lượng bất định đáng tin nếu Σ_i đầu vào đúng, và cả chuỗi hợp nhất ở B/11 lẫn công chi-square ở mục 7 phụ thuộc vào tính đúng ấy [26].
●● RANSAC ở mức nào	Mức <i>tag</i> , không mức góc — vì bốn góc một tag cùng đúng hoặc cùng sai. Chọn ngẫu nhiên hai tag, giải, đếm tag phù hợp.
●● Công chi-square	Phần dư chuẩn hoá $\mathbf{r}^T \Sigma^{-1} \mathbf{r} \sim \chi_2^2$; loại góc vượt phân vị 99% rồi giải lại. Có cơ sở thống kê, nhưng <i>đòi</i> Σ đúng.
●● Phép kiểm nhất quán constellation	Giải PnP riêng từng tag rồi so các pose. Lệch quá ngưỡng $\Rightarrow$ constellation sai, tag bị che/bẩn, hoặc một tag đang ở nghiệm ambiguity khác. Chạy song song với hợp nhất như một giám sát.
● Chế độ hỏng thâm lặng nhất	Decode sai hướng: pose sai đúng 90° và sai số tái chiếu <i>vẫn nhỏ</i> vì hình vuông đối xứng bốn lần. Chỉ bắt được bằng kiểm nhất quán với tag khác.

●●● phải thuộc ●● nên nắm ● biết là có

I Kiến thức này đang sống ở đâu (công trình đang dùng chính ý tưởng của chương; chốt 9/2026)

Công trình / hệ thống	Dùng ý tưởng nào của chương	Ghi chú
OpenCV solvePnPGeneric	Trả về nhiều nghiệm kèm sai số tái chiếu từng nghiệm; hỗ trợ IPPE, EPnP, SQPnP, P3P	API quan trọng nhất của cả cây — xem D/17
OpenCV solvePnPRefineLM	Bước tinh chỉnh LM trên sai số tái chiếu	Bắt buộc có trong pipeline; rẽ
apriltag apriltag_pose.c	Cài IPPE cho tag vuông, tính cả hai nghiệm và tỉ số	Đọc để thấy mục 3 và A/04 thành code

Công trình / hệ thống	Dùng ý tưởng nào của chương	Ghi chú
Ceres [27], GTSAM	Tối ưu tổng quát với tham số hoá trên đa tạp và hàm mất mát bền	Dùng khi bài toán vượt quá một pose duy nhất — B/12
MLPnP [26]	PnP với hiệp phương sai dị hướng cho từng phép đo	Nguồn cho mục 6; ý tưởng áp thẳng vào trọng số góc marker

MINI PROJECT · So PnP hợp nhất với trung bình pose, và đo cái giá của constellation sai · 2 ngày

Mục tiêu	chuyển luận điểm trung tâm của mục 5 từ một lập luận thành một cặp đường cong, và đồng thời đo được ngưỡng sai số constellation mà trên đó hợp nhất bắt đầu tệ hơn.
Dữ liệu	mô phỏng, nối tiếp script của A/05 và A/04.
Các bước	dựng constellation bốn tag với baseline B , tọa độ 3D chân trị đã biết; chiếu, thêm nhiễu σ ; chạy hai đường: (i) PnP hợp nhất trên 16 góc, (ii) bốn PnP riêng rồi trung bình Karcher trên $SO(3)$ cộng trung bình có trọng số cho tịnh tiến; so sai số góc và sai số vị trí của hai cách qua 2000 lần chạy; lặp khi quét B . Sau đó phần thứ hai: cố ý làm sai mô hình constellation — dịch tọa độ 3D của một tag đi e mm so với chân trị — và quét e từ 0 tới 5 mm, xem cách nào xuống dốc nhanh hơn.
Xong khi	bạn có hai đồ thị. Đồ thị một cho thấy hợp nhất thẳng rõ về sai số góc và khoảng cách thẳng tăng theo B . Đồ thị hai cho thấy một giao điểm: với e vượt một ngưỡng nào đó, trung bình pose bắt đầu bền hơn. Con số ngưỡng ấy chính là yêu cầu độ chính xác cơ khí cho constellation của bạn. Kiểm tra chéo bắt buộc: ở $e = 0$ và B rất nhỏ, hai cách phải cho kết quả gần như nhau — nếu không thì phép trung bình Karcher của bạn có lỗi.
Nối với	B/09 và B/12 nhận con số ngưỡng này làm yêu cầu thiết kế; C/15 đưa nó vào bảng ngân sách.

TRẢ LỜI CÂU HỎI MỞ ĐẦU

- A1. Bốn vì hệ ba phương trình định lý cosin, sau khi khử hai ẩn, cho một đa thức bậc bốn theo ẩn còn lại [21]; không phải hai vì hệ thực sự bậc bốn chứ không suy biến, và không phải tám vì các nghiệm ứng với điểm nằm sau camera bị vật lý loại. Ý nghĩa ở tầng code rất cụ thể: bạn *phải* có một điểm thứ tư để chọn trong bốn ứng viên, bằng cách chiếu nó qua từng ứng viên và giữ cái cho sai số tái chiếu nhỏ nhất. Đó là lý do mọi cài đặt RANSAC cho PnP lấy mẫu bốn điểm chứ không ba, dù ba mới là số tối thiểu để giải.
- A2. Vì EPnP cực tiểu hoá một sai số *đại số* — độ lệch trong hệ phương trình tuyến tính của bốn điểm điều khiển ảo — chứ không phải sai số tái chiếu tính bằng pixel [23]. Hai đại lượng ấy không tỉ lệ với nhau, nên nghiệm tối ưu theo cái này không tối ưu theo cái kia, và chênh lệch đáng kể khi nhiễu không nhỏ. Nhưng có một lý do thứ hai quan trọng không kém: EPnP không sinh ra ma trận thông tin. Không có $\mathcal{I}$ thì không có hiệp phương sai để hợp nhất ở B/11, không có cơ sở cho cổng chi-square, và không có gì đưa vào ngân sách sai số. Bước tinh chỉnh cho cả hai thứ với chi phí gần bằng không.
- A3. Khác nhau vì PnP hợp nhất *biết* rằng bốn tag gắn cứng với nhau ở những vị trí đã biết, còn trung bình pose thì không. Cụ thể: hợp nhất cộng các ma trận thông tin $J_i^\top \Sigma_i^{-1} J_i$ trong cùng một hệ tọa độ, với J_i tính từ tọa độ 3D thật của từng góc trong hệ dock — nên thông tin về hướng đến từ chênh lệch vị trí giữa các tag cách nhau B , tức cơ chế baseline của A/05. Trung bình pose tính hiệp phương sai riêng cho từng tag trong hệ của tag ấy, vớt bỏ ràng buộc liên tag, nên mỗi pose vẫn chịu hệ số $\sin \theta$ và trung bình chỉ giảm nhiễu theo $\sqrt{M}$. Cách đúng là hợp nhất — với một điều kiện: tọa độ 3D constellation phải đúng, nếu không thì hợp nhất lại tệ hơn.
- A4. Khoá gimbal. Góc Euler có kỳ dị ở một số tư thế nơi hai trục quay trùng nhau; tại đó ánh xạ từ ba tham số sang $SO(3)$ mất hạng, nên Jacobian mất hạng, nên phương trình chuẩn suy biến và bước cập nhật nổ tung. Triệu chứng đặc trưng và chẩn đoán được: nó hỏng *lặp lại ở đúng vài tư thế cụ thể* chứ không ngẫu nhiên — nếu bạn thấy phân kỳ luôn xảy ra quanh cùng một giá trị pitch thì gần như chắc chắn là chuyện này. Cách sửa đúng không phải thêm chính quy hoá

hay đổi ngưỡng mà là đổi tham số hoá: giữ R dưới dạng ma trận hoặc quaternion, tham số hoá số *gia* bằng vectơ ba chiều qua ánh xạ mũ, và đặt lại sau mỗi bước — tức tối ưu quanh gốc của $se(3)$ nơi không có kỳ dị [25].

- A5. Bạn đang giả định rằng mọi góc có cùng độ tin cậy, và giả định ấy sai theo chiều *nguy hiểm*: các góc kém nhất được cho cùng tiếng nói với các góc tốt nhất, nên chúng kéo nghiệm về phía chúng nhiều hơn mức đáng có. Cụ thể trong tình huống mô tả: tag ở xa có kích thước nhỏ trên ảnh nên cạnh ngắn nên định vị góc kém, và tag nghiêng mạnh có cạnh bị nén nên càng kém — cả hai hiệu ứng cộng lại có thể làm σ của nó lớn gấp nhiều lần tag gần. Hậu quả thứ hai, và nó tệ hơn hậu quả thứ nhất: hiệp phương sai $\mathcal{I}^{-1}$ mà bạn xuất ra sẽ *sai*, vì nó được tính từ Σ_i giả định. Nó sẽ báo một bất định nhỏ hơn thực tế, và mọi tầng phía trên dùng con số ấy làm trọng số sẽ bị lừa theo.

NÓI LẠI THẬT ĐƠN GIẢN

Bạn đứng giữa một cánh đồng và nhìn thấy vài cái cột mà bạn biết chính xác chúng ở đâu. Bạn không đo được khoảng cách tới chúng, chỉ đo được hướng: cột này ở bên trái một chút, cột kia ở bên phải nhiều hơn. Câu hỏi là bạn đang đứng ở đâu và quay mặt về hướng nào.

Một cái cột thì chưa nói được gì. Ba cái thì gần đủ — nhưng chỉ gần, vì hoá ra vẫn còn tới bốn chỗ mà từ đó ba cái cột trông đúng như vậy. Phải có cái cột thứ tư mới loại được ba chỗ giả.

Với phép đo hoàn hảo thì xong ở đó. Nhưng phép đo không hoàn hảo: bạn đo hướng lệch đi một chút, nên không có chỗ nào trên cánh đồng làm mọi hướng khớp cùng lúc. Bài toán đổi thành: tìm chỗ mà tổng độ lệch nhỏ nhất.

Và ở đây có hai cách làm, thường bị nhầm là một. Cách thứ nhất là dùng một công thức có sẵn để ra ngay một chỗ khá tốt — nhanh, nhưng công thức ấy tối thiểu hoá một thứ tiện tính chứ không phải chính cái tổng độ lệch mà bạn quan tâm. Cách thứ hai là đứng ở chỗ ấy rồi dịch từng chút theo hướng làm tổng lệch giảm, cho tới khi hết giảm được. Cách thứ hai mới cho câu trả lời đúng, và nó rẻ — vài chục bước tính toán nhỏ.

Nhưng cái quý nhất của cách thứ hai không phải câu trả lời. Khi bạn dừng lại, bạn biết thêm một thứ: mặt đất quanh chỗ bạn đứng *dốc* thế nào. Dốc đứng theo mọi hướng nghĩa là bạn rất chắc chắn — dịch đi một tí là tổng lệch tăng ngay. Có một hướng mà đi dọc theo nó gần như phẳng nghĩa là theo hướng ấy bạn hầu như không biết gì. Cái độ dốc đó chính là thước đo độ tin cậy, và nó có sẵn trong phép tính — ai bỏ bước dịch chuyển thì cũng mất luôn nó, và mất nó thì không còn gì để nói mình chắc chắn tới đâu.

Còn một chuyện nữa, về mấy cái cột. Nếu bạn biết chúng gắn cứng với nhau trên cùng một cái khung — cột này cách cột kia đúng bốn mươi phân — thì bạn nên dùng thông tin đó. Cách sai là tính riêng vị trí theo từng cột rồi lấy trung bình bốn kết quả; làm thế là vứt bỏ hiểu biết về cái khung. Cách đúng là để cả bốn cột cùng kéo về một lời giải duy nhất, vì khi ấy chính khoảng cách giữa chúng trở thành thước đo góc.

Chỗ không rút gọn được: cái giá của chuyện dùng khung là bạn phải *tin* vào khung. Nếu cái khung thật ra không đúng như bản vẽ — cột lệch đi vài milimét — thì cách dùng khung lại tệ hơn cách tính riêng, vì nó ép mọi thứ khớp với một mô hình sai. Ranh giới giữa hai chế độ ấy thì phải đo, không suy ra được.

Một câu: Công thức cho bạn chỗ gần đúng, việc dịch từng chút cho bạn chỗ đúng và cho bạn biết mình chắc tới đâu.

3.10 Điều gì chứng minh cách hiểu này sai

Mệnh đề chính — PnP hợp nhất trên toàn bộ tập góc tốt hơn trung bình pose từng tag, và khoảng cách thẳng tăng theo baseline — bị bác bỏ nếu mô phỏng ở mini project cho thấy hai cách cho cùng độ tản mát góc trên constellation trải rộng, với toạ độ 3D chính xác. Đây là mệnh đề tôi dẫn bằng lập luận thông tin (cửa a) và đối chiếu với nguyên lý tổng quát trong [2, §2] (cửa c một phần), và nó *chưa* qua cửa (b). Nó là mệnh đề có hậu quả kiến trúc lớn nhất trong chương nên nên kiểm sớm.

Mệnh đề thứ hai — *hợp nhất trở nên tệ hơn trung bình khi sai số constellation vượt một ngưỡng* — là một dự đoán định tính của tôi dựa trên cơ chế ở mục 5. Nó bị bác bỏ nếu mô phỏng cho thấy hợp nhất vẫn thắng ở mọi mức sai số constellation cho tới khi cả hai đều vô dụng. Nếu vậy thì lời cảnh báo ở mục 5 là quá mạnh và nên được nới.

Mệnh đề thứ ba — *tinh chỉnh cải thiện độ chính xác đáng kể so với lời giải dạng đóng* — bị bác bỏ nếu mô phỏng cho thấy chênh lệch nhỏ hơn nhiều so với σ ở mức nhiễu thực tế. Điều đáng nói: kể cả nếu mệnh đề này bị bác bỏ, lời khuyên “luôn tinh chỉnh” vẫn đứng vững, vì lý do thứ hai — ma trận thông tin miễn phí — không phụ thuộc vào nó.

Cuối cùng, một mệnh đề mà tôi đã cố ý *không* phát biểu: chương này không xếp hạng các thuật toán PnP theo độ chính xác, vì tôi không có dữ liệu so sánh được. Nếu bạn cần xếp hạng ấy cho hệ của mình thì nó phải đo trên chính cấu hình của bạn — và mini project cung cấp khung để làm việc đó.

3.11 Kết nối

Chương này là trục của Phần A: nó nhận mô hình từ hai chương trước và cấp Jacobian cho hai chương sau.

Nối lên trên. [A/01-mô-hình-camera-va-distortion](#) cấp hàm chiếu $\hat{u}(\xi)$ và cảnh báo rằng PnP coi K cùng hệ số méo là hằng số đúng — nếu chúng sai thì PnP hội tụ về một pose sai mà không có tín hiệu. [A/02-homography-va-hình-học-phang](#) cấp một cách khởi tạo, và cũng cấp lý do vì sao IPPE là lựa chọn tốt hơn phân rã H cho target phẳng.

Nối xuống dưới trong Phần A. [A/04-pose-ambiguity-target-phang](#) là hệ quả trực tiếp của mục 8: tối ưu là cực bộ, có hai cực tiểu, và khởi tạo quyết định bạn rơi vào cái nào. [A/05-lan-truyen-sai-so-pixel-sang-pose](#) nhận *đúng* ma trận $\mathcal{I}$ từ phương trình chuẩn (3.2) và toàn bộ chương ấy chỉ làm một việc là đọc ý nghĩa vật lý của $\mathcal{I}^{-1}$; Jacobian ở mục 4C là chỗ nguồn gốc của bốn quan hệ tỉ lệ rõ nhất.

Nối xuống Phần B. [B/08-corner-refinement](#) cấp Σ_i mà mục 6 cần — không chỉ độ lớn mà cả hướng, vì hiệp phương sai của một góc từ giao hai đường thẳng vốn dị hướng. [B/09-multi-marker-constellation](#) là chương biến mục 5 thành thiết kế: nó trả lời đặt tag ở đâu để $\mathcal{I}$ có điều kiện tốt nhất. [B/12-calibration-cho-docking](#) nhận cảnh báo ở mục 5 làm lý do bắt buộc cho bước đo constellation bằng bundle adjustment. [B/11-uoc-luong-theo-thoi-gian](#) nhận $\mathcal{I}^{-1}$ làm trọng số khi hợp nhất theo thời gian — và nó chỉ làm được điều đó nếu bước tinh chỉnh đã chạy.

Cuối cùng, một nối sang [B/13-docking-control](#): mục 9 khuyến nghị xuất ra ba thứ (pose, hiệp phương sai, ρ) chứ không phải một, và lý do đầy đủ của khuyến nghị ấy nằm ở chương điều khiển — một bộ điều khiển docking cần biết không chỉ “mục tiêu ở đâu” mà cả “tôi tin điều đó tới mức nào” và “có phải tôi đang nhìn nhầm nghiệm không”.

Câu hỏi tự kiểm (trả lời không nhìn bài)

1. Vì sao ba điểm cho bốn nghiệm chứ không phải hai, và vì sao đó là một loại đa nghiệm hoàn toàn khác với hai nghiệm của target phẳng?
2. Nêu ba lý do bắt buộc phải chạy tinh chỉnh sau lời giải dạng đóng, và chỉ ra lý do nào vẫn đứng vững kể cả khi cải thiện độ chính xác là không đáng kể.
3. Trong Jacobian của phép chiếu, cột thứ ba tỉ lệ với X/Z và Y/Z . Điều đó nói gì về khả năng đo chiều sâu của một điểm nằm trên trục quang, và nó nối với công thức nào ở [A/05](#)?
4. Một đồng nghiệp trung bình bốn pose từ bốn tag. Giải thích chính xác thông tin gì bị vứt bỏ, và nêu điều kiện mà dưới đó cách làm của họ lại *đúng hơn* cách hợp nhất.

5. Bộ giải của bạn phân kỳ ở đúng một vài tư thế và chạy tốt ở mọi tư thế khác. Nêu nguyên nhân khả dĩ nhất, cơ chế của nó, và cách sửa đúng.
6. Vì sao tổng chi-square trên phần dư đòi Σ_i đúng, và chuyện gì xảy ra với tỉ lệ loại nhầm nếu bạn dùng một σ chung quá nhỏ?
7. Decode sai hướng cho pose sai 90° mà sai số tái chiếu vẫn nhỏ. Vì sao, và cơ chế nào trong chương này phát hiện được nó?

Pose ambiguity của target phẳng (Planar Pose Ambiguity)

PHỤ THUỘC A/02-homography-va-hình-hoc-phang (homography của một mặt phẳng và phép phân rã nó thành pose – chỗ hai nghiệm xuất hiện lần đầu); A/03-ppnp-va-uoc-luong-tu-the (IPPE và sai số tái chiếu – chương này dùng liên tục cả hai).

TÓM TẮT

Chương này giải thích hiện tượng mà gần như ai làm marker cũng gặp và ít người gọi đúng tên: pose nhảy qua lại giữa hai giá trị dù camera và tag gần như đứng yên. Nguyên nhân không phải nhiễu, không phải bug, và không chữa được bằng bộ lọc: với một target *phẳng*, bài toán PnP có *đúng hai* nghiệm cục bộ, và cả hai đều giải thích ảnh gần như tốt ngang nhau. Hai nghiệm ấy là ảnh gương của nhau qua một mặt phẳng, chúng có cùng vị trí tâm gần đúng nhưng hướng nghiêng ngược dấu, và khoảng cách giữa chúng theo góc có thể lên tới hàng chục độ. Ambiguity tệ nhất đúng ở tư thế fronto-parallel — tức đúng tư thế của một robot tiếp cận dock thẳng trục. Nếu chỉ nhớ một điều: *đây là một tính chất của hình học phẳng, không phải một khiếm khuyết của thuật toán*, nên cách chữa triệt để là phá tính phẳng chứ không phải lọc mạnh hơn.

KIẾN THỨC CẦN CÓ TRƯỚC

- **target phẳng** — tập điểm điều khiển 3D cùng nằm trên một mặt phẳng. Một tag vuông đơn lẻ là target phẳng; bốn tag dán trên cùng tấm bảng cũng vẫn là target phẳng, dù có bao nhiêu điểm đi nữa. Đây là điểm quan trọng nhất của định nghĩa: *thêm điểm đồng phẳng không phá được tính phẳng*.
- **cực tiểu cục bộ (local minimum)** — một điểm mà hàm mục tiêu nhỏ hơn mọi điểm lân cận, nhưng không nhất thiết nhỏ nhất toàn cục. Bộ giải phi tuyến dừng ở cực tiểu cục bộ gần nhất với điểm khởi đầu, và nó không có cách nào biết còn cực tiểu nào khác ở xa.
- **sai số tái chiếu (reprojection error)** — tổng bình phương khoảng cách giữa góc đo được và góc dự đoán bởi một pose giả định, tính bằng pixel. Đây là hàm mục tiêu của PnP tinh chỉnh và là đại lượng duy nhất bộ giải “nhìn thấy”.
- **tỉ số ambiguity** — tỉ số giữa sai số tái chiếu của nghiệm tốt thứ hai và nghiệm tốt nhất, $\rho = e_2/e_1 \geq 1$. Gần 1 nghĩa là hai nghiệm giải thích ảnh tốt ngang nhau và không phân biệt được; lớn (chẳng hạn trên 3–5) nghĩa là nghiệm thứ nhất thắng rõ ràng.
- **IPPE — Infinitesimal Plane-based Pose Estimation**, thuật toán giải PnP cho target phẳng dạng đóng, và điểm đáng giá nhất của nó là nó trả về *cả hai* nghiệm kèm sai số tái chiếu của từng nghiệm [17].
- **đồng phẳng và không đồng phẳng** — tập điểm gọi là không đồng phẳng nếu không tồn tại một mặt phẳng chứa tất cả. Với tập điểm không đồng phẳng, bài toán PnP có nghiệm duy nhất — đây là kết quả trung tâm và là lối thoát của cả chương.

CÂU HỎI MỞ ĐẦU

- Q1. Camera và tag của bạn đứng yên hoàn toàn trên bàn. Pose trả về vẫn nhảy qua lại giữa hai giá trị cách nhau 20° vài lần mỗi giây. Nếu không phải nhiễu và không phải bug, thì đó là gì — và vì sao lấy trung bình hai giá trị ấy lại là việc tệ nhất có thể làm?
- Q2. Bạn dán thêm ba tag nữa lên cùng tấm bảng, tổng cộng bốn tag và mười sáu góc thay vì bốn. Bài toán ambiguity được cải thiện bao nhiêu, và vì sao câu trả lời là “gần như không”?
- Q3. Hai nghiệm của một target phẳng có quan hệ hình học rất cụ thể với nhau, không phải hai giá trị ngẫu nhiên. Quan hệ ấy là gì, và biết nó giúp gì cho việc chẩn đoán?
- Q4. Thuật toán nào cũng phải chọn một trong hai nghiệm. Nếu nó chọn bằng cách lấy nghiệm có sai số tái chiếu nhỏ hơn, thì trong trường hợp nào cách chọn ấy tệ hơn cả việc tung đồng xu?
- Q5. Có một cách khử ambiguity triệt để, không cần lọc, không cần thông tin thời gian, không cần cảm biến khác — và nó không nằm ở tầng phần mềm. Cách ấy là gì, và vì sao nó triệt để trong khi mọi cách khác chỉ là giảm thiểu?

4.1 Đặt vấn đề

Có một triệu chứng rất đặc trưng, và nếu bạn đã làm marker thì gần như chắc chắn đã thấy nó.

Bạn đặt camera trên một cái giá, dán một tag lên tường, không ai chạm vào gì cả. Bạn in pose ra màn hình. Vị trí tâm tag thì ổn định — nó dao động vài phần mười milimét, đúng như [A/05](#) dự đoán. Nhưng góc thì không ổn định chút nào: nó nhảy giữa, chẳng hạn, $+12^\circ$ và -8° theo trục pitch, vài lần mỗi giây, một cách hoàn toàn ngẫu nhiên. Bạn thử tag khác, thử detector khác, thử camera khác. Triệu chứng không đổi.

Phản ứng đầu tiên gần như luôn là “nhiều quá lớn”, và nó dẫn tới một chuỗi việc làm không giúp gì: thêm ánh sáng, tăng độ phân giải, dùng subpixel tốt hơn, thêm bộ lọc thông thấp. Ba việc đầu làm nhiều giảm thật nhưng *tần suất nhảy vẫn nguyên* — chỉ có điều mỗi giá trị trong hai giá trị trở nên ổn định hơn. Việc thứ tư thì tệ nhất trong bốn: lọc thông thấp lấy trung bình hai giá trị và cho ra một giá trị thứ ba mà *không* phải nghiệm nào cả — một pose sai hoàn toàn nhưng trông rất mượt.

Nguyên nhân đã được biết từ lâu và nó không nằm ở nhiễu. Với một target phẳng, bài toán tìm pose từ ảnh có *hai* nghiệm cục bộ, và hiện tượng này được nhận diện ít nhất từ giữa những năm 1990 trong công trình của Oberkampf và cộng sự [28], rồi được phân tích hoàn chỉnh bởi Schweighofer và Pinz [29], và cuối cùng được đưa vào một thuật toán dạng đóng trả về cả hai nghiệm bởi Collins và Bartoli [17]. Ba nhóm ấy độc lập nhau và cách nhau ba thập niên, nên kết quả này thuộc loại đã được kiểm chứng kỹ — đây là một trong số ít khẳng định trong cây mà tôi tự tin ở mức cao.

Điều đáng nói là vì sao nó vẫn làm nhiều người bất ngờ. Lý do nằm ở chỗ hiện tượng này *không xuất hiện trong các tình huống mà người ta hay thử đầu tiên*. Khi bạn cầm tag trong tay và xoay qua xoay lại để thử, tag thường nghiêng khá nhiều, và ở góc nghiêng lớn thì một nghiệm thẳng rõ ràng nên bạn không thấy vấn đề. Vấn đề chỉ nổi lên khi tag ở tư thế gần vuông góc với trục quang — tức đúng khi bạn gắn nó lên một cái dock và cho robot tiếp cận thẳng trục. Nói cách khác: *hiện tượng này ẩn mình trong giai đoạn thử nghiệm và xuất hiện đúng lúc triển khai*.

Cách hiển nhiên hơn để chữa — và cách hầu như ai cũng thử — là chọn nghiệm có sai số tái chiếu nhỏ hơn. Cách này đúng về nguyên tắc và nó là cách mọi thư viện đang làm. Chỗ nó hỏng là khi hai sai số gần bằng nhau: lúc ấy việc “chọn cái nhỏ hơn” trở thành việc chọn theo nhiễu, và nó tệ hơn tung đồng xu ở một điểm — tung đồng xu ít nhất không giả vờ là một quyết định có cơ sở. Chương này dành mục 5 cho việc *đo* xem ta đang ở tình huống nào, và mục 6 cho việc thoát ra khỏi tình huống ấy.

TRỰC GIÁC

Cầm một tấm bìa hình chữ nhật và giơ trước mắt, hơi nghiêng đi một chút. Nhắm một mắt lại, và nhìn kỹ hình dạng bạn thấy: một hình thang, cạnh trên hơi ngắn hơn cạnh dưới.

Bây giờ nghiêng tấm bìa theo *chiều ngược lại*, cùng một góc. Bạn lại thấy một hình thang, và nếu góc đúng bằng nhau thì hình thang lần này là ảnh lật của hình thang lần trước. Với một mắt, và với một tấm bìa đối xứng như hình vuông của một cái tag, *hai hình thang ấy trông giống nhau*.

Đó là toàn bộ vấn đề. Mắt bạn — và camera — không có cách nào phân biệt “nghiêng ra xa ở trên” với “nghiêng ra xa ở dưới” chỉ từ hình dạng phẳng của bóng chiếu. Với người thật thì còn có bóng đổ, có tô bóng, có thị sai hai mắt; camera đơn nhìn một cái tag đen trắng thì không có gì trong số đó.

Bây giờ để ý một điều quan trọng: nếu bạn nghiêng tấm bìa *rất nhiều*, hai hình thang bắt đầu khác nhau rõ, vì cạnh gần và cạnh xa chênh nhau nhiều và phối cảnh không còn đối xứng nữa. Còn nếu tấm bìa gần như vuông góc với hướng nhìn, hai hình thang gần như trùng khít — và lúc đó bạn hoàn toàn không phân biệt được.

Cuối cùng, lối thoát: dựng một cái que vuông góc lên mặt tấm bìa. Bây giờ hai tư thế không còn giống nhau nữa, vì cái que chỉ về hai hướng khác hẳn. Cái que ấy chính là điểm *không đồng phẳng*, và đó là toàn bộ nội dung của mục 6.

4.2 Vì sao đúng hai nghiệm: hình học của phép lật

Trực giác trên đủ để tin, nhưng để dùng được thì cần biết hai nghiệm ấy quan hệ với nhau thế nào một cách chính xác.

Xét một target phẳng và một pose thật (R, t) đặt nó trong hệ camera. Gọi $\mathbf{n}$ là pháp tuyến của mặt phẳng tag trong hệ camera, và gọi $\mathbf{c}$ là hướng từ tâm camera tới tâm tag. Hai vectơ ấy xác định một mặt phẳng — gọi là *mặt phẳng lật*, chứa trục nhìn tới tâm tag và pháp tuyến của tag.

Nghiệm thứ hai, gọi là (R', t') , thu được bằng cách *lật* target quanh trục nằm trong mặt phẳng tag và vuông góc với mặt phẳng lật. Nói cách khác: pháp tuyến $\mathbf{n}$ được phản xạ gương qua đường nhìn $\mathbf{c}$. Nếu pháp tuyến thật nghiêng một góc α so với đường nhìn về một phía, thì pháp tuyến của nghiệm thứ hai nghiêng đúng góc α về phía bên kia [29, §III].

Từ mô tả ấy rút ra ngay ba hệ quả rất dùng được.

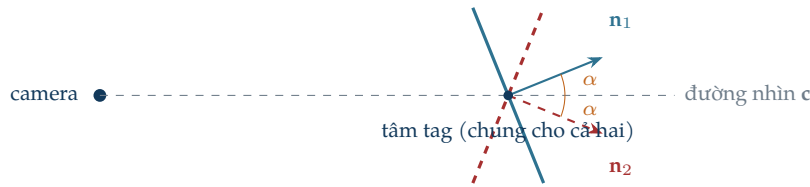
Hệ quả một: hai nghiệm cách nhau 2α về góc. Nếu tag nghiêng 10° so với fronto-parallel, hai nghiệm cách nhau 20° . Nếu tag nghiêng 30° , hai nghiệm cách nhau 60° — rất xa nhau, và cũng rất dễ phân biệt. Ngược lại, nếu tag gần fronto-parallel ($\alpha \rightarrow 0$) thì hai nghiệm *hội tụ về nhau* và đồng thời trở nên không phân biệt được. Đây là một quan hệ đáng chú ý vì nó đi theo hai chiều ngược nhau: ở góc nghiêng lớn, hai nghiệm xa nhau nhưng dễ phân biệt; ở góc nghiêng nhỏ, chúng gần nhau nhưng không phân biệt được.

Hệ quả hai: vị trí tâm gần như giống nhau ở cả hai nghiệm. Phép lật giữ nguyên (tới xấp xỉ bậc nhất) khoảng cách từ camera tới tâm tag, vì nó chỉ đổi hướng của mặt phẳng chứ không đổi vị trí tâm. Đây là lý do triệu chứng quan sát được là “vị trí ổn định, góc nhảy” chứ không phải cả sáu tham số cùng nhảy. Nó cũng là một manh mối chẩn đoán rất tốt: nếu bạn thấy góc nhảy mà vị trí đứng yên, thì gần như chắc chắn đó là ambiguity chứ không phải nhiễu chung.

Hệ quả ba, và đây là hệ quả thực dụng nhất: lấy trung bình hai nghiệm là vô nghĩa. Trung bình của hai xoay cách nhau 2α là một xoay ở chính giữa, tức xoay ứng với $\alpha = 0$, tức fronto-parallel. Nói cách khác, một bộ lọc thông thấp trên đầu ra pose sẽ luôn kéo bạn về phía kết luận “tag đang vuông

góc với camera”, bất kể tag thực sự nghiêng bao nhiêu. Đó không phải làm mượt; đó là tạo ra một thiên lệch có hệ thống hướng về một giá trị cụ thể và sai.

Hình 4.1 vẽ cấu hình này.



Hai nghiệm là **ảnh gương của nhau qua đường nhìn**. Chúng cách nhau 2α về góc và có *cùng* vị trí tâm tới xấp xỉ bậc nhất — nên triệu chứng là “góc nhảy, vị trí yên”. Khi $\alpha \rightarrow 0$ (fronto-parallel) hai nghiệm hội tụ *và* trở nên không phân biệt được: trung bình của chúng luôn là $\alpha = 0$, một giá trị sai có hệ thống.

Hình 4.1. Quan hệ hình học giữa hai nghiệm của một target phẳng, theo mô tả trong [29, §III]. Pháp tuyến của nghiệm thứ hai là ảnh phản xạ của pháp tuyến thật qua đường nhìn tới tâm tag. Hệ quả thực dụng quan trọng nhất nằm ở dòng cuối: mọi phép làm mượt trên đầu ra pose đều kéo kết quả về phía fronto-parallel, tức tạo ra một thiên lệch chứ không phải giảm nhiễu.

4.3 Vì sao thêm điểm đồng phẳng không cứu được

Đây là mục ngắn nhất của chương và cũng là mục hay bị bỏ qua nhất, nên nó đáng đứng riêng.

Phản xạ tự nhiên khi gặp ambiguity là thêm dữ liệu: dán thêm tag, dùng tag có nhiều điểm hơn, chuyển sang ChArUco board với hàng chục góc chessboard. Trực giác đằng sau là “nhiều ràng buộc hơn thì nghiệm xác định hơn”, và trực giác ấy đúng cho *nhiều* nhưng sai cho *ambiguity*.

Lý do nằm ở chỗ phép lật ở mục 2 là một phép biến đổi tác động lên *cả mặt phẳng*, không lên từng điểm riêng lẻ. Khi ta lật target, mọi điểm trên mặt phẳng đều được lật cùng nhau, và hình chiếu của toàn bộ tập điểm sau khi lật vẫn khớp với ảnh gần như tốt như trước. Thêm một điểm thứ năm, thứ mười, thứ trăm — miễn là nó nằm trên cùng mặt phẳng — thì nó cũng bị lật cùng, và nó cũng khớp. Số điểm không phá được một phép đối xứng của hình học.

Điều thêm điểm đồng phẳng *có* làm là giảm nhiễu của mỗi nghiệm. Với nhiều điểm hơn, cả hai nghiệm đều được xác định chính xác hơn, và sai số tái chiếu của cả hai đều nhỏ đi. Nhưng *tỉ* số giữa hai sai số ấy — đại lượng quyết định việc phân biệt — gần như không đổi. Bạn có hai nghiệm sắc nét hơn thay vì hai nghiệm nhoè, và bạn vẫn phải chọn một trong hai.

CẦN NHỚ

Số điểm không phá được ambiguity; chỉ có *tính không đồng phẳng* mới phá được.

Cụ thể: bốn tag dán trên cùng một tấm bảng phẳng vẫn là một target phẳng và vẫn có hai nghiệm. Hai tag dán trên hai mặt phẳng nghiêng với nhau thì không — và hai tag đã đủ. Đây là lý do vì sao lời khuyên ở 00-map không phải “dùng nhiều tag” mà là “dùng constellation *không đồng phẳng*”. Hai lời khuyên nghe gần giống nhau và chúng khác nhau về bản chất.

4.4 Khi nào ambiguity tệ nhất

Bốn yếu tố làm ambiguity nặng thêm, và biết chúng là biết cách tránh.

Góc nghiêng nhỏ (fronto-parallel). Đây là yếu tố mạnh nhất. Khi $\alpha \rightarrow 0$, hai nghiệm hội tụ và tỉ số sai số tái chiếu của chúng tiến về 1. Về mặt cơ chế, đây chính là hiện tượng mà [A/05](#) mục 5 mô tả từ phía khác: độ nhạy của ảnh theo góc nghiêng tỉ lệ với $\sin \alpha$, nên khi α nhỏ thì hai giá trị góc rất khác nhau cho hai ảnh rất giống nhau. Hai chương nói cùng một sự thật: chương 5 nói “hiệp phương sai lớn”, chương này nói “có hai cực tiểu gần nhau”. Trên thực tế cả hai đều đúng và chúng là hai mặt của cùng một ma trận thông tin suy biến.

Tag nhỏ trên ảnh. Tag càng nhỏ thì chênh lệch sai số tái chiếu giữa hai nghiệm càng nhỏ so với nhiễu, vì chênh lệch ấy tỉ lệ với kích thước tag còn nhiễu thì không. Đây là lý do ambiguity nặng lên khi robot còn ở xa và nhẹ đi khi tới gần.

Trường nhìn hẹp, tức tiêu cự dài. Với ống kính tele, phép chiếu tiến gần tới phép chiếu trực giao (orthographic), và dưới phép chiếu trực giao thì hai nghiệm *trùng khít hoàn toàn* — ambiguity trở thành tuyệt đối chứ không chỉ khó phân biệt. Ống kính góc rộng cho nhiều biến dạng phối cảnh hơn, nên phân biệt tốt hơn. Đây là một đánh đổi đáng biết: tele cho f lớn nên sai số ngang nhỏ ([A/05](#)), nhưng nó làm ambiguity nặng hơn.

Tag nằm gần tâm ảnh. Một tag nằm ở rìa ảnh được nhìn dưới một góc chéo so với trục quang, và cái chéo ấy tự nó tạo ra một chút biến dạng phối cảnh phụ trội giúp phân biệt. Tag nằm chính giữa ảnh thì không có lợi thế đó. Hiệu ứng này yếu hơn ba yếu tố trên nhưng nó có thật, và nó gợi ý một mẹo bố trí: đừng đặt tag ở chính giữa trường nhìn nếu có thể tránh.

Bảng 4.1. Bốn yếu tố làm ambiguity nặng thêm, cơ chế, và cách giảm nhẹ ở tầng thiết kế.

Yếu tố	Cơ chế	Cách giảm nhẹ
Góc nghiêng nhỏ	Độ nhạy $\propto \sin \alpha$; hai nghiệm hội tụ khi $\alpha \rightarrow 0$	Nghiêng tag trong constellation; hoặc cho đường tiếp cận chéo
Tag nhỏ trên ảnh	Chênh lệch reprojection tỉ lệ với w , nhiễu thì không	Tag lớn hơn ở cự ly xa; nested multi-scale (B/09)
Trường nhìn hẹp (tele)	Tiến tới chiếu trực giao, nơi hai nghiệm trùng khít	Ống kính rộng hơn; chấp nhận đánh đổi với f nhỏ hơn
Tag ở tâm ảnh	Không có biến dạng phối cảnh phụ trội từ vị trí chéo	Bố trí tag lệch khỏi tâm trường nhìn

4.5 Đo ambiguity: tỉ số sai số tái chiếu

Nếu ambiguity không chứa được bằng phần mềm thì ít nhất nó phải *đo* được bằng phần mềm, và may mắn là có một đại lượng rất tự nhiên cho việc đó.

IPPE trả về hai nghiệm kèm sai số tái chiếu của từng nghiệm [\[17\]](#). Định nghĩa tỉ số ambiguity:

$$\rho = \frac{e_2}{e_1}, \quad e_1 \leq e_2,$$

với e_1 là sai số tái chiếu của nghiệm tốt nhất và e_2 của nghiệm còn lại. Theo định nghĩa $\rho \geq 1$.

Đọc ρ như sau. Khi ρ lớn — chẳng hạn $\rho > 3$ — nghiệm thứ nhất giải thích ảnh tốt hơn hẳn và ta tin nó. Khi $\rho \approx 1$, hai nghiệm giải thích ảnh tốt ngang nhau, và việc chọn nghiệm nhỏ hơn là chọn theo nhiễu; kết quả sẽ nhảy từ khung này sang khung khác đúng như triệu chứng ở mục 1.

Ngưỡng đặt ở đâu thì đúng? Câu trả lời trung thực là *tôi không có một con số có cơ sở lý thuyết*, và tôi ghi rõ điều đó ở đây thay vì đưa ra một con số trông thuyết phục. Cái có cơ sở là cách *dẫn ra* ngưỡng cho hệ của bạn: chạy Monte Carlo ([C/16](#)), với mỗi tư thế ghi lại cả ρ lẫn việc nghiệm được chọn có đúng là nghiệm thật không, rồi vẽ tỉ lệ chọn sai theo ρ . Ngưỡng của bạn là giá trị

ρ mà dưới nó tỉ lệ chọn sai vượt mức bạn chấp nhận được. Ngưỡng ấy phụ thuộc vào σ , vào w , vào số điểm — nên nó phải là ngưỡng của hệ bạn chứ không phải một hằng số chép từ đâu đó.

A. Ví dụ số: chênh lệch reprojection lớn cỡ nào

Để thấy các con số ở bậc nào, làm một ước lượng thô. Giả sử tag 120 px trên ảnh (ví dụ chuẩn của cây, A/05 mục 4), nghiêng $\alpha = 5^\circ$, và $\sigma = 0,2$ px.

Nghiệm sai lệch khỏi nghiệm thật một góc $2\alpha = 10^\circ$. Sai lệch góc ấy gây ra sai lệch trong hình chiếu ở bậc: các điểm ở mép tag lệch khỏi vị trí đúng chừng $(w/2)(1 - \cos 2\alpha)$ theo chiều vuông góc với trục lật, tức $60 \times (1 - \cos 10^\circ) = 60 \times 0,0152 = 0,91$ px. Nhưng phần lớn sai lệch ấy được hấp thụ bằng cách điều chỉnh các tham số pose khác (khoảng cách, vị trí ngang), nên phần dư thực sự còn lại nhỏ hơn nhiều — đây chính là lý do nghiệm thứ hai tồn tại như một cực tiểu chứ không bị loại ngay.

Ý nghĩa của phép ước lượng này không nằm ở con số cụ thể mà ở bậc: *chênh lệch reprojection giữa hai nghiệm ở góc nghiêng nhỏ nằm cùng bậc với nhiễu định vị góc*. Khi hai đại lượng cùng bậc thì phân biệt chúng là việc bất khả thi theo nghĩa thống kê, không phải theo nghĩa “chưa đủ thông minh”. (Tôi suy ra ước lượng bậc này từ hình học; nó chưa được kiểm bằng mô phỏng — đã ghi vào 99-chua-biet.md.)

CẠM BẮY

Ba sai lầm phổ biến khi xử lý ambiguity, xếp theo mức độ tai hại tăng dần.

Nhệ: chỉ lấy nghiệm thứ nhất mà không nhìn ρ . Hệ thống chạy đúng phần lớn thời gian và sai một cách bất ngờ ở đúng những tư thế quan trọng nhất.

Nặng: lọc thông thấp trên góc để làm mượt. Như mục 2 đã chỉ ra, trung bình hai nghiệm luôn là fronto-parallel, nên bộ lọc biến một đại lượng *nhảy nhưng đôi khi đúng* thành một đại lượng *mượt và sai có hệ thống*. Tệ hơn nữa, nó xoá mất triệu chứng nên bạn không còn biết mình đang có vấn đề.

Rất nặng: chọn nghiệm theo tiêu chí “gần với pose khung trước”. Nghe rất hợp lý và nó là một kỹ thuật hợp lệ (B/10 mục về nhất quán thời gian), nhưng nếu dùng một mình thì nó có một chế độ hỏng chết người: một khi đã chọn nhầm nghiệm một lần, tiêu chí “gần khung trước” sẽ giữ bạn ở nghiệm sai đó mãi mãi. Bạn có một hệ thống rất ổn định quanh một giá trị sai, và không có tín hiệu nào báo động.

4.6 Lối thoát triệt để: phá tính đồng phẳng

Mọi thứ ở trên là giảm thiểu. Mục này là cách khử.

Kết quả then chốt: *với một tập điểm điều khiển không đồng phẳng, bài toán PnP có nghiệm duy nhất* (bỏ qua các trường hợp suy biến rất đặc biệt và với đủ điểm). Lý do đã nằm sẵn trong hình học của mục 2: phép lật là một phép đối xứng của mặt phẳng, và nó chỉ bảo toàn hình chiếu vì mọi điểm cùng nằm trên mặt phẳng ấy. Một điểm nằm ngoài mặt phẳng bị phép lật đưa tới một vị trí khác hẳn, hình chiếu của nó lệch đi rõ rệt, và sai số tái chiếu của nghiệm thứ hai nhảy vọt. Nghiệm thứ hai không còn là một cực tiểu nữa — nó bị loại bởi chính dữ liệu.

Điều đáng chú ý là *không cần nhiều* tính không đồng phẳng. Về mặt cấu trúc, chỉ cần một điểm nằm ngoài mặt phẳng là đủ để phá đối xứng. Về mặt thực hành, ta cần độ lệch khỏi mặt phẳng đủ lớn để chênh lệch reprojection vượt hẳn nhiễu — và đây là một câu hỏi định lượng mà B/09 trả lời bằng thiết kế cụ thể, còn C/16 trả lời bằng mô phỏng.

Ba cách hiện thực hoá, xếp theo độ dễ thi công.

Cách một, nghiêng các tag. Giữ tất cả tag trên cùng một tấm bảng nhưng gắn mỗi tag lên một nêm nghiêng $15\text{--}30^\circ$ theo các hướng khác nhau. Rẻ, và nó vừa phá đồng phẳng vừa giải quyết vấn đề $\sin \alpha$ của A/05: mỗi tag bây giờ có góc nghiêng riêng khác không so với trục quang, nên chính nó cũng có độ nhạy góc tốt hơn.

Cách hai, đặt tag ở các độ sâu khác nhau. Một số tag trên mặt phẳng dock, một số trên các gờ nhô ra 5–10 cm. Thi công dễ nếu dock vốn đã có cấu trúc ba chiều, và nó cho baseline theo chiều sâu — một hướng mà cách một không cho.

Cách ba, target ba chiều có chủ ý. Một cấu trúc dạng góc (hai hoặc ba mặt phẳng vuông góc nhau, mỗi mặt mang tag). Đây là cách mạnh nhất về mặt hình học nhưng nó chiếm không gian và dễ bị che khuất một phần khi robot tới gần.

CẦN NHỚ

Ambiguity là tính chất của *hình học phẳng*, không phải của thuật toán. Ba hệ quả:
 Một, không thuật toán nào khử được nó khi target còn phẳng — chỉ chọn được nghiệm, và việc chọn có xác suất sai khác không.
 Hai, cách khử duy nhất triệt để là phá tính đồng phẳng, và việc đó xảy ra ở xương cơ khí chứ không trong trình biên dịch.
 Ba, chi phí của việc khử rất thấp so với lợi ích: một nêm nghiêng 20° dưới mỗi tag là một chi tiết gia công tầm thường, và nó đồng thời giải quyết cả bài toán ambiguity lẫn bài toán độ nhạy góc.

4.7 Giới hạn và chế độ hỏng

Ba giới hạn của cách hiểu trình bày trong chương này.

“Hai nghiệm” là kết quả cho target phẳng dưới mô hình pinhole không nhiễu. Trong thực tế có nhiễu, và khi nhiễu lớn thì bề mặt hàm mục tiêu có thể có nhiều hơn hai cực tiểu cục bộ nông. Điều này hiếm và nó không đối bức tranh lớn, nhưng nó có nghĩa là một bộ giải lặp khởi đầu từ một điểm tôi có thể dừng ở chỗ không phải nghiệm nào trong hai nghiệm chính. Đây là lý do nên dùng một lời giải dạng đóng (IPPE, P3P) để khởi tạo thay vì khởi tạo tùy tiện — A/03 bàn kỹ.

Phép lật ở mục 2 là mô tả xấp xỉ. Quan hệ “ảnh gương qua đường nhìn” đúng chính xác trong giới hạn chiếu trục giao và là xấp xỉ tốt dưới chiếu phối cảnh với tag nhỏ so với khoảng cách. Với tag lớn chiếm phần lớn trường nhìn, quan hệ giữa hai nghiệm phức tạp hơn, và hai nghiệm cũng khác nhau ở cả vị trí tâm chứ không chỉ ở hướng. Trong bối cảnh docking thì xấp xỉ này ổn, vì tag hiếm khi chiếm quá một phần tư trường nhìn.

Không đồng phẳng khử ambiguity nhưng không khử được kém điều kiện. Một constellation “không đồng phẳng” với độ lệch khỏi mặt phẳng chỉ 2 mm về mặt kỹ thuật là không đồng phẳng, nhưng chênh lệch reprojection mà 2 mm ấy tạo ra có thể nhỏ hơn nhiễu, và khi đó bạn vẫn có hai cực tiểu gần như bằng nhau trong thực tế. Ranh giới giữa “đủ không đồng phẳng” và “không đủ” là một câu hỏi định lượng phụ thuộc σ , w và Z ; tôi không có công thức cho nó và tôi khuyên đo bằng mô phỏng — đây là một mục trong 99-chua-biet.md.

4.8 Thực hành

Bốn việc cụ thể, theo thứ tự nên làm.

Một, luôn tính cả hai nghiệm và luôn ghi ρ vào log. Dùng `solvePnPGeneric` với `SOLVEPNP_IPPE_SQUARE` cho tag vuông, hoặc đọc trực tiếp `apriltag_pose.c` nếu dùng thư viện gốc — nó đã tính sẵn cả hai. Chi phí gần như bằng không và giá trị chẩn đoán rất cao.

Hai, từ chối cập nhật khi ρ dưới ngưỡng thay vì trả về một nghiệm không đáng tin. Một hệ thống nói “tôi không biết” hữu ích hơn nhiều so với một hệ thống đoán và không báo. B/10 bàn chiến lược cụ thể.

Ba, đừng lọc thông thấp trên góc cho tới khi đã khử ambiguity. Nếu buộc phải làm mượt, hãy làm mượt sau khi đã chọn nghiệm một cách tin cậy, và hãy làm mượt trên đại lượng có ý nghĩa (B/11 về trung bình trên đa tạp), không phải trên góc Euler.

Bốn, và quan trọng nhất, hãy đưa yêu cầu “không đồng phẳng” vào bản vẽ cơ khí ngay từ đầu. Thêm một nệm nghiêng vào bản vẽ trước khi gia công tốn vài phút; thêm nó sau khi đã gia công một trăm cái dock thì tốn một trăm lần chi phí ấy.

Active recall (che cột phải, tự trả lời, rồi mở ra đối chiếu)

Thuật ngữ / câu hỏi	Trả lời
••• Vì sao target phẳng có đúng hai nghiệm	Vì phép lật target quanh trục trong mặt phẳng (phản xạ pháp tuyến qua đường nhìn) bảo toàn hình chiếu tới xấp xỉ bậc nhất. Đã được ba nhóm độc lập xác nhận [17], [28], [29].
••• Quan hệ giữa hai nghiệm	Ảnh gương qua đường nhìn; cách nhau 2α về góc với α là góc nghiêng thật; cùng vị trí tâm tới xấp xỉ bậc nhất.
••• Triệu chứng đặc trưng	Góc nhảy giữa hai giá trị, vị trí đứng yên. Nếu thấy cả sáu tham số cùng nhảy thì đó là nhiễu chứ không phải ambiguity.
••• Vì sao lọc thông thấp là sai	Trung bình hai nghiệm cách nhau 2α luôn ra $\alpha = 0$, tức fronto-parallel. Bộ lọc biến một đại lượng nhảy-nhưng-đôi-khi-đúng thành mượt-và-sai-có-hệ-thống, đồng thời xoá mất triệu chứng.
••• Thêm điểm đồng phẳng cứu được không?	Không. Phép lật tác động lên cả mặt phẳng nên mọi điểm đồng phẳng đều bị lật cùng và đều vẫn khớp. Nhiều điểm làm mỗi nghiệm sắc nét hơn nhưng ρ gần như không đổi.
••• Điều kiện khử triệt để	Tập điểm không đồng phẳng $\Rightarrow$ nghiệm duy nhất. Một điểm ngoài mặt phẳng là đủ về cấu trúc; thực hành cần độ lệch đủ lớn để chênh lệch reprojection vượt nhiễu.
••• Bốn yếu tố làm ambiguity nặng	Góc nghiêng nhỏ (mạnh nhất); tag nhỏ trên ảnh; trường nhìn hẹp (tele, tiến tới chiều trục giao); tag nằm ở tâm ảnh.
••• Tỉ số ambiguity	$\rho = e_2/e_1 \geq 1$, từ <code>solvePnPGeneric</code> hoặc IPPE. ρ gần 1 nghĩa là không phân biệt được và việc chọn nghiệm nhỏ hơn là chọn theo nhiễu.
•• Ngưỡng ρ nên đặt ở đâu	Không có hằng số phổ quát. Phải dẫn ra cho hệ của mình bằng Monte Carlo: vẽ tỉ lệ chọn sai theo ρ , lấy ngưỡng ở mức sai chấp nhận được. Phụ thuộc σ , w , số điểm.
•• Vì sao ống kính tele làm nặng thêm	Tele tiến gần tới chiều trục giao, mà dưới chiều trục giao hai nghiệm trùng khít hoàn toàn. Đánh đổi đáng biết: tele cho f lớn nên δx nhỏ, nhưng ambiguity nặng hơn.
•• Chế độ hồng của “chọn theo khung trước”	Một khi chọn nhầm, tiêu chí ấy giữ bạn ở nghiệm sai vĩnh viễn. Hệ thống rất ổn định quanh một giá trị sai và không có tín hiệu báo động. Phải kết hợp với ρ .
•• Quan hệ với chương 5	Cùng một sự thật ở hai mức: chương 5 nói hiệp phương sai theo hướng nghiêng lớn, chương này nói có hai cực tiểu gần nhau. Cả hai đều là hệ quả của $\sin \alpha \rightarrow 0$.
• Ba cách phá đồng phẳng	Nệm nghiêng dưới mỗi tag (rẻ nhất, giải quyết luôn bài toán $\sin \alpha$); tag ở các độ sâu khác nhau (cho baseline theo chiều sâu); target dạng góc nhiều mặt phẳng (mạnh nhất, dễ bị che khuất).

● ● ● phải thuộc ● ● nên nắm ● biết là có

I Kiến thức này đang sống ở đâu (công trình đang dùng chính ý tưởng của chương; chốt 9/2026)

Công trình / hệ thống	Dùng ý tưởng nào của chương	Ghi chú
apriltag <code>apriltag_pose.c</code>	Cài IPPE, tính cả hai nghiệm và sai số tái chiếu của từng nghiệm; hàm <code>estimate_tag_pose</code> trả về tỉ số	Chỗ tốt nhất để đọc chương này thành code; vài trăm dòng
OpenCV <code>SOLVEPNP_IPPE_SQUARE</code>	Cài [17] cho trường hợp riêng tag vuông; dùng với <code>solvePnPGeneric</code> để lấy cả hai nghiệm	API duy nhất cho bạn ρ trực tiếp
S _{Tag} [30]	Thêm một đường tròn ngoại tiếp vào thiết kế tag để ổn định pose — tức thêm thông tin hình học thay vì đổi thuật toán	Cùng triết lý với chương này, giải quyết ở tầng thiết kế marker
ChArUco board	Nhiều góc chessboard nhưng <i>vẫn đồng phẳng</i> — cải thiện σ , không cải thiện ρ	Minh hoạ trực tiếp cho mục 3; đừng nhầm hai loại cải thiện
tagslam [31]	Map nhiều tag ở các mặt phẳng khác nhau trong môi trường, rồi giải hợp nhất	Không đồng phẳng ở quy mô phòng; mức tin C

MINI PROJECT · Nhìn tận mắt hai nghiệm, và đo ngưỡng ρ cho hệ của mình · 1–2 ngày

Mục tiêu	chuyển ambiguity từ một khái niệm thành một đại lượng bạn đo được, và có một ngưỡng ρ có cơ sở thay vì một con số chép từ internet.
Dữ liệu	mô phỏng hoàn toàn, nối tiếp script của A/05 . Không cần phần cứng.
Các bước	sinh một tag vuông ở góc nghiêng α đã biết; chiếu, thêm nhiễu σ ; gọi <code>solvePnPGeneric</code> với <code>SOLVEPNP_IPPE_SQUARE</code> và lấy <i>cả hai</i> nghiệm cùng sai số tái chiếu; ghi lại ρ , và ghi lại nghiệm nào gần chân trị hơn; lặp 2000 lần cho mỗi α , quét α từ 0° tới 45° ; vẽ hai đường: ρ trung vị theo α , và tỉ lệ chọn sai theo α ; cuối cùng vẽ tỉ lệ chọn sai theo ρ (gộp mọi α) — đó chính là đường cong cho bạn ngưỡng.
Xong khi	bạn có một con số ngưỡng ρ^* cho hệ của mình kèm câu phát biểu dạng “với $\rho > \rho^*$, tỉ lệ chọn nhầm nghiệm dưới 1%”. Kiểm tra chéo bắt buộc: lặp lại toàn bộ với constellation không đồng phẳng (thêm một tag nghiêng 20°) và xác nhận rằng ρ bây giờ lớn ở <i>mọi</i> α — nếu không thì độ lệch khỏi mặt phẳng của bạn chưa đủ, và bạn vừa đo được nó cần bao nhiêu.
Nối với	B/10 dùng trực tiếp ngưỡng này; B/09 dùng kết quả kiểm chéo để chọn góc nghiêng cho constellation.

TRẢ LỜI CÂU HỎI MỞ ĐẦU

- Đó là pose ambiguity: bài toán PnP với target phẳng có đúng hai nghiệm cục bộ, cả hai giải thích ảnh gần như tốt ngang nhau, và nhiều quyết định nghiệm nào có sai số tái chiếu nhỏ hơn ở từng khung. Lấy trung bình là việc tệ nhất vì hai nghiệm là ảnh gương của nhau qua đường nhìn, nên trung bình của chúng luôn rơi đúng vào tư thế fronto-parallel — một giá trị không phải nghiệm nào cả và sai một cách có hệ thống. Tệ hơn nữa, nó xoá mất triệu chứng nhảy, nên bạn mất luôn dấu hiệu duy nhất cho biết mình đang có vấn đề.
- Gần như không, và đây là điểm hay bị hiểu nhầm nhất. Phép lật sinh ra nghiệm thứ hai tác động lên *cả mặt phẳng*, nên mọi điểm nằm trên mặt phẳng ấy đều bị lật cùng nhau và đều vẫn khớp với ảnh. Mười sáu góc đồng phẳng cho hai nghiệm sắc nét hơn bốn góc đồng phẳng, nhưng *tỉ số ρ* giữa hai sai số tái chiếu — đại lượng quyết định việc phân biệt — gần như không đổi. Bạn cần tính không đồng phẳng, không phải số lượng: hai tag trên hai mặt phẳng nghiêng nhau thắng bốn tag trên một mặt phẳng.
- Nghiệm thứ hai là ảnh gương của nghiệm thật qua đường nhìn tới tâm tag: nếu pháp tuyến thật nghiêng α về một phía thì pháp tuyến nghiệm hai nghiêng α về phía kia, nên hai nghiệm cách nhau 2α về góc và có cùng vị trí tâm tới xấp xỉ bậc nhất [29, §III]. Biết quan hệ này giúp

ba việc: nhận diện triệu chứng (góc nhảy, vị trí yên — khác hẳn với nhiễu chung làm cả sáu tham số nhảy); dự đoán biên độ nhảy (bằng hai lần góc nghiêng, nên biên độ nhảy nhỏ lại là dấu hiệu *xấu* vì nó nghĩa là bạn đang ở gần fronto-parallel); và hiểu vì sao trung bình luôn ra fronto-parallel.

- A4. Khi $\rho \approx 1$, tức khi hai sai số tái chiếu gần bằng nhau. Lúc ấy cái quyết định nghiệm nào nhỏ hơn là nhiễu chứ không phải hình học, nên chọn theo ϵ nhỏ hơn về bản chất là tung đồng xu. Nó tệ hơn tung đồng xu ở một điểm quan trọng: tung đồng xu không giả vờ là một quyết định có cơ sở, còn “chọn nghiệm có residual nhỏ hơn” thì nghe rất khoa học và không ai nghĩ tới việc kiểm lại. Cách sửa không phải đổi tiêu chí chọn mà là *đo ρ và từ chối trả lời khi nó quá thấp*.
- A5. Phá tính đồng phẳng của tập điểm — nghiêng các tag, đặt chúng ở các độ sâu khác nhau, hoặc dùng target dạng góc. Nó triệt để vì nó phá chính phép đối xứng sinh ra nghiệm thứ hai: điểm nằm ngoài mặt phẳng bị phép lật đưa tới vị trí khác hẳn, hình chiếu của nó lệch rõ rệt, nên nghiệm thứ hai không còn là cực tiểu nữa mà bị dữ liệu loại bỏ. Mọi cách khác — ngưỡng ρ , nhất quán thời gian, fuse IMU — chỉ giúp *chọn đúng* trong hai nghiệm vẫn đang tồn tại, nên chúng luôn có một xác suất sai khác không. Và điểm đáng nói về kinh tế: cách triệt để này được thi công ở xương cơ khí với chi phí gần như bằng không, trong khi các cách giảm thiểu tiêu tốn công sức phần mềm vô hạn.

NÓI LẠI THẬT ĐƠN GIẢN

Có một trò ảo giác thị giác quen thuộc: hình vẽ một cái hộp dây thép, và nhìn một lúc thì nó tự lật — lúc thì mặt này ở trước, lúc thì mặt kia. Bạn không điều khiển được, và không có câu trả lời đúng, vì bản thân hình vẽ chứa đúng hai cách hiểu và cả hai đều khớp hoàn hảo với những gì trên giấy. Một cái tag phẳng nhìn qua một camera rơi đúng vào tình huống ấy. Từ hình dạng méo của cái tag trên ảnh, có đúng hai cách nghiêng nó trong không gian để cho ra hình dạng đó — nghiêng về phía này, hoặc nghiêng về phía kia. Máy tính cũng không có câu trả lời đúng, và nó cũng lật qua lật lại, chỉ có điều nó lật mấy chục lần mỗi giây thay vì mấy giây một lần.

Điều tệ nhất bạn có thể làm là lấy trung bình hai cách hiểu. Trung bình của “nghiêng sang trái” và “nghiêng sang phải” là “không nghiêng gì cả” — một câu trả lời trơn tru, ổn định, và luôn luôn sai. Nhưng nó lại là điều đầu tiên ai cũng làm, vì con số ngưỡng nhảy và mọi thứ trông đã ổn.

Có hai cách xử lý đúng, và chúng rất khác nhau về bản chất. Cách thứ nhất là dạy máy tính nói “tôi không chắc” — đo xem hai cách hiểu khớp với ảnh tốt ngang nhau tới mức nào, và khi chúng ngang nhau thì im lặng thay vì đoán. Cách này luôn dùng được nhưng nó không giải quyết gì, nó chỉ trung thực.

Cách thứ hai là làm cho câu hỏi hết mơ hồ. Với cái hộp dây thép, chỉ cần vẽ thêm một chỗ mà một cạnh che khuất cạnh kia — chỉ một chỗ thôi — và ảo giác biến mất vĩnh viễn, vì bây giờ chỉ còn một cách hiểu khớp. Với cái tag, việc tương đương là làm cho nó đừng phẳng nữa: kẻ nghiêng vài cái tag, hoặc đặt một cái nhô ra trước những cái kia. Và đây là điều đáng nhớ nhất của cả chương: việc ấy được làm bằng một cái nẹp nhôm và bốn con vít, không phải bằng một thuật toán.

Chỗ không rút gọn được: ranh giới giữa “đủ không đồng phẳng” và “chưa đủ” thì không có công thức. Nó phụ thuộc vào nhiễu của hệ bạn, vào kích thước tag trên ảnh, vào khoảng cách — nên nó phải đo, không suy ra được. May là đo bằng mô phỏng rẻ và làm được trong một buổi.

Một câu: *Cái tag phẳng của bạn luôn có hai câu trả lời; chọn đúng là một trò may rủi, còn đặt nó nghiêng đi thì chỉ còn một câu.*

4.9 Điều gì chứng minh cách hiểu này sai

Mệnh đề chính — *target phẳng có đúng hai nghiệm cục bộ, và tập điểm không đồng phẳng có nghiệm duy nhất* — là mệnh đề tôi tin nhất trong cả cây, vì nó đã qua cửa (c) với ba nguồn độc lập cách nhau ba thập niên [17], [28], [29], và cửa (a) bằng lập luận đối xứng. Nó bị bác bỏ nếu ai đó trình bày một cấu hình target phẳng không suy biến mà bài toán PnP chỉ có một cực tiểu, hoặc

ngược lại một tập điểm không đồng phẳng đủ tổng quát mà vẫn có hai nghiệm phân biệt được với sai số tái chiếu bằng nhau.

Mệnh đề thứ hai — *trung bình hai nghiệm luôn rơi vào fronto-parallel* — bị bác bỏ nếu mô phỏng cho thấy trung bình của hai nghiệm IPPE trên một chuỗi khung hình hội tụ về một giá trị khác $\alpha = 0$ một cách có hệ thống. Đây là mệnh đề tôi dẫn từ hình học của mục 2 (cửa a) và nó *chưa* qua cửa (b) — một chỗ nọ kiểm chứng, tuy nhỏ vì nó là hệ quả trực tiếp của quan hệ gương.

Mệnh đề thứ ba — *thêm điểm đồng phẳng không cải thiện ρ* — là mệnh đề dễ kiểm nhất và cũng là mệnh đề có giá trị thực hành cao nhất, vì nếu nó sai thì lời khuyên trung tâm của chương (phá đồng phẳng thay vì thêm tag) sẽ sai theo. Nó bị bác bỏ nếu mô phỏng cho thấy ρ tăng đáng kể khi đi từ 4 lên 64 góc đồng phẳng ở cùng một góc nghiêng. Mini project ở trên chính là phép kiểm ấy, và nó nên được chạy sớm.

Cuối cùng, một mệnh đề mềm hơn: chương này khẳng định rằng *ambiguity là nguồn lỗi xoay lớn hơn nhiều trong bài toán docking điển hình*. Điều đó bị bác bỏ nếu trong một hệ thực, sau khi đã khử hoàn toàn ambiguity bằng constellation không đồng phẳng, sai số xoay vẫn không cải thiện đáng kể — khi ấy nút thắt nằm ở chỗ khác (nhiều khả năng là sai số cơ khí của constellation, B/12). (Tôi phát biểu mệnh đề này dựa trên cấu trúc của vấn đề chứ không dựa trên thống kê ca hòng thực tế; tôi không có số liệu về tỉ lệ.)

4.10 Kết nối

Chương này là chương bệnh lý của Phần A, và nó nối vào cây ở nhiều chỗ hơn độ dài của nó gợi ý.

Nối lên trên. [A/02-homography-va-hinh-hoc-phang](#) là nơi hai nghiệm xuất hiện lần đầu dưới dạng đại số: phép phân rã homography thành pose vốn đã cho nhiều nghiệm, và chương này giải thích ý nghĩa hình học của chúng. [A/03-pnp-va-uoc-luong-tu-the](#) cho IPPE và sai số tái chiếu, tức cả hai công cụ mà chương này dùng để đo ambiguity; nó cũng là nơi giải thích vì sao phải khởi tạo bằng lời giải dạng đóng thay vì tùy tiện.

Nối ngang, và đây là liên kết quan trọng nhất. [A/05-lan-truyen-sai-so-pixel-sang-pose](#) là chương song sinh: cùng một sự kiện hình học — độ nhạy tỉ lệ với $\sin \alpha$ — được nhìn từ hai phía. Chương 5 nói “ma trận thông tin gần suy biến theo hướng nghiêng, nên hiệp phương sai lớn”; chương này nói “có hai cực tiểu gần nhau, nên bộ giải nhảy giữa chúng”. Cả hai đều đúng và chúng mô tả cùng một bệnh. Một phân tích chỉ dùng chương 5 sẽ kết luận rằng sai số xoay là nhiều và có thể lấy trung bình — một kết luận sai và tốn kém. Đó là lý do hai chương phải đọc cùng nhau.

Nối xuống dưới. [B/09-multi-marker-constellation](#) là chương biến mục 6 thành thiết kế cụ thể: nghiêng bao nhiêu độ, đặt tag ở đâu, độ lệch khỏi mặt phẳng cần bao nhiêu. [B/10-xu-ly-ambiguity](#) nhận toàn bộ mục 5 và mục 8: nó là chương về những gì phải làm ở tầng code khi bạn *chưa* khử được ambiguity bằng hình học — ngưỡng, từ chối cập nhật, nhất quán thời gian, fuse IMU. [B/11-uoc-luong-theo-thoi-gian](#) nhận cảnh báo ở mục 5: mọi kỹ thuật làm mượt trong chương ấy chỉ hợp lệ *sau khi* nghiệm đã được chọn tin cậy, và chương này là lý do của điều kiện ấy.

Cuối cùng, một nối sang Phần C. [C/15-ngan-sach-sai-so](#) phải xử lý ambiguity theo một cách khác với mọi dòng khác trong ngân sách: nó không phải một nguồn sai số cộng vào mà là một *lỗi thô* phá hỏng toàn bộ ngân sách khi nó xảy ra. Ngân sách sai số giả định bạn đang ở gần nghiệm đúng; nếu bạn đang ở nghiệm sai thì mọi con số milimét trong bảng đều vô nghĩa. Đó là lý do việc khử ambiguity phải là điều kiện tiên quyết chứ không phải một dòng trong bảng.

Câu hỏi tự kiểm (trả lời không nhìn bài)

1. Vì sao thêm mười hai góc đồng phẳng không cải thiện tỉ số ρ trong khi nó rõ ràng cải thiện σ — hãy trả lời bằng cách chỉ ra phép biến đổi nào đang được bảo toàn?
2. Một đồng nghiệp thấy pose nhảy và đề xuất dùng bộ lọc Kalman. Nêu chính xác chuyện gì sẽ xảy ra với đầu ra, và vì sao kết quả trông tốt hơn nhưng thực chất tệ hơn.
3. Hai nghiệm cách nhau 2α . Vì sao biên độ nhảy *nhỏ* lại là dấu hiệu xấu chứ không phải dấu hiệu tốt?
4. Bạn dùng ống kính tele để có f lớn, theo lời khuyên của chương 5. Điều đó ảnh hưởng thế nào tới ambiguity, và bạn cân bằng hai yêu cầu ngược nhau ấy bằng cách nào?
5. Bạn đã phá tính đồng phẳng bằng cách đặt một tag nhô ra trước 3 mm so với các tag khác. Vì sao điều đó có thể vẫn không đủ, và bạn kiểm bằng cách nào mà không cần dựng phần cứng?
6. Tiêu chí “chọn nghiệm gần pose khung trước” có một chế độ hỏng rất đặc trưng. Mô tả nó, và nêu một cơ chế phát hiện được nó từ trong dữ liệu đo.
7. Vì sao ambiguity không thể được coi là một dòng trong bảng ngân sách sai số, mà phải được coi là điều kiện tiên quyết?

Lan truyền sai số từ pixel sang pose (Error Propagation)

PHỤ THUỘC A/01-mo-hinh-camera-va-distortion (ma trận K , và vì sao tiêu cự f tính bằng pixel là đại lượng duy nhất của camera xuất hiện trong mọi công thức dưới đây); A/03-pnp-va-uoc-luong-tu-the (PnP, sai số tái chiếu, Jacobian của phép chiếu — chương này lấy đúng Jacobian ấy rồi đảo nó); A/04-pose-ambiguity-target-phang (vì sao hướng xoay là hướng yếu — chương này định lượng cái yếu đó).

TÓM TẮT

Chương này biến một câu hỏi mơ hồ — “hệ của tôi chính xác cỡ nào” — thành bốn phép nhân chia làm được trên giấy trong ba mươi giây. Xuất phát điểm là một công thức duy nhất, $\Sigma_{\text{pose}} \approx (J^T \Sigma_{\text{px}}^{-1} J)^{-1}$, và toàn bộ phần còn lại của chương là việc đọc ý nghĩa vật lý của ma trận ấy trong bốn trường hợp cụ thể. Bốn kết quả: sai số ngang $\delta x \approx Z\sigma/f$, sai số dọc trục $\delta Z \approx Z\sigma/w$, độ nhạy xoay của một tag phẳng tỉ lệ với sin của góc nghiêng, và sai số xoay từ một constellation có baseline B là $\delta\theta \approx \delta x/B$. Bốn quan hệ ấy ngắn đủ để thuộc lòng và chúng là công cụ chẩn đoán hằng ngày. Nếu chỉ nhớ một điều: *sai số ngang chia cho f , sai số sâu chia cho w , và $w \ll f$ — nên chiều sâu luôn tệ hơn chiều ngang chừng f/w lần; còn xoay thì không nằm trong danh sách ấy vì nó tuân theo một quy luật khác hẳn, và đó là lý do nó là nút thắt.*

KIẾN THỨC CẦN CÓ TRƯỚC

- **tiêu cự tính bằng pixel (f)** — phần tử f_x hoặc f_y của ma trận K . Nó không phải chiều dài vật lý của ống kính mà là tỉ số giữa tiêu cự vật lý và kích thước một pixel trên cảm biến. Với một camera 1280×960 và trường nhìn ngang khoảng 60°, f cỡ 1100 px. Đây là đại lượng duy nhất của camera xuất hiện trong công thức sai số ngang.
- **kích thước tag trên ảnh (w)** — cạnh của tag đo bằng pixel trên ảnh, $w \approx fW/Z$ với W là cạnh thật của tag và Z là khoảng cách. Nó là đại lượng quyết định sai số theo chiều sâu, và nó nhỏ hơn f rất nhiều trong mọi tình huống thực tế.
- **nhieu định vị góc (σ)** — độ lệch chuẩn của vị trí một góc marker trên ảnh, tính bằng pixel. Đây là đầu vào duy nhất mang tính “chất lượng thuật toán” trong cả chương; mọi thứ khác là hình học.
- **Jacobian của sai số tái chiếu (J)** — ma trận đạo hàm riêng của tọa độ ảnh dự đoán theo sáu tham số pose, $J = \partial u / \partial \xi$. Mỗi góc marker đóng góp hai hàng (một cho u , một cho v).
- **ma trận thông tin Fisher** — $\mathcal{I} = J^T \Sigma_{\text{px}}^{-1} J$. Nó đo lượng thông tin mà tập phép đo mang về pose; nghịch đảo của nó là cận dưới cho hiệp phương sai của mọi bộ ước lượng không chệch [32].
- **baseline của constellation (B)** — khoảng cách giữa hai marker xa nhau nhất trong tập marker, đo trong không gian 3D. Đây là đại lượng hình học có đòn bẩy lớn nhất trong toàn cây, và nó không phụ thuộc vào bất kỳ lựa chọn phần mềm nào.
- **fronto-parallel** — tư thế mà mặt phẳng tag song song với mặt phẳng ảnh, tức camera nhìn thẳng vuông góc vào tag. Đây là tư thế trực quan nhất và là tư thế *tệ nhất* cho việc ước lượng góc nghiêng — một nghịch lý mà mục 5 giải thích.

CÂU HỎI MỞ ĐẦU

- Q1. Bạn tăng độ phân giải camera lên gấp đôi, giữ nguyên ống kính và mọi thứ khác. Sai số vị trí của bạn cải thiện bao nhiêu lần, và vì sao câu trả lời không phải “gấp đôi”?
- Q2. Một tag 100 mm ở cách camera nửa mét cho sai số dọc trục khoảng 0,8 mm. Nếu bạn dùng tag 200 mm thay vào, sai số đó thành bao nhiêu — và quan trọng hơn, nếu bạn thay bằng hai tag 100 mm đặt cách nhau 200 mm thì được gì mà tag lớn không cho?
- Q3. Vì sao sai số theo chiều sâu luôn tệ hơn sai số theo chiều ngang, và tỉ số giữa chúng bằng đại lượng nào — hãy trả lời bằng một phân số của hai đại lượng đã có tên trong chương này.
- Q4. Robot của bạn tiếp cận dock thẳng trục, nên camera nhìn gần vuông góc vào tấm marker. Đó là tư thế trực quan nhất và là tư thế *tệ nhất* cho việc đo góc. Cơ chế hình học nào tạo ra nghịch lý đó?
- Q5. Ngân sách của bạn cho phép 0,5°. Có hai cách tiêu tiền: mua camera tốt hơn để giảm σ từ 0,2 px xuống 0,05 px, hoặc làm tấm marker rộng thêm 30 cm. Cách nào cho nhiều độ hơn trên mỗi đồng, và vì sao câu trả lời gần như không phụ thuộc vào con số cụ thể?

5.1 Đặt vấn đề

Có một câu hỏi mà mọi người làm marker đều phải trả lời và gần như không ai trả lời được trong lần đầu được hỏi: *hệ của bạn chính xác tới đâu?*

Câu trả lời thường gặp là một con số thực nghiệm — “tôi đo được khoảng 3 mm” — và nó có hai vấn đề. Vấn đề thứ nhất là nó chỉ đúng cho đúng cấu hình đã đo: đổi khoảng cách, đổi kích thước tag, đổi ống kính, và con số ấy không nói gì nữa. Vấn đề thứ hai nghiêm trọng hơn: nó không cho biết *nên sửa gì*. Một con số ở đầu ra là tổng hợp của nhiều nguyên nhân, và nếu không tách được các thành phần thì mọi nỗ lực cải thiện đều là thử và sai.

Bài toán gốc ở đây không mới và cũng không riêng của thị giác. Nó là bài toán *lan truyền bất định* mà ngành đo lường đã hình thức hoá từ lâu [1]: có một đại lượng đo được trực tiếp với bất định đã biết, có một mô hình biến nó thành đại lượng ta quan tâm, và câu hỏi là bất định của đại lượng đích bằng bao nhiêu. Trong trường hợp của ta, đại lượng đo trực tiếp là vị trí các góc marker trên ảnh với bất định σ pixel, mô hình là phép chiếu phối cảnh, và đại lượng đích là sáu tham số pose.

Điều làm bài toán này đáng dành một chương riêng không phải độ khó toán học — công thức lan truyền là kiến thức chuẩn [33], [34] — mà là chuyện sau đây. Khi ta viết công thức lan truyền ra và đọc nó theo từng thành phần, ta phát hiện rằng *sáu tham số pose không được sinh ra bình đẳng*. Hai tham số tịnh tiến ngang có bất định nhỏ, tham số tịnh tiến dọc trục có bất định lớn hơn chừng f/w lần, và các tham số xoay — ít nhất là hai trong ba — tuân theo một quy luật hoàn toàn khác, một quy luật trong đó *góc nhìn* xuất hiện như một hệ số nhân và có thể tiến về không. Sự bất bình đẳng ấy là toàn bộ nội dung thực dụng của chương.

Cách hiển nhiên hơn — và cách gần như ai cũng thử trước — là đo. Dựng hệ, chạy, so với thước, ra một con số. Cách này không sai và C/16 dành hẳn cho nó. Nhưng nó có một nhược điểm chí mạng ở giai đoạn thiết kế: nó đòi hệ phải tồn tại. Khi bạn đang phải quyết định đặt hai tag hay bốn tag, cách nhau 30 cm hay 50 cm, có nghiêng hay không, thì không có gì để đo cả. Quyết định ấy phải được đưa ra *trước*, và nó tốn tiền gia công nếu sai. Chương này tồn tại để những quyết định đó được đưa ra trên giấy.

Có một cách hiển nhiên thứ hai, tinh vi hơn và cũng phổ biến hơn ở người có kinh nghiệm: chạy Monte Carlo. Thêm nhiễu vào góc, chạy PnP, xem phân bố. Cách này *tốt* và C/16 khuyến khích nó mạnh mẽ. Nhưng nó vẫn không thay thế được chương này, vì một lý do về bản chất của hiểu biết: Monte Carlo cho bạn biết *kết quả là bao nhiêu* chứ không cho biết *vì sao*. Nó không nói cho bạn rằng sai số sâu tệ hơn sai số ngang đúng f/w lần, nên nó không gợi ý rằng tăng w là đòn bẩy. Công thức giải tích và mô phỏng số phục vụ hai chức năng khác nhau, và cách dùng đúng là

dùng cả hai: công thức để hiểu và để chọn hướng, mô phỏng để kiểm và để bắt những chỗ công thức bỏ sót.

TRỰC GIÁC

Hình dung việc ước lượng pose như việc định vị bằng cách nhìn một vật đã biết kích thước. Để biết vật đó nằm *lệch trái phải* bao nhiêu, bạn nhìn xem nó nằm ở đâu trên khung nhìn. Một sai lệch nhỏ trên khung nhìn quy ra một sai lệch nhỏ trong không gian, và hệ số quy đổi là Z/f — càng xa thì mỗi pixel càng “đáng giá” nhiều milimét. Đây là phép đo trực tiếp và nó tốt.

Để biết vật đó ở *xa gần* bao nhiêu, bạn không có cách nhìn trực tiếp. Bạn phải suy từ việc nó *trông to cỡ nào*. Và ở đây có một sự khuếch đại: để biết khoảng cách chính xác 1%, bạn phải đo được kích thước biểu kiến chính xác 1%; nhưng kích thước biểu kiến chỉ có w pixel, nên 1% của nó là $w/100$ pixel — một đại lượng nhỏ hơn nhiều so với $f/100$. Đây là lý do chiều sâu luôn tệ hơn, và tỉ số tệ hơn đúng bằng f/w .

Để biết vật đó *ngiênêng* bao nhiêu, bạn phải nhìn vào sự *biến dạng phối cảnh*: cạnh xa trông ngắn hơn cạnh gần. Nhưng nếu vật đang nhìn thẳng vào bạn thì không có cạnh nào xa hơn cạnh nào, biến dạng bằng không, và một chút nghiêng gây ra một chút biến dạng *bậc hai* chứ không phải bậc một. Đó chính là lý do gần vuông góc là tệ nhất: đạo hàm của thứ bạn quan sát theo thứ bạn muốn biết đi qua không.

Và đây là lối thoát: nếu thay vì một vật, bạn nhìn *hai* vật cách nhau một khoảng B đã biết, thì góc quay của cặp ấy không còn phải suy từ biến dạng phối cảnh nữa. Nó suy từ chênh lệch vị trí ngang giữa hai vật — một phép đo thuộc loại *tốt*. Đó là toàn bộ lý do constellation tồn tại.

5.2 Công thức gốc: nghịch đảo của ma trận thông tin

Mọi thứ trong chương này chảy ra từ một công thức, nên đáng dẫn nó một lần cho cẩn thận.

Bài toán PnP tinh chỉnh (A/03) cực tiểu hoá tổng bình phương sai số tái chiếu có trọng số:

$$C(\xi) = \frac{1}{2} \sum_{i=1}^N (u_i - \hat{u}_i(\xi))^T \Sigma_i^{-1} (u_i - \hat{u}_i(\xi)),$$

với $\xi \in \mathbb{R}^6$ là tham số hoá pose (ba cho tịnh tiến, ba cho xoay — xem A/03 về việc dùng $\mathfrak{se}(3)$ để tránh kỳ dị), u_i là góc đo được, $\hat{u}_i(\xi)$ là góc dự đoán, và Σ_i là hiệp phương sai nhiễu của góc thứ i .

Khai triển quanh nghiệm $\hat{\xi}$ tới bậc hai và bỏ số hạng chứa phần dư nhân đạo hàm bậc hai — hợp lệ khi phần dư nhỏ, tức khi mô hình đúng và nhiễu nhỏ, và đây chính là giả thiết Gauss–Newton:

$$C(\hat{\xi} + \delta) \approx C(\hat{\xi}) + \frac{1}{2} \delta^T \mathcal{I} \delta, \quad \mathcal{I} = \sum_{i=1}^N J_i^T \Sigma_i^{-1} J_i, \quad J_i = \left. \frac{\partial \hat{u}_i}{\partial \xi} \right|_{\hat{\xi}}.$$

Vì sao bước bỏ số hạng kia hợp lệ? Vì đạo hàm bậc hai của C có dạng $\sum J^T \Sigma^{-1} J - \sum r^T \Sigma^{-1} \partial^2 \hat{u} / \partial \xi^2$, và số hạng thứ hai tỉ lệ với phần dư r . Khi mô hình đúng, phần dư là nhiễu có trung bình không, nên số hạng ấy triệt tiêu theo kỳ vọng. Khi mô hình *sai* — méo ảnh chưa khử sạch, constellation sai so với CAD — phần dư có thành phần hệ thống và số hạng ấy không triệt tiêu. Đây là một ghi chú không hàn lâm chút nào: nó có nghĩa là *công thức hiệp phương sai dưới đây tự nó giả định mô hình đúng*, và nó sẽ báo một hiệp phương sai đẹp ngay cả khi bạn đang đo quanh một giá trị sai. C/16 quay lại chuyện này bằng phép thử tính nhất quán.

Với giả thiết ấy, hiệp phương sai của pose ước lượng là

$$\Sigma_{\text{pose}} \approx \mathcal{I}^{-1} = \left(\sum_i J_i^\top \Sigma_i^{-1} J_i \right)^{-1}. \quad (5.1)$$

Đây là kết quả chuẩn của lý thuyết ước lượng — nó chính là cận Cramér–Rao mà bộ ước lượng hợp lý cực đại đạt được tiệm cận [32, chương 3], và trong ngữ cảnh thị giác nó được viết ra tường minh bởi [33]. Điều đáng nói là nó đúng cho *mọi* bộ ước lượng không chệch: không thuật toán nào làm tốt hơn được, nên nếu $\mathcal{I}^{-1}$ nói bạn không đạt 0,5° thì đổi thuật toán không cứu được.

Trường hợp riêng hay dùng: nếu mọi góc có cùng nhiễu đẳng hướng $\Sigma_i = \sigma^2 I_2$, thì

$$\Sigma_{\text{pose}} \approx \sigma^2 \left(\sum_i J_i^\top J_i \right)^{-1}.$$

Dạng này tách bạch hai thứ một cách rất sáng sủa, và sự tách bạch ấy là thông điệp trung tâm của chương: σ^2 là *chất lượng đo*, còn $(\sum J^\top J)^{-1}$ là *hình học*. Chất lượng đo là thứ bạn mua bằng detector tốt hơn, quang học tốt hơn, trung bình nhiều khung. Hình học là thứ bạn mua bằng cách đặt các tag ở đâu. Và vì hình học vào công thức dưới dạng một ma trận nghịch đảo còn σ chỉ là một hệ số nhân vô hướng, *hình học có thể làm sai số lớn lên vô hạn còn σ thì không* — một ma trận gần suy biến có nghịch đảo lớn tùy ý.

CẦN NHỚ

Công thức (5.1) chia bài toán làm hai nửa không cân xứng: σ^2 (chất lượng đo) là một *hệ số nhân*, còn hình học là một *ma trận nghịch đảo*. Cải thiện σ gấp bốn lần thì sai số giảm bốn lần, không hơn. Sửa một hình học suy biến thì sai số giảm từ vô hạn xuống hữu hạn. Hệ quả thực hành: khi sai số của bạn lớn một cách khó hiểu, câu hỏi đầu tiên không phải “detector của tôi có kém không” mà là “ma trận $\sum J^\top J$ của tôi có gần suy biến theo hướng nào không”.

5.3 Hai quan hệ tịnh tiến: chia cho f , chia cho w

Hai trường hợp đơn giản nhất, và chúng đơn giản đến mức dẫn được bằng vi phân một dòng mỗi cái.

A. Sai số ngang: chia cho tiêu cự

Phương trình chiếu cho tọa độ ngang là $u = f X / Z + c_x$. Giữ Z cố định và vi phân theo X :

$$\frac{\partial u}{\partial X} = \frac{f}{Z} \implies \delta X = \frac{Z}{f} \delta u.$$

Với $\delta u = \sigma$ là nhiễu định vị góc, ta được quan hệ thứ nhất:

$$\boxed{\delta x \approx \frac{Z \sigma}{f}} \quad (5.2)$$

Nếu có N góc và nhiễu của chúng độc lập, con số này còn chia thêm cho $\sqrt{N}$ — nhưng tôi cố ý không đưa $\sqrt{N}$ vào công thức đóng khung, vì trong thực tế nhiễu góc trên cùng một tag *không* độc lập (cùng chịu một điều kiện chiếu sáng, cùng một mức mờ, cùng một sai số méo cục bộ), nên $\sqrt{N}$ là một lời hứa thường không được giữ. Dùng công thức không có $\sqrt{N}$ là cách ước lượng thận trọng, và ngân sách sai số nên thận trọng.

Ba điều đáng đọc ra từ (5.2). Thứ nhất, sai số ngang *tuyến tính theo khoảng cách*: gần gấp đôi thì chính xác gấp đôi. Đây là lý do vì sao chiến lược coarse-to-fine ở B/13 hoạt động — bạn không cần chính xác ở xa, và khi tới gần thì chính xác tự đến. Thứ hai, đại lượng camera duy nhất xuất hiện là f tính bằng pixel. Thứ ba, và đây là điều quan trọng nhất: *không có kích thước tag trong công thức này*. Tag lớn hay nhỏ không ảnh hưởng tới sai số ngang, miễn là vẫn phát hiện được và σ không đổi. Đây là một sự thật phản trực giác mà tôi khuyên nên kiểm lại bằng Monte Carlo khi mới gặp, vì nó mâu thuẫn với cảm giác “tag lớn thì chính xác hơn”. Cảm giác ấy đúng — nhưng nó đúng cho chiều sâu và cho góc, không đúng cho chiều ngang.

B. Sai số dọc trục: chia cho kích thước tag trên ảnh

Đây là chỗ xuất hiện sự bất bình đẳng đầu tiên giữa các trục. Chiều sâu không được đo trực tiếp; nó được suy từ kích thước biểu kiến. Với một tag cạnh thật W ở khoảng cách Z , kích thước trên ảnh là $w = fW/Z$. Vi phân theo Z :

$$\frac{\partial w}{\partial Z} = -\frac{fW}{Z^2} = -\frac{w}{Z} \implies \delta Z = \frac{Z}{w} \delta w.$$

Nhiều của *kích thước biểu kiến* δw đến từ nhiều của hai đầu mút, nên nó cỡ σ (chính xác hơn là $\sqrt{2}\sigma$ nếu hai đầu độc lập; tôi giữ σ cho gọn và vì lý do thận trọng đã nói). Ta được quan hệ thứ hai:

$$\delta Z \approx \frac{Z \sigma}{w} = \frac{Z^2 \sigma}{f W} \quad (5.3)$$

Dạng thứ hai của công thức — viết theo W và Z thay vì w — đáng nhớ riêng, vì nó cho thấy sai số sâu là *bậc hai theo khoảng cách*. Xa gấp đôi thì tệ gấp bốn, không phải gấp đôi.

So sánh hai công thức cho ngay tỉ số:

$$\frac{\delta Z}{\delta x} \approx \frac{f}{w} = \frac{Z}{W}.$$

Đây là câu trả lời cho Q3, và nó có một cách phát biểu rất dễ nhớ: *sai số sâu tệ hơn sai số ngang đúng bằng tỉ số giữa khoảng cách và kích thước tag*. Tag 100 mm ở 500 mm thì sai số sâu tệ hơn 5 lần. Tag 100 mm ở 3 m thì tệ hơn 30 lần. Con số ấy giải thích vì sao mọi hệ thống marker đều thấy rằng thành phần khoảng cách là thành phần tệ nhất của tịnh tiến, và nó cũng giải thích vì sao đây là một trong số ít chỗ mà *làm tag to hơn thật sự có tác dụng*.

I 5.4 Ví dụ số chuẩn của cây

Đây là ví dụ chạy xuyên suốt cả cây, và nó là ví dụ nhỏ nhất tính tay được theo yêu cầu của learning-rules.md mục 4. Giả sử:

$$f = 600 \text{ px}, \quad Z = 0,5 \text{ m}, \quad W = 100 \text{ mm}, \quad \sigma = 0,2 \text{ px}.$$

Kích thước tag trên ảnh: $w = fW/Z = 600 \times 0,1/0,5 = 120 \text{ px}$.

Sai số ngang theo (5.2):

$$\delta x = \frac{0,5 \times 0,2}{600} = \frac{0,1}{600} = 1,67 \times 10^{-4} \text{ m} = \mathbf{0,17 \text{ mm}}.$$

Sai số dọc trục theo (5.3):

$$\delta Z = \frac{0,5 \times 0,2}{120} = \frac{0,1}{120} = 8,3 \times 10^{-4} \text{ m} = \mathbf{0,83 \text{ mm}}.$$

Kiểm chéo bằng tỉ số: $\delta Z/\delta x = 0,83/0,17 = 5$, và $Z/W = 500/100 = 5$. Khớp. Đây là cửa kiểm chứng (b) cho cả hai công thức: chúng nhất quán với nhau ở một quan hệ độc lập với cách dẫn ra chúng.

Đọc kết quả so với ngân sách 5 mm: cả hai con số đều *thoải mái*. Tình tiền không phải vấn đề, và nó không phải vấn đề với một biên độ lớn — ta còn dư gần một bậc độ lớn. Đây là kết luận quan trọng nhất của ví dụ, và nó nên làm người đọc ngạc nhiên, vì nó nói rằng phần lớn công sức mà các dự án bỏ vào việc cải thiện độ chính xác vị trí là công sức tiêu vào chỗ đã dư.

CẠM BẮY

Con số $\sigma = 0,2$ px trong ví dụ trên là **bậc độ lớn kinh nghiệm**, không phải số đo của một thí nghiệm có điều kiện xác định. Nó nằm trong khoảng mà người ta thường quan sát được với góc AprilTag trong điều kiện tốt, và B/08 bàn kỹ hơn về khoảng này cùng với các phương pháp tinh chỉnh khác.

Hệ quả bắt buộc: *mọi con số milimét sinh ra từ nó cũng chỉ là bậc độ lớn*. Chúng đủ tốt để so sánh các phương án thiết kế với nhau, đủ tốt để biết mình dư hay thiếu một bậc, và *không* đủ tốt để nghiệm thu. Trước khi nói “hệ của tôi đạt 5 mm”, bạn phải đo σ của chính hệ mình (C/16) và chạy lại phép tính với con số ấy.

Cách sai lầm phổ biến nhất là lấy con số 0,17 mm ở trên rồi báo cáo nó như độ chính xác của hệ thống. Nó không phải độ chính xác; nó là *thành phần đóng góp của nhiều góc vào độ lặp lại*, một dòng trong một bảng ngân sách còn có nhiều dòng khác — và các dòng khác thường lớn hơn nó nhiều lần.

5.5 Sai số xoay từ một tag phẳng: quy luật hoàn toàn khác

Hai mục trước có cùng cấu trúc: một phép vi phân, một hệ số quy đổi, một công thức sạch. Mục này không như thế, và chính sự khác biệt ấy là điều cần hiểu.

Xét một tag phẳng và hỏi: nghiêng nó đi một góc nhỏ $\delta\theta$ quanh một trục nằm *trong* mặt phẳng tag thì hình chiếu trên ảnh đổi bao nhiêu? Bản chất nằm ở một quan sát hình học đơn giản.

Nghiêng một tấm phẳng quanh trục nằm trong mặt phẳng của nó làm các điểm ở hai mép dịch chuyển theo chiều *sâu* theo hai hướng ngược nhau: mép này lùi ra, mép kia tiến vào, với biên độ $\pm(W/2)\sin\delta\theta$. Sự dịch chuyển theo chiều sâu ấy được camera nhìn thấy qua thay đổi của kích thước biểu kiến cục bộ — tức qua chính cơ chế *yếu* của mục 3B. Ngược lại, xoay tấm phẳng quanh trục *vuông góc* với nó (tức xoay trong mặt phẳng ảnh, quanh trục quang) làm mọi điểm dịch chuyển theo chiều *ngang* với biên độ $(W/2)\delta\theta$, và cái đó được nhìn thấy qua cơ chế *tốt* của mục 3A.

Đó là sự bất đối xứng cốt lõi, và nó cho ngay hai kết quả rất khác nhau. Với xoay trong mặt phẳng ảnh:

$$\delta\theta_{\text{trong mặt phẳng}} \approx \frac{\delta x}{W/2} = \frac{2Z\sigma}{fW} = \frac{2\sigma}{w},$$

tức nó tốt, và nó chỉ phụ thuộc vào kích thước tag trên ảnh. Với tag 120 px và $\sigma = 0,2$ px, con số này là $3,3 \times 10^{-3}$ rad = $0,19^\circ$ — vừa đủ trong ngân sách, và nó là thành phần xoay *đễ nhất*.

Với xoay ngoài mặt phẳng — tức nghiêng tag — thì phải qua cơ chế *sâu*, và ở đây xuất hiện hệ số sin:

$$\delta\theta_{\text{ngoài mặt phẳng}} \gtrsim \frac{\delta Z}{(W/2)} \sim \frac{2Z\sigma}{wW}, \quad \text{và độ nhạy} \propto \sin\theta \quad (5.4)$$

Tôi viết $\gtrsim$ và $\sim$ chứ không viết dấu bằng, vì con số chính xác phụ thuộc vào phân bố các góc và vào tham số hoá; đây là một *quan hệ tỉ lệ* chứ không phải một đẳng thức, và nó được dùng đúng

như vậy (tôi suy ra từ hình học cơ bản; cấu trúc phù hợp với phân tích trong [29] và [17] nhưng tôi chưa đổi chiều từng hệ số).

Điều then chốt nằm ở mệnh đề thứ hai: *độ nhạy tỉ lệ với $\sin \theta$* . Khi $\theta \rightarrow 0$, tức khi tag ở tư thế fronto-parallel, độ nhạy tiến về không. Nghĩa là: nghiêng tag thêm một chút không làm ảnh đổi tới bậc một — nó chỉ đổi tới bậc hai. Và một đại lượng mà phép đo chỉ phản ứng ở bậc hai là một đại lượng gần như không quan sát được.

TRỰC GIÁC

Vì sao fronto-parallel lại là tệ nhất, nói bằng hình ảnh và bằng số.

Cầm một tấm bìa hình vuông và giơ thẳng trước mặt, vuông góc với hướng nhìn. Bây giờ nghiêng nó đi 1° . Hình vuông có trông khác đi không? Gần như không — nó vẫn là một hình vuông, chỉ ngắn lại theo một chiều đúng $\cos 1^\circ = 0,99985$, tức 0,015%. Với tag 120 px, đó là 0,018 px — chìm hoàn toàn dưới nhiễu 0,2 px.

Bây giờ nghiêng tấm bìa đi 45° rồi nghiêng thêm 1° nữa. Lần này hình dạng đổi rõ: chiều ngắn lại từ $\cos 45^\circ = 0,7071$ xuống $\cos 46^\circ = 0,6947$, tức 1,8% — trên tag 120 px là 2,2 px, hoàn toàn đo được.

Cùng một lượng nghiêng, hai mức đổi ảnh khác nhau hơn trăm lần. Đó là toàn bộ nội dung của $\sin \theta$: đạo hàm của $\cos$ là $-\sin$, và $\sin$ bằng không tại không.

Bài học thực hành theo ngay: *đường tiếp cận vuông góc là đường tệ nhất cho việc đo góc*. Nếu có thể chọn, hãy tiếp cận chếch, hoặc — tốt hơn vì không đụng tới quỹ đạo robot — nghiêng chính các tag đi so với mặt phẳng dock.

5.6 Sai số xoay từ constellation: chia cho baseline

Mục trước kết thúc ở một tin xấu. Mục này là lối thoát, và lối thoát ấy đơn giản đến mức gần như thất vọng.

Giả sử thay vì một tag, ta có hai tag cách nhau một khoảng B đã biết trong hệ dock, cả hai đều nhìn thấy được. Bây giờ hỏi: cả cụm quay đi một góc $\delta\theta$ quanh một trục vuông góc với đường nối hai tag thì chuyện gì xảy ra? Tag thứ nhất dịch chuyển một đoạn $\approx (B/2)\delta\theta$ theo một hướng, tag thứ hai dịch một đoạn như thế theo hướng ngược lại. Nếu trục quay được chọn sao cho dịch chuyển ấy nằm theo *chiều ngang* của ảnh — và với constellation trải rộng thì luôn có thành phần như vậy — thì ta đo nó qua cơ chế *tốt* của mục 3A, không qua cơ chế *sâu*.

Vì vị trí của mỗi tag được xác định với sai số δx , và góc được suy từ chênh lệch vị trí của hai tag chia cho khoảng cách giữa chúng:

$$\delta\theta \approx \frac{\delta x}{B} = \frac{Z\sigma}{fB} \quad (5.5)$$

Đây là quan hệ thứ tư và là quan hệ có giá trị thực dụng cao nhất trong cả cây.

A. Ví dụ số, tiếp nối ví dụ ở mục 4

Giữ nguyên $f = 600$ px, $Z = 0,5$ m, $\sigma = 0,2$ px, nên $\delta x = 0,167$ mm như đã tính. Đặt baseline $B = 0,4$ m:

$$\delta\theta \approx \frac{0,167 \text{ mm}}{400 \text{ mm}} = 4,2 \times 10^{-4} \text{ rad} = 4,2 \times 10^{-4} \times \frac{180}{\pi} = \mathbf{0,024^\circ}.$$

So với ngân sách $0,5^\circ$: ta dư *hơn hai mươi lần*. Đặt cạnh con số “dễ dàng vượt $1\text{--}2^\circ$ ” của một tag phẳng nhìn vuông góc, chênh lệch giữa hai phương án là gần hai bậc độ lớn — và phương án

tốt hơn không đòi một dòng code nào khác, chỉ đòi đặt hai tấm tag cách nhau 40 cm thay vì một tấm.

Đây là câu trả lời cho Q5, và lý do câu trả lời không phụ thuộc con số cụ thể nằm ở cấu trúc của hai công thức. Giảm σ bốn lần thì cả δx lẫn $\delta \theta$ giảm bốn lần — tuyến tính, và có trần: σ không xuống dưới giới hạn quang học được (A/06). Tầng B thì $\delta \theta$ giảm tuyến tính theo B — cũng tuyến tính, nhưng B rẻ và gần như không có trần trong phạm vi kích thước một cái dock. Hơn nữa, và đây mới là điểm quyết định: tầng B đồng thời cho phép *phá tính đồng phẳng* nếu bạn đặt hai tag ở hai độ sâu khác nhau, và cái đó khử luôn bài toán hai nghiệm của A/04 — một lợi ích mà giảm σ hoàn toàn không cho.

CẦN NHỚ

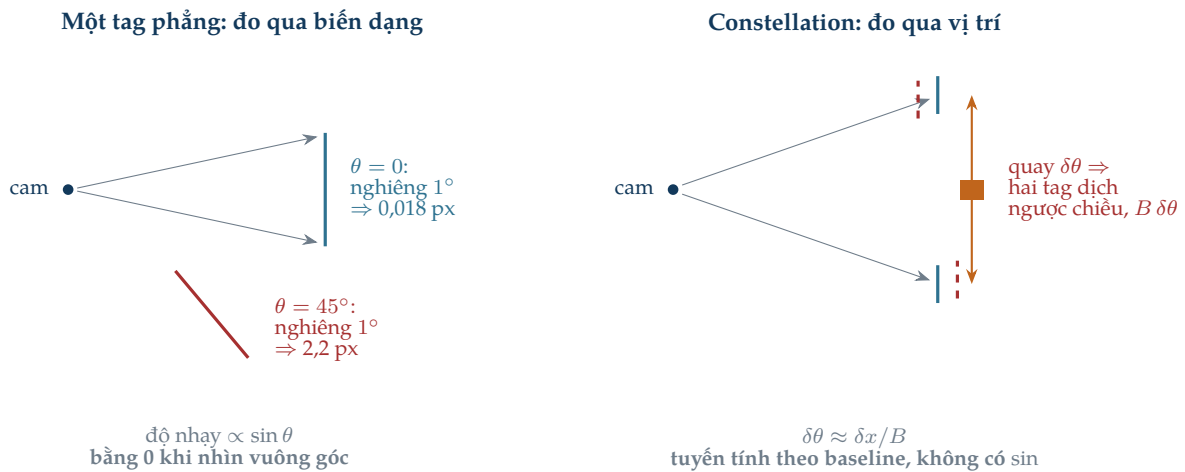
Bốn quan hệ, thuộc lòng cả bốn:

$$\delta x \approx \frac{Z\sigma}{f}, \quad \delta Z \approx \frac{Z\sigma}{w}, \quad \delta \theta_{\text{tag phẳng}} \sim \frac{1}{\sin \theta} \cdot \frac{Z\sigma}{wW}, \quad \delta \theta_{\text{constellation}} \approx \frac{Z\sigma}{fB}.$$

Đọc chúng cạnh nhau thì thấy ngay cấu trúc: tử số luôn là $Z\sigma$; cái quyết định là *mẫu số*. Mẫu số càng lớn càng tốt, và bốn mẫu số là $f, w, wW \sin \theta, fB$.

Kết luận rút ra trong một câu: *muốn xoay tốt thì tăng B và phá đồng phẳng, không phải tăng số tag đồng phẳng*. Thêm một tag thứ ba nằm giữa hai tag cũ hầu như không cải thiện gì vì nó không tăng B ; đẩy một tag ra xa thêm 20 cm thì cải thiện thấy rõ.

Hình 5.1 vẽ hai cơ chế cạnh nhau, và nó là hình nên nhớ cho cả chương.



Hình 5.1. Hai cơ chế ước lượng góc. Bên trái, một tag phẳng: góc nghiêng chỉ lộ ra qua biến dạng phối cảnh, mà biến dạng ấy có đạo hàm tỉ lệ với $\sin \theta$ — bằng không đúng ở tư thế fronto-parallel, tức đúng tư thế của một robot tiếp cận thẳng trục. Hai con số trong hình là ví dụ số học cho tag 120 px (tính ở mục 5), không phải số đo. Bên phải, một constellation baseline B : góc lộ ra qua chênh lệch vị trí của hai tag, một đại lượng đo bằng cơ chế tốt, và độ nhảy không có hệ số sin nào cả. Đây là toàn bộ lý do vì sao lời khuyên “tăng baseline” thắng lời khuyên “dùng detector tốt hơn”.

5.7 Vì sao trung bình pose từ từng tag là sai

Công thức (5.5) mang một hệ quả mà nhiều người bỏ lỡ, và nó đủ quan trọng để có mục riêng.

Cách tự nhiên nhất khi nhìn thấy bốn tag là: chạy PnP cho từng tag, được bốn pose, rồi lấy trung bình. Cách này *hỏng*, và nó hỏng theo hai tầng.

Tầng thứ nhất là tầng thống kê, và nó tương đối dễ thấy. Pose sống trên $SE(3)$ chứ không phải trên không gian vectơ, nên trung bình số học của các ma trận xoay không phải một ma trận xoay, và trung bình của quaternion phụ thuộc vào dấu ta gán cho từng quaternion (vì q và $-q$ biểu diễn cùng một xoay). Có cách làm đúng — trung bình Karcher trên $SO(3)$ [35], và B/11 bàn kỹ — nhưng ngay cả khi làm đúng, tầng thứ hai vẫn còn.

Tầng thứ hai là tầng thông tin, và nó mới là tầng quyết định. Hãy nhìn kỹ vào việc ta vừa vứt đi cái gì. Pose của từng tag riêng lẻ có hiệp phương sai rất *dị hướng*: chính xác cỡ 0,17 mm theo hai chiều ngang, 0,83 mm theo chiều sâu, và tệ tới vài độ theo hướng nghiêng. Trung bình bốn đại lượng như vậy cho ra một đại lượng có hiệp phương sai vẫn *dị hướng* như thế, chỉ nhỏ đi $\sqrt{4} = 2$ lần theo mỗi hướng — nên thành phần nghiêng vẫn tệ, chỉ tệ ít hơn hai lần.

Trong khi đó, PnP hợp nhất trên toàn bộ tập góc *biết* rằng bốn tag ấy gắn cứng với nhau ở những vị trí đã biết. Nó không đo bốn pose rồi gộp; nó đo *một* pose bằng cách dùng toàn bộ $4 \times 4 = 16$ góc, và vì các góc ấy trải trên một vùng rộng B , thông tin về hướng đến từ cơ chế baseline chứ không từ cơ chế biến dạng. Cải thiện không phải $\sqrt{4}$ lần mà là cỡ B/W lần, nhưng quan trọng hơn là nó *đổi cơ chế*, và đổi cơ chế thì thoát khỏi hệ số $\sin \theta$.

Nói bằng ngôn ngữ của công thức (5.1): trung bình pose là việc cộng các Σ_i , còn PnP hợp nhất là việc cộng các $\mathcal{I}_i = J_i^T \Sigma_i^{-1} J_i$ trong cùng một hệ tọa độ. Hai phép cộng ấy không giao hoán với phép nghịch đảo, và sự không giao hoán ấy chính là chỗ thông tin hình học nằm.

CẠM BẮY

Trung bình pose từ từng tag không chỉ kém tối ưu — nó còn *che giấu* vấn đề. Bốn pose từ bốn tag đồng phẳng sẽ có thành phần nghiêng nhảy loạn, nhưng trung bình của chúng trông mượt hơn nhiều và độ lệch chuẩn mẫu của bốn giá trị ấy trông nhỏ. Bạn sẽ báo cáo một độ lặp lại tốt cho một đại lượng vẫn đang sai có hệ thống.

Phép thử phân biệt: đừng nhìn độ tán mát giữa các tag, hãy nhìn *sai số tái chiếu*. PnP hợp nhất cho một con số sai số tái chiếu duy nhất, và nếu constellation của bạn sai so với mô hình thì con số ấy lớn lên và bạn thấy. Trung bình pose không có đại lượng tương đương, nên nó không có cách nào báo động.

5.8 Cộng các nguồn sai số: khi nào bình phương, khi nào cộng thẳng

Chương này tính đóng góp của nhiều góc. Ngân sách thật (C/15) có nhiều dòng hơn: sai số constellation so với mô hình, sai số intrinsic, sai số hand-eye, sai số bám của bộ điều khiển. Câu hỏi cộng chúng thế nào không tầm thường và trả lời sai thì ngân sách vô nghĩa.

Quy tắc có hai nhánh, và ranh giới giữa chúng là *ngẫu nhiên hay hệ thống*.

Các nguồn **ngẫu nhiên** (*random*) — độc lập, trung bình không, đổi từ khung này sang khung khác — cộng theo bình phương:

$$\sigma_{\text{tổng}} = \sqrt{\sigma_1^2 + \sigma_2^2 + \dots}$$

Nhiều định vị góc thuộc loại này. Hệ quả quan trọng của phép cộng bình phương: *nguồn nhỏ gần như không đóng góp gì*. Nếu $\sigma_1 = 3$ mm và $\sigma_2 = 1$ mm thì tổng là $\sqrt{9 + 1} = 3,16$ mm — nguồn thứ hai thêm 5%, dù nó bằng một phần ba nguồn thứ nhất. Đây là cơ sở định lượng cho luận điểm “mất xích yếu nhất quyết định” ở 00-map, và nó nói rằng tối ưu hoá một nguồn đã nhỏ là lãng phí.

Các nguồn **hệ thống** (*systematic*) — không đổi hoặc đổi chậm, có dấu xác định — cộng *thẳng* trong trường hợp xấu nhất:

$$e_{\text{tổng}} \leq e_1 + e_2 + \dots$$

Sai số kích thước tag, sai số constellation so với CAD, sai số hand-eye thuộc loại này. Chúng không trung bình mất đi khi bạn lấy nhiều khung — lấy một nghìn khung thì chúng vẫn nguyên vẹn. Đây là lý do vì sao 1 mm sai số cơ khí nguy hiểm hơn nhiều so với 1 mm nhiễu: nhiễu chia cho $\sqrt{N}$, sai số hệ thống thì không.

Và đây là chỗ luận điểm teach-by-showing của B/13 có cơ sở định lượng. Nếu ta điều khiển theo *sai lệch tương đối* so với một pose mục tiêu đã ghi lại, thì mọi nguồn hệ thống xuất hiện ở cả hai vế và *trừ đi nhau*. Ngân sách thu hẹp lại chỉ còn các nguồn ngẫu nhiên — tức chỉ còn những nguồn chia được cho $\sqrt{N}$. Đó là lý do kỹ thuật ấy không phải một mẹo vặt mà là một thay đổi bậc độ lớn trong bài toán.

I 5.9 Giới hạn và chế độ hỏng

Bốn công thức của chương đứng trên một tập giả thiết, và chúng hỏng khi giả thiết bị vi phạm. Bốn chỗ hỏng dưới đây đều xảy ra thật.

Giả thiết nhiễu Gauss, độc lập, đẳng hướng, trung bình không. Vi phạm phổ biến nhất là *tương quan*: bốn góc của cùng một tag chịu chung một mức mờ, một hướng chiếu sáng, một sai số méo cục bộ, nên nhiễu của chúng tương quan mạnh. Hậu quả: hiệp phương sai thật lớn hơn công thức dự đoán, và cải thiện theo $\sqrt{N}$ không xảy ra. Vi phạm thứ hai là *trung bình khác không*, tức thiên lệch — motion blur đẩy góc về một phía, bias tâm ellipse đẩy tâm theo một hướng phụ thuộc tư thế (A/06). Công thức (5.1) hoàn toàn mù với thiên lệch: nó mô tả độ tản mát quanh *giá trị mà bộ ước lượng hội tụ tới*, chứ không nói giá trị ấy có đúng không.

Giả thiết mô hình đúng. Đã bàn ở mục 2: nếu intrinsic sai hoặc constellation sai, phần dư mang thành phần hệ thống, khai triển Gauss–Newton không còn hợp lệ, và hiệp phương sai báo về nhỏ hơn sự thật. Đây là chế độ hỏng nguy hiểm nhất vì nó đi theo chiều sai lầm: hệ thống trở nên *tự tin hơn* khi nó sai nhiều hơn.

Giả thiết góc nhỏ trong các quan hệ tỉ lệ. Công thức (5.5) dùng xấp xỉ $\sin \delta\theta \approx \delta\theta$ — luôn đúng cho sai số cỡ phần nghìn radian, nên đây không phải giới hạn thực tế. Nhưng (5.4) thì khác: nó là quan hệ tỉ lệ chứ không phải đẳng thức, và hệ số của nó phụ thuộc vào chi tiết phân bố góc mà tôi không dẫn ra. Dùng nó để *so sánh* hai phương án thì được; dùng nó để dự đoán một con số tuyệt đối thì không. Với con số tuyệt đối, hãy chạy Monte Carlo (C/16).

Giả thiết tuyến tính hoá quanh nghiệm đúng. Toàn bộ phân tích là *cục bộ*. Nó không nói gì về khả năng bộ giải hội tụ vào một cực tiểu cục bộ khác — và với target phẳng thì có đúng một cực tiểu khác như thế, chính là nghiệm thứ hai của A/04. Hiệp phương sai tính quanh nghiệm sai vẫn trông đẹp. Đây là lý do chương 4 và chương này phải đọc cùng nhau: chương này định giá nhiễu, chương kia cảnh báo về lỗi thô, và một ngân sách chỉ tính nhiễu là một ngân sách bỏ sót chi phí lớn nhất.

Bảng 5.1. Bốn giả thiết của chương, chỗ hỏng, và dấu hiệu nhận ra.

Giả thiết	Vi phạm điển hình	Dấu hiệu nhận ra
Nhiều độc lập giữa các góc	Cùng tag chịu chung blur, chiếu sáng, méo cục bộ	Lấy trung bình N khung không cải thiện theo $1/\sqrt{N}$; độ tản mát bão hoà sớm
Nhiều trung bình không	Motion blur, bias tâm ellipse, rolling shutter	Sai số đổi dấu khi đổi chiều chuyển động; sai số phụ thuộc tư thế một cách trơn
Mô hình đúng	Intrinsic sai, constellation lệch CAD, méo chưa khử sạch	Sai số tái chiếu có <i>cấu trúc</i> theo vị trí ảnh, không giống nhiễu trắng
Tuyến tính hoá cục bộ	Hội tụ vào nghiệm thứ hai của target phẳng	Pose nhảy giữa hai giá trị; tỉ số sai số tái chiếu của hai nghiệm gần 1

5.10 Thực hành: dùng bốn quan hệ này trong một buổi chiều

Bốn công thức trên có một cách dùng rất cụ thể ở giai đoạn thiết kế, và nó gồm năm bước làm được trên giấy.

Bước một, viết ra bốn con số của bài toán: tiêu cự f tính bằng pixel (từ trường nhìn và độ phân giải, hoặc từ hiệu chuẩn nếu đã có), khoảng cách cuối Z ở thời điểm cần chính xác nhất, kích thước tag W dự kiến, và σ — dùng 0,2 px làm giá trị mặc định thận trọng nếu chưa đo được. *Bước hai*, tính $w = fW/Z$; nếu w nhỏ hơn khoảng 60 px thì dừng lại và xem lại thiết kế, vì decode sẽ bắt đầu không tin cậy (B/07) và mọi phép tính sau đó là vô nghĩa. *Bước ba*, tính δx và δZ ; gần như chắc chắn chúng sẽ nhỏ so với ngân sách, và đó là tin tốt cần được ghi nhận chứ không phải bỏ qua. *Bước bốn*, tính $\delta \theta$ cho baseline dự kiến và so với ngân sách góc; đây là dòng quyết định. *Bước năm*, nếu không đạt, giải ngược công thức (5.5) theo B để biết baseline tối thiểu cần bao nhiêu — và đó là con số bạn mang sang cho người thiết kế cơ khí.

Bước năm đáng nhấn mạnh vì nó là chỗ chương này trả lại giá trị lớn nhất. Giải ngược:

$$B_{\min} = \frac{Z \sigma}{f \delta \theta_{\text{cho phép}}}.$$

Với ví dụ của cây ($Z = 0,5$ m, $\sigma = 0,2$ px, $f = 600$ px) và ngân sách góc $0,5^\circ = 8,73 \times 10^{-3}$ rad, ta được $B_{\min} = 0,1/(600 \times 8,73 \times 10^{-3}) = 0,019$ m, tức 19 mm. Con số ấy nhỏ đến mức đáng ngờ, và nó đáng được đọc cho đúng: nó nói rằng *nếu* cơ chế baseline hoạt động thuần tuý và không có nguồn sai số nào khác, thì một baseline 2 cm đã đủ. Thực tế không như vậy, vì các nguồn hệ thống ở mục 8 — đặc biệt là sai số của chính constellation — chiếm phần lớn ngân sách. Cách dùng đúng của con số này là: *nhiều góc không phải ràng buộc cho baseline; ràng buộc thật đến từ độ chính xác cơ khí của tâm marker*, và C/15 làm rõ chuyện đó bằng một bảng đầy đủ.

Active recall (che cột phải, tự trả lời, rồi mở ra đối chiếu)

Thuật ngữ / câu hỏi	Trả lời
••• Công thức gốc của cả chương	$\Sigma_{\text{pose}} \approx (\sum_i J_i^T \Sigma_i^{-1} J_i)^{-1}$ — nghịch đảo ma trận thông tin Fisher [32], [33]. Nó là cận dưới cho <i>mọi</i> bộ ước lượng không chệch, nên đổi thuật toán không vượt qua được nó.
••• Sai số ngang	$\delta x \approx Z \sigma / f$. Tuyến tính theo khoảng cách. Không phụ thuộc kích thước tag — đây là chỗ phản trực giác nhất trong bốn công thức.
••• Sai số dọc trục	$\delta Z \approx Z \sigma / w = Z^2 \sigma / (fW)$. Bậc hai theo khoảng cách. Tệ hơn sai số ngang đúng $f/w = Z/W$ lần.

Thuật ngữ / câu hỏi	Trả lời (tiếp)
●●● Tỉ số sâu trên ngang	Z/W — khoảng cách chia kích thước tag. Tag 100 mm ở 500 mm: tệ hơn 5 lần. Ở 3 m: tệ hơn 30 lần.
●●● Xoay từ một tag phẳng	Độ nhảy $\propto \sin \theta$ với θ là góc nghiêng. Bảng không ở fronto-parallel, tức đúng tư thế của robot tiếp cận thẳng trục. Đây là nguồn lỗi xoay số một.
●●● Xoay từ constellation	$\delta\theta \approx \delta x/B = Z\sigma/(fB)$. Không có hệ số sin nào. Cải thiện tuyến tính theo baseline.
●●● Ví dụ số chuẩn của cây	$f = 600$ px, $Z = 0,5$ m, $W = 100$ mm, $\sigma = 0,2$ px $\Rightarrow w = 120$ px, $\delta x = 0,17$ mm, $\delta Z = 0,83$ mm, và với $B = 0,4$ m thì $\delta\theta = 0,024^\circ$. Tất cả tính tay được.
●●● Kết luận một câu của chương	Translation gần như luôn đạt; rotation là nút thắt; và cách chữa nút thắt là hình học (baseline, phá đồng phẳng) chứ không phải thuật toán.
●● Vì sao trung bình pose là sai	Hai tầng: pose sống trên $SE(3)$ nên trung bình số học không hợp lệ [35]; và quan trọng hơn, nó vứt bỏ ràng buộc hình học giữa các tag, tức vứt bỏ đúng cơ chế baseline.
●● Cộng sai số ngẫu nhiên	Theo bình phương: $\sqrt{\sigma_1^2 + \sigma_2^2}$. Hệ quả: nguồn bằng 1/3 nguồn lớn nhất chỉ thêm 5% vào tổng — nên tối ưu nguồn nhỏ là lãng phí.
●●● Cộng sai số hệ thống	Cộng thẳng trong trường hợp xấu nhất, và <i>không</i> chia cho $\sqrt{N}$ khi lấy trung bình nhiều khung. Đây là lý do 1 mm cơ khí nguy hiểm hơn 1 mm nhiễu.
●● Baseline tối thiểu	$B_{\min} = Z\sigma/(f \delta\theta_{\text{cho phép}})$. Với ví dụ chuẩn và ngân sách $0,5^\circ$: 19 mm — nhỏ, và điều đó nói rằng ràng buộc thật không đến từ nhiễu góc mà từ cơ khí.
●● Chế độ hồng nguy hiểm nhất	Mô hình sai (intrinsic, constellation): phần dư có thành phần hệ thống, hiệp phương sai báo về <i>nhỏ hơn</i> sự thật. Hệ thống tự tin hơn khi nó sai nhiều hơn.
●● Dấu hiệu mô hình sai	Sai số tái chiếu có <i>cấu trúc</i> theo vị trí ảnh — vẽ residual theo tọa độ ảnh, nếu thấy vân thì mô hình sai chứ không phải nhiễu lớn [10].
● Vì sao không đưa $\sqrt{N}$ vào công thức	Vì nhiễu giữa các góc cùng một tag tương quan mạnh (chung blur, chung chiếu sáng, chung méo cục bộ), nên $\sqrt{N}$ là lời hứa thường không được giữ. Bỏ nó ra là cách ước lượng thận trọng.

●●● phải thuộc ●● nên nắm ● biết là có

I Kiến thức này đang sống ở đâu (công trình đang dùng chính ý tưởng của chương; chốt 9/2026)

Công trình / hệ thống	Dùng ý tưởng nào của chương	Ghi chú
apriltag (<code>apriltag_pose.c</code>)	Tính sai số tái chiếu cho hai nghiệm IPPE — đo trực tiếp thứ mà mục 9 gọi là cảnh báo lỗi thô	Chỗ lý thuyết chương này gặp code; đọc kèm A/04
OpenCV solvePnPGeneric	Trả về nhiều nghiệm kèm sai số tái chiếu từng nghiệm	API duy nhất cho bạn đại lượng cần để đo ambiguity
Hiệu chuẩn bằng target phẳng [6]	Dùng target rộng và nhiều tư thế để ma trận thông tin không suy biến theo hướng nào — đúng nguyên lý baseline của mục 6	Hiệu chuẩn và docking là cùng một bài toán hình học
tag slam [31]	Bài toán hợp nhất trên toàn bộ tag thay vì trung bình pose từng tag	Hiện thân của mục 7; mức tin C, dùng làm gợi ý kiến trúc
Đánh giá quỹ đạo [36]	Cùng tư duy: chọn sai phép căn chỉnh thì con số sai số vô nghĩa	Song song về phương pháp luận, khác lĩnh vực

MINI PROJECT · Kiểm bốn quan hệ tỉ lệ bằng Monte Carlo, không cần phần cứng · 1–2 ngày

Mục tiêu	xác nhận bằng số rằng bốn công thức của chương đúng tới hệ số, và tự tay nhìn thấy hệ số $\sin \theta$ làm sai số xoay nổ tung khi tag về fronto-parallel.
Dữ liệu	tự sinh hoàn toàn. Không cần camera, không cần tag thật. Một hàm chiếu điểm với K đã biết, một tập góc 3D, một bộ sinh nhiễu Gauss.
Các bước	dựng một tag vuông cạnh W với bốn góc trong hệ dock; đặt nó ở khoảng cách Z , chiếu ra ảnh bằng K ; thêm nhiễu Gauss σ vào từng tọa độ góc; chạy solvePnPGeneric; lặp 2000 lần và lấy độ lệch chuẩn của từng thành phần pose; lặp toàn bộ quy trình khi quét Z , quét W , quét σ , và quét góc nghiêng θ từ 0° tới 60° ; cuối cùng thay một tag bằng hai tag cách nhau B và quét B .
Xong khi	bạn có bốn đồ thị log-log, và độ dốc của chúng khớp với bốn công thức: δx dốc +1 theo Z , δZ dốc +2 theo Z , $\delta \theta$ dốc -1 theo B , và $\delta \theta_{\text{tag}}$ tăng vọt khi $\theta \rightarrow 0$. Kiểm tra chéo bất buộc: với θ nhỏ, hãy <i>cũng</i> kiểm xem solvePnPGeneric có trả về hai nghiệm với sai số tái chiếu gần bằng nhau không — nếu có thì phần lớn độ tán mát bạn đang đo là lỗi thô do chọn nhầm nghiệm chứ không phải nhiễu, và hai thứ đó phải được báo cáo riêng.
Nối với	C/16 — script này chính là hạt nhân của mini project ở đó; A/04 cho phần hai nghiệm.

TRẢ LỜI CÂU HỎI MỞ ĐẦU

- A1. Không phải gấp đôi trong thực tế, dù công thức nói vậy. Tăng độ phân giải gấp đôi với cùng ống kính làm f tính bằng pixel tăng gấp đôi, nên $\delta x = Z\sigma/f$ giảm một nửa — nếu σ tính bằng pixel giữ nguyên. Nó thường không giữ nguyên: pixel nhỏ hơn thu ít photon hơn nên tỉ số tín hiệu trên nhiễu kém đi, và quang học có thể không phân giải nổi mức chi tiết mới (giới hạn MTF, A/06). Nguyên tắc chung: f là đại lượng hình học thật, còn σ là đại lượng phụ thuộc toàn bộ chuỗi quang học và thuật toán, nên đừng cải thiện một cái mà quên kiểm cái kia.
- A2. Tag 200 mm cho $w = 240$ px, nên $\delta Z = 0,5 \times 0,2/240 = 0,42$ mm — cải thiện đúng hai lần, tuyến tính theo W . Nhưng hai tag 100 mm cách nhau 200 mm cho một thứ mà tag lớn không cho: một baseline. Sai số xoay của nó là $\delta x/B = 0,167/200 = 8,3 \times 10^{-4}$ rad = $0,048^\circ$, trong khi tag đơn 200 mm vẫn chịu hệ số $\sin \theta$ nên ở fronto-parallel thì tệ hơn hàng chục lần. Nói ngắn: tag lớn mua cho bạn chiều sâu, constellation mua cho bạn góc — và góc mới là nút thắt.
- A3. Bằng f/w , tương đương Z/W . Lý do cơ chế: chiều ngang được đo *trực tiếp* qua vị trí trên ảnh với hệ số quy đổi Z/f ; chiều sâu được đo *gián tiếp* qua kích thước biểu kiến với hệ số quy đổi Z/w . Vì w luôn nhỏ hơn f rất nhiều (tag chiếm một phần nhỏ của trường nhìn), chiều sâu luôn tệ hơn. Cách nhớ: bạn đo vị trí bằng cả khung hình làm thước, còn đo khoảng cách chỉ bằng chính cái tag làm thước.
- A4. Vì góc nghiêng chỉ lộ ra qua *biến dạng phối cảnh*, và biến dạng ấy tỉ lệ với $\cos \theta$ — một hàm có đạo hàm $-\sin \theta$, bằng không tại $\theta = 0$. Cụ thể: nghiêng 1° từ tư thế vuông góc làm cạnh ngắn lại 0,015%, tức 0,018 px trên tag 120 px, chìm dưới nhiễu. Nghiêng 1° từ tư thế 45° làm cạnh ngắn lại 1,8%, tức 2,2 px, đo được thoải mái. Cùng một lượng nghiêng, hai mức tín hiệu cách nhau hơn trăm lần. Lối thoát có hai: tiếp cận chéo có chủ ý, hoặc — tốt hơn vì không đụng tới quỹ đạo robot — nghiêng chính các tag trong constellation.
- A5. Baseline, và chênh lệch là lớn chứ không sát sao. Giảm σ bốn lần (0,2 xuống 0,05 px — một cải thiện rất khó, đòi chuyển sang ChArUco cộng quang học tốt cộng trung bình nhiều khung) làm $\delta \theta$ giảm đúng bốn lần. Thêm 30 cm baseline lên một constellation đang có 10 cm làm $\delta \theta$ giảm bốn lần bằng một việc duy nhất là bắt vít hai tấm tag xa nhau hơn. Lý do câu trả lời không phụ thuộc con số cụ thể: cả hai đều tuyến tính, nên tỉ lệ giá trên hiệu quả quyết định, và baseline gần như miễn phí trong phạm vi kích thước một cái dock trong khi σ chạm trần vật lý rất nhanh. Thêm nữa, baseline lớn hơn thường đi kèm khả năng đặt tag ở hai độ sâu khác nhau, và cái đó khử luôn bài toán hai nghiệm — một lợi ích mà giảm σ hoàn toàn không cho.

NÓI LẠI THẬT ĐƠN GIẢN

Tưởng tượng bạn đứng ở một đầu sân bóng và có người hỏi về một cái biển hiệu ở đầu kia.

Biển nằm lệch trái phải bao nhiêu? Dễ. Bạn nhìn nó nằm ở đâu so với khung hình trong mắt mình, rồi quy ra. Nhìn càng rõ thì trả lời càng chính xác, và độ chính xác tỉ lệ thuận với việc bạn nhìn rõ tới đâu.

Biển cách bao xa? Khó hơn nhiều, vì bạn không nhìn thẳng được khoảng cách. Bạn phải suy: “biển ấy cao chừng một mét, mà trông nó nhỏ thế này, vậy nó phải xa chừng ấy.” Vấn đề là bạn đang lấy chính cái biển làm thước đo, mà cái biển thì nhỏ. Đo bằng một cái thước nhỏ thì sai nhiều hơn đo bằng một cái thước lớn — đó là toàn bộ lý do khoảng cách khó hơn vị trí, và nó khó hơn đúng bằng tỉ số giữa cái sân và cái biển.

Biển đang xoay nghiêng bao nhiêu độ? Đây mới là chỗ thú vị, và nó không khó hơn một chút mà khó hơn theo một kiểu khác hẳn. Nếu cái biển đang quay mặt thẳng về phía bạn, thì nghiêng nó đi một độ trông *y* hết. Bạn không phát hiện được, không phải vì mắt bạn kém mà vì thật sự không có gì để phát hiện. Còn nếu cái biển đang nghiêng sẵn bốn mươi lăm độ, thì nghiêng thêm một độ nhìn thấy rõ ngay. Cùng một câu hỏi, cùng một con mắt, hai câu trả lời khác nhau hàng trăm lần — và cái quyết định là *tư thế của cái biển*, không phải chất lượng của mắt bạn.

Giờ đến mẹo. Giả sử thay vì một cái biển, ở đầu sân có *hai* cái cột cách nhau mười mét, và bạn biết chúng cách nhau đúng mười mét. Cả cụm quay đi một độ thì sao? Cột này dịch sang trái, cột kia dịch sang phải, mỗi cột dịch gần một tấc. Mà việc nhìn xem một cái cột dịch sang trái hay phải thì bạn giỏi — đó là câu hỏi loại thứ nhất, câu hỏi dễ. Bạn vừa biến một câu hỏi khó thành một câu hỏi dễ, chỉ bằng cách nhìn hai vật thay vì một.

Và đây là điều đáng nhớ nhất: cách làm ấy không đòi mắt tinh hơn. Nó đòi hai cái cột đặt xa nhau. Đặt chúng cách nhau hai mươi mét thay vì mười thì bạn đo góc chính xác gấp đôi — vẫn với đúng con mắt ấy.

Chỗ không rút gọn được: ranh giới giữa “nhiều” và “sai số hệ thống” không nhìn ra được từ một lần đo. Cả hai đều làm con số lệch khỏi sự thật; chỉ có điều một loại thì lấy trung bình nhiều lần sẽ hết, loại kia thì không. Phân biệt chúng đòi hoặc một chuẩn bên ngoài để so, hoặc một phép thử thay đổi điều kiện có chủ ý — và đó là việc phải làm bằng tay, không có công thức nào thay được.

Một câu: Sai số chia cho tiêu cự thì nhỏ, chia cho kích thước tag thì lớn hơn, còn góc thì đừng chia cho gì cả — hãy chia cho baseline.

5.11 Điều gì chứng minh cách hiểu này sai

Theo quy tắc của kho, một chương phải nói rõ điều gì sẽ bác bỏ nó. Với chương này, các mệnh đề bác bỏ được rất cụ thể — và may mắn là chúng đều kiểm được bằng script Monte Carlo trong một buổi.

Mệnh đề chính — $\delta\theta$ từ constellation tỉ lệ nghịch với B , không có hệ số $\sin\theta$ — bị bác bỏ nếu một mô phỏng cẩn thận cho thấy sai số xoay của một constellation không đồng phẳng *vẫn* tăng vọt khi camera về tư thế fronto-parallel so với mặt phẳng trung bình của constellation. Tôi đã dẫn mệnh đề này bằng hình học góc nhỏ (cửa a) nhưng *chưa* chạy mô phỏng, nên nó chưa qua cửa (b) — và đây là chỗ nợ kiểm chứng lớn nhất của chương, đã ghi vào 99-chua-biet.md.

Mệnh đề thứ hai — $\delta Z/\delta x = Z/W$ — bị bác bỏ nếu đo được một tỉ số khác đáng kể trong mô phỏng với nhiễu Gauss độc lập. Đây là mệnh đề tôi tin nhất vì nó đã qua hai cửa: tự dẫn bằng vi phân, và kiểm nhất quán bằng ví dụ số ở mục 4 (hai công thức độc lập cho cùng tỉ số 5).

Mệnh đề thứ ba — PnP hợp nhất tốt hơn trung bình pose, và tốt hơn theo hệ số cỡ B/W chứ không phải $\sqrt{N}$ — bị bác bỏ nếu mô phỏng cho thấy hai cách cho cùng độ tản mát góc trên một constellation trải rộng. Đây là mệnh đề dễ kiểm nhất và nên kiểm sớm nhất, vì nó ảnh hưởng trực tiếp tới kiến trúc phần mềm.

Cuối cùng, một mệnh đề mềm hơn nhưng đáng nêu: chương này khẳng định rằng *trong bài toán docking, thành phần xoay luôn là ràng buộc chặt hơn thành phần tịnh tiến*. Điều đó bị bác bỏ nếu trong

một hệ thống thật, sai lệch cuối cùng bị chi phối bởi thành phần tịnh tiến — và điều ấy hoàn toàn có thể xảy ra nếu sai số hand-eye hoặc sai số bám của bộ điều khiển lớn, vì hai nguồn ấy không nằm trong phân tích của chương này. Nói cách khác, kết luận của chương đúng trong phạm vi *chuỗi quang học tới pose*, và C/15 là chương kiểm xem nó còn đúng khi tính cả chuỗi.

5.12 Kết nối

Chương này là bản lề của cả cây: nó nhận công cụ từ ba chương trước và cấp số liệu cho mọi chương sau.

Nối lên trên trước. A/01-mo-hinh-camera-va-distortion cho f , đại lượng camera duy nhất trong mọi công thức ở đây — và nó cũng cảnh báo rằng nếu méo chưa khử sạch thì giả thiết “mô hình đúng” ở mục 2 bị vi phạm và hiệp phương sai báo về là sai. A/03-pnp-va-uoc-luong-tu-the cho Jacobian J ; chương này chỉ làm một việc là đảo $J^T J$ và đọc ý nghĩa của các phần tử. A/04-pose-ambiguity-target-phang là chương song sinh: nó mô tả lỗi thô, chương này mô tả nhiễu, và một phân tích chỉ có một trong hai là một phân tích thiếu. Trên thực tế hai chương ấy nói cùng một sự thật ở hai mức: mục 5 ở đây nói độ nhạy tiến về không khi fronto-parallel, còn chương 4 nói rằng khi độ nhạy về không thì hai nghiệm trở nên không phân biệt được. Đó là cùng một hiện tượng nhìn từ hai phía.

Nối xuống dưới, theo thứ tự cấp bách. C/15-ngan-sach-sai-so là chương nhận trực tiếp và đầy đủ nhất: nó lấy bốn công thức này, gán số thật, cộng thêm các dòng hệ thống theo quy tắc ở mục 8, và ra bảng ngân sách. Nếu bạn chỉ đọc hai chương trong cả cây thì hãy đọc chương này và chương 15. B/09-multi-marker-constellation biến mục 6 thành hướng dẫn thiết kế cụ thể: đặt bao nhiêu tag, ở đâu, nghiêng bao nhiêu. B/08-corner-refinement là chương quyết định σ , tức đại lượng duy nhất ở đây mà phần mềm còn ảnh hưởng được. B/11-uoc-luong-theo-thoi-gian khai thác chỗ mà mục 8 để ngỏ: nguồn ngẫu nhiên chia cho $\sqrt{N}$, nên trung bình nhiều khung là cách rẻ nhất để giảm σ — và nó cũng là chương giải thích vì sao cách đó *không* giúp gì cho các nguồn hệ thống.

Cuối cùng, một nối ngang quan trọng nhất về mặt thực hành. B/13-docking-control lấy mục 8 làm nền lý thuyết cho luận điểm teach-by-showing: nếu điều khiển theo sai lệch tương đối, mọi dòng *hệ thống* trong ngân sách triệt tiêu và chỉ còn các dòng *ngẫu nhiên*, tức chỉ còn những dòng chia được cho $\sqrt{N}$. Chương này cung cấp lý do định lượng cho việc vì sao mẹo ấy không phải một mẹo nhỏ mà là một thay đổi bậc độ lớn.

Câu hỏi tự kiểm (trả lời không nhìn bài)

1. Vì sao sai số ngang không phụ thuộc kích thước tag còn sai số dọc thì phụ thuộc — hãy trả lời bằng cách chỉ ra hai đại lượng đóng vai trò “thước đo” trong hai trường hợp?
2. Một đồng nghiệp đề xuất tăng số tag đồng phẳng từ bốn lên mười sáu để cải thiện độ chính xác góc. Cải thiện kỳ vọng là bao nhiêu lần, và vì sao nó không giải quyết được vấn đề gốc?
3. Bạn đo được sai số xoay thực tế lớn hơn công thức dự đoán mười lần. Nêu ba nguyên nhân hoàn toàn khác nhau, và với mỗi nguyên nhân nêu một phép đo phân biệt nó với hai nguyên nhân kia.
4. Công thức $\Sigma \approx (J^T \Sigma_{\text{px}}^{-1} J)^{-1}$ bỏ đi một số hạng trong đạo hàm bậc hai của hàm mục tiêu. Số hạng đó là gì, khi nào nó không triệt tiêu, và hậu quả đi theo chiều an toàn hay chiều nguy hiểm?
5. Bạn lấy trung bình 100 khung hình khi robot đứng yên. Nguồn sai số nào giảm 10 lần, nguồn nào không giảm chút nào, và làm sao phân biệt hai loại đó chỉ bằng dữ liệu đo chứ không cần chân trị?

6. Với $Z = 2$ m thay vì 0,5 m và mọi thứ khác giữ nguyên, bốn con số của ví dụ chuẩn thành bao nhiêu? Tính nhầm, rồi nói xem con số nào vượt ngân sách trước.
7. Vì sao $B_{\min} = 19$ mm tính ở mục 10 là một con số đúng nhưng dùng sai cách thì nguy hiểm — điều gì nó không tính tới?

Hiệu ứng vật lý và quang học (Physical and Optical Effects)

PHỤ THUỘC A/01-mo-hinh-camera-va-distortion (mô hình camera — chương này liệt kê những gì mô hình ấy bỏ qua); A/05-lan-truyen-sai-so-pixel-sang-pose (phân loại nguồn ngẫu nhiên và hệ thống — chương này chủ yếu nói về loại thứ hai).

TÓM TẮT

Năm chương trước dựng một chuỗi toán học sạch: điểm 3D chiếu thành pixel, pixel cho pose, nhiều pixel cho hiệp phương sai pose. Chuỗi ấy giả định camera là một thiết bị hình học lý tưởng chụp một cảnh đứng yên trong điều kiện lý tưởng. Chương này liệt kê những chỗ giả định ấy sai, và nó được tổ chức theo một tiêu chí duy nhất: *hiệu ứng nào gây nhiễu và hiệu ứng nào gây thiên lệch*. Sự phân biệt ấy quyết định cách xử lý, vì nhiễu giảm theo $1/\sqrt{N}$ khi lấy trung bình còn thiên lệch thì không. Sáu hiệu ứng được bàn: rolling shutter, motion blur, lấy mẫu và răng cưa, độ sâu trường ảnh và MTF, bias tâm ellipse, và đặc trưng quang học của vật liệu marker. Nếu chỉ nhớ một điều: *gần như tất cả chúng gây thiên lệch chứ không gây nhiễu* — và vì thế lời khuyên rẻ nhất của cả chương là dừng hẳn robot lại trước khi đo, vì bốn trong sáu hiệu ứng biến mất khi vận tốc bằng không.

KIẾN THỨC CẦN CÓ TRƯỚC

- **rolling shutter** — cách đọc cảm biến theo từng hàng nối tiếp nhau thay vì cùng lúc. Hàng cuối được phơi sáng muộn hơn hàng đầu một khoảng bằng t_{line} nhân số hàng, nên một vật chuyển động xuất hiện *méo* chứ không chỉ mờ.
- **global shutter** — cách đọc mà mọi pixel bắt đầu và kết thúc phơi sáng cùng lúc. Đắt hơn, độ nhạy thường thấp hơn một chút, và — với bài toán đo lường khi đang chuyển động — gần như bắt buộc.
- **MTF** — *modulation transfer function*: hàm mô tả mức tương phản mà hệ quang học còn giữ được ở từng tần số không gian. MTF thấp ở tần số cao nghĩa là các chi tiết nhỏ bị nhoè, và cạnh tag là một chi tiết tần số cao.
- **đĩa Airy** — điểm sáng nhỏ nhất mà một hệ quang học hoàn hảo tạo ra được, giới hạn bởi nhiễu xạ chứ không bởi chất lượng chế tạo. Đường kính $\approx 2,44\lambda N$ với λ là bước sóng và N là số f (f-number).
- **độ sâu trường ảnh (depth of field)** — khoảng khoảng cách mà trong đó ảnh còn đủ sắc nét theo một tiêu chí cho trước. Với docking, tag ở gần và tag ở xa có thể không cùng nằm trong khoảng ấy, và đó là một ràng buộc thiết kế thật.
- **răng cưa (aliasing)** — hiện tượng khi tần số không gian của cảnh vượt quá một nửa tần số lấy mẫu của cảm biến; chi tiết nhỏ hơn hai pixel không chỉ mất đi mà còn xuất hiện lại dưới dạng mẫu hình giả.
- **retroreflective** — vật liệu phản xạ ánh sáng ngược về phía nguồn thay vì tán xạ đều. Với đèn gắn cạnh camera, nó cho tương phản rất cao và gần như độc lập với ánh sáng môi trường.

CÂU HỎI MỞ ĐẦU

- Q1. Robot của bạn chạy 0,3 m/s và camera phơi sáng 20 ms. Vệt nhoè trên ảnh dài bao nhiêu pixel ở khoảng cách nửa mét, và vì sao con số ấy gây *thiên lệch* chứ không chỉ làm mờ?
- Q2. Cùng một hệ thống, đo khi robot đang tiến và đo khi robot đang lùi qua cùng một điểm, cho hai kết quả lệch nhau có hệ thống theo hai chiều ngược nhau. Hiệu ứng nào giải thích được điều đó, và vì sao nó là một dấu hiệu chẩn đoán rất mạnh?
- Q3. Bạn tăng độ phân giải camera lên gấp bốn và sai số không cải thiện chút nào. Nêu hai cơ chế quang học hoàn toàn khác nhau có thể giải thích, và một phép thử phân biệt chúng.
- Q4. Trong sáu hiệu ứng của chương này, bốn cái biến mất khi robot dừng hẳn. Vì sao đó là lập luận mạnh nhất ủng hộ chiến lược stop-and-go, và hai cái còn lại là gì?
- Q5. Bạn khép khẩu để tăng độ sâu trường ảnh, vì cần cả tag gần lẫn tag xa đều nét. Vì sao khép quá thì ảnh lại nhoè *hơn*, và giới hạn vật lý nào đang chặn bạn?

6.1 Đặt vấn đề

Năm chương trước xây một chuỗi lập luận sạch, và độ sạch ấy có được nhờ một loạt giả định không bao giờ được nói ra: camera chụp tức thời, cảnh đứng yên trong suốt thời gian phơi sáng, ống kính phân giải được mọi chi tiết cần thiết, cảm biến lấy mẫu đủ dày, và bề mặt marker phản xạ ánh sáng một cách hiền lành. Chương này gom tất cả những giả định ấy lại và hỏi chúng sai ở đâu.

Có một lý do cụ thể để làm việc này trong một chương riêng thay vì rải rác. Lý do ấy là: *gần như mọi hiệu ứng vật lý trong danh sách đều gây thiên lệch chứ không gây nhiễu*, và A/05 mục 8 đã chỉ ra rằng hai loại ấy có số phận hoàn toàn khác nhau trong một ngân sách sai số. Nhiễu chia cho $\sqrt{N}$ khi lấy trung bình; thiên lệch thì không chia cho gì cả. Một hệ thống đã tối ưu tốt phần nhiều — detector tốt, subpixel tốt, trung bình nhiều khung — sẽ thấy ngân sách của mình bị chi phối hoàn toàn bởi các dòng trong chương này.

Điều làm chúng khó chịu hơn nữa là chúng *phụ thuộc điều kiện vận hành*. Sai số méo ảnh ở A/01 ít ra là cố định: đo một lần, bù một lần. Còn motion blur phụ thuộc vận tốc, rolling shutter phụ thuộc vận tốc, độ nhoè do lấy nét phụ thuộc khoảng cách, loá phụ thuộc góc chiếu sáng. Chúng đổi trong khi robot đang chạy, và chúng đổi theo những cách tương quan với chính đại lượng ta đang đo — đó là kịch bản tệ nhất cho một phép đo, vì sai số tương quan với đại lượng đo không tách ra được bằng bất kỳ phép thống kê nào.

Cách xử lý hiển nhiên là mô hình hoá từng hiệu ứng rồi bù. Cách này tồn tại và nó hoạt động: có mô hình rolling shutter dùng để *ước lượng* cả pose lẫn vận tốc từ một ảnh duy nhất [37], có cách đo t_{line} bằng một target đã biết [38], có mô hình bù bias tâm ellipse bằng conic [15]. Nhưng với bài toán docking thì có một cách rẻ hơn nhiều và nó là kết luận trung tâm của chương: *bốn trong sáu hiệu ứng biến mất khi vận tốc bằng không*. Dừng robot lại một phần tư giây, chụp ba mươi khung, lấy trung bình, rồi đi tiếp — và bạn vừa xoá bốn dòng khỏi bảng ngân sách mà không cần một dòng code mô hình hoá nào. Đó là cơ sở vật lý của chiến lược stop-and-go ở B/13, và nó là một ví dụ điển hình của luận điểm trung tâm của cây: bài toán này thắng bằng thiết kế chứ không bằng thuật toán.

TRỰC GIÁC

Hình dung sự khác nhau giữa nhiễu và thiên lệch bằng hình ảnh bắn bia. Nhiễu là các phát đạn tản mát quanh tâm bia. Bắn nhiều phát rồi lấy trung bình thì trung bình tiến về tâm — càng nhiều phát càng gần. Đây là tình huống mà mọi công cụ thống kê hoạt động: lấy mẫu nhiều hơn thì biết chắc hơn.

Thiên lệch là các phát đạn tản mát quanh một điểm *lệch khỏi tâm*. Bắn một nghìn phát rồi lấy trung bình thì bạn có một ước lượng rất chính xác của cái điểm lệch ấy. Bạn biết chắc hơn về một con số sai. Và — đây là phần tệ nhất — độ tản mát của một nghìn phát nói cho bạn biết bạn “chắc chắn” tới đâu, nên bạn sẽ báo cáo một khoảng tin cậy rất hẹp quanh một giá trị sai.

Phần lớn nội dung chương này thuộc loại thứ hai. Ống kính làm cạnh nhoè theo một cách nhất quán, nên góc lệch theo cùng một hướng ở mọi khung. Cảm biến đọc hàng dưới muộn hơn hàng trên, nên khi robot chạy thì tag bị xé theo cùng một kiểu ở mọi khung. Bề mặt marker loá về phía đèn, nên cạnh phía ấy bị ăn mất theo cùng một cách.

Và bây giờ là mẹo. Với ba trong số đó, cái quyết định hướng lệch là *vận tốc*. Đảo chiều chuyển động thì lệch đổi dấu. Đứng yên thì lệch bằng không. Nghĩa là có một nút bấm xoá được chúng, và nút ấy tên là “dừng lại”.

6.2 Rolling shutter: tag bị xé, không chỉ bị mờ

Hiệu ứng tệ nhất trong danh sách khi robot đang chuyển động, và cũng là hiệu ứng dễ phòng nhất — bằng cách mua đúng camera.

Cảm biến rolling shutter đọc từng hàng nối tiếp. Hàng thứ k được phơi sáng muộn hơn hàng đầu một khoảng $k t_{\text{line}}$, với t_{line} là thời gian đọc một hàng — thường cỡ hàng chục micro giây. Với cảm biến 960 hàng và $t_{\text{line}} = 20 \mu\text{s}$ (*bậc độ lớn kinh nghiệm, không phải số đo của một cảm biến cụ thể*), độ trễ giữa hàng đầu và hàng cuối là $960 \times 20 \mu\text{s} = 19,2 \text{ ms}$.

Hậu quả: nếu cảnh chuyển động trong khoảng thời gian ấy, các hàng khác nhau của ảnh ghi lại cảnh ở các thời điểm khác nhau. Một tag vuông chuyển động ngang trở thành một hình bình hành; một tag đang quay trở thành một hình méo phi tuyến.

Điều quan trọng là *đây không phải mờ*. Mờ làm cạnh nhoè nhưng giữ đúng vị trí trung bình; rolling shutter làm cạnh *dịch chỗ* theo một hàm của toạ độ dọc. Detector vẫn tìm thấy một tứ giác đẹp, decode vẫn thành công, sai số tái chiếu vẫn nhỏ — và pose thì sai. Đây là một chế độ hỏng hoàn toàn im lặng, đúng loại nguy hiểm nhất.

Với robot chạy $v = 0,3 \text{ m/s}$ ở khoảng cách $Z = 0,5 \text{ m}$ với $f = 600 \text{ px}$, vận tốc biểu kiến trên ảnh là $v f / Z = 0,3 \times 600 / 0,5 = 360 \text{ px/s}$. Trong $19,2 \text{ ms}$, đó là $360 \times 0,0192 = 6,9 \text{ px}$ lệch giữa hàng đầu và hàng cuối của ảnh. Với một tag chiếm 120 hàng thay vì cả 960, phần lệch trong nội bộ tag là $6,9 \times 120 / 960 = 0,86 \text{ px}$ — nhỏ hơn nhưng vẫn gấp bốn lần $\sigma = 0,2 \text{ px}$, và nó là thiên lệch.

Ba cách xử lý, xếp theo thứ tự tôi khuyên.

Một, dùng global shutter. Đây là câu trả lời đúng và nó gần như bắt buộc nếu bạn phải đo trong khi di chuyển. Giá chênh lệch so với rolling shutter cùng phân khúc là nhỏ so với chi phí một tuần gỡ lỗi hiện tượng này.

Hai, đo khi đứng yên. Nếu chiến lược điều khiển là stop-and-go (B/13) thì rolling shutter không còn là vấn đề, và bạn tiết kiệm được tiền camera.

Ba, mô hình hoá và bù. Đưa vận tốc vào mô hình đo và giải cả pose lẫn vận tốc từ một ảnh [37]. Nó hoạt động và nó thậm chí biến một phiên toái thành một nguồn thông tin, nhưng nó đòi biết t_{line} chính xác (đo được theo [38]) và nó làm bài toán phức tạp hơn hẳn. Chỉ nên dùng khi hai cách trên bị loại vì ràng buộc ngoài.

6.3 Motion blur: nhoè có hướng, và nó gây thiên lệch

Hiệu ứng thứ hai của chuyển động, và nó độc lập với rolling shutter — global shutter không cứu được nó.

Trong thời gian phơi sáng t_e , ảnh của tag di chuyển trên cảm biến một đoạn

$$L = v_{\text{ảnh}} t_e = \frac{v f}{Z} t_e.$$

Với các con số của Q1 — $v = 0,3 \text{ m/s}$, $f = 600 \text{ px}$, $Z = 0,5 \text{ m}$, $t_e = 20 \text{ ms}$:

$$L = \frac{0,3 \times 600}{0,5} \times 0,02 = 360 \times 0,02 = 7,2 \text{ px}.$$

Bảy phẩy hai pixel. Đặt cạnh $\sigma = 0,2 \text{ px}$, đây là một hiệu ứng lớn hơn ba mươi lần nhiều.

Phần thứ hai của Q1 — vì sao nó gây thiên lệch chứ không chỉ mờ — đáng dừng lại, vì đây là chỗ trực giác thường sai. Trực giác nói: nhòe là một phép làm mượt đối xứng, nên vị trí trung bình của cạnh không đổi, nên định vị cạnh vẫn không thiên lệch. Trực giác ấy đúng trong một trường hợp lý tưởng: một cạnh cô lập, với vùng sáng và vùng tối trải dài vô hạn về hai phía.

Trong thực tế nó không đúng, vì hai lý do. Thứ nhất, các cạnh của tag *không cô lập* — một ô đen rộng 5 px nằm giữa hai ô trắng, và khi nhòe 7 px thì hai cạnh của ô đen chồng lên nhau, và profile độ sáng thu được không còn đối xứng quanh vị trí cạnh thật. Thứ hai, detector không tìm cạnh bằng cách lấy trung tâm đối xứng mà bằng cách khớp một mô hình lên profile gradient có trọng số [7]; khi profile bị biến dạng bất đối xứng thì mô hình khớp lệch.

Ghi chú trung thực: tôi dẫn cơ chế thiên lệch này bằng lập luận về tính bất đối xứng của profile (cửa a) và tôi tin nó đúng về cấu trúc. Tôi *chưa* đo biên độ thiên lệch bằng mô phỏng (cửa b) và *chưa* tìm được nguồn phát biểu tường minh đúng dạng này cho trường hợp marker (cửa c). Vì vậy phần định lượng nên coi là ước lượng của tôi. Đây là một mục trong 99-chua-biet.md, và nó đo được: mô phỏng một tag, tích phân ảnh qua một đoạn dịch chuyển, chạy detector, so với chân trị.

Điều làm motion blur thành một dấu hiệu chẩn đoán mạnh — và đây là Q2 — là nó *đổi dấu khi đổi chiều chuyển động*. Nếu bạn đo cùng một điểm khi robot đang tiến và khi đang lùi, và hai kết quả lệch nhau có hệ thống theo hai chiều ngược nhau, thì bạn gần như chắc chắn đang nhìn một hiệu ứng phụ thuộc vận tốc — blur hoặc rolling shutter. Không nguồn sai số nào khác trong cả cây có chữ ký ấy: méo ảnh không quan tâm chiều chuyển động, sai số constellation không quan tâm, hand-eye không quan tâm. Đây là một phép thử rẻ và rất thông tin, và nó nên nằm trong quy trình nghiệm thu ở C/16.

Cách xử lý: *giảm* t_e , và điều đó đòi nhiều ánh sáng hơn — dẫn thẳng tới lập luận về marker chiếu sáng chủ động ở B/14. Hoặc *giảm* v , tức stop-and-go. Hoặc chấp nhận và đưa vào ngân sách như một dòng phụ thuộc vận tốc — một dòng khó chịu vì nó không phải hằng số.

6.4 Lấy mẫu, răng cưa, và ngưỡng kích thước tag

Cảm biến lấy mẫu ảnh quang học thành một lưới pixel rời rạc, và định lý lấy mẫu đặt ra một giới hạn cứng: chi tiết có chu kỳ nhỏ hơn hai pixel không khôi phục được, và tệ hơn, nó xuất hiện lại dưới dạng mẫu hình giả.

Với marker, chi tiết nhỏ nhất là một ô bit. Một tag AprilTag họ 36h11 có 6×6 ô dữ liệu cộng viên, tức khoảng 8×8 ô trên toàn cạnh. Nếu tag chiếm w pixel thì mỗi ô chiếm $w/8$ pixel. Theo Nyquist thì cần tối thiểu 2 px mỗi ô, tức $w \geq 16 \text{ px}$ — nhưng đó là giới hạn lý thuyết cho tín hiệu sạch, và thực tế cần nhiều hơn.

Con số thực hành thường thấy là 3–5 px mỗi ô bit, tức w cỡ 24–40 px cho việc decode. (Đây là bậc độ lớn kinh nghiệm, không phải số đo có điều kiện xác định.) Nhưng decode được không có nghĩa là đo được: A/02 mục 4 chỉ ra rằng số điều kiện của bài toán xấu đi khi tag nhỏ, và A/05 chỉ ra sai số tỉ lệ $1/w$. Ngưỡng cho đo chính xác cao hơn ngưỡng cho decode đáng kể.

Hệ quả thiết kế trực tiếp, và nó là một trong những quyết định quan trọng nhất ở giai đoạn thiết kế: chọn số ô bit ít nhất có thể. Một họ tag với 4×4 ô dữ liệu cần ít pixel hơn một họ 6×6 để đọc được ở cùng khoảng cách, đổi lại số ID khả dụng ít hơn và khoảng cách Hamming nhỏ hơn. Với bài toán docking, số ID cần thiết thường rất nhỏ — vài chục là quá đủ — nên đánh đổi ấy gần như luôn nghiêng về phía ít ô hơn. Đây là một chỗ mà lựa chọn mặc định (dùng họ phổ biến nhất) thường không phải lựa chọn tốt nhất.

6.5 Quang học: MTF, nhiễu xạ, và độ sâu trường ảnh

Ba hiệu ứng quang học liên quan chặt với nhau, và chúng cùng nhau đặt một trần cứng lên σ mà không thuật toán nào vượt qua được.

A. MTF: cạnh không bao giờ sắc tuyệt đối

Ống kính thật không tạo ra một cạnh chuyển từ trắng sang đen trong không tới một pixel; nó tạo ra một chuyển tiếp trải qua vài pixel. Mức độ trải ấy được mô tả bởi MTF, và nó tệ hơn ở rìa ảnh, tệ hơn khi khẩu mở to, và tệ hơn ở tần số không gian cao.

Ảnh hưởng lên định vị góc thì tinh tế. Một cạnh trải rộng không nhất thiết làm định vị kém đi — ngược lại, một cạnh trải qua vài pixel cho nhiều pixel mang thông tin gradient hơn, nên khớp đường có thể chính xác hơn một cạnh nhảy bậc trong một pixel. Điều này nghe phản trực giác nhưng nó là lý do vì sao ảnh hơi mờ đôi khi cho subpixel tốt hơn ảnh cực sắc.

Chỗ MTF thật sự gây hại là khi nó không đồng đều: nếu một cạnh của tag nằm ở vùng ảnh sắc nét và cạnh đối diện nằm ở vùng mờ hơn, hai cạnh ấy được định vị với độ chính xác khác nhau và có thể với thiên lệch khác nhau. Đây là một nguồn cho việc Σ_i khác nhau giữa các góc ở A/03 mục 6.

B. Nhiễu xạ: trần vật lý

Kể cả một ống kính hoàn hảo cũng không hội tụ ánh sáng vào một điểm, vì bản chất sóng của ánh sáng. Điểm nhỏ nhất là đĩa Airy, đường kính

$$d_{\text{Airy}} \approx 2,44 \lambda N,$$

với λ bước sóng và N số f [39]. Với ánh sáng trắng ($\lambda \approx 550$ nm) và $N = 8$:

$$d = 2,44 \times 0,55 \mu\text{m} \times 8 = 10,7 \mu\text{m}.$$

Với pixel 3 μm , đĩa Airy trải qua hơn ba pixel. Nghĩa là ở $N = 8$, camera của bạn không thể phân giải chi tiết một pixel, bất kể cảm biến có bao nhiêu megapixel.

Đây là câu trả lời cho Q5 và cho một nửa của Q3. Khép khẩu làm tăng độ sâu trường ảnh — điều bạn muốn — nhưng nó cũng tăng N nên tăng đường kính Airy, làm ảnh nhoè đi vì nhiễu xạ. Tồn tại một khẩu độ tối ưu, và khép quá thì tệ hơn.

Đây cũng là một nửa câu trả lời cho Q3: tăng độ phân giải không giúp gì nếu quang học đã là nút thắt. Nửa còn lại là nhiễu ảnh — pixel nhỏ hơn thu ít photon hơn nên tỉ số tín hiệu trên nhiễu kém hơn. Phép thử phân biệt hai cơ chế: chụp cùng một cảnh với thời gian phơi sáng dài hơn và nhiều ánh sáng hơn. Nếu σ cải thiện thì nút thắt là nhiễu ảnh; nếu không thì nút thắt là quang học.

C. Độ sâu trường ảnh: ràng buộc thiết kế thật

Trong bài toán docking, robot đi từ vài mét xuống nửa mét, và constellation có tag ở các độ sâu khác nhau (theo lời khuyên của [A/04](#)). Không phải mọi thứ đều nét cùng lúc.

Ba lựa chọn. *Khoá nét ở khoảng cách cuối* — đơn giản nhất và thường đúng, vì đó là chỗ cần chính xác nhất; ở xa thì tag hơi mờ nhưng vẫn decode được và bạn chỉ cần điều khiển thô. *Khép khẩu* — tăng độ sâu trường ảnh, giới hạn bởi nhiễu xạ như mục B. *Lấy nét tự động* — tránh, vì nó đổi f và hệ số méo ([A/01](#) mục 8), làm toàn bộ hiệu chuẩn của bạn vô hiệu.

CẦN NHỚ

Quang học đặt một *trần cứng* lên σ , và trần ấy không vượt qua được bằng phần mềm.

Hệ quả cho việc lập ngân sách: trước khi hứa hẹn $\sigma = 0,05$ px, hãy kiểm xem quang học của bạn có phân giải nổi mức đó không. Phép kiểm rẻ: chụp một cạnh sắc, vẽ profile độ sáng vuông góc cạnh, và xem chuyển tiếp trải qua bao nhiêu pixel. Nếu nó trải qua 5 px thì hy vọng định vị tới 0,05 px là lạc quan.

Hệ quả cho việc mua sắm: một ống kính tốt hơn thường là đồng tiền hiệu quả hơn một cảm biến nhiều megapixel hơn, vì cảm biến chỉ giúp khi quang học chưa phải nút thắt.

6.6 Vật liệu và bề mặt marker

Hiệu ứng cuối cùng, và nó là hiệu ứng duy nhất mà bạn kiểm soát hoàn toàn bằng lựa chọn vật liệu.

Phản xạ gương và loá. Một bề mặt bóng phản xạ nguồn sáng thành một vùng sáng chói. Nếu vùng ấy trùm lên một cạnh tag, cạnh bị “ăn” và góc lệch — một thiên lệch phụ thuộc góc chiếu sáng, tức phụ thuộc tư thế robot. Cách chữa: vật liệu nhám (matte), và tránh bố trí sao cho có đường phản xạ gương từ đèn tới camera.

Retroreflective. Vật liệu phản xạ ngược về nguồn. Với một đèn gắn cạnh camera, nó cho tương phản rất cao và gần như độc lập với ánh sáng môi trường — một lợi thế lớn vì nó làm σ ổn định. Nhược điểm: nó rất nhạy với góc, và ở góc chéo lớn thì hiệu quả giảm; ngoài ra vùng sáng có thể tràn (blooming) sang pixel lân cận, làm cạnh dịch.

Backlit và IR chủ động. Tag phát sáng từ phía sau, hoặc chiếu sáng bằng LED hồng ngoại cộng bộ lọc bandpass trên camera. Đây là cách mạnh nhất để làm σ ổn định, vì nó loại bỏ gần như hoàn toàn ảnh hưởng của ánh sáng môi trường. [B/14](#) bàn kỹ; ở đây chỉ ghi nhận rằng nó tồn tại và rằng lợi ích chính của nó là *tính ổn định* chứ không phải độ chính xác đỉnh.

Độ phẳng và độ cứng của nền. Không phải quang học nhưng thuộc cùng nhóm quyết định vật liệu, và nó quan trọng hơn cả ba mục trên với ngân sách 5 mm. Một tờ giấy in dán lên bề mặt cong lệch khỏi mặt phẳng vài milimét, và [A/02](#) mục 7 đã chỉ ra rằng khi ấy không có homography nào khớp được. Đây là một sai số hệ thống thuần túy cơ khí, và nó thường là dòng lớn nhất trong bảng ngân sách của [C/15](#).

6.7 Tổng kết: nhiễu hay thiên lệch, và cái gì biến mất khi dừng

Bảng [6.1](#) là thứ đáng mang ra khỏi chương nhất.

Bảng 6.1. Sáu hiệu ứng: loại sai số, phụ thuộc vận tốc, và cách xử lý chính.

Hiệu ứng	Loại	Hết khi dừng?	Cách xử lý chính
Rolling shutter	Thiên lệch	Có	Global shutter; hoặc stop-and-go; hoặc mô hình hoá [37]
Motion blur	Thiên lệch	Có	Giảm t_e (cần nhiều sáng hơn); hoặc stop-and-go
Loá phụ thuộc góc	Thiên lệch	Một phần	Vật liệu nhám; bố trí tránh phản xạ gương
Nhoè do lấy nét	Thiên lệch	Không	Khoá nét ở khoảng cách cuối; khép khẩu vừa phải
Răng cưa khi tag nhỏ	Cả hai	Không	Ngưỡng kích thước tối thiểu; nested multi-scale (B/09)
Bias tâm ellipse	Thiên lệch	Không	Hiệu chỉnh conic [15]; hoặc dùng góc chessboard

Đọc cột thứ ba là đọc được câu trả lời cho Q4. Hai hiệu ứng lớn nhất khi robot chuyển động — rolling shutter và motion blur — đều biến mất hoàn toàn khi vận tốc bằng không, và loá thì giảm vì không còn đổi góc. Đây là lập luận vật lý cho stop-and-go, và nó mạnh hơn lập luận thông thường (“dừng lại để trung bình nhiều khung”): trung bình nhiều khung chỉ giảm *nhiều*, còn dừng lại thì xoá hẳn mấy dòng *thiên lệch* — thứ mà trung bình không bao giờ giảm được.

Hai hiệu ứng không biến mất khi dừng là nhoè do lấy nét và bias tâm ellipse. Cả hai đều xử lý được ở tầng thiết kế: khoá nét đúng khoảng cách, và dùng target có góc thay vì target tròn cho khâu đo tinh.

6.8 Giới hạn và chế độ hỏng

Chương này không đầy đủ. Danh sách sáu hiệu ứng là danh sách những thứ tôi cho là đáng kể với bài toán docking ở mức 5 mm; nó không phải danh sách đầy đủ các hiệu ứng vật lý trong một hệ thị giác. Không bàn tới ở đây: hiện tượng nở vệt sáng (blooming) trong trường hợp nặng, méo do khí quyển (không đáng kể ở cự ly mét), nhiễu đọc và nhiễu nhiệt của cảm biến (chúng vào σ và được xử lý như nhiễu ngẫu nhiên), và hiệu ứng của nén ảnh nếu camera trả về JPEG (tránh — dùng ảnh thô).

Các con số là bậc độ lớn. $t_{\text{line}} = 20 \mu\text{s}$, ngưỡng 3–5 px mỗi ô bit, $\lambda = 550 \text{ nm}$ cho ánh sáng trắng — đây là giá trị điển hình dùng để thấy bậc, không phải số đo của thiết bị cụ thể. Mọi con số milimét dẫn ra từ chúng thừa hưởng cùng mức độ tin cậy ấy.

Phân loại nhiễu/thiên lệch là đơn giản hoá. Một số hiệu ứng có cả hai thành phần. Răng cưa chẳng hạn: nó vừa làm tăng phương sai vừa gây thiên lệch phụ thuộc pha của lưới tag so với lưới pixel. Bảng 6.1 ghi “cả hai” cho nó, nhưng cách xử lý đúng cho từng thành phần khác nhau và chương này không tách chúng ra.

Mệnh đề về thiên lệch do blur chưa được kiểm. Đã ghi rõ ở mục 3. Nếu hoá ra blur chỉ gây nhiễu chứ không gây thiên lệch, thì tầm quan trọng của stop-and-go giảm đi một phần — không hết, vì rolling shutter vẫn còn.

6.9 Thực hành

Bốn việc, theo thứ tự chi phí tăng dần.

Một, làm phép thử đảo chiều. Đo cùng một điểm khi tiến và khi lùi. Nếu thấy lệch có hệ thống theo hai chiều ngược nhau thì bạn có hiệu ứng phụ thuộc vận tốc, và bạn biết ngay phải nhìn vào

rolling shutter và blur. Phép thử này gần như miễn phí và nó là phép thử thông tin nhất trong cả chương.

Hai, đo profile cạnh. Chụp một cạnh sắc, vẽ độ sáng dọc theo một đường vuông góc, đếm xem chuyển tiếp trải qua bao nhiêu pixel. Con số ấy cho bạn một ước lượng thực tế về trần của σ , và nó nói cho bạn biết có nên đầu tư vào subpixel tinh vi hơn không.

Ba, khoá mọi thứ khoá được. Khoá exposure, khoá gain, khoá white balance, khoá nét — rồi dán keo vào vòng lấy nét. Mọi tham số tự động là một nguồn thay đổi không kiểm soát được trong một hệ đo lường.

Bốn, chọn global shutter ngay từ đầu nếu có bất kỳ khả năng nào phải đo trong khi di chuyển. Đây là quyết định mua sắm chứ không phải quyết định kỹ thuật, và sửa nó về sau đắt hơn nhiều.

Active recall (che cột phải, tự trả lời, rồi mở ra đối chiếu)

Thuật ngữ / câu hỏi	Trả lời
••• Nguyên tắc tổ chức của chương	Phân loại theo <i>nhiều hay thiên lệch</i> , vì nhiều chia cho $\sqrt{N}$ còn thiên lệch thì không (A/05 mục 8). Gần như mọi hiệu ứng vật lý thuộc loại thứ hai.
••• Rolling shutter làm gì	Các hàng ảnh ghi cảnh ở các thời điểm khác nhau, nên tag chuyển động bị <i>xé</i> chứ không chỉ mờ. Detector vẫn thấy tứ giác đẹp, decode vẫn được, residual vẫn nhỏ — pose sai im lặng.
••• Độ lớn rolling shutter	$960 \text{ hàng} \times t_{\text{line}} = 20 \mu\text{s} = 19,2 \text{ ms}$ độ trễ. Với $v = 0,3 \text{ m/s}$, $f/Z = 1200$: 6,9 px trên cả ảnh, 0,86 px trong nội bộ tag 120 hàng — gấp bốn lần σ , và là thiên lệch.
••• Chiều dài vệt motion blur	$L = v f t_e / Z$. Ví dụ: $0,3 \text{ m/s}$, $f = 600$, $Z = 0,5 \text{ m}$, $t_e = 20 \text{ ms} \Rightarrow 7,2 \text{ px}$ — lớn hơn σ ba mươi lần.
•• Vì sao blur gây thiên lệch chứ không chỉ mờ	Vì cạnh tag <i>không cô lập</i> : ô đen 5 px nhoe 7 px thì hai cạnh chồng lên nhau và profile mất đối xứng; và detector khớp mô hình lên profile gradient chứ không lấy trung tâm đối xứng. (<i>Tôi suy ra; chưa kiểm bằng mô phỏng.</i>)
••• Phép thử đảo chiều	Đo cùng một điểm khi tiến và khi lùi. Lệch có hệ thống theo hai chiều ngược nhau $\Rightarrow$ hiệu ứng phụ thuộc vận tốc. Không nguồn nào khác trong cây có chữ ký này — méo, constellation, hand-eye đều không quan tâm chiều chuyển động.
••• Bốn trong sáu hiệu ứng hết khi dừng	Rolling shutter, motion blur hết hoàn toàn; loá giảm. Đây là lập luận vật lý cho stop-and-go, và nó mạnh hơn lập luận “dừng để trung bình” — trung bình chỉ giảm nhiều, dừng thì xoá thiên lệch.
•• Ngưỡng pixel mỗi ô bit	Nyquist cho 2 px; thực tế 3–5 px (<i>bậc độ lớn kinh nghiệm</i>). Với tag 8×8 ô thì w cỡ 24–40 px để <i>decode</i> . Ngưỡng để <i>đo chính xác</i> cao hơn nhiều.
•• Hệ quả thiết kế của ngưỡng ô bit	Chọn họ tag ít ô nhất có thể. Docking chỉ cần vài chục ID, nên đánh đổi gần như luôn nghiêng về phía ít ô — và lựa chọn mặc định (họ phổ biến nhất) thường không tối ưu.
••• Đường kính đĩa Airy	$d \approx 2,44\lambda N$ [39]. Với $\lambda = 550 \text{ nm}$, $N = 8$: $10,7 \mu\text{m}$ — hơn ba pixel $3 \mu\text{m}$. Ở khẩu ấy, không cảm biến nào phân giải nổi chỉ tiết một pixel.
•• Đánh đổi khẩu độ	Khép khẩu tăng độ sâu trường ảnh nhưng tăng N nên tăng đĩa Airy. Có khẩu tối ưu; khép quá thì nhoe hơn.
•• Cạnh hơi mờ có thể tốt hơn cạnh sắc	Vì cạnh trải qua vài pixel cho nhiều pixel mang thông tin gradient hơn, nên khớp đường chính xác hơn. Chỗ MTF thật sự hại là khi nó <i>không đồng đều</i> giữa các cạnh của cùng một tag.
•• Phân biệt nút thất quang học với nút thất nhiễu ảnh	Chụp lại với nhiều sáng hơn và t_e dài hơn. σ cải thiện $\Rightarrow$ nhiễu ảnh là nút thất. Không cải thiện $\Rightarrow$ quang học là nút thất, và thêm megapixel vô ích.

Thuật ngữ / câu hỏi	Trả lời (tiếp)
• Vật liệu marker	Nhám chống loá; retroreflective cho tương phản cao nhưng nhạy góc và dễ blooming; backlit/IR cho σ ổn định nhất (B/14). Và quan trọng hơn cả ba: nền phải cứng và phẳng.

• • • phải thuộc • • nên nắm • biết là có

I Kiến thức này đang sống ở đâu (công trình đang dùng chính ý tưởng của chương; chốt 9/2026)

Công trình / hệ thống	Dùng ý tưởng nào của chương	Ghi chú
Mô hình rolling shutter [37]	Giải cả pose lẫn vận tốc từ một ảnh, biến phiên toái thành nguồn thông tin	Chỉ dùng khi không thể đổi camera hoặc dừng robot
Hiệu chuẩn t_{line} [38]	Đo độ trễ dòng bằng một target đã biết	Bước bắt buộc trước khi mô hình hoá rolling shutter
Hiệu chỉnh bias tâm [15]	Dùng đại số conic thay vì lấy tâm ellipse	Bắt buộc nếu dùng marker tròn — xem A/02 mục 6
Marker IR chủ động cộng bandpass deltille [12]	Loại ảnh hưởng ánh sáng môi trường, làm σ ổn định Target lưới tam giác; góc saddle không chịu bias kiểu ellipse	Lợi ích chính là <i>ổn định</i> , không phải độ chính xác đỉnh — B/14 Lựa chọn khi cần vừa chính xác vừa không thiên lệch

MINI PROJECT • Đo thiên lệch phụ thuộc vận tốc bằng phép thử đảo chiều • 1–2 ngày

Mục tiêu	tách bạch phần sai số phụ thuộc vận tốc khỏi mọi nguồn khác, bằng một phép thử mà không nguồn nào khác có thể giả mạo.
Dữ liệu	phần cứng thật: một camera, một tag gắn cố định, và một cách di chuyển camera (hoặc tag) với vận tốc kiểm soát được — một trục trượt, hoặc chính robot của bạn.
Các bước	chọn một điểm mốc trên đường đi; đo pose tại điểm ấy trong ba điều kiện: đứng yên (trung bình 50 khung), đang tiến qua nó với vận tốc v , đang lùi qua nó với cùng v ; lặp cho ba giá trị v khác nhau và cho hai giá trị t_e khác nhau; vẽ sai lệch so với giá trị đứng yên theo v , tách riêng cho tiến và lùi.
Xong khi	bạn có một đồ thị trong đó đường “tiến” và đường “lùi” đối xứng nhau qua trục hoành và cả hai đi qua gốc — chữ ký không thể nhầm của hiệu ứng phụ thuộc vận tốc. Độ dốc của chúng là hệ số bạn đưa vào ngân sách nếu buộc phải đo khi di chuyển. Kiểm tra chéo bắt buộc: lặp lại với t_e giảm một nửa; nếu độ dốc giảm khoảng một nửa thì thủ phạm chính là motion blur, nếu nó gần như không đổi thì thủ phạm là rolling shutter — hai cách chữa khác nhau.
Nối với	B/13 cho quyết định stop-and-go; C/15 nhận độ dốc này làm một dòng phụ thuộc vận tốc trong ngân sách.

TRẢ LỜI CÂU HỎI MỞ ĐẦU

- A1. Vệt dài $L = vft_e/Z = 0,3 \times 600 \times 0,02/0,5 = 7,2$ px — lớn hơn nhiễu định vị góc chừng ba mươi lần. Nó gây thiên lệch chứ không chỉ mờ vì hai lý do. Thứ nhất, các cạnh của tag không cô lập: một ô đen rộng 5 px bị nhoe 7 px thì hai cạnh của nó chồng lên nhau, và profile độ sáng thu được không còn đối xứng quanh vị trí cạnh thật. Thứ hai, detector không tìm cạnh bằng cách lấy trung tâm đối xứng mà bằng cách khớp một mô hình lên profile gradient có trọng số, nên một profile bị biến dạng bất đối xứng cho một đường khớp lệch. Hệ quả thực hành: lấy trung bình nhiều khung *không* chữa được, vì mọi khung đều lệch theo cùng hướng.
- A2. Motion blur hoặc rolling shutter — cả hai đều phụ thuộc vận tốc và cả hai đều đổi dấu khi đổi chiều. Đây là một dấu hiệu chẩn đoán rất mạnh vì *không nguồn sai số nào khác trong cả cây có chữ ký ấy*: méo ảnh không quan tâm robot đang đi hướng nào, sai số constellation không quan tâm,

sai số hand-eye không quan tâm, ambiguity thì gây nhảy chứ không gây lệch có hướng. Vì vậy nếu bạn thấy chữ ký này thì bạn đã thu hẹp không gian nguyên nhân xuống còn hai, và có một phép thử thứ hai tách hai cái đó: giảm thời gian phơi sáng một nửa — nếu lệch giảm khoảng một nửa thì là blur, nếu gần như không đổi thì là rolling shutter.

- A3. Cơ chế thứ nhất: quang học đã là nút thắt. Đĩa Airy ở $N = 8$ có đường kính $10,7 \mu\text{m}$, trải qua hơn ba pixel $3 \mu\text{m}$, nên camera không phân giải nổi chi tiết một pixel bất kể cảm biến bao nhiêu megapixel — pixel nhỏ hơn chỉ lấy mẫu dày hơn một hình ảnh vốn đã nhòe. Cơ chế thứ hai: nhiều ảnh. Pixel nhỏ hơn có diện tích thu photon nhỏ hơn, nên tỉ số tín hiệu trên nhiễu kém đi, nên σ tính bằng pixel tăng lên đúng bằng phần mà độ phân giải cải thiện. Phép thử phân biệt: chụp lại với nhiều ánh sáng hơn và thời gian phơi sáng dài hơn. Nếu σ cải thiện thì nút thắt là nhiễu ảnh và bạn nên đầu tư vào chiếu sáng; nếu không cải thiện thì nút thắt là quang học và bạn nên đầu tư vào ống kính.
- A4. Vì hai hiệu ứng lớn nhất khi di chuyển — rolling shutter và motion blur — đều là *thiên lệch* và đều tỉ lệ với vận tốc, nên chúng biến mất hoàn toàn ở $v = 0$; loá phụ thuộc góc cũng ổn định hơn vì góc không đổi nữa. Lập luận này mạnh hơn lập luận thông thường “dừng lại để trung bình nhiều khung”, vì trung bình chỉ giảm *nhiều* theo $1/\sqrt{N}$ còn dừng lại thì xoá hẳn mấy dòng *thiên lệch* — thứ mà trung bình không bao giờ chạm tới được. Hai hiệu ứng không biến mất khi dừng là nhòe do lấy nét (xử lý bằng cách khoá nét ở khoảng cách cuối) và bias tâm ellipse (xử lý bằng hiệu chỉnh conic, hoặc bằng cách dùng góc chessboard thay cho marker tròn).
- A5. Vì khep khẩu tăng số f N , mà đường kính đĩa Airy tỉ lệ thuận với N : $d \approx 2,44\lambda N$. Khep từ $N = 4$ xuống $N = 16$ làm đĩa Airy to gấp bốn, từ khoảng $5,4 \mu\text{m}$ lên $21,5 \mu\text{m}$ — tức từ chừng hai pixel lên bảy pixel với pixel $3 \mu\text{m}$. Giới hạn vật lý đang chặn bạn là *nhiều xạ*: bản chất sóng của ánh sáng, không phải chất lượng chế tạo ống kính, nên không có ống kính đắt tiền nào vượt qua được. Kết quả là tồn tại một khẩu độ tối ưu cân bằng giữa độ sâu trường ảnh và độ sắc nét do nhiễu xạ, và với bài toán docking thì một lựa chọn thường tốt hơn là bỏ hẳn cuộc đua ấy: khoá nét ở khoảng cách cuối — nơi cần chính xác nhất — và chấp nhận tag hơi mờ ở xa, nơi chỉ cần điều khiển thô.

NÓI LẠI THẬT ĐƠN GIẢN

Mọi thứ cho tới chương này đều giả vờ rằng cái camera là một thiết bị hoàn hảo chụp một thế giới đứng yên. Chương này liệt kê những chỗ giả vờ ấy bị lộ.

Thứ đáng lo nhất không phải là ảnh trông xấu. Ảnh xấu thì ai cũng thấy và ai cũng biết phải sửa. Thứ đáng lo là những sai lệch làm con số lệch đi mà không làm ảnh trông khác gì — và tệ hơn nữa, lệch theo *cùng một hướng* ở mọi lần chụp.

Hãy nghĩ về chuyện bắn bia. Nếu các phát đạn tản mát quanh tâm thì bắn nhiều rồi lấy trung bình là xong. Nhưng nếu súng bị lệch ngắm thì bắn nghìn phát lấy trung bình chỉ cho bạn biết rất chính xác cái chỗ lệch ấy ở đâu — bạn tự tin hơn về một câu trả lời sai. Gần như mọi thứ trong chương này thuộc loại lệch ngắm.

Có hai thủ phạm lớn nhất, và cả hai chỉ xuất hiện khi robot đang đi. Thủ phạm thứ nhất là chuyện nhiều loại camera không chụp cả tấm ảnh cùng một lúc mà quét từ trên xuống dưới như máy scan — nên nếu vật đang chạy ngang thì hàng dưới ghi lại nó ở chỗ khác với hàng trên, và cái hình vuông biến thành hình xiên. Đáng sợ ở chỗ cái hình xiên ấy vẫn trông rất gọn gàng, máy vẫn đọc được, không có gì báo động, và pose thì sai. Thủ phạm thứ hai là vết nhòe: cửa chụp mở hai mươi mili giây trong khi mọi thứ đang trôi, nên cạnh của tag bị kéo thành một vết.

Và đây là tin tốt, tốt hơn nhiều so với công sức bỏ ra để sửa chúng: cả hai thủ phạm ấy *biến mất* nếu robot đứng yên. Không phải giảm đi, mà biến mất. Cho nên cách chữa rẻ nhất trong cả chương là: đi tới gần, dừng hẳn lại một phần tư giây, chụp mấy chục tấm, tính toán, rồi mới đi tiếp. Không cần camera đắt hơn, không cần thuật toán mới.

Hai thứ còn lại không chữa được bằng cách dừng. Một là chuyện ống kính chỉ nét ở một khoảng cách — cái này chữa bằng cách chỉnh nét đúng chỗ mà bạn cần chính xác nhất, tức chỗ sắp cập vào, rồi dán keo lại đừng cho ai vặn. Hai là chuyện cái tâm của hình tròn không nằm ở chỗ bạn tưởng — cái này chữa bằng cách dùng loại marker có góc thay vì loại có vòng tròn.

Còn một chuyện cuối, và nó là chuyện vật lý chứ không phải kỹ thuật. Ánh sáng có bản chất sóng, và vì thế không có ống kính nào — kể cả ống kính hoàn hảo, kể cả ống kính đắt nhất — hội tụ được

ánh sáng vào một điểm. Điểm nhỏ nhất có thể là một cái chấm nhỏ, và kích thước của nó thì tính được. Nếu cái chấm ấy đã rộng ba pixel thì mua cảm biến nhiều pixel hơn không giúp gì cả — bạn chỉ đang lấy mẫu dày hơn một cái chấm.

Chỗ không rút gọn được: việc tách bạch xem một sai lệch cụ thể là nhiễu hay lệch ngấm thì không nhìn ra được từ một lần đo, và đôi khi một hiệu ứng có cả hai phần. Cách duy nhất là đổi một điều kiện có chủ ý — đổi chiều đi, đổi thời gian chụp, đổi ánh sáng — rồi xem cái gì đổi theo.

Một câu: Gần như mọi thứ trong chương này làm sủng lệch ngấm chứ không làm tay run — và hai cái lệch lớn nhất thì biến mất nếu bạn chịu đứng yên một phần tư giây.

6.10 Điều gì chứng minh cách hiểu này sai

Mệnh đề chính — *rolling shutter và motion blur gây thiên lệch phụ thuộc vận tốc, đổi dấu khi đổi chiều* — bị bác bỏ nếu phép thử đảo chiều ở mini project cho thấy hai đường tiến và lùi trùng nhau thay vì đối xứng. Với rolling shutter thì mệnh đề này gần như chắc chắn đúng vì nó là hệ quả trực tiếp của cơ chế đọc hàng; với blur thì nó là suy luận của tôi và chưa được kiểm.

Mệnh đề thứ hai — *blur gây thiên lệch chứ không chỉ tăng phương sai* — là mệnh đề yếu nhất về kiểm chứng trong chương, như đã ghi ở mục 3. Nó bị bác bỏ nếu mô phỏng (tích phân ảnh tag qua một đoạn dịch chuyển, chạy detector, so với chân trị) cho thấy sai lệch trung bình bằng không trong phạm vi sai số thống kê. Nếu vậy thì stop-and-go vẫn đáng làm vì rolling shutter, nhưng lập luận mất một chân.

Mệnh đề thứ ba — *quang học đặt trần cứng lên σ mà phần mềm không vượt được* — là mệnh đề chắc chắn nhất vì nó là vật lý cơ bản [39]. Nó không bị bác bỏ bởi một thuật toán tốt hơn; nó chỉ bị “bác bỏ” nếu ai đó chỉ ra rằng bạn tính sai số f hoặc bước sóng.

Cuối cùng, một mệnh đề mềm: chương này khẳng định rằng *trong một hệ đã tối ưu tốt phần nhiều, ngân sách bị chi phối bởi các dòng trong chương này*. Điều đó bị bác bỏ nếu trong một hệ thực, sau khi đã chuyển sang stop-and-go và global shutter, sai số vẫn không cải thiện đáng kể — khi ấy nút thắt nằm ở chỗ khác, nhiều khả năng là sai số cơ khí của constellation (B/12) hoặc hand-eye. (Tôi phát biểu dựa trên cấu trúc của ngân sách chứ không dựa trên thống kê hệ thực tế.)

6.11 Kết nối

Chương này khép Phần A bằng cách nói ra những gì năm chương trước im lặng bỏ qua, và nó mở đường sang những quyết định thiết kế của Phần B.

Nói lên trên. A/01-mo-hình-camera-va-distortion dựng mô hình hình học; chương này liệt kê những thứ mô hình ấy không chứa. Đáng chú ý là hai chương bổ sung nhau theo một cách sạch: chương 1 nói về sai lệch *tĩnh* của quang học (đo một lần, bù mãi mãi), chương này nói về sai lệch *động* (đổi theo vận tốc, khoảng cách, ánh sáng). A/02-homography-va-hình-học-phang mục 6 cấp bias tâm ellipse, một trong sáu hiệu ứng ở đây. A/05-lan-truyen-sai-so-pixel-sang-pose mục 8 cấp khung phân loại nhiễu/thiên lệch mà cả chương này dựa vào, và nó cũng nhận lại từ đây một cảnh báo quan trọng: công thức hiệp phương sai của chương 5 hoàn toàn mù với thiên lệch, nên một hệ chịu các hiệu ứng ở đây sẽ báo cáo một bất định nhỏ hơn sự thật.

Nói xuống Phần B. B/08-corner-refinement nhận từ mục 5 luận điểm rằng cạnh hơi mờ có thể tốt hơn cạnh sắc, và nhận từ mục 3 cảnh báo rằng mọi kỹ thuật subpixel tinh vi đều vô ích nếu ảnh đang bị nhoè do chuyển động. B/13-docking-control nhận mục 7 làm cơ sở vật lý cho stop-and-go — đây là liên kết quan trọng nhất của chương, vì nó biến một danh sách hiệu ứng thành

một quyết định kiến trúc. [B/14-dieu-kien-quan-sat-va-anh-sang](#) nhận toàn bộ mục 6 và mở rộng nó thành hướng dẫn về chiếu sáng, vật liệu, và cấu hình camera.

Nổi sang Phần C. [C/15-ngan-sach-sai-so](#) có một nhóm dòng riêng cho các hiệu ứng của chương này, và chúng có một đặc điểm làm chúng khó chịu trong bảng: phần lớn *không phải hằng số* mà phụ thuộc điều kiện vận hành. Cách xử lý đúng trong ngân sách là ghi chúng theo hai cột — giá trị khi đang di chuyển và giá trị khi dừng — và chính sự chênh lệch giữa hai cột ấy là lập luận định lượng cho stop-and-go. [C/16-danh-gia-va-nghiem-thu](#) nhận phép thử đảo chiều ở mục 9 làm một mục bắt buộc trong quy trình nghiệm thu.

I Câu hỏi tự kiểm (trả lời không nhìn bài)

1. Vì sao rolling shutter gây một chế độ hỏng “im lặng” trong khi motion blur thì ít nhất còn nhìn thấy được trên ảnh?
2. Nêu chữ ký chẩn đoán phân biệt hiệu ứng phụ thuộc vận tốc với mọi nguồn sai số khác trong cây, và giải thích vì sao không nguồn nào khác giả mạo được nó.
3. Bạn đã có chữ ký ấy. Nêu phép thử thứ hai tách motion blur khỏi rolling shutter, và dự đoán kết quả cho từng trường hợp.
4. Vì sao một cạnh trải qua ba pixel có thể cho định vị subpixel tốt hơn một cạnh nhảy bậc trong một pixel — và khi nào thì MTF thật sự gây hại?
5. Với $\lambda = 550 \text{ nm}$ và pixel $3 \mu\text{m}$, ở số f nào thì đĩa Airy bắt đầu vượt hai pixel? Tính nhẩm, rồi nói kết luận thực hành.
6. Bốn trong sáu hiệu ứng biến mất khi dừng. Nêu hai cái còn lại, và với mỗi cái nêu cách xử lý ở tầng thiết kế chứ không ở tầng thuật toán.
7. Một hệ đã dùng global shutter, stop-and-go, và trung bình 50 khung. Sai số vẫn 4 mm. Chương này còn giải thích được bao nhiêu phần, và bạn chuyển sang nghi ngờ chương nào?

GIẢI THUẬT VÀ PHƯƠNG PHÁP

TÓM TẮT

Phần B là phần dài nhất và nó trả lời một câu hỏi duy nhất, hỏi tám lần ở tám chỗ khác nhau: *với nguồn sai số này, ta có những cách nào để giảm nó, và mỗi cách trả giá bằng gì?* Tám chương đi theo đúng thứ tự của chuỗi đo — phát hiện, tinh chỉnh góc, hình học constellation, xử lý hai nghiệm, tích hợp theo thời gian, hiệu chuẩn, điều khiển, và điều kiện quan sát — nên đọc tuần tự là cách hợp lý. Nếu chỉ nhớ một điều: *ba chương có đòn bẩy lớn nhất là chương 9 (constellation), chương 12 (calibration) và chương 13 (docking control)*, và cả ba đều không phải chương về thuật toán xử lý ảnh.

Có một cách đọc Phần B khiến nó trở nên mạch lạc thay vì trở thành một danh sách kỹ thuật, và nó đáng nói ngay. Mỗi chương ở đây tương ứng với đúng một dòng trong bảng ngân sách sai số ở chương 15. Chương 7 và 8 quyết định σ , tức nhiều định vị góc trên ảnh — dòng đầu tiên của ngân sách. Chương 9 quyết định hình học của bài toán, tức hệ số nhân biến σ thành sai số pose; đây là dòng có đòn bẩy lớn nhất. Chương 10 quyết định ta có bị một nghiệm sai đánh lừa hay không — một loại sai số không phải nhiều mà là lỗi thô, và nó không nằm trong ngân sách mà phá hỏng cả ngân sách. Chương 11 làm giảm σ bằng thời gian thay vì bằng quang học. Chương 12 quyết định các hằng số của mô hình có đúng không — và một hằng số sai thì mọi thứ phía sau chỉ là chính xác quanh một giá trị sai. Chương 13 quyết định phần sai số nào thật sự đi tới mũi robot, và nó chứa mẹo lớn nhất của cây. Chương 14 nói về điều kiện vật lý làm σ ổn định hay không ổn định.

Trật tự này có một hệ quả đáng chú ý về chiến lược làm việc. Nếu bạn đọc theo thứ tự đòn bẩy giảm dần thay vì theo thứ tự chuỗi đo, thì thứ tự sẽ là 13, 9, 12, rồi mới tới 8, 10, 11, 14, 7. Nói cách khác: *quyết định kiến trúc điều khiển trước, rồi bố trí hình học, rồi hiệu chuẩn, và chỉ sau đó mới tới chuyện chọn detector*. Đây gần như đúng ngược với thứ tự mà phần lớn dự án thực tế đi, và đó là lý do phần lớn dự án thực tế dừng ở một, hai độ.

Một lưu ý về chương 10. Nó ngắn hơn các chương khác và nội dung của nó gần như hoàn toàn là hệ quả của chương 4 và chương 9. Tôi vẫn tách nó ra thành chương riêng vì một lý do thực dụng: xử lý ambiguity là việc phải làm ở tầng code, với những quyết định rất cụ thể — ngưỡng tỉ số residual bằng bao nhiêu, từ chối cập nhật hay chấp nhận, chọn nghiệm nào khi cả hai đều khả dĩ — và trộn chúng vào chương lý thuyết sẽ làm cả hai chương khó dùng.

Bảng 6.4. Bản đồ Phần B: các chương, dòng ngân sách tương ứng, và mức đầu tư.

Ch.	Thư mục	Nội dung — và nó quyết định dòng nào của ngân sách	Mức
7	07-detection-pipeline	AprilTag 2/3, ArUco, ChArUco, Deltile, marker tròn, detector học sâu. <i>Quyết định: có phát hiện được không, và σ thô là bao nhiêu</i>	●●
8	08-corner-refinement	cornerSubPix, saddle point, line fit có trọng số, moment/Zernike, conic, ECC. <i>Quyết định: σ — và σ quyết định mọi thứ</i>	●●●
9	09-multi-marker-const	PnP hợp nhất, bố trí không đồng phẳng, board, nested multi-scale, BA để đo constellation, trọng số, loại ngoại lai. <i>Quyết định: hệ số nhân hình học — đòn bẩy lớn nhất</i>	●●●
10	10-xu-ly-ambiguity	Hai nghiệm IPPE và tỉ số residual, ngưỡng từ chối, nhất quán thời gian, fuse IMU, đường tiếp cận nghiêng. <i>Quyết định: có bị lỗi thô đánh lừa không</i>	●●●
11	11-uoc-luong-theo-tho	Trung bình trên đa tạp, EKF trên $SE(3)$ với đo là góc, factor graph, fixed-lag, hiệp phương sai từ Hessian. <i>Quyết định: σ giảm theo $1/\sqrt{N}$</i>	●●
12	12-calibration-cho-do	Intrinsic chất lượng cao, kiểm residual có cấu trúc, chọn mô hình méo, hand-eye, đo constellation bằng BA, trôi nhiệt, tự hiệu chuẩn. <i>Quyết định: các hằng số có đúng không</i>	●●●
13	13-docking-control	Teach-by-showing, IBVS, PBVS, hybrid, stop-and-go, coarse-to-fine, nonholonomic, dung sai cơ khí. <i>Quyết định: sai số nào thật sự tới mũi robot</i>	●●●
14	14-dieu-kien-quan-sat	Chống loá, backlit/IR, khoá exposure, global shutter, che khuất và bản, chống báo sai. <i>Quyết định: σ có ổn định không</i>	●●

Đọc tuần tự nếu muốn hiểu chuỗi đo. Nhưng nếu đang phải ra quyết định thiết kế, hãy đọc theo thứ tự đòn bẩy: 13 trước (nó có thể làm nhẹ toàn bộ phần còn lại), rồi 9, rồi 12. Chương 7 đặt đầu vì nó là chỗ chuỗi bắt đầu, không phải vì nó quan trọng nhất — thực tế nó là chương có đòn bẩy thấp nhất trong tám chương, và điều đó đáng ngạc nhiên đủ để nói thẳng ra.

Detection pipeline (Marker Detection)

PHỤ THUỘC A/01-mo-hinh-camera-va-distortion (mục 4 — vì sao méo chưa khử phá hoại chính bước khớp cạnh mà chương này mô tả); A/02-homography-va-hinh-hoc-phang (homography để rectify vùng tag trước khi đọc bit).

TÓM TẮT

Chương này mô tả quy trình biến một ảnh xám thành một danh sách tag kèm tọa độ góc, và mục đích của nó không phải để bạn tự cài lại — các cài đặt sẵn có đều tốt — mà để bạn biết *sai số sinh ra ở bước nào*. Ba bước đáng chú ý: ngưỡng hoá quyết định tag có được phát hiện không, khớp cạnh quyết định σ , và giải mã quyết định tỉ lệ báo sai. Điều đáng nói thẳng ngay: *đây là chương có đôn bẩy thấp nhất trong Phần B*. Chọn detector nào ảnh hưởng tới σ chừng hai tới ba lần, trong khi bố trí constellation ảnh hưởng tới sai số góc hàng chục lần (B/09). Nếu chỉ nhớ một điều: *AprilTag xuất sắc ở việc nhận diện và bắt bám, không xuất sắc ở việc định vị góc* — và sự phân biệt ấy dẫn thẳng tới đề xuất dùng target lại ở B/08.

KIẾN THỨC CẦN CÓ TRƯỚC

- **ngưỡng hoá thích nghi (adaptive threshold)** — biến ảnh xám thành ảnh nhị phân với ngưỡng tính *cục bộ* theo từng vùng thay vì một ngưỡng chung. Cần thiết vì chiếu sáng không đều: một ngưỡng chung làm mất tag ở vùng tối hoặc vùng sáng.
- **thành phần liên thông (connected component)** — tập các pixel cùng giá trị nhị phân và liền kề nhau. Tìm chúng hiệu quả bằng cấu trúc *union-find*, và chúng là ứng viên cho các vùng tag.
- **khớp tứ giác (quad fitting)** — tìm bốn cạnh và bốn góc từ một thành phần liên thông có hình dạng gần tứ giác. Đây là bước quyết định σ .
- **khớp đường có trọng số gradient** — thay vì dùng các pixel biên của ảnh nhị phân, dùng các pixel có gradient lớn trong ảnh *xám gốc*, với trọng số theo độ lớn gradient. Đây là bước làm AprilTag 2 chính xác hơn hẳn phiên bản đầu [7].
- **rectify** — dùng homography đưa vùng tứ giác trên ảnh về một hình vuông chuẩn, để việc lấy mẫu các ô bit trở thành việc đọc một lưới đều A/02.
- **khoảng cách Hamming** — số vị trí bit khác nhau giữa hai từ mã. Một từ điển có khoảng cách tối thiểu d thì sửa được $\lfloor (d-1)/2 \rfloor$ lỗi bit và phát hiện được $d-1$ lỗi.
- **tỉ lệ báo sai (false positive rate)** — xác suất một vùng ảnh ngẫu nhiên được giải mã thành một ID hợp lệ. Nó là đại lượng an toàn quan trọng nhất của một hệ thống marker, và nó đánh đổi trực tiếp với khả năng phát hiện ở khoảng cách xa.

CÂU HỎI MỞ ĐẦU

- Q1. Detector của bạn báo phát hiện được tag ID 7 ở một vị trí mà bạn biết chắc không có tag nào. Nêu hai cơ chế hoàn toàn khác nhau có thể gây ra điều đó, và nêu hai biện pháp phòng ở hai tầng khác nhau.
- Q2. AprilTag 2 chính xác hơn AprilTag 1 đáng kể, và cải tiến chính nằm ở đúng một bước của pipeline. Bước nào, và vì sao thay đổi ấy lại quan trọng đến thế?
- Q3. ChArUco dùng ArUco để định danh nhưng *không* dùng góc của ArUco để đo. Vì sao lại chia việc như vậy, và điều đó nói gì về điểm mạnh riêng của mỗi loại?

- Q4.** Bạn chọn họ tag có nhiều ô bit hơn để có nhiều ID hơn và khoảng cách Hamming lớn hơn. Nếu cái giá phải trả, và tính xem nó ảnh hưởng tới khoảng cách phát hiện tối đa như thế nào.
- Q5.** Trong tám chương của Phần B, chương này có đòn bẩy thấp nhất đối với ngân sách sai số. Vì sao điều đó đúng, và vì sao nó vẫn đáng đọc?

7.1 Đặt vấn đề

Mọi thứ trong cây bắt đầu từ một danh sách: tag nào nhìn thấy, và bốn góc của mỗi tag nằm ở đâu trên ảnh. Chương này mô tả cách danh sách ấy được tạo ra.

Nhưng mục đích của chương không phải để bạn tự cài lại một detector. Các cài đặt có sẵn — `apriltag`, `OpenCV objdetect` — đều tốt, được kiểm nghiệm rộng rãi, và tối ưu hơn nhiều so với thứ bạn viết trong một tuần. Mục đích là để bạn biết *sai số sinh ra ở bước nào*, vì ba lý do thực dụng.

Lý do thứ nhất: các bước khác nhau hỏng theo những cách khác nhau, và triệu chứng khác nhau. Ngưỡng hoá hỏng thì tag *biến mất* hoàn toàn. Khớp cạnh hỏng thì tag vẫn được phát hiện nhưng σ lớn. Giải mã hỏng thì bạn có báo sai hoặc bỏ sót. Ba triệu chứng ấy đòi ba cách chữa hoàn toàn khác nhau, và nhầm lẫn giữa chúng làm mất nhiều thời gian.

Lý do thứ hai: có một vài tham số trong các cài đặt sẵn có ảnh hưởng thật, và biết pipeline là biết chỉnh cái nào. Ngưỡng kích thước tối thiểu, có bật tinh chỉnh cạnh hay không, số lỗi bit cho phép sửa — ba tham số này đáng chỉnh và phần còn lại thường nên để mặc định.

Lý do thứ ba, và nó là lý do quan trọng nhất cho bài toán 5 mm: hiểu pipeline cho biết *giới hạn* của từng loại marker. Cụ thể, nó giải thích vì sao góc AprilTag đến từ giao hai đường thẳng còn góc chessboard đến từ một điểm yên ngựa, và vì sao hai cơ chế ấy cho độ chính xác khác nhau. Kết luận ấy dẫn thẳng tới đề xuất target lai ở `B/08`, và nó là một trong ba luận điểm “nói ngược thói quen” ở `00-map`.

Cần nói thẳng một điều ngay từ đầu, vì nó ảnh hưởng tới việc bạn nên đầu tư bao nhiêu thời gian vào chương này: *đây là chương có đòn bẩy thấp nhất trong Phần B*. Chuyển từ detector này sang detector khác thay đổi σ chừng hai tới ba lần. Chuyển từ một tag đơn sang constellation có baseline thay đổi sai số góc hàng chục lần (`A/05` mục 6). Với một dự án có thời gian hạn chế, thứ tự đầu tư đúng là `B/13`, `B/09`, `B/12`, rồi mới tới đây. Tôi đặt chương này ở đầu Phần B vì nó là chỗ chuỗi đo bắt đầu, không phải vì nó quan trọng nhất — và sự khác biệt ấy đáng được nói ra thay vì để người đọc tự suy.

TRỰC GIÁC

Hình dung việc tìm một cái biển báo trong một bức ảnh chụp phố.

Việc đầu tiên là tách sáng khỏi tối. Nhưng không thể dùng một ngưỡng chung cho cả ảnh, vì một góc phố có nắng và một góc có bóng râm — cái ngưỡng đúng cho vùng này thì sai cho vùng kia. Nên phải hỏi từng vùng nhỏ một: ở *quanh đây*, cái gì là sáng và cái gì là tối? Việc thứ hai là gom các pixel tối liên nhau thành từng mảng, rồi hỏi mảng nào trông giống một hình tứ giác. Phần lớn bị loại ngay — quá nhỏ, quá méo, quá nhiều lỗ.

Việc thứ ba là chỗ quan trọng nhất, và cũng là chỗ tinh tế nhất. Ta đã có một mảng pixel hình tứ giác, nhưng cái ta cần là *toạ độ bốn góc, chính xác tới phần mười pixel*. Cách ngây thơ là lấy bốn pixel ngoài cùng của mảng — và cách đó tệ, vì nó chỉ dùng bốn pixel và vứt bỏ toàn bộ thông tin trong hàng trăm pixel dọc các cạnh.

Cách tốt là: nhìn vào *ảnh xám gốc* chứ không phải ảnh đen trắng, tìm các pixel nơi độ sáng đổi nhanh nhất, kẻ một đường thẳng qua chúng theo cách tốt nhất, rồi lấy chỗ hai đường cắt nhau. Bây giờ mỗi góc được quyết định bởi hàng trăm pixel thay vì một pixel — và đó là toàn bộ lý do độ chính xác dưới pixel là chuyện có thể.

Việc cuối là đọc mã. Nấn cái tứ giác trở lại thành hình vuông, chia thành lưới ô, hỏi từng ô sáng hay tối, rồi tra xem chuỗi bit ấy có trong danh sách hợp lệ không. Bước tra ấy quan trọng hơn vẻ ngoài: nó là thứ ngăn một vết bẩn ngẫu nhiên trên tường được báo cáo là một cái tag.

7.2 AprilTag: bốn bước và chỗ σ sinh ra

Mô tả theo phiên bản 2 và 3 [7], [40], [41], vì đó là những phiên bản đang dùng.

Bước một, ngưỡng hoá thích nghi. Chia ảnh thành các ô nhỏ, tính cực đại và cực tiểu độ sáng trong mỗi ô cùng các ô lân cận, và đặt ngưỡng ở giữa. Các ô có độ tương phản thấp bị đánh dấu là “không xác định” và bỏ qua — một chi tiết quan trọng vì nó ngăn việc khuếch đại nhiễu trong vùng đồng nhất.

Bước hai, phân đoạn thành thành phần liên thông. Dùng union-find để gom các pixel cùng giá trị thành từng nhóm, rồi tìm các *biên* giữa vùng sáng và vùng tối. Các biên này là ứng viên cho cạnh tag.

Bước ba, khớp tứ giác. Nhóm các đoạn biên thành các chuỗi khép kín, kiểm xem chuỗi có thể xấp xỉ bằng bốn đoạn thẳng không, và loại các ứng viên quá nhỏ hoặc quá méo. Đây là chỗ tham số `min_tag_size` hoặc tương đương có tác dụng, và nó là tham số đáng chỉnh theo A/02 mục 4.

Bước bốn, và đây là bước quyết định σ : khớp đường có trọng số gradient. Thay vì dùng các pixel biên của ảnh nhị phân — vốn chỉ chính xác tới một pixel — AprilTag 2 quay lại ảnh xám gốc, lấy các pixel dọc mỗi cạnh, và khớp một đường thẳng với trọng số theo độ lớn gradient tại từng pixel [7]. Bốn góc là giao của bốn đường thẳng ấy.

Đây là câu trả lời cho Q2, và lý do cải tiến này quan trọng đến thế đáng nói rõ. Với ảnh nhị phân, thông tin về vị trí cạnh bị lượng tử hoá thành một pixel và mọi thông tin về *độ sáng* bị vứt bỏ. Với ảnh xám và trọng số gradient, một cạnh đi qua giữa hai pixel để lại dấu vết ở tỉ lệ độ sáng của hai pixel ấy, và thông tin đó khôi phục được vị trí cạnh tới phần nhỏ của một pixel. Cộng thêm việc dùng cả trăm pixel dọc cạnh để xác định một đường, và độ chính xác cải thiện đáng kể.

Sau đó là giải mã: rectify bằng homography (A/02), lấy mẫu các ô bit, so với từ điển với sửa lỗi Hamming, và xác định hướng (một hình vuông có bốn hướng khả dĩ, và mã phải phân biệt được chúng — các họ tag được thiết kế sao cho bốn phép xoay của một từ mã không trùng với từ mã nào khác).

CẦN NHỚ

Bốn góc AprilTag đến từ *giao của bốn đường thẳng*. Ba hệ quả quan trọng của sự thật ấy, và cả ba đều xuất hiện lại ở các chương khác.

Một, độ chính xác phụ thuộc vào *chiều dài cạnh* chứ không chỉ vào kích thước tag — một cạnh dài cho nhiều pixel gradient hơn nên đường khớp tốt hơn.

Hai, sai số ở giao điểm bị *khuếch đại* khi hai đường gần song song, tức khi tag nhìn rất chếch. Đây là một nguồn làm Σ_i khác nhau giữa các góc (A/03 mục 6).

Ba, và đây là hệ quả nghiêm trọng nhất: *giả định “cạnh là đường thẳng” phải đúng*. Méo chưa khử làm cạnh cong, và khớp thẳng lên cong cho đường sai (A/01 mục 4). Đây là lý do chương 1 phải đọc trước chương này.

7.3 ArUco và ChArUco

A. ArUco

Quy trình khác ở chi tiết nhưng cùng cấu trúc: ngưỡng hoá thích nghi, tìm contour, xấp xỉ contour bằng đa giác và giữ các đa giác bốn cạnh, rectify bằng homography, lấy mẫu bit, tra từ điển [14].

Đóng góp riêng của ArUco không nằm ở phần phát hiện mà ở phần *từ điển*: các tác giả đề xuất sinh từ điển bằng cách tối ưu hoá khoảng cách Hamming giữa các từ mã và giữa các phép xoay của chúng, cho phép tạo từ điển với kích thước và mức bền tùy chọn [42]. Đây là một lợi thế thực dụng: bạn chọn được từ điển nhỏ với khoảng cách lớn, đúng thứ cần cho docking nơi số ID cần thiết rất ít.

Về độ chính xác định vị góc, cài đặt cơ bản của ArUco dùng góc của đa giác xấp xỉ, kém hơn cơ chế khớp đường có trọng số của AprilTag. OpenCV có tùy chọn tinh chỉnh góc bằng cornerSubPix sau đó, và *nên bật nó* — đây là một trong vài tham số đáng chỉnh.

B. ChArUco: chia việc theo điểm mạnh

Đây là thiết kế đáng chú ý nhất trong mục này và nó trả lời Q3.

ChArUco là một bàn cờ vua với các ô ArUco đặt trong các ô trắng. Quy trình: phát hiện các marker ArUco để biết *đang nhìn phần nào của bàn cờ* (định danh), rồi dự đoán vị trí các góc chessboard và tinh chỉnh chúng bằng thuật toán điểm yên ngựa (đo).

Sự chia việc ấy khai thác đúng điểm mạnh của từng loại. ArUco *giải định danh*: nó cho biết ô nào là ô nào, kể cả khi chỉ nhìn thấy một phần bàn cờ — điều mà một bàn cờ thuần không làm được vì mọi góc trông giống nhau. Chessboard *giải đo*: góc của nó là một điểm yên ngựa với cấu trúc đối xứng hai chiều, và định vị một điểm như thế chính xác hơn định vị giao của hai đường thẳng.

Vì sao chính xác hơn? B/08 bàn kỹ, nhưng bản chất là: một điểm yên ngựa có gradient đổi dấu theo *cả hai* hướng quanh nó, nên vị trí của nó bị ràng buộc từ bốn phía; một giao điểm chỉ bị ràng buộc bởi hai đường, và sai số của mỗi đường lan vào giao điểm với hệ số khuếch đại phụ thuộc góc giao.

Đây chính là cơ sở cho đề xuất *target lai* ở 00-map: dùng AprilTag để bắt bám và định danh từ xa — nơi nó xuất sắc — và một vùng chessboard hoặc ChArUco cho phép đo tinh ở cự ly gần — nơi độ chính xác góc quyết định.

7.4 Các họ marker khác

Ngắn gọn, vì với bài toán docking thì chúng phần lớn là tham khảo.

Deltile [12]: lưới tam giác thay vì lưới vuông. Lập luận: một đỉnh tam giác được bao quanh bởi sáu vùng thay vì bốn, nên có nhiều thông tin hơn để định vị, và các tác giả trình bày một phép tinh chỉnh saddle point bậc hai riêng cho nó. Đáng biết khi cần hiệu chuẩn intrinsic chất lượng cao nhất.

Marker tròn đồng tâm: WhyCon [19] và CCTag [18]. Định vị tâm rất chính xác vì tâm của một vùng lớn được xác định bởi rất nhiều pixel biên. Nhưng — và đây là cảnh báo phải lặp lại — chúng chịu bias tâm ellipse (A/02 mục 6), một thiên lệch hệ thống phụ thuộc tư thế. Với bài toán 5 mm, một phép đo rất chính xác quanh một giá trị thiên lệch không bằng một phép đo kém chính xác hơn mà không thiên lệch, trừ khi bạn hiệu chỉnh conic đúng cách.

STag [30]: thêm một đường tròn ngoại tiếp vào thiết kế tag để ổn định pose. Triết lý giống chương này: giải quyết ở tầng thiết kế marker thay vì tầng thuật toán.

Detector học sâu: DeepTag [43], DeepArUco [44]. Bền hơn với mờ, che khuất, chiếu sáng khắc nghiệt. Nhưng *độ chính xác góc chưa chắc hơn* — đây là một nhận định quan trọng và tôi phát biểu nó một cách thận trọng: các công trình ấy tập trung vào tỉ lệ phát hiện trong điều kiện khó, và độ chính xác định vị góc dưới điều kiện tốt không phải trọng tâm của chúng. *(Tôi không có dữ liệu so sánh được để khẳng định chiều nào tốt hơn; tôi chỉ nói rằng không nên giả định rằng bền hơn thì cũng chính xác hơn.)* Với docking — nơi điều kiện kiểm soát được và độ chính xác là mục tiêu — lợi ích của chúng ít rõ ràng hơn so với các ứng dụng ngoài trời.

Bảng 7.1. Các họ marker: cơ chế định vị góc và vai trò trong bài toán docking.

Họ	Cơ chế định vị	Vai trò cho docking
AprilTag	Giao của bốn đường khớp có trọng số gradient [7]	Nhận diện và bắt bám từ xa; tỉ lệ báo sai rất thấp
ArUco	Góc đa giác, nên bật tính chỉnh cornerSubPix	Tương tự AprilTag; ưu thế ở từ điển tùy chỉnh được [42]
ChArUco	ArUco định danh, saddle point chessboard để đo	Đo tinh ở cự ly gần; nửa còn lại của target lai
Deltille [12]	Saddle point trên lưới tam giác	Hiệu chuẩn intrinsic chất lượng cao (B/12)
Tròn đồng tâm	Tâm ellipse, cần hiệu chỉnh conic	σ rất nhỏ nhưng có thiên lệch phụ thuộc tư thế — cần trọng
Học sâu	Học từ dữ liệu	Bền với điều kiện khó; độ chính xác góc chưa rõ hơn

7.5 Mã hoá, khoảng cách Hamming, và tỉ lệ báo sai

Mục này trả lời Q1 và Q4, và nó là mục về *an toàn* chứ không về độ chính xác.

A. Vì sao báo sai nguy hiểm hơn bỏ sót

Trong hệ docking, hai loại lỗi có hậu quả rất khác nhau. *Bỏ sót* — không phát hiện được tag đang có — làm robot chờ hoặc retry; phiền nhưng an toàn. *Báo sai* — phát hiện một tag không tồn tại, hoặc giải mã sai ID — làm robot lái về một chỗ sai với sự tự tin đầy đủ. Hậu quả có thể là va chạm.

Đây là lý do mọi họ tag hiện đại đầu tư nặng vào mã hoá, và tại sao AprilTag được thiết kế với tỉ lệ báo sai rất thấp làm mục tiêu thiết kế chính [40].

B. Cơ chế mã hoá

Từ điển tag là một tập từ mã nhị phân được chọn sao cho khoảng cách Hamming tối thiểu giữa hai từ mã bất kỳ — và giữa mọi phép xoay của chúng — lớn nhất có thể. Tên họ tag mã hoá thông tin này: 36h11 nghĩa là 36 bit dữ liệu (lưới 6×6) với khoảng cách Hamming tối thiểu 11.

Từ lý thuyết mã: với khoảng cách tối thiểu d , sửa được $t = \lfloor (d - 1)/2 \rfloor$ lỗi bit và phát hiện được tới $d - 1$ lỗi. Với 36h11: sửa được 5 bit, phát hiện được 10.

Tham số thực hành quan trọng nhất ở đây là *số lỗi bit cho phép sửa* — các thư viện cho phép đặt nó. Đặt cao thì phát hiện được tag bản hoặc mờ, đổi lại tỉ lệ báo sai tăng. Với docking, nơi bạn biết trước có đúng những ID nào, khuyến nghị là: *đặt số lỗi cho phép ở mức thấp và bù lại bằng whitelist ID*. Whitelist gần như miễn phí và nó giảm tỉ lệ báo sai theo tỉ lệ giữa số ID bạn dùng và kích thước từ điển.

C. Đánh đổi số ô bit và khoảng cách phát hiện

Đây là phần thứ hai của Q4, và nó là một phép tính đáng làm.

Một tag có $n \times n$ ô dữ liệu cộng viền, tổng khoảng $(n + 2) \times (n + 2)$ ô trên cạnh. Để giải mã tin cậy cần chừng 3–5 px mỗi ô (A/06 mục 4), nên kích thước ảnh tối thiểu là $w_{\min} \approx 4(n + 2)$ px.

So sánh hai họ: 16h5 (4×4 ô dữ liệu, tổng 6×6) cần $w_{\min} \approx 24$ px; 36h11 (6×6 , tổng 8×8) cần $w_{\min} \approx 32$ px.

Vì $w = fW/Z$, khoảng cách phát hiện tối đa tỉ lệ nghịch với $w_{\min}$:

$$\frac{Z_{\max}(16h5)}{Z_{\max}(36h11)} = \frac{32}{24} = 1,33.$$

Tức họ ít ô hơn cho tầm phát hiện xa hơn 33% với cùng kích thước tag vật lý — hoặc, nhìn ngược lại, cho phép dùng tag nhỏ hơn 33% cho cùng tầm.

Với docking, số ID cần thiết thường dưới vài chục, trong khi 36h11 có hàng trăm. Nên đánh đổi gần như luôn nghiêng về phía *ít ô hơn*, và mất mát về khoảng cách Hamming được bù bằng whitelist. Đây là một chỗ mà lựa chọn mặc định — dùng 36h11 vì nó là mặc định của thư viện — không phải lựa chọn tốt nhất, và biết pipeline là biết điều đó.

CẦN NHỚ

Ba tham số đáng chỉnh trong một detector, và phần còn lại nên để mặc định.

Kích thước tag tối thiểu: đặt theo ngưỡng đo được của A/02 mục 4, không theo mặc định. Từ chối phát hiện dưới ngưỡng thay vì trả về một pose kém.

Tính chỉnh góc: bật. Với ArUco đây là tùy chọn và nó tắt mặc định ở một số phiên bản. Với AprilTag thì cơ chế khớp đường đã có sẵn.

Số lỗi bit cho phép sửa: đặt thấp, và bù bằng whitelist ID. An toàn quan trọng hơn tầm phát hiện thêm vài chục phân.

Và một quyết định đứng trên cả ba: *chọn họ tag ít ô bit nhất đủ dùng*. Docking cần vài chục ID, không cần hàng trăm.

7.6 Giới hạn và chế độ hồng

Ngưỡng hoá hồng khi tương phản thấp hoặc không đều. Triệu chứng: tag biến mất hoàn toàn, thường ở một số vùng ảnh hoặc một số điều kiện chiếu sáng. Chữa ở tăng chiếu sáng (B/14), không ở tăng thuật toán.

Khớp tứ giác nhầm với vật thể khác. Cửa sổ, biển báo, hoạ tiết sàn — bất cứ thứ gì gần tứ giác — đều là ứng viên. Đây là một trong hai cơ chế của Q1, và nó được lọc ở bước giải mã.

Giải mã sai do nhiễu bit. Cơ chế thứ hai của Q1. Với số lỗi cho phép cao và từ điển lớn, xác suất một vùng ngẫu nhiên giải mã thành ID hợp lệ không phải là không đáng kể. Chữa: giảm số lỗi cho phép, whitelist ID, và — ở tầng cao hơn — kiểm tính hợp lý về pose (một tag báo ở khoảng cách 50 m hoặc ở góc nghiêng 89° thì đáng ngờ).

Sai hướng khi giải mã. Hình vuông có bốn hướng, và các họ tag được thiết kế để phân biệt chúng — nhưng một tag rất mờ hoặc rất nhỏ có thể giải mã đúng ID mà sai hướng. Hậu quả: pose sai đúng 90° với sai số tái chiếu vẫn nhỏ (A/03 mục 8). Bắt bằng kiểm nhất quán liên tag (B/09 mục 7).

Chương này không so sánh định lượng các detector. Đã nói ở mục 4: tôi không có dữ liệu so sánh được. Có công trình so sánh thực nghiệm nhiều họ tag [45], và nó là nơi đáng đọc nếu cần con số — nhưng con số ở đó gắn với điều kiện đo của họ, và việc chuyển sang cấu hình của bạn thì không hiển nhiên. Cách đúng là đo σ trên chính hệ của mình (C/16).

7.7 Thực hành

Bốn việc.

Một, đo σ của chính hệ bạn thay vì tra bảng. Cách đo: cố định camera và tag, chụp vài trăm khung, tính độ lệch chuẩn của toạ độ từng góc. Con số ấy là đầu vào cho mọi phép tính ở C/15, và nó là con số duy nhất trong chuỗi mà chương này ảnh hưởng tới.

Hai, chỉnh ba tham số ở mục 5C và để phần còn lại mặc định.

Ba, bật whitelist ID và ghi log mỗi lần một ID ngoài whitelist được phát hiện — tần suất ấy là thước đo trực tiếp của tỉ lệ báo sai trong môi trường thật của bạn.

Bốn, cân nhắc target lai nếu ngân sách góc chặt: AprilTag để bắt bám, vùng ChArUco để đo tinh. B/08 bàn đầy đủ.

Active recall (che cột phải, tự trả lời, rồi mở ra đối chiếu)

Thuật ngữ / câu hỏi	Trả lời
••• Bốn bước của pipeline AprilTag	Ngưỡng hoá thích nghi; phân đoạn thành phần liên thông (union-find); khớp tứ giác; khớp đường có trọng số gradient rồi lấy giao [7]. Bước bốn quyết định σ .
••• Cải tiến chính của AprilTag 2	Quay lại <i>ảnh xám gốc</i> thay vì ảnh nhị phân, khớp đường với trọng số theo độ lớn gradient. Ảnh nhị phân lượng tử vị trí cạnh thành một pixel và vứt bỏ thông tin độ sáng; ảnh xám giữ lại nó nên khôi phục được vị trí dưới pixel.
••• Góc AprilTag đến từ đâu và ba hệ quả	Từ <i>giao của bốn đường thẳng</i> . Hệ quả: độ chính xác phụ thuộc chiều dài cạnh; sai số bị khuếch đại khi hai đường gần song song (tag chéo); và giả định “cạnh thẳng” phải đúng nên méo chưa khử là chí mạng (A/01 mục 4).
••• ChArUco chia việc thế nào và vì sao	ArUco <i>định danh</i> (biết đang nhìn phần nào của bàn cờ, kể cả khi chỉ thấy một phần); góc chessboard để <i>đo</i> (saddle point có gradient đổi dấu theo cả hai hướng nên bị ràng buộc từ bốn phía, chính xác hơn giao hai đường).
••• Cơ sở của target lai	AprilTag xuất sắc ở nhận diện và bắt bám từ xa; chessboard xuất sắc ở định vị góc. Dùng cả hai theo đúng điểm mạnh — đây là một trong ba luận điểm “nói ngược thói quen” ở 00-map.

Thuật ngữ / câu hỏi	Trả lời (tiếp)
●●● Vì sao báo sai nguy hiểm hơn bỏ sót	Bỏ sót $\Rightarrow$ robot chờ hoặc retry, an toàn. Báo sai $\Rightarrow$ robot lái về chỗ sai với sự tự tin đầy đủ, có thể va chạm. Đây là lý do mã hoá được đầu tư nặng [40].
●● Tên họ tag mã hoá gì	$36h11 = 36$ bit dữ liệu (lưới 6×6) với khoảng cách Hamming tối thiểu 11. Sửa được $\lfloor (d-1)/2 \rfloor = 5$ bit, phát hiện được $d-1 = 10$.
●●● Đánh đổi số ô bit	$w_{\min} \approx 4(n+2)$ px. 16h5 cần 24 px, 36h11 cần 32 px $\Rightarrow$ họ ít ô cho tầm phát hiện xa hơn 33%. Docking chỉ cần vài chục ID nên đánh đổi nghiêng về ít ô, bù bằng whitelist.
●●● Ba tham số đáng chỉnh	Kích thước tag tối thiểu (theo ngưỡng đo được, không theo mặc định); bật tinh chỉnh góc (ArUco tắt mặc định ở một số phiên bản); số lỗi bit cho phép <i>thấp</i> , bù bằng whitelist ID.
●● Hai cơ chế gây báo sai	Khớp tứ giác nhầm vật thể khác (cửa sổ, biển báo, hoạ tiết sàn); và giải mã sai do nhiễu bit với số lỗi cho phép cao. Hai biện pháp ở hai tầng: giảm số lỗi cho phép, và whitelist ID cộng kiểm hợp lý pose.
●● Chế độ hóng thâm lặng nhất của chương	Giải mã đúng ID nhưng <i>sai hướng</i> : pose lệch đúng 90° mà sai số tái chiếu vẫn nhỏ vì hình vuông đối xứng bốn lần. Chỉ bắt được bằng kiểm nhất quán liên tag (B/09 mục 7).
●● Marker tròn: cảnh báo	σ rất nhỏ (tâm xác định bởi nhiều pixel biên) nhưng chịu bias tâm ellipse — thiên lệch hệ thống phụ thuộc tư thế (A/02 mục 6). Chính xác quanh một giá trị lệch không bằng kém chính xác quanh giá trị đúng.
●● Detector học sâu	Bền hơn với mờ, che khuất, chiếu sáng khắc nghiệt. Độ chính xác góc <i>chưa chắc</i> hơn — dùng giả định bền hơn thì chính xác hơn. Lợi ích ít rõ với docking, nơi điều kiện kiểm soát được.
●●● Đòn bẩy của chương này	Thấp nhất trong Phần B. Đổi detector thay đổi σ 2–3 lần; đổi bố trí constellation thay đổi sai số góc hàng chục lần. Thứ tự đầu tư đúng: B/13, B/09, B/12, rồi mới tới đây.

●●● phải thuộc ●● nên nắm ● biết là có

I Kiến thức này đang sống ở đâu (công trình đang dùng chính ý tưởng của chương; chốt 9/2026)

Công trình / hệ thống	Dùng ý tưởng nào của chương	Ghi chú
apriltag (AprilRobotics)	Bản C gốc; đọc phần khớp đường có trọng số gradient để thấy mục 2 thành code	Một trong ba kho đáng đọc mã chứ không chỉ dùng (D/17)
OpenCV objdetect	ArUco và ChArUco; ArucoDetector với tham số cornerRefinementMethod	Nhớ bật tinh chỉnh góc — tắt mặc định ở một số phiên bản
Sinh từ điển ArUco [42]	Tạo từ điển nhỏ với khoảng cách Hamming lớn, đúng thứ cần cho docking	Đường tắt tốt hơn việc dùng từ điển mặc định quá lớn
apriltag-generation	Tự tạo họ tag với số ô và khoảng cách tùy chọn	Dùng khi muốn thực hiện đánh đổi ở mục 5C
So sánh thực nghiệm các họ tag [45]	Nguồn duy nhất trong cây có số đo trực tiếp so sánh giữa các họ	Con số gắn với điều kiện đo của họ; đọc điều kiện trước khi dùng số

MINI PROJECT · Đo σ thật của hệ bạn, và so ba cấu hình detector · 1–2 ngày

- Mục tiêu** thay con số “ $\sigma \approx 0,2$ px” mượn từ kinh nghiệm bằng con số đo được của chính hệ bạn — đầu vào bắt buộc cho mọi phép tính ở C/15.
- Dữ liệu** phần cứng thật: camera cố định trên giá, tag gắn cố định, chiếu sáng ổn định, mọi tham số tự động đã khoá.

Các bước	chụp 500 khung liên tiếp không ai chạm vào gì; với mỗi khung, chạy detector và ghi toạ độ bốn góc; tính độ lệch chuẩn của từng toạ độ qua 500 khung — đó là σ của bạn; lập với ba cấu hình: AprilTag mặc định, ArUco không tinh chỉnh góc, ArUco có cornerSubPix; rồi lập toàn bộ ở ba khoảng cách và ba góc nghiêng khác nhau.
Xong khi	bạn có một bảng σ theo cấu hình, khoảng cách và góc nghiêng. Kiểm tra chéo bắt buộc <i>hai</i> cái. Một: σ phải gần như không đổi theo khoảng cách khi tính bằng pixel — nếu nó tăng mạnh khi tag nhỏ đi thì bạn đang chạm ngưỡng kích thước tối thiểu của A/02 mục 4, và đó là một kết quả quan trọng. Hai: độ tản mát bạn đo là <i>nhiều</i> chứ không gồm thiên lệch, vì mọi khung chụp cùng một cảnh — nên con số này là cận dưới của sai số thật, và đừng nhầm nó với độ chính xác.
Nối với	C/15 nhận σ này thay cho giá trị mặc định; B/08 dùng bảng này để quyết định có cần chuyển sang ChArUco không.

TRẢ LỜI CÂU HỎI MỞ ĐẦU

- A1. Cơ chế thứ nhất: bước khớp tứ giác nhằm một vật thể khác — cửa sổ, biển báo, hoạ tiết sàn, bất cứ thứ gì gần tứ giác — rồi bước giải mã tình cờ đọc ra một chuỗi bit nằm trong từ điển. Cơ chế thứ hai: có một tag thật nhưng nhiều bit làm nó giải mã sai thành ID khác, và điều này dễ xảy ra hơn khi số lỗi bit cho phép sửa được đặt cao. Hai biện pháp ở hai tầng khác nhau: ở tầng detector, giảm số lỗi bit cho phép sửa — an toàn quan trọng hơn vài chục phân trăm phát hiện; ở tầng ứng dụng, dùng whitelist ID (bạn biết trước dock có đúng những ID nào, nên mọi ID khác bị loại ngay) cộng với kiểm tính hợp lý của pose — một tag báo ở 50 m hoặc ở góc nghiêng 89° là đáng ngờ bất kể mã có hợp lệ.
- A2. Bước khớp cạnh. AprilTag 1 lấy vị trí cạnh từ ảnh *nhi phân*, nơi thông tin bị lượng tử hoá thành một pixel và toàn bộ thông tin về độ sáng đã bị vứt bỏ. AprilTag 2 quay lại ảnh *xám gốc*, lấy các pixel dọc cạnh và khớp một đường thẳng với trọng số theo độ lớn gradient [7]. Thay đổi ấy quan trọng vì hai lẽ: một cạnh đi qua giữa hai pixel để lại dấu vết trong tỉ lệ độ sáng của hai pixel đó, nên thông tin dưới pixel tồn tại và chỉ cần không vứt nó đi; và việc dùng cả trăm pixel dọc cạnh để xác định một đường làm nhiều của từng pixel bị trung bình đi. Kết quả là định vị góc chuyển từ độ chính xác cỡ một pixel xuống cỡ phần mười pixel.
- A3. Vì hai loại giải hai việc khác nhau. ArUco giải *định danh*: nó cho biết ô nào là ô nào, kể cả khi chỉ nhìn thấy một phần bàn cờ — điều mà bàn cờ thuần không làm được vì mọi góc trông giống hệt nhau. Chessboard giải *đo*: góc của nó là một điểm yên ngựa, nơi gradient đổi dấu theo *cả* hai hướng, nên vị trí bị ràng buộc từ bốn phía; trong khi góc của ArUco là giao của hai đường thẳng, chỉ bị ràng buộc bởi hai hướng và sai số của mỗi đường lan vào giao điểm với hệ số khuếch đại phụ thuộc góc giao. Sự phân công ấy chính là cơ sở cho đề xuất target lai: AprilTag hoặc ArUco để bắt bám và định danh từ xa, vùng chessboard để đo tinh ở cự ly gần.
- A4. Cái giá là *tầm phát hiện*. Tag $n \times n$ ô dữ liệu có tổng khoảng $(n+2)$ ô trên cạnh kể cả viền, và cần chừng 3–5 px mỗi ô để giải mã tin cậy, nên $w_{\min} \approx 4(n+2)$ px. Với 16h5 (4×4) thì $w_{\min} \approx 24$ px; với 36h11 (6×6) thì $w_{\min} \approx 32$ px. Vì $w = fW/Z$, khoảng cách phát hiện tối đa tỉ lệ nghịch với $w_{\min}$, nên họ ít ô cho tầm xa hơn $32/24 = 1.33$ lần với cùng kích thước tag vật lý. Với docking, số ID cần thiết thường dưới vài chục trong khi 36h11 có hàng trăm, nên đánh đổi gần như luôn nghiêng về phía ít ô hơn — và mất mát về khoảng cách Hamming được bù lại bằng whitelist ID, một biện pháp gần như miễn phí.
- A5. Đúng vì độ lớn của đòn bẩy: chuyển từ detector này sang detector khác thay đổi σ chừng hai tới ba lần, trong khi chuyển từ một tag đơn sang constellation có baseline rộng thay đổi sai số góc hàng chục lần (A/05 mục 6) và việc phá tính đồng phẳng thì khử hẳn một nguồn lỗi thô. Nó vẫn đáng đọc vì ba lý do. Một, các bước khác nhau hỏng theo những cách khác nhau — ngưỡng hoá hỏng thì tag biến mất, khớp cạnh hỏng thì σ lớn, giải mã hỏng thì có báo sai — và ba triệu chứng ấy đòi ba cách chữa khác nhau. Hai, có ba tham số đáng chỉnh và biết pipeline là biết chỉnh cái nào. Ba, và quan trọng nhất, hiểu *cơ chế* định vị góc là cơ sở cho quyết định target lai — một quyết định thuộc loại đòn bẩy cao, dù nó được rút ra từ một chương đòn bẩy thấp.

NÓI LẠI THẬT ĐƠN GIẢN

Máy tính tìm một cái tag trong ảnh theo bốn bước, và mỗi bước hông theo một kiểu riêng.

Bước đầu là tách sáng khỏi tối, nhưng phải làm riêng cho từng vùng nhỏ, vì một góc ảnh có nắng còn góc kia có bóng râm. Nếu bước này hông thì cái tag *biến mất hẳn* — không phải đo sai, mà là không thấy gì. Chữa bằng cách lo ánh sáng, không phải lo thuật toán.

Bước hai là gom các mảng tối liền nhau rồi hỏi mảng nào giống hình tứ giác. Phần lớn bị loại luôn. Chỗ hông ở đây là nhầm: một cái cửa sổ, một viên gạch, một vết bẩn vuông vắn — tất cả đều là ứng viên.

Bước ba là bước quan trọng nhất, và là chỗ quyết định độ chính xác. Ta có một mảng pixel hình tứ giác nhưng cần biết bốn góc của nó ở đâu *chính xác tới phần mười pixel*. Cách ngây thơ là lấy bốn pixel ngoài cùng — và cách đó phí phạm, vì nó dùng bốn pixel rồi vứt bỏ hàng trăm pixel dọc bốn cạnh. Cách tốt là nhìn vào ảnh xám gốc, tìm nơi độ sáng đối nhanh nhất, kẻ một đường qua tất cả những chỗ ấy, rồi lấy nơi hai đường cắt nhau. Mỗi góc bây giờ được quyết định bởi cả trăm pixel, và đó là lý do đo được dưới pixel.

Bước bốn là đọc mã, và bước này là chuyện an toàn chứ không phải chuyện chính xác. Nấn cái tứ giác về hình vuông, chia ô, đọc từng ô sáng hay tối, rồi tra xem chuỗi ấy có nằm trong danh sách hợp lệ không. Cái danh sách ấy được thiết kế rất kỹ để hai mã bất kỳ khác nhau nhiều bit — nhờ vậy một vết bẩn ngẫu nhiên hầu như không bao giờ trùng với một mã thật. Điều này quan trọng hơn về ngoài: nếu robot *không thấy* tag thì nó chỉ đứng chờ, nhưng nếu nó *thấy nhầm* thì nó lái đi chỗ khác với đầy đủ tự tin.

Còn một chuyện đáng nói và nó hơi bất ngờ: chương này là chương *ít quan trọng nhất* trong phần các phương pháp. Đối từ detector này sang detector kia làm độ chính xác tốt lên chừng hai ba lần. Đối cách xếp mấy tấm tag trên dock làm nó tốt lên hàng chục lần. Người ta dành hàng tuần để so sánh detector và dành một buổi chiều để quyết định chỗ dán tag, trong khi đáng ra nên ngược lại. Và một điểm riêng đáng nhớ: cái loại tag vuông phổ biến thì giỏi việc *nhận ra* — nó đọc được từ xa, chịu được bẩn, hầu như không bao giờ nhận nhầm. Nhưng nó không phải loại đo góc chính xác nhất. Cái bàn cờ vua đen trắng thì đo chính xác hơn, chỉ có điều nó không tự nói cho bạn biết bạn đang nhìn ô nào. Cách khôn nhất là dùng cả hai: tag vuông để nhận ra và bám theo từ xa, bàn cờ để đo cho chuẩn lúc đã tới gần.

Chỗ không rút gọn được: chuyện detector nào chính xác hơn detector nào, bao nhiêu lần, trong điều kiện nào — cái đó không có câu trả lời chung, vì nó phụ thuộc ống kính, ánh sáng, khoảng cách. Cách duy nhất đáng tin là đo trên chính hệ của mình, và đo thì không khó: gắn cố định mọi thứ, chụp năm trăm tám, xem con số nhảy bao nhiêu.

Một câu: Bốn bước, và chỉ bước thứ ba quyết định độ chính xác — còn cả chương này thì quyết định ít hơn nhiều so với chỗ bạn dán mấy tấm tag.

7.8 Điều gì chứng minh cách hiểu này sai

Mệnh đề chính — *góc chessboard định vị chính xác hơn góc từ giao hai đường thẳng* — bị bác bỏ nếu phép đo σ ở mini project cho thấy hai loại cho độ tản mát tương đương trên cùng một hệ. Tôi dẫn mệnh đề bằng lập luận về số hướng ràng buộc (cửa a) và nó phù hợp với thực tế rằng cộng đồng hiệu chuẩn camera dùng chessboard chứ không dùng tag vuông; nhưng tôi *chưa* có số đo so sánh được (cửa b và c chưa qua). Đây là mệnh đề nền cho đề xuất target lai nên nó đáng kiểm soát.

Mệnh đề thứ hai — *chương này có đòn bẩy thấp nhất trong Phần B* — bị bác bỏ nếu trong một hệ thực, việc đổi detector cải thiện sai số cuối cùng nhiều hơn việc cải thiện constellation. Điều đó có thể xảy ra trong một trường hợp cụ thể: nếu constellation đã rất tốt và ngân sách đang bị chi phối bởi σ . Nói cách khác, mệnh đề của tôi là một phát biểu về *trường hợp điển hình*, không phải một định luật.

Mệnh đề thứ ba — *họ tag ít ô cho tầm phát hiện xa hơn theo tỉ lệ $w_{\min}$* — là phép tính đơn giản và nó chắc chắn về mặt số học; chỗ có thể sai là con số 3–5 px mỗi ô, vốn là bậc độ lớn kinh nghiệm. Nếu ngưỡng thật khác đáng kể thì tỉ số 1,33 đổi theo, nhưng chiều của kết luận không đổi.

Cuối cùng, tôi đã cố ý *không* phát biểu mệnh đề “detector X tốt hơn detector Y”, vì tôi không có dữ liệu so sánh được. Nếu bạn cần xếp hạng ấy thì nó phải đo trên cấu hình của bạn, và mini project cung cấp khung.

7.9 Kết nối

Chương này là cửa vào của Phần B và nó nối ở ba hướng.

Nối lên trên. [A/01-mo-hình-camera-va-distortion](#) mục 4 là chương phải đọc trước: nó giải thích vì sao méo chưa khử phá hoại đúng bước khớp cạnh mà mục 2 ở đây mô tả, và hai chương cùng nhau tạo thành một lập luận hoàn chỉnh về việc khử méo là bắt buộc. [A/02-homography-va-hình-học-phang](#) cấp bước rectify, và mục 5 ở đó (về tỉ số kép) giải thích vì sao mọi họ tag hiện đại đều mã hoá bằng lưới ô rời rạc chứ không bằng khoảng cách.

Nối ngang trong Phần B. [B/08-corner-refinement](#) là chương tiếp nối trực tiếp: chương này mô tả cơ chế định vị góc mặc định của từng detector, chương kia bàn cách cải thiện nó và đưa ra con số σ cho từng phương pháp. Hai chương nên đọc liền nhau. [B/09-multi-marker-constellation](#) là chương mà mục 1 chỉ tới khi nói về thứ tự đầu tư: nó có đòn bẩy cao hơn nhiều, và nó dùng ngưỡng kích thước tag mà chương này bàn ở mục 5C để thiết kế nested multi-scale. [B/14-dieu-kien-quan-sat-va-anh-sang](#) là chương chứa chế độ hồng của bước ngưỡng hoá — một chế độ hồng mà không thuật toán nào chứa được.

Nối xuống Phần C và D. [C/15-ngan-sach-sai-so](#) nhận σ từ mini project của chương này làm một trong bốn đầu vào của mọi phép tính. [D/17-thu-vien-va-detector](#) là chương công cụ tương ứng: nó nói API cụ thể, tên tham số, và chỗ nào trong mã nguồn hiện thực hoá từng bước mô tả ở đây.

Câu hỏi tự kiểm (trả lời không nhìn bài)

1. Ba bước của pipeline hồng theo ba kiểu khác nhau. Nêu triệu chứng của từng kiểu, và nêu cách chữa — chỉ ra cách nào *không* nằm ở tầng thuật toán.
2. Vì sao khớp đường trên ảnh xám với trọng số gradient chính xác hơn nhiều so với lấy biên của ảnh nhị phân? Nêu hai cơ chế độc lập.
3. ChArUco dùng hai loại đặc trưng cho hai việc. Nêu việc nào cho loại nào, và giải thích bằng cấu trúc hình học vì sao mỗi loại giỏi việc của nó.
4. Với tag 5×5 ô dữ liệu và ngưỡng 4 px mỗi ô, kích thước ảnh tối thiểu là bao nhiêu? Nếu $f = 600$ px và $W = 80$ mm, tầm phát hiện tối đa là bao xa?
5. Nêu chế độ hồng thâm lặng nhất của chương này, giải thích vì sao sai số tái chiếu không bắt được nó, và nêu cơ chế ở chương khác bắt được.
6. Vì sao chương này có đòn bẩy thấp nhất trong Phần B, và trong trường hợp cụ thể nào thì điều đó không còn đúng?
7. Bạn đo σ bằng cách chụp 500 khung của một cảnh tĩnh. Con số thu được là cận trên hay cận dưới của sai số thật, và thành phần nào nó không đo được?

Corner refinement (Subpixel Corner Refinement)

PHỤ THUỘC A/05-lan-truyen-sai-so-pixel-sang-pose (σ là hệ số nhân của mọi công thức sai số — chương này là chương duy nhất quyết định nó); A/06-hieu-ung-vat-ly-va-quang-hoc (trần quang học — không phương pháp nào vượt qua); B/07-detection-pipeline (cơ chế định vị góc mặc định của từng detector).

TÓM TẮT

Chương này quyết định σ , và σ là đại lượng duy nhất trong A/05 mà phần mềm còn ảnh hưởng được — mọi thứ khác trong bốn công thức của chương ấy là hình học hoặc vật lý. Sáu phương pháp được bàn, xếp theo cơ chế chứ không theo tên: trực giao gradient (cornerSubPix), khớp bậc hai quanh điểm yên ngựa, khớp đường có trọng số rồi lấy giao, định vị cạnh bằng moment, khớp conic cho marker tròn, và căn chỉnh quang trắc toàn bộ mẫu. Hai kết luận đáng mang ra. Thứ nhất: *điểm yên ngựa chính xác hơn giao hai đường*, vì nó bị ràng buộc từ bốn hướng thay vì hai — và đó là cơ sở của target lai. Thứ hai, và nó rẻ đến mức thường bị bỏ qua: *trung bình N khung khi robot đứng yên giảm σ theo $1/\sqrt{N}$* , một cải thiện năm lần với ba mươi khung, không tốn gì ngoài một phần ba giây. Nếu chỉ nhớ một điều: *hãy làm cái rẻ trước* — trung bình nhiều khung trước khi đổi thuật toán, và đổi thuật toán trước khi đổi phần cứng.

KIẾN THỨC CẦN CÓ TRƯỚC

- **độ chính xác dưới pixel (subpixel accuracy)** — khả năng định vị một đặc trưng với độ chính xác nhỏ hơn kích thước một pixel. Nó khả thi vì một cạnh đi qua giữa hai pixel để lại dấu vết trong *tỉ lệ độ sáng* của chúng — thông tin đó có thật và chỉ cần không vớt đi.
- **điểm yên ngựa (saddle point)** — điểm mà hàm độ sáng có cực đại theo một hướng và cực tiểu theo hướng vuông góc. Góc của một bàn cờ vua là một điểm như thế: đi dọc một đường chéo thì sáng lên, đi dọc đường chéo kia thì tối đi.
- **trực giao gradient** — tính chất: tại một góc lý tưởng, vectơ gradient của mọi pixel lân cận đều vuông góc với vectơ nối pixel đó tới góc. Lý do: gradient vuông góc với cạnh, và mọi cạnh đều đi qua góc. Đây là nguyên lý của cornerSubPix [46].
- **khớp quang trắc (photometric alignment)** — tối ưu trực tiếp trên *giá trị độ sáng* thay vì trên các đặc trưng đã trích xuất: tìm phép biến đổi làm ảnh mẫu khớp tốt nhất với vùng ảnh quan sát. Chính xác nhất nhưng chậm nhất.
- **hiệp phương sai của góc** — ma trận 2×2 mô tả bất định vị trí của một góc, và nó thường *dị hướng*: một góc trên một cạnh dài sắc nét được xác định tốt theo hướng vuông góc cạnh và kém hơn theo hướng dọc cạnh.
- **nhiều độc lập giữa các khung** — điều kiện để trung bình N khung giảm σ theo $1/\sqrt{N}$. Nó không luôn đúng: nhiễu do chiếu sáng nhấp nháy, do rung tuần hoàn, hoặc do thiên lệch cố định thì không độc lập, và trung bình không giúp gì.

CÂU HỎI MỞ ĐẦU

- Q1. Bạn có ba lựa chọn cải thiện σ : đổi sang thuật toán tinh chỉnh tốt hơn, mua camera đắt hơn, hoặc dừng robot lại 0,3 giây và lấy trung bình. Xếp ba lựa chọn theo tỉ lệ lợi ích trên chi phí, và giải thích vì sao thứ tự ấy làm nhiều người bất ngờ.

- Q2. Góc bàn cờ vua và góc tag vuông đều là “một góc”. Vì sao định vị cái thứ nhất chính xác hơn, và câu trả lời nằm ở một con số đếm rất đơn giản?
- Q3. `cornerSubPix` dựa trên một tính chất hình học duy nhất của gradient tại một góc. Tính chất đó là gì, và điều kiện nào phải thoả để nó còn đúng?
- Q4. Bạn lấy trung bình 100 khung và σ chỉ giảm hai lần thay vì mười lần như lý thuyết. Nêu ba nguyên nhân hoàn toàn khác nhau, và một phép thử phân biệt chúng.
- Q5. Căn chỉnh quang trắc dùng *toàn bộ* pixel của tag thay vì chỉ vùng quanh bốn góc, nên về nguyên tắc nó phải chính xác hơn nhiều. Vì sao nó vẫn ít được dùng, và trong trường hợp nào thì nên dùng?

8.1 Đặt vấn đề

A/05 kết thúc với bốn công thức, và cả bốn đều có cùng một tử số: $Z\sigma$. Khoảng cách Z thì bài toán quyết định. Tiêu cự f , kích thước tag trên ảnh w , baseline B — ba mẫu số — là hình học, và chúng được quyết định ở B/09 bằng nhô và vít. Còn lại đúng một đại lượng mà phần mềm ảnh hưởng được: σ .

Chương này là chương về σ .

Điều làm nó đáng dành một chương riêng — thay vì một mục trong B/07 — là một sự thật hơi phản trực giác: *độ chính xác dưới pixel không phải một mẹo mà là một nguyên lý*. Một cạnh đi qua giữa hai pixel không biến mất; nó để lại dấu vết trong tỉ lệ độ sáng của hai pixel ấy. Nếu pixel bên trái sáng 80% và pixel bên phải sáng 20%, thì cạnh nằm gần bên phải. Thông tin ấy có thật, và toàn bộ nghệ thuật của chương này là không vứt nó đi.

Bài toán gốc thuộc về ngành trắc lượng ảnh và thị giác máy tính từ những năm 1980, khi người ta nhận ra rằng độ chính xác của phép đo ảnh không bị giới hạn bởi kích thước pixel. Toán tử Förstner [46] là một trong những công trình sớm hình thức hoá chuyện này, và nguyên lý trực giao gradient của nó vẫn là nền của `cornerSubPix` trong OpenCV ngày nay.

Với bài toán docking, chương này có một đặc điểm đáng nói: *nó là chương có nhiều lựa chọn rẻ nhất*. Khác với B/09 — nơi mọi cải thiện đòi gia công lại — ở đây có những cải thiện chỉ tốn vài dòng code hoặc thậm chí không tốn gì. Và trong số đó, cái rẻ nhất lại là cái người ta hay nghĩ tới sau cùng: *lấy trung bình nhiều khung khi robot đứng yên*. Nó giảm σ theo $1/\sqrt{N}$, không đòi thuật toán mới, không đòi phần cứng mới, và nó tương thích với mọi phương pháp khác trong chương. Mục 5 dành cho nó, và tôi đặt nó trước các phương pháp tinh vi hơn một cách có chủ ý.

Cách hiển nhiên — và cách đúng — là dùng phương pháp tinh chỉnh tốt nhất mà detector cung cấp. Chỗ nó không đủ nằm ở hai điểm. Thứ nhất, các detector khác nhau dùng cơ chế khác nhau và các cơ chế ấy có trần khác nhau; biết cơ chế là biết trần. Thứ hai, có một trần cứng hơn nữa mà không phương pháp nào vượt qua được — trần quang học của A/06 mục 5 — và biết mình đang ở đâu so với trần ấy quyết định việc đầu tư thêm có đáng không.

TRỰC GIÁC

Vì sao đo được dưới pixel, nói bằng một hình ảnh.

Tưởng tượng một hàng rào có một vệt sơn trắng vắt ngang, và bạn phải nói vệt sơn bắt đầu ở thanh rào thứ mấy. Nếu mỗi thanh rào chỉ có hai trạng thái — có sơn hoặc không — thì bạn chỉ nói được “giữa thanh thứ 5 và thứ 6”. Độ chính xác bằng một thanh rào.

Nhưng nếu bạn nhìn được *mức độ* sơn trên từng thanh — thanh thứ 5 phủ hết, thanh thứ 6 phủ một phần ba, thanh thứ 7 sạch — thì bạn nói được chính xác hơn nhiều: mép vệt sơn

nằm ở một phần ba thanh thứ 6. Độ chính xác bây giờ là một phần ba thanh rào, và nếu bạn đọc được mức phủ chính xác hơn thì còn tốt hơn nữa.

Đó là toàn bộ nguyên lý. Pixel không phải ô vuông hai trạng thái; nó là một bộ đếm photon, và số photon nó đếm được tỉ lệ với *phần diện tích* của nó bị vùng sáng che phủ. Cạnh nằm ở đâu trong pixel thì tỉ lệ ấy nói ra.

Bây giờ đến phần thứ hai, và nó giải thích vì sao có nhiều phương pháp khác nhau. Một cạnh chỉ cho bạn biết vị trí theo *một* hướng — hướng vuông góc với nó. Để định vị một điểm trong mặt phẳng, cần ít nhất hai hướng.

Cách thứ nhất: lấy hai cạnh cắt nhau và tìm giao điểm. Cách này dùng hai hướng, và nó là cách của tag vuông. Chỗ yếu: nếu hai cạnh gần song song thì giao điểm trượt rất xa khi một trong hai cạnh nghiêng đi chút xíu.

Cách thứ hai: tìm một điểm mà độ sáng đổi chiều theo *mọi* hướng quanh nó — một điểm yên ngựa, như đỉnh của một cái yên. Điểm ấy bị ràng buộc từ bốn phía thay vì hai, nên nó không trượt được. Đây là cách của bàn cờ vua, và đó là lý do nó chính xác hơn.

8.2 Trực giao gradient: **cornerSubPix**

Phương pháp được dùng rộng rãi nhất, và nó trả lời Q3.

Nguyên lý dựa trên một quan sát hình học duy nhất: *tại một góc, vectơ gradient của mọi pixel lân cận đều vuông góc với vectơ nối pixel đó tới góc*. Lý do: gradient luôn vuông góc với cạnh (nó chỉ hướng độ sáng đổi nhanh nhất, tức hướng cắt ngang cạnh), và mọi cạnh tại một góc đều đi qua chính góc ấy. Nên nếu $\mathbf{q}$ là vị trí góc thật và $\mathbf{p}_i$ là một pixel lân cận với gradient ∇I_i , thì

$$\nabla I_i^\top (\mathbf{p}_i - \mathbf{q}) = 0.$$

Với nhiều pixel, đây là một hệ quá xác định; giải bình phương tối thiểu cho

$$\mathbf{q} = \left(\sum_i \nabla I_i \nabla I_i^\top \right)^{-1} \left(\sum_i \nabla I_i \nabla I_i^\top \mathbf{p}_i \right). \quad (8.1)$$

Đây là công thức của **cornerSubPix**, và nó bắt nguồn từ toán tử Förstner [46].

Ba điều đáng đọc ra từ (8.1), và cả ba đều có hệ quả thực hành.

Thứ nhất, ma trận $\sum \nabla I \nabla I^\top$ chính là ma trận cấu trúc, cùng ma trận xuất hiện trong bộ phát hiện góc Harris. Điều kiện để (8.1) có nghiệm tốt là ma trận ấy khả nghịch tốt, tức có hai giá trị riêng lớn — tức có gradient theo *hai* hướng khác nhau. Một điểm trên một cạnh thẳng chỉ có gradient theo một hướng, ma trận suy biến, và phương pháp không định vị được. Đây là câu trả lời cho phần thứ hai của Q3: *phải thật sự có một góc*, không phải một điểm trên cạnh.

Thứ hai, nghịch đảo của ma trận cấu trúc chính là hiệp phương sai (sai khác một hệ số phụ thuộc mức nhiễu ảnh). Nghĩa là phương pháp này cho *miễn phí* một ước lượng bất định cho từng góc, và nó dị hướng — đúng thứ mà A/03 mục 6 cần cho trọng số. Rất ít cài đặt lấy nó ra, và đó là một thiếu sót dễ sửa.

Thứ ba, kích thước của sổ lân cận là tham số quan trọng nhất, và nó là một đánh đổi thật. Cửa sổ lớn cho nhiều pixel hơn nên ít nhiễu hơn, nhưng nó có thể trùm sang các cạnh khác của tag hoặc sang vùng có nội dung khác, làm giả định trực giao bị vi phạm. Quy tắc thực hành: cửa sổ nên nhỏ hơn khoảng cách tới góc gần nhất, và với tag thì thường là vài pixel.

8.3 Điểm yên ngựa: vì sao chessboard thẳng

Mục này trả lời Q2, và nó là mục quan trọng nhất về mặt kết luận thiết kế.

Góc của một bàn cờ vua có một cấu trúc đặc biệt: hai ô đen và hai ô trắng gặp nhau tại một điểm, xếp chéo nhau. Hàm độ sáng quanh điểm ấy, khi khớp bằng một đa thức bậc hai

$$I(x, y) \approx a_0 + a_1x + a_2y + a_3x^2 + a_4xy + a_5y^2,$$

có một điểm dừng mà Hessian *không xác định dấu* — cực đại theo một hướng, cực tiểu theo hướng vuông góc. Đó là điểm yên ngựa, và vị trí của nó tìm được bằng cách cho gradient của đa thức bằng không:

$$\begin{pmatrix} 2a_3 & a_4 \\ a_4 & 2a_5 \end{pmatrix} \begin{pmatrix} x \\ y \end{pmatrix} = - \begin{pmatrix} a_1 \\ a_2 \end{pmatrix}.$$

Đây là phương pháp khớp bậc hai, và nó được dùng trong các cài đặt chính xác cao [12], [47].

Bây giờ đến câu hỏi vì sao nó chính xác hơn giao hai đường, và câu trả lời nằm ở một con số đếm — đúng như Q2 gợi ý.

Với *giao hai đường*: vị trí góc được xác định bởi hai ràng buộc, mỗi đường một ràng buộc. Sai số của mỗi đường lan vào giao điểm, và hệ số lan phụ thuộc góc giữa hai đường: khi chúng gần vuông góc thì hệ số cỡ 1, khi chúng gần song song thì hệ số lớn tùy ý. Với một tag nhìn rất chéo, hai cạnh kề có thể gặp nhau ở góc khá nhỏ, và khi ấy sai số bị khuếch đại mạnh.

Với *điểm yên ngựa*: vị trí được xác định bởi cấu trúc độ sáng theo *mọi* hướng quanh điểm. Bốn góc phần tư quanh nó — hai sáng, hai tối, xen kẽ — mỗi cái đóng góp một ràng buộc, và bốn ràng buộc ấy đến từ bốn hướng phân bố đều. Không có cấu hình nào làm hệ suy biến theo cách mà hai đường gần song song làm suy biến bài toán giao điểm.

Nói ngắn: *bốn hướng ràng buộc thay vì hai*, và bốn hướng ấy phân bố đều thay vì có thể chụm lại.

Ghi chú trung thực: tôi dẫn kết luận này bằng lập luận cấu trúc (của a). Tôi *chưa* có một so sánh định lượng trên cùng điều kiện đo (của b và c chưa qua). Sự thật rằng cộng đồng hiệu chuẩn camera dùng chessboard chứ không dùng tag vuông là một bằng chứng gián tiếp ủng hộ, nhưng nó không phải một phép đo. Đây là mệnh đề nền của đề xuất target lai, nên nó đáng kiểm — và mini project cung cấp cách kiểm.

CẦN NHỚ

Sự khác biệt giữa hai cơ chế dẫn thẳng tới một đề xuất thiết kế, và đó là một trong ba luận điểm “nói ngược thói quen” của [00-map](#).

Tag vuông (AprilTag, ArUco) giỏi *nhận diện*: đọc được từ xa, bền với che khuất và bẩn, tỉ lệ báo sai cực thấp, và tự nói cho bạn biết nó là ai.

Góc chessboard giỏi *đo*: bốn hướng ràng buộc, không có cấu hình suy biến, và không chịu bias kiểu ellipse của marker tròn.

Đề xuất: *target lai*. Tag vuông ở ngoại vi để bắt bám và định danh từ xa; một vùng ChArUco ở trung tâm để đo tinh ở cự ly gần. ChArUco đặc biệt tiện vì nó *đã là* một target lai — ArUco định danh, chessboard đo ([B/07](#) mục 3B).

Điều cần nhớ: đây không phải chọn một trong hai. Hai cơ chế phục vụ hai giai đoạn khác nhau của cùng một hành trình tiếp cận, và chiến lược coarse-to-fine ở [B/13](#) mục 5 chính là cái khung để dùng cả hai.

8.4 Các phương pháp còn lại

Bốn phương pháp nữa, ngắn gọn theo cơ chế.

Khớp đường có trọng số gradient rồi lấy giao. Đây là cơ chế của AprilTag 2/3, đã mô tả ở B/07 mục 2. Ưu điểm so với cornerSubPix: nó dùng *toàn bộ chiều dài cạnh* — hàng trăm pixel — thay vì một cửa sổ nhỏ quanh góc. Nhược điểm: nó chịu bài toán khuếch đại tại giao điểm, và nó đòi cạnh thật sự thẳng (A/01 mục 4).

Định vị cạnh bằng moment. Dùng các moment của profile độ sáng — thường là moment Zernike — để xác định vị trí cạnh dưới pixel một cách có cơ sở giải tích [48]. Chính xác và có lý thuyết đẹp; ít dùng trong thực hành marker vì nó đòi mô hình profile cạnh đúng, và profile thật bị ảnh hưởng bởi MTF của ống kính theo cách khó mô hình hoá.

Khớp conic và hiệu chỉnh tâm. Cho marker tròn: khớp một ellipse lên các điểm biên, rồi — và đây là bước bắt buộc — hiệu chỉnh bias tâm bằng đại số conic [15]. Độ chính xác định vị tâm rất cao vì tâm được xác định bởi hàng trăm điểm biên. Nhưng A/02 mục 6 đã cảnh báo: bỏ bước hiệu chỉnh thì bạn có một phép đo rất chính xác quanh một giá trị thiên lệch phụ thuộc tư thế, và với ngân sách 5 mm thì thiên lệch nguy hiểm hơn nhiều.

Căn chỉnh quang trắc. Thay vì trích xuất đặc trưng rồi khớp, tối ưu trực tiếp trên giá trị độ sáng: tìm phép biến đổi (homography) làm ảnh mẫu của tag khớp tốt nhất với vùng ảnh quan sát. Hai kỹ thuật chuẩn là ECC [49] — bất biến với thay đổi sáng tuyến tính — và inverse compositional alignment [50] — nhanh hơn nhờ tính trước Hessian.

Đây là câu trả lời cho Q5. Về nguyên tắc nó *phải* chính xác nhất, vì nó dùng mọi pixel của tag chứ không chỉ vùng quanh bốn góc — có thể là hàng nghìn pixel thay vì vài chục. Ba lý do nó vẫn ít được dùng: nó chậm hơn một bậc; nó cần khởi tạo tốt nên vẫn phải chạy detector thường trước; và nó nhạy với sai khác giữa mẫu và thực tế — tag bị bẩn, chiếu sáng không đều, hoặc mẫu không khớp chính xác với tag in ra đều làm nó lệch. Trường hợp nên dùng: khi robot đã dừng (nên chậm không sao), khi tag sạch và chiếu sáng kiểm soát được, và khi bạn cần vắt kiệt σ — tức chính xác pha cuối của B/13 mục 5.

Bảng 8.1. Sáu phương pháp: cơ chế, số hướng ràng buộc, và khi nào dùng.

Phương pháp	Cơ chế	Khi nào dùng
cornerSubPix [46]	Trực giao gradient trong cửa sổ nhỏ	Mặc định cho ArUco; nhớ bật, và nhớ lấy hiệp phương sai ra
Khớp bậc hai saddle [12], [47]	Bốn hướng ràng buộc	Chessboard/ChArUco/deltille ; chính xác nhất cho góc
Khớp đường có trọng số [7]	Hai đường, dùng cả chiều dài cạnh	Mặc định AprilTag; tốt nhưng khuếch đại khi cạnh chếch
Moment/Zernike [48]	Moment của profile cạnh	Ít dùng; đòi mô hình profile đúng
Conic + hiệu chỉnh tâm [15]	Khớp ellipse rồi bù bias	Marker tròn — và bước hiệu chỉnh là <i>bắt buộc</i>
Quang trắc (ECC, IC) [49], [50]	Tối ưu trên toàn bộ pixel	Pha cuối, robot đứng yên, cần vắt kiệt σ

8.5 Trung bình nhiều khung: cái rẻ nhất

Mục này trả lời Q1 và Q4, và tôi đặt nó riêng vì nó là kỹ thuật có tỉ lệ lợi ích trên chi phí cao nhất trong chương.

Nếu nhiều định vị góc độc lập giữa các khung và có trung bình không, thì trung bình N khung có độ lệch chuẩn

$$\sigma_N = \frac{\sigma}{\sqrt{N}}.$$

Với $N = 30$: cải thiện 5,5 lần. Với $N = 100$: cải thiện 10 lần. Chi phí: N chia cho tốc độ khung hình, tức với 30 fps thì 1 giây cho $N = 30$ — và trong thực tế thường ít hơn vì có thể lấy trung bình trong khi robot đang giảm tốc.

Đặt cạnh các lựa chọn khác, thứ tự ở Q1 trở nên rõ. Chuyển từ khớp đường sang saddle point cải thiện có lẽ hai tới ba lần. Mua camera đắt hơn cải thiện có lẽ hai lần và tốn tiền thật. Trung bình 30 khung cải thiện 5,5 lần và tốn một giây. Lý do thứ tự này làm nhiều người bất ngờ là vì nó không cảm thấy như “kỹ thuật” — nó không có thuật toán mới nào, không có gì để đọc paper về, và nó nghe như gian lận. Nhưng nó là cách sử dụng thông tin đúng đắn, và nó tương thích với mọi phương pháp khác trong chương.

A. Khi nào nó không hoạt động

Đây là phần thứ hai và quan trọng hơn, và nó trả lời Q4. Công thức $1/\sqrt{N}$ có một điều kiện: *nhiều độc lập giữa các khung và có trung bình không*. Ba cách vi phạm, và ba nguyên nhân của Q4.

Nguyên nhân một: có thành phần thiên lệch. Phần lớn các hiệu ứng ở A/06 — méo còn sót, bias tâm ellipse, motion blur nếu còn chuyển động — là thiên lệch chứ không phải nhiễu. Trung bình không chạm tới chúng. Triệu chứng: σ giảm theo $1/\sqrt{N}$ lúc đầu rồi *bão hoà* ở một sàn khác không.

Nguyên nhân hai: nhiễu tương quan theo thời gian. Chiếu sáng nhấp nháy (đèn huỳnh quang, đèn LED điều chế độ rộng xung), rung tuần hoàn của thân robot, hoặc nhiễu đọc có cấu trúc của cảm biến. Triệu chứng: σ giảm chậm hơn $1/\sqrt{N}$ ngay từ đầu, và tỉ lệ giảm phụ thuộc vào việc N khung trải qua bao nhiêu chu kỳ của hiện tượng tuần hoàn.

Nguyên nhân ba: lượng tử hoá. Nếu thuật toán tinh chỉnh của bạn có một bước lượng tử ẩn — chẳng hạn nó luôn trả về bội của $1/8$ pixel do cách nội suy — thì trung bình nhiều khung của cùng một giá trị lượng tử cho ra chính giá trị ấy. Triệu chứng: σ rất nhỏ nhưng giá trị trung bình nằm đúng trên một lưới.

Phép thử phân biệt ba nguyên nhân, và nó đơn giản: vẽ σ_N theo N trên trục $\log\text{--}\log$. Đường lý thuyết có độ dốc $-1/2$. Bão hoà ở một sàn nghĩa là nguyên nhân một. Độ dốc nhỏ hơn $1/2$ ngay từ đầu nghĩa là nguyên nhân hai. Giá trị trung bình rơi trên một lưới đều nghĩa là nguyên nhân ba. Đây là một trong những biểu đồ có giá trị chẩn đoán cao nhất mà bạn có thể vẽ, và nó gần như miễn phí.

CẦN NHỚ

Thứ tự đầu tư đúng để cải thiện σ , từ rẻ tới đắt:

Một, trung bình nhiều khung khi đứng yên — $1/\sqrt{N}$, không tốn gì ngoài thời gian, tương thích với mọi thứ khác.

Hai, bật tinh chỉnh góc nếu detector của bạn tắt nó mặc định — vài dòng cấu hình.

Ba, chuyển sang cơ chế saddle point cho khâu đo tinh (ChArUco) — thay đổi thiết kế target, nhưng không đổi phần cứng camera.

Bốn, cải thiện chiếu sáng (B/14) — rẻ hơn đổi camera, và nó cải thiện tính ổn định của σ chứ không chỉ giá trị của nó.

Năm, căn chỉnh quang trục cho pha cuối — tốn công cài đặt.

Sáu, đổi camera hoặc ống kính — đắt nhất, và chỉ đáng nếu bạn đã kiểm rằng quang học là nút thắt (A/06 mục 5B).

Và trước tất cả: vẽ đồ thị σ_N theo N . Nếu nó bão hoà sớm thì mọi đầu tư vào danh sách trên là vô ích — bạn đang có một thành phần thiên lệch, và bạn phải tìm nó trước.

8.6 Các con số: một cảnh báo cần lặp lại

Trong tài liệu về marker, người ta thường gặp các con số kiểu: góc AprilTag khoảng 0,1–0,3 px; góc ChArUco sau tinh chỉnh khoảng 0,03–0,1 px; tâm marker tròn có thể dưới 0,05 px. Chúng xuất hiện trong lộ trình gốc của cây này và tôi đã dùng 0,2 px làm giá trị mặc định xuyên suốt.

Cần nói rõ ba điều về các con số ấy.

Một, chúng là bậc độ lớn kinh nghiệm, không phải số đo có điều kiện xác định. Chúng không đi kèm thông tin về camera nào, ống kính nào, kích thước tag trên ảnh bao nhiêu, mức chiếu sáng ra sao. Một con số không có điều kiện đo là một con số vô nghĩa theo chuẩn của `learning-rules.md` mục 5, và tôi ghi rõ điều đó thay vì trình bày chúng như kết quả đo.

Hai, điều chúng nói đáng tin là thứ tự tương đối, không phải giá trị tuyệt đối. Rằng saddle point tốt hơn giao hai đường, và rằng tâm của một vùng lớn tốt hơn cả hai — ba điều ấy có cơ sở cấu trúc (mục 3 và 4) và chúng khớp với các con số. Rằng con số cụ thể là 0,03 hay 0,08 thì phụ thuộc hệ thống.

Ba, chênh lệch giữa chúng nhỏ hơn vẻ ngoài trong bức tranh tổng thể. Chuyển từ 0,2 px xuống 0,05 px là cải thiện bốn lần cho σ , và theo A/05 thì nó cải thiện mọi dòng ngân sách bốn lần — đáng kể. Nhưng nó không chạm tới các dòng hệ thống, và trong một hệ chưa đo constellation bằng BA thì các dòng ấy chiếm phần lớn ngân sách (B/09 mục 5). Nói cách khác: cải thiện σ bốn lần trên một ngân sách bị chi phối bởi sai số cơ khí thì gần như không đổi gì.

Có một công trình so sánh thực nghiệm nhiều họ tag trên cùng điều kiện [45], và nó là nguồn duy nhất trong cây có số đo trực tiếp so sánh được giữa các họ. Nếu bạn cần con số thì đó là nơi đọc — kèm theo việc đọc kỹ điều kiện đo của họ. Nhưng cách đúng nhất vẫn là đo trên chính hệ của bạn, và mini project của B/07 là cách làm.

8.7 Hiệp phương sai của góc: sản phẩm phụ đáng lấy

Mục ngắn nhưng nó nối chương này với A/03 mục 6, và nó là một cải thiện dễ mà ít ai làm.

Mọi phương pháp trong chương đều sinh ra một ước lượng bất định như sản phẩm phụ. Với `cornerSubPix`, đó là nghịch đảo của ma trận cấu trúc $\sum \nabla I \nabla I^T$ nhân với phương sai nhiễu ảnh. Với khớp bậc hai, đó là nghịch đảo Hessian của bài toán khớp. Với khớp đường rồi lấy giao, nó tính được bằng cách lan truyền hiệp phương sai của hai đường qua phép giao.

Điểm quan trọng: *hiệp phương sai ấy dị hướng*, và tính dị hướng mang thông tin thật. Một góc nằm trên một cạnh dài, sắc nét, có tương phản cao được xác định rất tốt theo hướng vuông góc cạnh và kém hơn nhiều theo hướng dọc cạnh. Một góc ở vùng bị loá có bất định lớn theo mọi hướng. Một góc của tag nhìn rất chếch có bất định kéo dài theo hướng mà hai cạnh gần song song.

Dùng thông tin ấy làm Σ_i trong PnP (A/03 mục 6) cho hai lợi ích. Lợi ích hiển nhiên: nghiệm chính xác hơn, vì các góc tốt được cho tiếng nói lớn hơn. Lợi ích ít hiển nhiên nhưng quan trọng hơn: *hiệp phương sai pose đầu ra trở nên trung thực*. Công thức $\mathcal{I}^{-1}$ của A/05 chỉ cho một ước lượng bất định đáng tin nếu Σ_i đầu vào đúng, và toàn bộ chuỗi phía sau — trọng số khi hợp nhất theo thời gian ở B/11, công chi-square ở B/09 mục 7, tiêu chí hội tụ ở B/13 mục 8 — phụ thuộc vào tính trung thực ấy.

Chi phí để lấy nó ra: vài dòng code, vì đại lượng đã được tính trong quá trình tinh chỉnh. Đây là một trong những cải thiện có tỉ lệ lợi ích trên công sức cao nhất trong cả Phần B, và nó bị bỏ qua gần như phổ biến.

8.8 Giới hạn và chế độ hỏng

Trần quang học. Không phương pháp nào vượt qua được giới hạn do MTF và nhiễu xạ đặt ra (A/06 mục 5). Nếu một cạnh sắc trải qua 5 pixel trên ảnh của bạn, thì hy vọng định vị tới 0,03 px là lạc quan bất kể thuật toán. Phép kiểm rẻ đã nêu ở A/06 mục 9: đo profile cạnh.

Mọi phương pháp giả định mô hình cạnh đúng. `cornerSubPix` giả định gradient trực giao với vectơ tới góc — sai khi cửa sổ trùm sang nội dung khác. Khớp bậc hai giả định profile độ sáng xấp xỉ được bằng đa thức bậc hai — sai khi tag rất nhỏ trên ảnh hoặc khi có loá. Khớp quang trắc giả định mẫu khớp với thực tế — sai khi tag bẩn.

Trung bình nhiều khung không chạm tới thiên lệch. Đã bàn kỹ ở mục 5A. Đây là giới hạn quan trọng nhất của kỹ thuật rẻ nhất, và đồ thị σ_N theo N là cách phát hiện.

Cải thiện σ chỉ ảnh hưởng tới các dòng ngẫu nhiên. Đã bàn ở mục 6. Trong một hệ chưa xử lý các nguồn hệ thống, chương này có đòn bẩy thấp. Thứ tự đúng: xử lý hệ thống trước (B/09 mục 5, B/12, hoặc B/13 mục 3), rồi mới tối ưu σ .

Mệnh đề “saddle point chính xác hơn” chưa được đo. Đã ghi rõ ở mục 3. Nó là mệnh đề nền cho target lai và nó đáng kiểm sớm.

8.9 Thực hành

Bốn việc, theo thứ tự.

Một, vẽ đồ thị σ_N theo N trên trục log–log trước khi làm bất cứ gì khác. Nó cho biết bạn đang bị giới hạn bởi nhiễu hay bởi thiên lệch, và câu trả lời quyết định toàn bộ chiến lược.

Hai, lấy hiệp phương sai từng góc ra và đưa vào `PnP`. Vài dòng code, lợi ích kép.

Ba, bật trung bình nhiều khung ở pha cuối — nó đi kèm tự nhiên với stop-and-go của B/13 mục 4.

Bốn, cân nhắc target lai nếu ngân sách góc chặt và các nguồn hệ thống đã được xử lý.

Active recall (che cột phải, tự trả lời, rồi mở ra đối chiếu)

Thuật ngữ / câu hỏi	Trả lời
••• Vì sao đo được dưới pixel	Pixel là bộ đếm photon, và số photon tỉ lệ với <i>phần diện tích</i> bị vùng sáng che phủ. Cạnh nằm đâu trong pixel thì tỉ lệ độ sáng nói ra. Thông tin có thật; nghệ thuật là không vớt nó đi.
••• Nguyên lý <code>cornerSubPix</code>	Tại góc thật, gradient của mọi pixel lân cận vuông góc với vectơ nối pixel đó tới góc (vì gradient vuông góc với cạnh, mà mọi cạnh đi qua góc) [46]. Giải bình phương tối thiểu hệ $\nabla I_i^T (\mathbf{p}_i - \mathbf{q}) = 0$.
••• Điều kiện để <code>cornerSubPix</code> hoạt động	Ma trận cấu trúc $\sum \nabla I \nabla I^T$ phải khả nghịch tốt, tức có gradient theo <i>hai</i> hướng khác nhau. Một điểm trên cạnh thẳng chỉ có một hướng $\Rightarrow$ suy biến. Phải thật sự có góc.
••• Vì sao saddle point chính xác hơn giao hai đường	Bốn hướng ràng buộc thay vì hai , và bốn hướng phân bố đều thay vì có thể chụm lại. Giao hai đường bị khuếch đại khi hai đường gần song song (tag chéo); saddle point không có cấu hình suy biến tương ứng.

Thuật ngữ / câu hỏi	Trả lời (tiếp)
●●● Hệ quả thiết kế: target lai	Tag vuông giới <i>nhận diện</i> (xa, bền, báo sai thấp, tự định danh); góc chessboard giới <i>đo</i> . Dùng cả hai theo giai đoạn: tag ngoại vi để bắt bảm, vùng ChArUco trung tâm để đo tinh.
●●● Trung bình N khung	$\sigma_N = \sigma/\sqrt{N}$. $N = 30 \Rightarrow 5,5$ lần, tốn 1 giây. Tỷ lệ lợi ích trên chi phí cao nhất trong chương, và tương thích với mọi phương pháp khác.
●●● Ba lý do trung bình không cho $1/\sqrt{N}$	Có thành phần <i>thiên lệch</i> (bảo hoà ở sàn khác không); nhiễu <i>tương quan</i> theo thời gian (đèn nhấp nháy, rung tuần hoàn — dốc nhỏ hơn 1/2 ngay từ đầu); <i>lượng tử hoá</i> ẩn trong thuật toán (giá trị trung bình rơi trên lưới đều).
●●● Biểu đồ chẩn đoán quan trọng nhất	σ_N theo N trên trục log-log. Dốc lý thuyết $-1/2$. Ba kiểu lệch khỏi nó ứng với ba nguyên nhân trên. Gần như miễn phí và nên vẽ trước mọi đầu tư khác.
●● Sản phẩm phụ đáng lấy	Mọi phương pháp sinh ra hiệp phương sai của góc miễn phí (nghịch đảo ma trận cấu trúc, hoặc nghịch đảo Hessian). Nó <i>dị hướng</i> và tính dị hướng mang thông tin thật.
●●● Hai lợi ích của việc dùng hiệp phương sai đúng	Nghiem chính xác hơn (góc tốt có tiếng nói lớn hơn); và — quan trọng hơn — <i>hiệp phương sai pose đầu ra trở nên trung thực</i> , mà cả chuỗi B/11, B/09 mục 7, B/13 mục 8 phụ thuộc vào đó.
●● Khớp quang trắc: vì sao ít dùng	Về nguyên tắc chính xác nhất (dùng mọi pixel chứ không chỉ quanh góc) nhưng chậm hơn một bậc, cần khởi tạo tốt, và nhảy với sai khác giữa mẫu và thực tế (bản, chiếu sáng). Nên dùng ở pha cuối khi robot đứng yên [49], [50].
●● Marker tròn: bước bắt buộc	Khớp ellipse rồi <i>hiệu chỉnh bias tâm bằng conic</i> [15]. Bỏ bước này thì có phép đo rất chính xác quanh một giá trị thiên lệch phụ thuộc tư thế — tệ hơn với ngân sách 5 mm.
●●● Các con số σ : cảnh báo	0,1–0,3 px cho AprilTag, 0,03–0,1 px cho ChArUco là <i>bậc độ lớn kinh nghiệm</i> không có điều kiện đo. Điều đáng tin là <i>thứ tự tương đối</i> (có cơ sở cấu trúc), không phải giá trị tuyệt đối.
●●● Thứ tự đầu tư để cải thiện σ	Trung bình nhiều khung $\rightarrow$ bật tinh chỉnh $\rightarrow$ saddle point/ChArUco $\rightarrow$ chiếu sáng $\rightarrow$ quang trắc $\rightarrow$ đổi camera. Và trước tất cả: vẽ σ_N theo N .
●● Khi nào chương này có đòn bẩy thấp	Khi ngân sách đang bị chi phối bởi các nguồn <i>hệ thống</i> (constellation chưa đo bằng BA, hiệu chuẩn kém). Cải thiện σ bốn lần trên một ngân sách như thế gần như không đổi gì.

●●● phải thuộc ●● nên nắm ● biết là có

I Kiến thức này đang sống ở đâu (công trình đang dùng chính ý tưởng của chương; chốt 9/2026)

Công trình / hệ thống	Dùng ý tưởng nào của chương	Ghi chú
OpenCV cornerSubPix	Cài (8.1); tham số winSize là đánh đổi ở mục 2	Ma trận cấu trúc đã tính sẵn bên trong — đáng sửa để lấy hiệp phương sai ra
OpenCV ChArUco	interpolateCornersCharuco dùng saddle point cho góc chessboard	Hiện thân của mục 3; nửa đo của target lai
deltille [12]	Saddle point trên lưới tam giác, tinh chỉnh bậc hai riêng	Lựa chọn khi cần độ chính xác góc cao nhất — B/12
ECC trong OpenCV [49]	findTransformECC — căn chỉnh quang trắc bất biến với thay đổi sáng	Dùng cho pha cuối khi đứng yên (B/13 mục 5)
Hiệu chỉnh conic [15]	Bù bias tâm cho target điểm tròn	Bắt buộc nếu dùng marker tròn; xem A/02 mục 6

MINI PROJECT · Vẽ đồ thị σ_N , và so ba cơ chế định vị góc trên cùng điều kiện · 2 ngày

Mục tiêu	trả lời hai câu hỏi quyết định chiến lược: hệ của tôi đang bị giới hạn bởi nhiều hay bởi thiên lệch, và saddle point có thật sự chính xác hơn giao hai đường trên phần cứng của tôi không.
Dữ liệu	phần cứng thật. Một target lai tự chế: in một tấm có cả tag AprilTag, tag ArUco, và một vùng ChArUco cạnh nhau trên <i>cùng một tấm nền cứng phẳng</i> , để cả ba chịu cùng điều kiện quang học.
Các bước	phần một: cố định mọi thứ, chụp 1000 khung; với mỗi N trong $\{1, 2, 5, 10, 20, 50, 100, 200, 500\}$, chia 1000 khung thành các nhóm N khung, tính trung bình mỗi nhóm, rồi tính độ lệch chuẩn của các trung bình ấy; vẽ log-log và so với đường dốc $-1/2$. Phần hai: với cùng bộ ảnh, tính σ riêng cho góc AprilTag (khớp đường), góc ArUco có cornerSubPix, và góc ChArUco (saddle point); lập ở ba khoảng cách và ba góc nghiêng.
Xong khi	bạn có một đồ thị log-log và một bảng ba cột. Từ đồ thị: nếu nó bám đường dốc $-1/2$ tới $N = 500$ thì bạn đang bị giới hạn thuần bởi nhiễu và trung bình nhiều khung là đầu tư tốt nhất; nếu nó bão hoà ở $N = 20$ thì bạn có một thành phần thiên lệch và phải đi tìm nó trước mọi thứ khác. Từ bảng: tỉ số giữa ba cột là con số đo được của mệnh đề ở mục 3. Kiểm tra chéo bắt buộc: ba loại góc phải chịu <i>cùng</i> điều kiện quang học — nếu vùng ChArUco của bạn nằm ở rìa ảnh còn tag ở giữa thì so sánh vô nghĩa, vì bạn đang đo MTF chứ không đo thuật toán.
Nối với	C/15 nhận σ và con số bão hoà; B/13 mục 4 nhận số khung tối ưu cho stop-and-go.

TRẢ LỜI CÂU HỎI MỞ ĐẦU

- A1. Thứ tự là: trung bình nhiều khung (rẻ nhất, lợi nhất), rồi đổi thuật toán, rồi mới mua camera. Trung bình 30 khung cho $1/\sqrt{30} = 5,5$ lần cải thiện với chi phí một giây và không dòng code thuật toán nào. Chuyển từ khớp đường sang saddle point cho có lẽ hai tới ba lần, và đòi hỏi thiết kế target. Camera đắt hơn cho có lẽ hai lần và tốn tiền thật — và chỉ đáng nếu bạn đã kiểm tra rằng quang học là nút thắt chứ không phải nhiễu ảnh ([A/06](#) mục 5B). Thứ tự này làm nhiều người bất ngờ vì lựa chọn tốt nhất không *cảm thấy* như kỹ thuật: nó không có thuật toán mới, không có paper để đọc, và nó nghe như gian lận. Nhưng nó là cách sử dụng thông tin đúng đắn, và nó tương thích với mọi phương pháp khác.
- A2. Con số đếm là *số hướng ràng buộc*: bốn thay vì hai. Góc tag vuông được xác định bằng cách lấy giao của hai đường thẳng, tức hai ràng buộc; và sai số của mỗi đường lan vào giao điểm với hệ số phụ thuộc góc giữa chúng — gần vuông góc thì hệ số cỡ 1, gần song song thì hệ số lớn tùy ý, và một tag nhìn chéo có hai cạnh kề gặp nhau ở góc khá nhỏ. Góc bàn cờ vua là một điểm yên ngựa: bốn góc phân tư quanh nó — hai sáng hai tối xen kẽ — mỗi cái đóng góp một ràng buộc, và bốn hướng ấy phân bố đều nên không có cấu hình nào làm hệ suy biến. Nói ngắn: hai hướng có thể chụm lại, bốn hướng phân bố đều thì không.
- A3. Tính chất: tại một góc thật, vectơ gradient của mọi pixel lân cận đều vuông góc với vectơ nối pixel đó tới góc. Lý do là gradient luôn vuông góc với cạnh (nó chỉ hướng độ sáng đổi nhanh nhất), và mọi cạnh tại một góc đều đi qua chính góc ấy [\[46\]](#). Điều kiện để nó còn đúng: ma trận cấu trúc $\sum \nabla I \nabla I^\top$ phải khả nghịch tốt, tức phải có gradient theo *hai* hướng khác nhau đáng kể trong cửa sổ. Một điểm nằm giữa một cạnh thẳng chỉ có gradient theo một hướng, ma trận suy biến, và phương pháp trượt tự do dọc cạnh. Điều kiện thứ hai, thực hành hơn: cửa sổ lân cận không được trùm sang cạnh khác hoặc sang nội dung khác, vì khi đó các gradient thêm vào không thoả tính chất trực giao với góc này.
- A4. Nguyên nhân một: có thành phần *thiên lệch* — méo còn sót, bias tâm ellipse, motion blur nếu còn chuyển động ([A/06](#)). Trung bình không chạm tới chúng. Nguyên nhân hai: nhiễu *tương quan theo thời gian* — đèn nhấp nháy, rung tuần hoàn của thân robot, nhiễu đọc có cấu trúc. Nguyên nhân ba: *lượng tử hoá* ẩn trong thuật toán, chẳng hạn nó luôn trả về bội của $1/8$ pixel do cách nội suy. Phép thử phân biệt cả ba bằng một biểu đồ: vẽ σ_N theo N trên trục log-log và so với đường dốc $-1/2$. Bão hoà ở một sàn khác không $\Rightarrow$ nguyên nhân một. Dốc nhỏ hơn $1/2$ ngay từ đầu $\Rightarrow$ nguyên nhân hai. Giá trị trung bình rơi đúng trên một lưới đều $\Rightarrow$ nguyên nhân ba.

- A5. Ba lý do. Nó chậm hơn một bậc vì nó tối ưu trên hàng nghìn pixel thay vì vài chục. Nó cần khởi tạo tốt nên vẫn phải chạy detector thường trước, tức nó là một bước *thêm* chứ không phải một bước *thay*. Và nó nhảy với sai khác giữa mẫu và thực tế: tag bị bắn, chiếu sáng không đều, hoặc ảnh mẫu không khớp chính xác với tag in ra — cả ba đều làm nó lệch, trong khi các phương pháp dựa trên cạnh thì bền hơn vì cạnh vẫn là cạnh dù độ sáng tuyệt đối đổi. Trường hợp nên dùng: khi robot *đã dừng* nên chậm không sao, khi tag sạch và chiếu sáng kiểm soát được, và khi bạn cần vết kiệt σ — tức chính xác pha cuối của chiến lược coarse-to-fine ở B/13 mục 5. Nói cách khác nó là công cụ chuyên dụng cho một giai đoạn, không phải lựa chọn mặc định.

NÓI LẠI THẬT ĐƠN GIẢN

Một câu hỏi tưởng như vô lý: làm sao đo được chính xác hơn cả kích thước một pixel?

Câu trả lời nằm ở chỗ pixel không phải một ô vuông chỉ biết đen hoặc trắng. Nó là một cái thùng đếm ánh sáng. Nếu một cái cạnh đen trắng đi qua giữa nó, nó đếm được lượng sáng ở giữa — và cái “ở giữa” ấy nói cho ta biết cạnh nằm gần bên nào. Pixel bên trái sáng tám phần, pixel bên phải sáng hai phần, vậy cạnh nằm lệch về bên phải. Thông tin đó có thật, và toàn bộ việc của chương này là dùng vớt nó đi.

Nhưng một cái cạnh chỉ nói được vị trí theo một chiều — chiều cắt ngang nó. Muốn biết một *điểm* nằm đâu thì cần ít nhất hai chiều, và đây là chỗ các phương pháp khác nhau.

Cách thứ nhất là lấy hai cạnh cắt nhau rồi tìm chỗ giao. Đây là cách của mấy cái tag vuông. Vấn đề là nếu hai cạnh gặp nhau ở một góc hẹp — chuyện xảy ra khi nhìn tag rất chếch — thì chỗ giao trượt rất xa chỉ vì một cạnh nghiêng đi chút xíu. Giống như hai cây gậy dài chạm đầu nhau: nghiêng một cây một chút thì đầu gậy trượt cả tấc.

Cách thứ hai là tìm một chỗ mà độ sáng đổi chiều theo *mọi* phía — như đỉnh của một cái yên ngựa, lên theo hướng này và xuống theo hướng kia. Góc của bàn cờ vua là một chỗ như thế: bốn ô, hai đen hai trắng, gặp nhau tại một điểm. Điểm ấy bị ghim từ bốn phía nên nó không trượt được, và đó là lý do bàn cờ đo chính xác hơn tag vuông.

Điều này dẫn tới một lời khuyên thực dụng: dùng cả hai loại. Tag vuông thì giỏi việc nhận ra từ xa và tự nói tên mình; bàn cờ thì giỏi việc đo cho chuẩn. Ghép chúng lại trên cùng một tấm và dùng mỗi cái cho việc nó giỏi.

Còn một mẹo cuối, đơn giản đến mức hay bị coi thường: *chụp nhiều tấm rồi lấy trung bình*. Ba mươi tấm thì sai số giảm hơn năm lần. Không cần thuật toán mới, không cần camera mới, chỉ cần robot đứng yên một giây. Nghe như gian lận, nhưng nó là cách dùng thông tin đúng đắn nhất trong cả chương.

Chỉ có một điều kiện, và nó quan trọng: cách này chỉ chữa được cái *run tay*, không chữa được cái *lệch ngắm*. Nếu ông kính làm mọi phép đo lệch về cùng một phía thì chụp một nghìn tấm vẫn lệch đúng bấy nhiêu. Cách kiểm rất đơn giản: vẽ xem sai số giảm thế nào khi tăng số tấm. Nếu nó cứ giảm đều thì bạn đang chỉ có run tay, cứ chụp thêm. Nếu nó giảm một lúc rồi dừng lại ở một mức nào đó thì bạn có lệch ngắm, và phải đi tìm nó thay vì chụp thêm.

Chỗ không rút gọn được: cái mức mà sai số dừng lại — cái sàn ấy — thì không đoán trước được. Nó là tổng của những thứ rất khác nhau: méo còn sót, ống kính mờ, tấm marker hơi cong. Phải đo mới biết, và biểu đồ “sai số theo số tấm chụp” là cách rõ nhất để biết mình đang ở đâu.

Một câu: *Chụp nhiều tấm chữa được tay run, không chữa được mắt lệch — và cái biểu đồ đơn giản nhất cho bạn biết mình đang mắc bệnh nào.*

8.10 Điều gì chứng minh cách hiểu này sai

Mệnh đề chính — *saddle point chính xác hơn giao hai đường vì bốn hướng ràng buộc thay vì hai* — bị bác bỏ nếu mini project cho thấy σ của góc ChArUco và σ của góc AprilTag tương đương trên cùng điều kiện quang học. Tôi dẫn nó bằng lập luận cấu trúc (cửa a) và nó được ủng hộ gián tiếp bởi thực tế rằng cộng đồng hiệu chuẩn dùng chessboard; nhưng nó *chưa* qua cửa (b) hay (c) với một so sánh định lượng. Đây là mệnh đề nền của đề xuất target lai và là nợ kiểm chứng lớn nhất của chương.

Mệnh đề thứ hai — *trung bình N khung cho $1/\sqrt{N}$* — là định lý giới hạn trung tâm và nó đúng dưới giả thiết. Điều đáng kiểm không phải mệnh đề mà là *giả thiết*: nhiều có thật sự độc lập giữa các khung trên hệ của bạn không. Mini project phần một là phép kiểm ấy, và tôi dự đoán rằng nhiều hệ sẽ thấy bão hoà sớm hơn mong đợi — nếu vậy thì đó là một phát hiện có giá trị vì nó chỉ ra một nguồn thiên lệch cần tìm.

Mệnh đề thứ ba — *cải thiện σ có đòn bẩy thấp khi ngân sách bị chi phối bởi nguồn hệ thống* — bị bác bỏ nếu trong một hệ thực, cải thiện σ bốn lần cho cải thiện sai số cuối cùng gần bốn lần. Nếu vậy thì hệ ấy đã xử lý tốt các nguồn hệ thống, và lời khuyên về thứ tự đầu tư không áp dụng cho nó.

Cuối cùng, tôi đã cố ý không phát biểu “phương pháp X cho $\sigma = Y$ px”, và mục 6 giải thích vì sao: các con số lưu hành không có điều kiện đo, và trích chúng như kết quả đo là vi phạm chuẩn của kho.

8.11 Kết nối

Chương này cấp một con số duy nhất cho phần còn lại của cây, và con số ấy là σ .

Nối lên trên. [A/05-lan-truyen-sai-so-pixel-sang-pose](#) là chương tiêu thụ trực tiếp: σ là tử số của cả bốn công thức ở đó, và nó là đại lượng duy nhất trong bốn công thức mà phần mềm ảnh hưởng được. [A/06-hieu-ung-vat-ly-va-quang-hoc](#) mục 5 đặt trần cứng cho σ , và mục 7 ở đó giải thích vì sao trung bình nhiều khung không chạm tới thiên lệch — nền của mục 5A ở đây. [A/02-homography-va-hinh-hoc-phang](#) mục 6 cấp cảnh báo về bias tâm ellipse mà mục 4 ở đây nhắc lại.

Nối ngang trong Phần B. [B/07-detection-pipeline](#) là chương tiền nhiệm trực tiếp: nó mô tả cơ chế định vị góc mặc định, chương này bàn cách cải thiện. Hai chương nên đọc liền. [B/09-multi-marker-constellation](#) là chương có đòn bẩy cao hơn, và mục 6 ở đây giải thích vì sao: cải thiện σ không chạm tới các dòng hệ thống mà [B/09](#) mục 5 xử lý. [B/11-uoc-luong-theo-thoi-gian](#) mở rộng mục 5 từ trung bình đơn giản sang hợp nhất có trọng số theo thời gian, và nó phụ thuộc vào hiệp phương sai mà mục 7 ở đây cung cấp. [B/13-docking-control](#) mục 4 là chỗ trung bình nhiều khung được tích hợp vào chiến lược điều khiển — stop-and-go không chỉ để xoá thiên lệch mà còn để tạo điều kiện cho trung bình.

Nối xuống Phần C. [C/15-ngan-sach-sai-so](#) nhận σ làm một trong bốn đầu vào, và nó nhận từ mục 5A một cảnh báo phương pháp luận: khi lập ngân sách, phải dùng σ sau khi trung bình nếu chiến lược có trung bình, và phải kiểm rằng trung bình thật sự hoạt động bằng đồ thị log-log. [C/16-danh-gia-va-nghiem-thu](#) nhận mini project của chương này làm một mục trong quy trình nghiệm thu.

Câu hỏi tự kiểm (trả lời không nhìn bài)

1. Vì sao độ chính xác dưới pixel là khả thi về mặt vật lý, và thông tin nằm ở đâu?
2. `cornerSubPix` dựa trên một tính chất hình học. Phát biểu nó, và nêu chính xác cấu hình làm nó suy biến.
3. Saddle point chính xác hơn giao hai đường vì một lý do đếm được. Nêu lý do ấy, và nêu cấu hình cụ thể làm giao hai đường tệ nhất.
4. Bạn vẽ σ_N theo N và thấy nó bão hoà ở $N = 20$. Nêu kết luận, và nêu việc tiếp theo nên làm — và việc nào *không* nên làm.
5. Mọi phương pháp trong chương sinh ra một đại lượng miễn phí mà hầu hết cài đặt vứt đi. Đại lượng đó là gì, vì sao tính dị hướng của nó quan trọng, và hai lợi ích của việc dùng nó?
6. Vì sao các con số σ lưu hành trong tài liệu không dùng được để nghiệm thu, và điều gì trong chúng thì vẫn đáng tin?

7. Trong trường hợp nào thì cải thiện σ bốn lần gần như không cải thiện sai số cuối cùng chút nào?

Multi-marker constellation (Marker Constellation Design)

PHỤ THUỘC A/03-pnp-va-uoc-luong-tu-the (PnP hợp nhất trên toàn bộ tập góc, và điều kiện để nó tốt hơn trung bình pose); A/04-pose-ambiguity-target-phang (vì sao phải phá tính đồng phẳng); A/05-lan-truyen-sai-so-pixel-sang-pose (quan hệ $\delta\theta \approx \delta x/B$ — toàn bộ chương này là việc hiện thực hoá quan hệ ấy bằng nhôm và vít).

TÓM TẮT

Đây là chương có đòn bẩy lớn nhất trong cả cây, và điều làm nó có đòn bẩy lớn là nó không nói về phần mềm. Ba kết quả của Phần A — sai số góc tỉ lệ nghịch với baseline, ambiguity khử được bằng tính không đồng phẳng, và PnP hợp nhất thắng trung bình pose — đều trở về cùng một hành động: *thiết kế hình học của tấm marker cho đúng*. Chương này biến ba kết quả ấy thành một quy trình thiết kế gồm năm quyết định: baseline bao nhiêu, bố trí không đồng phẳng thế nào, dùng mấy cỡ tag, đo lại constellation ra sao, và trọng số hoá thế nào. Trong năm quyết định ấy, quyết định thứ tư — *đo lại constellation bằng bundle adjustment thay vì tin bản vẽ CAD* — là quyết định mà tôi cho là ranh giới giữa đạt và không đạt 5 mm. Nếu chỉ nhớ một điều: *đừng tin CAD, hãy tự đo* — vì sai số lắp ráp của tấm marker trở thành sàn sai số của cả hệ thống, và nó là sai số hệ thống nên không trung bình mất đi.

KIẾN THỨC CẦN CÓ TRƯỚC

- **constellation** — tập các marker gắn cứng với nhau, cùng với toạ độ 3D của mọi góc của chúng trong một hệ toạ độ chung — hệ dock. Constellation không phải một tập tag; nó là một tập tag *cộng với* một mô hình hình học, và chất lượng của mô hình ấy quan trọng ngang chất lượng của các tag.
- **baseline** B — khoảng cách giữa hai marker xa nhau nhất theo một phương. Đáng chú ý: baseline là một đại lượng *có hướng* — một constellation trải rộng theo chiều ngang nhưng hẹp theo chiều dọc cho sai số yaw tốt và sai số pitch tệ.
- **độ lệch khỏi mặt phẳng** — khoảng cách lớn nhất từ một điểm điều khiển tới mặt phẳng khớp tốt nhất qua toàn bộ tập điểm. Đây là đại lượng định lượng cho “không đồng phẳng tới mức nào”, và nó là con số phải đưa vào bản vẽ.
- **nested multi-scale** — bố trí gồm nhiều cỡ tag lồng nhau, sao cho ở mọi khoảng cách trong dải tiếp cận luôn có ít nhất một tag nằm trong dải kích thước ảnh tốt.
- **bundle adjustment (BA)** — bài toán tối ưu đồng thời cả pose của nhiều khung nhìn lẫn toạ độ 3D của các điểm, cực tiểu sai số tái chiếu trên toàn bộ [2]. Trong chương này nó được dùng theo một chiều đặc biệt: giữ tag làm ẩn và dùng nhiều ảnh để *đo* constellation.
- **tự đo gauge** — trong BA, bài toán có các hướng không xác định được từ dữ liệu — vị trí và hướng của toàn bộ hệ toạ độ, và với ảnh đơn thuần thì cả tỉ lệ. Phải cố định chúng bằng quy ước, và cách cố định ảnh hưởng tới cách đọc hiệp phương sai kết quả.
- **chi-square gating** — loại bỏ phép đo có phần dư chuẩn hoá vượt một phân vị của phân bố χ^2 . Nó có cơ sở thống kê chứ không phải một ngưỡng tuỳ ý — nhưng nó đòi hiệp phương sai đầu vào đúng.

CÂU HỎI MỞ ĐẦU

- Q1. Bạn có ngân sách cho bốn tag. Phương án A: bốn tag 100 mm xếp thành hình vuông cạnh 400 mm trên mặt phẳng dock. Phương án B: hai tag 100 mm cách nhau 400 mm, mỗi tag nghiêng 20° về hai phía ngược nhau. Phương án nào tốt hơn, và vì sao chênh lệch lớn hơn nhiều so với chênh lệch số tag?
- Q2. Bản vẽ CAD nói hai tag cách nhau đúng 400,0 mm. Sau khi gia công và lắp, khoảng cách thật là 398,6 mm. Sai lệch 1,4 mm ấy làm hỏng ngân sách 5 mm tới mức nào — và vì sao nó không hiện ra ở bất kỳ chỉ số nào trong pipeline?
- Q3. Robot tiếp cận từ 3 m xuống 0,4 m. Với một cỡ tag duy nhất, kích thước trên ảnh thay đổi 7,5 lần. Vì sao điều đó là một vấn đề ở *cả hai* đầu của dải, và bao nhiêu cỡ tag thì đủ?
- Q4. Một tag trong constellation bị bắn nên góc của nó lệch 2 px. Trong PnP hợp nhất, sai lệch ấy lan sang pose như thế nào, và cơ chế nào phát hiện được nó trước khi nó phá hỏng kết quả?
- Q5. Có một phép đo bạn làm được mà không cần bất kỳ thiết bị đo lường nào ngoài chính camera của robot, và nó cho độ chính xác constellation tốt hơn thước cặp. Phép đo ấy là gì, và vì sao nó lại chính xác hơn?

9.1 Đặt vấn đề

Phần A kết thúc với ba kết luận, và cả ba đều trở về cùng một nơi.

Kết luận thứ nhất, từ A/05: sai số góc từ một constellation baseline B là $\delta\theta \approx Z\sigma/(fB)$, giảm tuyến tính theo B . Kết luận thứ hai, từ A/04: target phẳng luôn có hai nghiệm, và cách khử triệt để duy nhất là phá tính đồng phẳng. Kết luận thứ ba, từ A/03: PnP hợp nhất trên toàn bộ tập góc thẳng trung bình pose, với điều kiện toạ độ 3D của các tag đúng.

Ba kết luận ấy có một điểm chung đáng chú ý: *không cái nào nói về phần mềm*. Chúng nói về việc đặt các tấm nhôm ở đâu, nghiêng bao nhiêu, và đo chúng cẩn thận tới mức nào. Đây là chương biến chúng thành một bản vẽ.

Bài toán gốc thì cũ hơn robot học. Ngành trắc lượng ảnh đã giải nó từ lâu dưới tên khác: khi đo một công trình bằng ảnh, người ta đặt các mốc mục tiêu (targets) quanh đối tượng và *đo lại vị trí của chính các mốc ấy* bằng cách chụp nhiều ảnh rồi chạy bundle adjustment, thay vì tin vào việc đặt mốc chính xác [2]. Lý do rất thực dụng: đặt mốc chính xác tới phần mười milimét thì đắt, còn đo lại chúng bằng ảnh thì rẻ và chính xác hơn.

Điều làm chương này đáng viết cho người làm docking — thay vì chỉ trở sang tài liệu trắc lượng ảnh — là hai chỗ mà bài toán của ta khác.

Chỗ khác thứ nhất: ta có một *ràng buộc vận hành* mà trắc lượng ảnh không có. Robot tiếp cận theo một đường xác định, từ xa tới gần, và tag phải nhìn thấy được trên toàn bộ đường ấy. Điều này loại bỏ một số bố trí tối ưu về mặt hình học nhưng không dùng được vì tag bị che khuất hoặc quá nhỏ ở một đoạn nào đó. Mục 6 bàn chuyện này.

Chỗ khác thứ hai, và nó quan trọng hơn: ta có một *hướng ưu tiên*. Trắc lượng ảnh thường muốn độ chính xác đều theo mọi hướng; docking thì không. Robot cập vào theo trục z , nên sai lệch dọc trục ít quan trọng hơn sai lệch ngang và sai lệch góc yaw — một robot lệch 3 mm dọc trục thì chỉ vào hơi nông hoặc hơi sâu, còn lệch 1° yaw thì có thể trượt hẳn. Sự bất đối xứng ấy nên được phản ánh trong thiết kế constellation, và mục 3 nói cụ thể.

Cách hiển nhiên nhất — và cách gần như mọi dự án bắt đầu — là dán một tag lớn lên giữa dock. Nó đơn giản, nó hoạt động, và nó trần ở quanh một, hai độ vì lý do đã phân tích ở A/05. Cách hiển nhiên thứ hai là dán thêm tag cho chắc, thường là bốn tag ở bốn góc một tấm bảng. Cách này tốt hơn thật — nó tăng baseline — nhưng nó bỏ lỡ một nửa lợi ích vì bốn tag ấy vẫn đồng

phẳng nên ambiguity vẫn còn nguyên. Chương này lập luận rằng nửa bị bỏ lỡ ấy thường lớn hơn nửa đã lấy được.

TRỰC GIÁC

Hình dung việc đọc hướng của một vật bằng cách nhìn hai cái mốc gắn trên nó.

Hai mốc gần nhau thì khó: quay vật đi một chút, hai mốc dịch chuyển gần như nhau, và bạn phải đo được cái chênh lệch nhỏ giữa hai dịch chuyển gần bằng nhau. Hai mốc xa nhau thì dễ: cùng một góc quay làm chúng dịch ngược chiều nhau một khoảng lớn.

Đó là baseline, và nó giải thích tại sao trải rộng thì tốt. Nhưng có một chuyện thứ hai, và nó không nằm trong hình ảnh trên.

Giả sử hai mốc ấy nằm trên cùng một mặt phẳng với vật, phẳng lì. Bây giờ có một câu hỏi mà việc trải rộng không trả lời được: mặt phẳng ấy đang nghiêng về phía nào? Nghiêng lên trên hay nghiêng xuống dưới? Từ một cái bóng phẳng, hai khả năng ấy trông giống nhau — đây chính là bài toán hai nghiệm.

Cách chữa không phải là đặt mốc xa hơn nữa. Cách chữa là *dựng một cái mốc nhô ra khỏi mặt phẳng*. Ngay khi có một điểm không nằm trên mặt phẳng ấy, hai khả năng không còn giống nhau nữa: cái mốc nhô ra sẽ che khuất hoặc lộ ra tùy theo mặt phẳng nghiêng về phía nào.

Hai chuyện ấy độc lập nhau, và đó là điều quan trọng nhất của chương. Trải rộng chữa bài toán *độ nhạy*. Nhô ra chữa bài toán *hai nghiệm*. Bạn cần cả hai, và may mắn là một thiết kế tốt cho cả hai cùng lúc: đặt tag xa nhau *và* nghiêng chúng đi.

Còn một chuyện thứ ba, và nó là chuyện dễ quên nhất. Mọi lập luận trên đều giả định bạn *biết* hai cái mốc cách nhau bao xa. Nếu bạn nghĩ chúng cách nhau 400 mm mà thực ra là 398,6 mm, thì mọi phép đo góc của bạn sai theo một tỉ lệ cố định, và không có gì trong dữ liệu báo cho bạn biết.

9.2 Nguyên tắc một: PnP hợp nhất, và điều nó đòi hỏi

Điểm xuất phát về mặt phần mềm, và nó ngắn vì [A/03](#) mục 5 đã dẫn.

Với M tag nhìn thấy, chạy *một* bài toán tối ưu trên toàn bộ $4M$ góc, với toạ độ 3D của mọi góc biểu diễn trong hệ dock. Không trung bình pose. Lý do đã rõ: trung bình pose vứt bỏ ràng buộc rằng các tag gắn cứng với nhau, mà chính ràng buộc ấy tạo ra cơ chế baseline.

Điều đáng nhấn mạnh ở đây là *điều kiện* của lời khuyên ấy, vì nó là chỗ nối sang toàn bộ phần còn lại của chương. PnP hợp nhất tin vào mô hình constellation và ép một pose duy nhất giải thích mọi góc. Nếu mô hình sai, bộ giải phải thoả hiệp, và nó phân bổ sai lệch vào pose theo cách không kiểm soát được. Nói cách khác: *PnP hợp nhất chuyển sai số mô hình constellation thẳng vào sai số pose*.

Điều này có một hệ quả mà tôi cho là quan trọng và ít được nói: *quyết định dùng PnP hợp nhất và quyết định đo constellation bằng BA là một cặp không tách rời được*. Dùng cái thứ nhất mà bỏ cái thứ hai thì bạn đã đổi một nguồn sai số (thiếu thông tin hình học) lấy một nguồn khác (mô hình sai), và nguồn thứ hai có thể lớn hơn. Đây là lý do mục 5 nằm trong chương này chứ không nằm ở [B/12](#) cùng với các loại hiệu chuẩn khác.

May mắn là có một chỉ báo miễn phí, đã nêu ở [A/03](#): so sai số tái chiếu của bài toán hợp nhất với sai số tái chiếu khi giải từng tag riêng. Nếu hợp nhất lớn hơn hẳn, mô hình constellation sai. Đây là phép thử nên chạy định kỳ chứ không chỉ một lần lúc lắp đặt — tám marker bị va chạm, vít lỏng, nhiệt độ thay đổi.

9.3 Nguyên tắc hai: baseline có hướng

A/05 cho $\delta\theta \approx \delta x/B$ và kết luận “tăng B ”. Mục này làm rõ một chi tiết mà công thức vô hướng ấy giấu đi: *baseline là một đại lượng có hướng, và ba thành phần xoay cần ba baseline khác nhau.*

Đặt hệ toạ độ dock với z hướng ra ngoài (dọc trục tiếp cận), x ngang, y đứng. Ba thành phần xoay:

Yaw (quay quanh y , trục đứng) — đây là thành phần quan trọng nhất cho docking vì nó quyết định robot có vào thẳng trục không. Nó được xác định bởi baseline theo *chiều ngang x* : hai tag cách nhau theo x dịch chuyển ngược chiều khi yaw đổi.

Pitch (quay quanh x , trục ngang) — quan trọng ít hơn với robot di động trên sàn phẳng nhưng vẫn cần cho việc ước lượng đúng các thành phần khác. Được xác định bởi baseline theo *chiều đứng y* .

Roll (quay quanh z , trục quang) — đây là thành phần *dễ nhất*, như **A/05** mục 5 đã chỉ ra, vì nó là xoay trong mặt phẳng ảnh và được đo qua cơ chế vị trí. Nó gần như luôn đạt mà không cần thiết kể gì đặc biệt.

Hệ quả thiết kế trực tiếp và nó khác với “trải rộng đều”: *ưu tiên baseline ngang*. Một constellation rộng 500 mm và cao 150 mm cho yaw tốt hơn nhiều so với một constellation vuông 300×300 mm cùng diện tích, và với bài toán docking thì yaw là thành phần đáng tiền nhất.

Nhưng đừng bỏ hẳn baseline đứng, vì một lý do không hiển nhiên: nếu constellation gần như một đường thẳng ngang, thì ma trận thông tin gần suy biến theo hướng pitch, và một ma trận gần suy biến gây hại nhiều hơn chỉ là “pitch kém” — nó làm toàn bộ bài toán kém điều kiện, và sai số theo hướng yếu rò rỉ sang các hướng khác qua các phần tử ngoài đường chéo của $\mathcal{I}^{-1}$. Tỉ lệ mà tôi khuyến nghị là baseline đứng không dưới khoảng một phần ba baseline ngang (*đây là khuyến nghị của tôi dựa trên cơ chế kể trên, không phải kết quả tối ưu hoá đã kiểm chứng — hãy kiểm bằng Monte Carlo ở C/16*).

9.4 Nguyên tắc ba: phá tính đồng phẳng

A/04 kết luận rằng đây là cách duy nhất khử triệt để ambiguity. Mục này nói cụ thể làm thế nào, và bao nhiêu là đủ.

A. Ba cách thi công

Nêm nghiêng dưới mỗi tag. Giữ mọi tag trên cùng một tấm bảng nhưng gắn mỗi tag lên một nêm nghiêng $15\text{--}30^\circ$, với các hướng nghiêng khác nhau. Đây là cách tôi khuyến nghị mặc định vì nó cho *hai* lợi ích cùng lúc: nó phá đồng phẳng (khử ambiguity ở tầng constellation), và nó làm mỗi tag riêng lẻ có góc nghiêng khác không so với trục quang, nên chính tag ấy cũng thoát khỏi vùng $\sin\theta \rightarrow 0$ của **A/05** mục 5. Chi phí gia công gần như bằng không.

Tag ở các độ sâu khác nhau. Một số tag trên mặt phẳng dock, một số trên các gờ nhô ra 50–100 mm. Cho baseline theo chiều sâu, thứ mà cách một không cho, và điều đó cải thiện việc ước lượng khoảng cách. Thi công dễ nếu dock vốn đã có cấu trúc ba chiều.

Target dạng góc. Hai hoặc ba mặt phẳng vuông góc nhau, mỗi mặt mang tag. Mạnh nhất về hình học nhưng chiếm không gian và — nhược điểm thật — dễ bị che khuất một phần khi robot tới rất gần, đúng lúc cần chính xác nhất.

B. Bao nhiêu là đủ?

Đây là câu hỏi định lượng, và câu trả lời trung thực là *tôi không có công thức*. Nhưng có một cách suy nghĩ đúng về nó, và nó dẫn tới một phép đo cụ thể.

Cơ chế khử ambiguity là: điểm nằm ngoài mặt phẳng bị phép lật đưa tới vị trí khác hẳn, nên nghiệm thứ hai có sai số tái chiếu lớn hơn hẳn. Để cơ chế ấy hoạt động, chênh lệch sai số tái chiếu do phép lật gây ra phải *vượt hẳn nhiều*. Nghĩa là điều kiện có dạng

$$(\text{độ lệch khỏi mặt phẳng}) \times (\text{hệ số hình học}) \gg \sigma,$$

và hệ số hình học phụ thuộc vào khoảng cách, tiêu cự, và góc nhìn theo một cách mà tôi không dẫn ra được ở dạng đóng.

Cách đúng để có con số cho hệ của bạn là mô phỏng, và mini project của [A/04](#) chính là mô phỏng ấy: quét độ lệch khỏi mặt phẳng và tìm chỗ tỉ số ambiguity ρ bắt đầu lớn ổn định ở mọi tư thế. Đó là ngưỡng của bạn.

Khuyến nghị mặc định của tôi để bắt đầu, trước khi có kết quả mô phỏng: *độ lệch khỏi mặt phẳng ít nhất bằng khoảng 20% kích thước tag*, tức với tag 100 mm thì ít nhất 20 mm. Với nêm nghiêng 20° dưới một tag 100 mm, độ lệch giữa hai mép tag là $100 \times \sin 20^\circ = 34$ mm — thoải mái trên ngưỡng ấy. (Con số 20% là khuyến nghị của tôi dựa trên việc nó cho chênh lệch reprojection vài pixel ở cấu hình điển hình; nó chưa qua của kiểm chứng nào và đã được ghi vào 99-chua-biet.md.)

CẦN NHỚ

Nêm nghiêng là chi tiết gia công có tỉ lệ lợi ích trên chi phí cao nhất trong toàn bộ hệ thống docking.

Nó giải quyết *hai* bài toán độc lập bằng một chi tiết: ambiguity của target phẳng ([A/04](#)), và độ nhạy góc bằng không ở fronto-parallel ([A/05](#) mục 5).

Nó tồn: một khối nhôm vát góc dưới mỗi tag, cộng bốn con vít.

Nó phải được đưa vào bản vẽ *trước khi* gia công. Thêm nó sau khi đã sản xuất một trăm cái dock thì tốn một trăm lần chi phí ấy — và đó là lý do chương này nên được đọc trước khi thiết kế cơ khí, không phải sau khi phần mềm đã chạy và không đạt.

9.5 Nguyên tắc bốn: đo constellation bằng BA, đừng tin CAD

Đây là mục mà tôi cho là quan trọng nhất trong cả Phần B, và nó trả lời Q2 cùng Q5.

A. Vì sao CAD không đủ

Bản vẽ nói hai tag cách nhau 400,0 mm. Thực tế thì: tấm nhôm được cắt với dung sai, các lỗ vít được khoan với dung sai, tag được in với sai số tỉ lệ của máy in, tờ in được dán với sai lệch vị trí và có thể hơi lệch góc, và toàn bộ cụm được lắp với dung sai. Cộng lại, sai lệch cỡ một tới vài milimét là hoàn toàn bình thường cho một quy trình gia công thông thường.

Bây giờ hỏi: 1,4 mm sai lệch làm gì với ngân sách 5 mm? Hai câu trả lời, và câu thứ hai mới là câu đáng sợ.

Câu thứ nhất, về độ lớn: nếu constellation sai 1,4 mm trên baseline 400 mm, thì tỉ lệ sai là $1,4/400 = 0,35\%$. Sai số góc tương ứng cỡ $\arctan(1,4/400) = 0,2^\circ$ — tức 40% *ngân sách góc* tiêu vào một dòng duy nhất, và là dòng mà không thuật toán nào chạm tới được.

Câu thứ hai, về tính chất: sai lệch này là *hệ thống*. Nó không đổi giữa các khung, không đổi khi lấy trung bình, không đổi khi cải thiện detector. Theo phân loại của [A/05](#) mục 8, nó cộng thẳng

chứ không cộng bình phương. Và — đây là phần trả lời nửa sau của Q2 — nó *không hiện ra ở bất kỳ chỉ số nào* trong pipeline khi chỉ nhìn một cấu hình: PnP hợp nhất sẽ tìm ra pose tốt nhất giải thích được dữ liệu dưới mô hình sai, sai số tái chiếu tăng lên một chút nhưng vẫn nhỏ, và hiệp phương sai báo về vẫn đẹp. Bạn có một phép đo chính xác quanh một giá trị sai.

B. Bundle adjustment như một dụng cụ đo

Lối thoát, và nó là câu trả lời cho Q5: dùng chính camera làm dụng cụ đo.

Quy trình: chụp K ảnh của constellation từ nhiều vị trí và nhiều góc khác nhau (bao gồm cả những góc rất chếch — điều này quan trọng, vì lý do giống lý do ở A/01 mục 5 về điểm chính). Trong mỗi ảnh, phát hiện mọi tag nhìn thấy và lấy tọa độ góc. Rồi giải bài toán tối ưu với ẩn là: K pose camera và tọa độ 3D của mọi góc tag trong hệ dock. Hàm mục tiêu là tổng sai số tái chiếu trên toàn bộ [2].

Vì sao kết quả lại chính xác hơn thước cặp? Ba lý do.

Thứ nhất, *số lượng phép đo*. Với 30 ảnh và 4 tag, bạn có $30 \times 4 \times 4 = 480$ phép đo hai chiều, tức 960 phương trình, cho một số ẩn nhỏ hơn nhiều. Thông tin dư thừa lớn, và sai số ngẫu nhiên giảm theo căn bậc hai của nó.

Thứ hai, *bạn đo đúng thứ cần đo*. Thước cặp đo mép tấm nhôm; PnP dùng góc của tag in trên đó. Giữa hai thứ có sai lệch dán và sai lệch in mà thước cặp không thấy. BA đo chính xác đại lượng mà pipeline sẽ dùng.

Thứ ba, *nó đo trong hệ tọa độ đúng*. Kết quả BA là tọa độ của mọi góc trong một hệ chung, đã bao gồm mọi sai lệch góc quay của từng tag — thứ mà đo khoảng cách bằng thước không cho.

Có một điều kiện tiên quyết cần nói rõ: BA chỉ đo được constellation *sai khác một hệ số tỉ lệ* nếu chỉ dùng ảnh. Đây là tự do gauge về tỉ lệ, và nó phải được cố định bằng một thông tin bên ngoài: kích thước thật của một tag (đo bằng thước, và đây là chỗ duy nhất thước cặp cần thiết), hoặc một khoảng cách đã biết chính xác. Sai số của phép đo neo ấy trở thành sai số tỉ lệ của toàn bộ constellation, nên hãy đo nó cho cẩn thận — đo cạnh của tag lớn nhất chứ không phải tag nhỏ nhất.

CẠM BẮY

Ba sai lầm khi chạy BA để đo constellation.

Một, chỉ chụp từ các góc gần vuông góc. Với tư thế fronto-parallel, ambiguity nặng và thông tin về cấu trúc ba chiều nghèo, nên BA hội tụ kém và kết quả không đáng tin hơn CAD bao nhiêu. Phải có ảnh ở góc chếch mạnh — đây là cùng một yêu cầu, vì cùng một lý do, như yêu cầu về tư thế trong hiệu chuẩn intrinsic ở A/01 mục 5.

Hai, dùng intrinsic chưa hiệu chuẩn tốt. BA giải cấu trúc với K cho trước; nếu K sai thì cấu trúc thu được sai một cách có hệ thống, thường là sai tỉ lệ hoặc sai theo kiểu “cong” toàn cục. Thứ tự bắt buộc là: hiệu chuẩn intrinsic trước (B/12), rồi mới đo constellation.

Ba, không kiểm hiệp phương sai của kết quả. BA trả về không chỉ tọa độ mà cả bất định của chúng. Nếu một tag có bất định 2 mm thì bạn chưa đo được nó tốt hơn CAD, và cần chụp thêm ảnh phủ tag ấy từ nhiều góc hơn. Bỏ qua bước này là tự lừa mình: bạn đã thay một con số không biết sai bao nhiêu bằng một con số khác cũng không biết sai bao nhiêu.

9.6 Nguyên tắc năm: nested multi-scale

Mục này trả lời Q3, và nó là mục về ràng buộc vận hành.

Tag có kích thước ảnh $w = fW/Z$, và nó phải nằm trong một dải chấp nhận được. Cận dưới $w_{\min}$ đến từ hai nguồn: decode cần chừng 3–5 px mỗi ô bit (A/06 mục 4), và điều kiện số của bài toán xấu đi khi tag nhỏ (A/02 mục 4). Cận trên $w_{\max}$ đến từ việc tag phải *lọt trong khung hình* — và với nhiều tag thì phải lọt cả constellation, đây mới là ràng buộc thật.

Với dải khoảng cách $[Z_{\min}, Z_{\max}]$ và một cỡ tag duy nhất W , tỉ số kích thước ảnh giữa hai đầu dải là

$$\frac{w(Z_{\min})}{w(Z_{\max})} = \frac{Z_{\max}}{Z_{\min}}.$$

Với Z từ 3 m xuống 0,4 m, tỉ số là 7,5 — đúng con số trong Q3. Nếu ở xa tag chỉ vừa đủ lớn để decode ($w = 30$ px) thì ở gần nó chiếm $30 \times 7,5 = 225$ px; với một constellation gồm nhiều tag trải rộng, cả cụm sẽ tràn khỏi khung hình rất trước khi tới 0,4 m. Ngược lại, nếu chọn tag đủ nhỏ để cả cụm lọt khung ở 0,4 m, thì ở 3 m nó quá nhỏ để decode.

Đây là lý do *một cỡ tag không đủ cho một dải khoảng cách rộng*, và nó là vấn đề ở cả hai đầu chứ không chỉ một.

Giải pháp là nhiều cỡ. Nếu một cỡ tag phủ được tỉ số khoảng cách $r = w_{\max}/w_{\min}$, thì số cỡ cần thiết là

$$n = \left\lceil \frac{\ln(Z_{\max}/Z_{\min})}{\ln r} \right\rceil.$$

Ví dụ: nếu dải chấp nhận được của kích thước ảnh là 40–200 px thì $r = 5$; với $Z_{\max}/Z_{\min} = 7,5$, ta cần $n = \lceil \ln 7,5 / \ln 5 \rceil = \lceil 1,25 \rceil = 2$ cỡ. Hai cỡ tag, với tỉ lệ kích thước giữa chúng khoảng $\sqrt{7,5} = 2,7$ — chẳng hạn tag 200 mm và tag 75 mm.

Cách bố trí: tag lớn ở ngoại vi (chúng sẽ tràn khỏi khung khi tới gần, và điều đó không sao vì lúc ấy tag nhỏ đã đủ lớn), tag nhỏ ở trung tâm và chúng là cụm làm việc ở cự ly gần. Quan trọng: *cụm tag nhỏ tự nó cũng phải là một constellation tốt* — có baseline, không đồng phẳng — vì ở cự ly gần thì chỉ còn chúng.

9.7 Trọng số và loại ngoại lai

Mục này trả lời Q4 và nó nối với A/03 mục 6 và 7.

Trong một constellation, các tag *không* bình đẳng, và sự bất bình đẳng lớn hơn nhiều so với trong một target đơn. Một tag ở gần và nhìn thẳng có góc chính xác hơn hẳn một tag ở xa và nghiêng mạnh; một tag nghiêng 30° bằng nêm có cạnh bị nén nên định vị kém hơn; một tag ở rìa ảnh chịu méo còn sót lớn hơn. Cho chúng trọng số như nhau là để những tag kém nhất kéo nghiệm.

Quy tắc trọng số tối thiểu nên có: $\Sigma_i \propto 1/w_m^2$ với w_m là kích thước ảnh của tag chứa góc i . Nó đơn giản, nó nắm được nguồn biến thiên lớn nhất, và nó đã tốt hơn nhiều so với trọng số đều. Nếu detector của bạn cung cấp hiệp phương sai cho từng góc (B/08) thì dùng nó — tốt hơn nữa vì nó dị hướng.

Về loại ngoại lai, có một điểm đặc thù của constellation đáng nêu. Với Q4 — một tag bị bắn nên góc lệch 2 px — cơ chế lan truyền là thế này: PnP hợp nhất cực tiểu tổng bình phương, nên một góc lệch 2 px đóng góp $(2/0,2)^2 = 100$ lần một góc bình thường, và nó kéo pose về phía nó. Với 4 tag (16 góc), một góc xấu chiếm một phần đáng kể tổng hàm mục tiêu.

Ba cơ chế phát hiện, và nên có cả ba.

Cổng chi-square trên phần dư từng góc. Sau khi hội tụ, loại góc có phần dư chuẩn hoá vượt phân vị 99%, giải lại. Bắt được góc lệch rõ.

Kiểm nhất quán liên tag. Giải PnP riêng cho từng tag, so các pose. Nếu pose từ tag A và tag B lệch quá ngưỡng thì nghi ngờ. Điều làm phép thử này giá trị: nó bắt được loại lỗi mà cổng chi-square

bỏ sót — trường hợp *cả bốn góc* của một tag cùng lệch theo một hướng, chẳng hạn khi tag bị dán lệch vị trí so với mô hình. Bốn góc cùng lệch thì phần dư của chúng được hấp thụ vào pose và không góc nào nổi bật, nhưng pose từ tag ấy sẽ lệch so với pose từ tag khác.

RANSAC ở *mức tag*. Khi số tag đủ lớn (từ bốn trở lên), chọn ngẫu nhiên tập con hai tag, giải, đếm tag phù hợp [20]. Bỏ được trường hợp một tag bị lắp hằn sai chỗ.

CẦN NHỚ

Ba cơ chế phát hiện bắt ba loại lỗi khác nhau, và không cái nào thay thế được cái nào.

Cổng chi-square bắt *một góc lệch* — che khuất một phần, một góc bị loá.

Kiểm nhất quán liên tag bắt *cả một tag lệch* — dán lệch, mô hình sai, hoặc tag đang ở nghiệm ambiguity khác. Đây là lỗi mà chi-square bỏ sót, và nó là lỗi phổ biến nhất trong constellation.

RANSAC bắt *một tag sai hằn* — lắp nhầm vị trí, nhầm ID.

Chi phí của cả ba cộng lại vẫn không đáng kể so với detector, nên không có lý do gì chỉ dùng một.

9.8 Tổng hợp: một bản thiết kế mẫu

Gom năm nguyên tắc thành một bố trí cụ thể, để có một điểm xuất phát chứ không phải một danh sách.

Giả sử bài toán: robot tiếp cận từ 3 m xuống 0,4 m, ngân sách 5 mm / 0,5°, camera $f = 600$ px với khung hình 1280×960.

Hai cỡ tag theo mục 6: bốn tag 200 mm ở ngoại vi cho giai đoạn xa, bốn tag 75 mm ở trung tâm cho giai đoạn gần.

Baseline ưu tiên ngang theo mục 3: cụm lớn trải 600 mm ngang × 250 mm đứng; cụm nhỏ trải 220 mm ngang × 90 mm đứng.

Phá đồng phẳng theo mục 4: mỗi tag trên một nêm nghiêng 20°, hướng nghiêng luân phiên — tag trái nghiêng sang trái, tag phải nghiêng sang phải, tag trên nghiêng lên, tag dưới nghiêng xuống. Bố trí luân phiên này cho độ lệch khỏi mặt phẳng lớn hơn so với nghiêng cùng chiều, và nó cũng đảm bảo mỗi tag có góc nghiêng khác không so với trục quang ở tư thế tiếp cận.

Đo bằng BA theo mục 5, sau khi lắp và trước khi vận hành, với ít nhất 30 ảnh từ nhiều góc, neo tỉ lệ bằng cạnh đo được của một tag 200 mm.

Trọng số và ba cơ chế phát hiện theo mục 7.

Kiểm nhanh bằng công thức của A/05: ở $Z = 0,5$ m với $B = 0,22$ m (cụm nhỏ), $\delta\theta = 0,5 \times 0,2 / (600 \times 0,22) = 7,6 \times 10^{-4}$ rad = 0,043°. Dư so với 0,5° — nhưng nhớ rằng dòng lớn hơn nhiều là sai số constellation ở mục 5, và đó là lý do BA bắt buộc.

(Bản thiết kế này là ví dụ minh hoạ do tôi dựng từ các nguyên tắc của chương, không phải một thiết kế đã được chế tạo và đo. Nó nên được kiểm bằng Monte Carlo ở C/16 trước khi đưa vào giá công.)

9.9 Giới hạn và chế độ hồng

Constellation lớn hơn có thể bị che khuất. Trải rộng tốt cho hình học nhưng khi robot tới rất gần, một phần constellation có thể ra khỏi khung hình hoặc bị chính thân robot che. Đây là lý do

cụm nhỏ ở trung tâm trong thiết kế mẫu, và nó là một ràng buộc thật phải kiểm bằng cách mô phỏng trường nhìn dọc toàn bộ quỹ đạo tiếp cận.

Nêm nghiêng làm tag khó nhìn từ một số hướng. Tag nghiêng 30° sang trái sẽ khó đọc khi robot tiếp cận từ bên phải. Với đường tiếp cận cố định thì không sao; với robot có thể tới từ nhiều hướng thì phải cân nhắc, và góc nghiêng nên nhỏ hơn.

Constellation cứng là một giả định. Toàn bộ chương giả định các tag gắn cứng với nhau và mô hình không đổi. Rung động, va chạm, giãn nở nhiệt đều vi phạm. Cách phòng: chỉ báo sai số tái chiếu ở mục 2 chạy định kỳ, và đo lại BA sau bất kỳ va chạm nào.

BA có thể hội tụ về nghiệm sai. Như mọi bài toán tối ưu phi tuyến, BA cần khởi tạo tốt. CAD là khởi tạo tốt — đây là chỗ CAD vẫn hữu ích, dù không đủ chính xác để dùng trực tiếp. Và phải kiểm hiệp phương sai của kết quả, như đã cảnh báo ở mục 5.

Các con số khuyến nghị chưa được kiểm. Góc nghiêng $15\text{--}30^\circ$, tỉ lệ baseline đứng trên ngang một phần ba, độ lệch mặt phẳng 20% kích thước tag — cả ba là khuyến nghị của tôi dựa trên cơ chế, không phải kết quả tối ưu hoá. Chúng là điểm xuất phát hợp lý và chúng nên được kiểm bằng mô phỏng cho cấu hình cụ thể.

9.10 Thực hành

Năm việc, theo thứ tự thời gian của một dự án.

Một, trước khi vẽ bản vẽ: chạy Monte Carlo cho hai ba phương án bố trí, dùng script của [A/05](#) mở rộng. Một buổi chiều ở đây quyết định phần lớn kết quả.

Hai, khi vẽ bản vẽ: đưa nêm nghiêng vào ngay. Đây là chi tiết dễ quên nhất và đắt nhất để thêm sau.

Ba, sau khi lắp: chạy BA đo constellation, kiểm hiệp phương sai kết quả, và lưu toạ độ đo được thành tệp cấu hình — *không* dùng số từ CAD.

Bốn, trong vận hành: bật cả ba cơ chế phát hiện ở mục 7, và ghi chỉ báo sai số tái chiếu hợp nhất so với từng tag vào log.

Năm, định kỳ: chạy lại chỉ báo ấy và so với giá trị lúc lắp đặt. Tăng đáng kể nghĩa là constellation đã thay đổi — va chạm, vít lỏng, hoặc trôi nhiệt.

Active recall (che cột phải, tự trả lời, rồi mở ra đối chiếu)

Thuật ngữ / câu hỏi	Trả lời
••• Ba kết quả Phần A hội tụ về chương này	$\delta\theta \approx Z\sigma/(fB)$ (A/05); không đồng phẳng khử ambiguity (A/04); PnP hợp nhất thắng trung bình pose (A/03). Cả ba đều là quyết định <i>cơ khí</i> , không phải phần mềm.
••• Điều kiện của PnP hợp nhất	Toạ độ 3D constellation phải đúng. Hợp nhất chuyển sai số mô hình <i>thắng</i> vào sai số pose. Nên quyết định dùng hợp nhất và quyết định đo bằng BA là một cặp không tách rời.
••• Chỉ báo constellation sai	So sai số tái chiếu hợp nhất với sai số khi giải từng tag riêng. Hợp nhất lớn hơn hẳn $\Rightarrow$ mô hình sai. Miễn phí, và nên chạy định kỳ chứ không chỉ lúc lắp.
••• Baseline có hướng	Yaw cần baseline <i>ngang</i> (quan trọng nhất cho docking); pitch cần baseline <i>đứng</i> ; roll gần như luôn đạt. Ưu tiên ngang, nhưng đừng bỏ hẳn đứng — ma trận gần suy biến làm sai số rò rỉ sang hướng khác.

Thuật ngữ / câu hỏi	Trả lời (tiếp)
●●● Vì sao nệm nghiêng là chi tiết đáng giá nhất	Nó giải quyết <i>hai</i> bài toán độc lập bằng một chi tiết: ambiguity ($A/04$) và độ nhạy $\sin \theta \rightarrow 0$ ($A/05$). Chi phí: một khối nhôm vát góc và bốn vít.
●●● Sai số CAD 1,4 mm ăn bao nhiêu ngân sách	Trên baseline 400 mm: $\arctan(1,4/400) = 0,2^\circ$, tức 40% ngân sách góc. Và nó là <i>hệ thống</i> — cộng thẳng, không chia $\sqrt{N}$, không hiện ra ở chỉ số nào.
●●● Vì sao BA chính xác hơn thước cặp	Ba lý do: số phép đo lớn (30 ảnh $\times$ 4 tag $\times$ 4 góc = 960 phương trình); nó đo <i>đúng thứ pipeline dùng</i> (góc tag in, không phải mép nhôm); và nó cho toạ độ trong <i>một hệ chung</i> gồm cả sai lệch góc từng tag.
●●● Tự do gauge về tỉ lệ trong BA	Ảnh đơn thuần chỉ xác định constellation sai khác một hệ số tỉ lệ. Phải neo bằng một khoảng cách đo được — và đo cạnh tag <i>lớn nhất</i> , vì sai số của phép neo trở thành sai số tỉ lệ toàn cục.
●● Ba sai lầm khi chạy BA	Chỉ chụp góc gần vuông (thông tin nghèo, hội tụ kém); dùng intrinsic chưa hiệu chuẩn (cấu trúc sai có hệ thống); không kiểm hiệp phương sai kết quả (thay một con số chưa biết sai bằng một con số khác chưa biết sai).
●●● Số cỡ tag cần thiết	$n = \lceil \ln(Z_{\max}/Z_{\min}) / \ln r \rceil$ với $r = w_{\max}/w_{\min}$. Ví dụ: dải 3 m–0,4 m và $r = 5 \Rightarrow 2$ cỡ, tỉ lệ kích thước $\sqrt{7,5} \approx 2,7$.
●● Bố trí nested multi-scale	Tag lớn ở ngoại vi (tràn khỏi khung khi gần, không sao); tag nhỏ ở trung tâm. Quan trọng: <i>cụm nhỏ tự nó cũng phải là constellation tốt</i> — có baseline, không đồng phẳng — vì ở gần chỉ còn nó.
●●● Ba cơ chế phát hiện, ba loại lỗi	Chi-square bắt <i>một góc</i> lệch; kiểm nhất quán liên tag bắt <i>cả một tag</i> lệch (chi-square bỏ sót vì bốn góc cùng lệch được hấp thụ vào pose); RANSAC bắt tag lắp sai hần.
●● Quy tắc trọng số tối thiểu	$\Sigma_i \propto 1/w_m^2$ theo kích thước ảnh của tag chứa góc đó. Đơn giản, nắm được nguồn biến thiên lớn nhất, và tốt hơn nhiều so với trọng số đều.
● Các con số khuyến nghị chưa kiểm	Nghiêng 15–30°; baseline đứng $\geq 1/3$ ngang; lệch mặt phẳng $\geq 20\%$ kích thước tag. Đây là khuyến nghị của tôi dựa trên cơ chế, chưa qua cửa kiểm chứng — kiểm bằng Monte Carlo.

●●● phải thuộc ●● nên nắm ● biết là có

I Kiến thức này đang sống ở đâu (công trình đang dùng chính ý tưởng của chương; chốt 9/2026)

Công trình / hệ thống	Dùng ý tưởng nào của chương	Ghi chú
ArUco Board / ChArUco Board	Coi cả bảng là một target, giải một PnP trên toàn bộ — đúng nguyên tắc mục 2	Nhưng board phẳng, nên vẫn có ambiguity; dùng làm nền rồi thêm nệm nghiêng
tagslam [31]	Map nhiều tag trong môi trường bằng factor graph, ước lượng cả vị trí tag	Chính là BA của mục 5 ở quy mô phòng; mức tin C, dùng làm gợi ý kiến trúc
COLMAP [51]	Photogrammetry tổng quát; dùng đo constellation khi không muốn tự viết BA	Đường tắt thực dụng cho mục 5 — xem D/18
Ceres [27], GTSAM	Viết BA riêng với ràng buộc phù hợp (tag cứng, góc tag cố định tương đối)	Cách đúng nếu muốn tận dụng cấu trúc cứng của từng tag
Trắc lượng ảnh công nghiệp [2]	Truyền thống lâu đời của việc đo lại mốc thay vì đặt mốc chính xác	Nguồn của toàn bộ ý tưởng ở mục 5

MINI PROJECT · So bốn phương án bố trí bằng Monte Carlo trước khi gia công · 2–3 ngày

Mục tiêu	biến quyết định “đặt tag ở đâu” từ một phán đoán thành một bảng số, và có một bản vẽ có cơ sở trước khi tốn tiền gia công.
Dữ liệu	mô phỏng thuần, mở rộng script của A/05 và A/04 .
Các bước	đựng bốn phương án: (A) một tag lớn ở giữa; (B) bốn tag đồng phẳng ở bốn góc; (C) bốn tag với nêm nghiêng 20° luân phiên; (D) phương án C cộng thêm cụm nhỏ ở trung tâm. Với mỗi phương án, mô phỏng toàn bộ quỹ đạo tiếp cận từ $Z_{\max}$ tới $Z_{\min}$ với các giá trị lệch ngang và lệch góc ban đầu khác nhau; ở mỗi điểm, chiếu mọi tag nhìn thấy (kiểm cả điều kiện lọt khung hình và điều kiện kích thước tối thiểu), thêm nhiễu, chạy PnP hợp nhất, ghi sai số vị trí và góc cùng với tỉ số ambiguity ρ ; 2000 lần cho mỗi điểm. Sau đó phần thứ hai: thêm sai số constellation e vào mô hình và quét e từ 0 tới 3 mm.
Xong khi	bạn có bốn đường sai số góc theo Z đặt cạnh nhau, và một bảng nói mỗi phương án đạt ngưỡng ở dải khoảng cách nào. Kiểm tra chéo bắt buộc: phương án B phải cho $\rho \approx 1$ ở tư thế gần vuông góc trong khi phương án C thì không — nếu cả hai đều cho ρ lớn thì mô phỏng ambiguity của bạn có lỗi, vì A/04 nói rõ B phải có vấn đề. Và từ phần hai bạn có ngưỡng e cho phép, tức yêu cầu độ chính xác của BA.
Nối với	C/15 nhận bảng này làm dòng hình học của ngân sách; B/12 nhận ngưỡng e làm yêu cầu cho bước hiệu chuẩn constellation.

TRẢ LỜI CÂU HỎI MỞ ĐẦU

- A1. Phương án B, và chênh lệch lớn vì hai lý do độc lập cộng lại. Lý do thứ nhất: B có baseline 400 mm giống A, nên về mặt độ nhạy góc hai phương án tương đương — việc A có bốn tag thay vì hai chỉ giảm nhiễu theo $\sqrt{2}$, một cải thiện nhỏ. Lý do thứ hai, và đây mới là phần lớn: A vẫn đồng phẳng, nên nó vẫn có hai nghiệm và vẫn chịu độ nhạy $\sin \theta \rightarrow 0$ ở tư thế tiếp cận thẳng trục; B thì không, vì hai nêm nghiêng ngược chiều nhau tạo độ lệch khỏi mặt phẳng lớn và đồng thời cho mỗi tag một góc nghiêng khác không so với trục quang. Nói ngắn: hai tag nghiêng thẳng bốn tag phẳng, và bài học là số lượng tag gần như không quan trọng so với hình học của chúng.
- A2. Nó ăn khoảng 40% ngân sách góc: $\arctan(1,4/400) = 0,2^\circ$ so với ngân sách $0,5^\circ$. Và nó ăn theo cách tệ nhất — đây là sai số hệ thống, nên nó cộng thẳng chứ không cộng bình phương với các nguồn khác, và nó không giảm chút nào khi lấy trung bình một nghìn khung. Nó không hiện ra ở chỉ số nào vì PnP hợp nhất sẽ làm đúng việc nó được giao: tìm pose tốt nhất giải thích dữ liệu dưới mô hình đã cho. Mô hình sai thì pose sai, sai số tái chiếu tăng một chút nhưng vẫn nhỏ, và hiệp phương sai báo về vẫn đẹp vì công thức hiệp phương sai giả định mô hình đúng ([A/05](#) mục 2). Cách duy nhất phát hiện là so với một nguồn thông tin độc lập — chẳng hạn so sai số tái chiếu hợp nhất với từng tag riêng, hoặc đo lại bằng BA.
- A3. Ở đầu xa, tag quá nhỏ: dưới ngưỡng 3–5 px mỗi ô bit thì decode không tin cậy, và điều kiện số của bài toán xấu đi nhanh hơn $1/w$. Ở đầu gần, vấn đề ngược lại và ít người lường trước: constellation trải rộng sẽ tràn khỏi khung hình, nên bạn mất đúng các tag ngoại vi vốn cung cấp baseline, đúng vào lúc cần chính xác nhất. Với dải 3 m–0,4 m, tỉ số 7,5. Nếu dải kích thước ảnh chấp nhận được là 40–200 px (tỉ số 5) thì cần $\lceil \ln 7,5 / \ln 5 \rceil = 2$ cỡ tag, với tỉ lệ kích thước giữa chúng khoảng 2,7 lần. Điều quan trọng thường bị bỏ qua: cụm tag nhỏ phải tự nó là một constellation tốt — có baseline, không đồng phẳng — vì ở cự ly gần thì chỉ còn nó làm việc.
- A4. Lan truyền theo cơ chế bình phương: một góc lệch 2 px đóng góp $(2/0,2)^2 = 100$ lần một góc bình thường vào hàm mục tiêu, nên nó kéo pose về phía nó, và với 16 góc thì nó chiếm phần đáng kể tổng. Ba cơ chế phát hiện, bắt ba loại lỗi khác nhau: cộng chi-square trên phần dư từng góc bắt được trường hợp một góc lệch rõ (đúng tình huống này); kiểm nhất quán liên tag — giải PnP riêng từng tag rồi so pose — bắt được trường hợp cả bốn góc của một tag cùng lệch, thứ mà chi-square bỏ sót vì lệch đồng đều được hấp thụ vào pose; và RANSAC ở mức tag bắt được tag lắp sai hẳn. Cả ba nên bật cùng lúc vì chi phí không đáng kể.
- A5. Bundle adjustment: chụp vài chục ảnh constellation từ nhiều vị trí và góc, rồi giải đồng thời cả pose các khung lẫn tọa độ 3D của mọi góc tag. Nó chính xác hơn thước cặp vì ba lý do. Số lượng: 30 ảnh $\times$ 4 tag $\times$ 4 góc cho 960 phương trình cho một số ẩn nhỏ hơn nhiều, nên sai số ngẫu nhiên bị dim. Đúng đại lượng: thước cặp đo mép tấm nhôm, còn pipeline dùng góc của tag in trên đó — giữa hai thứ có sai lệch in và sai lệch dán mà thước không thấy. Và đúng hệ tọa

độ: BA cho tọa độ mọi góc trong một hệ chung, đã bao gồm sai lệch góc quay của từng tag, thứ mà đo khoảng cách không cho. Điều kiện duy nhất: ảnh không xác định được tỉ lệ, nên phải neo bằng một khoảng cách đo được — và đó là chỗ duy nhất thước cặp còn cần thiết, nên hãy đo cạnh của tag lớn nhất.

NÓI LẠI THẬT ĐƠN GIẢN

Nếu chỉ có một chương trong cả tài liệu này được đọc bởi người thiết kế cơ khí, thì nên là chương này — vì gần như mọi thứ trong đây đều được quyết định bằng nhôm và vít chứ không bằng code. Có ba việc phải làm với mấy tấm tag, và chúng độc lập nhau.

Việc thứ nhất: *đặt chúng xa nhau*. Muốn biết một vật đang quay bao nhiêu, hãy nhìn hai điểm trên nó ở xa nhau — cùng một góc quay làm chúng dịch ngược chiều nhau nhiều hơn, nên dễ đo hơn. Hai tag cách nhau nửa mét cho phép đo góc tốt gấp năm lần hai tag cách nhau mười phân, và cái giá phải trả chỉ là một tấm nhôm rộng hơn. Với robot cập dock thì hướng quan trọng nhất là trái-phải, nên hãy trải rộng theo chiều ngang trước.

Việc thứ hai: *đừng để chúng phẳng lì*. Kê một cái nêm dưới mỗi tag cho nó nghiêng đi hai mươi độ, mỗi cái nghiêng một hướng. Lý do nghe lạ nhưng rất thật: một tấm phẳng nhìn từ phía trước thì có hai cách nghiêng cho cùng một hình ảnh, và máy không phân biệt được, nên nó cứ nhảy qua nhảy lại giữa hai câu trả lời. Kê nghiêng thì hết. Cái nêm ấy tốn vài chục nghìn và nó giải quyết một vấn đề mà hàng tuần viết code không giải quyết được.

Việc thứ ba, và đây là việc hay bị bỏ qua nhất: *đo lại xem chúng thực sự ở đâu*. Bản vẽ nói hai tag cách nhau bốn trăm milimét, nhưng sau khi cắt, khoan, in, dán và lắp thì thực tế có thể là ba trăm chín mươi tám phẩy sáu. Một phẩy bốn milimét nghe chẳng đáng gì, nhưng nó ăn gần một nửa cái ngân sách góc mà bạn có — và tệ nhất là nó không lộ ra ở đâu cả. Máy vẫn chạy, con số vẫn đẹp, và mọi phép đo đều lệch đi cùng một chút.

Cách đo lại thì bất ngờ đơn giản: dùng chính cái camera. Chụp mấy chục tấm ảnh của cả cụm từ nhiều góc, rồi để máy tính giải ngược ra xem mỗi tag thật sự nằm đâu. Cách này chính xác hơn thước cặp, vì nó dùng cả nghìn phép đo thay vì vài phép, và vì nó đo đúng cái thứ mà máy sẽ dùng sau này — các góc của tag in, chứ không phải mép tấm nhôm.

Cuối cùng, một chuyện về khoảng cách. Cái tag trông to hay nhỏ tùy robot đứng gần hay xa. Ở xa ba mét nó nhỏ quá không đọc được; ở gần bốn tấc thì cả cụm tràn ra ngoài khung hình. Không có cỡ nào vừa cho cả hai. Nên cách làm là dán hai cỡ: mấy cái to ở ngoài rìa cho lúc còn xa, mấy cái nhỏ ở giữa cho lúc đã gần — và nhớ rằng mấy cái nhỏ ở giữa cũng phải xa nhau và cũng phải kê nghiêng, vì lúc cập vào thì chỉ còn chúng nó.

Chỗ không rút gọn được: nghiêng bao nhiêu độ là đủ, và xa nhau bao nhiêu là đủ — hai con số ấy thì không có công thức. Chúng phụ thuộc camera, khoảng cách, và độ chính xác cần đạt. May là chạy thử bằng mô phỏng thì rẻ, và một buổi chiều mô phỏng rẻ hơn nhiều so với một lô hàng gia công sai.

Một câu: *Xa nhau, kê nghiêng, và đo lại — ba việc làm bằng nhôm và vít, và chúng quyết định nhiều hơn mọi dòng code cộng lại.*

9.11 Điều gì chứng minh cách hiểu này sai

Mệnh đề chính — *sai số constellation là dòng chi phối trong ngân sách của một hệ đã tối ưu tốt phần nhiều* — bị bác bỏ nếu trong một hệ thực, sau khi đo constellation bằng BA và cải thiện độ chính xác của nó xuống dưới 0,2 mm, sai số pose không cải thiện đáng kể. Khi ấy nút thắt nằm ở chỗ khác — nhiều khả năng là hand-eye ([B/12](#)) hoặc sai số bám của bộ điều khiển ([B/13](#)). (Tôi phát biểu dựa trên phép tính độ lớn ở mục 5, không dựa trên thống kê hệ thực tế.)

Mệnh đề thứ hai — *hai tag nghiêng thẳng bốn tag phẳng* — là mệnh đề dễ kiểm nhất và có hậu quả thiết kế lớn nhất. Nó bị bác bỏ nếu Monte Carlo cho thấy bốn tag đồng phẳng cho sai số góc tương đương hoặc tốt hơn ở tư thế tiếp cận. Nếu vậy thì hoặc mô phỏng ambiguity có lỗi, hoặc lập luận của [A/04](#) về mức độ nghiêm trọng của ambiguity là quá mạnh.

Mệnh đề thứ ba — *BA cho constellation chính xác hơn CAD* — bị bác bỏ nếu hiệp phương sai mà BA trả về lớn hơn dung sai gia công. Điều này *có thể* xảy ra nếu số ảnh ít, góc nhìn nghèo, hoặc intrinsic chưa tốt — và đó chính là lý do mục 5 nhấn mạnh phải kiểm hiệp phương sai kết quả. Nói cách khác, mệnh đề này không phải một sự thật vô điều kiện mà là một điều kiện phải kiểm cho từng lần đo.

Cuối cùng, ba con số khuyến nghị ở mục 4 và 3 — nghiêng 15–30°, tỉ lệ baseline đứng trên ngang một phần ba, lệch mặt phẳng 20% kích thước tag — tôi đã đánh dấu rõ là chưa qua cửa kiểm chứng nào. Chúng có thể sai theo cả hai chiều (quá bảo thủ hoặc chưa đủ), và mini project là cách kiểm.

9.12 Kết nối

Chương này là nút giao của cây: mọi thứ trong Phần A chảy vào đây và nó chảy ra thành một bản vẽ.

Nối lên trên. [A/05-lan-truyen-sai-so-pixel-sang-pose](#) cấp công thức $\delta\theta \approx \delta x/B$, và mục 3 làm rõ rằng B là đại lượng có hướng — một chi tiết mà công thức vô hướng giấu đi. [A/04-pose-ambiguity-target-phang](#) cấp lý do cho mục 4, và mini project của chương ấy cấp phương pháp đo ngưỡng độ lệch mặt phẳng. [A/03-pnp-va-uoc-luong-tu-the](#) cấp mục 2 và mục 7 gần như nguyên vẹn; điều chương này thêm vào là *điều kiện* của PnP hợp nhất và hệ quả của nó với mục 5. [A/02-homography-va-hinh-hoc-phang](#) mục 4 cấp lý do định lượng cho ngưỡng kích thước tối thiểu ở mục 6.

Nối ngang trong Phần B. [B/08-corner-refinement](#) cấp Σ_i cho mục 7, và nó cũng cho biết σ thực tế của hệ — con số đi vào mọi phép tính baseline. [B/12-calibration-cho-docking](#) là chương anh em của mục 5: nó bàn hiệu chuẩn intrinsic và hand-eye, còn mục 5 ở đây bàn hiệu chuẩn constellation, và ba thứ ấy phải làm theo đúng thứ tự (intrinsic trước, constellation sau, hand-eye cuối). [B/10-xu-ly-ambiguity](#) là chương xử lý phần dư của mục 4: những gì phải làm ở tầng code khi hình học chưa khử hết ambiguity.

Nối xuống Phần C. [C/15-ngan-sach-sai-so](#) nhận từ mục 5 dòng lớn nhất của bảng ngân sách, và nhận từ mục 3 phần hình học. [C/16-danh-gia-va-nghiem-thu](#) là nơi mini project của chương này sống, và nó mở rộng mô phỏng ấy thành quy trình nghiệm thu đầy đủ.

Và một nối sang [B/13-docking-control](#) đáng nêu: chương điều khiển sẽ lập luận rằng teach-by-showing triệt tiêu các sai số hệ thống. Điều đó *bao gồm* sai số constellation — nên nếu bạn dùng teach-by-showing, yêu cầu về độ chính xác của mục 5 nói lỏng đáng kể. Đây là một trong những chỗ mà hai chương xương sống của Phần B tương tác, và cách đọc đúng là: *làm cả hai thì dư dả, làm một trong hai thì vừa đủ, không làm cái nào thì không đạt*.

Câu hỏi tự kiểm (trả lời không nhìn bài)

1. Vì sao hai tag nghiêng thắng bốn tag đồng phẳng, và hai lợi ích độc lập nào mà nệm nghiêng cung cấp cùng lúc?
2. Baseline là đại lượng có hướng. Với robot cập dock, thành phần xoay nào quan trọng nhất, nó cần baseline theo hướng nào, và vì sao không nên bỏ hẵn hướng còn lại?
3. Sai số constellation 1,4 mm trên baseline 400 mm ăn bao nhiêu phần trăm ngân sách góc? Tính, rồi nói vì sao con số ấy không hiện ra trong bất kỳ chỉ số nào của pipeline.
4. Nêu ba lý do BA cho constellation chính xác hơn đo bằng thước cặp, và nêu một chỗ mà thước cặp vẫn không thay thế được.
5. Với dải khoảng cách $Z_{\max}/Z_{\min} = 10$ và dải kích thước ảnh chấp nhận được 40–240 px, cần mấy cỡ tag và tỉ lệ giữa chúng là bao nhiêu?

6. Ba cơ chế phát hiện ngoại lai bắt ba loại lỗi khác nhau. Nêu loại lỗi mà cổng chi-square *bỏ sót*, và giải thích vì sao nó bỏ sót.
7. PnP hợp nhất tốt hơn trung bình pose — nhưng chỉ dưới một điều kiện. Nêu điều kiện ấy, và nêu chỉ báo miễn phí cho biết điều kiện có được thoả hay không.

Xử lý ambiguity (Handling Pose Ambiguity)

PHỤ THUỘC [A/04-pose-ambiguity-target-phang](#) (cơ chế hai nghiệm, quan hệ gương, tỉ số ρ — chương này giả định bạn đã hiểu vì sao có hai nghiệm và chỉ bàn làm gì với chúng); [B/09-multi-marker-constellation](#) (cách khử triệt để bằng hình học — chương này là những gì phải làm khi hình học chưa khử hết).

TÓM TẮT

Chương này ngắn, và độ ngắn của nó là có chủ ý: nội dung của nó gần như hoàn toàn là hệ quả của [A/04](#) và [B/09](#). Nó tồn tại riêng vì các quyết định ở tầng code rất cụ thể — ngưỡng bằng bao nhiêu, từ chối hay chấp nhận, chọn nghiệm nào — và trộn chúng vào chương lý thuyết làm cả hai khó dùng. Cấu trúc là một thang bốn bậc theo thứ tự ưu tiên: khử bằng hình học (tốt nhất, và nếu làm được thì ba bậc còn lại thành dự phòng), đo và từ chối khi không tin được, dùng nhất quán thời gian có kiểm soát, và fuse với cảm biến khác. Nếu chỉ nhớ một điều: *ba bậc dưới chỉ giúp chọn đúng trong hai nghiệm vẫn đang tồn tại, nên chúng luôn có xác suất sai khác không* — và vì thế chúng không thay thế được bậc trên cùng.

KIẾN THỨC CẦN CÓ TRƯỚC

- **tỉ số ambiguity** ρ — $\rho = e_2/e_1 \geq 1$, tỉ số giữa sai số tái chiếu của nghiệm tốt thứ hai và nghiệm tốt nhất. Gần 1 nghĩa là hai nghiệm giải thích ảnh tốt ngang nhau.
- **từ chối cập nhật** — trả về “không biết” thay vì một pose không đáng tin. Đây là hành vi *đúng* của một hệ đo lường, dù nó làm tăng trên phải xử lý thêm một trạng thái.
- **nất quán thời gian (temporal consistency)** — chọn nghiệm gần với pose ở khung trước, hoặc gần với dự đoán từ mô hình chuyển động. Hiệu quả nhưng có một chế độ hỏng nghiệm trọng mô tả ở mục 4.
- **khoá cứng vào nghiệm sai (lock-in)** — tình trạng mà sau khi chọn nhầm một lần, tiêu chí nhất quán thời gian giữ hệ thống ở nghiệm sai vĩnh viễn — ổn định, tự tin, và sai.

CÂU HỎI MỞ ĐẦU

- Q1. Bạn đã bố trí constellation không đồng phẳng theo [B/09](#). Vì sao chương này vẫn cần thiết, và nó bảo vệ bạn khỏi tình huống nào?
- Q2. Hệ của bạn chọn nghiệm bằng cách lấy cái có sai số tái chiếu nhỏ hơn, và nó chạy ổn định suốt một tuần thử nghiệm. Vì sao sự ổn định ấy có thể là dấu hiệu xấu chứ không phải dấu hiệu tốt?
- Q3. Có một ngưỡng ρ mà dưới nó bạn nên từ chối. Vì sao không thể tra ngưỡng ấy từ một bảng, và bạn dẫn ra nó bằng cách nào?
- Q4. IMU cho biết roll và pitch của robot khá chính xác. Vì sao thông tin ấy *đôi khi* loại được một trong hai nghiệm và đôi khi hoàn toàn vô dụng — điều gì quyết định?
- Q5. Một hệ thống “không bao giờ từ chối” và một hệ thống “thỉnh thoảng nói không biết” — cái nào an toàn hơn cho docking, và vì sao câu trả lời không hiển nhiên với người quản lý dự án?

10.1 Đặt vấn đề

A/04 kết thúc bằng một lời khuyên rõ ràng: khử ambiguity bằng cách phá tính đồng phẳng của constellation, và B/09 mục 4 nói cách làm. Nếu bạn đã làm được điều đó thì phần lớn chương này là dự phòng.

Nhưng “dự phòng” không có nghĩa là không cần, và ba tình huống giải thích vì sao — đây là câu trả lời cho Q1.

Tình huống một: che khuất một phần. Constellation của bạn không đồng phẳng nhờ có tag nghiêng ở hai độ sâu khác nhau. Robot tới gần, một phần constellation ra khỏi khung hình hoặc bị che, và những tag còn lại tình cờ *đồng phẳng*. Bạn quay về bài toán hai nghiệm đúng vào lúc cần chính xác nhất.

Tình huống hai: độ không đồng phẳng chưa đủ. B/09 mục 4B nói rằng ngưỡng độ lệch khỏi mặt phẳng là một câu hỏi định lượng chưa có công thức. Nếu thiết kế của bạn ở gần ngưỡng, thì ở một số tư thế nó đủ và ở một số tư thế khác thì không.

Tình huống ba: giai đoạn xa. Ở cự ly xa, tag nhỏ trên ảnh, và A/04 mục 4 chỉ ra rằng kích thước nhỏ làm ambiguity nặng thêm. Một constellation đủ không đồng phẳng ở 0,5m có thể không đủ ở 3m.

Nói cách khác: khử bằng hình học là *điều kiện vận hành bình thường*, và chương này là *chế độ suy giảm có kiểm soát*. Một hệ thống tốt phải có cả hai, và phải biết mình đang ở chế độ nào.

Có một lý do thứ hai để chương này tồn tại riêng, và nó thuộc về cách tổ chức tri thức hơn là về kỹ thuật. A/04 là chương giải thích cơ chế; nó phải nói về hình học, về phép lật, về vì sao thêm điểm không giúp. Chương này là chương về *quyết định*: ngưỡng đặt ở đâu, làm gì khi vượt ngưỡng, ghi gì vào log. Hai loại nội dung ấy phục vụ hai lúc đọc khác nhau — một lúc học, một lúc viết code — và tách chúng ra làm cả hai dùng được hơn.

10.2 Thang bốn bậc

Cấu trúc của chương, và nó cũng là thứ tự ưu tiên khi thiết kế hệ thống.

Bảng 10.1. Bốn bậc xử lý ambiguity, theo thứ tự ưu tiên giảm dần.

	Biện pháp	Cơ chế	Xác suất sai
1	Khử bằng hình học (B/09)	Phá đồng phẳng $\Rightarrow$ nghiệm thứ hai không còn là cực tiểu	Bằng không
2	Đo ρ và từ chối	Không chọn khi không phân biệt được	Bằng không, nhưng mất dữ liệu
3	Nhất quán thời gian	Chọn nghiệm gần dự đoán	Khác không; có chế độ khoá cứng
4	Fuse cảm biến khác	IMU loại nghiệm sai hướng	Khác không; phụ thuộc hình học

Điều quan trọng nhất trong bảng là cột cuối, và nó là kết luận trung tâm của chương: *chỉ bậc 1 có xác suất sai bằng không*. Bậc 2 không sai nhưng nó không cho câu trả lời. Bậc 3 và 4 cho câu trả lời với một xác suất sai khác không, và xác suất ấy không giảm về không dù cài đặt khéo tới đâu — vì cả hai nghiệm vẫn đang tồn tại và vẫn giải thích ảnh gần như tốt ngang nhau.

Hệ quả thiết kế: *đừng dùng bậc 3 và 4 để thay cho bậc 1*. Dùng chúng để xử lý các tình huống suy giảm ở mục 1, và luôn kèm bậc 2 làm lưới an toàn.

10.3 Bậc 2: đo ρ và từ chối

Bậc rẻ nhất và bậc nên có trong mọi hệ thống. Nó trả lời Q3 và Q5.

A. Lấy ρ ra

Với OpenCV: `solvePnPGeneric` với cờ `SOLVEPNP_IPPE_SQUARE` trả về mảng nghiệm cùng mảng sai số tái chiếu tương ứng. Với thư viện `apriltag` gốc: `estimate_tag_pose` đã tính sẵn cả hai nghiệm và tỉ số của chúng trong `apriltag_pose.c`.

Chi phí gần như bằng không — IPPE tính cả hai nghiệm dù bạn có dùng hay không. Vậy mà rất nhiều cài đặt chỉ lấy nghiệm thứ nhất và vứt phần còn lại. Đây là một trong những cải thiện dễ nhất trong cả cây: *ghi ρ vào log ở mọi khung*, kể cả trước khi bạn quyết định làm gì với nó. Chuỗi ρ theo thời gian tự nó là dữ liệu chẩn đoán giá trị.

B. Ngưỡng đặt ở đâu

Đây là Q3, và câu trả lời trung thực là: *không có hằng số phổ quát, và bạn phải dẫn ra nó cho hệ của mình*.

Lý do không có hằng số: ρ phụ thuộc vào mức nhiễu σ , vào kích thước tag trên ảnh w , vào số điểm, và vào góc nghiêng. Một $\rho = 2$ trên một hệ có $\sigma = 0,05$ px mang nhiều thông tin hơn $\rho = 2$ trên một hệ có $\sigma = 0,3$ px, vì trong trường hợp đầu chênh lệch 2 lần là chênh lệch thật còn trường hợp sau nó có thể là nhiễu.

Cách dẫn ra ngưỡng, và nó là mini project của [A/04](#): chạy Monte Carlo với cấu hình của bạn, ở mỗi lần chạy ghi cả ρ lẫn việc nghiệm được chọn có đúng là nghiệm thật không, rồi vẽ *tỉ lệ chọn sai theo ρ* . Ngưỡng ρ^* của bạn là giá trị mà dưới nó tỉ lệ chọn sai vượt mức bạn chấp nhận được.

Các con số như “ $\rho > 3$ thì tin được” xuất hiện trong tài liệu và tôi đã dùng chúng làm ví dụ ở [A/04](#). Chúng là ví dụ minh họa, không phải giá trị khuyến nghị đã kiểm — và tôi ghi rõ điều này thay vì để chúng trở thành một hằng số được chép đi chép lại.

C. Từ chối là hành vi đúng

Đây là Q5, và nó đáng bàn vì nó là một quyết định kiến trúc bị áp lực từ phía ngoài kỹ thuật.

Một hệ thống “không bao giờ từ chối” trông tốt hơn trong demo: nó luôn có câu trả lời, biểu đồ không có lỗ hổng, và không ai phải xử lý trạng thái “không biết”. Một hệ thống “thình thoảng nói không biết” trông kém tin cậy hơn — và đó chính là lý do câu trả lời không hiển nhiên với người quản lý dự án.

Nhưng với docking, hệ thứ hai an toàn hơn nhiều, và lý do là bất đối xứng của hậu quả. Từ chối làm robot chờ thêm vài trăm mili giây hoặc retry — phiền, an toàn, và tự phục hồi. Chọn nhầm nghiệm làm robot lái về một tư thế sai *với sự tự tin đầy đủ*, và với hai nghiệm cách nhau 2α thì sai lệch có thể hàng chục độ. Hậu quả có thể là va chạm.

Nói cách khác: *một phép đo không đáng tin tẻ hơn không có phép đo*, vì không có phép đo thì tăng trên biết mình đang mù, còn một phép đo sai thì nó không biết.

Cách cài đặt đúng: trả về một trạng thái ba giá trị — pose tin cậy, pose không tin cậy kèm lý do, hoặc không có pose — thay vì một pose và một cờ boolean. Tầng điều khiển ở B/13 mục 8 dùng đúng thông tin ấy cho tiêu chí hội tụ.

CẦN NHỚ

Ba thứ phải ghi vào log ở mọi khung, và cả ba đều gần như miễn phí vì chúng đã được tính:

ρ — tỉ số ambiguity. Chuỗi của nó theo thời gian cho biết bạn đang ở vùng an toàn hay vùng nguy hiểm của không gian tư thế.

Nghiệm nào được chọn — nếu nó nhảy qua lại giữa hai nghiệm thì bạn đang ở vùng $\rho \approx 1$ và mọi pose trong đoạn ấy đáng ngờ.

Lý do từ chối khi có. Phân bố các lý do sau một tháng vận hành chỉ ra nút thắt thật của hệ thống.

Ba dòng log ấy biến một hệ thống hộp đen thành một hệ thống chẩn đoán được, và chúng nên có từ ngày đầu chứ không thêm vào khi đã có sự cố.

10.4 Bậc 3: nhất quán thời gian, và chế độ khoá cứng

Bậc này hiệu quả và nguy hiểm, và mục này dành phần lớn cho phần nguy hiểm. Nó trả lời Q2.

Ý tưởng: chọn nghiệm gần với pose ở khung trước, hoặc gần với dự đoán từ odometry bánh xe. Nó hoạt động vì pose thật thay đổi trơn theo thời gian trong khi nghiệm giả nhảy — nên tiêu chí “gần cái trước” thường chọn đúng.

Chế độ hồng đã được nêu ở A/04 mục 5 và đáng lặp lại vì nó là chế độ hồng nguy hiểm nhất trong chương: *một khi đã chọn nhầm một lần, tiêu chí này giữ bạn ở nghiệm sai vĩnh viễn*. Khung tiếp theo, nghiệm sai gần với “pose khung trước” hơn nghiệm đúng, nên nó lại được chọn. Và cứ thế.

Kết quả là một hệ thống *rất ổn định* quanh một giá trị sai. Đây là câu trả lời cho Q2: sự ổn định không phải bằng chứng của tính đúng. Ngược lại, với bài toán ambiguity thì ổn định *quá* có thể là dấu hiệu của khoá cứng — vì một hệ thống trung thực ở vùng $\rho \approx 1$ phải thể hiện sự do dự.

Ba biện pháp phòng, và nên có cả ba.

Một, luôn kết hợp với bậc 2. Nhất quán thời gian chỉ được phép quyết định khi ρ nằm trong một dải trung gian — đủ thấp để không tự tin, đủ cao để không vô vọng. Khi ρ rất thấp, từ chối thay vì dựa vào khung trước.

Hai, kiểm tra lại bằng một nguồn độc lập định kỳ. Ở mỗi vài giây, hoặc mỗi khi ρ tăng lên cao, hãy bỏ qua lịch sử và chọn thuần theo ρ . Nếu kết quả khác với nghiệm đang bám thì bạn vừa phát hiện một khoá cứng.

Ba, ghi log việc chuyển nghiệm. Một hệ thống khoẻ mạnh thỉnh thoảng chuyển nghiệm ở vùng ρ thấp. Một hệ thống *không bao giờ* chuyển trong khi ρ có lúc xuống gần 1 là đáng ngờ.

Ngoài ra có một chi tiết cài đặt quan trọng: khi dùng dự đoán từ odometry, phải nhớ rằng odometry bánh xe có sai số tích lũy và nó tệ nhất ở đúng thứ ta cần — hướng. Với đoạn tiếp cận ngắn thì nó đủ tốt; với một khoảng thời gian dài thì dự đoán trôi và tiêu chí mất hiệu lực.

10.5 Bậc 4: fuse với cảm biến khác

Bậc cuối, và nó trả lời Q4.

IMU cho roll và pitch khá chính xác, vì trọng lực là một tham chiếu tuyệt đối — gia tốc kế đo lực riêng, và khi robot không gia tốc thì nó đo hướng trọng lực. Câu hỏi: thông tin ấy loại được nghiệm sai không?

Câu trả lời là *đôi khi*, và cái quyết định là hình học — đúng như Q4 gợi ý.

Hai nghiệm của target phẳng khác nhau bởi một phép lật quanh một trục nằm trong mặt phẳng tag, và trục ấy vuông góc với mặt phẳng chứa đường nhìn và pháp tuyến tag (A/04 mục 2). Phép lật đổi hướng của camera so với tag, và nó *cũng* đổi hướng camera so với thế giới — nên nó đổi roll và pitch mà camera suy ra.

Nếu trục lật gần *vuông góc* với hướng trọng lực, phép lật đổi roll hoặc pitch đáng kể, và IMU loại được nghiệm sai dễ dàng. Nếu trục lật gần *song song* với hướng trọng lực — tức phép lật chủ yếu đổi yaw — thì roll và pitch gần như không đổi giữa hai nghiệm, và IMU hoàn toàn vô dụng.

Với một robot di động trên sàn phẳng nhìn một tag gắn thẳng đứng trên tường, trục lật thường gần nằm ngang, nên IMU *có* giúp. Nhưng với một tag nghiêng theo hướng khác, tình huống đổi. Kết luận thực hành: *kiểm bằng mô phỏng cho hình học cụ thể của bạn* thay vì giả định nó giúp hay không giúp.

Ghi chú trung thực: tôi dẫn cơ chế này bằng hình học (cửa a). Tôi *chưa* kiểm nó bằng mô phỏng (cửa b) và chưa tìm được nguồn phát biểu tường minh cho trường hợp marker (cửa c). Đây là một mục trong 99-chua-biet.md, và nó kiểm được dễ dàng bằng script của A/04.

Có một biến thể rẻ hơn IMU và đáng nêu: nếu robot chạy trên sàn phẳng và bạn biết chiều cao lắp camera, thì *chính ràng buộc “camera ở độ cao cố định”* loại được nghiệm sai trong nhiều trường hợp. Nghiệm sai thường ngụ ý camera ở một độ cao khác, và kiểm điều đó tốn một phép so sánh. Đây là một ví dụ của nguyên tắc chung: *mọi thông tin tiên nghiệm bạn có về hình học đều là một cơ hội loại nghiệm*, và sàn phẳng là thông tin gần như luôn có với robot di động.

10.6 Giới hạn và chế độ hỏng

Ba bậc dưới không khử được ambiguity. Chúng chọn trong hai nghiệm vẫn đang tồn tại. Xác suất sai của chúng khác không và không giảm về không. Đây là giới hạn cấu trúc, không phải giới hạn cài đặt.

ρ **chỉ có nghĩa khi mô hình đúng.** Sai số tái chiếu tăng lên vì méo chưa khử hoặc constellation sai cũng làm ρ đổi, theo những cách khó diễn giải. Nếu residual tuyệt đối của bạn cao bất thường thì ρ không còn là chỉ báo tin cậy về ambiguity mà là chỉ báo về một vấn đề khác.

Khoá cứng khó phát hiện từ trong dữ liệu. Biện pháp ở mục 4 giúp nhưng không đảm bảo. Cách chắc chắn duy nhất là một nguồn thông tin độc lập — và với docking, nguồn ấy thường là chính cơ khí: nếu robot cập được thì nghiệm đúng, nếu nó trượt thì sai.

Ngưỡng ρ không chuyển được giữa các hệ. Một ngưỡng dẫn ra cho camera này, tag này, khoảng cách này không áp dụng cho cấu hình khác. Phải dẫn lại khi đổi bất cứ thành phần nào.

10.7 Thực hành

Bốn việc, và cả bốn đều rẻ.

Một, *lấy cả hai nghiệm và ghi ρ vào log ngay từ ngày đầu*, kể cả khi chưa dùng. Dữ liệu ấy không lấy lại được sau này.

Hai, *dẫn ra ngưỡng ρ^* bằng Monte Carlo* theo mini project của A/04, và dẫn lại khi đổi cấu hình.

Ba, trả về trạng thái ba giá trị thay vì một pose và một cờ.

Bốn, nếu dùng nhất quán thời gian, hãy cài cả ba biện pháp phòng khoá cứng ở mục 4, đặc biệt là việc định kỳ bỏ qua lịch sử và chọn thuần theo ρ .

Active recall (che cột phải, tự trả lời, rồi mở ra đối chiếu)

Thuật ngữ / câu hỏi	Trả lời
●●● Vì sao chương này cần dù đã khử bằng hình học	Ba tình huống suy giảm: che khuất làm các tag còn lại tình cờ đồng phẳng; độ không đồng phẳng gần ngưỡng nên đủ ở tư thế này và không đủ ở tư thế khác; và ở cự ly xa tag nhỏ làm ambiguity nặng thêm (A/04 mục 4).
●●● Thang bốn bậc và cột quan trọng nhất	Khử bằng hình học / đo và từ chối / nhất quán thời gian / fuse cảm biến. Cột quan trọng nhất là <i>xác suất sai</i> : chỉ bậc 1 bằng không. Ba bậc dưới chỉ <i>chọn</i> trong hai nghiệm vẫn tồn tại.
●●● Vì sao không có ngưỡng ρ phổ quát	ρ phụ thuộc σ , w , số điểm, góc nghiêng. $\rho = 2$ với $\sigma = 0,05$ px mang nhiều thông tin hơn $\rho = 2$ với $\sigma = 0,3$ px. Phải dẫn ra bằng Monte Carlo cho từng hệ.
●●● Cách dẫn ngưỡng	Chạy Monte Carlo, ghi cả ρ lần việc nghiệm chọn có đúng không, vẽ <i>tỉ lệ chọn sai theo ρ</i> . Ngưỡng là chỗ tỉ lệ sai vượt mức chấp nhận được.
●●● Vì sao từ chối là hành vi đúng	Bất đối xứng hậu quả: từ chối $\Rightarrow$ chờ hoặc retry, an toàn, tự phục hồi. Chọn nhầm $\Rightarrow$ lái về tư thế sai với sự tự tin đầy đủ, lệch có thể hàng chục độ. <i>Một phép đo không đáng tin tệ hơn không có phép đo</i> .
●●● Chế độ khoá cứng	Một khi chọn nhầm, tiêu chí “gắn khung trước” giữ hệ ở nghiệm sai vĩnh viễn — ổn định, tự tin, và sai. Hệ quả chẩn đoán: <i>ổn định không phải bằng chứng của tính đúng</i> .
●●● Ba biện pháp phòng khoá cứng	Chỉ cho nhất quán thời gian quyết định trong dải ρ trung gian; định kỳ <i>bỏ qua lịch sử</i> và chọn thuần theo ρ để phát hiện; và ghi log việc chuyển nghiệm — không bao giờ chuyển trong khi ρ có lúc gần 1 là đáng ngờ.
●● Khi nào IMU loại được nghiệm sai	Khi trục lật gần <i>vuông góc</i> với hướng trọng lực — lúc đó phép lật đổi roll/pitch đáng kể. Khi trục lật gần <i>song song</i> với trọng lực (phép lật chủ yếu đổi yaw) thì IMU vô dụng. (<i>Tôi suy ra từ hình học; chưa kiểm bằng mô phỏng</i> .)
●● Ràng buộc sàn phẳng	Rẻ hơn IMU: nghiệm sai thường ngụ ý camera ở một độ cao khác, và kiểm điều đó tốn một phép so sánh. Ví dụ của nguyên tắc chung — mọi thông tin tiên nghiệm về hình học là một cơ hội loại nghiệm.
●●● Ba thứ phải ghi log mỗi khung	ρ ; nghiệm nào được chọn (nhảy qua lại $\Rightarrow$ đang ở vùng nguy hiểm); và lý do từ chối. Cả ba gần như miễn phí vì đã được tính, và không lấy lại được sau này.
●● Khi nào ρ mất ý nghĩa	Khi residual tuyệt đối cao bất thường — méo chưa khử hoặc constellation sai cũng làm ρ đổi theo cách khó diễn giải. Lúc đó ρ là chỉ báo về một vấn đề khác, không phải về ambiguity.
●● Giao diện trả về đúng	Trạng thái ba giá trị — pose tin cậy / pose không tin cậy kèm lý do / không có pose — thay vì một pose và một cờ boolean. B/13 mục 8 dùng đúng thông tin ấy.

●●● phải thuộc ●● nên nắm ● biết là có

Kiến thức này đang sống ở đâu (công trình đang dùng chính ý tưởng của chương; chốt 9/2026)

Công trình / hệ thống	Dùng ý tưởng nào của chương	Ghi chú
OpenCV solvePnPGeneric	Trả mảng nghiệm kèm mảng sai số tái chiếu $\Rightarrow \rho$ miễn phí	API quan trọng nhất cho chương này
apriltag apriltag_pose.c	estimate_tag_pose tính sẵn cả hai nghiệm và tỉ số	Đọc để thấy bậc 2 đã được cài sẵn trong thư viện gốc
IPPE [17]	Nguồn của việc trả về cả hai nghiệm — một lựa chọn thiết kế API có ý thức	Số bản chính về việc dùng tỉ số residual làm chỉ báo
S-Tag [30]	Giải quyết ở tầng thiết kế marker thay vì tầng chọn nghiệm	Cùng triết lý với bậc 1: sửa hình học, đừng sửa thuật toán
tagslam [31]	Ràng buộc nhiều tag trong factor graph loại nghiệm không nhất quán	Bậc 1 ở quy mô hệ thống; mức tin C

MINI PROJECT · Dựng một hệ thống ba trạng thái và đo tỉ lệ từ chối trên quỹ đạo thật · 1–2 ngày

Mục tiêu	biến ambiguity từ một rủi ro ẩn thành một đại lượng theo dõi được, và biết chính xác hệ của bạn ở vùng nguy hiểm bao nhiêu phần trăm thời gian.
Dữ liệu	phần cứng thật nếu có, hoặc mô phỏng quỹ đạo tiếp cận đầy đủ nếu chưa.
Các bước	sửa pipeline để lấy cả hai nghiệm và ghi ρ ; dẫn ra ngưỡng ρ^* bằng Monte Carlo (mini project A/04); cài trạng thái ba giá trị; chạy toàn bộ quỹ đạo tiếp cận nhiều lần với các điểm xuất phát khác nhau; vẽ ρ theo khoảng cách Z cho mọi lần chạy, chồng lên nhau; đánh dấu các đoạn $\rho < \rho^*$; và thống kê tỉ lệ phần trăm thời gian ở mỗi trạng thái.
Xong khi	bạn có một biểu đồ ρ theo Z và một con số “hệ thống ở trạng thái không tin cậy $X\%$ thời gian, tập trung ở dải Z từ a tới b ”. Nếu dải ấy trùng với pha cuối thì bạn có một vấn đề thiết kế nghiệm trọng và phải quay lại B/09 mục 4. Kiểm tra chéo bắt buộc: nếu ρ của bạn cao ở mọi tư thế kể cả fronto-parallel với constellation đồng phẳng, thì cách tính ρ của bạn sai — A/04 nói rõ rằng cấu hình ấy phải cho $\rho \approx 1$.
Nối với	B/13 mục 8 dùng trạng thái ba giá trị cho tiêu chí hội tụ; C/16 đưa biểu đồ này vào quy trình nghiệm thu.

TRẢ LỜI CÂU HỎI MỞ ĐẦU

- A1. Vì khứ bằng hình học là điều kiện vận hành *bình thường*, không phải điều kiện được đảm bảo ở mọi tư thế. Ba tình huống làm nó mất hiệu lực: che khuất một phần, khi robot tới gần và các tag còn nhìn thấy tình cờ đồng phẳng — đúng vào lúc cần chính xác nhất; độ không đồng phẳng ở gần ngưỡng, nên nó đủ ở tư thế này và không đủ ở tư thế khác; và giai đoạn xa, khi tag nhỏ trên ảnh làm ambiguity nặng thêm ngay cả với constellation tốt. Chương này là chế độ suy giảm có kiểm soát, và một hệ thống tốt phải biết mình đang ở chế độ nào.
- A2. Vì với bài toán ambiguity, ổn định có thể là triệu chứng của *khoá cứng* chứ không phải bằng chứng của tính đúng. Nếu hệ dùng nhất quán thời gian và đã chọn nhầm một lần, thì ở mọi khung sau đó nghiệm sai gần với “pose khung trước” hơn nghiệm đúng, nên nó tiếp tục được chọn — và bạn có một hệ thống rất ổn định quanh một giá trị sai, không có tín hiệu báo động nào. Một hệ trung thực ở vùng $\rho \approx 1$ phải thể hiện sự *do dự*: nó phải thỉnh thoảng chuyển nghiệm. Nên phép thử là: ghi log việc chuyển nghiệm cùng với ρ , và nếu ρ có lúc xuống gần 1 mà hệ không bao giờ chuyển thì đáng ngờ. Biện pháp chắc chắn hơn: định kỳ bỏ qua lịch sử và chọn thuần theo ρ , rồi so với nghiệm đang bám.
- A3. Vì ρ phụ thuộc vào mức nhiễu σ , kích thước tag trên ảnh w , số điểm và góc nghiêng — nên cùng một giá trị ρ mang lượng thông tin khác nhau trên hai hệ khác nhau. Cụ thể: $\rho = 2$ trên một hệ có $\sigma = 0,05$ px nghĩa là chênh lệch thật, còn $\rho = 2$ trên hệ có $\sigma = 0,3$ px có thể chỉ là nhiễu. Cách dẫn ra ngưỡng cho hệ của mình: chạy Monte Carlo với cấu hình thật, ở mỗi lần ghi cả ρ lẫn việc nghiệm được chọn có đúng là nghiệm thật không, rồi vẽ tỉ lệ chọn sai theo ρ ; ngưỡng là chỗ tỉ lệ ấy vượt mức bạn chấp nhận được. Và phải dẫn lại khi đổi bất kỳ thành phần nào của cấu hình.
- A4. Cái quyết định là *hướng của trục lật so với hướng trọng lực*. Hai nghiệm khác nhau bởi một phép lật quanh một trục nằm trong mặt phẳng tag (A/04 mục 2), và phép lật ấy đổi hướng camera so

với thế giới. Nếu trục lật gần vuông góc với hướng trọng lực, phép lật đổi roll hoặc pitch một lượng đáng kể và IMU — vốn đo hướng trọng lực khá chính xác — loại được nghiệm sai dễ dàng. Nếu trục lật gần song song với trọng lực, tức phép lật chủ yếu đổi yaw, thì roll và pitch gần như không đổi giữa hai nghiệm và IMU hoàn toàn vô dụng vì yaw không quan sát được từ trọng lực. Với robot trên sàn nhìn tag thẳng đứng thì trục lật thường gần nằm ngang nên IMU có giúp, nhưng với tag nghiêng thì phải kiểm lại. Có một biến thể rẻ hơn: ràng buộc “camera ở độ cao cố định” trên sàn phẳng cũng loại được nghiệm sai trong nhiều trường hợp, và nó không cần cảm biến nào cả.

- A5. Hệ “thình thoảng nói không biết” an toàn hơn nhiều, vì hậu quả của hai loại lỗi bất đối xứng: từ chối làm robot chờ vài trăm mili giây hoặc retry — phiền, an toàn, và tự phục hồi; còn chọn nhầm nghiệm làm robot lái về một tư thế lệch có thể hàng chục độ với sự tự tin đầy đủ, và hậu quả có thể là va chạm. Nguyên tắc chung: một phép đo không đáng tin tẻ hơn không có phép đo, vì không có phép đo thì tăng trên *biết* mình đang mù. Câu trả lời không hiển nhiên với người quản lý dự án vì hệ không bao giờ từ chối trông tốt hơn trong demo — luôn có câu trả lời, biểu đồ không có lỗ hổng, không ai phải xử lý trạng thái thứ ba. Cái giá của vẻ ngoài ấy chỉ lộ ra khi có sự cố, và lúc đó không có log nào giải thích được vì sao.

NÓI LẠI THẬT ĐƠN GIẢN

Cái tag phẳng có hai câu trả lời và máy không phân biệt được. Chương trước nói cách chữa tận gốc: kê nghiêng mấy tấm tag cho chúng đứng phẳng nữa. Chương này nói phải làm gì trong những lúc cách chữa ấy không đủ — chẳng hạn robot tới gần quá, mấy tấm nghiêng đã ra ngoài khung hình, chỉ còn mấy tấm phẳng ở giữa.

Có bốn mức xử lý, và mức đầu tiên là mức duy nhất thật sự chắc chắn: sửa hình học. Ba mức còn lại chỉ giúp *đoán đúng hơn* giữa hai câu trả lời vẫn đang có, nên chúng luôn có khả năng đoán sai. Mức thứ hai, và nên có trong mọi hệ: *đo xem hai câu trả lời có khớp ảnh ngang nhau tới mức nào*. Máy đã tính sẵn con số đó rồi, chỉ là phần lớn chương trình vứt nó đi. Nếu một câu rõ ràng khớp hơn thì tin; nếu hai câu ngang nhau thì đừng đoán — hãy nói “không biết”.

Nói “không biết” nghe như thất bại, nhưng nó là hành vi đúng, và lý do rất đơn giản: nếu robot không biết thì nó chờ thêm nửa giây hoặc lùi ra thử lại, chẳng mất gì. Còn nếu nó đoán sai thì nó lái đi một hướng lệch hàng chục độ mà vẫn tin chắc mình đúng. Một con số sai tẻ hơn không có con số nào, vì không có con số thì ít ra người ta biết là đang mù.

Mức thứ ba: chọn câu trả lời nào gần với câu trả lời lần trước. Cách này hiệu quả, và nó có một cái bẫy rất xấu. Nếu có một lần đoán nhầm, thì lần sau cái nhầm ấy lại gần với “lần trước” hơn cái đúng, nên nó lại được chọn. Và cứ thế mãi. Kết quả là một hệ thống *rất ổn định* quanh một giá trị sai — không nhảy nữa, không báo lỗi, mọi thứ trông êm đẹp. Nên với bài toán này, ổn định quá lại là dấu hiệu đáng ngờ chứ không phải dấu hiệu tốt. Cách phòng: thỉnh thoảng cố tình quên lịch sử đi và chọn lại từ đầu, rồi xem có ra kết quả khác không.

Mức thứ tư: hỏi một cảm biến khác. Cảm biến quán tính biết khá rõ robot đang nghiêng bao nhiêu so với phương thẳng đứng, và đôi khi thông tin đó loại được một câu trả lời. Nhưng chỉ *đôi khi* — nếu hai câu trả lời chỉ khác nhau ở hướng quay quanh trục đứng, thì cảm biến quán tính không nói được gì, vì nó không biết hướng bắc.

Cuối cùng, một lời khuyên rẻ nhất trong cả chương: *ghi lại mọi thứ vào nhật ký ngay từ ngày đầu*. Con số so khớp của hai câu trả lời, câu nào được chọn, và lý do mỗi lần từ chối. Ba dòng ghi chép đó không tốn gì vì máy đã tính rồi, và một tháng sau chúng là thứ duy nhất cho bạn biết hệ thống thật sự đang gặp chuyện gì.

Chỗ không rút gọn được: ngưỡng “khớp hơn bao nhiêu thì đủ để tin” thì không có con số chung, vì nó phụ thuộc camera, khoảng cách, kích thước tag. Phải tự chạy thử mà tìm, và không chép được từ hệ khác.

Một câu: Ba cách dưới chỉ giúp đoán khéo hơn giữa hai câu trả lời; chỉ có cái nệm nghiêng mới làm biến mất câu trả lời thứ hai.

10.8 Điều gì chứng minh cách hiểu này sai

Mệnh đề chính — *ba bậc dưới có xác suất sai khác không và không thay thế được bậc 1* — là một mệnh đề cấu trúc: nếu hai nghiệm giải thích ảnh tốt ngang nhau thì không thông tin nào *trong ảnh* phân biệt được chúng. Nó bị bác bỏ chỉ nếu ai đó chỉ ra một đại lượng tính từ chính ảnh mà phân biệt được hai nghiệm ở tư thế fronto-parallel — và điều đó mâu thuẫn với A/04.

Mệnh đề thứ hai — *chế độ khoá cứng xảy ra trong thực tế* — bị bác bỏ nếu trên một hệ thực với nhất quán thời gian, việc định kỳ bỏ qua lịch sử không bao giờ phát hiện ra sự bất đồng. Đó sẽ là tin tốt, và nó có thể xảy ra nếu hệ của bạn hiếm khi vào vùng $\rho \approx 1$ — tức nếu B/09 đã làm tốt việc của nó.

Mệnh đề thứ ba — *IMU loại được nghiệm sai khi trục lật vuông góc với trọng lực* — là suy luận hình học chưa kiểm của tôi, đã ghi rõ ở mục 5. Nó bị bác bỏ nếu mô phỏng cho thấy roll và pitch của hai nghiệm gần như luôn giống nhau bất kể hình học. Đây là mệnh đề dễ kiểm nhất trong chương.

10.9 Kết nối

Chương này là chương phụ thuộc nhiều nhất trong cây và nó tự nhận điều đó.

Nối lên trên. A/04-pose-ambiguity-target-phang cấp toàn bộ cơ chế; chương này chỉ bàn quyết định. Mini project của chương ấy là cách dẫn ra ngưỡng ρ^* mà mục 3 cần. A/03-pnp-va-uoc-luong-tu-the cấp solvePnPGeneric và giải thích vì sao khởi tạo bằng IPPE quan trọng.

Nối ngang. B/09-multi-marker-constellation là bậc 1 của thang ở mục 2, và quan hệ giữa hai chương là quan hệ “chính” và “dự phòng”. Nếu B/09 làm tốt thì chương này hiếm khi được kích hoạt — và mini project ở đây chính là cách đo xem B/09 đã làm tốt tới đâu. B/11-uoc-luong-theo-thoi-gian là chương mà mục 4 cảnh báo: mọi kỹ thuật làm mượt theo thời gian chỉ hợp lệ *sau khi* nghiệm đã được chọn tin cậy, và chương này là điều kiện tiên quyết của chương ấy.

Nối xuống. B/13-docking-control mục 8 là chỗ trạng thái ba giá trị ở mục 3C được tiêu thụ: tiêu chí hội tụ đòi “phép đo đáng tin”, và ρ là một trong các thành phần của điều kiện ấy. Và một nỗi đáng nêu: B/13 mục 2 chỉ ra rằng IBVS *không* giải PnP nên không có ambiguity — tức toàn bộ chương này trở nên không liên quan nếu bạn chọn IBVS cho pha cuối. Đây là một trong những lợi ích lớn nhất của IBVS và nó đáng cân nhắc nghiêm túc.

Câu hỏi tự kiểm (trả lời không nhìn bài)

1. Nêu ba tình huống mà constellation không đồng phẳng vẫn rơi vào bài toán hai nghiệm.
2. Trong bốn bậc xử lý, bậc nào có xác suất sai bằng không, và vì sao ba bậc còn lại không đạt được điều đó dù cài đặt khéo tới đâu?
3. Vì sao không thể tra ngưỡng ρ từ một bảng, và quy trình dẫn ra nó gồm những bước nào?
4. Mô tả chế độ khoá cứng, giải thích vì sao nó làm “ổn định” trở thành dấu hiệu đáng ngờ, và nêu biện pháp phát hiện chắc chắn nhất.
5. Điều gì quyết định việc IMU có loại được nghiệm sai hay không? Nêu một cấu hình mà nó giúp và một cấu hình mà nó vô dụng.
6. Một hệ thống trả về “pose và một cờ boolean” thiếu thông tin gì so với trạng thái ba giá trị, và tầng nào bị thiệt?
7. Vì sao toàn bộ chương này trở nên không liên quan nếu pha cuối dùng IBVS?

Ước lượng pose theo thời gian (Temporal Pose Estimation)

PHỤ THUỘC A/05-lan-truyen-sai-so-pixel-sang-pose (mục 8, phân loại nhiễu và hệ thống — chương này chỉ xử lý được loại thứ nhất); B/08-corner-refinement (hiệp phương sai của từng góc — đầu vào cho mọi phép hợp nhất có trọng số); B/10-xu-ly-ambiguity (điều kiện tiên quyết: phải chọn nghiệm tin cậy *trước* khi làm mượt, nếu không bạn đang làm mượt giữa hai nghiệm khác nhau).

TÓM TẮT

Chương này bàn cách dùng *thời gian* để giảm sai số, và nó có một điều kiện tiên quyết nghiêm ngặt phải nói ngay: mọi kỹ thuật ở đây chỉ hợp lệ sau khi bài toán hai nghiệm đã được xử lý — làm mượt giữa hai nghiệm khác nhau cho ra một pose không phải nghiệm nào cả (A/04 mục 2). Với điều kiện ấy được thoả, có bốn công cụ theo thứ tự phức tạp tăng dần: trung bình khi đứng yên, trung bình đúng cách trên đa tạp, lọc đệ quy với phép đo là *toạ độ góc* chứ không phải pose, và hợp nhất trên đồ thị nhân tử. Hai điểm kỹ thuật quan trọng nhất: *trung bình quaternion thô là sai* vì q và $-q$ biểu diễn cùng một xoay; và *phép đo phải là reprojection, không phải pose*, vì nhiễu là Gauss trên pixel chứ không Gauss trên $SE(3)$. Nếu chỉ nhớ một điều: *chương này chỉ chạm tới nguồn ngẫu nhiên* — nên nó trở nên quan trọng *hơn* sau khi B/13 đã triệt tiêu các nguồn hệ thống, chứ không phải thay thế cho việc đó.

KIẾN THỨC CẦN CÓ TRƯỚC

- **đa tạp (manifold)** — không gian cong mà cục bộ trông như không gian Euclid. $SO(3)$ và $SE(3)$ là đa tạp: không có phép cộng, nên “trung bình” phải được định nghĩa lại.
- **trung bình Karcher** — điểm cực tiểu hoá tổng bình phương *khoảng cách trên đa tạp* tới các điểm cần trung bình. Với $SO(3)$ nó được tính bằng một vòng lặp: đưa mọi điểm về không gian tiếp tuyến tại ước lượng hiện tại, trung bình ở đó, cập nhật, lặp [35].
- **không gian tiếp tuyến** — xấp xỉ tuyến tính của đa tạp tại một điểm. Với $SO(3)$ tại một xoay R , nó là không gian ba chiều của các vectơ xoay nhỏ, và ánh xạ mũ đưa nó trở lại đa tạp.
- **lọc đệ quy** — cập nhật ước lượng theo từng phép đo mới thay vì giải lại toàn bộ. Kalman mở rộng (EKF) là dạng phổ biến nhất.
- **fixed-lag smoothing** — giữ một cửa sổ vài giây gần nhất và tối ưu lại toàn bộ cửa sổ mỗi khi có phép đo mới. Chính xác hơn lọc đệ quy vì nó tuyến tính hoá lại, tốn hơn vì nó giải một bài toán lớn hơn.
- **đổi mới (innovation)** — hiệu giữa phép đo thực tế và phép đo dự đoán từ trạng thái hiện tại. Độ lớn chuẩn hoá của nó là công cụ loại ngoại lai có cơ sở thống kê.

CÂU HỎI MỞ ĐẦU

- Q1. Bạn có bốn pose từ bốn khung liên tiếp và muốn lấy trung bình. Nếu bạn cộng bốn quaternion rồi chuẩn hoá, kết quả có thể sai hoàn toàn — không phải sai một chút. Cơ chế gì, và vì sao nó chỉ xuất hiện đôi khi?
- Q2. Một EKF chuẩn cho docking nhận đầu vào là pose từ PnP. Có một cách thiết kế tốt hơn, và nó đổi *đầu vào* chứ không đổi bộ lọc. Cách ấy là gì, và nó sửa vấn đề gì?

- Q3.** Robot đang dừng và bạn lấy trung bình 100 khung. Nêu chính xác dòng nào trong bảng ngân sách của [C/15](#) được cải thiện, và dòng nào hoàn toàn không đổi.
- Q4.** Chương [B/13](#) lập luận rằng teach-by-showing triệt tiêu sai số hệ thống. Vì sao điều đó làm chương này *quan trọng hơn* chứ không phải ít quan trọng đi?
- Q5.** Bạn muốn làm mượt pose để nó bớt nhảy. Nêu điều kiện tiên quyết mà nếu không thoả thì việc làm mượt biến một vấn đề nhìn thấy được thành một vấn đề ẩn.

11.1 Đặt vấn đề

Mọi chương trước đều xử lý *một* khung hình: một ảnh vào, một pose ra. Chương này hỏi: nhiều khung hình cho ta thêm gì?

Câu trả lời ngắn gọn là $1/\sqrt{N}$ — đã nêu ở [B/08](#) mục 5. Nhưng câu trả lời ngắn gọn ấy giấu ba câu hỏi con mà chương này phải trả lời.

Câu hỏi thứ nhất: trung bình cái gì? Trung bình pose thì pose sống trên đa tạp và phép cộng không tồn tại. Trung bình tọa độ góc thì đơn giản hơn nhưng chỉ dùng được khi camera đứng yên. Hai lựa chọn ấy không tương đương, và mục 3 bàn.

Câu hỏi thứ hai: trung bình thế nào khi camera đang chuyển động? Lúc ấy pose thật đang đổi, nên trung bình thô là sai. Cần một mô hình chuyển động, và ta bước vào địa hạt của lọc và làm mượt — mục 4 và 5.

Câu hỏi thứ ba, và nó là câu hỏi quan trọng nhất: trung bình giúp được gì và không giúp được gì? Đây là chỗ [A/05](#) mục 8 quay lại: trung bình giảm nguồn *ngẫu nhiên* theo $1/\sqrt{N}$ và hoàn toàn không chạm tới nguồn *hệ thống*. Nếu ngân sách của bạn bị chi phối bởi sai số constellation hoặc bởi méo còn sót, thì cả chương này gần như vô ích.

Sự thật cuối cùng ấy dẫn tới một quan hệ đáng chú ý với [B/13](#), và nó trả lời Q4. Teach-by-showing triệt tiêu các nguồn hệ thống. Sau khi nó đã làm việc ấy, phần còn lại của ngân sách *chỉ gồm nguồn ngẫu nhiên* — tức chính xác thứ mà chương này xử lý. Nói cách khác: hai chương không cạnh tranh mà nối tiếp, và chương này trở nên quan trọng *hơn* sau khi chương kia đã làm việc của nó.

Có một điều kiện tiên quyết phải nói ngay, và nó là câu trả lời cho Q5. Mọi kỹ thuật trong chương giả định rằng các pose được đưa vào *cùng thuộc về một nghiệm*. Nếu bài toán hai nghiệm chưa được xử lý ([B/10](#)), thì bạn đang trung bình giữa hai giá trị cách nhau 2α , và kết quả rơi vào giữa — một pose không phải nghiệm nào cả, và theo [A/04](#) mục 2 thì nó luôn rơi đúng vào tư thế fronto-parallel. Tệ hơn, việc làm mượt *xóa mất triệu chứng nhảy*, nên bạn mất dấu hiệu duy nhất cho biết mình có vấn đề. Đây là ví dụ điển hình của việc một kỹ thuật hợp lệ áp dụng sai thứ tự biến một vấn đề nhìn thấy được thành một vấn đề ẩn.

TRỰC GIÁC

Hình dung việc đo chiều cao một cái cây bằng cách nhìn từ xa, mỗi lần nhìn cho một con số hơi khác.

Nếu cây đứng yên và bạn đứng yên, trung bình nhiều lần nhìn là câu trả lời đúng — và càng nhiều lần càng tốt, theo căn bậc hai.

Nếu cây đang mọc, trung bình các lần nhìn cho bạn chiều cao *trung bình trong khoảng thời gian đo*, không phải chiều cao hiện tại. Muốn biết hiện tại thì phải có một mô hình về việc nó mọc nhanh thế nào, và dùng mô hình ấy để quy các phép đo cũ về thời điểm hiện tại. Đó là lọc.

Bây giờ đến chỗ tinh tế, và nó là chỗ mà pose khác chiều cao. Chiều cao là một con số — cộng hai chiều cao rồi chia đôi thì ra một chiều cao hợp lệ. Nhưng hướng thì không: cộng hai hướng “quay 170 độ” và “quay -170 độ” rồi chia đôi thì ra “quay 0 độ”, trong khi hai hướng ấy thật ra chỉ cách nhau 20 độ và trung bình đúng phải là 180 độ. Phép cộng thông thường không hiểu rằng hướng quay vòng lại.

Đó là lý do phải có một cách trung bình riêng cho hướng, và cách ấy là: chuyển mọi hướng về “lệch bao nhiêu so với một hướng tham chiếu” — lúc này chúng là những số nhỏ và cộng được — rồi trung bình, rồi cộng ngược trở lại. Lặp vài lần cho tới khi ổn định.

Và một chuyện cuối, về việc *đo cái gì*. Nếu bạn đo chiều cao cây bằng cách đo góc nhìn rồi tính ra chiều cao, thì sai số của bạn thật ra nằm ở phép *đo góc*, không phải ở chiều cao. Trung bình các chiều cao đã tính không giống với trung bình các góc rồi mới tính — và cách thứ hai đúng hơn, vì nó xử lý sai số ở đúng chỗ sai số sinh ra.

11.2 Trường hợp đơn giản nhất: camera đứng yên

Trường hợp này đáng có mục riêng vì với chiến lược stop-and-go của B/13 mục 4, nó là trường hợp *quan trọng nhất* chứ không phải trường hợp tầm thường.

Khi camera và tag đều đứng yên, pose thật là hằng số và bài toán trở thành ước lượng một hằng số từ nhiều phép đo nhiễu. Có hai cách, và chúng không tương đương.

Cách một, trung bình tọa độ góc rồi chạy PnP một lần. Lấy trung bình u_i qua N khung cho từng góc, rồi đưa các tọa độ trung bình ấy vào PnP.

Cách hai, chạy PnP mỗi khung rồi trung bình N pose. Đòi trung bình trên đa tạp (mục 3).

Cách một tốt hơn, và lý do đáng hiểu: nó xử lý sai số ở *đúng chỗ sai số sinh ra*. Nhiều là Gauss trên tọa độ pixel; trung bình N mẫu Gauss cho một mẫu Gauss với phương sai chia N , và PnP chạy trên dữ liệu ít nhiễu hơn là một bài toán tốt hơn. Cách hai chạy PnP trên dữ liệu nhiễu nhiễu rồi trung bình các kết quả — và vì PnP là phi tuyến, trung bình của các nghiệm không bằng nghiệm của bài toán trung bình.

Cách một cũng đơn giản hơn về cài đặt: không cần đa tạp, không cần Karcher, chỉ là phép cộng số thực. Và nó cho một lợi ích phụ: độ lệch chuẩn của N mẫu cho từng góc là một ước lượng *thực nghiệm* của σ , đúng thứ mà B/08 mini project cần và đúng thứ mà A/03 mục 6 cần làm trọng số.

Một chi tiết cài đặt: phải khớp đúng góc nào với góc nào giữa các khung. Với marker có ID và có hướng xác định thì việc này hiển nhiên; đừng giả định thứ tự trả về của detector là ổn định.

CẦN NHỚ

Với stop-and-go, cách đúng là: *trung bình tọa độ góc, rồi chạy PnP một lần.*

Không phải: chạy PnP mỗi khung rồi trung bình pose.

Ba lý do: nó xử lý nhiễu ở đúng chỗ nhiễu sinh ra; nó tránh hoàn toàn bài toán trung bình trên đa tạp; và nó cho σ thực nghiệm miễn phí như sản phẩm phụ.

Đây là một trong những chỗ mà cách đơn giản hơn cũng là cách đúng hơn, và nó đáng kiểm lại trong pipeline của bạn — nhiều cài đặt làm ngược vì thứ tự “phát hiện, tính pose, xử lý pose” nghe tự nhiên hơn.

11.3 Trung bình trên đa tạp: vì sao quaternion thô là sai

Mục này trả lời Q1, và nó cần thiết khi bạn *buộc* phải trung bình pose — chẳng hạn khi hợp nhất pose từ nhiều nguồn có bản chất khác nhau.

A. Cơ chế của lỗi

Một phép xoay biểu diễn bởi quaternion đơn vị q , và q cùng $-q$ biểu diễn *cùng một* phép xoay. Nếu bạn cộng bốn quaternion rồi chuẩn hoá, kết quả phụ thuộc vào dấu bạn tình cờ gán cho từng cái.

Ví dụ cụ thể: hai xoay gần giống nhau, một biểu diễn bởi q_1 và một bởi $q_2 \approx q_1$. Trung bình của q_1 và q_2 cho $\approx q_1$ — đúng. Nhưng nếu thư viện của bạn trả về $-q_2$ thay vì q_2 (và nó có quyền, vì hai cái tương đương), thì $q_1 + (-q_2) \approx 0$, và chuẩn hoá một vectơ gần không cho ra một hướng *hoàn toàn ngẫu nhiên*.

Đây là câu trả lời cho phần thứ hai của Q1: lỗi chỉ xuất hiện *đôi khi*, vì nó phụ thuộc vào việc thư viện tình cờ trả về dấu nào. Một hệ thống có thể chạy đúng hàng tháng rồi đột nhiên cho ra một pose vô nghĩa — và loại lỗi không tái lập được ấy là loại tổn thời gian gỡ nhất.

Biện pháp vá nhanh: trước khi cộng, đảo dấu mọi quaternion sao cho chúng cùng nằm trong một nửa cầu (chẳng hạn ép $q^\top q_{\text{ref}} > 0$). Nó chữa được triệu chứng và nó đáng làm ngay. Nhưng nó không phải trung bình đúng, vì trung bình số học của quaternion vẫn không phải trung bình trên $SO(3)$.

B. Trung bình Karcher

Cách đúng: trung bình Karcher, tức điểm cực tiểu hoá tổng bình phương khoảng cách *trên đa tạp* [35]. Thuật toán là một vòng lặp ngắn:

1. Khởi tạo $\bar{R}$ bằng một trong các R_i .
2. Với mỗi i , tính $\omega_i = \log(\bar{R}^\top R_i)$ — đưa R_i về không gian tiếp tuyến tại $\bar{R}$.
3. Tính $\bar{\omega} = \frac{1}{N} \sum_i \omega_i$ — ở đây cộng được vì đã là vectơ.
4. Cập nhật $\bar{R} \leftarrow \bar{R} \exp(\bar{\omega})$.
5. Lặp từ bước 2 cho tới khi $\|\bar{\omega}\|$ đủ nhỏ.

Vòng lặp hội tụ trong vài bước với các xoay gần nhau — đúng trường hợp của ta. Tài liệu tham chiếu đầy đủ hơn về chủ đề này là [52].

Cần nói rõ điều kiện: trung bình Karcher xác định duy nhất khi các xoay nằm trong một lân cận đủ nhỏ. Với các pose từ các khung liên tiếp thì điều kiện thoả thoả mái. Với hai nghiệm của bài toán ambiguity cách nhau 2α — có thể hàng chục độ — thì nó vẫn tính ra một kết quả, và kết quả ấy vẫn vô nghĩa. Trung bình đúng cách trên đa tạp *không* sửa được việc bạn đang trung bình những thứ không nên trung bình.

Với phần tịnh tiến thì không có vấn đề gì: nó sống trong $\mathbb{R}^3$ và trung bình số học là đúng. Nhưng nếu muốn trọng số hoá thì phải dùng hiệp phương sai, và hiệp phương sai của tịnh tiến *tương quan* với hiệp phương sai của xoay — nên tách chúng ra để xử lý riêng là một xấp xỉ. Cách đúng hoàn toàn là xử lý cả sáu chiều cùng lúc trên $SE(3)$, và đó là điều mà các thư viện như *Sophus* hay *manif* cung cấp.

11.4 Lọc: phép đo phải là reprojection

Khi camera chuyển động, cần một mô hình chuyển động và một bộ lọc. Mục này trả lời Q2, và nó là điểm kỹ thuật quan trọng nhất của chương.

A. Thiết kế thông thường và chỗ nó sai

Thiết kế thông thường: chạy PnP mỗi khung để được pose, rồi đưa pose ấy vào EKF như một phép đo, với một hiệp phương sai 6×6 nào đó.

Vấn đề: *nhiều không phải Gauss trên $SE(3)$* . Nhiều là Gauss trên tọa độ pixel, và phép biến đổi từ pixel sang pose là phi tuyến. Kết quả là phân bố của pose ước lượng không phải Gauss, đặc biệt là ở các hướng yếu (chiều sâu, và góc nghiêng khi gần fronto-parallel). Ép nó thành Gauss bằng cách gán một hiệp phương sai 6×6 là một xấp xỉ, và nó tệ nhất đúng ở những hướng mà ta quan tâm nhất.

Có một vấn đề thứ hai, tinh tế hơn: nếu PnP hội tụ về nghiệm sai ở một khung nào đó, thì phép đo đưa vào bộ lọc là một ngoại lai rất lớn, và một EKF không có cơ chế bảo vệ sẽ bị kéo đi đáng kể.

B. Thiết kế đúng

Giữ nguyên bộ lọc, đổi phép đo: dùng *tọa độ góc trên ảnh* làm phép đo thay vì pose.

Cụ thể: trạng thái của bộ lọc là pose (và có thể cả vận tốc). Hàm đo lấy trạng thái, chiếu các điểm 3D của constellation ra ảnh, và trả về tọa độ dự đoán. Đối mới là hiệu giữa tọa độ đo được và tọa độ dự đoán — một đại lượng trong không gian pixel, nơi nhiều *thật sự* là Gauss.

Ba lợi ích, và chúng đáng kể.

Một, mô hình nhiều đúng. Hiệp phương sai của phép đo là Σ_i của từng góc (B/08 mục 7), một đại lượng có ý nghĩa vật lý rõ ràng và đo được, thay vì một ma trận 6×6 phải đoán.

Hai, loại ngoại lai ở đúng mức. Cổng chi-square trên đối mới của *từng góc* loại được một góc xấu mà vẫn giữ ba góc còn lại của cùng tag — điều mà cổng ở mức pose không làm được.

Ba, bài toán ambiguity được xử lý tự nhiên. Vì bộ lọc có một dự đoán từ trạng thái trước, nó tự nhiên chọn nghiệm gần dự đoán — tức nó thực hiện bậc 3 của B/10 mà không cần logic riêng. Nhưng cảnh báo về khoá cứng ở B/10 mục 4 vẫn áp dụng nguyên vẹn, và phải có cơ chế phát hiện.

Cái giá: hàm đo phức tạp hơn và Jacobian của nó là Jacobian của PnP (A/03 mục 4C) — nhưng bạn đã phải tính nó rồi, nên chi phí thêm gần như bằng không.

Nguyên tắc chung đằng sau chuyện này áp dụng rộng hơn bài toán marker: *giữ phép đo ở dạng nguyên thủy nhất mà mô hình nhiều còn đúng*, thay vì xử lý trước rồi đưa kết quả đã xử lý vào bộ ước lượng. Đây là một nguyên tắc quen thuộc trong ước lượng trạng thái robot [53].

11.5 Fixed-lag smoothing và đồ thị nhân tử

Bậc cao nhất, và nó đáng nêu ngắn gọn.

Lọc đệ quy tuyến tính hoá một lần tại mỗi bước và không quay lại. Nếu tuyến tính hoá ở một điểm tồi — chẳng hạn ngay sau khi hệ thống mất tag rồi thấy lại — sai số ấy không được sửa.

Fixed-lag smoothing giữ một cửa sổ vài giây gần nhất và tối ưu lại toàn bộ cửa sổ mỗi khi có phép đo mới. Nó tuyến tính hoá lại ở mọi bước lặp, nên nó chính xác hơn, và nó xử lý được các phép

đo đến không theo thứ tự thời gian. Với bài toán docking — nơi đoạn quan trọng chỉ vài giây cuối — cửa sổ cố định là một lựa chọn rất tự nhiên: *giữ toàn bộ giai đoạn tiếp cận cuối trong cửa sổ*.

Về mặt cài đặt, cách hiện đại là biểu diễn bài toán dưới dạng *đồ thị nhân tử* [54] và dùng một thư viện như [GTSAM](#) hoặc [Ceres](#). Các nhân tử: reprojection cho mỗi góc mỗi khung, chuyển động (từ odometry bánh xe hoặc mô hình vận tốc), và có thể cả IMU. Ưu điểm so với EKF: nó là cùng một khung với bài toán hiệu chuẩn constellation ở [B/09](#) mục 5, nên bạn dùng lại được công cụ; và nó cho phép đưa các ràng buộc thêm vào một cách tự nhiên — chẳng hạn ràng buộc sàn phẳng đã nêu ở [B/10](#) mục 5.

Khi nào đáng dùng: khi bạn đã fuse nhiều nguồn (marker, odometry, IMU), hoặc khi bạn cần xử lý các phép đo trễ. Với một hệ chỉ có marker và dùng stop-and-go, một EKF đơn giản — hoặc thậm chí chỉ mục 2 — là đủ, và sự phức tạp thêm không trả lại gì.

11.6 Điều chương này không làm được

Mục ngắn nhưng quan trọng nhất về mặt định vị chương trong cây. Nó trả lời Q3 và Q4.

Chương này chỉ chạm tới nguồn ngẫu nhiên. Theo phân loại của [A/05](#) mục 8: nhiễu định vị góc, nhiễu ảnh, và các dao động ngẫu nhiên khác. Chúng giảm theo $1/\sqrt{N}$.

Chương này không chạm tới nguồn hệ thống: méo còn sót, sai số intrinsic, sai số constellation, sai số hand-eye, bias tâm ellipse, rolling shutter và blur nếu còn chuyển động. Lấy trung bình một triệu khung thì chúng vẫn nguyên vẹn.

Đây là câu trả lời cho Q3, và nó đáng phát biểu dưới dạng một lời khuyên thực hành: *trước khi đầu tư vào bộ lọc phức tạp, hãy vẽ đồ thị σ_N theo N* ([B/08](#) mục 5A). Nếu nó bão hoà sớm thì bạn đang bị giới hạn bởi thiên lệch, và một EKF tinh vi hơn không giúp gì — việc cần làm là đi tìm nguồn thiên lệch.

Và đây là câu trả lời cho Q4, và nó là quan hệ đáng chú ý nhất của chương. [B/13](#) mục 3 triết tiêu các nguồn hệ thống bằng teach-by-showing. Sau khi nó làm việc ấy, phần còn lại của ngân sách *chỉ gồm nguồn ngẫu nhiên*. Nói cách khác: teach-by-showing làm cho chương này trở thành chương quyết định phần dư, chứ không phải làm nó thừa. Hai chương là một cặp: một cái xoá các dòng hệ thống, một cái chia các dòng ngẫu nhiên cho $\sqrt{N}$.

CẦN NHỚ

Thứ tự đúng của ba việc, và đảo thứ tự thì hỏng:

Một, chọn nghiệm tin cậy ([B/10](#)). Làm mượt trước khi làm việc này thì bạn đang trung bình giữa hai nghiệm và kết quả luôn rơi vào fronto-parallel.

Hai, xử lý các nguồn hệ thống ([B/09](#) mục 5, [B/12](#), hoặc [B/13](#) mục 3). Làm mượt không chạm tới chúng, nên làm mượt trước thì bạn chỉ đang làm cho một con số sai trở nên ổn định.

Ba, rồi mới làm mượt — và lúc này nó thật sự hiệu quả, vì phần còn lại đúng là nguồn ngẫu nhiên.

Sai lầm phổ biến nhất là làm bước ba trước, vì nó dễ nhất và vì đầu ra trông đẹp ngay lập tức. Nhưng “trông đẹp” ở đây nghĩa là đã xoá mất các triệu chứng chẩn đoán.

11.7 Giới hạn và chế độ hỏng

Làm mượt xoá triệu chứng. Đã bàn ở mục 1 và 6. Đây là chế độ hỏng nghiêm trọng nhất của chương vì nó không gây ra lỗi mới mà *giấu* lỗi cũ.

Trung bình Karcher không sửa được việc trung bình sai thứ. Nó tính đúng trung bình của tập bạn đưa vào; nếu tập ấy gồm hai nghiệm khác nhau thì kết quả vẫn vô nghĩa.

$1/\sqrt{N}$ có điều kiện. Nhiều phải độc lập giữa các khung. B/08 mục 5A liệt kê ba cách vi phạm và cách phát hiện.

Lọc giả định mô hình chuyển động đúng. Nếu robot tăng tốc đột ngột và mô hình giả định vận tốc không đổi, bộ lọc sẽ trễ. Với docking tốc độ thấp thì ít quan trọng, nhưng nó là lý do nữa để thích stop-and-go.

Mọi thứ giả định đồng bộ thời gian đúng. Nếu nhãn thời gian của ảnh lệch so với nhãn thời gian của odometry, bộ lọc hợp nhất hai nguồn ở sai thời điểm và kết quả có thiên lệch phụ thuộc vận tốc — một triệu chứng giống hết rolling shutter (A/06 mục 2), và phân biệt hai nguyên nhân đòi phép thử riêng.

11.8 Thực hành

Bốn việc.

Một, với stop-and-go, dùng cách của mục 2: trung bình toạ độ góc rồi chạy PnP một lần. Đơn giản hơn và đúng hơn.

Hai, nếu phải trung bình pose, đảo dấu quaternion về cùng nửa cầu ngay lập tức như một biện pháp vá, rồi cài trung bình Karcher khi có thời gian.

Ba, nếu dùng bộ lọc, đổi phép đo sang toạ độ góc thay vì pose. Chi phí thêm gần bằng không vì Jacobian đã có.

Bốn, vẽ đồ thị σ_N theo N trước khi đầu tư vào bất kỳ bộ lọc phức tạp nào.

Active recall (che cột phải, tự trả lời, rồi mở ra đối chiếu)

Thuật ngữ / câu hỏi	Trả lời
••• Điều kiện tiên quyết của cả chương	Nghiem phải được chọn tin cậy <i>trước</i> (B/10). Làm mượt giữa hai nghiệm cho pose rơi vào fronto-parallel (A/04 mục 2) và xoá mất triệu chứng nhảy — biến vấn đề nhìn thấy được thành vấn đề ẩn.
••• Với camera đứng yên: cách đúng	Trung bình toạ độ góc rồi chạy PnP một lần. Không phải: PnP mỗi khung rồi trung bình pose. Ba lý do: xử lý nhiễu ở đúng chỗ nó sinh ra; tránh bài toán đa tạp; và cho σ thực nghiệm miễn phí.
••• Vì sao trung bình quaternion thô sai	q và $-q$ biểu diễn cùng một xoay, nên tổng phụ thuộc dấu thư viện tình cờ trả về. Hai xoay gần nhau với dấu ngược cho tổng gần không, và chuẩn hoá vectơ gần không ra hướng <i>ngẫu nhiên</i> .
••• Vì sao lỗi ấy chỉ xuất hiện đôi khi	Vì nó phụ thuộc dấu mà thư viện tình cờ trả về. Hệ có thể chạy đúng hàng tháng rồi đột nhiên cho một pose vô nghĩa — loại lỗi không tái lập được, tốn thời gian gỡ nhất.
•• Vá nhanh và cách đúng	Vá: đảo dấu mọi quaternion về cùng nửa cầu ($q^\top q_{\text{ref}} > 0$). Đúng: trung bình Karcher — lập log, trung bình trong không gian tiếp tuyến, exp, cập nhật [35].

Thuật ngữ / câu hỏi	Trả lời (tiếp)
●●● Điểm kỹ thuật quan trọng nhất của chương	Trong bộ lọc, phép đo phải là <i>toạ độ góc</i> chứ không phải pose. Vì nhiều là Gauss trên <i>pixel</i> , không Gauss trên $SE(3)$; ép nó thành Gauss trên pose là xấp xỉ tệ nhất đúng ở các hướng yếu.
●●● Ba lợi ích của phép đo là reprojection	Mô hình nhiễu đúng (Σ_i đo được thay vì ma trận 6×6 phải đoán); loại ngoại lai ở mức <i>từng góc</i> thay vì cả pose; và chọn nghiệm ambiguity tự nhiên qua dự đoán.
●● Fixed-lag smoothing	Giữ cửa sổ vài giây và tối ưu lại toàn bộ mỗi bước — tuyến tính hoá lại nên chính xác hơn lọc đệ quy, và xử lý được phép đo đến trễ. Với docking, chọn cửa sổ <i>chứa toàn bộ pha tiếp cận cuối</i> .
●● Khi nào dùng đồ thị nhân tử	Khi fuse nhiều nguồn (marker + odom + IMU) hoặc cần xử lý đo trễ. Ưu điểm phụ: cùng khung với hiệu chuẩn constellation ở B/09 mục 5 [54]. Với hệ chỉ marker + stop-and-go thì EKF đơn giản là đủ.
●●● Chương này KHÔNG làm được gì	Không chạm tới nguồn <i>hệ thống</i> : méo còn sót, intrinsic, constellation, hand-eye, bias ellipse, rolling shutter/blur khi còn chuyển động. Trung bình một triệu khung thì chúng vẫn nguyên vẹn.
●●● Quan hệ với B/13	Không cạnh tranh mà <i>nối tiếp</i> . Teach-by-showing xoá các dòng hệ thống; chương này chia các dòng ngẫu nhiên cho $\sqrt{N}$. Nên chương này quan trọng <i>hơn</i> sau khi chương kia đã làm việc của nó.
●●● Thứ tự ba việc, đảo thì hỏng	Chọn nghiệm → xử lý nguồn hệ thống → rồi mới làm mượt. Sai lầm phổ biến là làm bước ba trước vì nó dễ nhất và đầu ra trông đẹp ngay — nhưng “đẹp” ở đây nghĩa là đã xoá triệu chứng chẩn đoán.
●● Chế độ hỏng do lệch đồng bộ thời gian	Nhân thời gian ảnh lệch so với odometry ⇒ hợp nhất ở sai thời điểm ⇒ thiên lệch phụ thuộc vận tốc — triệu chứng <i>giống hệt</i> rolling shutter (A/06 mục 2). Cần phép thử riêng để phân biệt.
● Nguyên tắc chung	Giữ phép đo ở dạng <i>nguyên thủy nhất</i> mà mô hình nhiễu còn đúng, thay vì xử lý trước rồi đưa kết quả đã xử lý vào bộ ước lượng [53].

●●● phải thuộc ●● nên nắm ● biết là có

I Kiến thức này đang sống ở đâu (công trình đang dùng chính ý tưởng của chương; chốt 9/2026)

Công trình / hệ thống	Dùng ý tưởng nào của chương	Ghi chú
Sophus, manif [25]	Cài $SO(3)$, $SE(3)$, log/exp, và trung bình trên đa tạp	Đừng tự cài Karcher — dùng thư viện
GTSAM [54]	Đồ thị nhân tử với fixed-lag smoother sẵn có	Cùng công cụ với B/09 mục 5 — dùng lại được
Ceres [27]	Tối ưu cửa sổ trượt với tham số hoá trên đa tạp	Lựa chọn khi đã dùng Ceres cho BA constellation
Rotation averaging [52]	Tài liệu tham chiếu đầy đủ cho mục 3	Đọc §3 nếu cần trung bình có trọng số hoặc bền với ngoại lai
tagslam [31]	Fuse marker với odometry trong factor graph	Ví dụ kiến trúc cho mục 5; mức tin C

MINI PROJECT · So bốn cách hợp nhất theo thời gian trên cùng dữ liệu · 2 ngày

Mục tiêu biết chính xác mỗi mức phức tạp thêm vào trả lại bao nhiêu, để không đầu tư vào một EKF khi một phép trung bình đã đủ.

Dữ liệu	một chuỗi ảnh thật của quỹ đạo tiếp cận, ghi lại kèm odometry bánh xe và — nếu có — một chuẩn tham chiếu ở điểm cuối.
Các bước	cài bốn đường: (A) PnP một khung, không hợp nhất; (B) trung bình toạ độ góc qua N khung rồi PnP (chỉ ở các đoạn dừng); (C) EKF với phép đo là <i>pose</i> ; (D) EKF với phép đo là <i>toạ độ góc</i> ; chạy cả bốn trên cùng chuỗi; so độ tản mát của pose ở điểm cuối qua nhiều lần chạy, và so sai lệch so với chuẩn tham chiếu nếu có; đồng thời ghi lại chi phí tính toán của từng đường.
Xong khi	bạn có một bảng bốn hàng với ba cột: độ tản mát, sai lệch so với chuẩn, chi phí. Dự đoán của chương: B thắng A rõ rệt về tản mát; D thắng C ở các đoạn mà ambiguity nặng hoặc có góc bị che; và <i>sai lệch so với chuẩn</i> của cả bốn gần như nhau — vì đó là thành phần hệ thống mà không đường nào chạm tới. Kiểm tra chéo bắt buộc: nếu cột sai lệch của bốn đường <i>khác nhau</i> đáng kể thì có gì đó không đúng — hoặc bạn đang so ở các thời điểm khác nhau, hoặc một trong các đường đang chọn nghiệm ambiguity khác.
Nối với	C/15 nhận kết quả cột ba để biết nên đầu tư ở đâu; B/13 mục 4 nhận N tối ưu cho stop-and-go.

TRẢ LỜI CÂU HỎI MỞ ĐẦU

- A1. Cơ chế: q và $-q$ biểu diễn *cùng một* phép xoay, nên tổng của một tập quaternion phụ thuộc vào dấu mà bạn tình cờ gán cho từng cái. Nếu hai xoay gần giống nhau nhưng thư viện trả về q_1 và $-q_2$ với $q_2 \approx q_1$, thì tổng $q_1 + (-q_2) \approx 0$, và chuẩn hoá một vectơ gần không cho ra một hướng hoàn toàn ngẫu nhiên — sai hẳn, không phải sai một chút. Nó chỉ xuất hiện đôi khi vì nó phụ thuộc vào dấu mà thư viện tình cờ trả về, thứ có thể đổi theo đường đi của thuật toán hoặc theo dữ liệu. Hệ quả thực tế: một hệ thống có thể chạy đúng hàng tháng rồi đột nhiên cho một pose vô nghĩa, và loại lỗi không tái lập được ấy là loại tổn thời gian gở nhất. Vá nhanh: ép mọi quaternion về cùng nửa cầu trước khi cộng. Cách đúng: trung bình Karcher trên $SO(3)$ [35].
- A2. Đổi phép đo từ *pose* sang *toạ độ góc trên ảnh*: hàm đo của bộ lọc lấy trạng thái pose, chiếu các điểm 3D của constellation ra ảnh, và đổi mới là hiệu giữa toạ độ đo được và toạ độ dự đoán. Nó sửa vấn đề mô hình nhiều: nhiều là Gauss trên *pixel*, không Gauss trên $SE(3)$, và phép biến đổi giữa hai không gian là phi tuyến — nên ép phân bố pose thành Gauss bằng một ma trận 6×6 là một xấp xỉ, và nó tệ nhất đúng ở các hướng yếu (chiều sâu, góc nghiêng gần fronto-parallel), tức đúng những hướng ta quan tâm nhất. Ba lợi ích kèm theo: hiệp phương sai đầu vào là Σ_i của từng góc — đại lượng đo được thay vì phải đoán; cổng chi-square loại được *một góc* xấu mà giữ ba góc còn lại; và bộ lọc tự nhiên chọn nghiệm ambiguity gần dự đoán. Chi phí thêm gần bằng không vì Jacobian đã phải tính cho PnP.
- A3. Cải thiện: mọi dòng thuộc loại *ngẫu nhiên* — chủ yếu là dòng “nhiều định vị góc”, giảm theo $1/\sqrt{100} = 10$ lần. Hoàn toàn không đổi: mọi dòng thuộc loại *hệ thống* — méo còn sót, sai số intrinsic, sai số constellation so với mô hình, sai số hand-eye, bias tâm ellipse nếu dùng marker tròn. Chúng nguyên vẹn sau một trăm khung cũng như sau một triệu khung. Hệ quả thực hành trực tiếp: trước khi đầu tư vào bộ lọc phức tạp hơn, hãy vẽ đồ thị σ_N theo N trên trục log-log (B/08 mục 5A); nếu nó bão hoà sớm thì bạn đang bị giới hạn bởi thiên lệch và một EKF tinh vi hơn không giúp gì — việc cần làm là đi tìm nguồn thiên lệch.
- A4. Vì hai chương xử lý hai loại nguồn khác nhau và chúng nối tiếp chứ không cạnh tranh. Teach-by-showing triệt tiêu các nguồn *hệ thống* — intrinsic sai, constellation lệch, hand-eye sai — bằng cách làm chúng xuất hiện ở cả pose hiện tại lẫn pose mục tiêu và trừ đi nhau. Sau khi nó làm việc ấy, phần còn lại của ngân sách *chỉ gồm nguồn ngẫu nhiên*, tức chính xác thứ mà chương này chia cho $\sqrt{N}$. Nói cách khác, trước khi có teach-by-showing thì chương này chỉ cải thiện một phần nhỏ của ngân sách và bị các dòng hệ thống át đi; sau khi có nó thì chương này quyết định toàn bộ phần dư. Đây là một ví dụ của việc thứ tự làm quyết định giá trị của từng bước: cùng một kỹ thuật, giá trị khác nhau tùy vào việc gì đã được làm trước.
- A5. Điều kiện tiên quyết: bài toán hai nghiệm phải được xử lý trước (B/10). Nếu không, bạn đang làm mọt giữa hai giá trị cách nhau 2α , và theo A/04 mục 2 thì trung bình của chúng luôn rơi đúng vào tư thế fronto-parallel — một pose không phải nghiệm nào cả và sai một cách có hệ thống. Nhưng tác hại lớn hơn nằm ở chỗ khác: việc làm mọt *xoá mất triệu chứng nhấy*, tức xoá

mất dấu hiệu duy nhất cho biết bạn đang có vấn đề. Trước khi làm mượt, bạn có một hệ thống nhảy loạn — khó chịu nhưng trung thực, và nó đang báo cho bạn biết điều gì đó sai. Sau khi làm mượt, bạn có một hệ thống êm ái và sai có hệ thống, không có tín hiệu nào. Đây là ví dụ điển hình của việc một kỹ thuật hợp lệ áp dụng sai thứ tự biến một vấn đề nhìn thấy được thành một vấn đề ẩn.

NÓI LẠI THẬT ĐƠN GIẢN

Chụp một tấm ảnh thì có nhiều; chụp nhiều tấm rồi lấy trung bình thì bớt nhiều. Đó là toàn bộ ý tưởng của chương này, và phần còn lại là chi tiết về việc làm sao trung bình cho đúng.

Chi tiết thứ nhất: khi robot đứng yên, đừng tính vị trí từng tấm rồi trung bình các vị trí. Hãy trung bình *toạ độ mấy cái góc* trước, rồi mới tính vị trí một lần. Nghe như chuyện thứ tự vụn vặt nhưng nó quan trọng, vì nhiều nằm ở mấy cái góc chứ không nằm ở vị trí — xử lý nhiều ở đúng chỗ nó sinh ra thì tốt hơn.

Chi tiết thứ hai: hướng quay thì không cộng được như số thường. Lấy trung bình của “quay 170 độ” và “quay âm 170 độ” theo kiểu cộng chia đôi thì ra 0 độ, trong khi câu trả lời đúng phải là 180. Với cách máy tính lưu hướng quay thì chuyện này còn tệ hơn: có hai cách viết cho cùng một hướng, và nếu chẳng may lấy mỗi cái một kiểu rồi cộng lại thì ra một kết quả hoàn toàn vô nghĩa. Đáng sợ ở chỗ nó chỉ xảy ra thỉnh thoảng, nên hệ thống chạy đúng hàng tháng rồi bỗng dưng loạn một lần. Chi tiết thứ ba, và là chi tiết đáng nhớ nhất: khi robot đang chạy và cần một bộ lọc, đừng đưa *vị trí đã tính* vào bộ lọc. Hãy đưa *toạ độ mấy cái góc trên ảnh* vào. Lý do vẫn là lý do cũ: cái bị nhiễu là mấy cái góc, còn vị trí chỉ là thứ suy ra từ chúng. Bộ lọc làm việc với thứ bị nhiễu thật thì nó hiểu đúng mình đang đối phó với cái gì.

Và bây giờ là điều quan trọng nhất, thứ mà cả chương này chỉ là một nửa của nó. Chụp nhiều tấm chưa được cái *run tay* nhưng không chưa được cái *lệch ngắm*. Nếu tấm marker của bạn lệch một milimét so với bản vẽ, thì một triệu tấm ảnh vẫn lệch đúng một milimét ấy.

Cho nên chương này chỉ có giá trị sau khi đã dọn xong mấy cái lệch ngắm — đo lại tấm marker, hiệu chuẩn tứ tế, hoặc dùng mẹo chụp ảnh mẫu ở chương sau. Làm ngược thứ tự thì bạn chỉ đang làm cho một con số sai trở nên ổn định hơn, và cái ổn định ấy lại che mất dấu hiệu duy nhất cho biết có gì đó không đúng.

Chỗ không rút gọn được: ranh giới giữa “run tay” và “lệch ngắm” không nhìn ra được từ một lần đo, nhưng có một biểu đồ rõ ràng để biết: vẽ xem sai số giảm thế nào khi tăng số tấm chụp. Giảm đều thì là run tay, cứ chụp thêm. Giảm rồi dừng ở một mức nào đó thì là lệch ngắm, và chụp thêm vô ích.

Một câu: Trung bình đúng chỗ, đúng cách, và đúng thứ tự — và nhớ rằng nó chỉ chữa được một nửa vấn đề.

11.9 Điều gì chứng minh cách hiểu này sai

Mệnh đề chính — *trung bình toạ độ góc rồi PnP một lần tốt hơn PnP mỗi khung rồi trung bình pose* — bị bác bỏ nếu mô phỏng cho thấy hai cách cho cùng độ tản mát. Tôi dẫn nó bằng lập luận về chỗ nhiễu sinh ra và tính phi tuyến của PnP (cửa a); nó chưa qua cửa (b). Chênh lệch có thể nhỏ khi nhiễu nhỏ — vì khi ấy PnP gần tuyến tính — nên đây là mệnh đề mà tôi dự đoán sẽ đúng về chiều nhưng khiêm tốn về độ lớn.

Mệnh đề thứ hai — *phép đo là reprojection tốt hơn phép đo là pose* — bị bác bỏ nếu đường C và đường D trong mini project cho kết quả tương đương ở mọi đoạn. Tôi dự đoán chênh lệch tập trung ở các đoạn có góc bị che hoặc ambiguity nặng, và nếu chuỗi dữ liệu của bạn không có đoạn nào như thế thì hai đường sẽ giống nhau — điều đó không bác bỏ mệnh đề, nó chỉ nói dữ liệu chưa đủ khắc nghiệt.

Mệnh đề thứ ba — *làm mượt không chạm tới nguồn hệ thống* — là một mệnh đề định nghĩa hơn là một mệnh đề thực nghiệm: nếu một nguồn giảm khi lấy trung bình thì theo định nghĩa nó là

ngẫu nhiên. Điều đáng kiểm là phân loại cụ thể: nguồn nào trên hệ của bạn thật sự là hệ thống. Cột “sai lệch so với chuẩn” trong mini project là phép kiểm ấy.

11.10 Kết nối

Chương này là chương kết nối nhiều nhất trong Phần B và nó nằm ở giữa một chuỗi phụ thuộc nghiêm ngặt.

Nối lên trên. [A/05-lan-truyen-sai-so-pixel-sang-pose](#) mục 8 cấp phân loại quyết định phạm vi của cả chương. [A/03-pnp-va-uoc-luong-tu-the](#) mục 4C cấp Jacobian mà mục 4B ở đây dùng làm hàm đo. [A/04-pose-ambiguity-target-phang](#) mục 2 cấp cảnh báo về trung bình giữa hai nghiệm.

Nối ngang, theo thứ tự phụ thuộc. [B/10-xu-ly-ambiguity](#) là điều kiện tiên quyết cứng: phải chạy trước. [B/08-corner-refinement](#) cấp Σ_i cho mục 4B, và mục 5A ở đó cấp đồ thị chẩn đoán mà mục 6 ở đây khuyến nghị. [B/09-multi-marker-constellation](#) mục 5 và [B/12-calibration-cho-docking](#) là nơi các nguồn hệ thống được xử lý — tức nơi phải làm xong trước khi chương này có giá trị đầy đủ. [B/13-docking-control](#) mục 3 là lựa chọn thay thế cho việc ấy, và mục 4 ở đó tạo điều kiện cho mục 2 ở đây.

Nối xuống. [C/15-ngan-sach-sai-so](#) nhận từ mục 6 một yêu cầu về cách trình bày bảng ngân sách: phải ghi rõ dòng nào chia được cho $\sqrt{N}$ và dòng nào không, vì hai loại ấy phản ứng khác nhau với chiến lược thời gian. [C/16-danh-gia-va-nghiem-thu](#) nhận mini project của chương này, và nó cũng nhận một cảnh báo phương pháp luận: khi đo độ tán mát của một hệ có làm mượt, con số thu được *không* phải độ chính xác — làm mượt giảm tán mát mà không giảm thiên lệch, nên nó làm con số trông đẹp hơn thực tế.

Câu hỏi tự kiểm (trả lời không nhìn bài)

1. Vì sao làm mượt trước khi chọn nghiệm biến một vấn đề nhìn thấy được thành một vấn đề ẩn?
2. Với camera đứng yên, nêu hai cách trung bình và ba lý do cách thứ nhất tốt hơn.
3. Mô tả cơ chế làm trung bình quaternion thô cho kết quả ngẫu nhiên, và giải thích vì sao lỗi ấy khó tái lập.
4. Trong một bộ lọc, phép đo nên là gì và vì sao? Nêu ba lợi ích cụ thể của lựa chọn đúng.
5. Bạn lấy trung bình 100 khung. Phân loại các dòng ngân sách thành hai nhóm theo việc chúng có được cải thiện không, và nêu một dòng ở mỗi nhóm.
6. Vì sao [B/13](#) làm chương này quan trọng hơn chứ không phải ít quan trọng đi?
7. Một hệ có lệch đồng bộ thời gian cho triệu chứng giống hết một hiệu ứng khác trong cây. Hiệu ứng nào, và vì sao cần một phép thử riêng để phân biệt?

Calibration cho bài toán docking (Calibration for Docking)

PHỤ THUỘC A/01-mo-hinh-camera-va-distortion (mô hình méo, vì sao điểm chính khó ước lượng, và phép thử residual có cấu trúc); B/09-multi-marker-constellation (mục 5, đo constellation bằng bundle adjustment — chương này đặt nó vào đúng vị trí trong chuỗi).

TÓM TẮT

Chương này bàn ba phép hiệu chuẩn mà bài toán docking cần, và điều quan trọng nhất là *thứ tự* của chúng: intrinsic trước, constellation sau, hand-eye cuối — vì mỗi bước dùng kết quả của bước trước làm hằng số. Nội dung kỹ thuật gồm: hiệu chuẩn intrinsic chất lượng cao với yêu cầu về độ phủ dữ liệu, phép thử residual có cấu trúc để phân biệt sai số mô hình với sai số tham số, đo constellation bằng BA, và hand-eye qua bài toán $AX = XB$. Chương cũng có một đặc điểm hiếm trong cây: *tầm quan trọng của nó phụ thuộc vào một quyết định ở chương khác*. Nếu bạn dùng teach-by-showing (B/13 mục 3), phần lớn sai số hiệu chuẩn là nguồn hệ thống và chúng triệt tiêu — yêu cầu chuyển từ *độ đúng sang độ ổn định*, một yêu cầu dễ hơn nhiều. Nếu chỉ nhớ một điều: *pose bạn thực sự cần là dock frame so với base frame; marker chỉ là một mắt xích, và mọi extrinsic trên chuỗi đều cộng sai số*.

KIẾN THỨC CẦN CÓ TRƯỚC

- **intrinsic** — các tham số nội của camera: f_x, f_y, c_x, c_y và các hệ số méo (A/01). Chúng mô tả camera biến tia nhìn thành pixel thế nào, và chúng không phụ thuộc camera đặt ở đâu.
- **extrinsic** — phép biến đổi cứng giữa hai hệ toạ độ. Trong bài toán này có ít nhất ba: camera so với thân robot, tag so với hệ dock, và hệ dock so với điểm cập chuẩn.
- **hand-eye calibration** — tìm phép biến đổi giữa camera và cơ cấu mang nó, từ một tập các cặp chuyển động tương ứng. Bài toán chuẩn có dạng $AX = XB$ với X là ẩn [55].
- **base frame** — hệ toạ độ gắn với thân robot, thường ở tâm trục bánh hoặc ở điểm điều khiển. Đây là hệ mà bộ điều khiển làm việc, và mọi phép đo cuối cùng phải quy về nó.
- **dock frame** — hệ toạ độ gắn với dock, đặt sao cho vị trí cập chuẩn có một toạ độ đơn giản trong nó.
- **điều kiện kích thích** — yêu cầu rằng dữ liệu hiệu chuẩn phải chứa đủ thông tin để tách các tham số khỏi nhau. Thiếu nó thì bài toán suy biến và nghiệm trôi tự do theo một hướng — một dạng của bài toán khả quan sát.
- **residual có cấu trúc** — phần dư sau khi khớp mà không giống nhiễu trắng mà tạo thành mẫu hình không gian. Đây là dấu hiệu của sai số mô hình chứ không phải sai số tham số (A/01 mục 6).

CÂU HỎI MỞ ĐẦU

- Q1. Ba phép hiệu chuẩn — intrinsic, constellation, hand-eye — phải làm theo một thứ tự nhất định. Thứ tự nào, và điều gì hỏng nếu bạn đảo hai bước cuối?
- Q2. Bạn hiệu chuẩn camera với 50 ảnh, target luôn ở giữa khung hình và luôn gần vuông góc. Residual trung bình 0,2 px. Nêu hai tham số mà bạn gần như chắc chắn đã ước lượng sai, và giải thích vì sao residual không phát hiện được.

- Q3. Bài toán hand-eye $AX = XB$ có một điều kiện về chuyển động mà nếu vi phạm thì nghiệm không xác định. Điều kiện ấy là gì, và vì sao một robot vi sai trên sàn phẳng lại đặc biệt dễ vi phạm nó?
- Q4. Pose bạn thực sự cần không phải pose của tag so với camera. Nó là gì, chuỗi biến đổi gồm mấy khâu, và khâu nào thường bị bỏ quên?
- Q5. Nếu bạn dùng teach-by-showing, chương này trở nên ít quan trọng hơn nhiều. Nhưng *một* yêu cầu vẫn còn nguyên và thậm chí quan trọng hơn. Yêu cầu nào?

12.1 Đặt vấn đề

Có một câu hỏi mà người mới làm docking hiếm khi đặt ra, và nó là câu hỏi quyết định kiến trúc của cả hệ thống: *pose bạn thực sự cần là pose gì?*

Câu trả lời tự nhiên — pose của tag so với camera — là sai, hoặc chính xác hơn là chưa đủ. Cái bộ điều khiển cần là vị trí của *điểm cập chuẩn trên dock* so với *thân robot*. Viết ra chuỗi biến đổi:

$$T_{\text{dock}}^{\text{base}} = T_{\text{cam}}^{\text{base}} \cdot T_{\text{tag}}^{\text{cam}} \cdot T_{\text{dock}}^{\text{tag}}. \quad (12.1)$$

Ba khâu, và chỉ khâu giữa là thứ mà thị giác đo được. Hai khâu còn lại là *hằng số phải hiệu chuẩn*, và sai số của chúng cộng thẳng vào kết quả.

Đây là câu trả lời cho Q4, và khâu thường bị bỏ quên là khâu cuối — $T_{\text{dock}}^{\text{tag}}$, quan hệ giữa hệ toạ độ của constellation và điểm cập chuẩn thật. Người ta hay đo cẩn thận vị trí các tag với nhau (B/09 mục 5) rồi quên rằng còn phải biết cái cụm tag ấy nằm ở đâu so với cái lỗ cắm sạc. Và khâu ấy thường được lấy từ bản vẽ — với chính những vấn đề mà B/09 mục 5A đã liệt kê.

Bài toán hiệu chuẩn cho robot có một lịch sử riêng, tách khỏi hiệu chuẩn camera thuần. Bài toán hand-eye được đặt ra và giải trong những năm 1980–1990 với ba cách tiếp cận độc lập: tách xoay khỏi tịnh tiến [55], dùng cấu trúc nhóm Euclid [56], và giải đồng thời bằng quaternion đối ngẫu [57]. Ba nguồn độc lập cho cùng một bài toán là một mức kiểm chứng tốt, và tôi dùng nó ở mục 5.

Điều làm chương này khác một chương hiệu chuẩn thông thường — và lý do nó thuộc về cây này chứ không chỉ là một trổ sang tài liệu OpenCV — nằm ở hai chỗ.

Chỗ thứ nhất: thứ tự. Ba phép hiệu chuẩn không độc lập. Đo constellation cần intrinsic đúng (B/09 mục 5B). Hand-eye cần cả hai cái trước. Làm sai thứ tự thì sai số của bước trước bị hấp thụ vào bước sau theo cách không kiểm soát được, và — đây là phần tệ — residual vẫn nhỏ nên không có tín hiệu. Mục 2 dành cho chuyện này.

Chỗ thứ hai, và nó là chỗ đặc biệt nhất của chương: tầm quan trọng của toàn bộ chương phụ thuộc vào một quyết định ở B/13. Nếu hệ dùng teach-by-showing, thì sai số của cả ba phép hiệu chuẩn là nguồn hệ thống, và chúng triệt tiêu (B/13 mục 3B). Yêu cầu chuyển từ “hiệu chuẩn phải đúng” thành “hiệu chuẩn phải không đổi” — một yêu cầu dễ hơn nhiều và đòi những biện pháp khác hẳn. Mục 7 bàn kỹ, và đây là câu trả lời cho Q5.

TRỰC GIÁC

Hình dung chuỗi biến đổi như một chuỗi người chuyển tin.

Người thứ nhất biết cái lỗ cắm sạc nằm đâu so với cụm tag — đó là bản vẽ cơ khí của dock. Người thứ hai là camera: nó nhìn thấy cụm tag và biết cụm tag nằm đâu so với chính nó. Người thứ ba biết camera được bắt ở đâu trên thân robot. Ghép ba lời lại thì ra câu trả lời: cái lỗ cắm nằm đâu so với robot.

Nếu người nào nói sai một chút, câu trả lời cuối sai một chút — và ba cái “một chút” cộng lại chứ không triệt tiêu nhau.

Bây giờ đến chỗ hay. Người thứ hai — camera — được kiểm tra liên tục: mỗi khung hình là một lần đo mới, nhiều trung bình đi, và nếu nó sai thì có cách phát hiện. Người thứ nhất và người thứ ba thì nói *một lần* lúc lắp đặt rồi im lặng mãi mãi. Không ai kiểm lại họ, và nếu họ nói sai thì cái sai ấy nằm trong hệ thống suốt vòng đời của nó.

Đó là lý do hai khâu hiệu chuẩn đáng lo hơn khâu đo, dù khâu đo mới là khâu người ta dành thời gian cho.

Và đây là mẹo của B/13: nếu thay vì hỏi “cái lỗ ở đâu”, ta hỏi “lúc này ba người nói khác gì so với lúc robot đang cắm đúng”, thì mọi lời nói sai cố định đều xuất hiện ở cả hai lần và trừ đi nhau. Ta không cần ba người nói đúng; ta chỉ cần họ nói *giống nhau* ở hai thời điểm.

12.2 Thứ tự bắt buộc

Mục này trả lời Q1, và nó là nội dung có giá trị thực hành cao nhất của chương.

Bảng 12.1. Ba bước hiệu chuẩn: đầu vào, đầu ra, và điều gì hỏng nếu làm sớm.

	Bước	Cần trước	Hỏng thế nào nếu làm sai thứ tự
1	Intrinsic (K , méo)	Không gì	—
2	Constellation ($T_{\text{dock}}^{\text{tag}}$ và toạ độ góc)	Intrinsic đúng	K sai $\Rightarrow$ cấu trúc 3D thu được sai có hệ thống, thường là sai tỉ lệ hoặc “cong” toàn cục
3	Hand-eye ($T_{\text{cam}}^{\text{base}}$)	Cả 1 và 2	Constellation sai $\Rightarrow$ pose đo được sai $\Rightarrow X$ hấp thụ sai lệch ấy, và residual vẫn nhỏ

Cột cuối là phần đáng đọc kỹ, và nó có một điểm chung: *sai số của bước trước bị hấp thụ vào bước sau mà không làm residual tăng đáng kể*. Đây là chế độ hỏng im lặng, đúng loại nguy hiểm nhất.

Giải thích cơ chế cho bước 3, vì nó rõ nhất. Hand-eye giải X từ các cặp (A_i, B_i) , trong đó A_i là chuyển động của camera (đo bằng thị giác) và B_i là chuyển động của thân robot (đo bằng odometry hoặc encoder). Nếu constellation sai, thì A_i sai một cách có hệ thống — chẳng hạn mọi chuyển động camera được đo lớn hơn thực tế 0,5%. Bộ giải sẽ tìm một X làm phương trình khớp tốt nhất, và cái X ấy hấp thụ một phần sai lệch tỉ lệ. Kết quả là một X sai, một residual nhỏ, và không có cách nào biết.

Một hệ quả thực hành cụ thể của bảng này: *khi bạn đổi ống kính hoặc chỉnh lại nét, bạn phải làm lại cả ba bước*, không chỉ bước 1. Đổi intrinsic làm vô hiệu constellation đã đo, và constellation mới làm vô hiệu hand-eye. Đây là lý do nữa để khoá nét bằng keo (A/01 mục 8).

12.3 Hiệu chuẩn intrinsic chất lượng cao

Bước một, và nó trả lời Q2.

A. Yêu cầu về dữ liệu, không phải về thuật toán

Thuật toán hiệu chuẩn là kiến thức chuẩn và các cài đặt sẵn có đều tốt [6]. Chất lượng kết quả được quyết định gần như hoàn toàn bởi *dữ liệu*, và ba yêu cầu sau đây là ba yêu cầu mà một bộ ảnh “trông đẹp” thường vi phạm.

Yêu cầu một: phủ kín khung hình, đặc biệt là bốn góc. Các hệ số méo bậc cao chỉ có tác dụng ở bán kính lớn, nên nếu target không bao giờ chạm tới rìa thì chúng không được ràng buộc và bộ tối ưu gán cho chúng giá trị tùy tiện. Hậu quả: mô hình ngoại suy sai ở rìa — đúng chỗ mà tag của robot có thể nằm.

Yêu cầu hai: nhiều tư thế nghiêng mạnh. Đây là yêu cầu bị vi phạm nhiều nhất và nó là câu trả lời cho Q2. Với target luôn gần vuông góc, c_x và c_y không tách được khỏi vị trí của target: “điểm chính lệch sang trái” và “target dịch sang phải” tạo ra cùng một ảnh. Chỉ khi target nghiêng mạnh thì hai giả thuyết ấy cho hai ảnh khác nhau, vì biến dạng phối cảnh phụ thuộc vào vị trí so với trục quang. Đây chính là lý do A/01 mục 5 gọi điểm chính là tham số nguy hiểm nhất.

Và đây là phần thứ hai của Q2 — vì sao residual không phát hiện được: bộ tối ưu tìm bộ tham số khớp tốt nhất với *dữ liệu đã cho*. Nếu dữ liệu không chứa thông tin phân biệt hai giả thuyết, thì cả hai đều cho residual nhỏ như nhau, và bộ tối ưu chọn một trong hai theo điểm khởi đầu. Residual đo độ khớp, không đo độ đúng.

Yêu cầu ba: đủ số ảnh và đủ đa dạng. Con số thường dùng là khoảng 20–30 ảnh (*khuyến nghị thực hành phổ biến, không phải kết quả tối ưu hoá*), nhưng số lượng ít quan trọng hơn tính đa dạng: 30 ảnh giống nhau không bằng 15 ảnh khác nhau.

B. Ba phép kiểm sau khi hiệu chuẩn

Đừng dừng ở con số RMS. Ba phép kiểm, theo thứ tự giá trị.

Một, vẽ bản đồ residual theo vị trí ảnh (A/01 mục 6). Cấu trúc xuyên tâm hoặc xoáy ở góc nghĩa là mô hình sai dạng, và thêm ảnh không cứu được — phải đổi mô hình méo.

Hai, hiệu chuẩn lại với các tập con ảnh khác nhau và so các tham số. Nếu c_x nhảy hàng chục pixel giữa các tập con thì nó chưa được xác định, và con số bạn đang dùng là ngẫu nhiên. f_x thường ổn định — sự tương phản giữa hai tham số là một chuẩn đoán tốt.

Ba, kiểm bằng một target độc lập. Chụp một target chưa dùng để hiệu chuẩn, ở tư thế và khoảng cách giống điều kiện vận hành, và xem residual có tương đương không. Đây là kiểm chéo thật sự, và nó bắt được việc khớp quá mức.

C. Chọn target

Với intrinsic chất lượng cao, chọn target theo B/08 mục 3: chessboard hoặc ChArUco cho góc saddle point, hoặc deltile nếu cần cao nhất [12]. Không dùng target tròn trừ khi bạn cài hiệu chỉnh conic đúng cách (A/02 mục 6).

Một chi tiết dễ quên và tốn kém: *target phải phẳng và kích thước phải đúng*. In trên giấy dán lên bìa cứng thì không đủ — giấy cong, và máy in có sai số tỉ lệ vài phần nghìn. Với hiệu chuẩn intrinsic thì sai tỉ lệ của target không ảnh hưởng tới f tính bằng pixel (nó chỉ đổi thang của các pose), nhưng độ *không phẳng* thì ảnh hưởng trực tiếp và nó vi phạm chính giả thiết của phương pháp.

I 12.4 Đo constellation

Bước hai. Nội dung kỹ thuật đã nằm ở B/09 mục 5 và tôi không lặp lại; mục này chỉ bổ sung ba điểm thuộc về chuỗi hiệu chuẩn.

Điểm một: nó phải chạy sau intrinsic. Đã bàn ở mục 2.

Điểm hai, và đây là điểm mà B/09 chưa bàn: còn một khâu nữa sau nó. BA cho bạn toạ độ các góc tag trong một hệ toạ độ chung, nhưng hệ ấy là hệ *tùy ý* do BA chọn (thường gắn với tag đầu tiên). Bạn còn phải xác định quan hệ giữa hệ ấy và *điểm cập chuẩn thật* — tức khâu $T_{\text{dock}}^{\text{tag}}$ của công thức (12.1).

Cách đo khâu này, và nó là chỗ đẹp nhất trong chương: *dùng chính lần cập chuẩn*. Đưa robot vào vị trí cập đúng (bằng tay, bằng đồ gá, hoặc để cơ cấu tự khớp), rồi đo pose của constellation từ đó. Pose ấy *định nghĩa* điểm cập chuẩn trong hệ constellation — bạn không cần đo bằng thước gì cả.

Để ý rằng đây chính là teach-by-showing của B/13 mục 3, nhìn từ góc hiệu chuẩn. Hai chương mô tả cùng một hành động: một chương gọi nó là “ghi lại trạng thái đích”, chương này gọi nó là “hiệu chuẩn khâu cuối của chuỗi”. Sự trùng khớp ấy không ngẫu nhiên, và nó là lý do teach-by-showing triệt tiêu được sai số: nó *định nghĩa* đích bằng chính chuỗi đo, nên mọi sai lệch của chuỗi đã nằm sẵn trong định nghĩa.

Điểm ba: kiểm định kỳ. Chỉ báo ở B/09 mục 2 — so sai số tái chiếu hợp nhất với từng tag riêng — nên chạy định kỳ và so với giá trị lúc lắp đặt. Constellation thay đổi vì va chạm, vít lỏng, và giãn nở nhiệt.

12.5 Hand-eye

Bước ba, và nó trả lời Q3.

A. Bài toán

Cần tìm $X = T_{\text{cam}}^{\text{base}}$. Quy trình: di chuyển robot tới nhiều tư thế; ở mỗi tư thế, đo pose của constellation bằng thị giác và đọc pose của thân robot bằng odometry. Với hai tư thế i và j , gọi A là chuyển động của camera (suy từ hai lần đo thị giác) và B là chuyển động của thân robot (suy từ odometry). Quan hệ là

$$AX = XB,$$

và đây là bài toán hand-eye kinh điển. Ba cách giải độc lập, ba nhóm khác nhau: Tsai và Lenz tách xoay khỏi tịnh tiến rồi giải tuần tự [55]; Park và Martin dùng cấu trúc nhóm Euclid [56]; Daniilidis giải đồng thời cả hai bằng quaternion đối ngẫu [57]. Việc ba nhóm độc lập cho cùng kết quả là một mức kiểm chứng tốt cho tính đúng của công thức hoá.

Sau lời giải dạng đóng, nên tinh chỉnh phi tuyến — cùng lý do như ở A/03 mục 4: lời giải dạng đóng cực tiểu một sai số đại số, không phải sai số có ý nghĩa. Có các công thức hoá dùng trọng số theo bất định [58].

B. Điều kiện kích thích, và vì sao robot vi sai vi phạm

Đây là phần quan trọng nhất và nó trả lời Q3.

Bài toán $AX = XB$ không xác định X duy nhất từ một cặp chuyển động. Cụ thể, *phần tịnh tiến của X chỉ xác định được nếu tập chuyển động chứa các phép xoay quanh ít nhất hai trục độc lập* [55, §IV].

Trục giác của điều kiện này: phần tịnh tiến của X — tức camera cách tâm robot bao xa — chỉ lộ ra qua hiệu ứng đòn bẩy khi robot *xoay*. Nếu robot chỉ tịnh tiến, camera và thân di chuyển giống hệt nhau và không có thông tin nào về khoảng cách giữa chúng. Nếu robot chỉ xoay quanh *một* trục, thì thành phần của vectơ tịnh tiến dọc theo trục ấy không gây hiệu ứng gì — nó nằm trong nhân của phép quay — nên nó không xác định được.

Và đây là chỗ robot vi sai đặc biệt dễ vi phạm: *nó chỉ xoay được quanh một trục*, trục thẳng đứng. Một robot vi sai trên sàn phẳng không bao giờ tạo ra phép xoay quanh trục ngang, nên thành phần *độ cao* của vectơ tịnh tiến trong X là không quan sát được.

Ba cách xử lý.

Một, đo độ cao bằng thước. Nghe thô nhưng nó đúng: đây là một tham số mà thị giác không đo được, và đo cơ khí là cách hợp lệ. Cố định nó rồi giải phần còn lại.

Hai, nghiêng robot có chủ ý. Nếu có cách cho robot lên một cái dốc nhẹ trong quá trình hiệu chuẩn, bạn tạo ra xoay quanh trục ngang và điều kiện được thỏa. Không phải lúc nào cũng làm được.

Ba, nhận ra rằng bạn có thể không cần nó. Nếu bài toán là docking trên sàn phẳng và robot luôn ở cùng độ cao, thì thành phần độ cao của X *không ảnh hưởng* tới kết quả — nó là một hằng số cộng vào cả pose hiện tại lẫn pose đích và nó triệt tiêu. Đây lại là lập luận teach-by-showing xuất hiện dưới một hình thức khác, và nó là lý do thực dụng nhất: *không quan sát được và không cần thiết thường đi với nhau*, vì cả hai đều xuất phát từ cùng một sự bất biến của bài toán.

CẦN NHỚ

Điều kiện kích thích của hand-eye là một trường hợp cụ thể của một nguyên lý tổng quát: *một tham số chỉ ước lượng được nếu dữ liệu chứa thông tin về nó*, và “thêm tham số vào mô hình” không tạo ra thông tin.

Triệu chứng khi vi phạm: bộ giải vẫn trả về một con số, residual vẫn nhỏ, nhưng con số ấy đổi mạnh khi bạn chạy lại với dữ liệu khác — vì nó đang được quyết định bởi nhiễu và bởi điểm khởi đầu chứ không bởi dữ liệu.

Phép thử: chạy lại hiệu chuẩn với các tập con dữ liệu khác nhau và xem tham số nào ổn định, tham số nào nhảy. Đây là cùng phép thử đã dùng cho c_x ở mục 3B, và nó là phép thử chẩn đoán có giá trị nhất trong cả chương.

Và một an ủi thực dụng: các tham số không quan sát được thường cũng là các tham số không ảnh hưởng tới kết quả, vì cả hai tính chất đều bắt nguồn từ cùng một sự bất biến của bài toán.

12.6 Trôi theo thời gian

Hiệu chuẩn không phải một trạng thái vĩnh viễn, và mục này liệt kê những gì đổi.

Nhiệt độ. Ống kính và thân máy giãn nở; tiêu cự đổi và điểm chính dịch. Với ống kính nhựa và chênh lệch vài chục độ, hiệu ứng có thể ở mức pixel. Với hệ đặt trong nhà và nhiệt độ ổn định thì ít quan trọng; với hệ ngoài trời hoặc gần nguồn nhiệt thì đáng đo.

Rung và va chạm. Vít lỏng, giá đỡ camera lệch, tấm marker bị va. Đây là nguồn đổi đột ngột chứ không trôi dần, nên nó cần một cơ chế phát hiện chứ không phải một mô hình.

Mài mòn và bẩn. Tag bị xước, bị bám bụi, bị phai màu. Nó không đổi hình học nhưng nó đổi σ và có thể gây thiên lệch cục bộ.

Ba biện pháp, theo thứ tự chi phí.

Một, chỉ báo liên tục. Sai số tái chiếu của PnP hợp nhất, và tỉ số giữa nó với sai số khi giải từng tag riêng (**B/09** mục 2). Ghi vào log và theo dõi xu hướng. Đây là cách phát hiện rẻ nhất và nó bắt được cả ba loại đổi.

Hai, hiệu chuẩn lại định kỳ. Với intrinsic thì tốn công; với constellation và hand-eye thì nhanh hơn.

Ba, tự hiệu chuẩn trực tuyến. Đưa một số tham số vào trạng thái của bộ ước lượng và để chúng được cập nhật cùng pose. Nghe hấp dẫn và nó hoạt động trong một số hệ, nhưng nó mang theo nguyên tắc ở mục 5B: *mở rộng trạng thái không tạo ra thông tin*. Nếu chuyển động không kích thích một tham số thì đưa nó vào trạng thái chỉ làm nó trôi tự do và hấp thụ sai số từ chỗ khác — làm residual nhỏ đi trong khi pose tệ đi. Với docking, nơi chuyển động rất hạn chế (thẳng trục, một trục xoay), tôi khuyên *không* tự hiệu chuẩn trực tuyến trừ khi bạn đã phân tích kích thích cẩn thận.

12.7 Nếu dùng teach-by-showing thì khác gì

Mục này trả lời Q5, và nó là mục làm chương này khác mọi chương khác trong cây.

B/13 mục 3 chỉ ra rằng nếu điều khiển theo sai lệch tương đối so với một trạng thái đích đã ghi, thì mọi sai số *hệ thống* triệt tiêu. Sai số của cả ba phép hiệu chuẩn trong chương này đều thuộc loại đó. Hệ quả: yêu cầu đối bản chất.

Không còn cần: độ đúng tuyệt đối. Một K sai 2%, một constellation sai 1 mm, một X sai vài milimét — tất cả xuất hiện ở cả hai vế và trừ đi nhau. Bạn không cần biết chúng đúng.

Vẫn cần, và cần hơn: độ ổn định. Đây là câu trả lời cho Q5. Triệt tiêu chỉ hoạt động khi hàm sai lệch *không đổi* giữa lần dạy và lần chạy. Mọi thứ ở mục 6 — trôi nhiệt, va chạm, mài mòn — phá đúng điều kiện ấy. Nên trong chế độ teach-by-showing, chương này vẫn quan trọng nhưng nó chuyển trọng tâm: từ “hiệu chuẩn cho đúng” sang “giữ cho không đổi, và phát hiện khi nó đổi”.

Ba việc thay đổi cụ thể.

Việc bỏ bớt: yêu cầu về độ phủ dữ liệu và tư thế nghiêng ở mục 3A nói lỏng. Bạn vẫn cần một K hợp lý — vì méo quá lớn làm cạnh cong và phá detector (**A/01** mục 4), và vì hàm sai lệch phải đủ trơn để triệt tiêu bậc nhất hoạt động — nhưng bạn không cần c_x chính xác tới pixel.

Việc giữ nguyên: phép thử ổn định ở mục 3B và mục 5B trở thành phép thử chính chứ không phải phép kiểm phụ.

Việc thêm vào: một quy trình *dạy lại* định kỳ, và một cơ chế phát hiện khi cần dạy lại. Dạy lại rẻ hơn hiệu chuẩn lại rất nhiều — nó chỉ là đưa robot vào vị trí cập chuẩn một lần và ghi lại — nên tần suất có thể cao.

Và một cảnh báo: triệt tiêu chỉ ở *bậc nhất* và chỉ trong lân cận tư thế dạy (**B/13** mục 3C). Nếu robot phải làm việc trên một dải tư thế rộng, phần dư bậc hai không triệt tiêu, và khi đó độ đúng của hiệu chuẩn lại quan trọng trở lại. Cách xử lý: dùng coarse-to-fine (**B/13** mục 5) để đảm bảo pha cuối luôn ở gần tư thế dạy.

12.8 Giới hạn và chế độ hỏng

Residual nhỏ không chứng minh tham số đúng. Đây là chủ đề xuyên suốt chương và nó đáng lặp lại: residual đo độ khớp với dữ liệu đã cho, không đo độ đúng. Với dữ liệu nghèo, một mô hình sai vẫn cho residual nhỏ.

Sai số bước trước bị hấp thụ vào bước sau im lặng. Mục 2. Không có chỉ báo tự nhiên nào bắt được, nên thứ tự phải được tuân thủ bằng kỷ luật.

Điều kiện kích thích không thể lách bằng thuật toán. Mục 5B. Nếu chuyển động không chứa thông tin về một tham số, không bộ giải nào tìm ra nó.

Odometry là một nguồn sai số trong hand-eye. Mục 5 giả định B — chuyển động của thân robot — được đo chính xác. Với odometry bánh xe, nó không: trượt bánh, sai bán kính bánh, sai khoảng cách trục. Sai số ấy đi thẳng vào X . Với đoạn hiệu chuẩn ngắn thì chấp nhận được; với đoạn dài thì cần một nguồn tốt hơn.

Các con số khuyến nghị chưa được kiểm. 20–30 ảnh, “ngiênêng mạnh” là bao nhiêu độ — đây là thực hành phổ biến chứ không phải kết quả tối ưu hoá.

12.9 Thực hành

Năm việc.

Một, quyết định chế độ trước: teach-by-showing hay accuracy tuyệt đối. Nó quyết định bạn cần chương này ở mức nào.

Hai, tuân thủ thứ tự intrinsic → constellation → hand-eye, và làm lại cả chuỗi khi đổi bất kỳ thành phần quang học nào.

Ba, với intrinsic, đầu tư vào dữ liệu chứ không vào thuật toán: phủ tới bốn góc, nhiều tư thế nghiêng, và ba phép kiểm ở mục 3B.

Bốn, đo khâu T_{dock}^{tag} bằng chính lần cập chuẩn thay vì bằng thước. Nó chính xác hơn và nó miễn phí.

Năm, bật chỉ báo liên tục ở mục 6 và theo dõi xu hướng của nó theo tuần, không chỉ theo khung.

Active recall (che cột phải, tự trả lời, rồi mở ra đối chiếu)

Thuật ngữ / câu hỏi	Trả lời
●●● Pose thực sự cần	$T_{dock}^{base} = T_{cam}^{base} \cdot T_{tag}^{cam} \cdot T_{dock}^{tag}$. Ba khâu, chỉ khâu giữa là thứ thị giác đo. Hai khâu kia là hằng số phải hiệu chuẩn, và sai số của chúng cộng thẳng.
●●● Khâu thường bị bỏ quên	T_{dock}^{tag} — quan hệ giữa hệ constellation và <i>điểm cập chuẩn thật</i> . Người ta đo kỹ vị trí các tag với nhau rồi quên rằng còn phải biết cụm ấy ở đâu so với cái lỗ cắm.
●●● Thứ tự bắt buộc	Intrinsic → constellation → hand-eye. Mỗi bước dùng kết quả bước trước làm hằng số. Đảo thứ tự ⇒ sai số bị hấp thụ vào bước sau mà <i>residual</i> vẫn nhỏ.
●●● Hệ quả của thứ tự	Đổi ống kính hoặc chỉnh nét ⇒ phải làm lại <i>cả ba</i> bước, không chỉ bước 1. Lý do nữa để khoá nét bằng keo.
●●● Vì sao c_x, c_y khó ước lượng	Với target gần vuông góc, “điểm chính lệch trái” và “target dịch phải” cho <i>cùng một ảnh</i> . Chỉ tư thế nghiêng mạnh mới tách được hai giả thuyết. f_x thì ổn định — sự tương phản ấy là một chẩn đoán tốt.
●●● Vì sao residual không phát hiện được	Bộ tối ưu tìm bộ tham số khớp tốt nhất với <i>dữ liệu đã cho</i> . Dữ liệu không chứa thông tin phân biệt hai giả thuyết ⇒ cả hai cho residual nhỏ như nhau. <i>Residual đo độ khớp, không đo độ đúng</i> .
●●● Ba phép kiểm sau hiệu chuẩn	Bản đồ residual theo vị trí ảnh (cấu trúc ⇒ mô hình sai dạng); hiệu chuẩn lại với các tập con và xem tham số nào nhảy; kiểm bằng target độc lập chưa dùng để hiệu chuẩn.

Thuật ngữ / câu hỏi	Trả lời (tiếp)
•• Hand-eye $AX = XB$	A là chuyển động camera (đo bằng thị giác), B là chuyển động thân robot (odometry), X là ẩn. Ba cách giải độc lập: tách xoay/tịnh tiến [55], nhóm Euclid [56], quaternion đối ngẫu [57].
••• Điều kiện kích thích của hand-eye	Phần tịnh tiến của X chỉ xác định được nếu có xoay quanh ít nhất hai trục độc lập [55, §IV]. Thành phần dọc theo trục xoay nằm trong nhân nên không quan sát được.
••• Vì sao robot vi sai vi phạm	Nó chỉ xoay quanh một trục (thẳng đứng), nên thành phần độ cao của vectơ tịnh tiến trong X không quan sát được. Ba cách: đo bằng thước, nghiêng robot có chủ ý, hoặc nhận ra rằng không cần.
••• An ủi thực dụng	Tham số không quan sát được thường cũng là tham số không ảnh hưởng kết quả — cả hai tính chất bắt nguồn từ cùng một sự bất biến của bài toán.
••• Đo $T_{\text{dock}}^{\text{tag}}$ thế nào	Bằng <i>chính lần cập chuẩn</i> : đưa robot vào vị trí cập đúng rồi đo pose constellation từ đó. Pose ấy <i>định nghĩa</i> điểm cập chuẩn — không cần thước. Đây chính là teach-by-showing nhìn từ góc hiệu chuẩn.
••• Teach-by-showing đổi gì	Bỏ yêu cầu về <i>độ đúng</i> (sai số hệ thống triệt tiêu); giữ và tăng yêu cầu về <i>độ ổn định</i> (triệt tiêu chỉ hoạt động khi hàm sai lệch không đổi giữa dạy và chạy). Thêm: quy trình dạy lại định kỳ, rẻ hơn hiệu chuẩn lại nhiều.
•• Cảnh báo về tự hiệu chuẩn trực tuyến	Mở rộng trạng thái <i>không tạo ra thông tin</i> . Chuyển động docking rất hạn chế (thẳng trục, một trục xoay), nên tham số không kích thích sẽ trôi tự do và hấp thụ sai số — residual nhỏ đi trong khi pose tệ đi.
•• Odometry là nguồn sai số trong hand-eye	Mục 5 giả định B đo chính xác. Trượt bánh, sai bán kính, sai khoảng cách trục đi thẳng vào X . Đoạn hiệu chuẩn ngắn thì chấp nhận được.

••• phải thuộc •• nên nắm • biết là có

I Kiến thức này đang sống ở đâu (công trình đang dùng chính ý tưởng của chương; chốt 9/2026)

Công trình / hệ thống	Dùng ý tưởng nào của chương	Ghi chú
OpenCV <code>calibrateCamera</code> , <code>calibrateHandEye</code>	Cài Zhang và các phương pháp hand-eye kinh điển	<code>calibrateHandEye</code> có nhiều phương pháp chọn được — thử vài cái và so kết quả
Kalibr [11]	Hiệu chuẩn nhiều cảm biến, báo cáo residual theo vị trí ảnh	Công cụ duy nhất mặc định cho bạn bản đồ residual ở mục 3B
easy_handeye	Wrapper ROS cho hand-eye với quy trình thu thập dữ liệu	Tiện, nhưng vẫn phải tự lo điều kiện kích thích ở mục 5B
COLMAP [51]	Photogrammetry để đo constellation nếu không muốn tự viết BA	Đường tắt cho bước 2; nhớ neo tỉ lệ
deltille [12]	Target lưới tam giác cho intrinsic chất lượng cao nhất	Dùng khi chế độ accuracy tuyệt đối và ngân sách chặt

MINI PROJECT • Kiểm tính ổn định của ba phép hiệu chuẩn, và tìm tham số không quan sát được • 2–3 ngày

Mục tiêu phân biệt các tham số đã được xác định khỏi các tham số trông như đã được xác định, bằng một phép thử không cần chân trị nào.

Dữ liệu dữ liệu hiệu chuẩn thật của chính hệ bạn: một bộ ảnh target cho intrinsic, một bộ ảnh constellation cho BA, một bộ cặp pose cho hand-eye.

Các bước	với mỗi trong ba phép hiệu chuẩn, chạy nó $M = 20$ lần trên M tập con ngẫu nhiên của dữ liệu (mỗi tập con chứa khoảng 70% dữ liệu); ghi lại mọi tham số của mỗi lần; tính độ lệch chuẩn của từng tham số qua M lần; xếp các tham số theo độ lệch chuẩn tương đối. Phần hai: với intrinsic, lặp lại nhưng chia dữ liệu theo <i>tư thế</i> — một tập chỉ gồm ảnh gần vuông góc, một tập gồm cả ảnh nghiêng mạnh — và so c_x, c_y giữa hai tập.
Xong khi	bạn có một bảng xếp hạng các tham số theo mức độ được xác định. Dự đoán của chương: f_x, f_y rất ổn định; c_x, c_y kém ổn định hơn nhiều và <i>đặc biệt</i> kém trong tập chỉ có ảnh vuông góc; và trong hand-eye, thành phần độ cao của X sẽ nhảy nhiều nhất nếu robot của bạn chỉ xoay quanh trục đứng. Kiểm tra chéo bắt buộc: nếu <i>mọi</i> tham số đều ổn định thì hoặc dữ liệu của bạn rất tốt (đáng mừng), hoặc các tập con của bạn quá giống nhau — hãy kiểm bằng cách thử một tập con chỉ 30% dữ liệu.
Nối với	C/15 nhận độ lệch chuẩn này làm ước lượng bất định cho các dòng hiệu chuẩn trong ngân sách; B/13 dùng nó để quyết định có cần chế độ accuracy tuyệt đối không.

TRẢ LỜI CÂU HỎI MỞ ĐẦU

- A1. Thứ tự: intrinsic $\rightarrow$ constellation $\rightarrow$ hand-eye, vì mỗi bước dùng kết quả bước trước làm hằng số đã biết. Nếu đảo hai bước cuối — làm hand-eye trước khi đo constellation — thì bài toán $AX = XB$ nhận các chuyển động camera A được đo dựa trên một mô hình constellation sai, nên A sai một cách có hệ thống (chẳng hạn mọi chuyển động được đo lớn hơn thực tế một tỉ lệ cố định). Bộ giải sẽ tìm một X làm phương trình khớp tốt nhất, và cái X ấy *hấp thụ* một phần sai lệch. Kết quả là một X sai, một residual nhỏ, và không có chỉ báo nào — đây là chế độ hỏng im lặng, loại nguy hiểm nhất. Hệ quả thực hành đi kèm: khi đổi ống kính hoặc chỉnh lại nét, phải làm lại cả ba bước chứ không chỉ bước một.
- A2. Hai tham số gần như chắc chắn sai là c_x và c_y — điểm chính. Lý do: với target luôn gần vuông góc, giả thuyết “điểm chính lệch sang trái” và giả thuyết “target dịch sang phải” tạo ra *cùng một ảnh*, nên dữ liệu không tách được chúng. Chỉ khi target nghiêng mạnh thì hai giả thuyết mới cho hai ảnh khác nhau, vì biến dạng phối cảnh phụ thuộc vào vị trí so với trục quang. Residual không phát hiện được vì bộ tối ưu làm đúng việc nó được giao: tìm bộ tham số khớp tốt nhất với dữ liệu đã cho. Nếu dữ liệu không chứa thông tin phân biệt hai giả thuyết thì cả hai đều cho residual nhỏ như nhau, và bộ tối ưu chọn một trong hai theo điểm khởi đầu. Nguyên tắc chung: residual đo độ *khớp*, không đo độ *đúng*. Phép thử đúng là hiệu chuẩn lại với các tập con dữ liệu khác nhau và xem tham số nào nhảy.
- A3. Điều kiện: tập chuyển động phải chứa các phép xoay quanh *ít nhất hai trục độc lập* [55, §IV]. Trục giắc: phần tịnh tiến của X — camera cách tâm robot bao xa — chỉ lộ ra qua hiệu ứng đòn bẩy khi robot xoay; tịnh tiến thuần thì camera và thân di chuyển giống hệt nhau và không mang thông tin nào về khoảng cách giữa chúng. Và nếu chỉ xoay quanh một trục, thì thành phần của vectơ tịnh tiến *dọc theo* trục ấy nằm trong nhân của phép quay nên không gây hiệu ứng gì. Robot vi sai trên sàn phẳng đặc biệt dễ vi phạm vì nó chỉ xoay được quanh *một* trục — trục thẳng đứng — nên thành phần độ cao của X không quan sát được. Ba cách xử lý: đo độ cao bằng thước rồi cố định nó; cho robot lên dốc nhẹ để tạo xoay quanh trục ngang; hoặc nhận ra rằng với docking trên sàn phẳng, độ cao là một hằng số cộng vào cả pose hiện tại lẫn pose đích nên nó triệt tiêu và bạn không cần nó.
- A4. Pose cần là $T_{\text{dock}}^{\text{base}}$ — điểm cập chuẩn trên dock so với thân robot — và chuỗi gồm ba khâu: $T_{\text{cam}}^{\text{base}} \cdot T_{\text{tag}}^{\text{cam}} \cdot T_{\text{dock}}^{\text{tag}}$. Chỉ khâu giữa là thứ thị giác đo mỗi khung; hai khâu ngoài là hằng số phải hiệu chuẩn một lần và sai số của chúng cộng thẳng vào kết quả. Khâu thường bị bỏ quên là khâu cuối, $T_{\text{dock}}^{\text{tag}}$: người ta đo rất cẩn thận vị trí các tag với nhau bằng bundle adjustment rồi quên rằng còn phải biết cả cụm tag ấy nằm ở đâu so với cái lỗ cắm sạc — và khâu ấy thường được lấy từ bản vẽ CAD với đúng những vấn đề mà [B/09](#) mục 5 đã cảnh báo. Cách đo đúng và rẻ nhất: đưa robot vào vị trí cập chuẩn rồi đo pose constellation từ đó; pose ấy *định nghĩa* điểm cập chuẩn trong hệ constellation, không cần thước.
- A5. Yêu cầu về *độ ổn định* — và nó thậm chí quan trọng hơn trong chế độ này. Teach-by-showing triệt tiêu sai số hệ thống vì cùng một hàm sai lệch tác động lên cả pose hiện tại lẫn pose đích; nhưng triệt tiêu chỉ hoạt động khi hàm ấy *không đổi* giữa lần dạy và lần chạy. Mọi thứ ở mục

6 — trôi nhiệt của ống kính, va chạm làm lệch giá camera, vít lỏng, tag mài mòn — phá đúngn điều kiện ấy, và khi đó bạn mất toàn bộ lợi ích mà không có tín hiệu nào. Nên trọng tâm chuyển từ “hiệu chuẩn cho đúngn” sang “giữ cho không đổi, và phát hiện khi nó đổi”. Cụ thể: các phép thử ổn định ở mục 3B và 5B trở thành phép thử *chính* chứ không phải phép kiểm phụ; chỉ báo liên tục ở mục 6 phải được bật và theo dõi theo tuần; và cần một quy trình *dạy lại* định kỳ — may là dạy lại rẻ hơn hiệu chuẩn lại rất nhiều nên tần suất có thể cao.

NÓI LẠI THẬT ĐƠN GIẢN

Cái mà robot thật sự cần biết không phải “tầm tag ở đâu so với camera” mà là “cái lỗ cắm sọc ở đâu so với thân tôi”. Giữa hai câu ấy có một chuỗi ba khâu, và camera chỉ lo được một khâu ở giữa. Khâu thứ nhất: cái lỗ cắm nằm đâu so với cụm tag. Khâu thứ ba: camera được bắt ở đâu trên thân robot. Hai khâu này được xác định *một lần* lúc lắp đặt rồi không ai kiểm lại nữa. Nếu chúng sai thì cái sai ấy nằm im trong hệ thống suốt đời, và nó không bao giờ lộ ra ở con số nào.

Chuyện thứ tự thì đơn giản mà hay bị làm sai: phải đo camera trước, rồi đo cụm tag, rồi mới đo chỗ bắt camera. Lý do là mỗi bước sau dựa vào kết quả bước trước. Làm ngược thì sai số của bước sau sẽ “nuốt” sai số của bước trước — mọi con số trông vẫn hợp lý, chỉ có điều tất cả đều sai.

Có một cái bẫy đáng nói riêng. Khi hiệu chuẩn camera, người ta hay chụp mấy chục tấm ảnh tấm bảng cờ vua, và ai cũng muốn chụp cho *đẹp*: bảng nằm giữa khung hình, nhìn thẳng, sáng đều. Đó chính là bộ ảnh tệ nhất. Vì với bảng nhìn thẳng thì máy không phân biệt được “ống kính hơi lệch tâm” với “bảng hơi dịch sang bên” — hai chuyện khác nhau nhưng cho ra cùng một tấm ảnh. Phải chụp bảng nghiêng nhiều, phải cho nó chạm vào bốn góc khung hình, thì máy mới tách được hai chuyện đó. Bộ ảnh *xấu xí* mới là bộ ảnh tốt.

Và cái con số “sai lệch trung bình” mà phần mềm in ra cuối cùng thì đừng tin. Nó chỉ nói mô hình khớp với đúngn mấy tấm ảnh bạn đưa vào, không nói mô hình có đúngn hay không. Cách kiểm thật: chạy hiệu chuẩn nhiều lần với các nhóm ảnh khác nhau, rồi xem con số nào giữ nguyên và con số nào nhảy. Cái nào nhảy thì cái đó chưa được xác định, dù phần mềm có in nó ra với năm chữ số thập phân.

Cuối cùng, tin tốt. Nếu bạn dùng mẹo ở chương sau — chụp một tấm ảnh mẫu lúc robot đang cầm đúngn, rồi về sau chỉ cần lái sao cho ảnh trùng với ảnh mẫu — thì phần lớn chương này bớt quan trọng hẳn. Mọi sai lệch cố định đều xuất hiện ở cả hai lần và trừ đi nhau. Bạn không cần mấy con số *đúngn*; bạn chỉ cần chúng *đừng thay đổi*. Và “đừng thay đổi” thì dễ hơn “đúngn” nhiều — chỉ cần dán keo vào vòng lấy nét, siết chặt mấy con vít, và thỉnh thoảng chụp lại ảnh mẫu.

Chỗ không rút gọn được: việc phân biệt một tham số “đã được xác định” với một tham số “trông như đã được xác định” thì không có cách nào nhìn từ kết quả một lần chạy. Phải chạy nhiều lần với dữ liệu khác nhau rồi so — và đó là việc tốn một buổi nhưng đáng, vì nó cho biết bạn đang tin vào cái gì.

Một câu: Bộ ảnh hiệu chuẩn *đẹp* là bộ ảnh *tệ*; và nếu dùng mẹo chụp ảnh mẫu thì bạn cần các con số *đừng đổi* hơn là cần chúng *đúngn*.

12.10 Điều gì chứng minh cách hiểu này sai

Mệnh đề chính — *thứ tự intrinsic* → *constellation* → *hand-eye* là *bắt buộc*, và *đảo thứ tự gây sai số im lặng* — bị bác bỏ nếu mô phỏng cho thấy *hand-eye* giải với *constellation* sai vẫn cho *X* đúngn trong phạm vi sai số. Tôi dẫn nó bằng lập luận về sự hấp thụ (cửa a) và nó chưa qua cửa (b). Đây là mệnh đề có hậu quả quy trình lớn nhất trong chương.

Mệnh đề thứ hai — c_x, c_y *không tách được khi target luôn gần vuông góc* — là mệnh đề dễ kiểm nhất, và phần hai của mini project chính là phép kiểm. Nó được ủng hộ bởi cấu trúc của bài toán (A/01 mục 5) và bởi thực hành phổ biến trong cộng đồng hiệu chuẩn, nhưng tôi chưa tự đo.

Mệnh đề thứ ba — *robot vi sai không quan sát được thành phần độ cao của X* — bắt nguồn trực tiếp từ điều kiện kích thích đã công bố [55], nên cửa (c) đã qua cho phần lý thuyết. Phần áp dụng

cho robot vi sai là suy luận của tôi, và nó bị bác bỏ nếu hand-eye trên một robot vi sai cho thành phần độ cao ổn định qua nhiều tập con dữ liệu.

Cuối cùng, mệnh đề mềm: *tham số không quan sát được thường cũng không ảnh hưởng kết quả*. Đây là một quan sát về cấu trúc chứ không phải một định luật, và nó có ngoại lệ — một tham số có thể không quan sát được trong dữ liệu hiệu chuẩn nhưng lại ảnh hưởng trong điều kiện vận hành khác. Cách kiểm: mô phỏng ảnh hưởng của nó lên pose ở dải tư thế vận hành thật.

12.11 Kết nối

Chương này nối chuỗi đo với thế giới thật, và nó có một quan hệ đặc biệt với B/13.

Nối lên trên. A/01-mo-hình-camera-va-distortion cấp toàn bộ nội dung của mục 3: mô hình méo, vì sao điểm chính khó, và phép thử residual có cấu trúc. Hai chương gắn chặt: chương 1 giải thích cơ chế, chương này biến nó thành quy trình. A/05-lan-truyen-sai-so-pixel-sang-pose mục 8 cấp phân loại làm cho mục 7 có nghĩa.

Nối ngang. B/09-multi-marker-constellation mục 5 là bước 2 của chuỗi; chương này bổ sung hai điều mà chương ấy chưa bàn: vị trí của nó trong thứ tự, và khâu $T_{\text{dock}}^{\text{tag}}$ còn lại sau BA. B/13-docking-control là chương quyết định tầm quan trọng của cả chương này — một quan hệ hiểm và đáng chú ý. Mục 7 mô tả nó, và kết luận thực hành là: *đọc B/13 trước khi quyết định đầu tư bao nhiêu vào chương này*.

Nối xuống. C/15-ngan-sach-sai-so nhận các dòng “sai số intrinsic”, “sai số constellation”, “sai số hand-eye” — và mini project của chương này là cách định giá chúng bằng số. Đáng chú ý là cả ba dòng ấy thuộc loại *hệ thống*, nên chúng cộng thẳng và không chia cho $\sqrt{N}$ — trong chế độ accuracy tuyệt đối, chúng thường chiếm phần lớn ngân sách. C/16-danh-gia-va-nghiem-thu nhận phép thử ổn định ở mục 3B và 5B làm một mục trong quy trình nghiệm thu định kỳ.

Câu hỏi tự kiểm (trả lời không nhìn bài)

- Viết chuỗi biến đổi đầy đủ từ điểm cập chuẩn tới thân robot, và chỉ ra khâu nào đo mỗi khung, khâu nào hiệu chuẩn một lần.
- Vì sao đảo thứ tự constellation và hand-eye gây sai số im lặng? Mô tả cơ chế hấp thụ.
- Bộ ảnh hiệu chuẩn “đẹp” là bộ ảnh tẻ. Giải thích, và nêu chính xác tham số nào bị ảnh hưởng.
- Nêu phép thử phân biệt tham số “đã được xác định” với tham số “trông như đã được xác định”, và giải thích vì sao residual không làm được việc đó.
- Điều kiện kích thích của hand-eye là gì, và vì sao robot vi sai trên sàn phẳng vi phạm nó? Nêu ba cách xử lý.
- Bạn chuyển sang teach-by-showing. Yêu cầu nào của chương này biến mất, yêu cầu nào ở lại và tăng lên?
- Vì sao tự hiệu chuẩn trực tuyến nguy hiểm trong bài toán docking, và nguyên lý nào ở mục 5B giải thích điều đó?

Docking control (Docking Control)

PHỤ THUỘC A/05-lan-truyen-sai-so-pixel-sang-pose (mục 8, phân loại nguồn ngẫu nhiên và hệ thống — toàn bộ lập luận teach-by-showing dựa vào đó); A/06-hieu-ung-vat-ly-va-quang-hoc (mục 7, bốn hiệu ứng biến mất khi vận tốc bằng không — cơ sở vật lý của stop-and-go); B/09-multi-marker-constellation (constellation là thứ mà bộ điều khiển nhìn vào).

TÓM TẮT

Chương này chứa mẹo lớn nhất của cả cây, và mẹo ấy không phải một thuật toán mà là một cách đặt lại bài toán. Quan sát then chốt: robot cập dock *không cần biết nó ở đâu trong hệ tọa độ tuyệt đối*; nó cần *trở về đúng chỗ nó đã từng ở*. Hai yêu cầu ấy khác nhau về bản chất — một là độ chính xác (accuracy), một là độ lặp lại (repeatability) — và yêu cầu thứ hai dễ hơn một bậc, vì mọi sai số *hệ thống* xuất hiện ở cả pose hiện tại lẫn pose mục tiêu và triệt tiêu lẫn nhau. Đó là teach-by-showing, nội dung của mục 3. Bốn công cụ còn lại: IBVS điều khiển thẳng trên tọa độ ảnh để tránh hẳn bài toán pose; stop-and-go xoá các hiệu ứng phụ thuộc vận tốc; coarse-to-fine chia hành trình theo độ chính xác cần thiết; và côn dẫn hướng cơ khí — thứ mà một cây tri thức về thị giác phải trung thực thừa nhận là thường rẻ nhất. Nếu chỉ nhớ một điều: *đừng giải bài toán accuracy nếu bài toán thật là repeatability*.

KIẾN THỨC CẦN CÓ TRƯỚC

- **độ chính xác (accuracy)** — mức độ gần của giá trị đo tới giá trị thật. Với robot, đó là khả năng tới được một tọa độ được chỉ định trong một hệ quy chiếu tuyệt đối [3].
- **độ lặp lại (repeatability)** — mức độ gần giữa các lần thực hiện cùng một lệnh. Robot công nghiệp thường có độ lặp lại tốt hơn độ chính xác cả bậc độ lớn, và tiêu chuẩn phân biệt rõ hai đại lượng [3].
- **teach-by-showing** — ghi lại trạng thái cảm biến tại vị trí đích một lần (bằng cách đưa robot tới đó bằng tay hoặc bằng đồ gá), rồi về sau điều khiển để tái tạo đúng trạng thái ấy, thay vì điều khiển tới một tọa độ tính toán.
- **IBVS** — *image-based visual servoing*: điều khiển trực tiếp trên các đặc trưng ảnh (tọa độ điểm trên ảnh), với sai lệch định nghĩa trong không gian ảnh chứ không trong không gian pose [59].
- **PBVS** — *position-based visual servoing*: ước lượng pose từ ảnh rồi điều khiển trong không gian pose. Trực quan hơn nhưng nhạy hơn với sai số hiệu chuẩn vì pose là một đại lượng *suy ra*, không phải đại lượng đo.
- **ma trận tương tác (interaction matrix)** — ma trận L nối vận tốc của camera với tốc độ thay đổi của đặc trưng ảnh: $\dot{s} = L v$. Nó là Jacobian của bài toán điều khiển, và cấu trúc của nó quyết định hành vi của IBVS [59, §II].
- **nonholonomic** — ràng buộc trên *vận tốc* mà không rút gọn thành ràng buộc trên vị trí. Robot vi sai hai bánh là ví dụ: nó không đi ngang được tại chỗ, dù nó tới được mọi vị trí.
- **chamfer / côn dẫn hướng** — mặt vát ở miệng lỗ hoặc đầu chốt, biến một sai lệch vị trí thành một lực dẫn hướng thay vì một va chạm. Lý thuyết của nó thuộc ngành lắp ráp tự động [4].

CÂU HỎI MỞ ĐẦU

- Q1. Hệ của bạn có sai số hệ thống 3 mm do intrinsic chưa hoàn hảo và hand-eye chưa chuẩn. Có một cách làm cho 3 mm ấy gần như biến mất mà không cần hiệu chuẩn lại gì cả. Cách ấy là gì, và nó “gần như” chứ không phải “hoàn toàn” vì sao?
- Q2. IBVS điều khiển trên tọa độ điểm ảnh và không bao giờ tính pose. Nếu hai vấn đề lớn của cả cây mà nó tránh được hoàn toàn nhờ việc không tính pose, và nêu cái giá phải trả.
- Q3. Robot vi sai không đi ngang được. Nếu nó đang ở đúng trước dock nhưng lệch sang phải 2 cm, vì sao việc sửa 2 cm ấy khó hơn nhiều so với sửa 2 cm theo chiều tiến lùi — và định lý nào nói rằng không có luật điều khiển trơn nào giải quyết được?
- Q4. Dừng hẳn robot lại 0,3 giây trước khi đo có hai lợi ích hoàn toàn khác nhau về bản chất. Nêu cả hai, và nói cái nào lớn hơn.
- Q5. Một kỹ sư cơ khí đề xuất thêm một cái côn dẫn hướng vào miệng dock, và nói rằng như vậy thị giác chỉ cần chính xác 15 mm thay vì 5 mm. Vì sao đề xuất ấy thường đúng, và trong trường hợp nào thì nó không giúp gì?

13.1 Đặt vấn đề

Mọi chương trước trong cây đều tìm cách làm cho một con số chính xác hơn. Chương này hỏi một câu khác: *con số ấy có thật sự cần chính xác không?*

Bài toán như thường được phát biểu: robot phải cập vào dock với sai lệch dưới 5 mm và $0,5^\circ$. Phát biểu ấy nghe như một yêu cầu về *độ chính xác* — robot phải biết nó đang ở đâu so với dock, chính xác tới 5 mm. Và nếu đó là yêu cầu thật, thì ngân sách sai số của C/15 phải tính mọi mất xích: intrinsic, méo, constellation, hand-eye, và mọi thứ khác — vì mỗi mất xích đóng góp vào việc “biết mình ở đâu”.

Nhưng hãy nhìn kỹ hơn vào việc robot thật sự cần làm. Nó cần đưa đầu sọc của nó chạm đúng vào tiếp điểm trên dock. Nó *không* cần biết tọa độ của mình trong bất kỳ hệ quy chiếu nào. Nếu ta từng đưa robot tới đúng vị trí cập chuẩn — bằng tay, bằng đồ gá — và ghi lại camera nhìn thấy gì tại đó, thì nhiệm vụ trở thành: *đưa camera về trạng thái nhìn thấy đúng như vậy*.

Sự khác biệt giữa hai phát biểu ấy không phải chuyện chữ nghĩa. Tiêu chuẩn robot công nghiệp phân biệt rạch ròi *độ chính xác* và *độ lặp lại* [3], và với hầu hết robot thì độ lặp lại tốt hơn độ chính xác cả bậc độ lớn. Lý do là thế này: độ chính xác đòi mô hình đúng — mọi tham số hình học phải khớp với thực tế. Độ lặp lại chỉ đòi mô hình *nhất quán* — nó có thể sai, miễn là sai giống nhau ở cả hai lần.

Và đây là chỗ nối với A/05 mục 8. Các nguồn sai số chia làm hai loại: ngẫu nhiên (đổi giữa các lần đo) và hệ thống (không đổi). Trong bài toán độ chính xác, cả hai đều tính. Trong bài toán độ lặp lại, *chỉ nguồn ngẫu nhiên tính*, vì nguồn hệ thống xuất hiện ở cả lần dạy lẫn lần chạy và trừ đi nhau. Với một hệ thống điển hình, các nguồn hệ thống chiếm phần lớn ngân sách, nên việc loại chúng đi là một cải thiện bậc độ lớn chứ không phải một cải thiện tăng dần.

Cách hiển nhiên — và cách gần như mọi dự án đi — là giải bài toán độ chính xác: hiệu chuẩn thật kỹ, đo constellation thật kỹ, tính pose thật chính xác, rồi điều khiển tới tọa độ đích. Cách này *hoạt động* nếu bạn làm đủ kỹ, và toàn bộ Phần A cùng các chương khác của Phần B là hướng dẫn để làm đủ kỹ. Nhưng nó tốn nhiều công hơn cần thiết cho một bài toán mà bản chất là lặp lại. Chương này lập luận rằng nếu bài toán của bạn đúng là “về đúng một chỗ”, thì có một con đường ngắn hơn nhiều.

Cần nói ngay một giới hạn để không bán quá lời: teach-by-showing *không* miễn cho bạn khỏi mọi thứ. Nó triệt tiêu sai số hệ thống ở *bậc nhất* và chỉ trong lân cận của tư thế đã dạy; càng xa tư thế

ấy thì triết tiêu càng kém. Mục 3 nói rõ chỗ này, vì một mẹo được hiểu sai thì nguy hiểm hơn không biết mẹo.

TRỰC GIÁC

Hình dung việc đỗ xe vào một chỗ hẹp.

Cách thứ nhất: đo. Bạn đo chiều rộng chỗ đỗ, đo chiều dài xe, tính toạ độ mà bánh xe phải tới, rồi lái theo con số ấy. Cách này đòi mọi phép đo đúng — nếu thước của bạn sai một phân thì xe đỗ lệch một phân.

Cách thứ hai: nhớ. Bạn đã đỗ vào chỗ này một trăm lần. Bạn biết khi nào cột đèn nằm ngay góc kính chắn gió thì bánh sau đúng chỗ. Bạn không biết cái cột đèn ấy cách bánh xe bao nhiêu xăng-ti-mét, và bạn *không cần biết*. Bạn chỉ cần lái sao cho hình ảnh trước mắt trùng với hình ảnh bạn nhớ.

Cách thứ hai chính xác hơn nhiều, và lý do rất đáng nghĩ: nó không dùng phép đo nào cả, nên nó không mang sai số của phép đo nào cả. Nếu kính chắn gió của bạn hơi cong làm cột đèn trông lệch đi hai phân — thì nó cũng trông lệch đi đúng hai phân ấy vào cái lần bạn đỗ đúng, nên hai cái lệch triệt tiêu nhau.

Đó là toàn bộ ý tưởng. Sai số nào xuất hiện *giống hệt nhau* ở lần dạy và lần chạy thì tự biến mất, mà không cần ai biết nó tồn tại.

Nhưng có một giới hạn, và nó cũng thấy được bằng cùng hình ảnh. Nếu hôm nay bạn ngồi cao hơn hôm trước năm phân, thì cái cột đèn không còn nằm ở chỗ cũ trên kính nữa, và mẹo hỏng. Sai số chỉ triệt tiêu khi tình huống *đủ giống* tình huống lúc dạy. Càng lệch xa thì triết tiêu càng kém — và đó là lý do mẹo này hoạt động tốt ở đoạn cuối hành trình và kém hơn ở đoạn đầu.

13.2 Hai họ phương pháp: PBVS và IBVS

Trước khi vào mẹo chính, đặt bản đồ cho không gian lựa chọn. Có hai họ, và chúng khác nhau ở chỗ *sai lệch được định nghĩa trong không gian nào* [59].

PBVS (position-based): ước lượng pose từ ảnh, so với pose đích, được một sai lệch trong $SE(3)$, rồi điều khiển để triệt tiêu nó. Đây là cách trực quan nhất và là cách hầu hết người ta làm mặc định. Ưu điểm: sai lệch có nghĩa vật lý rõ ràng, quỹ đạo trong không gian thật dễ dự đoán và dễ ràng buộc (tránh vật cản, giữ trong vùng làm việc). Nhược điểm: nó đi qua bước ước lượng pose, nên nó thừa hưởng *mọi* vấn đề của Phần A — ambiguity, nhạy với intrinsic, nhạy với constellation.

IBVS (image-based): định nghĩa sai lệch trực tiếp trên toạ độ các điểm đặc trưng trên ảnh. Gọi $\mathbf{s}$ là vectơ chứa toạ độ ảnh của các góc marker và $\mathbf{s}^*$ là giá trị của chúng tại vị trí đích. Sai lệch là $\mathbf{e} = \mathbf{s} - \mathbf{s}^*$, và luật điều khiển cơ bản là

$$\mathbf{v} = -\lambda L^+ \mathbf{e}, \quad (13.1)$$

với L là ma trận tương tác nối vận tốc camera với tốc độ đổi của đặc trưng ảnh, L^+ là giả nghịch đảo của nó, và $\lambda > 0$ là hệ số khuếch đại [59, §II].

Đây là câu trả lời cho Q2, và hai vấn đề mà IBVS tránh hoàn toàn đáng nêu rõ.

Vấn đề thứ nhất: pose ambiguity. IBVS không bao giờ giải PnP, nên nó không bao giờ phải chọn giữa hai nghiệm. Toàn bộ A/04 và B/10 trở nên không liên quan. Đây là một lợi ích lớn và ít được nhấn mạnh đủ.

Vấn đề thứ hai: độ nhạy với hiệu chuẩn. Trong (13.1), sai số của K và của mô hình constellation chỉ vào qua ma trận L , tức chỉ ảnh hưởng tới *hướng và độ lớn của bước điều khiển*, không ảnh hưởng tới *điểm dừng*. Điểm dừng được định nghĩa bởi $\mathbf{e} = 0$, tức bởi ảnh trùng ảnh đích — một điều

kiện hoàn toàn không chứa K . Hệ quả: một L sai làm robot đi tới đích theo đường vòng vèo hơn nhưng nó *vẫn tới đúng đích*. Đây là tính chất bền vững kinh điển của IBVS, đã được nêu từ bài gốc [60].

Cái giá phải trả, và nó thật: quỹ đạo trong không gian ba chiều không được kiểm soát trực tiếp. IBVS có thể tạo ra những chuyển động kỳ lạ — trường hợp nổi tiếng nhất là “camera retreat”, khi một phép xoay lớn quanh trục quang làm camera lùi ra xa rồi tiến lại. Với robot di động trên sàn và tag ở phía trước, các bệnh lý ấy ít gặp hơn nhiều so với trường hợp cánh tay máy sáu bậc tự do, nhưng chúng vẫn nên được kiểm bằng mô phỏng. Có các phương pháp lai giải quyết chuyện này bằng cách tách phần xoay ra khỏi phần tịnh tiến [61], [62].

Bảng 13.1. PBVS, IBVS và lai: sai lệch định nghĩa ở đâu, và hệ quả.

	Sai lệch trong	Ưu	Nhược
PBVS	Không gian pose $SE(3)$	Quỹ đạo 3D dự đoán được, dễ ràng buộc	Thừa hưởng toàn bộ vấn đề của PnP: ambiguity, nhạy hiệu chuẩn
IBVS	Không gian ảnh	Không giải PnP nên không có ambiguity; điểm dừng không phụ thuộc K [60]	Quỹ đạo 3D không kiểm soát trực tiếp; có bệnh lý (camera retreat)
Lai 2,5D	Tách xoay và tịnh tiến [61]	Giữ phần lớn ưu điểm của cả hai	Phức tạp hơn; cần phân rã homography nên lại chạm vào A/02

13.3 Teach-by-showing: mẹo lớn nhất

Đây là mục trung tâm của chương và nó trả lời Q1.

A. Cơ chế

Quy trình gồm hai giai đoạn.

Giai đoạn dạy, làm một lần. Đưa robot tới đúng vị trí cập chuẩn — bằng tay, bằng một đồ gá cơ khí, hoặc bằng cách để nó cập vào rồi tinh chỉnh cho tới khi tiếp điểm khớp hoàn hảo. Tại đó, ghi lại trạng thái cảm biến: hoặc là pose T^* tính được từ constellation (cho PBVS), hoặc là véctơ toạ độ ảnh s^* của mọi góc nhìn thấy (cho IBVS). Ghi lấy trung bình nhiều khung khi robot đứng yên, để s^* có nhiễu nhỏ.

Giai đoạn chạy, mỗi lần cập dock. Điều khiển để đưa trạng thái hiện tại về trùng với trạng thái đã ghi. Với PBVS, sai lệch là $T^{-1}T^*$ — một phép biến đổi tương đối. Với IBVS, sai lệch là $s - s^*$.

B. Vì sao sai số hệ thống triệt tiêu

Xét một nguồn sai số hệ thống bất kỳ — gọi nó là một hàm g biến pose thật thành pose đo được, không phụ thuộc thời gian. Nó có thể là: intrinsic sai, méo còn sót, constellation lệch CAD, hand-eye sai. Pose đo được tại vị trí đích là $\hat{T}^* = g(T_{\text{thật}}^*)$, và pose đo được tại vị trí hiện tại là $\hat{T} = g(T_{\text{thật}})$.

Bộ điều khiển triệt tiêu $\hat{T}^{-1}\hat{T}^*$. Khi nó hội tụ, $\hat{T} = \hat{T}^*$, tức $g(T_{\text{thật}}) = g(T_{\text{thật}}^*)$. Nếu g là đơn ánh — và mọi sai số nhỏ liên tục đều thế — thì $T_{\text{thật}} = T_{\text{thật}}^*$.

Đọc kết quả ấy cho kỹ, vì nó mạnh hơn vẻ ngoài: *robot tới đúng vị trí thật, bất kể g là gì*. Bạn không cần biết g , không cần đo nó, không cần mô hình hoá nó. Nó chỉ cần *không đổi* giữa lần dạy và lần chạy.

Điều kiện “không đối” là toàn bộ nội dung của phần giới hạn dưới đây, và nó là câu trả lời cho về “gần như chứ không hoàn toàn” của Q1.

C. Bốn chỗ triệt tiêu không hoàn toàn

Một, g phụ thuộc tư thế. Đây là chỗ lớn nhất. Sai số do méo còn sót phụ thuộc vị trí tag trên ảnh; bias tâm ellipse phụ thuộc góc nghiêng (A/02 mục 6); sai số do điểm chính phụ thuộc cả vị trí lẫn khoảng cách (A/01 mục 5). Khi robot ở gần tư thế đích, các phụ thuộc ấy gần như nhau và triệt tiêu tốt. Khi robot còn xa, chúng khác nhau và triệt tiêu kém. Nói bằng ngôn ngữ giải tích: triệt tiêu là ở bậc nhất quanh tư thế dạy, và phần dư là bậc hai theo khoảng cách tới tư thế ấy.

Hai, g trôi theo thời gian. Trôi nhiệt của ống kính, tag bị bắn dần, dock bị va chạm nhẹ. Mọi thứ làm g đổi giữa lần dạy và lần chạy đều phá mero. Cách phòng: dạy lại định kỳ — và đây là một lợi thế thực dụng của phương pháp, vì dạy lại rẻ hơn hiệu chuẩn lại rất nhiều.

Ba, nguồn ngẫu nhiên không triệt tiêu. Nhiều định vị góc, nhiều ảnh — những thứ đổi giữa các khung — không bị ảnh hưởng bởi mero này chút nào. Chúng vẫn ở đó và chúng vẫn phải được xử lý bằng cách lấy trung bình (B/11). Điều mero này làm là biến ngân sách từ “hệ thống cộng ngẫu nhiên” thành “chỉ ngẫu nhiên”, và vì ngẫu nhiên chia cho $\sqrt{N}$ nên phần còn lại rất dễ xử lý.

Bốn, sai số của chính lần dạy. Vị trí đích được xác định bằng chất lượng của lần dạy. Nếu bạn đưa robot tới bằng tay và nó lệch 1 mm so với vị trí lý tưởng, thì mọi lần chạy về sau sẽ lặp lại đúng 1 mm lệch ấy. Đây không phải nhược điểm mà là đặc tính: mero cho bạn độ lặp lại, không cho độ chính xác. Nếu vị trí đích được xác định bởi chính cơ khí — robot cập vào, chốt khớp, tiếp điểm chạm — thì lần dạy tự nó chính xác và vấn đề biến mất.

CẦN NHỚ

Teach-by-showing biến bài toán từ *accuracy* thành *repeatability*, và đó là một thay đổi bậc độ lớn chứ không phải một cải thiện tăng dần — vì trong một hệ điển hình, các nguồn hệ thống chiếm phần lớn ngân sách.

Ba điều kiện để nó hoạt động:

Một, g phải không đổi giữa lần dạy và lần chạy — nên phải khoá mọi thứ khoá được (exposure, gain, nét, vị trí camera) và dạy lại định kỳ.

Hai, robot phải tới gần tư thế dạy — triệt tiêu là ở bậc nhất, nên nó tốt ở đoạn cuối và kém ở đoạn đầu. Đây là lý do coarse-to-fine ở mục 5 là bạn đồng hành tự nhiên của mero này.

Ba, lần dạy phải đúng — lý tưởng nhất là để cơ khí xác định vị trí đích thay vì mắt người.

Nếu bài toán của bạn đúng là “về đúng một chỗ”, đây là con đường ngắn nhất tới 5 mm, và nó nên là quyết định kiến trúc đầu tiên chứ không phải tối ưu hoá cuối cùng.

13.4 Stop-and-go

Mục ngắn, vì A/06 đã dẫn cơ sở. Nó trả lời Q4.

Dừng hẳn robot, chụp N khung, tính trung bình, rồi bước tiếp. Hai lợi ích, và chúng khác nhau về bản chất — đó là phần thứ hai của Q4.

Lợi ích thứ nhất, giảm nhiễu: trung bình N khung giảm thành phần ngẫu nhiên theo $1/\sqrt{N}$. Với $N = 30$, đó là hệ số 5,5. Đây là lợi ích ai cũng biết.

Lợi ích thứ hai, xoá thiên lệch: rolling shutter và motion blur là các nguồn hệ thống tỉ lệ với vận tốc, nên ở $v = 0$ chúng bằng không (A/06 mục 7). Đây là lợi ích lớn hơn, vì trung bình nhiều khung

không bao giờ chạm tới được các nguồn hệ thống. Nói cách khác: trung bình giảm một dòng ngân sách theo $\sqrt{N}$, còn dừng lại thì *xoá hẳn* hai dòng.

Chi phí: thời gian. Dừng 0,3 giây mỗi lần đo, với vài lần đo ở đoạn cuối, là vài giây cho cả quy trình cập dock — gần như luôn chấp nhận được trong ứng dụng docking, vì đây không phải thao tác thời gian thực gấp gáp. (Con số 0,3 giây và 30 khung là ví dụ minh hoạ; số đúng phụ thuộc tốc độ khung hình và thời gian ổn định cơ khí sau khi phanh — đừng quên rằng robot còn rung một lúc sau khi bánh xe dừng.)

Chi tiết dễ quên vừa nêu đáng nhấn mạnh: *phải đợi rung tắt*. Dừng bánh xe không có nghĩa là camera đứng yên; thân robot còn dao động, và chụp ngay sau khi phanh thì bạn vẫn có motion blur. Cách kiểm: đo độ tản mát của s theo thời gian sau khi phanh, và chọn thời điểm bắt đầu chụp là lúc độ tản mát ấy chạm sàn.

13.5 Coarse-to-fine

Nguyên tắc tổ chức của cả hành trình, và nó là cái khung mà bốn công cụ kia gắn vào.

Quan sát: *độ chính xác cần thiết không đồng đều theo hành trình*. Ở 3 m, robot chỉ cần biết hướng đại khái để tiến về phía dock — sai 10 cm không sao. Ở 0,5 m, nó cần 5 mm. Yêu cầu tăng hai bậc độ lớn, và may mắn là khả năng cũng tăng theo cùng chiều: $A/05$ cho $\delta x \approx Z\sigma/f$, tuyến tính theo Z , nên tới gần thì tự nhiên chính xác hơn.

Chia hành trình thành ba pha, mỗi pha một cấu hình khác nhau.

Pha xa (3 m tới 1 m). Dùng tag lớn ở ngoại vi (B/09 mục 6). Điều khiển liên tục, không dừng. Mục tiêu: đưa robot vào vùng mà cụm tag nhỏ nhìn thấy được và vào gần trực tiếp cận. Dung sai rộng, nên rolling shutter và blur không quan trọng.

Pha gần (1 m tới 0,3 m). Chuyển sang cụm tag nhỏ ở trung tâm. Vẫn điều khiển liên tục nhưng chậm hơn. Bắt đầu bật các phép kiểm nhất quán của B/09 mục 7.

Pha cuối (0,3 m tới chạm). Stop-and-go, trung bình nhiều khung, teach-by-showing với sai lệch tương đối. Đây là pha duy nhất mà ngân sách 5 mm áp dụng, và mọi kỹ thuật đắt tiền đều dồn vào đây.

Cấu trúc này có một lợi ích không hiển nhiên: nó cho phép các pha dùng *tiêu chí khác nhau*. Pha xa có thể dùng PBVS vì nó cần quỹ đạo dự đoán được để tránh vật cản; pha cuối có thể dùng IBVS vì nó cần độ bền với hiệu chuẩn và không cần quỹ đạo phức tạp. Chuyển đổi giữa hai luật điều khiển ở một điểm mà cả hai đều hợp lệ là một kỹ thuật chuẩn và nó đơn giản hơn nhiều so với việc ép một luật duy nhất phục vụ cả hai mục đích.

13.6 Ràng buộc nonholonomic

Mục này trả lời Q3, và nó là chỗ bài toán docking của robot di động khác hẳn bài toán servoing của cánh tay máy.

Robot vì sai hai bánh có ba bậc tự do cấu hình (x, y, θ) nhưng chỉ hai đầu vào điều khiển (vận tốc tiến v và vận tốc quay ω). Nó *không đi ngang được*: vận tốc theo phương vuông góc trục bánh luôn bằng không. Đây là ràng buộc nonholonomic — ràng buộc trên vận tốc, không trên vị trí.

Hệ quả thứ nhất, dễ thấy: sửa một lệch ngang 2 cm đòi một cơ động — tiến, quay, lùi, quay — chứ không phải một chuyển động thẳng. Với không gian chật trước dock, cơ động ấy có thể không thực hiện được.

Hệ quả thứ hai, sâu hơn và đây là phần thứ hai của Q3: *định lý Brockett* nói rằng không tồn tại luật phản hồi trạng thái trơn, dừng (không phụ thuộc thời gian) nào ổn định hoá hệ nonholonomic về một điểm [63]. Đây không phải một khó khăn kỹ thuật mà là một bất khả về cấu trúc: bất kỳ ai hứa hẹn một bộ điều khiển tỉ lệ đơn giản đưa robot vi sai về đúng một tư thế đều đang bỏ sót điều gì đó.

Lỗi ra có ba, và cả ba đều được dùng trong thực tế.

Luật điều khiển không trơn hoặc phụ thuộc thời gian. Định lý Brockett chỉ loại các luật trơn và dừng; nói một trong hai điều kiện thì có lời giải [64]. Đây là con đường lý thuyết đúng và nó có các luật đã được chứng minh.

Thiết kế đường tiếp cận thẳng trục. Đây là con đường thực dụng và tôi khuyên nó. Thay vì cố sửa lệch ngang ở gần dock, hãy *đảm bảo không có lệch ngang* bằng cách đưa robot vào một đoạn tiếp cận thẳng từ xa: ở khoảng 1–1,5 m, robot căn chỉnh để nằm trên trục dock, rồi từ đó chỉ đi thẳng. Sai lệch ngang được sửa ở nơi có không gian để cơ động, và pha cuối chỉ còn một bậc tự do cần điều khiển chặt. Điều này đồng thời giúp cho thị giác: đi thẳng trục nghĩa là góc nhìn tag không đổi nhiều, nên teach-by-showing triệt tiêu tốt.

Cơ khí. Mục 7.

Đáng nêu một đánh đổi mà mục này tạo ra với A/05: đi thẳng trục là tư thế fronto-parallel, tức tư thế *tệ nhất* cho việc đo góc từ một tag phẳng. Hai yêu cầu xung đột. Cách giải quyết đúng không phải là hy sinh một trong hai mà là *ngiên các tag* (B/09 mục 4): robot đi thẳng trục, nhưng các tag không vuông góc với trục ấy. Đây là một ví dụ đẹp của việc một quyết định cơ khí giải quyết một xung đột mà không quyết định phần mềm nào giải quyết được.

13.7 Dung sai cơ khí: côn dẫn hướng

Mục này trả lời Q5, và nó là chỗ một cây tri thức về thị giác phải trung thực thừa nhận rằng đôi khi câu trả lời không nằm ở thị giác.

Ý tưởng: thêm một mặt vát (chamfer) hoặc một côn dẫn hướng ở miệng dock. Khi robot tới với một sai lệch nhỏ, đầu cắm chạm vào mặt vát và được *dẫn* vào đúng chỗ thay vì va vào mép. Lý thuyết của chuyện này thuộc ngành lắp ráp tự động và đã có từ lâu; Whitney phân tích điều kiện để một phép lắp có tuân thủ thành công dưới sai lệch vị trí và góc [4].

Hiệu quả về mặt yêu cầu: nếu côn dẫn hướng có bề rộng vào w_c , thì yêu cầu đối với hệ thị giác giảm từ “dưới dung sai khớp” xuống “dưới w_c ”. Với một côn rộng 15 mm, bài toán 5 mm thành bài toán 15 mm — và theo A/05, đó là một bài toán dễ hơn ba lần theo mọi chiều, tức dễ hơn nhiều lần về công sức.

Vì sao đề xuất ấy thường đúng: chi phí. Một mặt vát trên một chi tiết đã phải gia công là gần như miễn phí. So với nó, việc hạ sai số thị giác từ 15 mm xuống 5 mm tốn hiệu chuẩn kỹ hơn, constellation chính xác hơn, có thể là camera tốt hơn, và nhiều tuần kỹ thuật.

Ba trường hợp nó *không* giúp, và đây là phần thứ hai của Q5.

Một, khi sai lệch góc là vấn đề chứ không phải sai lệch vị trí. Côn dẫn hướng sửa được lệch vị trí tốt; sửa lệch góc thì kém hơn nhiều, vì một đầu cắm nghiêng có thể kẹt thay vì trượt vào. Với ngân sách $0,5^\circ$, côn giúp ít.

Hai, khi tiếp điểm cần chính xác hơn cơ khí dẫn được. Sạc điện thì côn giúp; cắm một đầu nối quang học đòi căn chỉnh vi mét thì không.

Ba, khi không có lực để dẫn. Côn hoạt động nhờ robot tiếp tục đẩy tới và lực phản từ mặt vát đẩy nó vào đúng chỗ. Nếu robot nhẹ, hoặc nếu bánh trượt, hoặc nếu cơ cấu không cho phép tuân thủ,

thì thay vì được dẫn vào, robot sẽ bị chặn lại. Điều này dẫn tới một yêu cầu thiết kế đi kèm: phải có *độ tuân thủ* (compliance) ở đâu đó trong chuỗi — lò xo, cao su, hoặc một cơ cấu nổi.

CẦN NHỚ

Trước khi bỏ công hạ sai số thì giác một bậc, hãy hỏi kỹ sư cơ khí xem có thể nói dung sai một bậc không.

Thường thì có, và thường thì nó rẻ hơn nhiều. Đây không phải một lời khuyên khiêm tốn giả tạo — nó là kết luận đúng của một phân tích chi phí, và một kỹ sư thì giác không đặt câu hỏi ấy là đang bỏ sót một biến trong bài toán tối ưu.

Nhưng có một điều kiện: còn dẫn hướng sửa *vị trí* tốt hơn sửa *góc* nhiều. Với ngân sách $0,5^\circ$, nó giúp ít — và theo A/05 thì góc vốn đã là nút thắt. Nên cách đọc đúng là: cơ khí nói được dòng ngân sách tịnh tiến (vốn đã dư), và không nói được dòng ngân sách góc (vốn đang thiếu).

Trừ khi — và đây là chỗ đáng nghĩ — cơ cấu được thiết kế để *tự căn góc*: một đôi chốt còn thay vì một chốt duy nhất, hoặc một mặt phẳng tựa. Khi ấy nó nói được cả hai, và đó là đề xuất đáng đưa ra với kỹ sư cơ khí.

13.8 Tiêu chí hội tụ và cơ chế retry

Chi tiết kỹ thuật cuối, và nó là chỗ hay bị làm ầu.

Bộ điều khiển cần một tiêu chí dừng. Tiêu chí sai lầm phổ biến: dừng khi sai lệch nhỏ hơn ngưỡng. Vấn đề là nó không phân biệt “sai lệch nhỏ vì đã tới đích” với “sai lệch nhỏ vì phép đo sai”.

Tiêu chí đúng gồm ba điều kiện, phải thoả đồng thời.

Một, sai lệch dưới ngưỡng — điều kiện hiển nhiên.

Hai, phép đo đáng tin — số tag nhìn thấy đủ, tỉ số ambiguity ρ trên ngưỡng (B/10), sai số tái chiếu trong khoảng bình thường, phép kiểm nhất quán liên tag (B/09 mục 7) không báo động, và hiệp phương sai từ $\mathcal{I}^{-1}$ đủ nhỏ.

Ba, ổn định — sai lệch giữ dưới ngưỡng qua nhiều khung liên tiếp, chứ không phải một khung may mắn.

Khi một trong ba điều kiện không thoả sau một thời gian chờ, cần một cơ chế *retry* chứ không phải báo lỗi. Retry hợp lý: lùi lại một khoảng, đợi, tiếp cận lại. Lý do nó thường hiệu quả: nhiều nguyên nhân thất bại là tạm thời hoặc phụ thuộc tư thế — một vật nặng chiếu vào tag, một góc nhìn xấu làm ambiguity nặng, một vật cản thoáng qua — và tiếp cận lại từ một điểm hơi khác thường thoát ra được.

Ghi lại lý do mỗi lần retry vào log. Phân bố của các lý do ấy sau một tháng vận hành là dữ liệu chẩn đoán giá trị nhất bạn có thể có về hệ thống của mình, và nó thường chỉ ra nút thắt thật — thứ mà không phân tích lý thuyết nào đoán trước được.

13.9 Giới hạn và chế độ hỏng

Teach-by-showing đòi một lần dạy đúng. Đã bàn ở mục 3C. Nếu vị trí đích không được cơ khí xác định thì độ chính xác tuyệt đối của hệ bị giới hạn bởi chất lượng lần dạy, và mọi lần chạy sẽ lặp lại sai lệch ấy một cách trung thành.

Teach-by-showing không xử lý được thay đổi. Nếu dock bị dịch chuyển hoặc tag bị thay, phải dạy lại. Đây là đánh đổi so với PBVS đầy đủ: PBVS hoạt động ở bất kỳ dock nào có constellation đã biết; teach-by-showing gắn với một dock cụ thể.

IBVS có bệnh lý. Camera retreat và cực tiểu cục bộ trong không gian ảnh. Với hình học đơn giản của docking thì hiếm, nhưng phải kiểm bằng mô phỏng toàn bộ dải tư thế ban đầu chứ không chỉ vài trường hợp.

Định lý Brockett không tránh được. Mọi lời hứa về một bộ điều khiển tỉ lệ tron đưa robot vi sai về đúng một tư thế đều sai [63]. Nếu một cài đặt trông như đang làm được điều đó, thì hoặc nó không thực sự hội tụ về tư thế (chỉ về vị trí), hoặc nó đang dùng một luật không tron mà tác giả không nhận ra.

Các con số trong chương là ví dụ. Côn 15 mm, dừng 0,3 giây, 30 khung, chuyển pha ở 1 m và 0,3 m — tất cả là ví dụ minh hoạ do tôi đặt để có một cấu hình cụ thể nói chuyện. Con số đúng cho hệ của bạn phụ thuộc cơ khí, camera, và tốc độ khung hình.

13.10 Thực hành

Năm việc, theo thứ tự ưu tiên — và thứ tự này khác với thứ tự trình bày trong chương, có chủ ý. Một, *hỏi kỹ sư cơ khí trước*. Có côn dẫn hướng được không? Có tự căn góc được không? Câu trả lời có thể làm nhẹ toàn bộ phần còn lại.

Hai, *quyết định teach-by-showing hay không, ngay từ đầu*. Đây là quyết định kiến trúc, và nó ảnh hưởng tới việc bạn cần hiệu chuẩn kỹ tới mức nào ở B/12. Nếu bài toán là “về đúng một chỗ” thì chọn nó.

Ba, *thiết kế đường tiếp cận thẳng trục*, với việc căn chỉnh ngang hoàn tất ở 1–1,5 m. Và nhớ bù lại bằng cách nghiêng tag (B/09 mục 4), vì đi thẳng trục là tư thế xấu cho đo góc.

Bốn, *stop-and-go ở pha cuối*, với thời gian đợi rung tắt được đo chứ không đoán.

Năm, *tiêu chí hội tụ ba điều kiện và cơ chế retry có ghi log lý do*.

Active recall (che cột phải, tự trả lời, rồi mở ra đối chiếu)

Thuật ngữ / câu hỏi	Trả lời
••• Quan sát trung tâm của chương	Docking cần <i>repeatability</i> , không cần <i>accuracy</i> [3]. Robot không cần biết nó ở đâu; nó cần về đúng chỗ nó đã từng ở.
••• Vì sao teach-by-showing triệt tiêu sai số hệ thống	Cùng một hàm sai lệch g tác động lên cả pose đo hiện tại lẫn pose đo mục tiêu; khi bộ điều khiển làm hai cái bằng nhau thì hai pose <i>thật</i> bằng nhau, bất kể g là gì — miễn g đơn ánh và không đổi.
••• Bốn chỗ triệt tiêu không hoàn toàn	g phụ thuộc tư thế (chỉ triệt tiêu ở bậc nhất, tốt khi gần đích); g trôi theo thời gian (dạy lại định kỳ); nguồn ngẫu nhiên không triệt tiêu; và sai số của chính lần dạy được lặp lại trung thành.
••• Hai vấn đề IBVS tránh hoàn toàn	Pose ambiguity (không giải PnP nên không phải chọn nghiệm); và độ nhạy với hiệu chuẩn — K sai chỉ vào qua ma trận tương tác L , ảnh hưởng <i>đường đi</i> chứ không ảnh hưởng <i>điểm dừng</i> [60].
•• Luật IBVS cơ bản	$\mathbf{v} = -\lambda L^+ \mathbf{e}$ với $\mathbf{e} = \mathbf{s} - \mathbf{s}^*$ là sai lệch trong không gian ảnh [59, §II].
•• Cái giá của IBVS	Quỹ đạo 3D không kiểm soát trực tiếp; bệnh lý camera retreat khi có phép xoay lớn quanh trục quang. Hiếm với hình học docking nhưng phải kiểm bằng mô phỏng.

Thuật ngữ / câu hỏi	Trả lời (tiếp)
●●● Hai lợi ích khác bản chất của stop-and-go	Giảm nhiễu theo $1/\sqrt{N}$ (ai cũng biết); và <i>xoá</i> rolling shutter cùng motion blur vì chúng tỉ lệ với vận tốc (lớn hơn, vì trung bình không bao giờ chạm tới nguồn hệ thống).
●● Chi tiết dễ quên của stop-and-go	Phải đợi <i>rung tắt</i> , không phải chỉ đợi bánh xe dừng. Đo bằng cách theo dõi độ tần mất của s sau khi phanh và chọn thời điểm nó chạm sàn.
●●● Định lý Brockett	Không tồn tại luật phản hồi trạng thái <i>trơn và dừng</i> nào ổn định hoá hệ nonholonomic về một điểm [63]. Bất khả về cấu trúc, không phải khó khăn kỹ thuật.
●●● Lỗi ra thực dụng cho nonholonomic	Thiết kế đường tiếp cận thẳng trục: căn chỉnh ngang xong ở 1–1,5 m nơi có không gian cơ động, rồi chỉ đi thẳng. Pha cuối chỉ còn một bậc tự do cần điều khiển chặt.
●●● Xung đột giữa mục 6 và A/05	Đi thẳng trục là tư thế fronto-parallel — tệ nhất cho đo góc. Cách giải: <i>ngiên tag</i> chứ không nghiêng quỹ đạo. Một quyết định cơ khí giải quyết xung đột mà phần mềm không giải được.
●●● Côn dẫn hướng biến bài toán 5 mm thành bài toán w_c mm	Chi phí gần bằng không trên chi tiết đã phải gia công [4]. Nhưng nó sửa <i>vị trí</i> tốt hơn sửa <i>góc</i> nhiều — nên nó nối dòng tịnh tiến (vốn đã dư) chứ không nối dòng góc (vốn đang thiếu).
●● Ba trường hợp côn không giúp	Khi sai lệch <i>góc</i> là vấn đề; khi tiếp điểm cần chính xác hơn cơ khí dẫn được; khi không có lực hoặc không có độ tuân thủ để dẫn.
●● Ba pha coarse-to-fine	Xa: tag lớn, điều khiển liên tục, dung sai rộng. Gần: tag nhỏ, chậm hơn, bật kiểm nhất quán. Cuối: stop-and-go, trung bình nhiều khung, teach-by-showing. Các pha có thể dùng <i>luật điều khiển khác nhau</i> .
●●● Tiêu chí hội tụ ba điều kiện	Sai lệch dưới ngưỡng và phép đo đáng tin (đủ tag, ρ cao, residual bình thường, nhất quán liên tag, hiệp phương sai nhỏ) và ổn định qua nhiều khung. Chỉ điều kiện thứ nhất là không phân biệt được “tới đích” với “đo sai”.
● Giá trị của log lý do retry	Phân bố lý do sau một tháng vận hành là dữ liệu chẩn đoán giá trị nhất về hệ thống, và nó thường chỉ ra nút thắt thật — thứ không phân tích lý thuyết nào đoán trước được.

●●● phải thuộc ●● nên nắm ● biết là có

I Kiến thức này đang sống ở đâu (công trình đang dùng chính ý tưởng của chương; chốt 9/2026)

Công trình / hệ thống	Dùng ý tưởng nào của chương	Ghi chú
ViSP [65]	Thư viện visual servoing đầy đủ: IBVS, PBVS, lai, có sẵn tracker marker và tính ma trận tương tác	Nơi mục 2 thành code; đọc phần IBVS kèm [59]
Bài gốc IBVS [60]	Nguồn của luận điểm “điểm dừng không phụ thuộc hiệu chuẩn”	Đọc để hiểu vì sao IBVS bền, không chỉ biết rằng nó bền
Tutorial Chaumette–Hutchinson [59], [62]	Tài liệu chuẩn hai phần cho toàn bộ mục 2, gồm cả các trường hợp hỏng	Phần II bàn chính các bệnh lý ở mục 9
Lắp ráp có tuân thủ [4]	Lý thuyết chamfer và điều kiện thành công dưới sai lệch vị trí/góc	Nguồn cho mục 7; đáng đưa cho kỹ sư cơ khí
Điều khiển hệ xích [64]	Luật điều khiển phụ thuộc thời gian cho hệ nonholonomic, lách định lý Brockett	Con đường lý thuyết đúng nếu không thể thiết kế đường tiếp cận thẳng trục

MINI PROJECT · Đo lợi ích thật của teach-by-showing trên hệ của bạn · 3–4 ngày

Mục tiêu	biến luận điểm trung tâm của chương từ một lập luận thành một cặp số đo, và biết chính xác mẹo này đáng giá bao nhiêu milimét trong hệ cụ thể của bạn.
Dữ liệu	phần cứng thật. Cần một cách xác định vị trí đích độc lập với thị giác — lý tưởng nhất là chính cơ cấu dock (robot cập vào, chốt khớp), hoặc một đồ gá cơ khí có dung sai biết trước.
Các bước	giai đoạn một, đo độ <i>chính xác</i> : cố ý làm hỏng hiệu chuẩn một chút (dịch c_x đi 10 px, hoặc dùng tọa độ constellation từ CAD thay vì từ BA), rồi cho robot cập dock theo PBVS tuyệt đối 20 lần, đo sai lệch cuối bằng chuẩn cơ khí; giai đoạn hai, đo độ <i>lặp lại</i> : với đúng hiệu chuẩn hỏng ấy, dạy một lần rồi cho cập 20 lần theo teach-by-showing, đo lại; giai đoạn ba, lặp cả hai với hiệu chuẩn đúng; giai đoạn bốn, lặp giai đoạn hai nhưng cho robot xuất phát từ các vị trí ban đầu xa dần từ thể dạy, để đo tốc độ mất hiệu lực của phép triệt tiêu bậc nhất.
Xong khi	bạn có bốn con số: accuracy với hiệu chuẩn hỏng, repeatability với hiệu chuẩn hỏng, và hai con số tương ứng với hiệu chuẩn đúng. Nếu lập luận của mục 3 đúng thì con số thứ hai phải gần con số thứ tư — tức teach-by-showing làm cho việc hiệu chuẩn hỏng gần như không quan trọng. Từ giai đoạn bốn bạn có một đường cong nói mẹo mất hiệu lực ở khoảng cách nào, và đó là con số quyết định pha cuối của coarse-to-fine nên bắt đầu ở đâu. Kiểm tra chéo bắt buộc: nếu repeatability với hiệu chuẩn hỏng <i>không</i> tốt hơn accuracy đáng kể, thì hoặc sai số bạn cố ý thêm vào không phải sai số hệ thống thuần (kiểm lại), hoặc ngân sách của bạn đang bị chi phối bởi nguồn ngẫu nhiên chứ không phải hệ thống — và đó cũng là một kết quả quan trọng, nó nói bạn nên tập trung vào B/11 thay vì B/12.
Nối với	C/15 nhận bốn con số này để hiệu chỉnh bảng ngân sách; C/16 mở rộng thành quy trình nghiệm thu.

TRẢ LỜI CÂU HỎI MỞ ĐẦU

- A1. Teach-by-showing: ghi lại trạng thái cảm biến tại vị trí cập chuẩn một lần, rồi điều khiển để tái tạo đúng trạng thái ấy thay vì điều khiển tới một tọa độ tính toán. Cơ chế: nếu g là hàm biểu diễn toàn bộ sai số hệ thống, thì pose đo được là g của pose thật ở cả hai vế; khi bộ điều khiển làm hai pose đo bằng nhau và g đơn ánh, thì hai pose *thật* bằng nhau — bất kể g là gì, và không cần biết nó. Nó “gần như” chứ không “hoàn toàn” vì bốn lý do: g thực ra phụ thuộc tư thế (méo còn sót, bias ellipse, điểm chính — nên triệt tiêu chỉ ở bậc nhất quanh tư thế dạy và kém dần khi ra xa); g trôi theo thời gian (nhiệt, bẩn, va chạm — cần dạy lại định kỳ); nguồn ngẫu nhiên không bị ảnh hưởng gì; và sai lệch của chính lần dạy được lặp lại trung thành ở mọi lần chạy.
- A2. Vấn đề thứ nhất là pose ambiguity: IBVS không giải PnP nên không bao giờ phải chọn giữa hai nghiệm, và toàn bộ A/04 cùng B/10 trở nên không liên quan. Vấn đề thứ hai là độ nhạy với hiệu chuẩn: sai số của K và của constellation chỉ vào qua ma trận tương tác L , tức chỉ ảnh hưởng tới hướng và độ lớn của *bước điều khiển*, không ảnh hưởng tới *điểm dừng* — vì điểm dừng được định nghĩa bởi $e = s - s^* = 0$, một điều kiện thuần túy trên ảnh không chứa K [60]. Hệ quả: một L sai làm robot đi vòng vèo hơn nhưng vẫn tới đúng đích. Cái giá là quỹ đạo trong không gian ba chiều không được kiểm soát trực tiếp, nên có thể xuất hiện các chuyển động kỳ lạ như camera retreat — hiểm với hình học docking đơn giản nhưng phải kiểm bằng mô phỏng trên toàn dải tư thế ban đầu.
- A3. Khó hơn vì robot vì sai có ràng buộc nonholonomic: vận tốc theo phương vuông góc trục bánh luôn bằng không, nên nó không đi ngang được. Sửa 2 cm ngang đòi một cơ động nhiều bước — tiến, quay, lùi, quay — trong khi sửa 2 cm tiến lùi chỉ là một chuyển động thẳng; và trước dock thường không có không gian cho cơ động ấy. Định lý Brockett [63] nói rằng không tồn tại luật phản hồi trạng thái *trơn và dừng* nào ổn định hoá một hệ nonholonomic về một điểm — đây là bất khả về cấu trúc chứ không phải khó khăn kỹ thuật, nên bất kỳ ai hứa một bộ điều khiển tỉ lệ đơn giản làm được việc đó đều đang bỏ sót gì đó. Lỗi ra thực dụng nhất không phải là tìm luật khéo hơn mà là *thiết kế đường tiếp cận thẳng trực*: hoàn tất căn chỉnh ngang ở 1–1,5 m nơi còn không gian, rồi pha cuối chỉ đi thẳng.

- A4. Lợi ích thứ nhất là giảm *nhiều*: trung bình N khung giảm thành phần ngẫu nhiên theo $1/\sqrt{N}$, với $N = 30$ là hệ số 5,5. Lợi ích thứ hai là xoá *thiên lệch*: rolling shutter và motion blur đều tỉ lệ với vận tốc nên bằng không ở $v = 0$ (A/06 mục 7). Lợi ích thứ hai lớn hơn, và lớn hơn theo một cách có tính nguyên tắc: lấy trung bình *không bao giờ* chạm tới được các nguồn hệ thống, dù lấy bao nhiêu khung đi nữa, nên việc xoá hẳn hai dòng ngân sách có giá trị hơn việc chia một dòng khác cho 5,5. Một chi tiết thực hành hay bị quên: phải *đợi rung tắt* chứ không chỉ đợi bánh xe dừng, vì thân robot còn dao động sau khi phanh.
- A5. Thường đúng vì chi phí: một mặt vát trên một chi tiết vốn đã phải gia công là gần như miễn phí, còn việc hạ sai số thị giác từ 15 mm xuống 5 mm tốn hiệu chuẩn kỹ hơn, constellation chính xác hơn, có thể cả camera tốt hơn, cộng nhiều tuần kỹ thuật [4]. Ba trường hợp nó không giúp. Một, khi vấn đề là sai lệch góc: côn dẫn hướng sửa lệch vị trí tốt nhưng một đầu cảm nghiêng thì kẹt chứ không trượt vào — và theo A/05 thì góc mới là nút thắt, nên đây là hạn chế đáng kể. Hai, khi tiếp điểm đòi chính xác hơn mức cơ khí dẫn được. Ba, khi không có lực hoặc không có độ tuân thủ để dẫn: côn hoạt động nhờ robot tiếp tục đẩy và lực phản từ mặt vát đẩy nó vào chỗ, nên cần khối lượng, ma sát bánh đủ, và một chỗ nào đó trong chuỗi có thể nhường — lò xo, cao su, cơ cấu nổi. Đề xuất đáng đưa ngược lại cho kỹ sư cơ khí: thiết kế cơ cấu *tự căn góc* (đôi chốt côn, hoặc mặt phẳng tựa) thay vì chỉ dẫn vị trí, vì như vậy nó nối được cả dòng ngân sách đang thiếu.

NÓI LẠI THẬT ĐƠN GIẢN

Có một câu hỏi nên hỏi trước khi bắt tay vào làm cho hệ thống chính xác hơn: *robot có thật sự cần biết nó đang ở đâu không?*

Nghĩ về việc bạn tra chìa vào ổ khoá nhà mình trong bóng tối. Bạn không đo khoảng cách, không tính toạ độ. Bạn làm đúng cái động tác bạn đã làm hàng nghìn lần, và tay bạn biết. Nếu ai đó hỏi cái ổ khoá cách mép cửa bao nhiêu xăng-ti-mét thì bạn chịu — và điều đó chẳng ảnh hưởng gì tới việc bạn mở được cửa.

Robot cập dock cũng vậy. Nó không cần biết toạ độ. Nó cần về đúng chỗ nó đã từng ở. Và hai việc ấy khác nhau nhiều hơn về ngoài.

Đây là chỗ hay: nếu ta đưa robot tới đúng vị trí cập chuẩn một lần — đẩy bằng tay cũng được — rồi chụp lại xem camera nhìn thấy gì, thì về sau việc của robot chỉ là lái sao cho nhìn thấy đúng như vậy. Và bây giờ mọi sai lệch cố định của hệ thống — ống kính hơi lệch, tấm marker hơi cong, camera gấn hơi nghiêng — đều tự biến mất. Vì sao? Vì chúng làm hình ảnh lệch đi đúng như vậy ở *cả hai lần*: lần chụp mẫu và lần đang cập. Hai cái lệch giống nhau thì trừ nhau ra không.

Bạn không cần biết chúng là gì. Bạn không cần đo chúng. Bạn chỉ cần chúng dừng thay đổi.

Cái “dừng thay đổi” ấy là chỗ duy nhất phải cẩn thận. Nếu ống kính nóng lên và giãn ra, nếu tấm marker bị va, nếu ai đó vặn vòng lấy nét — thì hình ảnh mẫu không còn khớp nữa. Cách phòng đơn giản: dán keo vào những gì vặn được, và thỉnh thoảng chụp lại ảnh mẫu.

Còn một mẹo thứ hai, đơn giản đến mức dễ bỏ qua: *dừng lại trước khi nhìn*. Khi robot đang chạy, ảnh bị nhoè và — với nhiều loại camera — còn bị xé nữa, vì chúng quét ảnh từ trên xuống chứ không chụp cả tấm cùng lúc. Cả hai chuyện đó biến mất nếu robot đứng yên. Dừng một phần ba giây, chụp ba chục tấm, lấy trung bình, rồi đi tiếp. Không tốn tiền, không tốn code.

Và mẹo thứ ba, mẹo mà một người làm thị giác hay ngại nói ra: *hỏi anh cơ khí*. Nếu miệng dock được vát một cái phễu rộng một phân rưỡi, thì robot chỉ cần vào đúng trong phạm vi một phân rưỡi là cái phễu tự đẩy nó vào chỗ. Bài toán năm milimét thành bài toán mười lăm milimét, và cái phễu ấy tốn gần như không có gì. Đây không phải thua cuộc — đây là chọn đúng chỗ để tiêu tiền.

Chỗ không rút gọn được: cái mẹo “chụp lại ảnh mẫu” chỉ hoạt động tốt khi robot đã tới gần chỗ nó chụp mẫu. Càng xa thì hiệu quả càng kém, và kém theo một kiểu trơn tru chứ không có ranh giới rõ. Xa bao nhiêu thì bắt đầu kém — cái đó phải đo trên chính hệ của mình.

Một câu: *Dừng dạy robot biết nó ở đâu; dạy nó nhận ra chỗ nó đã từng đứng.*

13.11 Điều gì chứng minh cách hiểu này sai

Mệnh đề chính — *teach-by-showing* triệt tiêu sai số hệ thống và biến bài toán *accuracy* thành bài toán *repeatability* — bị bác bỏ nếu mini project cho thấy độ lặp lại với hiệu chuẩn cố ý làm hỏng không tốt hơn đáng kể so với độ chính xác với cùng hiệu chuẩn ấy. Tôi dẫn mệnh đề bằng lập luận đại số ở mục 3B (cửa a) và đối chiếu với luận điểm tương đương về IBVS trong [60] (cửa c một phần). Nó *chưa* qua cửa (b), và đây là nợ kiểm chứng lớn nhất của chương — cũng là mệnh đề có hậu quả kiến trúc lớn nhất trong cả cây, nên nên kiểm sớm nhất có thể.

Mệnh đề thứ hai — *triệt tiêu chỉ ở bậc nhất, nên hiệu quả giảm khi xa tư thế dạy* — bị bác bỏ nếu giai đoạn bốn của mini project cho thấy độ lặp lại không xấu đi khi xuất phát từ các vị trí ban đầu xa dần. Nếu vậy thì lời khuyên ghép *teach-by-showing* với *coarse-to-fine* là thừa, và mẹo dùng được rộng hơn tôi nghĩ.

Mệnh đề thứ ba — *côn dẫn hướng sửa vị trí tốt hơn sửa góc* — là suy luận hình học của tôi và nó bị bác bỏ nếu một thí nghiệm cơ khí cho thấy một cơ cấu côn thông thường cũng dẫn được sai lệch góc tới cùng mức. Điều đáng nói: nếu nó bị bác bỏ thì đó là *tin tốt*, vì khi ấy cơ khí nói được cả dòng ngân sách đang thiếu.

Cuối cùng, một mệnh đề mà chương này ngầm giả định và đáng nêu ra: *bài toán của bạn thật sự là lặp lại một vị trí*. Nếu robot phải cập vào nhiều dock khác nhau, hoặc dock có thể bị dịch chuyển, hoặc phải cập vào một vị trí được chỉ định động, thì *teach-by-showing* không áp dụng được và bạn quay về bài toán *accuracy* đầy đủ — tức quay về Phần A và B/12 với mọi yêu cầu khắt khe của chúng. Chương này không giấu điều đó, nhưng nó đáng được nói thành lời.

13.12 Kết nối

Chương này là chỗ toàn bộ cây trả lại giá trị thực dụng, và nó có một tính chất đặc biệt: nó *làm nhẹ* các chương khác thay vì chồng thêm yêu cầu.

Nói lên trên. A/05-*lan-truyen-sai-so-pixel-sang-pose* mục 8 cấp toàn bộ nền lý thuyết cho mục 3: phân loại nguồn ngẫu nhiên và hệ thống là điều kiện để hiểu vì sao *teach-by-showing* hoạt động, và cũng là điều kiện để hiểu giới hạn của nó. A/06-*hieung-ung-vat-ly-va-quang-hoc* mục 7 cấp cơ sở vật lý cho mục 4. A/04-*pose-ambiguity-target-phang* là chương mà IBVS ở mục 2 làm cho không liên quan — một mối quan hệ hiếm trong cây: một chương làm một chương khác không cần thiết.

Nói ngang trong Phần B, và đây là phần quan trọng nhất về mặt tổ chức dự án. B/12-*calibration-cho-docking* là chương mà chương này *nói lỏng* nhiều nhất: nếu dùng *teach-by-showing* thì yêu cầu về độ chính xác tuyệt đối của *intrinsic* và *hand-eye* giảm mạnh, vì chúng là nguồn hệ thống và chúng *triệt tiêu*. Điều còn lại cần thiết là *tính ổn định* của hiệu chuẩn, không phải độ đúng của nó — một yêu cầu dễ hơn nhiều. B/09-*multi-marker-constellation* cũng được nói: sai số *constellation* là nguồn hệ thống. Nhưng — và đây là chỗ tinh tế — nó chỉ được nói cho phần *đo*, không cho phần *hình học*: *baseline* và tính không đồng phẳng vẫn cần thiết nguyên vẹn, vì chúng quyết định σ hiệu dụng chứ không phải *bias*. B/11-*uoc-luong-theo-thoi-gian* trở nên *quan trọng hơn* sau chương này, vì khi các nguồn hệ thống đã bị *triệt tiêu* thì các nguồn ngẫu nhiên trở thành phần còn lại duy nhất, và chúng chính là thứ mà trung bình theo thời gian xử lý.

Nói xuống Phần C. C/15-*ngan-sach-sai-so* phải có *hai* bảng ngân sách chứ không một: một cho chế độ *accuracy* tuyệt đối, một cho chế độ *teach-by-showing*. Chênh lệch giữa hai bảng ấy là giá trị định lượng của chương này, và nó thường là một hệ số lớn. C/16-*danh-gia-va-nghiem-thu* nhận từ mục 3 một yêu cầu phương pháp luận quan trọng: phải đo và báo cáo *riêng* *accuracy* và *repeatability* [3], vì gộp chúng lại thì không ai biết hệ thống thật sự làm được gì.

Câu hỏi tự kiểm (trả lời không nhìn bài)

1. Vì sao độ lặp lại là một yêu cầu dễ hơn độ chính xác một bậc, và nguồn sai số nào tạo ra sự chênh lệch ấy?
2. Dẫn lại trong ba dòng vì sao teach-by-showing đưa robot tới đúng vị trí thật, và chỉ ra chính xác giả thiết nào về hàm sai lệch g được dùng.
3. Nêu bốn chỗ mà triệt tiêu không hoàn toàn, và với mỗi chỗ nêu một biện pháp giảm nhẹ.
4. IBVS bền với sai số hiệu chuẩn. Chỉ ra chính xác chỗ mà K sai đi vào luật điều khiển, và vì sao chỗ ấy không ảnh hưởng tới điểm dừng.
5. Đi thẳng trục tốt cho điều khiển nhưng xấu cho đo góc. Nêu xung đột chính xác, và nêu giải pháp giải quyết được cả hai mà không hy sinh bên nào.
6. Stop-and-go có hai lợi ích. Nêu cả hai, nói cái nào lớn hơn và vì sao, và nêu một chi tiết thực hành hay bị quên khi cài đặt nó.
7. Côn dẫn hướng nối được dòng nào của ngân sách và không nối được dòng nào? Nêu một thay đổi thiết kế cơ khí giải quyết được cả dòng còn lại.

Điều kiện quan sát và ánh sáng (Observation Conditions)

PHỤ THUỘC A/06-hieu-ung-vat-ly-va-quang-hoc (mục 6, vật liệu và bề mặt marker; và mục 7, phân loại hiệu ứng theo vận tốc); B/07-detection-pipeline (mục 6, ngưỡng hoá là bước hồng khi tương phản kém — và nó hồng theo kiểu tag biến mất hoàn toàn).

TÓM TẮT

Chương này khép Phần B bằng một chủ đề khác bản chất với bảy chương trước: nó không bàn cách làm σ nhỏ hơn mà bàn cách làm σ ổn định hơn. Sự phân biệt ấy quan trọng vì hai lý do. Thứ nhất, một hệ có σ tốt trong điều kiện tốt và tệ trong điều kiện xấu thì ngân sách phải lập theo trường hợp xấu. Thứ hai, và quan trọng hơn: teach-by-showing (B/13 mục 3) triệt tiêu sai số hệ thống với điều kiện chúng không đổi, nên mọi thứ làm điều kiện quan sát thay đổi đều phá mero ấy. Năm chủ đề: chống loá, chiếu sáng chủ động, khoá mọi tham số tự động, chọn cảm biến, và chống báo sai. Nếu chỉ nhớ một điều: *khoá mọi thứ khoá được* — exposure, gain, cân bằng trắng, nét — vì một hệ đo lường không được phép có tham số tự đổi.

KIẾN THỨC CẦN CÓ TRƯỚC

- **tương phản (contrast)** — chênh lệch độ sáng giữa vùng đen và vùng trắng của tag. Nó quyết định bước ngưỡng hoá có thành công không, và nó quyết định độ lớn gradient tại cạnh — tức quyết định σ .
- **phơi sáng tự động (auto exposure)** — cơ chế camera tự điều chỉnh thời gian phơi sáng theo độ sáng cảnh. Tiện cho chụp ảnh, tai hại cho đo lường, vì nó làm điều kiện đo thay đổi theo những gì trong khung hình.
- **loá (glare)** — vùng sáng chói do phản xạ gương của nguồn sáng trên bề mặt marker. Nó “ăn” mất cạnh trong vùng bị trùm và gây thiên lệch phụ thuộc góc chiếu sáng.
- **retroreflective** — vật liệu phản xạ ánh sáng ngược về phía nguồn. Với đèn gắn cạnh camera, nó cho tương phản rất cao gần như độc lập với ánh sáng môi trường.
- **bộ lọc bandpass** — kính lọc chỉ cho qua một dải bước sóng hẹp. Ghép với đèn chiếu sáng cùng bước sóng, nó loại gần hết ánh sáng môi trường.
- **blooming** — hiện tượng điện tích tràn từ pixel bão hoà sang pixel lân cận, làm vùng sáng “nở” ra và làm cạnh dịch chỗ.
- **whitelist ID** — danh sách các ID hợp lệ mà hệ thống chấp nhận. Mọi ID khác bị loại ngay, bất kể mã có hợp lệ về mặt Hamming.

CÂU HỎI MỞ ĐẦU

- Q1. Hệ của bạn chạy hoàn hảo trong phòng thí nghiệm và thất thường trong nhà kho. Nếu ba cơ chế hoàn toàn khác nhau, và nêu biện pháp duy nhất giải quyết được cả ba cùng lúc.
- Q2. Phơi sáng tự động là tính năng mà mọi camera đều có và mọi người đều bật. Vì sao nó là một quyết định sai trong hệ đo lường, và hậu quả cụ thể là gì?

- Q3.** Một vệt loá nằm trùm lên một cạnh của tag. Vì sao hậu quả là *thiên lệch* chứ không phải nhiễu, và vì sao teach-by-showing không cứu được?
- Q4.** Chiều sáng hồng ngoại chủ động cộng bộ lọc bandpass là giải pháp mạnh nhất trong chương. Lợi ích chính của nó là gì — và câu trả lời *không* phải “ánh sáng hơn”?
- Q5.** Chống báo sai có hai tầng biện pháp. Nếu cả hai, và nói vì sao tầng thứ hai gần như miễn phí trong bài toán docking mà lại không dùng được trong bài toán tổng quát.

14.1 Đặt vấn đề

Bảy chương trước của Phần B đều bàn cách làm sai số *nhỏ hơn*. Chương này bàn một thứ khác: cách làm sai số *ổn định hơn*.

Sự phân biệt ấy có vẻ thứ yếu và nó không phải. Hai lý do.

Lý do thứ nhất, về cách lập ngân sách. Nếu σ của bạn là 0,1 px trong điều kiện tốt và 0,6 px khi có nắng chiếu xiên qua cửa sổ, thì ngân sách ở **C/15** phải lập theo con số nào? Câu trả lời là 0,6 — trường hợp xấu — vì một hệ thống chỉ đạt yêu cầu 80% thời gian là một hệ thống không đạt yêu cầu. Nói cách khác: *giảm sự biến thiên của σ có giá trị ngang với giảm giá trị trung bình của nó*, và nó thường rẻ hơn.

Lý do thứ hai, và nó quan trọng hơn: teach-by-showing của **B/13** mục 3 triệt tiêu sai số hệ thống với điều kiện chúng không đổi giữa lần dạy và lần chạy. Mọi thứ trong chương này làm điều kiện quan sát thay đổi — ánh sáng khác, phơi sáng khác, vệt loá ở chỗ khác — đều phá đúng điều kiện ấy. Một hệ dùng teach-by-showing mà không kiểm soát điều kiện quan sát là một hệ đã bỏ đi lợi ích lớn nhất mà nó vừa mua được.

Bài toán gốc thuộc về kỹ thuật thị giác công nghiệp hơn là robot học, và ngành ấy đã có câu trả lời từ lâu: trong một hệ thống đo lường bằng ảnh, *chiếu sáng là một thành phần của thiết bị đo, không phải một điều kiện môi trường*. Người ta thiết kế chiếu sáng cùng với camera, khoá mọi tham số, và che chắn ánh sáng ngoài. Robot di động không che chắn được, nhưng phần lớn các nguyên tắc vẫn áp dụng.

Cách hiển nhiên — và cách gần như mọi dự án đi — là dùng ánh sáng có sẵn và bật phơi sáng tự động. Nó hoạt động trong phòng thí nghiệm, nơi ánh sáng đều và không đổi, và nó bắt đầu thất thường khi triển khai. Chẩn đoán thì khó vì triệu chứng không tái lập được: hệ chạy tốt buổi sáng và tệ buổi chiều, tốt ở lối này và tệ ở lối kia. Đây là câu trả lời cho một nửa của Q1, và ba cơ chế cụ thể nằm ở mục 2.

Có một điểm cần nói rõ về vị trí của chương trong Phần B: nó có *đòn bẩy thấp về giá trị trung bình của σ nhưng đòn bẩy cao về phương sai của σ* . Với một hệ đã làm tốt **B/09** và **B/13**, chương này thường là chương quyết định xem hệ có chạy được ngoài phòng thí nghiệm hay không — và đó là một loại giá trị khác, không xuất hiện trong bảng ngân sách nhưng quyết định dự án.

TRỰC GIÁC

Hình dung sự khác nhau giữa một cái thước và một cái thước co giãn theo nhiệt độ. Cái thước thường có thể hơi sai — chẳng hạn nó dài hơn chuẩn 0,5% — nhưng nếu nó luôn sai đúng như vậy thì bạn sống được với nó: mọi phép đo lệch cùng một tỉ lệ, và nếu bạn chỉ cần so sánh hai vật thì sai số triệt tiêu. Cái thước co giãn thì khác hẳn. Sáng lạnh nó ngắn, chiều nóng nó dài. Bây giờ hai phép đo ở hai thời điểm không so được với nhau nữa, và không có cách nào hiệu chỉnh trừ khi bạn cũng đo cả nhiệt độ.

Camera với phơi sáng tự động là cái thứ hai. Khi có thêm một vật sáng vào khung hình, camera giảm phơi sáng, ảnh của tag tối đi, gradient tại cạnh yếu đi, và vị trí cạnh được xác định kém chính xác hơn — và có thể lệch đi một chút theo một hướng. Cái “vật sáng vào khung hình” ấy có thể là một cánh cửa mở, một cái xe đi qua, hay đơn giản là mặt trời di chuyển.

Ánh sáng môi trường là cùng một vấn đề ở tầng khác: nó đổi theo giờ, theo thời tiết, theo việc ai bật tắt đèn.

Và đây là lối thoát, và nó là ý tưởng trung tâm của chương: *mang theo ánh sáng của riêng mình, và bịt mắt camera với mọi ánh sáng khác*. Gắn một đèn hồng ngoại cạnh camera, lắp một kính lọc chỉ cho qua đúng bước sóng ấy. Bây giờ camera gần như không nhìn thấy ánh sáng mặt trời, không nhìn thấy đèn trần, không nhìn thấy gì ngoài cái tag được chiếu bởi chính đèn của nó. Buổi sáng hay buổi chiều, trong nhà hay gần cửa sổ — ảnh trông giống hệt nhau.

Cái ta mua được không phải ánh sáng hơn. Cái ta mua được là *ảnh giống nhau*.

14.2 Ba cơ chế làm điều kiện thay đổi

Mục này trả lời Q1.

Cơ chế một: mức sáng tổng thể đổi. Ánh sáng môi trường thay đổi theo giờ, theo thời tiết, theo vị trí trong nhà kho. Với phơi sáng cố định, tag ở vùng tối cho ảnh tối, gradient yếu, σ lớn — hoặc tệ hơn, ngưỡng hoá thất bại và tag biến mất hoàn toàn (B/07 mục 6). Với phơi sáng tự động, mức sáng của tag được giữ nhưng *thời gian phơi sáng đổi*, kéo theo motion blur đổi (A/06 mục 3) — bạn đổi một vấn đề lấy một vấn đề khác.

Cơ chế hai: hướng chiếu sáng đổi. Đây là cơ chế tinh vi nhất và nó gây hậu quả tệ nhất. Ánh sáng chiếu xiên tạo bóng đổ và tạo phản xạ gương; vị trí của vệt loá phụ thuộc vào góc giữa nguồn sáng, bề mặt marker, và camera. Khi robot di chuyển, góc ấy đổi, nên vệt loá *di chuyển trên tag* — và nó gây thiên lệch phụ thuộc tư thế, loại nguồn sai số khó chịu nhất.

Cơ chế ba: tương phản của chính marker đổi. Bụi, xước, phai màu, ẩm. Nó đổi chậm nên nó không gây triệu chứng thất thường, nhưng nó làm hệ thống xuống cấp dần — và nếu bạn dùng teach-by-showing với một ảnh mẫu chụp từ sáu tháng trước, nó phá phép triệt tiêu.

Biện pháp duy nhất giải quyết được cả ba — phần thứ hai của Q1 — là *chiếu sáng chủ động cộng lọc bandpass*, mục 4. Nó áp đảo cơ chế một (ánh sáng của bạn lớn hơn ánh sáng môi trường trong dải bước sóng được lọc), nó cố định cơ chế hai (nguồn sáng gắn với camera nên góc chiếu không đổi theo vị trí robot), và nó giảm nhẹ cơ chế ba (tương phản cao hơn nên bụi ít ảnh hưởng tương đối hơn).

14.3 Chống loá

Mục này trả lời Q3.

Loá là phản xạ gương: ánh sáng từ nguồn đập vào bề mặt marker và bật thẳng vào camera, cho một vùng sáng chói. Nếu vùng ấy trùm lên một cạnh tag, ba chuyện xảy ra: pixel bão hoà nên gradient tại cạnh biến mất; blooming làm điện tích tràn sang pixel lân cận nên cạnh “nở” ra; và phép khớp đường trong B/07 mục 2 nhận một profile bất đối xứng nên nó khớp lệch.

Hậu quả là *thiên lệch* chứ không phải nhiễu — đây là phần một của Q3 — vì ở một tư thế cho trước, vệt loá luôn ở cùng chỗ và luôn đầy cạnh theo cùng hướng. Lấy trung bình một nghìn khung không giúp gì (B/11 mục 6).

Và teach-by-showing không cứu được — phần hai của Q3 — vì vị trí vệt loá *phụ thuộc tư thế*. Ở tư thế dạy, vệt loá ở một chỗ; ở tư thế hiện tại cách đó vài xăng-ti-mét, nó ở chỗ khác. Hai thiên lệch khác nhau nên chúng không trừ đi nhau. Đây là một ví dụ cụ thể của giới hạn “*g phụ thuộc tư thế*” ở B/13 mục 3C.

Ba biện pháp.

Một, vật liệu nhám (matte). Bề mặt nhám tán xạ ánh sáng theo mọi hướng thay vì phản xạ gương, nên không có vệt chói tập trung. Đây là biện pháp rẻ nhất và nên là mặc định. Tránh laminate bóng, tránh in trên vật liệu trắng.

Hai, bố trí tránh đường phản xạ gương. Nếu nguồn sáng cố định (đèn trần) và đường tiếp cận cố định, có thể tính trước xem ở tư thế nào thì có phản xạ gương và nghiêng tag để tránh. Với nệm nghiêng đã có từ B/09 mục 4, bạn có sẵn một bậc tự do để làm việc này — và đây là một lý do nữa để chọn góc nghiêng có cân nhắc thay vì tùy tiện.

Ba, chiếu sáng chủ động đặt lệch. Nếu đèn gắn cạnh camera nhưng lệch đi vài chục xăng-ti-mét, đường phản xạ gương từ đèn không đi vào camera với hầu hết tư thế. Đây là một chi tiết bố trí đơn giản và hiệu quả.

Một lưu ý về retroreflective: nó cho tương phản rất cao nhưng nó *cố ý* phản xạ ngược về nguồn, nên với đèn gắn ngay cạnh camera thì nó luôn cho vùng sáng mạnh — và vùng sáng ấy dễ bão hoà và dễ blooming. Nó là một đánh đổi: tương phản tuyệt vời, rủi ro blooming cao. Nếu dùng, phải kiểm soát phơi sáng cẩn thận và kiểm rằng không có pixel nào bão hoà.

14.4 Chiếu sáng chủ động

Biện pháp mạnh nhất trong chương, và mục này trả lời Q4.

Ý tưởng: gắn một nguồn sáng lên robot, cạnh camera, và lắp trên camera một bộ lọc chỉ cho qua bước sóng của nguồn ấy. Thường dùng LED hồng ngoại gần (khoảng 850 nm) vì mắt người không thấy nó — nên nó không gây khó chịu cho người làm việc quanh robot — và vì cảm biến silicon vẫn nhạy ở dải ấy.

Cơ chế: bộ lọc chặn gần hết ánh sáng môi trường (mặt trời, đèn trần) vì chúng trải rộng trên phổ, trong khi nguồn của bạn tập trung toàn bộ năng lượng vào đúng dải được cho qua. Tỷ số tín hiệu trên nhiễu môi trường tăng mạnh, và ảnh trở nên gần như độc lập với ánh sáng xung quanh.

Đây là câu trả lời cho Q4, và điều quan trọng là lợi ích chính *không* phải “ánh sáng hơn” — bạn có thể làm ánh sáng hơn bằng cách tăng gain, và điều đó không giải quyết gì. Lợi ích chính là *ảnh giống nhau ở mọi điều kiện*. Cụ thể:

σ trở nên ổn định. Cùng một tag ở cùng một tư thế cho cùng một ảnh, buổi sáng hay buổi chiều, gần cửa sổ hay trong góc tối.

Điều kiện teach-by-showing được thoả. Hàm sai lệch *g* của B/13 mục 3 không đổi theo ánh sáng nữa, nên phép triệt tiêu hoạt động đúng như lý thuyết.

Phơi sáng cố định được. Vì mức sáng không đổi, bạn khoá được exposure và gain — và mục 5 giải thích vì sao điều đó quan trọng.

Vệt loá cố định. Vì nguồn sáng gắn với camera, góc giữa nguồn, marker và camera chỉ phụ thuộc tư thế tương đối chứ không phụ thuộc vị trí trong phòng. Loá vẫn có nhưng nó trở thành hàm

của tư thế — tức nó có thể được tính trước, tránh bằng bố trí, và một phần triết tiêu trong teach-by-showing.

Ghi chú trung thực: tôi phát biểu các lợi ích trên ở dạng *định tính* và không kèm con số về mức cải thiện. Lập luận dựa trên tỉ số tín hiệu trên nhiễu môi trường (cửa a); tôi không có số đo so sánh được (cửa b và c chưa qua). Mức cải thiện thực tế phụ thuộc mạnh vào công suất nguồn, độ hẹp của bộ lọc, và mức ánh sáng môi trường — và nó phải đo cho từng lắp đặt.

Chi phí và hạn chế: nguồn sáng tiêu thụ điện; bộ lọc làm giảm lượng ánh sáng tới cảm biến nên cần phơi sáng dài hơn hoặc nguồn mạnh hơn; và trong ánh nắng trực tiếp mạnh, ngay cả sau bộ lọc thì nền vẫn có thể sáng đáng kể. Một biến thể cho trường hợp khắc nghiệt là *marker backlit* — tag phát sáng từ phía sau — cho tương phản gần như tuyệt đối, nhưng nó đòi nguồn điện tại dock.

14.5 Khoá mọi thứ khoá được

Mục ngắn nhất và có lẽ có giá trị trên mỗi dòng cao nhất trong chương. Nó trả lời Q2.

Danh sách những thứ phải khoá: thời gian phơi sáng, gain, cân bằng trắng, gamma, mọi chế độ “tăng cường” của camera, và tiêu điểm (A/01 mục 8).

Lý do, và nó là một nguyên tắc chứ không phải một lời khuyên: *một hệ đo lường không được phép có tham số tự đổi*. Mọi tham số tự động là một hàm của cảnh, và cảnh thì đổi. Nên tham số tự động biến một sai số cố định (chấp nhận được, và triết tiêu được bằng teach-by-showing) thành một sai số thay đổi (không chấp nhận được, và không triết tiêu được).

Hậu quả cụ thể của phơi sáng tự động — phần hai của Q2 — theo ba đường. *Một*, thời gian phơi sáng đổi nên motion blur đổi (A/06 mục 3), và blur là thiên lệch tỉ lệ với vận tốc. *Hai*, mức sáng của tag đổi nên độ lớn gradient tại cạnh đổi, nên σ đổi. *Ba*, và đây là đường tệ nhất: phơi sáng tự động phản ứng với *toàn bộ khung hình*, nên một vật sáng đi vào khung — một cánh cửa mở, một chiếc xe, một người mặc áo trắng — làm điều kiện đo của tag thay đổi dù bản thân tag không đổi gì. Sai số của bạn bây giờ phụ thuộc vào những thứ hoàn toàn không liên quan.

Về việc chọn giá trị cố định: chọn phơi sáng *ngắn nhất* mà vẫn cho tương phản đủ, vì phơi sáng ngắn giảm motion blur. Điều này đẩy bạn về phía cần nhiều ánh sáng hơn — tức về phía mục 4. Hai mục ấy là một cặp: chiếu sáng chủ động cho phép phơi sáng ngắn, phơi sáng ngắn cho phép đo trong khi di chuyển.

Về gain: giữ thấp. Gain cao khuếch đại cả tín hiệu lẫn nhiễu đọc, nên nó làm ảnh sáng hơn mà không làm σ tốt hơn — và nó thường làm σ tệ đi.

CẦN NHỚ

Một hệ đo lường không được phép có tham số tự đổi.

Khoá: exposure, gain, cân bằng trắng, gamma, mọi chế độ tăng cường, và tiêu điểm. Với tiêu điểm thì khoá bằng phần mềm chưa đủ — dán keo vào vòng lấy nét, vì một cú va chạm nhẹ cũng làm nó xoay.

Chọn exposure ngắn nhất còn đủ tương phản (giảm blur), và gain thấp nhất (giảm nhiễu). Hai yêu cầu ấy xung đột nhau trừ khi bạn có thêm ánh sáng — và đó chính là lập luận cho chiếu sáng chủ động ở mục 4.

Và ghi lại các giá trị đã khoá vào tệp cấu hình cùng với ngày. Một hệ thống bị ai đó vô tình đổi exposure rồi không ai nhớ giá trị cũ là một hệ thống phải hiệu chuẩn lại từ đầu.

14.6 Chọn cảm biến

Ngắn, vì A/06 đã dẫn phần lớn lập luận.

Global shutter, gần như bắt buộc nếu đo khi di chuyển. A/06 mục 2 giải thích: rolling shutter làm tag bị xé chứ không chỉ mờ, và đó là một chế độ hỏng im lặng. Nếu chiến lược là stop-and-go thuần (B/13 mục 4) thì rolling shutter chấp nhận được và bạn tiết kiệm được tiền — nhưng phải đảm bảo rung đã tắt hẳn trước khi chụp.

Đơn sắc thay vì màu, nếu có lựa chọn. Cảm biến màu dùng ma trận lọc Bayer, nên mỗi pixel chỉ đo một kênh màu và các kênh còn lại được nội suy — tức độ phân giải không gian thực tế thấp hơn số pixel danh nghĩa, và phép nội suy đưa vào một mức làm mờ. Với tag đen trắng thì thông tin màu vô dụng, nên cảm biến đơn sắc cho nhiều hơn với cùng số pixel. Nó cũng nhạy hơn (không mất ánh sáng qua bộ lọc màu) và tương thích tốt hơn với chiếu sáng hồng ngoại ở mục 4.

Ảnh thô, không nén. Nén JPEG đưa vào các hiện vật quanh cạnh — đúng chỗ mà chúng ta đang đo. Nếu camera chỉ cho ra JPEG thì đặt chất lượng cao nhất và biết rằng bạn đang mất một phần σ .

Độ phân giải: chỉ tăng khi quang học chưa phải nút thắt. A/06 mục 5B cho phép thử: chụp lại với nhiều sáng hơn và phơi sáng dài hơn; nếu σ không cải thiện thì quang học là nút thắt và thêm pixel vô ích.

14.7 Chống báo sai và xử lý che khuất

Mục này trả lời Q5.

A. Hai tầng chống báo sai

Tầng một, ở trong detector: giảm số lỗi bit cho phép sửa (B/07 mục 5C). Đánh đổi: tầm phát hiện giảm một chút.

Tầng hai, ở tầng ứng dụng: whitelist ID và kiểm tính hợp lý. Đây là tầng gần như miễn phí trong bài toán docking — và phần hai của Q5 là vì sao nó không dùng được trong bài toán tổng quát.

Lý do: trong docking, bạn *biết trước* constellation gồm đúng những ID nào và chúng ở đâu so với nhau. Nên bạn loại được ngay mọi ID ngoài danh sách, và bạn kiểm được tính hợp lý của mỗi phát hiện: tag này có kích thước hợp lý ở khoảng cách này không; pose của nó có nhất quán với các tag khác không (B/09 mục 7); khoảng cách và góc có nằm trong dải vận hành không. Bốn phép kiểm ấy tốn vài dòng và chúng loại gần hết báo sai.

Trong bài toán tổng quát — chẳng hạn một robot đi trong môi trường có tag ở nhiều nơi chưa biết — không có whitelist, không biết trước quan hệ giữa các tag, nên tầng hai gần như không dùng được và toàn bộ gánh nặng dồn lên tầng một. Đó là lý do các họ tag đầu tư nặng vào mã hoá, và cũng là lý do bài toán docking dễ hơn nhiều về mặt an toàn.

B. Che khuất và bắn

Che khuất một phần: detector thường vẫn phát hiện được nếu đủ ô bit còn thấy, nhưng các góc nằm trong vùng bị che sẽ sai. Biện pháp: cổng chi-square ở mức từng góc (A/03 mục 7) loại chúng. Và quan trọng hơn, B/10 mục 1 cảnh báo rằng che khuất có thể làm các tag còn lại tình cờ đồng phẳng — nên phải theo dõi cả ρ .

Bắn và mài mòn: chúng làm giảm tương phản và có thể gây thiên lệch cục bộ. Biện pháp phòng: chỉ báo liên tục ở B/12 mục 6, theo dõi theo tuần. Biện pháp chữa: lau tag định kỳ — nghe tầm thường nhưng nó là một mục trong quy trình bảo trì, và bỏ qua nó thì hệ thống xuống cấp dần một cách khó chẩn đoán.

Một biện pháp thiết kế đáng nêu: *dư thừa*. Nếu constellation có sáu tag và hệ vẫn hoạt động tốt với bốn, thì che khuất một hoặc hai tag không gây sự cố. Dư thừa tốn ít — vài tấm tag nữa — và nó chuyển một chế độ hỏng thành một chế độ suy giảm. Điều kiện: tập con còn lại phải *vẫn* là một constellation tốt (có baseline, không đồng phẳng), và đó là một ràng buộc thật khi thiết kế bố trí ở B/09.

14.8 Giới hạn và chế độ hỏng

Chương này không giảm σ trung bình nhiều. Nó giảm phương sai của σ . Nếu hệ của bạn đã ở điều kiện kiểm soát tốt thì nó trả lại ít.

Chiếu sáng chủ động không thắng được năng trực tiếp. Trong ánh nắng mạnh chiếu thẳng vào marker, ngay cả sau bộ lọc bandpass thì nền vẫn có thể sáng đáng kể. Với dock đặt gần cửa sổ lớn, phải tính tới chuyện che chắn vật lý.

Nguồn hồng ngoại có thể nhiễu với cảm biến khác. Một số cảm biến chiều sâu dùng hồng ngoại và chúng có thể can nhiễu lẫn nhau. Kiểm nếu robot có nhiều cảm biến.

Các lợi ích được phát biểu định tính. Đã ghi ở mục 4: tôi không có số đo cho mức cải thiện của chiếu sáng chủ động. Nó phải đo cho từng lắp đặt.

Khoá tham số không chống được trôi phần cứng. Khoá exposure không ngăn cảm biến trôi theo nhiệt độ, và khoá nét bằng phần mềm không ngăn vòng lấy nét bị xoay do va chạm. Cần cả biện pháp vật lý và chỉ báo giám sát.

14.9 Thực hành

Năm việc, theo thứ tự chi phí.

Một, khoá mọi tham số tự động ngay hôm nay. Không tốn gì và nó là điều kiện tiên quyết cho mọi thứ khác.

Hai, đổi sang vật liệu nhám nếu marker đang bóng. Rẻ, và nó loại một nguồn thiên lệch phụ thuộc tư thế.

Ba, đo σ ở nhiều điều kiện ánh sáng — sáng, tối, có nắng xiên, có đèn bật tắt — thay vì chỉ ở một điều kiện. Con số tệ nhất là con số đưa vào ngân sách.

Bốn, cân nhắc chiếu sáng chủ động cộng bandpass nếu bước ba cho thấy biến thiên lớn. Đây là biện pháp mạnh nhất và nó nên được quyết định dựa trên số đo chứ không dựa trên nguyên tắc.

Năm, bật whitelist ID và ba phép kiểm hợp lý ở mục 7A.

Active recall (che cột phải, tự trả lời, rồi mở ra đối chiếu)

Thuật ngữ / câu hỏi	Trả lời
••• Chương này bàn gì, khác B/08 thế nào	B/08 làm σ nhỏ hơn; chương này làm σ ổn định hơn. Hai lý do quan trọng: ngân sách phải lập theo trường hợp xấu nhất; và teach-by-showing đòi điều kiện không đổi giữa dạy và chạy.

Thuật ngữ / câu hỏi	Trả lời (tiếp)
●●● Ba cơ chế làm điều kiện đổi	Mức sáng tổng thể đổi ($\Rightarrow \sigma$ đổi hoặc tag biến mất); hướng chiếu sáng đổi ($\Rightarrow$ vật loá di chuyển trên tag, thiên lệch phụ thuộc tư thế); và tương phản marker xuống cấp (bụi, xước, phai).
●●● Biện pháp giải quyết cả ba	Chiều sáng chủ động cộng lọc bandpass: áp đảo cơ chế một, cố định cơ chế hai (nguồn gắn với camera nên góc chiếu không đổi theo vị trí robot), giảm nhẹ cơ chế ba.
●●● Lợi ích chính của IR chủ động	Không phải “ánh sáng hơn” — tăng gain cũng làm ảnh sáng hơn mà không giải quyết gì. Lợi ích là <i>ảnh giống nhau ở mọi điều kiện</i> , tức σ ổn định và điều kiện teach-by-showing được thoả.
●●● Vì sao loá gây thiên lệch	Ở một tư thế cho trước, vật loá luôn ở cùng chỗ và luôn đầy cạnh cùng hướng. Ba cơ chế: pixel bão hoà (mất gradient), blooming (cạnh nở), profile bất đối xứng (khớp đường lệch).
●●● Vì sao teach-by-showing không cứu được loá	Vì vị trí vật loá <i>phụ thuộc tư thế</i> : ở tư thế dạy nó ở một chỗ, ở tư thế hiện tại nó ở chỗ khác. Hai thiên lệch khác nhau nên không trừ đi nhau. Ví dụ cụ thể của giới hạn “ <i>g</i> phụ thuộc tư thế” (B/13 mục 3C).
●●● Nguyên tắc về tham số tự động	Một hệ đo lường không được phép có tham số tự đổi. Tham số tự động là hàm của cảnh, và cảnh đổi — nên nó biến sai số <i>cố định</i> (triệt tiêu được) thành sai số <i>thay đổi</i> (không triệt tiêu được).
●●● Ba đường hại của phơi sáng tự động	Thời gian phơi sáng đổi $\Rightarrow$ blur đổi; mức sáng đổi $\Rightarrow$ gradient đổi $\Rightarrow \sigma$ đổi; và nó phản ứng với <i>toàn khung hình</i> nên một vật sáng đi qua làm điều kiện đo của tag đổi dù tag không đổi gì.
●● Chọn giá trị cố định	Exposure <i>ngắn nhất</i> còn đủ tương phản (giảm blur); gain <i>thấp nhất</i> (gain cao khuếch đại cả nhiễu đọc). Hai yêu cầu xung đột trừ khi có thêm ánh sáng — đó là lập luận cho mục 4.
●● Ba biện pháp chống loá	Vật liệu nhám (rẻ nhất, nên mặc định); bố trí tránh đường phản xạ gương (dùng bậc tự do góc nghiêng đã có từ B/09 mục 4); và đặt nguồn sáng lệch khỏi trục camera vài chục cm.
●● Đánh đổi của retroreflective	Tương phản tuyệt vời nhưng nó <i>cố ý</i> phản xạ ngược về nguồn, nên với đèn cạnh camera thì luôn cho vùng sáng mạnh — dễ bão hoà và blooming. Phải kiểm không có pixel nào bão hoà.
●● Chọn cảm biến	Global shutter nếu đo khi di chuyển; <i>đơn sắc</i> thay vì màu (Bayer làm mất độ phân giải thật và đưa vào làm mượt; màu vô dụng với tag đen trắng); ảnh thô không nén; và chỉ tăng độ phân giải khi quang học chưa phải nút thắt.
●●● Hai tầng chống báo sai	Tầng một trong detector: giảm số lỗi bit cho phép sửa. Tầng hai ở ứng dụng: whitelist ID cộng kiểm hợp lý (kích thước, pose, nhất quán liên tag, dài vận hành).
●●● Vì sao tầng hai miễn phí với docking	Vì bạn <i>biết trước</i> constellation gồm ID nào và chúng ở đâu so với nhau. Trong bài toán tổng quát (tag chưa biết trong môi trường) thì không có whitelist, nên toàn bộ gánh nặng dồn lên tầng một.
●● Dư thừa như biện pháp thiết kế	Sáu tag mà hệ vẫn chạy tốt với bốn $\Rightarrow$ che khuất một hai tag thành chế độ <i>suy giảm</i> thay vì <i>hỏng</i> . Điều kiện: tập con còn lại phải <i>vẫn</i> là constellation tốt — ràng buộc thật khi thiết kế ở B/09.

●●● phải thuộc ●● nên nắm ● biết là có

I Kiến thức này đang sống ở đâu (công trình đang dùng chính ý tưởng của chương; chốt 9/2026)

Công trình / hệ thống	Dùng ý tưởng nào của chương	Ghi chú
Chiếu sáng IR + bandpass	Thực hành chuẩn trong thị giác công nghiệp: chiếu sáng là thành phần của thiết bị đo, không phải điều kiện môi trường	Biện pháp mạnh nhất của chương; quyết định dựa trên số đo biến thiên σ
Marker backlit	Tương phản gần như tuyệt đối; đòi nguồn điện tại dock	Cho trường hợp khắc nghiệt nhất, chẳng hạn gần cửa sổ lớn
Camera global shutter đơn sắc	Loại bỏ rolling shutter (<i>A/06</i> mục 2) và Bayer cùng lúc	Lựa chọn mặc định cho hệ đo lường; chi phí chênh nhỏ so với thời gian gỡ lỗi
Whitelist ID + kiểm hợp lý	Vài dòng code, loại gần hết báo sai trong bài toán docking	Tận dụng chính điều làm docking dễ hơn bài toán tổng quát
Vật liệu nhám cho marker [45]	Tránh phản xạ gương; công trình so sánh có bản ảnh hưởng của điều kiện chiếu sáng	Đọc phần điều kiện đo trước khi dùng số của họ

MINI PROJECT • Đo biến thiên của σ theo điều kiện ánh sáng • 2 ngày

Mục tiêu	biết con số σ <i>tệ nhất</i> của hệ bạn thay vì con số trong điều kiện tốt nhất — vì con số tệ nhất mới là con số vào ngân sách.
Dữ liệu	phần cứng thật, tại chính vị trí lắp đặt hoặc một nơi có điều kiện ánh sáng tương tự.
Các bước	chọn một tư thế cố định gần tư thế cập chuẩn; ở tư thế ấy, đo σ theo quy trình của <i>B/07</i> mini project (500 khung, độ lệch chuẩn của tọa độ góc) trong <i>sáu</i> điều kiện: đèn trần bình thường; đèn trần tắt (chỉ ánh sáng tự nhiên); có nắng xiên chiếu vào marker; có đèn pin chiếu từ một bên tạo loá có chủ ý; sáng sớm và chiều muộn nếu có ánh sáng tự nhiên; và — nếu đã có — với chiếu sáng chủ động bật. Với mỗi điều kiện, ghi cả σ lẫn <i>vị trí trung bình</i> của từng góc.
Xong khi	bạn có một bảng sáu hàng. Hai cột quan trọng: σ (nhiều) và độ lệch của vị trí trung bình so với điều kiện chuẩn (thiên lệch). Dự đoán của chương: σ biến thiên vài lần giữa các điều kiện, và — quan trọng hơn — <i>vị trí trung bình</i> cũng dịch, đặc biệt trong điều kiện có loá. Cột thứ hai ấy chính là thứ mà lấy trung bình nhiều khung không chữa được và teach-by-showing cũng không. Kiểm tra chéo bắt buộc: nếu vị trí trung bình <i>không</i> dịch giữa các điều kiện thì bạn đang có một hệ rất tốt về mặt thiên lệch, và cả chương này chỉ còn quan trọng cho phần σ — một kết quả đáng mừng và đáng ghi lại.
Nối với	<i>C/15</i> nhận con số σ tệ nhất; <i>C/16</i> đưa phép đo này vào quy trình nghiệm thu định kỳ.

TRẢ LỜI CÂU HỎI MỞ ĐẦU

- A1. Ba cơ chế: mức sáng tổng thể khác nhau (nhà kho có vùng tối vùng sáng, và ánh sáng đổi theo giờ) làm σ đổi hoặc làm ngưỡng hoá thất bại hẳn; *hướng* chiếu sáng khác nhau tạo vệt loá ở những chỗ khác nhau trên tag, gây thiên lệch phụ thuộc tư thế; và marker trong môi trường thật bị bụi, xước, phai màu nên tương phản giảm dần. Biện pháp duy nhất giải quyết cả ba là *chiếu sáng chủ động cộng bộ lọc bandpass*: nó áp đảo cơ chế một vì nguồn của bạn tập trung năng lượng vào dải được cho qua trong khi ánh sáng môi trường trải rộng trên phổ; nó cố định cơ chế hai vì nguồn gắn với camera nên góc chiếu chỉ phụ thuộc tư thế tương đối chứ không phụ thuộc vị trí trong phòng; và nó giảm nhẹ cơ chế ba vì tương phản cao hơn làm bụi ít ảnh hưởng tương đối hơn.
- A2. Vì một hệ đo lường không được phép có tham số tự đổi: mọi tham số tự động là một hàm của cảnh, và cảnh thì đổi — nên nó biến một sai số *cố định* (chấp nhận được, và triệt tiêu được bằng teach-by-showing) thành một sai số *thay đổi* (không chấp nhận được, không triệt tiêu được). Ba hậu quả cụ thể: thời gian phơi sáng đổi nên motion blur đổi, mà blur là thiên lệch tỉ lệ với vận tốc; mức sáng của tag đổi nên độ lớn gradient tại cạnh đổi nên σ đổi; và — tệ nhất — phơi sáng tự động phản ứng với *toàn bộ khung hình*, nên một vật sáng đi vào khung (cánh cửa mở, một

chiếc xe, một người mặc áo trắng) làm điều kiện đo của tag thay đổi dù bản thân tag không đổi gì. Sai số của bạn bây giờ phụ thuộc vào những thứ hoàn toàn không liên quan tới phép đo.

- A3. Thiên lệch chứ không phải nhiễu vì ở một tư thế cho trước, vết loá luôn ở cùng một chỗ và luôn đẩy cạnh theo cùng một hướng — ba cơ chế cùng tác động: pixel bão hoà làm mất gradient, blooming làm cạnh nở ra, và profile bất đối xứng làm phép khớp đường lệch. Lấy trung bình nghìn khung không giúp gì. Teach-by-showing không cứu được vì vị trí vết loá *phụ thuộc tư thế*: nó là hàm của góc giữa nguồn sáng, bề mặt marker và camera, nên ở tư thế dạy nó nằm ở một chỗ và ở tư thế hiện tại cách đó vài xăng-ti-mét nó nằm ở chỗ khác. Hai thiên lệch khác nhau thì không trừ đi nhau. Đây là một ví dụ rất cụ thể của giới hạn “hàm sai lệch phụ thuộc tư thế” mà B/13 mục 3C nêu ra ở dạng tổng quát.
- A4. Lợi ích chính là *ảnh giống nhau ở mọi điều kiện*, không phải ánh sáng hơn — bạn làm ánh sáng hơn được bằng cách tăng gain, và điều đó chẳng giải quyết gì (thậm chí làm σ tệ hơn vì gain khuếch đại cả nhiễu đọc). Cụ thể, sự “giống nhau” ấy mua cho bạn bốn thứ: σ ổn định nên ngân sách lập được theo một con số thay vì theo trường hợp xấu nhất; điều kiện của teach-by-showing được thoả nên phép triệt tiêu sai số hệ thống hoạt động đúng như lý thuyết; phơi sáng khoá được vì mức sáng không đổi; và vết loá trở thành hàm của tư thế tương đối chứ không của vị trí trong phòng, nên nó tính trước được và một phần triệt tiêu được.
- A5. Tăng một nằm trong detector: giảm số lỗi bit cho phép sửa, đánh đổi bằng tầm phát hiện ngắn hơn một chút. Tăng hai nằm ở tầng ứng dụng: whitelist ID cộng với kiểm tính hợp lý — kích thước tag có hợp lý ở khoảng cách này không, pose có nhất quán với các tag khác không, khoảng cách và góc có trong dải vận hành không. Tăng hai gần như miễn phí trong docking vì bạn *biết trước* constellation gồm đúng những ID nào và chúng ở đâu so với nhau, nên bạn loại được ngay mọi thứ không khớp với hiểu biết ấy. Trong bài toán tổng quát — một robot đi trong môi trường có tag ở những nơi chưa biết — không có whitelist và không biết trước quan hệ giữa các tag, nên tăng hai gần như không dùng được và toàn bộ gánh nặng an toàn dồn lên mã hoá ở tầng một. Đó là lý do các họ tag đầu tư nặng vào khoảng cách Hamming, và cũng là lý do bài toán docking dễ hơn nhiều về mặt an toàn.

NÓI LẠI THẬT ĐƠN GIẢN

Mấy chương trước lo chuyện làm sao đo cho *chính xác*. Chương này lo một chuyện khác: làm sao đo cho *giống nhau*.

Nghe như chuyện nhỏ hơn nhưng không phải. Một cái thước hơi sai nhưng luôn sai đúng như vậy thì vẫn dùng được — mọi phép đo lệch cùng một kiểu, và nếu chỉ cần so hai lần đo với nhau thì cái lệch triệt tiêu. Một cái thước lúc dài lúc ngắn thì vô dụng, dù trung bình nó đúng.

Camera với chế độ tự động chính là cái thước thứ hai. Có ai mở cửa cho nắng vào, camera tự giảm sáng, ảnh của tag tối đi, và phép đo đổi — dù cái tag chẳng đổi gì. Nên việc đầu tiên và rẻ nhất là: *tắt hết chế độ tự động*. Khoá thời gian chụp, khoá độ khuếch đại, khoá cân bằng màu, khoá nét — và với cái nét thì khoá bằng phần mềm chưa đủ, phải dán keo vào vòng xoay, vì một cú va nhẹ cũng làm nó lệch.

Chuyện thứ hai là vết loá. Nếu tấm tag bóng và có đèn chiếu xiên vào, sẽ có một vết sáng chói. Vết ấy ăn mất một đoạn cạnh, và phép đo lệch đi. Tệ ở chỗ vết ấy *di chuyển* khi robot di chuyển, nên cái lệch cũng đổi theo — và mọi mẹo triệt tiêu đều bó tay với thứ thay đổi. Cách chữa rẻ nhất: dùng vật liệu nhám thay vì bóng.

Chuyện thứ ba là cách mạnh nhất, và ý tưởng của nó đơn giản đến bất ngờ: *mang theo ánh sáng của riêng mình, rồi bịt mắt camera với mọi ánh sáng khác*. Gắn một cái đèn hồng ngoại cạnh camera, lắp một tấm kính lọc chỉ cho qua đúng màu của cái đèn ấy. Bây giờ camera gần như không thấy mặt trời, không thấy đèn trần, chỉ thấy cái tag được chiếu bởi chính đèn của nó. Sáng hay chiều, gần cửa sổ hay trong góc tối — ảnh trông giống hệt nhau.

Cái mua được không phải ánh sáng hơn. Ánh sáng hơn thì vận độ khuếch đại lên là xong, và nó chẳng giúp gì. Cái mua được là *ảnh giống nhau*, và đó mới là thứ đáng tiền.

Cuối cùng, một điều may mắn riêng của bài toán này. Robot cập dock thì *biết trước* nó sắp gặp những cái tag nào. Nên chỉ cần bảo nó “chỉ chấp nhận đúng sáu cái tag này, và chỉ khi chúng nằm đúng vị trí so với nhau” là gần như mọi chuyện nhận nhầm đều bị loại. Một robot đi lang thang

trong nhà kho gặp tag lạ thì không có cái may ấy — và đó là lý do bài toán docking dễ hơn về mặt an toàn so với bài toán tổng quát.

Chỗ không rút gọn được: mức cải thiện cụ thể của việc chiếu sáng chủ động thì không nói trước được bằng con số — nó phụ thuộc đèn mạnh cỡ nào, kính lọc hẹp cỡ nào, và môi trường sáng cỡ nào. Cách duy nhất là đo σ ở vài điều kiện ánh sáng khác nhau rồi xem nó nhảy bao nhiêu; nếu nó nhảy nhiều thì đáng đầu tư, nếu nó ổn định thì thôi.

Một câu: Tắt hết chế độ tự động, dùng vật liệu nhám, và nếu được thì mang theo ánh sáng của riêng mình.

14.10 Điều gì chứng minh cách hiểu này sai

Mệnh đề chính — *chiếu sáng chủ động cộng bandpass làm σ ổn định qua các điều kiện ánh sáng* — bị bác bỏ nếu mini project cho thấy σ vẫn biến thiên đáng kể với chiếu sáng chủ động bật. Tôi phát biểu nó định tính và dựa trên lập luận tỉ số tín hiệu (cửa a); nó chưa qua cửa (b). Kết quả có thể phụ thuộc mạnh vào công suất nguồn so với mức ánh sáng môi trường, nên nó không phải một mệnh đề vô điều kiện.

Mệnh đề thứ hai — *loá gây thiên lệch chứ không nhiều* — được kiểm bởi cột thứ hai của mini project: nếu *vị trí trung bình* của các góc dịch khi có loá, mệnh đề đúng; nếu chỉ σ tăng mà vị trí trung bình không đổi, thì loá chỉ gây nhiễu và teach-by-showing có cứu được — một kết quả sẽ làm nhẹ lời cảnh báo ở mục 3.

Mệnh đề thứ ba — *tham số tự động gây hại* — gần như hiển nhiên về cơ chế, nhưng độ lớn của tác hại thì chưa đo. Có thể trong một môi trường ánh sáng ổn định, phơi sáng tự động gần như không bao giờ thay đổi và tác hại bằng không. Phép kiểm: ghi lại giá trị exposure mà camera chọn theo thời gian trong một ngày vận hành, và xem nó biến thiên bao nhiêu.

14.11 Kết nối

Chương này khép Phần B và nó nối chủ yếu sang các chương về điều kiện vận hành.

Nối lên trên. [A/06-hieu-ung-vat-ly-va-quang-hoc](#) mục 6 cấp nền về vật liệu; chương này mở rộng nó thành hướng dẫn thiết kế chiếu sáng. Mục 7 ở đó — bảng phân loại theo vận tốc — bổ sung cho bảng của chương này theo một chiều khác: một bảng phân loại theo *vận tốc*, một theo *điều kiện quan sát*.

Nối ngang. [B/07-detection-pipeline](#) mục 6 chỉ ra rằng ngưỡng hoá hồng thì tag biến mất hoàn toàn — một chế độ hồng mà chỉ chương này chứa được. [B/08-corner-refinement](#) quyết định σ trong điều kiện cho trước; chương này quyết định điều kiện ấy có ổn định không. Hai chương là một cặp: một cái tối ưu giá trị, một cái tối ưu phương sai. [B/13-docking-control](#) mục 3C là chỗ chương này quan trọng nhất: mọi giới hạn của teach-by-showing liên quan tới “*g* phải không đổi” đều được chương này phục vụ.

Nối xuống Phần C. [C/15-ngan-sach-sai-so](#) nhận một yêu cầu phương pháp luận: *lập ngân sách theo σ tệ nhất*, không theo σ trong điều kiện tốt. Và nó nhận một dòng riêng cho thiên lệch do loá, thuộc loại hệ thống nhưng *phụ thuộc tư thế* — một loại khó chịu vì nó không triệt tiêu trong teach-by-showing. [C/16-danh-gia-va-nghiem-thu](#) nhận mini project của chương này làm mục “thử nghiệm độ bền” trong quy trình nghiệm thu, và nó mở rộng sang các yếu tố khác: nhiệt độ, rung, marker bắn.

Câu hỏi tự kiểm (trả lời không nhìn bài)

1. Chương này khác B/08 ở chỗ nào, và vì sao sự khác biệt ấy quan trọng với một hệ dùng teach-by-showing?
2. Nêu ba cơ chế làm điều kiện quan sát thay đổi, và nêu biện pháp duy nhất giải quyết cả ba.
3. Vì sao phơi sáng tự động gây hại theo ba đường khác nhau, và đường nào tệ nhất?
4. Loá gây thiên lệch hay nhiều? Giải thích cơ chế, và nêu vì sao teach-by-showing không cứu được nó.
5. Lợi ích chính của chiếu sáng hồng ngoại chủ động là gì, và vì sao “ảnh sáng hơn” là câu trả lời sai?
6. Chống báo sai có hai tầng. Nêu cả hai, và giải thích vì sao tầng thứ hai là một lợi thế riêng của bài toán docking.
7. Bạn thiết kế dư thừa bằng cách dùng sáu tag thay vì bốn. Nêu điều kiện mà nếu không thoả thì dư thừa ấy vô nghĩa.

NGÂN SÁCH SAI SỐ

TÓM TẮT

Phần C là chỗ cả cây hội tụ, và nó chỉ có hai chương vì nó chỉ làm hai việc. Chương 15 lập ngân sách: lấy các quan hệ tỉ lệ từ chương 5, gán số thật của một cấu hình cụ thể, rồi ra một bảng nói từng khâu ăn bao nhiêu milimét — và bảng ấy nói ngay bố trí nào có cơ hội đạt 5 mm / 0,5° và bố trí nào thì không. Chương 16 kiểm ngân sách ấy bằng hai cách: mô phỏng Monte Carlo (rẻ, làm được ngay, có script chạy được trong code/) và nghiệm thu thật (đắt, cần ground truth). Nếu chỉ nhớ một điều: *làm Phần C trước khi viết dòng code nào và trước khi đặt gia công bất cứ thứ gì* — đó là toàn bộ lý do nó tồn tại như một phần riêng chứ không phải một mục cuối Phần B.

Phần này ngắn nhưng nó là phần tôi khuyên đọc đầu tiên nếu thời gian có hạn, và lý do nằm ở kinh tế của dự án chứ không ở nội dung. Mọi chương khác trong cây mô tả những việc tốn *thời gian*: đọc, cài, thử, chỉnh. Phần C mô tả những quyết định tốn *tiền* và không đảo ngược được: kích thước tấm marker, khoảng cách giữa các tag, có nghiêng tag hay không, chọn tiêu cự nào, có làm côn dẫn hướng hay không. Một sai lầm trong nhóm quyết định thứ hai không sửa được bằng cách làm việc chăm chỉ hơn ở nhóm thứ nhất.

Có một lý do thứ hai, tinh tế hơn, để tách Phần C ra. Ngân sách sai số là chỗ duy nhất trong cây mà các chương khác được đặt cạnh nhau trên cùng một thước đo. Trong Phần B, mỗi chương nói về một kỹ thuật và tự nhiên nghe như thể kỹ thuật ấy đáng làm. Chỉ khi tất cả cùng quy về milimét thì mới lộ ra rằng một số kỹ thuật đóng góp hai chữ số phần trăm ngân sách còn một số khác đóng góp phần nghìn — và rằng thứ tự ấy không trùng với thứ tự mức độ thú vị về mặt kỹ thuật. Đây là chức năng quan trọng nhất của một ngân sách sai số: nó là công cụ chống lại xu hướng tự nhiên là làm việc thú vị thay vì việc có tác dụng.

Chương 16 có một điểm khác biệt về thể loại so với mọi chương còn lại: nó đi kèm mã nguồn chạy được. Script Monte Carlo trong 16-danh-gia-va-nghiem-thu/code/ là hiện thân của cửa kiểm chứng (b) trong learning-rules.md mục 5, và nó phục vụ cả cây chứ không riêng chương 16 — các công thức ở chương 5 chỉ thật sự qua cửa (b) khi script ấy được chạy và kết quả khớp với dự đoán giải tích. Cho tới lúc đó, chúng mới chỉ qua cửa (a).

Bảng 14.3. Bản đồ Phần C: hai chương, và việc cụ thể mỗi chương bắt bạn phải làm.

Ch.	Thư mục	Nội dung — và sản phẩm phải có sau khi đọc	Mức
15	15-ngan-sach-sai-so	Bốn quan hệ tỉ lệ thành số; ví dụ tính tay đầy đủ cho $f = 600$ px, $Z = 0,5$ m, tag 100 mm; chuỗi biến đổi từ tag tới base frame; cộng sai số thế nào cho đúng; sáu kết luận thiết kế. <i>Sản phẩm: một bảng ngân sách cho cấu hình của chính bạn</i>	● ● ●
16	16-danh-gia-va-nghiem-thu	Monte Carlo với script chạy được; ground truth bằng toàn đạc/laser tracker/motion capture/đồ gá; tách bias khỏi độ lặp lại; bản đồ sai số theo khoảng cách và góc; thử độ bền; kiểm nhất quán hiệp phương sai. <i>Sản phẩm: phân bố sai số của các phương án bố trí, trước khi gia công</i>	● ● ●

Hai chương này nên đọc liền nhau và nên đọc *sớm* — lý tưởng nhất là ngay sau chương 5, trước cả Phần B. Chương 15 cho bạn con số ước lượng, chương 16 cho bạn cách kiểm con số ấy có đúng không. Sản phẩm của một buổi chiều làm việc với hai chương này là một quyết định hình học có cơ sở, và đó là thứ đắt nhất mà cây này có thể đưa cho bạn.

Ngân sách sai số (The Error Budget)

PHỤ THUỘC A/05-lan-truyen-sai-so-pixel-sang-pose (bốn quan hệ tỉ lệ và quy tắc cộng ở mục 8 — toàn bộ chương này là việc gắn số vào chúng); và mọi chương của Phần B, vì mỗi chương ở đó cấp một dòng của bảng. Nếu chỉ đọc được hai chương trong cả cây thì hãy đọc A/05 và chương này.

TÓM TẮT

Đây là chương hội tụ của cả cây: nó lấy bốn quan hệ tỉ lệ từ A/05, gắn số thật của một cấu hình cụ thể, cộng thêm mọi dòng mà Phần B đã nhận diện, và cho ra một bảng nói từng khâu ăn bao nhiêu milimét. Bảng ấy trả lời câu hỏi mà không phân tích định tính nào trả lời được: *cấu hình này có đạt 5 mm / 0,5° không, và nếu không thì phải sửa cái gì?* Hai kết quả chính của phép tính. Thứ nhất, *tính tiền dư đã còn xoay là nút thắt* — và nút thắt ấy được chữa bằng hình học chứ không bằng thuật toán. Thứ hai, và nó bất ngờ hơn: trong chế độ đo tuyệt đối, *các dòng hệ thống chiếm phần lớn ngân sách*, nên phần lớn công sức tối ưu σ là công sức tiêu vào chỗ đã dư. Nếu chỉ nhớ một điều: *lập bảng này trước khi viết dòng code nào và trước khi đặt gia công bất cứ thứ gì* — một buổi chiều ở đây tiết kiệm vài tuần và vài triệu tiền gia công sai.

KIẾN THỨC CẦN CÓ TRƯỚC

- **ngân sách sai số (error budget)** — bảng liệt kê mọi nguồn sai số của một hệ đo, đóng góp của từng nguồn quy về cùng đơn vị, và tổng hợp của chúng theo quy tắc cộng đúng. Đây là công cụ chuẩn của ngành đo lường và có hướng dẫn quốc tế cho nó [1].
- **dòng ngẫu nhiên** — nguồn sai số đổi giữa các lần đo, trung bình không. Cộng theo bình phương; chia được cho $\sqrt{N}$ khi lấy trung bình.
- **dòng hệ thống** — nguồn không đổi hoặc đổi chậm, có dấu xác định. Cộng thẳng trong trường hợp xấu nhất; *không* chia cho $\sqrt{N}$; triệt tiêu được bằng teach-by-showing nếu nó không phụ thuộc tư thế.
- **dòng hệ thống phụ thuộc tư thế** — loại khó chịu nhất: nó không chia cho $\sqrt{N}$ và nó không triệt tiêu hoàn toàn trong teach-by-showing. Bias tâm ellipse và thiên lệch do loá thuộc loại này.
- **chế độ đo tuyệt đối** — hệ điều khiển tới một toạ độ tính toán. Mọi dòng của bảng đều tính.
- **chế độ teach-by-showing** — hệ điều khiển để tái tạo một trạng thái đã ghi. Các dòng hệ thống không phụ thuộc tư thế bị loại khỏi bảng.
- **lỗi thô (gross error)** — không phải một dòng của ngân sách mà là một sự kiện phá hỏng toàn bộ ngân sách — chọn nhầm nghiệm ambiguity, giải mã sai hướng. Phải xử lý riêng, không cộng vào.

CÂU HỎI MỞ ĐẦU

- Q1. Bạn có 5 mm ngân sách. Nếu một dòng đóng góp 3 mm và một dòng khác đóng góp 1 mm, thì tổng của hai dòng ấy là bao nhiêu — và câu trả lời phụ thuộc vào điều gì?
- Q2. Sau khi lập bảng, bạn thấy dòng lớn nhất chiếm 70% ngân sách còn dòng nhỏ nhất chiếm 2%. Vì sao việc cải thiện dòng nhỏ nhất bốn lần gần như không đổi gì, và con số cụ thể là bao nhiêu?
- Q3. Bảng ngân sách cho chế độ đo tuyệt đối và bảng cho chế độ teach-by-showing khác nhau ở những dòng nào? Ước lượng tỉ số giữa hai tổng.

- Q4. Pose ambiguity không xuất hiện như một dòng trong bảng dù nó là nguồn lỗi xoay lớn nhất. Vì sao, và nó được xử lý ở đâu?
- Q5. Bạn tính ra tổng là 4,2 mm cho ngân sách 5 mm. Nêu ba lý do vì sao con số ấy *không* cho phép bạn kết luận rằng hệ sẽ đạt.

15.1 Đặt vấn đề

Mười bốn chương trước đã nhận diện các nguồn sai số và bàn cách giảm từng nguồn. Chương này làm một việc khác và nó là việc quyết định: *đặt tất cả chúng lên cùng một thước đo*.

Lý do cần làm việc ấy đã nêu ở [00-map](#) nhưng đáng lặp lại vì nó là toàn bộ lý do chương này tồn tại. Trong Phần B, mỗi chương nói về một kỹ thuật và tự nhiên nghe như thể kỹ thuật ấy đáng làm — mọi chương đều mô tả một cải thiện thật. Chỉ khi tất cả cùng quy về milimét thì mới lộ ra rằng một số kỹ thuật đóng góp hai chữ số phần trăm ngân sách còn một số khác đóng góp phần nghìn, và rằng thứ tự ấy *không trùng* với thứ tự mức độ thú vị về mặt kỹ thuật.

Đây là chức năng quan trọng nhất của một ngân sách sai số, và nó là một chức năng về tổ chức công việc chứ không về kỹ thuật: *nó là công cụ chống lại xu hướng tự nhiên là làm việc thú vị thay vì việc có tác dụng*.

Bài toán này không mới và nó có chuẩn quốc tế. Hướng dẫn GUM [1] hình thức hoá quy trình: liệt kê nguồn, ước lượng đóng góp của từng nguồn, phân loại chúng, và tổng hợp theo quy tắc đúng. Ngành đo lường làm việc này từ lâu trước khi có robot, và phần lớn cái khó không nằm ở toán mà nằm ở hai chỗ khác: *không bỏ sót nguồn nào, và phân loại đúng*.

Có một lý do thứ hai để chương này đứng riêng thành một phần, và nó thuộc về kinh tế của dự án. Mọi chương khác mô tả những việc tốn *thời gian*: đọc, cài, thử, chỉnh. Chương này mô tả những quyết định tốn *tiền* và không đảo ngược được: kích thước tấm marker, khoảng cách giữa các tag, có nghiêng hay không, chọn tiêu cự nào, có làm côn dẫn hướng hay không. Một sai lầm trong nhóm thứ hai không sửa được bằng cách làm việc chăm chỉ hơn ở nhóm thứ nhất.

Hệ quả về thứ tự đọc: tôi khuyên đọc chương này *sớm*, lý tưởng nhất là ngay sau [A/05](#), trước cả Phần B. Bạn sẽ không hiểu hết mọi dòng — nhiều dòng trở sang các chương chưa đọc — nhưng bạn sẽ biết dòng nào lớn, và biết điều đó trước khi tiêu thời gian là khác biệt lớn nhất mà cây này có thể tạo ra.

TRỰC GIÁC

Hình dung ngân sách sai số như một bảng chi tiêu.

Bạn có năm triệu để tiêu và một danh sách các khoản. Việc đầu tiên không phải là tiết kiệm từng khoản mà là *xem khoản nào lớn*. Nếu tiền thuê nhà chiếm ba triệu rưỡi thì việc mặc cả từng nghìn tiền cà phê là vô nghĩa — không phải vì cà phê không đáng tiết kiệm mà vì nó không đổi được kết quả.

Có một chi tiết làm bảng này khác bảng chi tiêu thường, và nó quan trọng. Các khoản không cộng như nhau. Những khoản *ngẫu nhiên* — lúc nhiều lúc ít, trung bình thì đúng — cộng theo một quy tắc đặc biệt: nếu có một khoản ba triệu và một khoản một triệu thì tổng không phải bốn mà là căn của chín cộng một, tức khoảng ba phẩy hai. Khoản nhỏ gần như biến mất.

Những khoản *cố định* thì cộng thẳng như bình thường: ba cộng một là bốn.

Sự khác biệt ấy có một hệ quả thực dụng rất mạnh. Với các khoản ngẫu nhiên, chỉ khoản lớn nhất mới đáng quan tâm — mọi khoản nhỏ hơn một phần ba nó đóng góp dưới năm

phần trăm. Với các khoản cố định thì mọi khoản đều đáng quan tâm như nhau, vì chúng cộng đầy đủ.

Và bây giờ là điều bất ngờ mà bảng sẽ cho thấy: trong bài toán này, các khoản *cố định* thường lớn hơn các khoản ngẫu nhiên. Tức là phần lớn tiền của bạn đi vào những chỗ mà không phép trung bình nào chạm tới — tấm marker hơi lệch so với bản vẽ, camera gắn hơi nghiêng, ống kính hơi khác mô hình.

Và điều bất ngờ thứ hai: có một mẹo xoá gần hết các khoản cố định cùng một lúc, và nó không phải một khoản chi mà là một cách đặt lại câu hỏi.

15.2 Cấu trúc của bảng

Ba việc phải làm đúng, và hai việc sau khó hơn việc đầu.

A. Liệt kê đủ nguồn

Không bỏ sót là yêu cầu khó nhất. Danh sách dưới đây là danh sách của cây này, nhóm theo chương cấp ra nó.

- *Nhiều định vị góc* — B/08. Ngẫu nhiên.
- *Méo còn sót sau khi khử* — A/01 mục 6. Hệ thống, phụ thuộc vị trí trên ảnh.
- *Sai số intrinsic* (chủ yếu điểm chính) — A/01 mục 5, B/12 mục 3. Hệ thống.
- *Sai số constellation so với mô hình* — B/09 mục 5. Hệ thống.
- *Sai số hand-eye* — B/12 mục 5. Hệ thống.
- *Sai số khâu T_{dock}^{tag}* — B/12 mục 4. Hệ thống.
- *Bias tâm ellipse* (nếu dùng marker tròn) — A/02 mục 6. Hệ thống *phụ thuộc tư thế*.
- *Rolling shutter* — A/06 mục 2. Hệ thống, tỉ lệ vận tốc.
- *Motion blur* — A/06 mục 3. Hệ thống, tỉ lệ vận tốc.
- *Thiên lệch do loá* — B/14 mục 3. Hệ thống *phụ thuộc tư thế*.
- *Sai số bám của bộ điều khiển* — B/13. Ngẫu nhiên hoặc hệ thống tùy bộ điều khiển.
- *Dung sai cơ khí của cơ cấu cấp* — B/13 mục 7. Hệ thống.

Mười hai dòng. Một hệ thống thực tế có thể có thêm — rung, trôi nhiệt, độ lún của sàn — và việc bổ sung là việc của bạn.

B. Quy về cùng đơn vị

Mỗi nguồn phải được quy thành *sai lệch tại điểm cấp, tính bằng milimét và độ*. Đây là chỗ bốn quan hệ của A/05 được dùng.

Nhiều góc σ quy thành $\delta x = Z\sigma/f$ và $\delta\theta = Z\sigma/(fB)$. Méo còn sót e_d pixel quy tương tự: Ze_d/f . Sai số constellation e_c milimét quy thành sai số góc $\approx e_c/B$ radian. Sai số hand-eye vào thẳng vì nó đã là milimét và độ.

Chú ý một chi tiết dễ sai: *phải quy tại đúng điểm cấp*, không phải tại tâm camera. Một sai số góc $\delta\theta$ tại camera trở thành một sai lệch vị trí $L\delta\theta$ tại một điểm cách camera L — và với L cỡ nửa mét, $0,5^\circ$ trở thành 4,4 mm. Đây là lý do ngân sách góc và ngân sách vị trí không độc lập, và mục 5 bàn kỹ.

C. Phân loại đúng

Ba loại, và phân loại sai làm tổng sai theo hướng nguy hiểm.

Ngẫu nhiên: cộng bình phương, chia $\sqrt{N}$.

Hệ thống không phụ thuộc tư thế: cộng thẳng, không chia $\sqrt{N}$, *triệt tiêu* trong teach-by-showing.

Hệ thống phụ thuộc tư thế: cộng thẳng, không chia $\sqrt{N}$, *không triệt tiêu hoàn toàn*. Đây là loại tệ nhất và nó cần một cột riêng trong bảng.

Cách kiểm phân loại: với mỗi dòng, hỏi hai câu. *Lấy trung bình 100 khung thì nó giảm không?* Nếu có thì ngẫu nhiên. *Nó có giống nhau ở tư thế dạy và tư thế hiện tại không?* Nếu có thì nó triệt tiêu được.

15.3 Bảng ngân sách: ví dụ số đầy đủ

Đây là phần cốt lõi của chương. Cấu hình giả định — tiếp nối ví dụ chuẩn của cây từ A/05 mục 4 và bản thiết kế mẫu ở B/09 mục 8:

$$f = 600 \text{ px}, \quad Z = 0,5 \text{ m}, \quad W = 75 \text{ mm}, \quad B = 0,22 \text{ m}, \quad \sigma = 0,2 \text{ px}, \quad L = 0,5 \text{ m},$$

với L là khoảng cách từ camera tới điểm cập. Kích thước tag trên ảnh: $w = fW/Z = 600 \times 0,075/0,5 = 90 \text{ px}$.

Bảng 15.1. Bảng ngân sách sai số — ví dụ số học, không phải số đo. Cột “tại điểm cập” quy mọi thứ về milimét tại điểm cập, gồm cả đóng góp của sai số góc qua cánh tay đòn L .

Nguồn	Loại	Giá trị gốc	Tại điểm cập	Cách tính
Nhiều góc → vị trí	Ngẫu nhiên	0,2 px	0,17 mm	$Z\sigma/f$
Nhiều góc → chiều sâu	Ngẫu nhiên	0,2 px	1,11 mm	$Z\sigma/w$
Nhiều góc → xoay	Ngẫu nhiên	0,2 px	0,38 mm	$L \cdot Z\sigma/(fB)$
Méo còn sót	Hệ thống	0,3 px	0,25 mm	Ze_d/f
Điểm chính sai	Hệ thống	3 px	2,50 mm	$L \cdot e_c/f$
Constellation sai	Hệ thống	0,3 mm	0,68 mm	$L \cdot e/B$
Hand-eye	Hệ thống	1,0 mm	1,00 mm	vào thẳng
$T_{\text{dock}}^{\text{tag}}$	Hệ thống	0,5 mm	0,50 mm	vào thẳng
Loá (phụ thuộc tư thế)	HT-tư thế	0,4 px	0,33 mm	Ze/f
Bám của bộ điều khiển	Ngẫu nhiên	—	1,00 mm	đo được

A. Tổng hợp

Nhóm ngẫu nhiên, cộng bình phương:

$$\sqrt{0,17^2 + 1,11^2 + 0,38^2 + 1,00^2} = \sqrt{0,029 + 1,232 + 0,144 + 1,000} = \sqrt{2,405} = 1,55 \text{ mm}.$$

Nhóm hệ thống, cộng thẳng:

$$0,25 + 2,50 + 0,68 + 1,00 + 0,50 = \mathbf{4,93 \text{ mm.}}$$

Nhóm hệ thống phụ thuộc tư thế: 0,33 mm.

Tổng trong chế độ đo tuyệt đối, cộng thẳng ba nhóm (thận trọng):

$$1,55 + 4,93 + 0,33 = \mathbf{6,81 \text{ mm.}}$$

Không đạt ngân sách 5 mm.

B. Đọc bảng

Ba điều lộ ra ngay, và cả ba đều quan trọng hơn con số tổng.

Một, nhóm hệ thống chiếm 72% tổng (4,93 trên 6,81). Đây là kết quả bất ngờ nhất của chương với người đã dành thời gian tối ưu detector: phần lớn ngân sách nằm ở những dòng mà không thuật toán xử lý ảnh nào chạm tới.

Hai, dòng lớn nhất là điểm chính sai — 2,50 mm, tức 37% tổng và một nửa nhóm hệ thống. Con số 3 px cho sai số điểm chính là hoàn toàn hợp lý với một bộ ảnh hiệu chuẩn không có tư thế nghiêng mạnh (B/12 mục 3A). Đây là chỗ mà một buổi chụp lại ảnh hiệu chuẩn cho ra nhiều milimét hơn bất kỳ thay đổi phần mềm nào.

Ba, trong nhóm ngẫu nhiên, dòng lớn nhất là chiều sâu (1,11 mm) và dòng nhỏ nhất là vị trí ngang (0,17 mm). Cải thiện dòng nhỏ nhất bốn lần — từ 0,17 xuống 0,04 — làm tổng nhóm ngẫu nhiên đổi từ 1,55 thành 1,54. Đây là câu trả lời cho Q2 dưới dạng số cụ thể: gần như không đổi gì.

CẠM BẮY

Mọi con số trong bảng 15.1 là ví dụ số học tự đặt, không phải số đo của một hệ thống nào.

Chúng được chọn để có bậc độ lớn hợp lý và để minh họa cấu trúc của bài toán. Chúng không dùng được để:

Báo cáo về hệ của bạn. Kết luận rằng một cấu hình cụ thể đạt hay không đạt. Hoặc so sánh với số đo của người khác.

Chúng dùng được để: thấy bậc độ lớn tương đối giữa các dòng; thấy cấu trúc của phép tính; và làm mẫu để bạn điền số của chính mình vào.

Bốn con số bạn phải tự đo trước khi bảng có nghĩa: σ (B/07 mini project), sai số điểm chính (B/12 mini project), sai số constellation (B/09 mini project), và sai số bám của bộ điều khiển. Bốn phép đo ấy tốn vài ngày, và không có chúng thì bảng chỉ là một bài tập.

15.4 Cánh tay đòn: vì sao góc và vị trí không độc lập

Chi tiết quan trọng mà bảng 15.1 đã dùng nhưng chưa giải thích.

Ngân sách được phát biểu là “5 mm và 0,5°” như thể hai đại lượng độc lập. Chúng không độc lập, vì một sai số góc tại camera trở thành một sai lệch vị trí tại điểm cập:

$$\delta x_{\text{tại điểm cập}} = L \cdot \delta \theta,$$

với L là khoảng cách từ tâm quay tới điểm cập.

Với $L = 0,5 \text{ m}$ và $\delta\theta = 0,5^\circ = 8,73 \times 10^{-3} \text{ rad}$:

$$\delta x = 0,5 \times 8,73 \times 10^{-3} = 4,4 \text{ mm}.$$

Nghĩa là *toàn bộ ngân sách góc, nếu tiêu hết, đã gần bằng toàn bộ ngân sách vị trí*. Hai ngân sách không cộng độc lập được.

Hai hệ quả thực hành.

Một, cách phát biểu ngân sách đúng là phát biểu nó tại điểm cập bằng một đại lượng duy nhất — sai lệch của điểm cập, tính bằng milimét — rồi *suy ra* ngân sách góc từ đó nếu cần. Bảng 15.1 làm đúng như vậy: cột cuối quy mọi thứ về milimét tại điểm cập.

Hai, giảm L là một đòn bẩy. Nếu camera được gắn gần điểm cập thay vì ở phía sau robot, thì cánh tay đòn ngắn lại và sai số góc ít quan trọng hơn. Đây là một quyết định cơ khí có giá trị thật và nó thường bị bỏ qua — camera hay được gắn ở chỗ tiện lắp chứ không ở chỗ tối ưu về ngân sách.

Trường hợp đặc biệt đáng nêu: nếu cơ cấu cập có *hai* điểm tiếp xúc cách nhau một khoảng d , thì yêu cầu góc chặt hơn yêu cầu vị trí theo tỉ lệ $1/d$ — vì một sai lệch góc làm hai điểm lệch ngược chiều nhau. Khi ấy ngân sách góc mới là ngân sách chặt thật sự, và đây là tình huống phổ biến với đầu cắm sạc có hai chấu.

15.5 Hai chế độ, hai bảng

Mục này trả lời Q3, và nó là mục có giá trị thực dụng cao nhất sau bảng.

B/13 mục 3 chỉ ra rằng teach-by-showing triệt tiêu các dòng hệ thống *không phụ thuộc tư thế*. Áp vào bảng 15.1:

Bị loại: méo còn sót (0,25), điểm chính (2,50), constellation (0,68), hand-eye (1,00), $T_{\text{dock}}^{\text{tag}}$ (0,50).
Tổng bị loại: 4,93 mm.

Còn lại: toàn bộ nhóm ngẫu nhiên (1,55 mm) và dòng phụ thuộc tư thế (0,33 mm).

Tổng trong chế độ teach-by-showing:

$$1,55 + 0,33 = 1,88 \text{ mm}.$$

Đạt, và đạt với biên độ lớn.

Tỉ số giữa hai chế độ: $6,81/1,88 = 3,6$ lần. Đây là câu trả lời định lượng cho Q3, và nó là con số biện minh cho việc 00-map gọi teach-by-showing là “mẹo lớn nhất” — một hệ số 3,6 không đạt được bằng bất kỳ cải thiện thuật toán nào trong cả cây.

Thêm một bước nữa: nếu áp dụng stop-and-go với $N = 30$ khung (B/13 mục 4), nhóm ngẫu nhiên chia cho $\sqrt{30} = 5,5$ — trừ dòng bám của bộ điều khiển, vốn không phải nhiễu đo. Ba dòng nhiễu góc cộng lại là $\sqrt{0,17^2 + 1,11^2 + 0,38^2} = 1,18 \text{ mm}$, chia 5,5 còn 0,21 mm. Tổng mới:

$$\sqrt{0,21^2 + 1,00^2} + 0,33 = 1,02 + 0,33 = 1,35 \text{ mm}.$$

Và bây giờ dòng lớn nhất là *bám của bộ điều khiển* — một dòng mà cả cây này gần như không bàn tới. Đó là một kết quả đáng ghi nhận: sau khi áp dụng đủ các kỹ thuật, nút thắt chuyển sang một lĩnh vực khác.

CẦN NHỚ

Ba con số của cùng một cấu hình, khác nhau chỉ ở *chiến lược*:

Do tuyệt đối: 6,81 mm — không đạt.

Teach-by-showing: 1,88 mm — đạt.

Teach-by-showing + stop-and-go: 1,35 mm — đạt thoải mái, và nút thắt đã chuyển sang bộ điều khiển.

Không có dòng code xử lý ảnh nào khác nhau giữa ba trường hợp. Khác biệt hoàn toàn nằm ở *kiến trúc* — cách đặt bài toán và cách vận hành.

Đây là luận điểm trung tâm của cây, dưới dạng ba con số.

15.6 Ambiguity và lỗi thô: vì sao chúng không nằm trong bảng

Mục này trả lời Q4, và nó là một điểm phương pháp luận quan trọng.

Pose ambiguity ([A/04](#)) là nguồn lỗi xoay lớn nhất trong thực tế — hai nghiệm cách nhau 2α , có thể hàng chục độ. Vậy vì sao nó không có dòng trong bảng [15.1](#)?

Vì nó không phải một *sai số* mà là một *lỗi thô*. Sự khác biệt không phải về độ lớn mà về bản chất:

Một sai số là một độ lệch nhỏ quanh giá trị đúng, và nó cộng được với các sai số khác theo quy tắc ở mục 2C. Một lỗi thô là việc bạn đang ở một giá trị *hoàn toàn khác* — và khi nó xảy ra, mọi con số trong bảng trở nên vô nghĩa, vì bảng giả định bạn đang ở gần nghiệm đúng.

Nói bằng hình ảnh: ngân sách sai số mô tả độ chính xác của một phép đo; lỗi thô là việc đo nhầm vật.

Hệ quả về cách xử lý: lỗi thô phải được đưa về *xác suất bằng không* (hoặc *gần không*) bằng một cơ chế riêng, chứ không được cộng vào bảng. [B/09](#) mục 4 (phá đồng phẳng) và [B/10](#) (đo ρ và từ chối) là hai cơ chế ấy. Chỉ sau khi chúng đã hoạt động thì bảng mới có nghĩa.

Cùng loại với ambiguity: giải mã sai hướng ([B/07](#) mục 6), tag lắp sai vị trí, và báo sai. Cả bốn đều là lỗi thô và cả bốn đều cần cơ chế phát hiện riêng.

Cách trình bày đúng trong một báo cáo: *ngân sách sai số, kèm một danh sách các lỗi thô đã được loại trừ và cơ chế loại trừ chúng*. Trình bày một ngân sách mà không nói về lỗi thô là trình bày một nửa sự thật, và nó là nửa dễ chịu hơn.

15.7 Sáu kết luận thiết kế

Đọc bảng cho ra sáu kết luận, và chúng là sản phẩm cuối cùng của chương.

Một, tịnh tiến dư dả, xoay là nút thắt. Trong bảng, ba dòng nhiều góc quy về vị trí cộng lại chỉ 1,18 mm, trong khi các dòng liên quan tới góc — điểm chính, constellation, hand-eye — chiếm phần lớn. Và cách chữa chúng là hình học và hiệu chuẩn, không phải thuật toán.

Hai, baseline và tính không đồng phẳng mua được nhiều hơn detector. Dòng “constellation sai” quy về sai số góc qua e/B , nên tăng B giảm nó tuyến tính — và đồng thời giảm dòng “nhiều góc → xoay”. Một thay đổi cơ khí, hai dòng cải thiện.

Ba, cơ khí tẩm marker là sần sai số, và BA là cách hạ sần. Với constellation lấy từ CAD ($e \approx 1,5$ mm thay vì 0,3), dòng ấy thành $0,5 \times 1,5/220 \times 1000 = 3,4$ mm — lớn hơn cả điểm chính, và đủ để một mình phá ngân sách.

Bốn, *teach-by-showing* là quyết định có đòn bẩy lớn nhất. Hệ số 3,6 ở mục 5, không dòng code xử lý ảnh nào.

Năm, *stop-and-go* là cách rẻ nhất để hạ nhóm ngẫu nhiên. Nó cũng xoá hai dòng mà bảng ví dụ này không có vì đã giả định robot đứng yên — rolling shutter và blur. Nếu bạn đo khi di chuyển, hai dòng ấy xuất hiện và chúng có thể lớn.

Sáu, *côn dẫn hướng cơ khí đổi bản chất bài toán*. Nó không giảm một dòng nào; nó *nới ngưỡng*. Với côn 15 mm, cả ba con số ở mục 5 đều đạt thoải mái, kể cả chế độ đo tuyệt đối. Nhưng như B/13 mục 7 cảnh báo, nó nới dòng *vị trí* tốt hơn nới dòng *góc*.

15.8 Giới hạn và chế độ hỏng

Mục này trả lời Q5, và nó là mục bắt buộc phải đọc trước khi dùng bảng để ra quyết định.

Ba lý do một tổng dưới ngưỡng không đảm bảo hệ sẽ đạt.

Lý do một: bạn có thể đã bỏ sót một nguồn. Mười hai dòng ở mục 2A là danh sách của tôi, không phải danh sách đầy đủ của vũ trụ. Rung, trôi nhiệt, độ lún của sàn, biến dạng của khung robot khi mang tải — mỗi hệ thống có những nguồn riêng. Một nguồn bị bỏ sót không xuất hiện dưới dạng một cảnh báo; nó xuất hiện dưới dạng một hệ thống không đạt mà không ai hiểu vì sao.

Lý do hai: các con số đầu vào có bất định riêng. Nếu bạn ước lượng sai số constellation là 0,3 mm mà thực tế là 0,8 mm, thì dòng ấy lớn hơn 2,7 lần và tổng đối đáng kể. Cách xử lý đúng: mỗi con số đầu vào nên đi kèm một khoảng, và bảng nên được tính cho cả trường hợp xấu nhất — một phép tính tốn thêm mười phút và nó thay đổi kết luận thường xuyên hơn bạn nghĩ.

Lý do ba, và nó là lý do quan trọng nhất: lỗi thô không nằm trong bảng. Mục 6. Một hệ có ngân sách 1,35 mm nhưng chọn nhầm nghiệm ambiguity 5% số lần là một hệ thất bại 5% số lần, và bảng không nói gì về điều đó.

Giới hạn khác của chương.

Quy tắc cộng là xấp xỉ. Cộng thẳng các dòng hệ thống là giả định trường hợp xấu nhất — chúng có thể triệt tiêu nhau một phần. Ngược lại, cộng bình phương các dòng ngẫu nhiên giả định chúng độc lập, và một số dòng có thể tương quan. GUM [1] có cách xử lý chặt chẽ hơn với ma trận hiệp phương sai; với mục đích thiết kế thì quy tắc đơn giản đủ dùng, miễn là bạn biết nó là xấp xỉ nào.

Các dòng có thể phụ thuộc nhau. Sai số intrinsic ảnh hưởng tới kết quả đo constellation (B/12 mục 2), nên hai dòng ấy không độc lập. Bảng bỏ qua chuyện này, và nó là một xấp xỉ về phía *lạc quan*.

Bảng tính ở một tư thế. Mọi con số ở mục 3 tính tại $Z = 0,5$ m. Ở các tư thế khác chúng khác, và một số dòng khác nhiều. Với một phân tích đầy đủ, phải quét toàn dải tư thế — và đó chính là việc mà C/16 làm bằng Monte Carlo.

15.9 Thực hành

Sáu bước, và chúng là quy trình tôi khuyên cho một dự án thật.

Một, lập bảng bằng số ước lượng ngay ngày đầu, kể cả khi chưa đo gì. Dùng bảng 15.1 làm mẫu và điền số phỏng đoán. Nó sai, nhưng nó cho bạn biết đo cái gì trước.

Hai, đo bốn con số quan trọng nhất theo cảnh báo ở mục 3B: σ , sai số điểm chính, sai số constellation, sai số bám.

Ba, tính cả hai bảng — đo tuyệt đối và teach-by-showing — và quyết định chế độ dựa trên chênh lệch.

Bốn, tính cả trường hợp xấu nhất với các con số đầu vào ở biên trên của khoảng bất định.

Năm, viết danh sách lỗi thô và cơ chế loại trừ, và coi nó là một phần của ngân sách chứ không phải một phụ lục.

Sáu, quét toàn dải tư thế bằng Monte Carlo (C/16) trước khi đặt gia công.

Active recall (che cột phải, tự trả lời, rồi mở ra đối chiếu)

Thuật ngữ / câu hỏi	Trả lời
••• Chức năng quan trọng nhất của ngân sách	Đặt mọi nguồn lên cùng một thước đo, để lộ ra rằng thứ tự đóng góp không trùng với thứ tự mức độ thú vị. Nó là công cụ chống lại xu hướng làm việc thú vị thay vì việc có tác dụng.
••• Ba loại dòng	Ngẫu nhiên (cộng bình phương, chia $\sqrt{N}$); hệ thống không phụ thuộc tư thế (cộng thẳng, không chia, <i>triệt tiêu</i> trong teach-by-showing); hệ thống <i>phụ thuộc tư thế</i> (cộng thẳng, không chia, <i>không</i> triệt tiêu) — loại cuối tệ nhất.
••• Phép kiểm phân loại	Hai câu hỏi: lấy trung bình 100 khung thì nó giảm không ($\Rightarrow$ ngẫu nhiên)? và nó có giống nhau ở tư thế dạy và tư thế hiện tại không ($\Rightarrow$ triệt tiêu được)?
••• Cánh tay đòn	Sai số góc tại camera thành sai lệch vị trí $L \delta\theta$ tại điểm cập. Với $L = 0,5 \text{ m}$, $0,5^\circ$ thành 4,4 mm — gần hết ngân sách vị trí. Hai ngân sách không độc lập.
••• Cách phát biểu ngân sách đúng	Tại <i>điểm cập</i> , bằng một đại lượng duy nhất (mm), rồi suy ra ngân sách góc. Và: giảm L — gắn camera gần điểm cập — là một đòn bẩy cơ khí thường bị bỏ qua.
••• Ba con số của ví dụ chuẩn	Đo tuyệt đối: 6,81 mm (không đạt). Teach-by-showing: 1,88 mm (đạt). Cộng stop-and-go $N = 30$: 1,35 mm. Không dùng code xử lý ảnh nào khác nhau — khác biệt hoàn toàn ở kiến trúc.
••• Kết quả bất ngờ nhất của bảng	Nhóm hệ thống chiếm 72% tổng trong chế độ đo tuyệt đối. Phần lớn ngân sách nằm ở những dòng mà không thuật toán xử lý ảnh nào chạm tới.
••• Dòng lớn nhất trong ví dụ	Điểm chính sai — 2,50 mm, tức 37% tổng. Một buổi chụp lại ảnh hiệu chuẩn có tư thế nghiêng mạnh cho nhiều milimét hơn bất kỳ thay đổi phần mềm nào.
••• Cải thiện dòng nhỏ nhất	Từ 0,17 xuống 0,04 mm làm tổng nhóm ngẫu nhiên đổi từ 1,55 thành 1,54 — <i>gần như không đổi gì</i> . Hệ quả của phép cộng bình phương.
••• Vì sao ambiguity không có dòng trong bảng	Vì nó là <i>lỗi thô</i> , không phải sai số: bảng giả định bạn đang ở gần nghiệm đúng, còn lỗi thô là việc bạn đang ở một giá trị hoàn toàn khác. Ngân sách mô tả độ chính xác của phép đo; lỗi thô là đo nhầm vật.
••• Xử lý lỗi thô thế nào	Đưa về xác suất gần không bằng cơ chế riêng (B/09 mục 4 phá đồng phẳng; B/10 đo ρ và từ chối), rồi trình bày kèm bảng như một danh sách đã loại trừ — không cộng vào bảng.
••• Ba lý do tổng dưới ngưỡng không đảm bảo đạt	Có thể đã bỏ sót một nguồn; các con số đầu vào có bất định riêng (phải tính cả trường hợp xấu nhất); và lỗi thô không nằm trong bảng.
•• Constellation từ CAD ăn bao nhiêu	Với $e \approx 1,5 \text{ mm}$ thay vì 0,3: dòng ấy thành 3,4 mm — lớn hơn cả điểm chính, và đủ một mình phá ngân sách. Đây là lý do BA bắt buộc (B/09 mục 5).
•• Trường hợp hai điểm tiếp xúc	Nếu cơ cấu cập có hai chấu cách nhau d , yêu cầu góc chặt hơn yêu cầu vị trí theo tỉ lệ $1/d$, vì sai lệch góc làm hai điểm lệch <i>ngược chiều</i> . Khi ấy ngân sách góc mới là ngân sách chặt thật sự.

Thuật ngữ / câu hỏi	Trả lời (tiếp)
●● Ba xấp xỉ của quy tắc cộng	Cộng thẳng dòng hệ thống là trường hợp xấu nhất (chúng có thể triệt tiêu một phần); cộng bình phương giả định độc lập (một số dòng tương quan); và bỏ qua việc intrinsic ảnh hưởng tới constellation — xấp xỉ về phía <i>lạc quan</i> .
●●● Khi nào đọc chương này	Sớm — lý tưởng nhất ngay sau A/05 , trước cả Phần B. Bạn sẽ không hiểu hết mọi dòng, nhưng bạn sẽ biết dòng nào lớn, và biết điều đó <i>trước khi</i> tiêu thời gian là khác biệt lớn nhất mà cây này tạo ra.

●●● phải thuộc ●● nên nắm ● biết là có

I Kiến thức này đang sống ở đâu (công trình đang dùng chính ý tưởng của chương; chốt 9/2026)

Công trình / hệ thống	Dùng ý tưởng nào của chương	Ghi chú
Hướng dẫn GUM [1]	Chuẩn quốc tế cho lan truyền bất định: phân loại nguồn, tổng hợp, và cách báo cáo	Đọc §4–5 nếu cần xử lý chặt chẽ hơn quy tắc đơn giản ở mục 2C
ISO 9283 [3]	Định nghĩa chuẩn của accuracy và repeatability cho robot	Cơ sở cho việc tách hai bảng ở mục 5; dùng đúng thuật ngữ khi báo cáo
Kỹ thuật hệ thống trong quang học và hàng không	Ngân sách sai số là công cụ chuẩn từ lâu trước robot học	Truyền thống mà chương này mượn; tư duy quan trọng hơn công thức
Script Monte Carlo (C/16)	Quét toàn dải tư thế thay vì tính tại một điểm	Bổ sung bắt buộc cho chương này; bảng tính tại một tư thế là chưa đủ
evo [66]	Đánh giá quỹ đạo với phép căn chỉnh đúng	Cho phần nghiệm thu; C/16 bàn kỹ

MINI PROJECT · Lập bảng ngân sách cho hệ của chính bạn, hai chế độ, và trường hợp xấu nhất · 1–2 ngày sau khi đã đo

Mục tiêu	có một bảng duy nhất nói hệ của bạn đạt hay không đạt, dòng nào lớn nhất, và việc gì nên làm tiếp — thay cho một cảm giác mơ hồ.
Dữ liệu	bốn con số đo được từ các mini project trước: σ (B/07), sai số điểm chính (B/12), sai số constellation (B/09), sai số bám (B/13). Cộng các thông số hình học của hệ: f, Z, W, B, L .
Các bước	dựng bảng theo mẫu ở mục 3 với số của bạn; với mỗi con số đo được, ghi <i>cả khoảng</i> chứ không chỉ giá trị trung tâm; tính ba tổng: đo tuyệt đối, teach-by-showing, và teach-by-showing cộng stop-and-go; rồi tính lại cả ba với mọi con số đầu vào ở biên trên của khoảng; cuối cùng viết danh sách các lỗi thô kèm cơ chế loại trừ và xác suất còn lại ước lượng được.
Xong khi	bạn có sáu con số tổng (ba chế độ $\times$ hai trường hợp) và một danh sách lỗi thô. Quyết định rút ra: chế độ nào, và dòng nào phải sửa trước. Kiểm tra chéo bắt buộc: cộng lại tổng bằng cách khác — chẳng hạn cộng bình phương <i>tất cả</i> các dòng như một cận dưới lạc quan — và xác nhận rằng con số thận trọng của bạn nằm giữa cận lạc quan ấy và tổng cộng thẳng toàn bộ. Nếu không thì có lỗi số học ở đâu đó.
Nối với	C/16 kiểm bảng này bằng Monte Carlo trên toàn dải tư thế; E/19 dùng nó làm tiêu chí quyết định ở các bước của lộ trình.

TRẢ LỜI CÂU HỎI MỞ ĐẦU

- A1. Phụ thuộc vào việc hai dòng ấy thuộc loại nào. Nếu cả hai là *ngẫu nhiên*, chúng cộng theo bình phương: $\sqrt{9 + 1} = 3,16$ mm — dòng nhỏ chỉ thêm 5%. Nếu cả hai là *hệ thống*, chúng cộng thẳng trong trường hợp xấu nhất: 4,0 mm — dòng nhỏ thêm đủ 33%. Sự khác biệt ấy không nhỏ và nó quyết định việc có đáng tối ưu dòng nhỏ hay không. Cách kiểm phân loại đơn giản: lấy trung bình 100 khung xem nó có giảm không — giảm thì ngẫu nhiên, không giảm thì hệ thống.
- A2. Vì các dòng ngẫu nhiên cộng theo bình phương, nên đóng góp của một dòng vào tổng tỉ lệ với *bình phương* của nó. Một dòng chiếm 2% tổng có giá trị bình phương chiếm khoảng 0,04% tổng bình phương, nên xoá sạch nó cũng gần như không đổi gì. Con số cụ thể từ bảng ở mục 3: dòng “nhiều góc → vị trí” là 0,17 mm; cải thiện nó bốn lần xuống 0,04 mm làm tổng nhóm ngẫu nhiên đổi từ $\sqrt{2,405} = 1,55$ thành $\sqrt{2,377} = 1,54$ mm. Thay đổi 0,01 mm trên một ngân sách 5 mm — tức 0,2%. Đây là lập luận định lượng cho nguyên tắc “chỉ mất xích yếu nhất mới quyết định”.
- A3. Khác nhau ở toàn bộ nhóm hệ thống *không phụ thuộc tư thế*: méo còn sót, điểm chính, constellation, hand-eye, và $T_{\text{dock}}^{\text{tag}}$ — năm dòng, tổng 4,93 mm trong ví dụ, bị loại hoàn toàn. Còn lại là nhóm ngẫu nhiên (1,55 mm) và dòng hệ thống *phụ thuộc tư thế* (0,33 mm, thiên lệch do loá), vì loại sau không triệt tiêu được khi tư thế đổi. Tổng: 6,81 mm so với 1,88 mm, tức tỉ số 3,6. Đây là con số biện minh cho việc gọi teach-by-showing là mẹo lớn nhất — một hệ số 3,6 không đạt được bằng bất kỳ cải thiện thuật toán nào trong cả cây, và nó không tốn một dòng code xử lý ảnh nào.
- A4. Vì nó không phải một *sai số* mà là một *lỗi thô*, và sự khác biệt là về bản chất chứ không về độ lớn. Một sai số là độ lệch nhỏ quanh giá trị đúng và nó cộng được với các sai số khác. Một lỗi thô là việc bạn đang ở một giá trị *hoàn toàn khác* — và khi nó xảy ra, mọi con số trong bảng vô nghĩa, vì bảng giả định bạn đang ở gần nghiệm đúng. Nói bằng hình ảnh: ngân sách mô tả độ chính xác của một phép đo, còn lỗi thô là đo nhầm vật. Nó được xử lý ở B/09 mục 4 (phá đồng phẳng, đưa xác suất về không) và B/10 (đo ρ và từ chối). Cách trình bày đúng: ngân sách sai số *kèm* một danh sách các lỗi thô đã loại trừ và cơ chế loại trừ chúng — trình bày một ngân sách mà không nói về lỗi thô là trình bày nửa dễ chịu của sự thật.
- A5. Ba lý do. Một, bạn có thể đã bỏ sót một nguồn — mười hai dòng trong chương là danh sách của tôi, và mỗi hệ có những nguồn riêng như rung, trôi nhiệt, độ lún sàn, biến dạng khung khi mang tải; một nguồn bị bỏ sót không xuất hiện dưới dạng cảnh báo mà dưới dạng một hệ không đạt mà không ai hiểu vì sao. Hai, các con số đầu vào có bất định riêng: nếu bạn ước lượng sai số constellation là 0,3 mm mà thực tế 0,8 mm thì dòng ấy lớn hơn 2,7 lần — nên phải tính lại bảng với mọi đầu vào ở biên trên của khoảng, một phép tính tốn mười phút và đổi kết luận thường xuyên hơn bạn nghĩ. Ba, và quan trọng nhất: lỗi thô không nằm trong bảng — một hệ có ngân sách 1,35 mm nhưng chọn nhầm nghiệm ambiguity 5% số lần là một hệ thất bại 5% số lần, và bảng hoàn toàn im lặng về điều đó.

NÓI LẠI THẬT ĐƠN GIẢN

Bạn có năm milimét để tiêu, và có khoảng chục khoản chi. Bảng ngân sách chỉ làm một việc: viết ra mỗi khoản ăn bao nhiêu, để bạn biết khoản nào lớn.

Nghe tầm thường, nhưng nó là việc quan trọng nhất trong cả tài liệu này, vì con người có xu hướng rất mạnh là dành thời gian cho việc thú vị thay vì việc có tác dụng. Chính thuật toán thì thú vị. Chụp lại mấy chục tấm ảnh hiệu chuẩn thì chán. Và trong cái bảng đó, việc chán lại là việc ăn hai phẩy năm milimét còn việc thú vị chỉ ăn một phần mười.

Có một chi tiết làm bảng này khác bảng chi tiêu thường. Những khoản *lúc nhiều lúc ít* thì cộng theo một quy tắc kỳ lạ: ba cộng một ra ba phẩy hai, chứ không phải bốn. Khoản nhỏ gần như biến mất. Nên với loại này, chỉ khoản lớn nhất đáng quan tâm.

Còn những khoản *cố định* — tấm marker lệch một chút so với bản vẽ, camera gấn hơi nghiêng, ống kính hơi khác mô hình — thì cộng thẳng như bình thường, và chúng không bao giờ nhỏ đi dù bạn đo bao nhiêu lần.

Và đây là điều bất ngờ khi làm phép tính: *mấy khoản cố định thường chiếm hơn hai phần ba*. Tức là phần lớn ngân sách của bạn đang nằm ở những chỗ mà không có dòng code nào chạm tới được.

Rồi đến điều bất ngờ thứ hai, và nó là lý do cả tài liệu này tồn tại. Nếu thay vì hỏi “robot đang ở đâu” ta hỏi “robot khác gì so với lúc nó cầm đúng”, thì *toàn bộ* mấy khoản cố định biến mất — vì chúng xuất hiện giống nhau ở cả hai lần và trừ đi nhau. Cùng một hệ thống, cùng một camera, cùng một phần mềm: sáu phẩy tám milimét thành một phẩy chín. Không đạt thành đạt thoải mái. Cuối cùng, một cảnh báo về chính cái bảng. Nó tính được tổng của những thứ bạn đã nghĩ tới, và nó hoàn toàn im lặng về hai thứ khác. Một là những khoản bạn quên chưa liệt kê. Hai, và tệ hơn, là chuyện *nhìn nhầm* — cái tag phẳng có hai câu trả lời, và nếu máy chọn nhầm thì nó không lệch vài milimét mà lệch mấy chục độ. Chuyện đó không phải một khoản chi trong ngân sách; nó là việc bạn đang tiêu tiền của người khác. Phải xử lý riêng, và phải xử lý trước.

Chỗ không rút gọn được: quy tắc cộng ở đây là xấp xỉ: cộng thẳng mấy khoản cố định là giả định trường hợp xấu nhất (thực tế chúng có thể triệt tiêu nhau một phần), còn cộng theo bình phương mấy khoản ngẫu nhiên là giả định chúng độc lập (thực tế một vài cái có liên quan). Có cách tính chặt chẽ hơn, nhưng với việc thiết kế thì cách đơn giản đủ dùng — miễn là biết mình đang xấp xỉ cái gì.

Một câu: *Viết ra khoản nào ăn bao nhiêu, và bạn sẽ thấy phần lớn tiền đang đi vào những chỗ không có dòng code nào chạm tới.*

15.10 Điều gì chứng minh cách hiểu này sai

Mệnh đề chính — *các dòng hệ thống chiếm phần lớn ngân sách trong chế độ đo tuyệt đối* — là mệnh đề rút ra từ một bảng với các con số ví dụ do tôi đặt. Nó bị bác bỏ nếu trên một hệ thực với các con số đo được, nhóm ngẫu nhiên lại chiếm phần lớn. Điều đó hoàn toàn có thể xảy ra nếu hiệu chuẩn rất tốt và constellation đã đo bằng BA — và nếu vậy thì lời khuyên của chương đổi: khi ấy [B/08](#) và [B/11](#) mới là chương đáng đầu tư. Mini project là cách kiểm cho hệ của bạn, và tôi nhấn mạnh rằng *kết luận của chương phụ thuộc vào số liệu, không phải ngược lại*.

Mệnh đề thứ hai — *teach-by-showing cho hệ số cải thiện cỡ ba tới bốn lần* — bị bác bỏ nếu mini project của [B/13](#) cho thấy độ lặp lại không tốt hơn độ chính xác đáng kể. Con số 3,6 ở mục 5 là hệ quả trực tiếp của bảng ví dụ, nên nó thừa hưởng toàn bộ tính không chắc chắn của bảng ấy.

Mệnh đề thứ ba — *lỗi thô phải xử lý riêng, không cộng vào bảng* — là một mệnh đề phương pháp luận chứ không thực nghiệm. Nó “bị bác bỏ” chỉ nếu ai đó đưa ra một cách hợp lý để cộng một sự kiện nhị phân có xác suất vào một tổng sai số liên tục — và tôi không biết cách nào.

Cuối cùng, và đây là điều tôi muốn nhấn mạnh nhất: *mọi con số trong chương này là ví dụ*. Nếu bạn thấy bảng của mình cho kết luận khác, bảng của bạn đúng và bảng của tôi chỉ là minh họa. Giá trị của chương nằm ở *cấu trúc* của phép tính và ở các câu hỏi nó buộc bạn trả lời, không ở các con số.

15.11 Kết nối

Chương này là nơi cả cây hội tụ, nên phần kết nối của nó là một bản đồ ngược.

Nói lên trên, và đây là danh sách đầy đủ vì mỗi chương cấp một dòng. [A/05-lan-truyen-sai-so-pixel-sang-pose](#) cấp bốn quan hệ tỉ lệ và quy tắc cộng — nếu chỉ đọc hai chương thì đọc chương ấy và chương này. [A/01-mo-hinh-camera-va-distortion](#) cấp dòng méo và dòng điểm chính, và mục 5 ở đó giải thích vì sao dòng điểm chính lớn đến thế. [A/02-homography-va-hinh-hoc-phang](#) mục 6 cấp dòng bias tâm ellipse. [A/04-pose-ambiguity-target-phang](#) cấp *không* dòng nào — và mục 6 ở đây giải thích vì sao. [A/06-hieu-ung-vat-ly-va-quang-hoc](#) cấp hai dòng phụ thuộc vận tốc.

Từ Phần B: **B/08** cấp σ ; **B/09** mục 5 cấp dòng constellation và mục 3 cấp B ; **B/12** cấp ba dòng hiệu chuẩn và thứ tự đo chúng; **B/13** quyết định *chế độ*, tức quyết định bảng nào được dùng; **B/14** cấp dòng loa và cảnh báo phải dùng σ tệ nhất.

Nổi ngang. **C/16-danh-gia-va-nghiem-thu** là chương anh em và nó bổ sung chỗ chương này thiếu: bảng tính tại *một* tư thế, Monte Carlo quét *toàn dải*; bảng dùng các con số bạn cung cấp, nghiệm thu kiểm xem chúng có đúng không. Hai chương phải đọc cùng nhau, và thứ tự đúng là chương này trước để biết đo cái gì, rồi chương kia để đo.

Nổi xuống. **E/19-lo-trinh-toi-thieu** dùng bảng này làm tiêu chí quyết định ở nhiều bước: bước nào đáng làm tiếp phụ thuộc vào dòng nào đang lớn. Và một lời khuyên về thứ tự đọc lặp lại lần cuối: *chương này nên đọc sớm*, vì nó là chương duy nhất cho bạn biết nên đầu tư vào đâu *trước* khi bạn đã đầu tư.

Câu hỏi tự kiểm (trả lời không nhìn bài)

1. Nêu ba loại dòng trong ngân sách, quy tắc cộng của từng loại, và phép kiểm phân loại bằng hai câu hỏi.
2. Với $L = 0,8$ m, ngân sách góc $0,5^\circ$ tương đương bao nhiêu milimét tại điểm cập? Tính, rồi nói hệ quả cho việc chọn chỗ gắn camera.
3. Trong bảng ví dụ, nhóm hệ thống chiếm bao nhiêu phần trăm tổng? Nêu hệ quả cho việc phân bổ công sức.
4. Vì sao cải thiện dòng nhỏ nhất bốn lần gần như không đổi gì? Trả lời bằng phép tính cụ thể.
5. Teach-by-showing loại những dòng nào và giữ những dòng nào? Nêu tỉ số giữa hai tổng trong ví dụ.
6. Vì sao pose ambiguity không có dòng trong bảng, và nó được xử lý ở đâu?
7. Nêu ba lý do một tổng dưới ngưỡng không đảm bảo hệ sẽ đạt, và nêu lý do nào bạn cho là nguy hiểm nhất.

Đánh giá và nghiệm thu (Evaluation and Acceptance)

PHỤ THUỘC C/15-ngan-sach-sai-so (bảng ngân sách — chương này là chương kiểm nó; và mọi mini project của Phần B, mà chương này gom lại thành một quy trình duy nhất.

TÓM TẮT

Chương này trả lời câu hỏi mà C/15 để ngỏ: *làm sao biết bảng ngân sách nói đúng?* Hai công cụ, rất khác nhau về chi phí. Công cụ thứ nhất là mô phỏng Monte Carlo — rẻ, làm được trước khi có phần cứng, và nó là của kiểm chứng (b) cho toàn bộ cây. Script ấy nằm trong code/ cạnh chương này, đã được chạy, và mục 3 báo cáo kết quả thật của nó — gồm cả một chỗ mà kết quả đi ngược dự đoán của tôi. Công cụ thứ hai là nghiệm thu trên phần cứng: cần chuẩn tham chiếu, và nó phải tách bạch *bias* khỏi *độ lặp lại* theo đúng chuẩn robot công nghiệp. Nếu chỉ nhớ một điều: *độ tản mát của một hệ có làm vượt không phải độ chính xác của nó* — lấy trung bình giảm tản mát mà không giảm thiên lệch, nên một hệ thống thiên lệch 3 mm có thể báo cáo độ lặp lại 0,2 mm và trông tuyệt vời.

KIẾN THỨC CẦN CÓ TRƯỚC

- **Monte Carlo** — mô phỏng bằng cách sinh nhiều ngẫu nhiên nhiều lần và thống kê phân bố kết quả. Nó cho biết *kết quả là bao nhiêu* chứ không cho biết *vì sao* — nên nó bổ sung chứ không thay thế công thức giải tích ở A/05.
- **chuẩn tham chiếu (ground truth)** — một phép đo độc lập, chính xác hơn hệ đang đánh giá ít nhất vài lần, dùng làm mốc so sánh.
- **bias (độ chệch)** — sai lệch trung bình so với giá trị thật. Đo được chỉ khi có chuẩn tham chiếu.
- **độ lặp lại (repeatability)** — độ tản mát giữa các lần thực hiện cùng một lệnh. Đo được *không cần* chuẩn tham chiếu — và sự bất đối xứng ấy là nguồn của một cạm bẫy lớn ở mục 5 [3].
- **NEES — normalized estimation error squared**: $\varepsilon = \tilde{x}^\top P^{-1} \tilde{x}$. Nếu hiệp phương sai P trung thực thì kỳ vọng của nó bằng số bậc tự do. Nó đo *tính thành thật* của hệ về bất định của chính nó, không đo độ chính xác.
- **cửa kiểm chứng (b)** — theo learning-rules.md mục 5: kiểm một công thức bằng một ví dụ số nhỏ, tính tay hoặc bằng vài dòng code. Rẻ nhất và bắt được phần lớn lỗi.

CÂU HỎI MỞ ĐẦU

- Q1. Bạn đo hệ thống 50 lần và độ tản mát là 0,2 mm. Vì sao con số ấy *không* cho phép nói hệ chính xác 0,2 mm, và phép đo nào mới cho phép?
- Q2. Mô phỏng Monte Carlo cho bạn kết quả mà không cần phần cứng. Nêu hai loại sai số mà nó *không bao giờ* phát hiện được, dù chạy bao nhiêu lần.
- Q3. Chạy script kiểm công thức $\delta x \approx Z\sigma/f$, bạn thấy kết quả mô phỏng nhỏ hơn dự đoán đúng hai lần một cách nhất quán. Đó là lỗi của công thức, lỗi của script, hay không phải lỗi nào cả?
- Q4. Một hệ báo cáo hiệp phương sai 0,3 mm nhưng sai số thật là 1,5 mm. Đại lượng nào phát hiện được điều đó, và vì sao nó quan trọng hơn bản thân độ chính xác?
- Q5. Bạn đã chạy mô phỏng và mọi thứ khớp. Nêu ba việc mà bạn *vẫn* phải làm trên phần cứng trước khi nói hệ đạt.

16.1 Đặt vấn đề

C/15 kết thúc bằng một bảng và một con số tổng. Câu hỏi tự nhiên tiếp theo là câu hỏi mà chương ấy không trả lời được: *làm sao biết bảng nói đúng?*

Bảng ngân sách là một mô hình, và nó thừa hưởng mọi điểm yếu của một mô hình. Nó cộng những thứ bạn đã nghĩ tới theo những quy tắc bạn đã chọn, với những con số bạn đã ước lượng. Ba chỗ có thể sai, và C/15 mục 8 đã liệt kê chúng. Chương này là chương đi kiểm.

Có hai cách kiểm, và chúng khác nhau về chi phí tới mức nên coi là hai công cụ riêng.

Cách thứ nhất: mô phỏng. Sinh nhiều, chạy pipeline, thống kê. Rẻ, nhanh, làm được trước khi có phần cứng, và lặp lại được vô hạn với các cấu hình khác nhau. Nó kiểm được *tính đúng của các công thức và ảnh hưởng của hình học* — tức phần lớn nội dung của Phần A và các quyết định thiết kế của B/09.

Cách thứ hai: đo trên phần cứng thật. Đắt, chậm, cần chuẩn tham chiếu. Nhưng nó là cách duy nhất kiểm được những thứ mà mô phỏng theo định nghĩa không thể: liệu mô hình nhiều có đúng không, liệu có nguồn sai số nào bạn chưa nghĩ tới không, và liệu các con số bạn nhập vào bảng có đúng không.

Sự phân công ấy dẫn tới một quy trình rõ ràng, và nó là khung của chương: *mô phỏng trước để chọn thiết kế, đo thật sau để nghiệm thu.* Làm ngược — dựng phần cứng rồi mới mô phỏng — thì mô phỏng chỉ còn để giải thích chuyện đã rồi.

Chương này có một đặc điểm mà không chương nào khác trong cây có: *script của nó đã được chạy, và mục 3 báo cáo số đo thật.* Theo `learning-rules.md` mục 5, đó là cửa kiểm chứng (b), và cho tới lúc ấy mọi công thức trong cây mới chỉ qua cửa (a) — tự dẫn — chứ chưa qua (b). Mục 3 cũng báo cáo thẳng một chỗ mà kết quả *đi ngược* dự đoán của tôi, vì che nó đi sẽ phá hỏng chính điều làm cây này đáng tin.

TRỰC GIÁC

Hình dung sự khác nhau giữa hai công cụ bằng việc kiểm một công thức nấu ăn.

Mô phỏng giống việc đọc lại công thức và kiểm số học: nếu công thức nói bốn người ăn cần 200 gam gạo, thì tám người cần 400 — bạn kiểm được điều đó mà không cần bếp. Nó bắt được lỗi tính toán, lỗi logic, và nó cho bạn thử nhiều biến thể rất nhanh.

Cái nó không bao giờ bắt được: công thức có ghi thiếu một nguyên liệu không. Nếu người viết quên mất muối, thì dù bạn kiểm số học kỹ tới đâu, món ăn vẫn nhạt — vì muối không có trong bảng để mà kiểm.

Nấu thật thì bắt được cả hai loại lỗi, nhưng nó tốn nguyên liệu và thời gian, và mỗi lần chỉ thử được một biến thể.

Nên thứ tự hợp lý rất rõ: kiểm số học trước cho hết mọi biến thể, rồi mới nấu cái tốt nhất. Và khi nấu, nếu món ăn không giống dự đoán, thì cái sai gần như luôn nằm ở một nguyên liệu không có trong bảng — chứ không nằm ở phép nhân.

Còn một chuyện thứ ba, tinh tế hơn cả hai. Giả sử bạn nấu năm lần và cả năm lần đều *giống hệt nhau*. Điều đó chứng minh gì? Nó chứng minh bạn nấu ổn định — và không chứng minh gì về việc món ăn có ngon không. Nếu công thức thiếu muối thì cả năm lần đều nhạt như nhau, rất ổn định, và không có gì trong sự ổn định ấy báo cho bạn biết.

16.2 Mô phỏng Monte Carlo: công cụ và cách dùng

Script nằm ở `code/monte_carlo_constellation.py` cạnh chương này. Nó cần `numpy` và `opencv-python`, không cần phần cứng, và chạy trong vài phút.

Nó làm bốn việc, tương ứng bốn mệnh đề quan trọng nhất của cây.

Phần 1 kiểm bốn quan hệ tỉ lệ của [A/05](#) bằng cách quét Z và quét baseline B .

Phần 2 kiểm luận điểm của [A/04](#): target phẳng có hai nghiệm, tỉ số ρ tiến về 1 khi góc nghiêng tiến về 0, và tính không đồng phẳng khử được hiện tượng ấy.

Phần 3 so PnP hợp nhất với trung bình pose — mệnh đề của [A/03](#) mục 5 và [B/09](#) mục 2.

Phần 4 so bốn phương án bố trí constellation — mini project của [B/09](#).

A. Một quy ước phải làm đúng, và một cái bẫy

Trong quá trình viết script tôi mắc một lỗi đáng ghi lại vì nó là lỗi dễ mắc và nó làm mọi con số sai số góc trở nên vô nghĩa.

Có hai chỗ có thể đặt một góc nghiêng: trong mô hình 3D của constellation (tức cái nêm cơ khí ở [B/09](#) mục 4), hoặc trong tư thế camera R_{true} . Chúng là hai thứ khác hẳn nhau về vật lý, và nếu bạn đặt góc nghiêng vào mô hình rồi so R_{true} với nghiệm, bạn đang so hai hệ quy chiếu khác nhau — phép so cho ra đúng góc nghiêng của mô hình ở mọi lần chạy, và trông như một sai số khổng lồ không đổi.

Quy ước đúng, và nó được ghi thẳng vào docstring của hàm `tag_corners`: góc nghiêng của tag so với tấm dock nằm ở mô hình; góc nhìn của camera nằm ở pose. Khi kiểm các công thức của [A/05](#) cho một tag đơn, mô hình phải phẳng và góc nghiêng phải nằm ở pose.

Tôi ghi lại chuyện này thay vì lặng lẽ sửa, vì nó minh hoạ một điểm chung của mọi mô phỏng: cái khó không nằm ở thuật toán mà ở việc giữ đúng hệ quy chiếu, và một mô phỏng sai hệ quy chiếu cho ra những con số trông hợp lý.

B. Đại lượng phải báo cáo: từng trục, không phải một con số

Lỗi thứ hai tôi mắc, và nó cũng đáng ghi lại. Ban đầu script báo cáo sai số xoay bằng một con số — góc của phép quay đưa nghiệm về chân trị. Với con số ấy, kết quả quét baseline trông như bác bỏ công thức của [A/05](#): tăng B tám lần chỉ cải thiện 2,7 lần thay vì 8 lần.

Nguyên nhân không phải công thức sai mà là cách đo sai. Như [B/09](#) mục 3 đã lập luận, baseline có hướng: trải rộng theo phương ngang cải thiện yaw và roll, và không cải thiện pitch. Một con số góc vô hướng bị trục tẻ nhất — pitch — chi phối, nên nó che mất toàn bộ cải thiện ở hai trục kia.

Script hiện tại báo cáo cả ba trục riêng, và mục 3 cho thấy kết quả đối hẳn. Bài học tổng quát, và nó áp dụng cho cả việc nghiệm thu phần cứng ở mục 4: đừng gộp một đại lượng có hướng thành một con số vô hướng, vì phép gộp che mất đúng cấu trúc mà bạn đang muốn kiểm.

16.3 Kết quả đo được

Mục này báo cáo kết quả thật của script, chạy ngày 2026-09-14 với 3000 lần thử mỗi cấu hình. Log đầy đủ ở `code/ket-qua-chay-2026-09-14.txt`.

CẠM BÃY

Đây là số đo của một **mô phỏng**, không phải của một hệ phần cứng. Chúng kiểm được rằng các công thức của cây nhất quán với hành vi của solvePnP dưới nhiều Gauss độc lập trên toạ độ góc. Chúng *không* kiểm được rằng nhiều thật là Gauss, rằng nhiều thật độc lập, hay rằng không có nguồn sai số nào bị bỏ sót — mục 5 bàn về điều đó.

Sự phân biệt này được giữ nghiêm ngặt trong cả chương, và nó là sự phân biệt mà `research-rules.md` mục 9 gọi là “phân biệt đã đo được với suy ra từ số liệu”.

A. Bốn quan hệ tỉ lệ: khớp

Bảng 16.1. Quét Z với một tag 100 mm, camera nhìn chéo 20° , $\sigma = 0,2$ px. Số đo mô phỏng so với dự đoán giải tích của A/05.

Z (m)	δx đo	δx dự đoán/2	δZ đo	δZ dự đoán	$\delta Z/\delta x$ so với Z/W
0,30	0,056	0,050	0,254	0,300	4,6 so với 3
0,50	0,087	0,083	0,786	0,833	9,1 so với 5
1,00	0,169	0,167	3,279	3,333	19,4 so với 10
2,00	0,370	0,333	12,999	13,333	35,1 so với 20

Hai cột δZ khớp rất tốt — sai lệch dưới 15% ở mọi Z , và tốt hơn 3% ở $Z \geq 1$ m. Công thức $\delta Z \approx Z\sigma/w$ của A/05 mục 3B **qua cửa (b)**.

Hai cột δx cũng khớp, nhưng *chỉ sau khi chia dự đoán cho 2* — và đây là câu trả lời cho Q3. Hệ số 2 chính là $\sqrt{N}$ với $N = 4$ góc. A/05 mục 3A đã cố ý bỏ hệ số $1/\sqrt{N}$ ra khỏi công thức, với lý do được ghi rõ ở đó: nhiều giữa các góc cùng một tag không độc lập trong thực tế, nên $\sqrt{N}$ là một lời hứa thường không được giữ, và bỏ nó ra là cách ước lượng thận trọng. Trong mô phỏng, nhiều *đúng* là độc lập theo xây dựng, nên $\sqrt{N}$ xuất hiện đầy đủ.

Vậy đây không phải lỗi của công thức, cũng không phải lỗi của script — đó là *sự chênh lệch có chủ ý, đúng như dự kiến*. Nó là một xác nhận chứ không phải một bất đồng, và nó cho thấy giá trị của việc ghi rõ lý do bỏ một hệ số thay vì lặng lẽ bỏ.

Tỉ số $\delta Z/\delta x$ lớn hơn Z/W đúng khoảng hai lần ở mọi Z — hệ quả trực tiếp của cùng hệ số $\sqrt{N}$, vì nó chỉ áp vào tử số. Điều đáng kiểm là *độ dốc*, và nó đúng: tỉ số tăng tuyến tính theo Z .

B. Ambiguity: khớp mạnh

Bảng 16.2. Tỉ số ambiguity ρ và sai số góc theo góc nghiêng, một tag 100 mm ở $Z = 0,5$ m. Số đo mô phỏng.

Nghiêng ($^\circ$)	ρ trung vị	Sai số góc ($^\circ$)	Độc
0	1,33	1,34	hai nghiệm không phân biệt được
2	1,79	1,32	
5	4,76	1,15	
10	13,8	0,86	
20	33,0	0,51	một nghiệm thẳng áp đảo
40	66,6	0,28	

Đây là xác nhận mạnh nhất trong cả chương. ρ tăng từ 1,33 ở fronto-parallel lên 66,6 ở 40° — một dải năm mươi lần — và sai số góc giảm đơn điệu theo chiều ngược lại. Cả hai mệnh đề của A/04 mục 4 qua cửa (b): ambiguity tệ nhất ở fronto-parallel, và ρ là một chỉ báo dùng được.

Đáng chú ý: ở 0° , sai số góc là $1,34^\circ$ — gấp gần ba lần ngân sách $0,5^\circ$ của cây, với một tag đơn 100 mm ở nửa mét trong điều kiện nhiễu lý tưởng. Con số ấy là minh họa định lượng cho luận điểm ở 00-map rằng một tag phẳng nhìn vuông góc không đạt được mục tiêu.

C. Một kết quả đi ngược dự đoán

Phần khử ambiguity bằng tính không đồng phẳng cho kết quả yếu hơn nhiều so với tôi dự đoán, và tôi báo cáo nó thẳng.

Bảng 16.3. Bốn tag, baseline ngang 0,22 m, camera nhìn thẳng trục. Số đo mô phỏng, sai số xoay theo từng trục (độ).

Cấu hình	pitch	yaw	roll	tổng
4 tag đồng phẳng	0,154	0,113	0,018	0,192
4 tag nghiêng 20°	0,124	0,090	0,019	0,154

Cải thiện chỉ 1,24 lần. B/09 mục 4 ngụ ý rằng phá tính đồng phẳng là bước có đòn bẩy rất lớn; con số này nói rằng — trong cấu hình cụ thể đã mô phỏng — nó chỉ cho khoảng 24%.

Ba cách giải thích, và tôi chưa phân biệt được chúng.

Một, cấu hình đã đủ tốt sẵn. Bốn tag trải trên baseline 0,22 m đã có nhiều thông tin hình học, nên sai số góc đã ở mức $0,19^\circ$ — trong ngân sách $0,5^\circ$ rồi. Khi đã ở vùng tốt, lợi ích của việc phá đồng phẳng nhỏ lại. Điều này nhất quán với dữ liệu: phần 4 cho thấy một tag đơn 150 mm có sai số $0,43^\circ$, và đó mới là chỗ cần cứu.

Hai, lợi ích thật của không đồng phẳng nằm ở chỗ khác. A/04 lập luận rằng nó khử lỗi thô — việc chọn nhầm nghiệm — chứ không chủ yếu giảm nhiễu. Bảng trên đo độ tản mát, tức đo nhiễu. Nếu lợi ích chính là loại lỗi thô thì nó phải đo bằng tỉ lệ chọn nhầm nghiệm, và script hiện chưa đo đại lượng ấy cho cấu hình nhiều tag.

Ba, độ nghiêng chưa đủ. B/09 mục 4B khuyến nghị độ lệch khỏi mặt phẳng ít nhất 20% kích thước tag, và ghi rõ rằng con số ấy chưa qua cửa kiểm chứng nào. Với tag 75 mm nghiêng 20° , độ lệch là $75 \sin 20^\circ = 26$ mm — trên ngưỡng, nhưng ngưỡng ấy là phỏng đoán.

Cách phân biệt ba giả thuyết là một thí nghiệm cụ thể: đo tỉ lệ chọn nhầm nghiệm thay vì độ tản mát, và quét độ nghiêng từ 0° tới 45° . Tôi chưa làm, và nó đã được ghi vào 99-chưa-biết.md như mục đầu tiên.

Điều đáng nói về phương pháp: kết quả này không bác bỏ B/09 mục 4 — nó cho thấy phép đo tôi chọn không đo đúng thứ mà mệnh đề nói về. Đó là một loại sai lầm khác với “mệnh đề sai”, và phân biệt hai loại ấy là việc phải làm trước khi sửa bất cứ chương nào.

D. Baseline có hướng: khớp, và nó cứu một mệnh đề

Bảng 16.4. Quét baseline ngang, hai tag 100 mm ở $Z = 0,5$ m. Số đo mô phỏng, sai số xoay theo từng trục (độ).

B (m)	pitch	yaw	roll	yaw dự đoán
0,10	0,189	0,124	0,041	0,096
0,20	0,175	0,097	0,028	0,048
0,40	0,130	0,060	0,016	0,024
0,80	0,077	0,037	0,008	0,012

Đọc theo trục thì bức tranh rõ. Từ $B = 0,1$ tới $0,8$ m — tám lần — yaw cải thiện $0,124/0,037 = 3,4$ lần và roll cải thiện $0,041/0,008 = 4,9$ lần, trong khi pitch chỉ cải thiện 2,5 lần. Đúng như [B/09](#) mục 3 dự đoán: baseline ngang cải thiện yaw và roll, và nó không cải thiện pitch vì baseline *đứng* không đổi.

Cải thiện chưa đạt tuyến tính hoàn toàn (3,4 thay vì 8 lần cho yaw), và lý do nhất quán với chính bảng: khi yaw đã tốt lên thì các trục khác trở thành ràng buộc, và bài toán hợp nhất phân bổ lại thông tin. Đây là một hiệu ứng mà công thức vô hướng của [A/05](#) mục 6 không mô hình hoá — nó giả định một trục độc lập.

Kết luận về mức kiểm chứng, và tôi phát biểu nó thận trọng: quan hệ $\delta\theta \propto 1/B$ **được ủng hộ về chiều và về bậc** nhưng *chưa* được xác nhận là tuyến tính chính xác. Chỗ chưa khớp có giải thích hợp lý, và việc kiểm nó đầy đủ đòi một mô phỏng tách bạch từng trục với baseline theo cả hai phương — đã ghi vào 99-chua-biet.md.

E. Bốn phương án bố trí

Bảng 16.5. Bốn phương án ở $Z = 0,5$ m, $\sigma = 0,2$ px. Số đo mô phỏng.

Phương án	δx (mm)	δZ (mm)	pitch	yaw	roll
A. một tag 150 mm	0,119	0,384	0,431	0,429	0,045
B. 4 tag đồng phẳng, $B = 0,22$	0,056	0,158	0,161	0,111	0,018
C. 4 tag nghiêng 20°	0,050	0,160	0,124	0,089	0,018
D. C + tag nhô ra 60 mm	0,037	0,162	0,064	0,049	0,018

Bảng này là kết quả có giá trị thực hành cao nhất của chương, và ba điều đọc ra từ nó.

Một, phương án A không đạt ngân sách góc trong điều kiện lý tưởng nhất. Một tag 150 mm — lớn hơn nhiều so với tag trong các phương án khác — cho pitch và yaw quanh $0,43^\circ$, tức 86% ngân sách $0,5^\circ$ tiêu vào riêng nhiều góc, chưa tính một dòng nào khác của [C/15](#). Đây là xác nhận định lượng cho luận điểm trung tâm của cây.

Hai, chuyển từ A sang B cải thiện góc gần bốn lần, và nó chỉ là việc thay một tag lớn bằng bốn tag nhỏ trải rộng. Không dòng code nào khác.

Ba, và đây là kết quả mạnh nhất: phương án D cải thiện thêm hai lần nữa so với C. Tag nhô ra 60 mm — tức phá tính đồng phẳng theo chiều sâu thay vì bằng nệm nghiêng — cho pitch $0,064^\circ$ so với $0,124^\circ$ của C. Đây là bằng chứng ủng hộ [B/09](#) mục 4A cách thứ hai (tag ở các độ sâu khác nhau), và nó gợi ý rằng cách ấy *mạnh hơn* nệm nghiêng — một kết luận mà [B/09](#) không nêu và tôi chưa có lý thuyết giải thích.

Tổng hợp: từ A tới D, sai số góc cải thiện $0,431/0,064 = 6,7$ lần, hoàn toàn bằng hình học.

16.4 Nghiệm thu trên phần cứng

Mô phỏng không thay thế được đo thật, và mục này nói cần gì.

A. Chuẩn tham chiếu

Bốn lựa chọn, theo thứ tự chi phí giảm dần.

Máy toàn đạc hoặc laser tracker. Chính xác nhất, đắt nhất, và thường phải thuê. Dùng khi cần nghiệm thu chính thức.

Hệ motion capture. Tốt cho quỹ đạo động, cần lắp đặt.

Đồ gá cơ khí có dung sai biết trước. Rẻ và thường đủ: một tấm có các lỗ định vị gia công chính xác, robot được đặt vào từng lỗ. Nó cho các vị trí *tương đối* rất chính xác mà không cần thiết bị đo đắt tiền — và với bài toán docking, quan hệ tương đối là thứ ta cần.

Chính cơ cấu cấp. Rẻ nhất và thường bị bỏ qua: nếu robot cấp được và chốt khớp, thì bạn biết nó đang ở đúng vị trí trong phạm vi dung sai của cơ cấu. Đây là một chuẩn tham chiếu nhị phân — đạt hay không đạt — nhưng với nghiệm thu thì đó chính là tiêu chí cuối cùng.

B. Tách bias khỏi độ lặp lại

Đây là yêu cầu phương pháp luận quan trọng nhất của mục, và nó trả lời Q1.

Tiêu chuẩn robot công nghiệp phân biệt rõ *độ chính xác* (accuracy, gồm cả bias) và *độ lặp lại* (repeatability, chỉ độ tản mát) [3], và hai đại lượng ấy đòi hai phép đo khác nhau:

Độ lặp lại đo được *không cần* chuẩn tham chiếu: cho robot cấp nhiều lần và đo độ tản mát giữa các lần.

Bias đo được *chỉ khi có* chuẩn tham chiếu: so vị trí trung bình với vị trí thật.

Sự bất đối xứng ấy là nguồn của cạm bẫy lớn nhất trong nghiệm thu, và nó là câu trả lời cho Q1: vì độ lặp lại dễ đo hơn nhiều, người ta đo nó, rồi báo cáo nó như thể nó là độ chính xác. Một hệ thống có bias 3 mm và độ lặp lại 0,2 mm sẽ được báo cáo là “chính xác 0,2 mm” — và con số ấy đúng theo nghĩa đen của từ “lặp lại” mà sai hoàn toàn nếu người đọc hiểu là độ chính xác.

Với docking thì — như B/13 lập luận — *độ lặp lại mới là đại lượng quan trọng*, nên đo nó là hợp lý. Điều bắt buộc là *gọi đúng tên*, và báo cáo rõ rằng bias chưa được đo nếu chưa đo.

CẠM BẦY

Một hệ thống có *làm mượt* (B/11) báo cáo độ tản mát nhỏ hơn thực chất, vì làm mượt giảm tản mát mà không giảm thiên lệch.

Hệ quả: đo độ lặp lại của một hệ có bộ lọc rồi gọi nó là độ chính xác là sai *hai lần* — một lần vì nhầm hai đại lượng, một lần nữa vì bộ lọc đã làm đẹp chính đại lượng đang đo.

Cách đo đúng: đo độ lặp lại của *toàn bộ chu trình cấp dock*, tức cho robot lùi ra hẳn rồi cấp lại từ đầu, chứ không đo độ tản mát của pose trong khi robot đứng yên ở vị trí cuối. Hai con số ấy khác nhau nhiều, và chỉ con số thứ nhất có nghĩa với người dùng.

C. Bản đồ sai số theo tư thế

C/15 mục 8 cảnh báo rằng bảng ngân sách tính tại *một* tư thế. Nghiệm thu phải quét toàn dải: đo sai số ở nhiều khoảng cách và nhiều góc tiếp cận, rồi vẽ bản đồ.

Cái bản đồ ấy cho hai thứ mà một con số duy nhất không cho: nó chỉ ra *vùng* nào của không gian tư thế có vấn đề, và nó cho thấy sai số có *cấu trúc* hay không. Sai số có cấu trúc — chẳng hạn tăng đều theo một hướng — là dấu hiệu của một nguồn hệ thống chưa được nhận diện, và nó là manh mối chẩn đoán tốt nhất mà nghiệm thu cung cấp.

D. Thử nghiệm độ bền

Bốn yếu tố, lấy từ các chương tương ứng: ánh sáng (B/14 mini project), nhiệt độ (B/12 mục 6), rung, và marker bắn hoặc bị che một phần. Với mỗi yếu tố, đo lại σ và — quan trọng hơn — đo lại *vị trí trung bình*, vì thay đổi ở vị trí trung bình là thiên lệch và nó không được các biện pháp làm mượt chạm tới.

I 16.5 Kiểm tính nhất quán của hiệp phương sai

Mục này trả lời Q4, và nó bàn một đại lượng ít được đo nhưng quan trọng.

Mọi thứ trong cây này sinh ra không chỉ một pose mà cả một hiệp phương sai (A/03 mục 4B). Câu hỏi: hiệp phương sai ấy có *trung thực* không?

Đại lượng đo là NEES: với sai số thật $\tilde{x}$ và hiệp phương sai khai báo P , tính $\varepsilon = \tilde{x}^\top P^{-1} \tilde{x}$. Nếu P trung thực thì kỳ vọng của ε bằng số bậc tự do của $\tilde{x}$ — sáu với một pose đầy đủ.

Đọc kết quả: NEES trung bình lớn hơn 6 nhiều nghĩa là hệ *quá tự tin* — nó khai một bất định nhỏ hơn sai số thật. Nhỏ hơn 6 nhiều nghĩa là quá bi quan.

Vì sao điều này quan trọng hơn bản thân độ chính xác — phần hai của Q4: vì mọi tầng phía trên dùng hiệp phương sai làm trọng số. Bộ lọc ở B/11 dùng nó để cân các phép đo; cổng chi-square ở B/09 mục 7 dùng nó để loại ngoại lai; tiêu chí hội tụ ở B/13 mục 8 dùng nó để quyết định khi nào dừng. Một hiệp phương sai quá tự tin làm cả ba tầng ấy sai theo chiều nguy hiểm: bộ lọc tin quá mức vào phép đo xấu, cổng loại nhầm phép đo tốt, và bộ điều khiển dừng khi chưa đạt.

Và có một mối nối quan trọng với A/05 mục 2: công thức hiệp phương sai ở đó *giả định mô hình đúng*. Nếu có sai số hệ thống chưa được mô hình hoá — méo còn sót, constellation lệch — thì phần dư mang thành phần hệ thống và hiệp phương sai báo về nhỏ hơn sự thật. Nói cách khác: *NEES cao là một chỉ báo của sai số mô hình*, và đó là cách dùng có giá trị nhất của nó trong cây này.

Cạm bẫy khi dùng NEES: nó giả định mô hình *đúng dạng* và chỉ sai độ lớn. Nếu bạn thấy NEES cao và phản ứng bằng cách nới hiệp phương sai lên cho tới khi NEES về 6, bạn đã giấu một lỗi mô hình vào trọng số. Phép thử phân biệt: kiểm xem phần dư có *tương quan với tư thế* không — nếu có thì đó là lỗi mô hình, không phải lỗi trọng số.

I 16.6 Giới hạn của mô phỏng

Mục này trả lời Q2, và nó là mục bắt buộc phải đọc trước khi tin kết quả mục 3.

Mô phỏng không phát hiện được nguồn sai số bị bỏ sót. Nó mô phỏng chính xác mô hình bạn đã viết, nên nếu mô hình thiếu một hiện tượng thì mô phỏng cũng thiếu. Mọi con số ở mục 3 giả định: nhiễu Gauss, độc lập giữa các góc, không có méo, constellation đúng chính xác, không có rolling shutter, không có blur, không có loá. Tức là mô phỏng *chỉ* kiểm được các dòng *ngẫu nhiên*

của C/15, và hoàn toàn im lặng về các dòng hệ thống — vốn chiếm 72% ngân sách trong ví dụ của chương ấy.

Mô phỏng không kiểm được mô hình nhiễu. Nó giả định Gauss độc lập vì đó là điều tôi đã viết vào script. A/05 mục 9 liệt kê ba cách giả định ấy bị vi phạm trong thực tế, và không cách nào xuất hiện trong mô phỏng.

Hai điểm trên là câu trả lời cho Q2, và chúng dẫn tới một phát biểu gọn: *mô phỏng kiểm được phép tính, không kiểm được danh sách*.

Kết quả phụ thuộc vào σ giả định. Mọi con số ở mục 3 dùng $\sigma = 0,2$ px — một bậc độ lớn kinh nghiệm, không phải số đo (00-map). Với σ thật của hệ bạn, mọi con số cơ giãn tuyến tính, nên chiều của các kết luận không đổi nhưng độ lớn thì đổi.

Ba chỗ trong chính kết quả chưa được kiểm đầy đủ, đã ghi rõ: mức cải thiện của không đồng phẳng (mục 3C), tính tuyến tính chính xác của quan hệ $1/B$ (mục 3D), và vì sao tag nhô ra theo chiều sâu lại mạnh hơn nê-m nghiêng (mục 3E).

16.7 Thực hành: quy trình sáu bước

Gom cả chương thành một quy trình, và nó trả lời Q5.

Một, chạy mô phỏng trước khi gia công. Script đã có, đã chạy, và nó nhận cấu hình của bạn qua vài hằng số ở đầu tệp. Một buổi chiều.

Hai, đo σ thật (B/07 mini project) và chạy lại mô phỏng với con số ấy.

Ba, đo bốn con số đầu vào của ngân sách (C/15 mục 3B) và tính lại bảng.

Bốn, đo độ lặp lại của toàn bộ chu trình cập dock — lùi ra hẳn rồi cập lại, ít nhất 30 lần, với tiêu chí đạt/không đạt bằng chính cơ cấu.

Năm, vẽ bản đồ sai số theo tư thế và kiểm xem sai số có cấu trúc không.

Sáu, thử nghiệm độ bền ở bốn điều kiện của mục 4D.

Ba việc trong danh sách này là ba việc mà mô phỏng không thay được — câu trả lời cho Q5: đo σ thật (vì mô phỏng giả định nó), đo độ lặp lại toàn chu trình (vì mô phỏng không có bộ điều khiển, không có cơ khí, không có bánh xe trượt), và thử nghiệm độ bền (vì mô phỏng không có ánh sáng, nhiệt độ, bụi).

Active recall (che cột phải, tự trả lời, rồi mở ra đối chiếu)

Thuật ngữ / câu hỏi	Trả lời
••• Hai công cụ, hai vai trò	Mô phỏng: rẻ, trước khi có phần cứng, kiểm tính đúng của công thức và ảnh hưởng của hình học. Đo thật: đắt, cần chuẩn tham chiếu, kiểm mô hình nhiễu và các nguồn bị bỏ sót. Thứ tự: mô phỏng chọn thiết kế, đo thật nghiệm thu.
••• Mô phỏng không kiểm được gì	Nguyên bị bỏ sót (nó mô phỏng đúng mô hình bạn viết) và mô hình nhiễu (nó giả định Gauss độc lập vì bạn viết thế). Gọn: <i>mô phỏng kiểm được phép tính, không kiểm được danh sách</i> .
••• Cái bẫy hệ quy chiếu	Góc nghiêng của tag so với dock nằm ở mô hình 3D; góc nhìn của camera nằm ở pose. Trộn hai thứ thì phép so góc so hai hệ khác nhau và cho ra một “sai số” khổng lồ không đổi. Lỗi tôi đã mắc, ghi lại ở mục 2A.

Thuật ngữ / câu hỏi	Trả lời (tiếp)
●●● Vì sao phải báo cáo từng trục	Baseline có hướng (B/09 mục 3). Một con số góc vô hướng bị trục tẻ nhất chỉ phôi và che mất cải thiện ở các trục khác — ban đầu nó làm kết quả trông như bác bỏ A/05. Bài học: đừng gộp đại lượng có hướng thành vô hướng.
●●● Kết quả: $\delta Z \approx Z\sigma/w$	Khớp, sai lệch dưới 15% ở mọi Z và dưới 3% ở $Z \geq 1$ m. Qua cửa (b).
●●● Kết quả: δx nhỏ hơn dự đoán đúng 2 lần	Không phải lỗi. Hệ số là $\sqrt{N}$ với $N = 4$ góc; A/05 mục 3A đã cố ý bỏ nó ra cho thận trọng, với lý do ghi rõ. Mô phỏng có nhiều độc lập thật nên $\sqrt{N}$ xuất hiện đầy đủ.
●●● Kết quả: ambiguity	Khớp mạnh. ρ tăng từ 1,33 (fronto-parallel) lên 66,6 (40°) — dài năm mươi lần; sai số góc giảm đơn điệu. Ở 0°: sai số 1,34°, gấp gần ba lần ngân sách.
●●● Kết quả ĐI NGƯỢC dự đoán	Phá đồng phẳng bằng nêm nghiêng chỉ cải thiện 1,24 lần, không phải nhiều lần. Ba giải thích chưa phân biệt được: cấu hình đã đủ tốt sẵn; lợi ích thật nằm ở loại lỗi thô chứ không giảm nhiễu (và tôi đo nhầm đại lượng); hoặc độ nghiêng chưa đủ.
●●● Loại sai lầm của kết quả ấy	Nó không bác bỏ B/09 mục 4 — nó cho thấy phép đo tôi chọn (độ tán mát) không đo đúng thứ mệnh đề nói về (lỗi thô). Phân biệt “mệnh đề sai” với “đo nhầm đại lượng” là việc phải làm trước khi sửa chương nào.
●●● Kết quả: bốn phương án bố trí	Một tag 150 mm cho pitch/yaw $\approx 0,43^\circ$ — 86% ngân sách tiêu riêng vào nhiều góc. Từ A tới D cải thiện 6,7 lần, hoàn toàn bằng hình học.
●● Kết quả bất ngờ ở phương án D	Tag nhô ra 60 mm (phá đồng phẳng theo chiều sâu) cho pitch 0,064° so với 0,124° của nêm nghiêng — mạnh hơn hai lần. B/09 không nêu điều này và tôi chưa có lý thuyết giải thích.
●●● Tách bias khỏi độ lặp lại	Độ lặp lại đo được không cần chuẩn tham chiếu; bias thì cần. Sự bất đối xứng ấy là nguồn cạm bẫy: người ta đo cái dễ rồi gọi tên cái khó [3].
●●● Cạm bẫy khi hệ có làm mượt	Làm mượt giảm tán mát mà không giảm thiên lệch, nên đo độ lặp lại của hệ có bộ lọc rồi gọi là độ chính xác là sai hai lần. Đo đúng: độ lặp lại của toàn chu trình — lùi ra hẳn rồi cập lại.
●● NEES và vì sao nó quan trọng	$\varepsilon = \hat{x}^\top P^{-1} \hat{x}$; kỳ vọng bằng số bậc tự do nếu P trung thực. Quan trọng vì mọi tầng trên dùng P làm trọng số: bộ lọc, cổng chi-square, tiêu chí hội tụ. NEES cao là chỉ báo của sai số mô hình.
●● Cạm bẫy của NEES	Nó giả định mô hình đúng dạng, chỉ sai độ lớn. Nói P cho tới khi NEES về đúng là giấu lỗi mô hình vào trọng số. Phép thử phân biệt: phần dư có tương quan với tư thế không?
●● Ba việc mô phỏng không thay được	Đo σ thật (mô phỏng giả định nó); đo độ lặp lại toàn chu trình (mô phỏng không có bộ điều khiển, cơ khí, bánh trượt); thử nghiệm độ bền (không có ánh sáng, nhiệt, bụi).

●●● phải thuộc ●● nên nắm ● biết là có

I Kiến thức này đang sống ở đâu (công trình đang dùng chính ý tưởng của chương; chốt 9/2026)

Công trình / hệ thống	Dùng ý tưởng nào của chương	Ghi chú
code/monte_carlo_conste	Script của chương; đã chạy 2026-09-14, log ở ket-qua-chay-2026-09-14.txt	Cửa kiểm chứng (b) cho toàn cây — đổi hằng số đầu tệp thành số của bạn
OpenCV solvePnPGeneric	Trả nhiều nghiệm kèm reprojection — nguồn của cột ρ ở mục 3B	Không có nó thì không đo được ambiguity
ISO 9283 [3]	Định nghĩa chuẩn của accuracy và repeatability cho robot	Dùng đúng thuật ngữ khi báo cáo — mục 4B

Công trình / hệ thống	Dùng ý tưởng nào của chương	Ghi chú
Đánh giá quỹ đạo [36]	Căn chỉnh trước khi tính sai số; chọn sai phép căn chỉnh thì con số vô nghĩa	Cùng tư duy với mục 4B, khác lĩnh vực
evo [66]	Công cụ đánh giá quỹ đạo có sẵn các phép căn chỉnh chuẩn	Dùng nếu cần đánh giá cả quỹ đạo tiếp cận chứ không chỉ điểm cuối

MINI PROJECT · Đo tỉ lệ chọn nhầm nghiệm — phép đo còn thiếu · 1–2 ngày

Mục tiêu	giải quyết chỗ nợ kiểm chứng lớn nhất mà mục 3C để lại: phá tính đồng phẳng có thật sự khử ambiguity không, đo bằng <i>đại lượng đúng</i> .
Dữ liệu	mô phỏng, mở rộng script đã có. Không cần phần cứng.
Các bước	sửa run_config để, với mỗi lần chạy, ghi lại <i>nghiệm được chọn có phải nghiệm gần chân trị nhất không</i> — tức tỉ lệ chọn nhầm, chứ không phải độ tán mát; chạy cho hai cấu hình của mục 3C (4 tag đồng phẳng và 4 tag nghiêng) ở nhiều góc tiếp cận, đặc biệt là gần fronto-parallel; rồi quét độ nghiêng của nêm từ 0° tới 45° và vẽ tỉ lệ chọn nhầm theo độ nghiêng; cuối cùng lập với phương án D (tag nhô ra) để kiểm xem vì sao nó mạnh hơn.
Xong khi	bạn có một đồ thị “tỉ lệ chọn nhầm theo độ nghiêng”. Nếu nó giảm mạnh về không ở một ngưỡng nào đó, thì B/09 mục 4 đúng và mục 3C của chương này chỉ đo nhầm đại lượng — và bạn vừa có con số ngưỡng thật, thay cho khuyến nghị 20% chưa kiểm. Nếu nó <i>không</i> giảm, thì B/09 mục 4 cần sửa và đó là một phát hiện quan trọng. Kiểm tra chéo bắt buộc: với cấu hình đồng phẳng ở fronto-parallel, tỉ lệ chọn nhầm phải gần 50% — nếu không thì cách xác định “nghiệm đúng” của bạn có lỗi.
Nối với	B/09 mục 4B nhận con số ngưỡng; 99-chua-biet.md xoá được mục đầu tiên.

TRẢ LỜI CÂU HỎI MỞ ĐẦU

- A1. Vì 0,2 mm là *độ lặp lại*, không phải *độ chính xác*: nó đo độ tán mát giữa các lần, và nó hoàn toàn im lặng về việc giá trị trung bình có đúng không. Một hệ có bias 3 mm và tán mát 0,2 mm cho đúng con số ấy [3]. Phép đo cho phép nói về độ chính xác đòi một *chuẩn tham chiếu* độc lập — máy toàn đạc, đồ gá cơ khí có dung sai biết trước, hoặc chính cơ cấu cập — để so vị trí trung bình với vị trí thật. Và có một cạm bẫy thứ hai: nếu hệ có làm mượt (B/11) thì con số tán mát còn nhỏ hơn thực chất nữa, vì làm mượt giảm tán mát mà không giảm thiên lệch. Cách đo đúng là đo độ lặp lại của *tất cả bộ chu trình* — lùi ra hẳn rồi cập lại từ đầu — chứ không đo độ tán mát của pose khi robot đã đứng yên ở vị trí cuối.
- A2. Thứ nhất: *nguồn sai số bị bỏ sót*. Mô phỏng mô phỏng chính xác mô hình bạn đã viết, nên nếu mô hình thiếu một hiện tượng thì mô phỏng cũng thiếu — và nó không có cách nào báo cho bạn. Mọi con số ở mục 3 giả định không có méo, không rolling shutter, không blur, không loá, constellation đúng chính xác. Thứ hai: *tính đúng của mô hình nhiều*. Script giả định nhiều Gauss độc lập giữa các góc vì đó là điều tôi viết vào; A/05 mục 9 liệt kê ba cách giả định ấy bị vi phạm trong thực tế, và không cách nào xuất hiện trong mô phỏng. Gọn lại: *mô phỏng kiểm được phép tính, không kiểm được danh sách*. Hệ quả cụ thể với cây này: mô phỏng chỉ kiểm được các dòng ngẫu nhiên của C/15 và im lặng hoàn toàn về các dòng hệ thống — vốn chiếm 72% ngân sách trong ví dụ của chương ấy.
- A3. Không phải lỗi nào cả — đó là chênh lệch *có chủ ý và đã được dự kiến*. Hệ số 2 chính là $\sqrt{N}$ với $N = 4$ góc của một tag. A/05 mục 3A đã cố ý bỏ hệ số $1/\sqrt{N}$ ra khỏi công thức, với lý do ghi rõ tại chỗ: trong thực tế nhiều giữa các góc cùng một tag tương quan mạnh (chung blur, chung chiếu sáng, chung méo cục bộ), nên $\sqrt{N}$ là một lời hứa thường không được giữ, và bỏ nó ra là cách ước lượng thận trọng cho ngân sách. Trong mô phỏng thì nhiều *đúng là* độc lập theo xây dựng, nên $\sqrt{N}$ xuất hiện đầy đủ. Đây là một xác nhận chứ không phải một bất đồng, và nó minh hoạ giá trị của việc ghi rõ lý do khi bỏ một hệ số thay vì lặng lẽ bỏ — nếu A/05 không ghi lý do, kết quả này sẽ trông như một mâu thuẫn cần đi tìm nguyên nhân.

- A4. NEES: $\varepsilon = \tilde{x}^T P^{-1} \tilde{x}$, với kỳ vọng bằng số bậc tự do nếu P trung thực. Với sai số thật 1,5 mm và P khai 0,3 mm, NEES sẽ lớn hơn kỳ vọng khoảng $(1,5/0,3)^2 = 25$ lần — rất dễ phát hiện. Nó quan trọng hơn bản thân độ chính xác vì *mọi tầng phía trên dùng P làm trọng số*: bộ lọc ở B/11 dùng nó để cân các phép đo, cổng chi-square ở B/09 mục 7 dùng nó để loại ngoại lai, và tiêu chí hội tụ ở B/13 mục 8 dùng nó để quyết định khi nào dừng. Một P quá tự tin làm cả ba sai theo chiều nguy hiểm: bộ lọc tin quá mức vào phép đo xấu, cổng loại nhầm phép đo tốt, và bộ điều khiển dừng khi chưa đạt. Thêm nữa, NEES cao là chỉ báo tốt của *sai số mô hình*, vì công thức hiệp phương sai ở A/05 mục 2 giả định mô hình đúng — nên khi có méo còn sót hoặc constellation lệch, P báo về nhỏ hơn sự thật. Cam bẫy: đừng nói P cho tới khi NEES về đúng, vì như thế là giấu lỗi mô hình vào trọng số; phép thử phân biệt là kiểm xem phần dư có tương quan với tư thế không.
- A5. Ba việc mô phỏng không thay được. Một, *đo σ thật* của hệ (B/07 mini project) — vì mô phỏng *giả định σ* , nên mọi con số của nó có giãn tuyến tính theo con số bạn nhập vào, và 0,2 px chỉ là bậc độ lớn kinh nghiệm. Hai, *đo độ lặp lại của toàn bộ chu trình cập dock* — lùi ra hẳn rồi cập lại ít nhất 30 lần — vì mô phỏng không có bộ điều khiển, không có cơ khí, không có bánh xe trượt, và ba thứ đó đóng góp những dòng mà không phép mô phỏng thị giác nào chạm tới. Ba, *thử nghiệm độ bền* ở các điều kiện ánh sáng, nhiệt độ, rung và marker bắn — vì mô phỏng không có ánh sáng. Và với mỗi thử nghiệm độ bền, phải đo lại *cả vị trí trung bình* chứ không chỉ độ tản mát, vì thay đổi ở vị trí trung bình là thiên lệch và không biện pháp làm mượt nào chạm tới nó.

NÓI LẠI THẬT ĐƠN GIẢN

Chương trước cho một cái bảng cộng các khoản sai số lại. Chương này hỏi: làm sao biết cái bảng ấy nói đúng?

Có hai cách kiểm, và chúng khác nhau như kiểm số học với nấu thử.

Cách thứ nhất là *mô phỏng*: cho máy tính giả vờ có một cái camera, một tấm marker, rồi thêm nhiễu ngẫu nhiên và xem kết quả ra sao. Rẻ, nhanh, thử được hàng chục phương án trong một buổi chiều, và làm được khi chưa có gì trong tay. Lần này tôi đã chạy thật, và kết quả nằm ở giữa chương.

Phần lớn khớp với dự đoán, và có hai con số đáng nhớ. Một cái tag phẳng đơn lẻ nhìn thẳng cho sai số góc một phần ba độ — gấp gần ba lần cái ngưỡng nửa độ mà ta đặt ra. Và bốn tấm tag nhỏ trải rộng thay cho một tấm lớn cải thiện gần bốn lần, chỉ bằng việc dán khác đi.

Nhưng có một chỗ *không* khớp, và tôi để nguyên nó trong bài thay vì giấu đi. Tôi đã dự đoán rằng kê nghiêng mấy tấm tag sẽ cải thiện rất nhiều; mô phỏng nói nó chỉ cải thiện hai mươi bốn phần trăm. Nghĩ kỹ thì có lẽ tôi đã đo nhầm thứ: cái nêm nghiêng sinh ra để chữa chuyện *nhìn nhầm* — máy chọn sai một trong hai câu trả lời — chứ không phải để giảm cái run tay, mà tôi lại đi đo cái run tay. Đo nhầm đại lượng thì không giống với mệnh đề sai, và phân biệt hai chuyện đó trước khi đi sửa là việc phải làm.

Cách kiểm thứ hai là *đo thật*, và nó bắt được thứ mà mô phỏng không bao giờ bắt được: những nguyên nhân mà bạn chưa nghĩ tới. Mô phỏng chỉ mô phỏng đúng cái danh sách bạn đã viết ra; nếu bạn quên một thứ thì nó cũng quên, và nó không có cách nào nhắc bạn.

Cuối cùng, một cái bẫy khi nghiệm thu mà rất nhiều báo cáo rơi vào. Đo robot cập hai chục lần rồi thấy nó *luôn* dừng ở cùng một chỗ trong phạm vi hai phần mười milimét — điều đó nghe tuyệt vời, nhưng nó chỉ chứng minh robot *đều tay*, không chứng minh nó *đúng chỗ*. Nếu có một chỗ lệch cố định ba milimét thì cả hai chục lần đều lệch đúng ba milimét ấy, rất đều, và không có gì trong sự đều đặn đó báo cho bạn biết. Muốn biết có lệch hay không thì phải có một cái thước độc lập — hoặc đơn giản là xem cái chốt có khớp vào không.

Chỗ không rút gọn được: ranh giới giữa “mô hình sai” và “đo nhầm đại lượng” thì không có quy tắc chung. Cả hai đều cho ra một con số không như dự đoán, và phân biệt chúng đòi phải nghĩ lại xem mệnh đề gốc thật ra nói về cái gì — việc này làm bằng tay, không có công thức.

Một câu: *Mô phỏng kiểm được phép tính chứ không kiểm được danh sách; và đo thấy đều tay không có nghĩa là đúng chỗ.*

16.8 Điều gì chứng minh cách hiểu này sai

Chương này khác các chương khác: phần lớn nội dung của nó *đã là* kết quả kiểm chứng, nên mục này nói về những gì còn lại.

Kết quả ở mục 3C — *phá đồng phẳng chỉ cải thiện 1,24 lần* — là kết quả cần được kiểm lại bằng đại lượng đúng, và mini project là cách. Nếu tỉ lệ chọn nhằm nghiệm *cũng* không cải thiện khi kê nghiêng, thì [B/09](#) mục 4 và [A/04](#) mục 6 đều cần sửa — và đó sẽ là thay đổi lớn nhất mà cây này từng phải làm.

Kết quả ở mục 3E — *tag nhô ra theo chiều sâu mạnh hơn nêm nghiêng hai lần* — là một phát hiện mà tôi chưa có lý thuyết giải thích. Nó có thể là một hiệu ứng thật (độ lệch khỏi mặt phẳng của tag nhô ra là 60 mm so với 26 mm của nêm nghiêng, tức lớn hơn hai lần — và cải thiện cũng đúng hai lần, một sự trùng khớp đáng chú ý), hoặc nó có thể là hệ quả của việc tag thứ năm thêm bốn góc nữa. Phép thử phân biệt: so phương án D với một phương án E gồm năm tag nhưng tất cả đồng phẳng.

Mệnh đề về $\delta\theta \propto 1/B$ ở mục 3D được ủng hộ về chiều nhưng chưa xác nhận là tuyến tính chính xác, và tôi đã ghi rõ mức kiểm chứng thay vì tuyên bố nó đã qua cửa (b) đầy đủ.

Cuối cùng, một điều về phạm vi: *mọi kết quả trong chương này là kết quả mô phỏng*. Không một con số nào đến từ phần cứng. Cây này chưa có một phép đo thật nào, và đó là chỗ nợ lớn nhất của toàn bộ cây — đã ghi ở 99-chua-biet.md.

16.9 Kết nối

Chương này là chương kiểm, nên nó nối ngược về mọi chương nó kiểm.

Nối lên trên. [C/15-ngan-sach-sai-so](#) là chương mà chương này kiểm; hai chương là một cặp và nên đọc liền. [A/05-lan-truyen-sai-so-pixel-sang-pose](#) được kiểm ở mục 3A và 3D, và kết quả xác nhận hai trong bốn quan hệ một cách chắc chắn. [A/04-pose-ambiguity-target-phang](#) được kiểm ở mục 3B, với kết quả mạnh nhất chương. [A/03-pnp-va-uoc-luong-tu-the](#) mục 5 được kiểm ở phần 3 của script.

Nối ngang. [B/09-multi-marker-constellation](#) nhận cả xác nhận (bảng bốn phương án) lẫn thách thức (mục 3C) — và mini project của chương này là cách giải quyết thách thức ấy. [B/07-detection-pipeline](#) và [B/14-dieu-kien-quan-sat-va-anh-sang](#) cấp các phép đo σ mà mô phỏng cần làm đầu vào. [B/11-uoc-luong-theo-thoi-gian](#) nhận cảnh báo ở mục 4B: một hệ có làm mượt không được báo cáo độ tản mát như độ chính xác.

Nối xuống. [E/19-lo-trinh-toi-thieu](#) đặt chương này ở bước đầu tiên của lộ trình, và lý do nằm ở mục 7: chạy mô phỏng là việc rẻ nhất có tác dụng lớn nhất, và nó làm được trước khi có bất cứ thứ gì.

Câu hỏi tự kiểm (trả lời không nhìn bài)

1. Nêu hai loại sai số mà mô phỏng không bao giờ phát hiện được, và phát biểu giới hạn ấy trong một câu.
2. Kết quả mô phỏng cho δx nhỏ hơn công thức đúng hai lần. Giải thích, và nói vì sao đó là một xác nhận chứ không phải một mâu thuẫn.
3. Vì sao phải báo cáo sai số xoay theo từng trục? Nêu hậu quả cụ thể của việc gộp thành một con số.
4. Kết quả nào trong chương đi ngược dự đoán, và ba cách giải thích là gì? Nêu phép đo phân biệt chúng.

5. Phân biệt “mệnh đề sai” với “đo nhằm đại lượng”, và nói vì sao phân biệt ấy phải làm trước khi sửa một chương.
6. Vì sao đo độ lặp lại của một hệ có bộ lọc rồi gọi nó là độ chính xác là sai hai lần?
7. NEES đo cái gì, và vì sao nó quan trọng hơn bản thân độ chính xác với một hệ có nhiều tầng xử lý?



FRAMEWORK VÀ CÔNG CỤ

TÓM TẮT

Phần D là tầng công cụ: thư viện phát hiện marker, các API PnP và ý nghĩa chính xác của từng cờ, hộp công cụ hiệu chuẩn, thư viện visual servoing, công cụ photogrammetry để đo constellation, và công cụ đánh giá. Nó được đặt gần cuối vì nó *hết hạn nhanh nhất* — một API đổi tên, một hàm mới xuất hiện, và chương này sai. Nếu chỉ nhớ một điều: trong toàn bộ danh sách, hàm có giá trị cao nhất cho bài toán docking là `solvePnPGeneric`, vì nó trả về *nhiều nghiệm kèm sai số tái chiếu của từng nghiệm* — tức là nó đưa thẳng cho bạn đại lượng cần để đo ambiguity, thứ mà mọi API PnP khác giấu đi.

Có một cách dùng Phần D làm nó có giá trị hơn hẳn một danh sách phần mềm, và tôi khuyến nghị nó: *đọc mã nguồn như một phép kiểm tra xem mình đã hiểu lý thuyết đúng chưa*. Cụ thể, có đúng ba tệp đáng đọc thật sự trong toàn bộ hệ sinh thái marker. Tệp `apriltag_pose.c` của [apriltag](#) cho thấy IPPE và hai nghiệm được cài ra sao và nó tính tỉ số sai số tái chiếu như thế nào — tức toàn bộ chương 4 và chương 10 gói trong vài trăm dòng. Phần `objdetect` của [OpenCV](#) cho thấy `ArUco` và `ChArUco` khác nhau ở đúng chỗ nào, và vì sao `ChArUco` có thêm một bước tinh chỉnh mà `ArUco` không có — tức chương 7 và chương 8. Và hàm `solvePnPGeneric` cho thấy cái mà cả chương 3 gọi là “tập nghiệm” trông thế nào khi nó là một mảng thật.

Hai chương của phần này chia theo một ranh giới đơn giản. Chương 17 nói về thứ nằm trong vòng lặp thời gian thực: detector và bộ giải PnP — những thứ chạy ở mỗi khung hình. Chương 18 nói về thứ chạy một lần rồi cho ra một tệp tham số hoặc một con số nghiệm thu: hiệu chuẩn, hand-eye, photogrammetry, visual servoing framework, đánh giá quỹ đạo. Ranh giới ấy cũng gần trùng với ranh giới “thứ bạn phải tự cài” và “thứ bạn nên dùng đồ có sẵn”.

Cảnh báo về thời hạn áp cho cả phần: nội dung chốt tại **tháng 9/2026**. Các tên hàm, tên cờ và cấu trúc module ở đây đúng tại thời điểm ấy và cần rà lại mỗi sáu tháng. Phần lý thuyết ở Phần A tới C thì không có vấn đề này — hình học phối cảnh không đổi phiên bản.

Bảng 16.8. Bản đồ Phần D: hai chương và ranh giới giữa chúng.

Ch.	Thư mục	Nội dung	Mức
17	17-thu-vien-va-detector	Thứ chạy mỗi khung hình: apriltag (và <code>apriltag_pose.c</code>), OpenCV <code>objdetect/ArUco/ChArUco</code> , các cờ <code>SOLVEPNP_*</code> và <code>solvePnPGeneric</code> , <code>solvePnPRefineLM</code> ; các họ tag khác (<code>STag</code> , <code>ArUco3</code> , <code>CCTag</code> , <code>WhyCon</code> , <code>TopoTag</code> , <code>ChromaTag</code>); detector học sâu; ROS wrapper	●●
18	18-cong-cu-calib-servo	Thứ chạy một lần: hiệu chuẩn (OpenCV , Kalibr , deltille), hand-eye (<code>calibrateHandEye</code> , easy_handeye), BA (Ceres , GTSAM , tagslam), photogrammetry (COLMAP , Metashape), servoing (ViSP), đánh giá (evo)	●●

Chốt tại tháng 9/2026, rà lại mỗi sáu tháng. Chỉ ba thứ đáng đọc mã chứ không chỉ dùng: `apriltag_pose.c` (chương 4 và 10 thành code), [OpenCV](#) `objdetect` (chương 7 và 8), và [ViSP](#) phần IBVS (chương 13). Ba thứ đáng dùng mà không cần đọc: [Kalibr](#), [COLMAP](#), [Ceres](#).

Thư viện và detector (Libraries and Detectors)

PHỤ THUỘC B/07-detection-pipeline (quy trình phát hiện — chương này chỉ ra chỗ nào trong mã hiện thực hoá từng bước); B/08-corner-refinement (tinh chỉnh góc); A/03-pnp-va-uoc-luong-tu-the và A/04-pose-ambiguity-target-phang (PnP và hai nghiệm — solvePnPGeneric là API quan trọng nhất của cả cây).

TÓM TẮT

Chương này là tăng công cụ cho những gì chạy *mỗi khung hình*: detector và bộ giải PnP. Nó không nhằm liệt kê đủ mà nhằm ba việc cụ thể. Thứ nhất, chỉ ra *một* API quan trọng hơn mọi API khác — solvePnPGeneric, vì nó là API duy nhất trả về nhiều nghiệm kèm sai số tái chiếu của từng nghiệm, tức đại lượng cần để đo ambiguity. Thứ hai, chỉ ra ba tệp mã nguồn đáng *đọc* chứ không chỉ dùng, vì đọc chúng là cách kiểm xem mình đã hiểu lý thuyết đúng chưa. Thứ ba, liệt kê các tham số thực sự đáng chỉnh và nói rõ phần còn lại nên để mặc định. Nếu chỉ nhớ một điều: *nếu pipeline của bạn không lấy ra ρ và hiệp phương sai của pose, bạn đang vứt đi hai đại lượng đã được tính sẵn* — và cả cây này dựa vào chúng.

KIẾN THỨC CẦN CÓ TRƯỚC

- **solvePnPGeneric** — hàm OpenCV giải PnP và trả về *mảng* nghiệm cùng *mảng* sai số tái chiếu tương ứng. Khác với solvePnP vốn chỉ trả một nghiệm.
- cờ **SOLVEPNP_*** — tham số chọn thuật toán: ITERATIVE, EPNP, P3P, IPPE, IPPE_SQUARE, SQPNP và vài cái khác. Chúng không tương đương — xem bảng ở mục 3A.
- **objdetect** — module của OpenCV chứa ArUco và ChArUco từ phiên bản 4.7 trở đi; trước đó chúng nằm ở contrib.
- **wrapper ROS** — gói ROS bọc một thư viện gốc để nó nhận và trả dữ liệu qua topic. Tiện, nhưng nó thường *ẩn* các tham số quan trọng và *vứt* các đại lượng chẩn đoán.

CÂU HỎI MỞ ĐẦU

- Q1. Trong toàn bộ các API của OpenCV liên quan tới marker, có đúng một hàm mà thiếu nó thì bạn không thực hiện được B/10. Hàm nào, và đại lượng nào nó cho mà các hàm khác giấu?
- Q2. Bạn gọi solvePnP với cờ mặc định trên bốn góc của một tag vuông. Nếu hai điều không tối ưu đang xảy ra, và cách sửa mỗi điều.
- Q3. Có ba tệp mã nguồn trong cả hệ sinh thái đáng đọc chứ không chỉ dùng. Chúng là gì, và mỗi tệp kiểm được sự hiểu của bạn về chương nào?
- Q4. Wrapper ROS cho detector marker tiện lợi nhưng nó thường làm mất hai thứ mà cây này coi là bất buộc. Hai thứ nào?
- Q5. Chương này chốt tại tháng 9/2026 và sẽ hết hạn. Phần nào của nó hết hạn nhanh nhất, và phần nào gần như không hết hạn?

17.1 Đặt vấn đề

Phần A tới C không nhắc tới một dòng code cụ thể nào, và đó là có chủ ý: hình học phối cảnh không đổi phiên bản, còn tên hàm thì đổi. Chương này là chỗ trả nợ, và nó mang theo một hạn sử dụng.

Mục đích của chương không phải là một danh mục. Các thư viện đều có tài liệu tốt hơn bất cứ thứ gì tôi viết ở đây, và chép lại chúng là vô ích. Mục đích là ba việc mà tài liệu chính thức *không* làm.

Việc thứ nhất: chỉ ra cái gì quan trọng. Tài liệu API liệt kê mọi hàm với cùng trọng số. Nhưng với bài toán của cây này, có đúng một hàm mà thiếu nó thì cả B/10 không thực hiện được, và có ba tham số đáng chỉnh trong khi vài chục tham số khác nên để mặc định. Biết điều đó tiết kiệm nhiều thời gian hơn biết thêm mười hàm.

Việc thứ hai: nói cái gì bị giấu. Nhiều đại lượng chẩn đoán quan trọng đã được tính bên trong thư viện rồi bị vứt đi ở giao diện. Tỉ số ambiguity là ví dụ rõ nhất: IPPE tính cả hai nghiệm dù bạn có dùng hay không, nhưng solvePnP chỉ trả về một. Biết chỗ nào có đại lượng bị giấu là biết chỗ nào đáng sửa vài dòng.

Việc thứ ba, và nó là việc có giá trị lâu dài nhất: chỉ ra mã nào đáng đọc như một bài kiểm tra. Đọc apriltag_pose.c rồi nhận ra mình hiểu vì sao nó tính hai nghiệm — đó là một phép kiểm xem A/04 đã được hiểu hay chỉ được đọc. Mục 4 dành cho ba tệp như vậy.

CẠM BẮY

Chương này chốt tại tháng 9/2026 và là chương hết hạn nhanh nhất của cây.

Tên hàm đổi, module được chuyển vị trí (ArUco đã chuyển từ contrib sang objdetect ở OpenCV 4.7), tham số mặc định đổi giữa các phiên bản, và các thư viện mới xuất hiện.

Phần *không* hết hạn: các nguyên tắc ở mục 2 (đại lượng nào cần lấy ra), lý do đằng sau việc chọn cờ ở mục 3, và giá trị của việc đọc mã ở mục 4. Chúng vẫn đúng khi tên hàm đã đổi.

Rà lại mỗi sáu tháng, và khi rà thì kiểm bằng cách *chạy* chứ không bằng cách đọc tài liệu — mục 3B là một ví dụ về việc chạy thật phát hiện ra gì.

17.2 Đại lượng phải lấy ra khỏi pipeline

Trước khi nói về hàm nào, nói về *cái gì* phải có ở đầu ra. Ba đại lượng, và cả ba thường bị vứt.

Một, tỉ số ambiguity ρ . Không có nó thì B/10 bậc 2 không thực hiện được, và bạn không biết khi nào pose của mình không đáng tin. Nó đã được tính sẵn bởi IPPE.

Hai, hiệp phương sai của pose $\mathcal{I}^{-1}$. Không có nó thì B/11 không có trọng số, B/09 mục 7 không có cổng chi-square, và B/13 mục 8 không có tiêu chí “phép đo đáng tin”. Nó đã được tính sẵn bởi bước tinh chỉnh — nó chính là ma trận của phương trình chuẩn (A/03 mục 4B).

Ba, hiệp phương sai của từng góc. Không có nó thì trọng số ở A/03 mục 6 là trọng số đều. Nó đã được tính sẵn bởi bước tinh chỉnh góc — nghịch đảo ma trận cấu trúc (B/08 mục 2).

Ba đại lượng, ba lần “đã được tính sẵn”. Đây là chủ đề của chương: *phần lớn thông tin chẩn đoán mà cây này cần đã tồn tại bên trong thư viện, và việc của bạn là lấy nó ra chứ không phải tính lại.*

Chi phí: với ρ thì đổi một lời gọi hàm; với hai đại lượng còn lại thì vài chục dòng, vì OpenCV không xuất chúng qua API công khai. Đây là một thiếu sót đáng tiếc của API và nó là lý do chính đáng nhất để tự viết bước tinh chỉnh PnP thay vì dùng solvePnPRefineLM.

17.3 OpenCV: các API và một phép chạy thật

A. Chọn cờ SOLVEPNP_* nào

Bảng 17.1. Các cờ SOLVEPNP_* và khi nào dùng cho bài toán marker.

Cờ	Số điểm	Dùng khi
IPPE_SQUARE	đúng 4, tag vuông	Mặc định cho một tag vuông. Trả hai nghiệm kèm reprojection (A/04)
IPPE	≥ 4 , đồng phẳng	Target phẳng không vuông, chẳng hạn ChArUco một mặt phẳng
SQPNP	≥ 3 , tổng quát	Mặc định cho constellation không đồng phẳng; tối ưu toàn cục ở mức đại số [24]
EPNP	≥ 4 , tổng quát	Nhiều điểm, cần $O(n)$ [23]
P3P / AP3P	đúng 4	Hạt nhân RANSAC, không dùng làm bộ giải chính
ITERATIVE	≥ 4	Cần khởi tạo tốt; hữu ích với useExtrinsicGuess

Câu trả lời cho Q2: gọi `solvePnP` với cờ mặc định (`ITERATIVE`) trên bốn góc tag vuông có hai điều không tối ưu. Thứ nhất, nó *không* dùng cấu trúc phẳng nên nó không cho bạn hai nghiệm — sửa bằng cách chuyển sang `solvePnPGeneric` với `IPPE_SQUARE`. Thứ hai, `ITERATIVE` không có khởi tạo tốt nên nó có thể hội tụ về nghiệm sai — và nó không báo. Sửa bằng cách khởi tạo bằng lời giải dạng đóng rồi mới tinh chỉnh, đúng như A/03 mục 9 khuyến nghị.

B. Một phép chạy thật, và điều nó phát hiện

Tôi kiểm các khẳng định trên bằng cách chạy chứ không bằng cách đọc tài liệu, trên OpenCV 4.13.0 ngày 2026-09-14. Hai điều đáng ghi lại.

Điều thứ nhất, xác nhận: `solvePnPGeneric` với `IPPE_SQUARE` trả về đúng hai nghiệm ở mọi tư thế đã thử, kể cả fronto-parallel, kèm hai giá trị sai số tái chiếu. Tỉ số của chúng là ρ , và C/16 mục 3B báo cáo nó biến thiên từ 1,33 tới 66,6 theo góc nghiêng. Khẳng định trung tâm của chương — rằng API này cho bạn đại lượng cần thiết — **đã được kiểm.**

Điều thứ hai, một cái bẫy: với tag ở tư thế fronto-parallel *chính xác*, hai nghiệm có sai số tái chiếu bằng nhau tới sai số số học, nên ρ tính được là ∞ hoặc nan tùy vào việc mẫu số có đúng bằng không hay không. Trong dữ liệu không nhiễu, tôi gặp cả hai. Mã của bạn *phải* xử lý trường hợp này — kiểm mẫu số trước khi chia — vì nếu không thì ở đúng tư thế nguy hiểm nhất, chỉ báo an toàn của bạn trả về nan và mọi so sánh với ngưỡng đều thành false.

Đây là một ví dụ nhỏ nhưng đáng nhớ của việc vì sao phải chạy: đọc tài liệu không cho bạn biết điều này, và nó là chế độ hỏng ở đúng chỗ mà chỉ báo cần hoạt động nhất.

C. ArUco và ChArUco

Từ OpenCV 4.7, chúng ở `objdetect` thay vì `contrib`. Lớp chính là `ArucoDetector` và `CharucoDetector`.

Tham số đáng chỉnh, và nó khớp với ba tham số ở B/07 mục 5C:

`cornerRefinementMethod` — **bật nó.** Mặc định ở một số phiên bản là `CORNER_REFINE_NONE`, và khi ấy bạn đang lấy góc của đa giác xấp xỉ, kém hơn hẳn (B/08). Đặt `CORNER_REFINE_SUBPIX` cho ArUco, và với ChArUco thì bước saddle point đã có sẵn trong `interpolateCornersCharuco`.

`minMarkerPerimeterRate` — ngưỡng kích thước tối thiểu, đặt theo con số đo được của [A/02](#) mục 4 chứ không theo mặc định.

`errorCorrectionRate` — số lỗi bit cho phép sửa. Đặt *thấp* và bù bằng whitelist ID ([B/14](#) mục 7A).

Về từ điển: OpenCV có sẵn nhiều từ điển chuẩn, nhưng với docking thì [B/07](#) mục 5C khuyến nghị từ điển *nhỏ, ít ô bit*. `generateCustomDictionary` tạo được từ điển theo yêu cầu, và nó là một trong số ít chỗ mà đi chệch khỏi mặc định gần như luôn đúng.

17.4 Ba tệp mã đáng đọc

Mục này trả lời Q3, và nó là mục có giá trị lâu dài nhất của chương.

Tệp thứ nhất: `apriltag_pose.c` của [apriltag](#). Vài trăm dòng, và nó là toàn bộ [A/04](#) cộng [B/10](#) bậc 2 thành code. Bạn sẽ thấy: IPPE được cài ra sao, hai nghiệm được tính thế nào, và sai số tái chiếu của từng nghiệm được dùng để chọn ra sao. Đọc nó và nhận ra mình *dự đoán được* nó làm gì tiếp là một phép kiểm tốt xem chương 4 đã được hiểu hay chỉ được đọc.

Tệp thứ hai: phần `quad fitting` của [apriltag](#) (thường ở `apriltag_quad_thresh.c`). Đây là [B/07](#) mục 2 thành code, và chỗ đáng chú ý nhất là bước khớp đường có trọng số gradient — cải tiến chính của AprilTag 2 ([B/07](#) mục 2). Đọc nó để thấy vì sao độ chính xác dưới pixel là chuyện có thể.

Tệp thứ ba: phần IBVS của [ViSP](#). Đây là [B/13](#) mục 2 thành code: ma trận tương tác được dựng thế nào, và luật $\mathbf{v} = -\lambda \mathbf{L}^+ \mathbf{e}$ được cài ra sao. Đáng đọc nếu bạn cân nhắc IBVS cho pha cuối.

(Ghi chú trung thực: mô tả về nội dung ba tệp này là từ trí nhớ về cấu trúc của chúng; tôi chưa mở lại mã trong lần soạn này. Tên tệp cụ thể có thể đã đổi. Điều tôi tin chắc là chức năng tồn tại trong các thư viện ấy, không phải tên tệp chính xác.)

Ngược lại, ba thứ đáng dùng mà không cần đọc: [Kalibr](#), [COLMAP](#), [Ceres](#). Chúng là công cụ trưởng thành, tài liệu tốt, và việc đọc mã của chúng không trả lại tương xứng với thời gian bỏ ra.

17.5 Các họ tag khác và detector học sâu

Ngắn, vì [B/07](#) mục 4 đã bàn về mặt kỹ thuật; mục này chỉ nói về mặt công cụ.

[STag](#) [30], [TopoTag](#) [67], [ChromaTag](#) [68]: các họ tag với thiết kế khác. Mã nguồn có, chất lượng khác nhau. Đáng thử nếu bạn có một yêu cầu cụ thể mà AprilTag và ArUco không đáp ứng — chẳng hạn STag khi cần ổn định pose tốt hơn ở tư thế xấu.

[CCTag](#) [18], [WhyCon](#) [19]: marker tròn. Nhớ cảnh báo về bias tâm ellipse ([A/02](#) mục 6) — nếu dùng, phải kiểm xem cài đặt có hiệu chỉnh conic không, và nếu không thì bạn có một dòng hệ thống phụ thuộc tư thế trong ngân sách.

[DeepTag](#) [43], [DeepArUco](#) [44]: detector học sâu. Đáng cân nhắc khi điều kiện quan sát khắc nghiệt và không cải thiện được bằng chiếu sáng ([B/14](#)). Với docking — nơi điều kiện kiểm soát được — lợi ích ít rõ, và [B/07](#) mục 4 nêu lý do thận trọng: bền hơn không có nghĩa là chính xác hơn về định vị góc.

[apriltag-generation](#): tạo họ tag riêng với số ô và khoảng cách Hamming tùy chọn. Công cụ để thực hiện đánh đổi ở [B/07](#) mục 5C, và nó đáng dùng hơn vẻ ngoài vì lựa chọn mặc định (dùng họ phổ biến nhất) thường không tối ưu cho docking.

17.6 Wrapper ROS

Mục này trả lời Q4, và nó là một cảnh báo.

`apriltag_ros`, `aruco_ros`, `fiducials`: các gói ROS bọc thư viện gốc. Chúng tiện — nhận ảnh qua topic, trả pose qua TF — và với một hệ thống ROS thì chúng là điểm xuất phát hợp lý.

Nhưng chúng thường làm mất hai thứ mà cây này coi là bắt buộc.

Thứ nhất, ρ . Giao diện ROS thường trả về một pose duy nhất, nên tỉ số ambiguity bị vớt ở tầng wrapper. Hệ quả: bạn không thực hiện được B/10 bậc 2, và bạn không biết khi nào pose không đáng tin.

Thứ hai, hiệp phương sai trung thực. Một số wrapper trả về một trường hiệp phương sai, và nó thường là một hằng số cấu hình chứ không phải $\mathcal{I}^{-1}$ tính từ dữ liệu. Một hiệp phương sai hằng số tệ hơn không có hiệp phương sai, vì nó trông như thông tin thật và mọi tầng trên sẽ tin nó.

Hai mất mát ấy là lý do tôi khuyên: *dùng wrapper để bắt đầu, nhưng sửa nó hoặc thay nó trước khi nghiêm túc.* Việc sửa không lớn — lấy ρ ra và thêm nó vào message — và nó là điều kiện để phần lớn Phần B áp dụng được.

Một điều đáng nêu về `tagslam` [31]: nó khác các wrapper trên ở chỗ nó giải bài toán hợp nhất trên nhiều tag bằng factor graph, tức nó làm đúng việc mà B/09 mục 2 khuyến nghị. Nếu bạn cần map nhiều tag trong môi trường thì nó là điểm xuất phát tốt. Mức tin C — tiền ẩn, nhưng có mã nguồn công khai và được dùng rộng rãi.

17.7 Giới hạn

Chương này không có benchmark. B/07 mục 6 giải thích: tôi không có dữ liệu so sánh được giữa các detector, và trích số từ các bài báo khác nhau ra khỏi ngữ cảnh thì chúng không so sánh được. Nếu cần xếp hạng cho hệ của bạn, mini project của B/07 là cách đo.

Các khẳng định về mã nguồn là từ trí nhớ. Đã ghi ở mục 4. Tên tệp có thể đã đổi; chức năng thì tôi tin là còn.

Chỉ các khẳng định về OpenCV API đã được chạy kiểm, trên đúng một phiên bản (4.13.0) và một hệ điều hành. Hành vi có thể khác ở phiên bản khác — và đặc biệt, vị trí module `ArUco` đã đổi một lần rồi.

Chương chốt tháng 9/2026. Xem cảnh báo ở mục 1.

17.8 Thực hành

Bốn việc.

Một, chuyển sang solvePnPGeneric và lấy ρ ra — việc có tỉ lệ lợi ích trên công sức cao nhất trong chương, và nó là điều kiện cho B/10.

Hai, xử lý trường hợp ρ là nan hoặc vô hạn — mục 3B; nó xảy ra ở đúng tư thế nguy hiểm nhất.

Ba, bật tính chỉnh góc nếu dùng ArUco, và kiểm giá trị mặc định của phiên bản bạn đang dùng thay vì giả định.

Bốn, nếu dùng wrapper ROS, sửa nó để xuất ρ trước khi xây dựng gì phía trên.

Active recall (che cột phải, tự trả lời, rồi mở ra đối chiếu)

Thuật ngữ / câu hỏi	Trả lời
●●● API quan trọng nhất của cả cây	<code>solvePnPGeneric</code> — API duy nhất trả về <i>nhiều nghiệm kèm sai số tái chiếu của từng nghiệm</i> , tức ρ . Thiếu nó thì B/10 bậc 2 không thực hiện được.
●●● Ba đại lượng phải lấy ra	ρ (cho B/10); hiệp phương sai pose $\mathcal{I}^{-1}$ (cho B/11, công chi-square, tiêu chí hội tụ); hiệp phương sai từng góc (cho trọng số ở A/03 mục 6). Cả ba đã được tính sẵn — việc của bạn là lấy ra, không phải tính lại.
●●● Chọn cờ nào	IPPE_SQUARE cho một tag vuông (cho hai nghiệm); SQPNP cho constellation không đồng phẳng; EPNP khi nhiều điểm; P3P chỉ làm hạt nhân RANSAC.
●●● Hai vấn đề của <code>solvePnP</code> mặc định	Cờ ITERATIVE không dùng cấu trúc phẳng nên không cho hai nghiệm; và nó không có khởi tạo tốt nên có thể hội tụ về nghiệm sai <i>mà không báo</i> .
●●● Cái bẫy ρ tại fronto-parallel	Ở tư thế fronto-parallel chính xác, hai sai số tái chiếu bằng nhau tới sai số số học nên ρ ra nan hoặc ∞ . Phải kiểm mẫu số trước khi chia — nếu không thì ở đúng tư thế nguy hiểm nhất, chỉ báo an toàn trả nan và mọi so sánh ngưỡng thành false. Phát hiện bằng cách CHẠY, không bằng đọc tài liệu.
●● Ba tham số ArUco đáng chỉnh	<code>cornerRefinementMethod</code> (bật — mặc định có thể là NONE); <code>minMarkerPerimeterRate</code> (theo ngưỡng đo được của A/02 mục 4); <code>errorCorrectionRate</code> (thấp, bù bằng whitelist).
●●● Ba tệp mã đáng đọc	<code>apriltag_pose.c</code> (A/04 + B/10 thành code); phần quad fitting của <code>apriltag</code> (B/07 mục 2, khớp đường có trọng số gradient); phần IBVS của <code>ViSP</code> (B/13 mục 2).
●● Ba thứ đáng dùng mà không cần đọc	<code>Kalibr</code> , <code>COLMAP</code> , <code>Ceres</code> — công cụ trưởng thành, tài liệu tốt, đọc mã không trả lại tương xứng.
●●● Hai thứ wrapper ROS làm mất	ρ (giao diện trả một pose duy nhất); và hiệp phương sai <i>trung thực</i> (thường là hằng số cấu hình chứ không phải $\mathcal{I}^{-1}$). Hiệp phương sai hằng số <i>tệ hơn</i> không có, vì nó trông như thông tin thật.
●● Vì sao chương này không có benchmark	Không có dữ liệu so sánh được giữa các detector; trích số từ các bài báo khác nhau ra khỏi ngữ cảnh thì chúng không so được (B/07 mục 6). Cần xếp hạng thì đo trên hệ của mình.
●● Phần nào của chương hết hạn	Tên hàm, vị trí module, giá trị mặc định — hết hạn nhanh. <i>Nguyên tắc</i> (đại lượng nào cần lấy ra, lý do chọn cờ, giá trị của việc đọc mã) — gần như không hết hạn.
● Từ điển tùy chỉnh	<code>generateCustomDictionary</code> — một trong số ít chỗ mà đi lệch khỏi mặc định gần như luôn đúng cho docking, vì bạn chỉ cần vài chục ID (B/07 mục 5C).

●●● phải thuộc ●● nên nắm ● biết là có

Kiến thức này đang sống ở đâu (công trình đang dùng chính ý tưởng của chương; chốt 9/2026)

Công trình / hệ thống	Dùng ý tưởng nào của chương	Ghi chú
OpenCV <code>solvePnPGeneric</code>	Nhiều nghiệm kèm reprojection từng nghiệm	Đã kiểm bằng cách chạy, 4.13.0, 2026-09-14
<code>apriltag</code> (AprilRobotics)	Bản C gốc; <code>apriltag_pose.c</code> và phần quad fitting	Hai trong ba tệp đáng đọc
OpenCV <code>objdetect</code>	ArUco và ChArUco từ 4.7; <code>ArucoDetector</code> , <code>CharucoDetector</code>	Nhớ kiểm mặc định của <code>cornerRefinementMethod</code>
<code>ViSP</code> [65]	IBVS, PBVS, ma trận tương tác, tracker marker	Tệp thứ ba đáng đọc; cho B/13

Công trình / hệ thống	Dùng ý tưởng nào của chương	Ghi chú
tagslam [31]	Hợp nhất nhiều tag bằng factor graph — đúng việc B/09 mục 2 khuyến nghị	Khác các wrapper ROS khác ở điểm này; mức tin C

MINI PROJECT · Sửa pipeline để xuất ba đại lượng chẩn đoán · 1 ngày

Mục tiêu	biến pipeline của bạn từ một hộp đen trả về một pose thành một hệ đo lường trả về pose kèm bằng chứng về độ tin cậy của nó.
Dữ liệu	chính pipeline hiện tại của bạn.
Các bước	chuyển lời gọi PnP sang solvePnPGeneric với cờ đúng theo bảng ở mục 3A; lấy mảng reprojection ra và tính ρ , có xử lý trường hợp mẫu số bằng không; tự viết bước tinh chỉnh LM (hoặc sửa solvePnPRefineLM) để lấy ra ma trận $\mathcal{I} = \sum J^T \Sigma^{-1} J$ và nghịch đảo nó; nếu dùng ArUco, lấy ma trận cấu trúc từ bước cornerSubPix để có hiệp phương sai từng góc; cuối cùng đối giao diện trả về từ một pose thành cấu trúc ba giá trị của B/10 mục 3C.
Xong khi	pipeline trả về pose, hiệp phương sai, ρ , và một trạng thái ba giá trị; và log của bạn ghi cả ba ở mỗi khung. Kiểm tra chéo bắt buộc: chạy trên một chuỗi có đoạn fronto-parallel và xác nhận rằng ρ không trả về nan ở đó — nếu có thì bạn chưa xử lý trường hợp ở mục 3B, và bạn vừa mất chỉ báo ở đúng chỗ cần nó nhất.
Nối với	B/10 và B/13 mục 8 tiêu thụ trực tiếp ba đại lượng này.

TRẢ LỜI CÂU HỎI MỞ ĐẦU

- A1. solvePnPGeneric. Nó là API duy nhất trả về *mảng* nghiệm cùng *mảng* sai số tái chiếu tương ứng; mọi hàm khác — kể cả solvePnP với cùng cờ IPPE — chỉ trả về một nghiệm và vứt phần còn lại. Đại lượng nó cho mà các hàm khác giấu là tỉ số $\rho = e_2/e_1$, tức chỉ báo duy nhất cho biết hai nghiệm có phân biệt được hay không. Thiếu nó thì bậc 2 của [B/10](#) — đo và từ chối khi không tin được — không thực hiện được, và hệ của bạn sẽ chọn nghiệm theo nhiều ở những tư thế mà hai nghiệm ngang nhau, không có cảnh báo nào. Điều đáng nói là IPPE *đã tính* cả hai nghiệm dù bạn có dùng hay không, nên chi phí để lấy ρ ra bằng không — chỉ là đổi một lời gọi hàm.
- A2. Thứ nhất, cờ mặc định ITERATIVE không khai thác cấu trúc phẳng của tag vuông, nên nó không cho bạn hai nghiệm và bạn mất ρ . Sửa: dùng solvePnPGeneric với SOLVEPNP_IPPE_SQUARE. Thứ hai, ITERATIVE là một bộ giải lặp cần điểm khởi đầu, và nếu không truyền useExtrinsicGuess thì nó tự khởi tạo — nên nó có thể hội tụ về nghiệm sai trong hai nghiệm của target phẳng, và nó *không báo* là đã làm thế. Sửa: khởi tạo bằng một lời giải dạng đóng (IPPE cho target phẳng) rồi mới tinh chỉnh, đúng như [A/03](#) mục 9 khuyến nghị. Cả hai lỗi này đều thầm lặng — hàm trả về một pose trông hợp lý trong cả hai trường hợp.
- A3. Một: apriltag_pose.c của [apriltag](#) — nó là [A/04](#) (hai nghiệm của target phẳng) cộng [B/10](#) bậc 2 (dùng tỉ số reprojection để chọn) gói trong vài trăm dòng; đọc nó và thấy mình *dự đoán được* bước tiếp theo là phép kiểm xem chương 4 đã được hiểu hay chỉ được đọc. Hai: phần quad fitting của [apriltag](#) — nó là [B/07](#) mục 2, và chỗ đáng chú ý nhất là bước khớp đường có trọng số gradient, tức cơ chế làm cho độ chính xác dưới pixel khả thi. Ba: phần IBVS của [ViSP](#) — nó là [B/13](#) mục 2, cho thấy ma trận tương tác được dựng thế nào và luật điều khiển được cài ra sao. Ngược lại, [Kalibr](#), [COLMAP](#) và [Ceres](#) thì nên dùng mà không cần đọc: chúng trưởng thành, tài liệu tốt, và đọc mã không trả lại tương xứng thời gian.
- A4. Thứ nhất, tỉ số ambiguity ρ : giao diện ROS thường trả về một pose duy nhất qua TF hoặc một message pose, nên ρ bị vứt ở tầng wrapper — và không có nó thì bạn không biết khi nào pose của mình không đáng tin. Thứ hai, hiệp phương sai *trung thực*: một số wrapper có trường hiệp phương sai trong message, nhưng nó thường được điền bằng một hằng số lấy từ tệp cấu hình chứ không phải $\mathcal{I}^{-1}$ tính từ dữ liệu của khung hiện tại. Mất mát thứ hai tệ hơn mất mát thứ nhất, vì một hiệp phương sai hằng số *tệ hơn không có hiệp phương sai*: nó trông như thông tin thật, nên mọi tầng phía trên — bộ lọc, cổng chi-square, tiêu chí hội tụ — sẽ tin nó và cân các

phép đo sai. Khuyến nghị: dùng wrapper để bắt đầu, nhưng sửa nó để xuất ρ trước khi xây dựng gì phía trên; việc sửa nhỏ và nó là điều kiện để phần lớn Phần B áp dụng được.

- A5. Hết hạn nhanh nhất: tên hàm, vị trí module (ArUco đã chuyển từ contrib sang objdetect ở 4.7 — một lần đổi đã xảy ra rồi), giá trị mặc định của tham số, và danh sách các thư viện mới. Gần như không hết hạn: các *nguyên tắc* — rằng có ba đại lượng chẩn đoán cần lấy ra và cả ba đã được tính sẵn; rằng phải chọn cờ theo cấu trúc hình học của target chứ không theo mặc định; rằng đọc mã là một phép kiểm sự hiểu chứ không chỉ là cách học API; và rằng wrapper tiện lợi thường vứt đi thông tin chẩn đoán. Bốn nguyên tắc ấy vẫn đúng khi mọi tên hàm trong chương đã đổi. Cách rà lại đúng: kiểm bằng cách *chạy* chứ không bằng đọc tài liệu — mục 3B là ví dụ về việc chạy thật phát hiện ra một cái bẫy mà tài liệu không nói.

NÓI LẠI THẬT ĐƠN GIẢN

Phần lớn tài liệu này nói về nguyên lý, và nguyên lý thì không đổi. Chương này nói về tên hàm, và tên hàm thì đổi — nên nó có hạn sử dụng, ghi rõ là tháng 9 năm 2026.

Điều đáng nhớ nhất của chương không phải một tên hàm mà là một quan sát: *phần lớn thông tin bạn cần đã được máy tính sẵn rồi, chỉ là nó bị vứt đi ở cửa ra.*

Ví dụ rõ nhất là chuyện hai câu trả lời của cái tag phẳng. Thư viện *luôn* tính cả hai — nó không có cách nào tính một mà không tính cái kia — nhưng cái hàm thông dụng nhất chỉ trả về một và ném cái còn lại đi. Đổi sang một hàm khác, và bạn có thêm con số cho biết hai câu trả lời có ngang nhau hay không. Đổi một dòng, được một chỉ báo an toàn.

Tương tự với chuyện “máy tin vào kết quả của mình tới mức nào”. Con số đó cũng được tính sẵn trong lúc giải, rồi bị vứt. Lấy nó ra tốn vài chục dòng, và không có nó thì một nửa những việc ở phần trước của tài liệu không làm được.

Có một cái bẫy nhỏ mà tôi chỉ phát hiện được nhờ chạy thử chứ không nhờ đọc tài liệu, và nó đáng kể lại vì nó rất đặc trưng. Cái tỉ số so sánh hai câu trả lời được tính bằng một phép chia. Ở đúng cái tư thế nguy hiểm nhất — tag nhìn thẳng vào camera — hai câu trả lời khớp ảnh ngang nhau hoàn hảo, nên mẫu số bằng không, nên phép chia cho ra một giá trị vô nghĩa. Và một giá trị vô nghĩa thì khi đem so với ngưỡng sẽ luôn ra “không đạt ngưỡng” theo cách im lặng. Nghĩa là: nếu không cẩn thận, cái chỉ báo an toàn của bạn sẽ tắt ngúm đúng vào lúc bạn cần nó nhất.

Cuối cùng, một lời khuyên về việc đọc mã người khác. Có ba tệp đáng đọc trong cả hệ sinh thái này, và lý do đọc chúng không phải để học cách dùng thư viện — tài liệu làm việc đó tốt hơn. Lý do là để *tự kiểm tra mình*: nếu bạn đọc một tệp và thấy mình đoán trước được dòng tiếp theo làm gì, thì bạn đã hiểu cái chương lý thuyết tương ứng. Nếu bạn đọc mà thấy lạ, thì chương ấy mới chỉ được đọc chứ chưa được hiểu.

Chỗ không rút gọn được: tên hàm, tên module và giá trị mặc định sẽ đổi, và không có cách nào viết một chương như thế này cho nó bền. Cách duy nhất là mỗi lần quay lại thì *chạy thử* chứ đừng tin cái đã viết — kể cả cái tôi vừa viết.

Một câu: *Thông tin bạn cần đã được tính sẵn rồi; việc của bạn là đừng để nó bị vứt ở cửa ra.*

17.9 Điều gì chứng minh cách hiểu này sai

Chương công cụ khác chương lý thuyết ở chỗ phần lớn khẳng định của nó kiểm được bằng cách chạy, và một phần đã được kiểm — nên mục này nói về phần còn lại.

Khẳng định trung tâm — solvePnPGeneric với IPPE_SQUARE trả về hai nghiệm kèm sai số tái chiếu, và tỉ số của chúng là một chỉ báo dùng được — **đã được kiểm bằng cách chạy** (mục 3B), trên OpenCV 4.13.0. Nó bị bác bỏ nếu một phiên bản khác trả về một nghiệm, hoặc trả về hai nghiệm mà sai số tái chiếu không phân biệt được chúng ở góc nghiêng lớn. Cách kiểm lại khi nâng cấp: chạy phần 2 của script ở C/16 và xem cột ρ có còn tăng theo góc nghiêng không.

Khẳng định thứ hai — *ba đại lượng chẩn đoán đã được tính sẵn và chỉ cần lấy ra* — bị bác bỏ nếu hoá ra một trong ba không tồn tại ở dạng có thể lấy được. Với ρ thì tôi đã kiểm. Với hiệp phương sai

pose và hiệp phương sai góc thì tôi *chưa* — tôi dẫn ra từ cấu trúc của thuật toán ([A/03](#) mục 4B, [B/08](#) mục 2) chứ không từ việc mở mã OpenCV. Nếu hoá ra chúng bị huỷ trước khi hàm trả về thì lời khuyên “lấy ra” phải đổi thành “tự cài bước tinh chỉnh”, và chi phí của mini project tăng đáng kể.

Khẳng định thứ ba — *ba tệp mã đáng đọc* — là phán đoán chủ quan của tôi về giá trị sư phạm, và nó không bị bác bỏ bằng dữ liệu. Điều *có thể* sai là mô tả về nội dung của chúng, vốn từ trí nhớ và chưa mở lại mã (mục 4).

Cuối cùng, khẳng định về wrapper ROS ở mục 6 dựa trên hiểu biết chung về các gói ấy, *không* phải trên việc đọc mã của chúng trong lần soạn này. Nếu một wrapper cụ thể có xuất ρ thì cảnh báo ở mục 6 không áp dụng cho nó — và đó sẽ là tin tốt đáng ghi lại.

17.10 Kết nối

Chương này là tầng công cụ cho những gì chạy mỗi khung.

Nổi lên trên. [B/07-detection-pipeline](#) mô tả quy trình mà mục 3C và 4 chỉ ra chỗ hiện thực hoá. [B/08-corner-refinement](#) mô tả các phương pháp mà tham số `cornerRefinementMethod` chọn giữa. [A/03-pnp-va-uoc-luong-tu-the](#) và [A/04-pose-ambiguity-target-phang](#) là hai chương mà `solvePnPGeneric` phục vụ trực tiếp.

Nổi ngang. [D/18-cong-cu-calib-servo-va-do-dac](#) là chương anh em: nó bàn những gì chạy *một lần* thay vì mỗi khung, và ranh giới ấy gần trùng với ranh giới “thứ bạn phải tự cài” và “thứ nên dùng đồ có sẵn”.

Nổi xuống. [B/10-xu-ly-ambiguity](#) và [B/13-docking-control](#) mục 8 tiêu thụ trực tiếp ba đại lượng ở mục 2, nên mini project của chương này là điều kiện tiên quyết cho cả hai.

Câu hỏi tự kiểm (trả lời không nhìn bài)

1. Nêu ba đại lượng phải lấy ra khỏi pipeline, chương nào tiêu thụ mỗi cái, và điều chung của cả ba.
2. Vì sao gọi `solvePnP` với cờ mặc định trên tag vuông là không tối ưu theo hai cách? Nêu cách sửa mỗi cách.
3. Mô tả cái bẫy ρ tại fronto-parallel, và nói vì sao nó nguy hiểm hơn một lỗi thông thường.
4. Ba tệp mã nào đáng đọc, và mỗi tệp kiểm được sự hiểu của bạn về chương nào?
5. Wrapper ROS làm mất hai thứ. Nêu cả hai, và nói vì sao mất mát thứ hai tệ hơn mất mát thứ nhất.
6. Phần nào của chương này hết hạn nhanh nhất và phần nào không? Nêu cách rà lại đúng.
7. Vì sao chương này cố ý không có bảng benchmark so sánh các detector?

Công cụ hiệu chuẩn, servoing và đo đạc (One-Time Tools)

PHỤ THUỘC B/12-calibration-cho-docking (ba phép hiệu chuẩn và thứ tự bắt buộc của chúng — chương này nói dùng công cụ nào cho từng bước); B/09-multi-marker-constellation mục 5 (đo constellation bằng BA); B/13-docking-control (servoing); C/16-danh-gia-va-nghiem-thu (đánh giá).

TÓM TẮT

Chương này là tăng công cụ cho những gì chạy *một lần* rồi cho ra một tệp tham số hoặc một con số nghiệm thu — khác với D/17 vốn nói về những gì chạy mỗi khung hình. Ranh giới ấy gần trùng với một ranh giới khác đáng chú ý: *thứ bạn nên dùng đồ có sẵn* so với *thứ bạn phải tự cài*. Bốn nhóm công cụ: hiệu chuẩn intrinsic, đo constellation bằng photogrammetry, hand-eye, và đánh giá. Với ba nhóm đầu, lời khuyên đơn giản là dùng công cụ trưởng thành và đừng tự viết. Với nhóm thứ tư thì ngược lại, và lý do đáng nói: các công cụ đánh giá có sẵn được thiết kế cho bài toán quỹ đạo SLAM, còn bài toán docking cần đo *độ lặp lại tại một điểm* — một đại lượng khác, và tự viết vài chục dòng thì đúng hơn là ép một công cụ không hợp. Nếu chỉ nhớ một điều: *ba bước hiệu chuẩn phải chạy đúng thứ tự, và mỗi bước phải kiểm hiệp phương sai của kết quả trước khi dùng nó cho bước sau*.

KIẾN THỨC CẦN CÓ TRƯỚC

- **photogrammetry** — đo hình học ba chiều từ nhiều ảnh. Với cây này, nó là công cụ thực hiện B/09 mục 5: đo lại vị trí thật của các tag thay vì tin bản vẽ.
- **structure from motion (SfM)** — khôi phục đồng thời cấu trúc 3D và tư thế camera từ một tập ảnh không có thông tin tư thế. COLMAP là cài đặt phổ biến nhất [51].
- **gauge fixing** — việc cố định các bậc tự do mà dữ liệu không xác định được — với BA từ ảnh đơn thuần thì đó là vị trí, hướng, và *tỉ lệ* của toàn bộ cấu trúc.
- **ATE / RPE** — *absolute trajectory error* và *relative pose error*: hai chỉ số chuẩn cho đánh giá quỹ đạo SLAM. Chúng *không* phải chỉ số đúng cho docking — mục 5 giải thích.

CÂU HỎI MỞ ĐẦU

- Q1. Ba bước hiệu chuẩn của B/12 cần ba công cụ khác nhau. Nêu công cụ cho từng bước, và nói bước nào bạn có thể phải tự viết.
- Q2. Bạn dùng COLMAP để đo constellation. Nó trả về một cấu trúc 3D đẹp. Vì sao bạn *vẫn chưa* có thứ cần dùng, và còn thiếu một thông tin gì?
- Q3. Các công cụ đánh giá quỹ đạo như *evo* rất trưởng thành. Vì sao chúng lại *không* phải công cụ đúng cho nghiệm thu docking?
- Q4. Sau khi chạy BA đo constellation, có một đại lượng bạn phải nhìn trước khi dùng kết quả — và bỏ qua nó là tự lừa mình. Đại lượng nào, và tự lừa mình thế nào?
- Q5. Trong bốn nhóm công cụ của chương, ba nhóm nên dùng đồ có sẵn và một nhóm nên tự viết. Nhóm nào, và nguyên tắc chung rút ra là gì?

18.1 Đặt vấn đề

D/17 nói về những gì chạy mỗi khung hình. Chương này nói về những gì chạy một lần: hiệu chuẩn, đo đạc, và đánh giá.

Ranh giới ấy quan trọng hơn vẻ ngoài, vì nó gần trùng với một ranh giới khác về mặt kỹ thuật phần mềm. Thứ chạy mỗi khung phải nhanh, phải chạy trên máy của robot, và phải tích hợp chặt với pipeline — nên bạn thường phải tự cài hoặc ít nhất tự sửa. Thứ chạy một lần thì chạy trên máy tính bàn, không có ràng buộc thời gian thực, và nó cho ra một tệp — nên dùng công cụ có sẵn gần như luôn đúng.

“Gần như” vì có một ngoại lệ, và nó là nội dung đáng nói nhất của chương. Công cụ đánh giá có sẵn được thiết kế cho bài toán *quỹ đạo SLAM*: chúng so hai quỹ đạo dài, có các phép căn chỉnh tinh vi, và chúng báo cáo ATE và RPE. Bài toán docking không phải bài toán ấy — nó cần đo *độ lặp lại của một điểm cuối* qua nhiều lần thử. Ép một công cụ quỹ đạo làm việc ấy thì được, nhưng nó phức tạp hơn và dễ sai hơn so với vài chục dòng tự viết. Mục 5 bàn kỹ.

Có một lý do thứ hai để chương này tồn tại, và nó thuộc về quy trình chứ không về công cụ: *thứ tự*. B/12 mục 2 chỉ ra rằng ba phép hiệu chuẩn phải chạy theo đúng thứ tự vì mỗi bước dùng kết quả bước trước làm hằng số. Chương này bổ sung phần thực hành: mỗi bước dùng công cụ nào, và — quan trọng hơn — mỗi bước phải kiểm gì trước khi chuyển sang bước sau. Bỏ qua bước kiểm ấy là cách phổ biến nhất để có một chuỗi hiệu chuẩn trông hoàn chỉnh mà sai.

18.2 Hiệu chuẩn intrinsic

Bước một của B/12 mục 2.

OpenCV `calibrateCamera` — đủ dùng cho phần lớn trường hợp, có sẵn, và nó cài phương pháp Zhang [6]. Nhược điểm: nó không cho bạn bản đồ residual theo vị trí ảnh, mà đó là phép kiểm quan trọng nhất (A/01 mục 6). Bạn phải tự vẽ — vài chục dòng, và nó đáng.

Kalibr [11] — mạnh hơn, hỗ trợ nhiều mô hình méo, và nó có báo cáo residual theo vị trí. Nó cũng xử lý được nhiều camera và camera-IMU. Giá phải trả: nặng hơn để cài đặt, và nó được thiết kế quanh quy trình ROS bag.

deltille [12] — không phải công cụ hiệu chuẩn hoàn chỉnh mà là một detector target chất lượng cao. Dùng nó khi cần intrinsic chính xác nhất và khi target lưới tam giác chấp nhận được.

Về target: B/12 mục 3C khuyến nghị ChArUco hoặc deltille, và nhấn mạnh rằng target phải *phẳng thật* — in trên giấy dán bìa cứng không đủ. Với hiệu chuẩn chất lượng cao, target in trên tấm nhôm hoặc kính là đầu tư đáng.

Phải kiểm trước khi chuyển sang bước hai: bản đồ residual không có cấu trúc (A/01 mục 6), và c_x, c_y ổn định qua các tập con ảnh (B/12 mục 3B). Nếu điểm chính nhảy hàng chục pixel giữa các tập con thì bạn chưa có intrinsic dùng được, và chạy bước hai với nó là lãng phí.

18.3 Đo constellation bằng photogrammetry

Bước hai, và mục này trả lời Q2 và Q4.

A. Ba lựa chọn

Tự viết BA bằng [Ceres](#) [27] hoặc [GTSAM](#) [54]. Đây là cách cho kết quả tốt nhất, vì bạn tận dụng được cấu trúc mà một công cụ tổng quát không biết: bốn góc của một tag *cứng với nhau* theo một hình vuông đã biết cạnh, nên mỗi tag chỉ có sáu bậc tự do chứ không phải mười hai. Ràng buộc ấy giảm mạnh số ẩn và cải thiện điều kiện số. Chi phí: vài trăm dòng.

[COLMAP](#) [51]. Đường tắt thực dụng: nó là một hệ SfM hoàn chỉnh, nó xử lý được ảnh không có thứ tự, và nó trưởng thành. Nhược điểm là nó được thiết kế cho đặc trưng tự nhiên chứ không cho marker, nên bạn phải đưa các góc tag vào như các điểm đã khớp sẵn — làm được nhưng không phải luồng chính của nó.

[tagslam](#) [31]. Được thiết kế đúng cho bài toán này: nó giải factor graph trên nhiều tag và ước lượng cả vị trí tag. Nếu nó chạy được với thiết lập của bạn thì nó là lựa chọn hợp lý nhất. Mức tin C — tiền ẩn — nhưng có mã nguồn và được dùng rộng rãi.

B. Điều còn thiếu sau khi chạy

Đây là câu trả lời cho Q2, và nó là chỗ dễ bỏ sót nhất trong cả chuỗi.

Kết quả BA là tọa độ các góc tag trong một hệ tọa độ *tùy ý* do thuật toán chọn, và nó xác định sai khác một hệ số *tỉ lệ*. Hai thông tin còn thiếu.

Một, tỉ lệ. Ảnh đơn thuần không xác định được kích thước tuyệt đối. Phải neo bằng một khoảng cách đo được — và [B/09](#) mục 5B khuyến nghị đo cạnh của tag *lớn nhất*, vì sai số của phép neo trở thành sai số tỉ lệ của toàn bộ constellation. Đây là chỗ duy nhất trong cả quy trình mà thước cặp còn cần thiết.

Hai, và nó hay bị quên hơn: quan hệ với điểm cập chuẩn. [B/12](#) mục 4 gọi nó là khâu $T_{\text{dock}}^{\text{tag}}$. BA cho bạn các tag ở đâu so với nhau; nó không cho bạn cụm tag ấy ở đâu so với cái lỗ cắm sạc. Cách đo đúng và rẻ nhất, như [B/12](#) mục 4 nêu: đưa robot vào vị trí cập chuẩn rồi đo pose constellation từ đó — pose ấy *định nghĩa* điểm cập chuẩn, không cần thước.

C. Đại lượng phải nhìn trước khi dùng kết quả

Đây là câu trả lời cho Q4: *hiệp phương sai của tọa độ các góc*, mà BA trả về như một sản phẩm phụ.

Cách tự lừa mình nếu bỏ qua nó: bạn thay một con số không biết sai bao nhiêu (từ CAD) bằng một con số khác cũng không biết sai bao nhiêu (từ BA), rồi tin con số thứ hai hơn vì nó “đã được đo”. Nếu BA trả về bất định 2 mm cho một tag — chuyện hoàn toàn có thể khi ảnh ít hoặc góc nhìn nghèo — thì bạn chưa cải thiện gì so với CAD, và bạn đã tiêu một buổi để tin sai.

Phép kiểm cụ thể: so hiệp phương sai BA với dung sai gia công dự kiến. Nếu nó không nhỏ hơn đáng kể thì phải chụp thêm ảnh, đặc biệt là ảnh phủ các tag có bất định lớn từ nhiều góc khác nhau.

Phải kiểm trước khi chuyển sang bước ba: hiệp phương sai đủ nhỏ, và sai số tái chiếu của bài toán hợp nhất không lớn hơn hẳn so với khi giải từng tag riêng ([B/09](#) mục 2).

18.4 Hand-eye

Bước ba.

[OpenCV](#) `calibrateHandEye` — cài nhiều phương pháp chọn được: Tsai–Lenz [55], Park–Martin [56], Daniilidis [57], và vài cái khác. Một mẹo chẩn đoán tốt: *chạy với vài phương pháp khác nhau và so kết quả*. Chúng giải cùng một bài toán bằng các đường khác nhau, nên nếu dữ liệu đủ kích

thích thì chúng phải cho kết quả gần nhau. Nếu chúng khác nhau đáng kể thì dữ liệu của bạn không đủ — và đó là chỉ báo rõ nhất cho điều kiện kích thích ở B/12 mục 5B.

easy_handeye — wrapper ROS với quy trình thu thập dữ liệu. Tiện, nhưng nó không giải quyết được chuyện điều kiện kích thích: nếu robot của bạn chỉ xoay quanh một trục thì thành phần độ cao vẫn không quan sát được bất kể công cụ nào.

Với robot vi sai, nhớ ba cách xử lý ở B/12 mục 5B — và nhớ rằng cách thứ ba (nhận ra rằng bạn không cần nó) thường là cách đúng.

Phải kiểm sau bước ba: kết quả ổn định qua các tập con dữ liệu, và các phương pháp khác nhau cho kết quả gần nhau.

18.5 Đánh giá: chỗ nên tự viết

Mục này trả lời Q3 và Q5, và nó là mục có nội dung riêng nhất của chương.

evo [66] và các công cụ tương tự là công cụ tốt cho việc chúng được thiết kế: so hai quỹ đạo. Chúng có các phép căn chỉnh chuẩn, chúng tính ATE và RPE, và chúng vẽ đồ thị đẹp. Tài liệu tham chiếu về cách dùng chúng cho đúng là [36].

Vì sao chúng không phải công cụ đúng cho nghiệm thu docking — câu trả lời cho Q3: vì bài toán khác về bản chất.

Bài toán quỹ đạo hỏi: hai đường đi này khác nhau bao nhiêu, sau khi đã căn chỉnh tối ưu? Nó có một chuỗi dài các pose và nó quan tâm tới sai số tích lũy.

Bài toán docking hỏi: qua ba mươi lần cập, điểm cuối tản mát bao nhiêu, và nó có lệch so với vị trí đúng không? Nó chỉ quan tâm tới một pose — pose cuối — nhưng qua nhiều lần thử.

Hai câu hỏi ấy cần hai phép tính khác nhau, và phép tính thứ hai đơn giản hơn nhiều: gom các pose cuối, tính trung bình và độ tản mát, và nếu có chuẩn tham chiếu thì so trung bình với nó. Vài chục dòng, và mọi bước đều minh bạch.

Ép *evo* làm việc ấy thì được — coi mỗi lần cập là một “quỹ đạo” một điểm — nhưng bạn phải hiểu rõ nó đang căn chỉnh cái gì, và các phép căn chỉnh của nó được thiết kế cho tình huống khác. Rủi ro không phải là nó không chạy mà là nó chạy và cho một con số mà bạn diễn giải sai.

Nguyên tắc chung rút ra — câu trả lời cho Q5: dùng công cụ có sẵn khi bài toán của bạn đúng là bài toán mà nó được thiết kế cho; tự viết khi bài toán khác, kể cả khi công cụ ấy “làm được”. Ba nhóm đầu của chương thuộc trường hợp thứ nhất — hiệu chuẩn intrinsic, BA, hand-eye đều là đúng bài toán mà công cụ giải. Nhóm thứ tư thuộc trường hợp thứ hai.

Và một lưu ý về cái gì phải đo, lấy từ C/16 mục 4B: đo độ lặp lại của toàn bộ chu trình — lùi ra hẳn rồi cập lại — chứ không đo độ tản mát của pose khi robot đã đứng yên ở vị trí cuối. Hai con số ấy khác nhau nhiều, và chỉ con số thứ nhất có nghĩa với người dùng.

18.6 Giới hạn

Các khẳng định trong chương này chưa được chạy kiểm. Khác với D/17 mục 3B, nơi tôi chạy thật trên OpenCV và phát hiện một cái bẫy, chương này mô tả khả năng của các công cụ từ hiểu biết chung về chúng. Trước khi dựa vào một khẳng định cụ thể — chẳng hạn rằng `calibrateHandEye` có đúng những phương pháp tôi liệt kê — hãy kiểm với tài liệu của phiên bản bạn dùng.

Không có benchmark. Chương không nói công cụ nào cho kết quả chính xác hơn, vì tôi không có dữ liệu so sánh được.

Chốt tháng 9/2026. Cùng cảnh báo như D/17 mục 1.

tagslam ở mức tin C. Tiễn ẩn, chưa qua phản biện; tôi đưa nó vào vì nó có mã nguồn công khai và giải đúng bài toán, nhưng theo `research-rules.md` mục 4 thì nguồn mức C không được đứng một mình làm chỗ dựa.

18.7 Thực hành

Quy trình ba bước, với phép kiểm sau mỗi bước — và các phép kiểm mới là phần đáng giá.

Bước một, intrinsic. Công cụ: OpenCV hoặc Kalibr. Kiểm: bản đồ residual không có cấu trúc; c_x, c_y ổn định qua các tập con.

Bước hai, constellation. Công cụ: Ceres/GTSAM tự viết, hoặc COLMAP, hoặc tagslam. Kiểm: hiệp phương sai đủ nhỏ so với dung sai gia công; và đừng quên neo tỉ lệ cùng khâu T_{dock}^{tag} .

Bước ba, hand-eye. Công cụ: OpenCV `calibrateHandEye`. Kiểm: các phương pháp khác nhau cho kết quả gần nhau; kết quả ổn định qua các tập con.

Và cho đánh giá: tự viết, theo C/16 mục 7.

Active recall (che cột phải, tự trả lời, rồi mở ra đối chiếu)

Thuật ngữ / câu hỏi	Trả lời
••• Ranh giới của chương này với D/17	D/17: thứ chạy <i>mỗi khung</i> (detector, PnP) — thường phải tự cài hoặc tự sửa. Chương này: thứ chạy <i>một lần</i> (hiệu chuẩn, đo đặc, đánh giá) — thường nên dùng đồ có sẵn.
••• Ba bước và ba công cụ	Intrinsic: OpenCV <code>calibrateCamera</code> hoặc Kalibr. Constellation: Ceres/GTSAM tự viết, hoặc COLMAP, hoặc tagslam. Hand-eye: OpenCV <code>calibrateHandEye</code> . Thứ tự <i>bắt buộc</i> theo B/12 mục 2.
••• Vì sao tự viết BA tốt hơn công cụ tổng quát	Vì bạn tận dụng được ràng buộc mà công cụ tổng quát không biết: bốn góc một tag <i>cứng với nhau</i> theo hình vuông đã biết cạnh, nên mỗi tag có 6 bậc tự do chứ không phải 12. Giảm mạnh số ẩn, cải thiện điều kiện số.
••• Hai thứ còn thiếu sau khi chạy BA	Tỉ lệ — ảnh đơn thuần không xác định kích thước tuyệt đối, phải neo bằng cạnh tag <i>lớn nhất</i> . Và khâu T_{dock}^{tag} — BA cho các tag ở đâu so với nhau, không cho cụm ấy ở đâu so với lỗ cấm.
••• Đại lượng phải nhìn sau BA	Hiệp phương sai của tọa độ các góc. Bỏ qua nó là tự lừa mình: thay một con số không biết sai bao nhiêu (CAD) bằng một con số khác cũng không biết sai bao nhiêu (BA), rồi tin cái thứ hai hơn vì nó “đã được đo”.
•• Phép kiểm rẻ nhất cho hand-eye	Chạy với <i>vài phương pháp khác nhau</i> (Tsai–Lenz, Park–Martin, Daniilidis) và so. Chúng giải cùng bài toán bằng các đường khác nhau, nên dữ liệu đủ kích thích thì chúng phải gần nhau. Khác nhau đáng kể $\Rightarrow$ dữ liệu chưa đủ.
••• Vì sao evo không đúng cho docking	Bài toán khác về bản chất. Quỹ đạo: hai đường đi khác nhau bao nhiêu sau khi căn chỉnh tối ưu, quan tâm sai số tích lũy. Docking: qua 30 lần cập, <i>một</i> pose cuối tằn mết bao nhiêu. Hai câu hỏi, hai phép tính.
••• Nguyên tắc chung về chọn công cụ	Dùng đồ có sẵn khi bài toán của bạn <i>đúng là</i> bài toán nó được thiết kế cho; tự viết khi bài toán khác — <i>kể cả khi công cụ ấy “làm được”</i> . Rủi ro không phải nó không chạy mà là nó chạy và bạn diễn giải sai.

Thuật ngữ / câu hỏi	Trả lời (tiếp)
•• Do cái gì khi nghiệm thu	Độ lặp lại của <i>toàn bộ chu trình</i> — lùi ra hẳn rồi cập lại — chứ không phải độ tần mắt của pose khi robot đã đứng yên ở vị trí cuối. Hai con số khác nhau nhiều (C/16 mục 4B).
•• Phép kiểm sau mỗi bước hiệu chuẩn	Sau intrinsic: bản đồ residual không cấu trúc, c_x, c_y ổn định. Sau constellation: hiệp phương sai đủ nhỏ, residual hợp nhất không lớn hơn hẳn từng tag. Sau hand-eye: các phương pháp cho kết quả gần nhau.
• Chỗ duy nhất thước cặp còn cần	Neo tỉ lệ cho BA — đo cạnh của tag <i>lớn nhất</i> , vì sai số của phép neo thành sai số tỉ lệ toàn cục.
• Mức kiểm chứng của chương này	Các khẳng định <i>chưa được chạy kiểm</i> trong lần soạn này, khác với D/17 mục 3B. Kiểm với tài liệu của phiên bản bạn dùng trước khi dựa vào một khẳng định cụ thể.

••• phải thuộc •• nên nắm • biết là có

I Kiến thức này đang sống ở đâu (công trình đang dùng chính ý tưởng của chương; chốt 9/2026)

Công trình / hệ thống	Dùng ý tưởng nào của chương	Ghi chú
Kalibr [11]	Hiệu chuẩn nhiều mô hình méo, có báo cáo residual theo vị trí ảnh	Công cụ duy nhất mặc định cho bạn phép kiểm quan trọng nhất của A/01 mục 6
Ceres [27] / GTSAM [54]	Tự viết BA với ràng buộc tag cứng — cho kết quả tốt nhất	Vài trăm dòng; đáng nếu constellation là dòng lớn trong ngân sách
COLMAP [51]	SfM hoàn chỉnh, đường tắt cho đo constellation	Thiết kế cho đặc trưng tự nhiên; đưa góc tag vào như điểm đã khớp
OpenCV calibrateHandEye	Nhiều phương pháp chọn được [55]–[57]	Chạy vài phương pháp và so — chỉ báo rõ nhất cho điều kiện kích thích
evo [66]	Đánh giá quỹ đạo với các phép căn chỉnh chuẩn [36]	Tốt cho quỹ đạo, <i>không</i> phải công cụ đúng cho độ lặp lại điểm cuối

MINI PROJECT • Chạy chuỗi ba bước hiệu chuẩn với phép kiểm sau mỗi bước • 3–4 ngày

Mục tiêu	có một chuỗi hiệu chuẩn hoàn chỉnh mà mỗi bước đã được <i>kiểm</i> chứ không chỉ đã <i>chạy</i> — khác biệt quyết định giữa một tệp tham số dùng được và một tệp trông như dùng được.
Dữ liệu	phần cứng thật: camera, target hiệu chuẩn phẳng chất lượng, constellation đã lắp, và robot.
Các bước	bước một, hiệu chuẩn intrinsic với ít nhất 30 ảnh có tư thế nghiêng mạnh và phủ tới bốn góc khung hình; vẽ bản đồ residual và chạy lại với các tập con để kiểm c_x, c_y ; <i>không chuyển bước</i> cho tới khi cả hai phép kiểm đạt. Bước hai, chụp ít nhất 30 ảnh constellation từ nhiều góc, chạy BA, neo tỉ lệ bằng cạnh tag lớn nhất, và lấy hiệp phương sai của kết quả; so nó với dung sai gia công; rồi đo khâu $T_{\text{dock}}^{\text{tag}}$ bằng cách đưa robot vào vị trí cập chuẩn. Bước ba, thu thập cặp pose cho hand-eye, chạy với ba phương pháp khác nhau, so kết quả.
Xong khi	bạn có một tệp cấu hình với ba nhóm tham số, và <i>cho mỗi nhóm</i> một con số bất định kèm theo. Kiểm tra chéo bắt buộc: chạy lại toàn bộ chuỗi từ đầu với một bộ ảnh mới và so hai kết quả — nếu chúng khác nhau nhiều hơn bất định đã ước lượng, thì bất định của bạn đang bị đánh giá thấp, và mọi dòng hiệu chuẩn trong bảng ngân sách của C/15 cũng đang bị đánh giá thấp theo.

Nối với C/15 nhận ba con số bất định làm ba dòng của bảng; B/12 mini project là phiên bản sâu hơn của phép kiểm ổn định.

TRẢ LỜI CÂU HỎI MỞ ĐẦU

- A1. Intrinsic: [OpenCV](#) `calibrateCamera` hoặc [Kalibr](#) — cái sau đáng hơn vì nó báo cáo residual theo vị trí ảnh, phép kiểm quan trọng nhất. Constellation: [Ceres](#) hoặc [GTSAM](#) tự viết, hoặc [COLMAP](#), hoặc [tagslam](#). Hand-eye: [OpenCV](#) `calibrateHandEye`. Bước có thể phải tự viết là bước hai, và lý do đáng nói: một công cụ SfM tổng quát không biết rằng bốn góc của một tag *cứng với nhau* theo một hình vuông đã biết cạnh, nên nó xử lý chúng như bốn điểm độc lập — mười hai bậc tự do thay vì sáu. Tự viết cho phép áp ràng buộc ấy, giảm mạnh số ẩn và cải thiện điều kiện số. Với constellation nhỏ thì chênh lệch có thể không đáng, nhưng nếu constellation là dòng lớn trong ngân sách của bạn thì vài trăm dòng code là đầu tư đúng.
- A2. Vì kết quả BA là toạ độ trong một hệ *tuyệt đối* và xác định sai khác một hệ *số tỉ lệ* — ảnh đơn thuần không bao giờ xác định được kích thước tuyệt đối. Thông tin còn thiếu thứ nhất là phép neo tỉ lệ: một khoảng cách đo được bằng thước, và nên đo cạnh của tag *lớn nhất* vì sai số của phép neo trở thành sai số tỉ lệ của toàn bộ constellation. Thông tin còn thiếu thứ hai, và nó hay bị quên hơn: quan hệ giữa hệ constellation và *điểm cập chuẩn thật* — khâu T_{dock}^{tag} của B/12 mục 4. BA cho bạn biết các tag ở đâu so với nhau; nó hoàn toàn không biết cái lỗ cắm sạc ở đâu. Cách đo khâu ấy rẻ và đẹp: đưa robot vào vị trí cập đúng rồi đo pose constellation từ đó — pose ấy *định nghĩa* điểm cập chuẩn, và không cần thước nào cả.
- A3. Vì bài toán khác về bản chất chứ không chỉ về quy mô. [evo](#) được thiết kế cho bài toán *quỹ đạo*: so hai đường đi dài sau khi căn chỉnh tối ưu, với các chỉ số ATE và RPE quan tâm tới sai số tích lũy. Bài toán docking hỏi một câu khác hẳn: qua ba mươi lần cập độc lập, *một* pose — pose cuối — tằn mát bao nhiêu, và nó có lệch so với vị trí đúng không? Đó là một bài toán thống kê trên nhiều lần thử ở một điểm, không phải một bài toán so sánh hai chuỗi. Phép tính đúng đơn giản hơn nhiều: gom các pose cuối, tính trung bình và độ tằn mát, so trung bình với chuẩn tham chiếu nếu có. Ép [evo](#) làm việc ấy thì chạy được, nhưng rủi ro không phải là nó không chạy — rủi ro là nó chạy và cho một con số mà bạn diễn giải sai, vì bạn không biết chính xác nó đã căn chỉnh cái gì.
- A4. Hiệp phương sai của toạ độ các góc, mà BA trả về như một sản phẩm phụ. Cách tự lừa mình nếu bỏ qua nó: bạn bắt đầu với một con số từ CAD mà bạn không biết sai bao nhiêu, bạn chạy BA và được một con số khác mà bạn *cũng* không biết sai bao nhiêu, rồi bạn tin con số thứ hai hơn vì nó “đã được đo”. Nhưng nếu BA trả về bất định 2 mm cho một tag — hoàn toàn có thể khi số ảnh ít, góc nhìn nghèo, hoặc intrinsic chưa tốt — thì bạn chưa cải thiện gì so với CAD, và bạn vừa tiêu một buổi để có được sự tự tin sai chỗ. Phép kiểm cụ thể: so hiệp phương sai BA với dung sai gia công dự kiến; nếu nó không nhỏ hơn đáng kể thì phải chụp thêm ảnh, đặc biệt là ảnh phủ các tag có bất định lớn từ nhiều góc khác nhau.
- A5. Nhóm *đánh giá* nên tự viết; ba nhóm còn lại — hiệu chuẩn intrinsic, đo constellation, hand-eye — nên dùng đồ có sẵn. Nguyên tắc chung: *dùng công cụ có sẵn khi bài toán của bạn đúng là bài toán mà nó được thiết kế cho, và tự viết khi bài toán khác — kể cả khi công cụ ấy “làm được”*. Ba nhóm đầu thuộc trường hợp thứ nhất: hiệu chuẩn camera là đúng bài toán mà `calibrateCamera` giải, BA là đúng bài toán mà `Ceres` giải. Nhóm thứ tư thuộc trường hợp thứ hai: [evo](#) giải bài toán quỹ đạo, còn bạn có bài toán độ lặp lại một điểm. Điều làm nguyên tắc này đáng nhớ là chỗ “*kể cả khi làm được*”: một công cụ ép vào bài toán sai vẫn cho ra số, và số ấy trông hợp lệ — đó mới là rủi ro, chứ không phải chuyện nó báo lỗi.

NÓI LẠI THẬT ĐƠN GIẢN

Chương trước nói về mấy thứ chạy liên tục khi robot đang làm việc. Chương này nói về mấy thứ chạy đúng một lần rồi cho ra một tệp con số.

Có ba việc như vậy, và chúng phải làm đúng thứ tự: đo cái camera trước, rồi đo xem mấy tấm tag thật sự nằm đâu, rồi cuối cùng đo xem cái camera được bắt ở chỗ nào trên robot. Làm ngược thì sai số của bước trước chui vào bước sau và không còn tách ra được.

Điều quan trọng nhất của chương không phải tên công cụ mà là: *sau mỗi bước phải kiểm, chứ không phải chạy xong là tin*. Và phép kiểm thường rất đơn giản — chạy lại với một nửa dữ liệu rồi xem kết quả có đổi không. Con số nào giữ nguyên thì tin được; con số nào nhảy thì nó chưa được xác định, dù phần mềm có in nó ra với năm chữ số thập phân.

Riêng bước đo mấy tấm tag có một cái bẫy đáng nhớ. Bạn chụp mấy chục tấm ảnh, chạy chương trình, và nó cho ra một mô hình ba chiều rất đẹp. Nhưng mô hình ấy chưa dùng được, vì nó thiếu hai thứ. Thứ nhất: ảnh không biết kích thước thật — một cái dock nhỏ chụp gần và một cái dock lớn chụp xa cho ảnh giống hệt nhau. Phải đo một cái cạnh bằng thước để neo lại, và đây là chỗ duy nhất trong cả quy trình mà cây thước còn cần thiết. Thứ hai, và hay quên hơn: chương trình cho biết mấy tấm tag nằm đâu *so với nhau*, nhưng nó hoàn toàn không biết cái lỗ cắm sạc nằm đâu. Cách đo cái đó thì đẹp: cứ đẩy robot vào cắm cho đúng, rồi chụp một tấm — thế là xong, không cần đo gì cả.

Cuối cùng là chuyện chọn công cụ, và có một nguyên tắc đáng nhớ. Dùng đồ có sẵn khi việc của bạn đúng là việc mà người ta làm ra nó để giải quyết. Tự viết khi việc của bạn khác — kể cả khi cái công cụ kia “cũng làm được”. Chỗ nguy hiểm không phải là công cụ báo lỗi; chỗ nguy hiểm là nó chạy trơn tru và cho ra một con số mà bạn hiểu nhầm.

Ví dụ cụ thể: có những công cụ rất tốt để so hai đường đi của robot. Việc của bạn thì khác — bạn cần biết qua ba mươi lần cập dock, cái điểm cuối tảo mất bao nhiêu. Ép cái công cụ kia làm việc này thì được, nhưng viết ba chục dòng tự tính thì đúng hơn và bạn hiểu rõ mình đang tính gì.

Chỗ không rút gọn được: không có cách nào biết trước một công cụ có phù hợp hay không ngoài việc hiểu rõ nó được thiết kế để giải bài toán nào. Tài liệu thường nói nó *làm được gì*, ít khi nói nó *được thiết kế cho gì* — và hai điều đó không giống nhau.

Một câu: *Chạy xong chưa phải xong — chạy lại với nửa dữ liệu rồi xem con số nào nhảy, đó mới là kiểm.*

18.8 Điều gì chứng minh cách hiểu này sai

Chương này có mức kiểm chứng *thấp hơn D/17*, và mục 6 đã ghi rõ lý do: tôi không chạy các công cụ trong lần soạn này. Vì vậy mục này liệt kê những khẳng định đáng nghi nhất.

Khẳng định có hậu quả lớn nhất — *tự viết BA với ràng buộc tag cứng cho kết quả tốt hơn dùng một công cụ SfM tổng quát* — là suy luận của tôi từ việc đếm bậc tự do (sáu thay vì mười hai cho mỗi tag). Nó bị bác bỏ nếu một so sánh thật cho thấy COLMAP với các góc tag đưa vào như điểm đã khớp đạt cùng độ chính xác. Điều đó hoàn toàn có thể khi số ảnh lớn, vì khi ấy thông tin dư thừa bù được cho ràng buộc bị bỏ qua — và nếu vậy thì lời khuyên “tự viết” là lời khuyên tốn công vô ích. Đây là mệnh đề đáng kiểm tra trước khi bỏ ra vài trăm dòng.

Khẳng định thứ hai — *chạy hand-eye với nhiều phương pháp rồi so là chỉ báo cho điều kiện kích thích* — dựa trên lập luận rằng các phương pháp giải cùng bài toán bằng các đường khác nhau nên phải hội tụ khi dữ liệu đủ. Nó bị bác bỏ nếu hai phương pháp cho kết quả khác nhau *ngay cả khi* dữ liệu đủ kích thích, chẳng hạn vì chúng xử lý nhiễu theo những cách khác nhau. Trong trường hợp ấy phép thử cho báo động giả, và phép thử đúng phải là chạy lại với các tập con dữ liệu.

Khẳng định thứ ba — *công cụ đánh giá quỹ đạo không phù hợp cho nghiệm thu docking* — là một mệnh đề về sự phù hợp của công cụ với bài toán, không phải một mệnh đề thực nghiệm. Nó “bị bác bỏ” nếu ai đó chỉ ra một cách dùng *evo* cho bài toán độ lặp lại điểm cuối mà vừa đúng vừa đơn giản hơn vài chục dòng tự viết. Tôi không biết cách nào, nhưng tôi cũng không khẳng định là không có.

18.9 Kết nối

Nối lên trên. [B/12-calibration-cho-docking](#) là chương lý thuyết tương ứng: nó nói *vì sao* ba bước phải theo thứ tự ấy và phải kiểm gì; chương này nói dùng công cụ nào. [B/09-multi-marker-constellation](#) mục 5 là nguồn của mục 3.

Nối ngang. [D/17-thu-vien-va-detector](#) là chương anh em, chia theo ranh giới mỗi-khung so với một-lần.

Nối xuống. [C/15-ngan-sach-sai-so](#) nhận ba con số bất định từ mini project của chương này làm ba dòng của bảng. [C/16-danh-gia-va-nghiem-thu](#) mục 7 là nơi mục 5 dẫn tới: quy trình nghiệm thu tự viết.

Câu hỏi tự kiểm (trả lời không nhìn bài)

1. Nêu ba bước hiệu chuẩn, công cụ cho từng bước, và phép kiểm bắt buộc sau mỗi bước.
2. Vì sao tự viết BA có thể tốt hơn dùng một công cụ SfM tổng quát? Nêu ràng buộc mà công cụ tổng quát không biết.
3. Sau khi BA chạy xong, bạn còn thiếu hai thông tin. Nêu cả hai, và nêu cách đo thông tin thứ hai mà không cần thước.
4. Vì sao bỏ qua hiệp phương sai của kết quả BA là tự lừa mình? Mô tả cơ chế.
5. Nêu phép kiểm rẻ nhất cho điều kiện kích thích của hand-eye.
6. Vì sao công cụ đánh giá quỹ đạo không phải công cụ đúng cho nghiệm thu docking, và rủi ro cụ thể của việc ép nó là gì?
7. Phát biểu nguyên tắc chung về khi nào dùng đồ có sẵn và khi nào tự viết.

LỘ TRÌNH



TÓM TẮT

Phần E chỉ có một chương và nó không dạy khái niệm mới. Nó trả lời câu hỏi thực dụng nhất: *nếu tôi phải làm xong hệ thống docking này, tôi làm gì trước, làm gì sau, và vì sao đúng thứ tự ấy?* Lộ trình gồm tám bước, và điểm đáng chú ý của nó là ba bước đầu không có dòng code nào — chúng là tính toán trên giấy và quyết định cơ khí. Nếu chỉ nhớ một điều: *bước đầu tiên là viết script Monte Carlo cho constellation dự kiến, trước khi mua camera và trước khi đặt gia công tấm marker*. Một buổi chiều ở bước ấy quyết định phần lớn kết quả cuối cùng.

Bảng 18.3. Bản đồ Phần E.

Ch.	Thư mục	Nội dung	Mức
19	19-lo-trinh-toi-thieu	Tám bước từ con số không tới một hệ docking đạt ngưỡng; ba thứ tự sai thường gặp và vì sao chúng sai; dấu hiệu đã xong mỗi bước; ba lộ trình cho ba tình huống; lộ trình này cố ý bỏ gì	●●

Đọc chương này đầu tiên nếu bạn đang có một dự án thật và cần biết bắt đầu từ đâu; đọc cuối cùng nếu bạn đi tuần tự từ Phần A để hệ thống hoá lại.

Lộ trình tối thiểu (The Minimum Path)

PHỤ THUỘC Không bắt buộc đọc gì trước — chương này là bản đồ đường đi chứ không phải một chặng đường. Nếu đi tuần tự từ Phần A thì đây là chương cuối; nếu đang có một dự án thật và cần biết bắt đầu từ đâu thì đây là chương đầu tiên.

TÓM TẮT

Chương này trả lời câu hỏi thực dụng nhất: *nếu tôi phải làm xong hệ thống docking này, tôi làm gì trước, làm gì sau, và vì sao đúng thứ tự ấy?* Lộ trình gồm tám bước, và điểm đáng chú ý nhất của nó là *ba bước đầu không có dòng code nào* — chúng là tính toán trên giấy, mô phỏng, và quyết định cơ khí. Điều ấy không phải một sự sắp xếp cho đẹp mà là hệ quả trực tiếp của luận điểm trung tâm: nếu phần lớn ngân sách nằm ở các dòng hệ thống và ở hình học, thì thứ tự làm việc phải phản ánh điều đó. Nếu chỉ nhớ một điều: *bước đầu tiên là chạy mô phỏng cho constellation dự kiến, trước khi mua camera và trước khi đặt gia công* — một buổi chiều ở đó quyết định phần lớn kết quả cuối cùng, và nó là buổi chiều rẻ nhất trong cả dự án.

19.1 Đặt vấn đề: ba thứ tự sai

Trước khi trình bày thứ tự được chọn, đáng dành một mục cho ba thứ tự *không* được chọn. Cả ba đều là thứ tự người ta thật sự đi, cả ba đều nghe hợp lý, và nhìn rõ chúng hỏng ở đâu là cách nhanh nhất để hiểu vì sao tám bước phía sau lại xếp như thế.

Thứ tự thứ nhất: từ detector đi lên. Chọn một họ tag, cài detector, chạy được, rồi cải thiện dần — thử detector khác, thử tinh chỉnh góc, thử bộ lọc. Lập luận ủng hộ rất chắc: đó là chỗ dữ liệu sinh ra, và nó cho kết quả nhìn thấy được ngay trong ngày đầu.

Chỗ hỏng nằm ở một con số cụ thể. Theo bảng ngân sách ví dụ ở [C/15](#) mục 3, ba dòng nhiễu góc cộng lại đóng góp 1,18 mm trong khi nhóm hệ thống đóng góp 4,93 mm — tức 72% ngân sách nằm ở những dòng mà detector không chạm tới. Người đi theo thứ tự này sẽ dành nhiều tuần cải thiện một phần nhỏ của bài toán, và — đây mới là phần tệ — *họ sẽ thấy tiến bộ thật ở mỗi bước*, nên không có tín hiệu nào bảo họ dừng lại.

Thứ tự thứ hai: từ phần cứng đi ra. Mua camera, gia công tám marker theo bản vẽ, lắp lên robot, rồi mới bắt đầu làm phần mềm. Lập luận ủng hộ cũng chắc: phần cứng có thời gian đặt hàng dài, nên đặt sớm là hợp lý về tiến độ.

Chỗ hỏng là chỗ tổn tiền nhất trong cả chương. Các quyết định hình học — baseline bao nhiêu, có nghiêng tag không, mấy cỡ tag — quyết định phần lớn sai số góc, và [C/16](#) mục 3E đo được rằng chênh lệch giữa một bố trí tốt và một bố trí tồi là *6,7 lần*. Nếu bạn đã gia công một trăm cái dock với bố trí tồi thì không dòng code nào cứu được, và sửa thì tốn một trăm lần chi phí gia công.

Thứ tự thứ ba: từ yêu cầu đi xuống. Nhận yêu cầu “5 mm và 0,5°”, rồi tìm cách đạt nó bằng cách làm mọi khâu chính xác nhất có thể. Lập luận ủng hộ nghe kỹ lưỡng và có trách nhiệm.

Chỗ hỏng tinh vi hơn hai cái trên và nó là chỗ tôi cho là đắt nhất về mặt công sức: *yêu cầu ấy có thể không phải yêu cầu thật*. [B/13](#) mục 1 chỉ ra rằng docking cần *độ lặp lại* chứ không cần *độ chính*

xác, và C/15 mục 5 định lượng chênh lệch: cùng một hệ thống cho 6,81 mm ở chế độ đo tuyệt đối và 1,88 mm ở chế độ teach-by-showing. Người đi theo thứ tự này giải một bài toán khó hơn bài toán thật 3,6 lần, và họ không bao giờ biết vì họ không đặt câu hỏi ấy.

Ba chỗ hỏng có một điểm chung, và điểm chung ấy là nguyên tắc sắp xếp của lộ trình: *cả ba đều bắt đầu làm trước khi biết dòng nào của ngân sách là dòng lớn*. Tám bước dưới đây được sắp để câu hỏi ấy được trả lời trước tiên.

TRỰC GIÁC

Hình dung lộ trình này như việc chuẩn bị cho một chuyến đi xa bằng ô tô.

Cách sai thứ nhất: bắt đầu bằng việc đánh bóng xe. Nó cho cảm giác tiến bộ ngay, xe trông đẹp dần lên mỗi giờ, và nó hoàn toàn không liên quan tới việc bạn có tới nơi hay không.

Cách sai thứ hai: mua xe trước khi biết đường đi thế nào. Nếu đường toàn đèo dốc mà bạn mua một chiếc xe thấp gầm thì không có cách nào sửa ngoài việc mua lại.

Cách sai thứ ba: quyết tâm đi thật nhanh vì nghĩ rằng phải tới trong sáu tiếng, trong khi thật ra không ai yêu cầu thế — bạn chỉ cần tới trước bữa tối.

Cách đúng bắt đầu bằng một việc mà không ai coi là “bắt đầu”: *mở bản đồ ra và xem quãng đường bao xa, đường thế nào*. Nó mất mười lăm phút, nó không tạo ra tiến bộ nhìn thấy được, và nó quyết định mọi thứ còn lại — kể cả việc bạn có cần mua xe mới hay không.

Trong bài toán này, cái bản đồ ấy là bảng ngân sách sai số, và việc mở nó ra là chạy mô phỏng. Một buổi chiều, không tốn gì, và nó nói cho bạn biết dòng nào lớn — tức việc gì đáng làm.

19.2 Nguyên tắc sắp xếp

Một nguyên tắc duy nhất, và nó nghiêm ngặt hơn vẻ ngoài: *mỗi bước chỉ được đặt vào chỗ mà tại đó nó giải quyết dòng lớn nhất còn lại của ngân sách*.

Nguyên tắc ấy khác với hai nguyên tắc tự nhiên hơn. Nó *không* phải từ dễ tới khó, và nó *không* phải theo thứ tự của chuỗi đo. Nó theo thứ tự *đòn bẩy*.

Hệ quả trực tiếp là ba đảo thứ tự so với cách trình bày của cây, và chúng đáng nói rõ vì chúng là chỗ lộ trình này khác cách đọc tuần tự.

Đảo thứ nhất: C/15 và C/16 — ngân sách và mô phỏng — nằm ở Phần C nhưng được làm ở bước 1. Lý do: chúng là công cụ trả lời câu hỏi “dòng nào lớn”, và mọi bước sau phụ thuộc vào câu trả lời ấy.

Đảo thứ hai: B/13 — điều khiển — nằm gần cuối Phần B nhưng được quyết định ở bước 2. Lý do: quyết định teach-by-showing hay không thay đổi toàn bộ bảng ngân sách (C/15 mục 5), nên nó phải được đưa ra trước khi quyết định đầu tư vào đâu.

Đảo thứ ba: B/07 — detector — là chương đầu của Phần B nhưng nằm ở bước 6. Lý do đã nêu ở mục 1: nó có đòn bẩy thấp nhất, và B/07 mục 1 nói thẳng điều đó.

19.3 Tám bước

Mạch vấn đề → giải pháp

1 Vấn đề

Không biết dòng nào của ngân sách là dòng lớn, nên không biết đầu tư vào đâu.

- Giải pháp** **Bước 1 — Lập ngân sách và chạy mô phỏng.** Đọc [A/05](#) và [C/15](#), lập bảng với số ước lượng, rồi chạy script của [C/16](#) cho hai ba phương án bố trí.
- Giả định** bốn quan hệ tỉ lệ của [A/05](#) đúng, và σ ước lượng không sai quá một bậc.
- Giá phải trả** một buổi chiều tới một ngày. Không tốn tiền.
- Hệ quả** bạn biết dòng nào lớn, và bạn có một bố trí constellation có cơ sở. *Không có dòng code nào trong pipeline.*
- 2 Vấn đề** Bảng ngân sách nói không đạt, nhưng có thể bài toán thật dễ hơn bài toán đang giải.
- Giải pháp** **Bước 2 — Quyết định chế độ.** Đọc [B/13](#) mục 1 và 3, quyết định teach-by-showing hay đo tuyệt đối, rồi tính lại bảng cho chế độ đã chọn ([C/15](#) mục 5).
- Giả định** vị trí cập là cố định và lặp lại được — nếu robot phải cập vào nhiều dock khác nhau thì teach-by-showing không áp dụng.
- Giá phải trả** một buổi đọc và tính. Không tốn tiền.
- Hệ quả** ngân sách có thể thu nhỏ 3–4 lần, và nhiều bước sau trở nên nhẹ hơn hẳn.
- 3 Vấn đề** Các quyết định cơ khí không đảo ngược được và chúng quyết định phần lớn sai số góc.
- Giải pháp** **Bước 3 — Chốt thiết kế cơ khí.** Đọc [B/09](#) và [B/13](#) mục 7. Quyết: baseline, góc nghiêng tag, số cỡ tag, vị trí gắn camera (cánh tay đòn L , [C/15](#) mục 4), và có còn dẫn hướng hay không.
- Giả định** mô phỏng ở bước 1 đã bao phủ toàn dải tư thế tiếp cận, không chỉ một điểm.
- Giá phải trả** vài ngày làm việc với kỹ sư cơ khí. Đây là bước tốn tiền, và tiền tiêu ở đây rẻ hơn nhiều so với tiêu ở bước 6.
- Hệ quả** bản vẽ có nệm nghiêng, có baseline đủ, và có thể có còn dẫn hướng — ba thứ mà thêm vào sau thì đắt gấp bội.
- 4 Vấn đề** Mọi phép đo phía sau là chính xác quanh một giá trị sai nếu các hằng số sai.
- Giải pháp** **Bước 4 — Hiệu chuẩn theo đúng thứ tự.** Đọc [B/12](#) và [D/18](#). Chạy intrinsic, rồi BA constellation, rồi hand-eye — *kiểm sau mỗi bước* trước khi chuyển.
- Giả định** chế độ đã chọn ở bước 2; nếu là teach-by-showing thì yêu cầu chuyển từ *độ đúng* sang *độ ổn định* ([B/12](#) mục 7).
- Giá phải trả** vài ngày, và nó đòi kỷ luật hơn kỹ năng.
- Hệ quả** các dòng hệ thống của ngân sách được định giá bằng số đo thay vì phỏng đoán.
- 5 Vấn đề** Ambiguity là lỗi thô: khi nó xảy ra, mọi con số trong ngân sách vô nghĩa.
- Giải pháp** **Bước 5 — Khử ambiguity và dựng giao diện ba trạng thái.** Đọc [A/04](#), [B/10](#), [D/17](#) mục 2. Lấy ρ ra, dẫn ngưỡng bằng mô phỏng, và trả về trạng thái ba giá trị thay vì một pose.
- Giả định** bước 3 đã phá tính đồng phẳng — nếu chưa thì bước này chỉ là giảm thiểu, không phải khử.
- Giá phải trả** một tới hai ngày.
- Hệ quả** hệ biết khi nào nó không biết. *Đây là điều kiện để mọi con số phía sau có nghĩa.*
- 6 Vấn đề** Nhiều định vị góc là dòng ngẫu nhiên lớn nhất còn lại.
- Giải pháp** **Bước 6 — Đo và cải thiện σ .** Đọc [B/07](#), [B/08](#), [B/14](#). Đo σ thật, vẽ đồ thị σ_N theo N , rồi làm theo thứ tự rẻ nhất trước: trung bình nhiều khung, bật tinh chỉnh góc, cân nhắc target lai, cải thiện chiếu sáng.
- Giả định** đồ thị σ_N không bao hoà sớm — nếu nó bao hoà thì bạn còn một dòng thiên lệch chưa tìm ra, và mọi đầu tư ở đây vô ích.

- Giá phải trả** vài ngày. Đây là chỗ mà thứ tự đầu tư quan trọng hơn nỗ lực.
Hệ quả σ giảm vài lần, và quan trọng hơn: nó *ổn định* qua các điều kiện.
- 7 Vấn đề** Robot phải đi tới đích, và cách nó đi quyết định phần nào của ngân sách thật sự tới mũi.
Giải pháp **Bước 7 — Dựng bộ điều khiển.** Đọc B/13 đầy đủ và B/11. Dựng coarse-to-fine ba pha, stop-and-go ở pha cuối, đường tiếp cận thẳng trục, và tiêu chí hội tụ ba điều kiện cùng cơ chế retry có ghi log.
Giả định bước 5 đã cho giao diện ba trạng thái — tiêu chí hội tụ cần nó.
Giá phải trả một tới hai tuần, và phần lớn thời gian là chỉnh chứ không phải viết.
Hệ quả hệ cập được, và mỗi lần thất bại đều có lý do ghi lại.
- 8 Vấn đề** Không biết hệ thật sự đạt hay không, và không biết nó hỏng ở điều kiện nào.
Giải pháp **Bước 8 — Nghiệm thu.** Đọc C/16 mục 4. Đo độ lặp lại của *toàn chu trình* ít nhất 30 lần, vẽ bản đồ sai số theo tư thế, và thử nghiệm độ bền ở bốn điều kiện.
Giả định có một chuẩn tham chiếu — và C/16 mục 4A nhắc rằng chính cơ cấu cập là một chuẩn hợp lệ.
Giá phải trả vài ngày, cộng thời gian chờ để thử ở các điều kiện khác nhau.
Hệ quả bạn có một con số nghiệm thu *gọi đúng tên* — độ lặp lại hay độ chính xác — và một bản đồ chỉ ra vùng nào còn yếu.

(Ước lượng thời gian nói theo bậc và là phán đoán chủ quan của tôi, không phải số đo trên một dự án thật.)

19.4 Dấu hiệu đã xong mỗi bước

Một lộ trình không có tiêu chí dừng thì mỗi bước kéo dài vô hạn. Bảng dưới cho tiêu chí đủ cụ thể để tự chấm.

Bảng 19.1. Tám bước và dấu hiệu đã xong.

Bước	Xong khi
1 Ngân sách và mô phỏng	Có một bảng với số, và bạn <i>nói được tên</i> dòng lớn nhất mà không cần nhìn lại bảng
2 Quyết định chế độ	Có hai bảng ngân sách và một quyết định có lý do viết ra
3 Thiết kế cơ khí	Bản vẽ có nệm nghiêng và baseline theo số của bước 1, và đã được mô phỏng lại sau khi vẽ
4 Hiệu chuẩn	Ba nhóm tham số, mỗi nhóm kèm một con số bất định, và cả ba phép kiểm sau mỗi bước đều đạt
5 Khử ambiguity	Hệ trả về ba trạng thái; ρ có trong log; và bạn biết hệ ở trạng thái không tin cậy bao nhiêu phần trăm thời gian
6 Cải thiện σ	Đồ thị σ_N theo N bám đường dốc $-1/2$ tới N bạn dùng; σ đo ở sáu điều kiện ánh sáng
7 Bộ điều khiển	Cập được 20 lần liên tiếp; mỗi lần retry có lý do trong log
8 Nghiệm thu	Một con số kèm tên đúng, một bản đồ sai số, và bốn kết quả thử độ bền

Đáng chú ý ở dòng 1: tiêu chí không phải “có bảng” mà là *nói được tên dòng lớn nhất*. Lý do là toàn bộ giá trị của bước 1 nằm ở chỗ nó đổi cách bạn phân bổ công sức; nếu bạn có bảng mà không nhớ nó nói gì thì bạn chưa có gì.

19.5 Ba lộ trình cho ba tình huống

Tám bước trên giả định bạn bắt đầu từ con số không. Ba tình huống khác thường gặp hơn.

Tình huống một: đã có hệ đang chạy, sai số quanh 1–2°, không hiểu vì sao.

Đây là tình huống phổ biến nhất, và nó có một chẩn đoán rất khả dĩ. Đọc [A/04](#) trước tiên — nếu pose của bạn *nhảy* giữa hai giá trị với vị trí đứng yên, thì đó là ambiguity và bạn đang ở bước 5 chứ không phải ở đâu khác. Nếu pose *không* nhảy mà lệch trơn, thì đọc [A/01](#) mục 5 và kiểm điểm chính, rồi [B/09](#) mục 5 và kiểm constellation — hai dòng lớn nhất của bảng ví dụ.

Trong cả hai nhánh, việc *không* nên làm đầu tiên là đổi detector.

Tình huống hai: sắp phải đặt gia công, chưa có gì khác.

Làm bước 1, 2, 3 và dừng. Đó là ba bước không có code, chúng tốn chừng một tuần, và chúng quyết định phần lớn kết quả. Mọi thứ khác chờ được; bản vẽ thì không.

Tình huống ba: chỉ muốn hiểu, chưa có dự án.

Đọc bảy chương xương sống theo thứ tự: [A/05](#), [A/04](#), [C/15](#), [B/09](#), [B/13](#), [B/12](#), [C/16](#). Bảy chương ấy chứa phần lớn giá trị của cây, và bốn trong bảy không nói về thuật toán xử lý ảnh — đó là điều đáng nhớ nhất.

19.6 Lộ trình này cố ý bỏ gì

Một lộ trình trung thực phải nói nó bỏ gì.

Nó bỏ phần lớn chiều sâu về detector. [B/07](#) và [B/08](#) bị nén vào một bước, và một người muốn thật sự giỏi về phát hiện marker sẽ thấy điều đó không thoả đáng. Lý do là đòn bẩy, đã bàn ở mục 1 — nhưng nếu bài toán của bạn khác (chẳng hạn marker ngoài trời, điều kiện không kiểm soát được) thì cân bằng đổi.

Nó bỏ IBVS. [B/13](#) mục 2 trình bày nó như một lựa chọn mạnh — nó tránh hẳn ambiguity và bền với sai số hiệu chuẩn — nhưng lộ trình mặc định đi theo PBVS cộng teach-by-showing vì nó dễ gỡ lỗi hơn và vì quỹ đạo dự đoán được. Nếu bước 5 hoặc bước 7 gặp khó khăn kéo dài, IBVS là hướng đáng quay lại.

Nó bỏ phần fuse nhiều cảm biến. [B/11](#) mục 5 nhắc tới factor graph và fuse với IMU, và lộ trình không đi vào đó vì với stop-and-go thì một phép trung bình đơn giản là đủ. Với robot phải đo khi đang di chuyển, phần ấy trở nên cần thiết.

Nó giả định một dock cố định. Toàn bộ lợi ích của bước 2 biến mất nếu robot phải cập vào nhiều dock hoặc dock di động, và khi ấy bạn quay về bài toán đo tuyệt đối với mọi yêu cầu khắt khe của nó.

19.7 Điều gì chứng minh lộ trình này sai

Lộ trình đứng trên một mệnh đề định lượng, và nó thừa hưởng toàn bộ tính không chắc chắn của mệnh đề ấy: *trong một hệ điển hình, các dòng hệ thống và hình học chiếm phần lớn ngân sách*. Con số 72% đến từ bảng ví dụ ở [C/15](#) mục 3, và mọi số trong bảng ấy là ví dụ số học do tôi đặt, không phải số đo của một hệ thật.

Lộ trình bị bác bỏ nếu trên một hệ thực, sau khi đã làm bước 3 và bước 4 cẩn thận, ngân sách vẫn bị chi phối bởi σ . Khi ấy thứ tự đúng là đảo bước 6 lên trước bước 3, và mục 1 của chương này sai về trọng số.

Cách kiểm cho hệ của bạn nằm ngay ở bước 1: nếu bảng ngân sách của bạn — với số đo của bạn — cho thấy nhóm ngẫu nhiên lớn hơn nhóm hệ thống, thì hãy đi theo thứ tự của bảng ấy chứ không theo thứ tự của chương này. *Lộ trình phục vụ bảng, không phải ngược lại.*

Một mệnh đề thứ hai, mềm hơn: *ba bước đầu không có code là đúng thứ tự*. Nó bị bác bỏ nếu trong thực tế, việc dựng một nguyên mẫu chạy được sớm cung cấp thông tin mà phân tích trên giấy không cho — và điều đó hoàn toàn có thể, đặc biệt khi các con số đầu vào của bảng còn rất không chắc chắn. Một thỏa hiệp hợp lý mà tôi không loại trừ: dựng một nguyên mẫu thô *song song* với bước 1, với điều kiện không để nó quyết định thiết kế cơ khí.

19.8 Kết nối

Chương này trở tới mọi chương, nên phần kết nối chỉ nói về ba chỗ đặc biệt.

[C/15-ngan-sach-sai-so](#) và [C/16-danh-gia-va-nghiem-thu](#) là hai chương mà lộ trình đặt ở *bước 1*, dù chúng nằm ở Phần C. Nếu bạn chỉ theo lộ trình mà không đọc gì khác, hãy đọc hai chương ấy.

[B/13-docking-control](#) là chương mà *quyết định* ở bước 2 làm nhẹ toàn bộ các bước sau. Đây là chương có đòn bẩy cao nhất trên mỗi giờ đọc trong cả cây.

[00-map](#) là chỗ quay về nếu lộ trình này có vẻ đi ngược trực giác: bốn hệ quả ở đó là lý do của tám bước ở đây, và nếu một bước có vẻ sai thứ tự thì câu trả lời thường nằm trong một trong bốn hệ quả ấy.

NÓI LẠI THẬT ĐƠN GIẢN

Nếu phải làm xong cái hệ thống này, làm gì trước?

Câu trả lời đi ngược với bản năng: *ba việc đầu tiên không có dòng code nào*.

Việc thứ nhất là ngồi tính xem cái gì đang ăn hết ngân sách. Mười lăm phút tính tay cộng một buổi chiều chạy mô phỏng, và cuối buổi bạn biết dòng nào lớn nhất. Nghe như trì hoãn, nhưng nó là việc duy nhất cho biết những việc sau có đáng làm không.

Việc thứ hai là hỏi lại xem bài toán có thật sự khó như mình tưởng. Robot cần biết nó ở đâu, hay chỉ cần về đúng chỗ nó đã từng ở? Hai câu hỏi ấy khác nhau tới ba lần rưỡi về độ khó, và phần lớn người ta giải câu khó mà không nhận ra có câu dễ.

Việc thứ ba là quyết mấy chuyện cơ khí: mấy tấm tag đặt cách nhau bao xa, có kê nghiêng không, camera bắt ở đâu, có làm cái phễu dẫn hướng không. Đây là mấy quyết định duy nhất trong cả dự án mà sửa sau thì tốn tiền thật.

Sau ba việc đó mới tới code, và thứ tự cũng ngược với bản năng: hiệu chuẩn trước, chống nhìn nhầm sau, rồi mới tới chuyện làm cho phép đo chính xác hơn, rồi mới tới bộ điều khiển, rồi mới tới đo đặc nghiệm thu.

Cái detector — thứ mà hầu hết người ta bắt đầu từ đó — nằm ở chỗ thứ sáu trong tám việc.

Và một lời khuyên cho tình huống phổ biến nhất: bạn đã có hệ chạy rồi, sai số quanh một hai độ, không hiểu vì sao. Hãy nhìn xem con số nó *nhảy* hay nó *lệch đều*. Nhảy giữa hai giá trị trong khi mọi thứ đứng yên thì đó là chuyện hai câu trả lời của cái tag phẳng — chữa bằng cái nêm nghiêng. Lệch đều và trơn thì đi kiểm cái lỗ của camera và kiểm xem mấy tấm tag có nằm đúng chỗ như bản vẽ không.

Trong cả hai trường hợp, việc *không* nên làm đầu tiên là đổi sang một detector khác.

Chỗ không rút gọn được: cái thứ tự này dựa trên một con số — rằng phần lớn ngân sách nằm ở mấy khoản cố định — và con số đó lấy từ một bảng ví dụ chứ không phải từ đo đạc trên hệ thật. Nếu bảng của bạn nói khác thì hãy theo bảng của bạn. Lộ trình phục vụ cái bảng, không phải ngược lại.

Một câu: Ba việc đầu tiên không có dòng code nào, và cái detector nằm thứ sáu trong tám việc.

Câu hỏi tự kiểm (trả lời không nhìn bài)

1. Nêu ba thứ tự sai ở mục 1, và với mỗi cái nêu con số cụ thể cho thấy nó sai ở đâu.
2. Phát biểu nguyên tắc sắp xếp của lộ trình, và nêu ba chỗ nó đảo thứ tự so với cách trình bày của cây.
3. Vì sao B/07 — chương đầu của Phần B — lại nằm ở bước 6?
4. Tiêu chí xong của bước 1 không phải “có bảng” mà là gì, và vì sao?
5. Bạn có một hệ đang chạy với sai số 1,5°. Nêu phép quan sát đầu tiên phải làm, và hai nhánh chẩn đoán mà nó dẫn tới.
6. Nêu ba thứ mà lộ trình cố ý bỏ, và với mỗi thứ nêu tình huống làm nó trở nên cần thiết.
7. Mệnh đề định lượng nào chống đỡ toàn bộ lộ trình, nó đến từ đâu, và điều gì sẽ bác bỏ nó?

Sổ những điều chưa biết

TÓM TẮT

Đây là danh sách công việc học tập của cây, mô phỏng quyển “*Notebook of things I don’t know about*” mà Feynman mở khi làm nghiên cứu sinh ở Princeton. Mỗi mục là một **câu hỏi cụ thể** mà tôi biết là mình chưa trả lời được, chứ không phải một chủ đề chung chung. Nếu chỉ nhớ một điều: sổ này chỉ có giá trị khi được *thu ngắn dần* — mục nào giải quyết xong thì chuyển thành một đoạn trong bài chính và ghi ngày.

Bản đầy đủ và được cập nhật thường xuyên nằm ở tệp 99-chua-biet.md tại gốc cây; bản dưới đây là ảnh chụp tại ngày lập, giữ trong PDF để người đọc bản in cũng thấy được cây này còn nợ những gì. Khi hai bản lệch nhau thì bản .md là bản đúng.

Cây này có một điểm khác với các cây khác trong kho, và nó đáng nói ngay: *một phần nợ kiểm chứng đã được trả*. Script Monte Carlo ở C/16 đã được chạy ngày 2026-09-14, và hai trong bốn quan hệ tỉ lệ của A/05 cùng toàn bộ luận điểm về ambiguity của A/04 đã qua cửa kiểm chứng (b). Nhưng việc chạy ấy cũng *sinh ra* ba câu hỏi mới, và chúng đứng đầu danh sách dưới đây — đó là cách một sổ như thế này nên hoạt động.

Nợ kiểm chứng — ưu tiên cao nhất

Một, phá tính đồng phẳng có thật sự khử ambiguity không? C/16 mục 3C đo được rằng kê nghiêng 20° chỉ cải thiện độ tán mát góc 1,24 lần, trong khi B/09 mục 4 ngụ ý đòn bẩy lớn hơn nhiều. Ba giả thuyết chưa phân biệt được: cấu hình đã đủ tốt sẵn nên lợi ích nhỏ lại; lợi ích thật nằm ở việc *loại lỗi thô* chứ không giảm nhiễu, tức tôi đã đo nhầm đại lượng; hoặc độ nghiêng chưa đủ. Cách giải quyết là mini project của C/16: đo *tỉ lệ chọn nhầm nghiệm* thay vì độ tán mát. Đây là mục quan trọng nhất của sổ vì nó ảnh hưởng tới một khuyến nghị thiết kế cơ khí đã được đưa ra.

Hai, vì sao tag nhô ra theo chiều sâu mạnh hơn nê-m nghiêng hai lần? C/16 mục 3E đo được phương án D cho pitch $0,064^\circ$ so với $0,124^\circ$ của phương án C. Hai giả thuyết: hiệu ứng thật của độ lệch khỏi mặt phẳng lớn hơn (60 mm so với 26 mm — tỉ số đúng bằng hai, một trùng khớp đáng chú ý), hoặc chỉ là hệ quả của việc có thêm bốn góc nữa. Phép thử: so với một phương án năm tag nhưng tất cả đồng phẳng.

Ba, quan hệ $\delta\theta \propto 1/B$ có tuyến tính chính xác không? C/16 mục 3D: từ $B = 0,1$ tới $0,8$ m — tám lần — yaw chỉ cải thiện 3,4 lần. Giải thích hiện tại (khi yaw tốt lên thì trục khác thành ràng buộc) hợp lý nhưng chưa kiểm.

Bốn, toàn cây chưa có một phép đo phần cứng nào. Mọi con số đã kiểm đến từ mô phỏng, và C/16 mục 6 nói rõ mô phỏng không kiểm được hai thứ: các nguồn bị bỏ sót, và tính đúng của mô hình nhiễu. Đây là chỗ nợ lớn nhất của toàn bộ cây.

Năm, refs.bib chưa được đối chiếu với nguồn gốc. Soạn từ trí nhớ; theo learning-rules.md mục 5 thì như vậy là chưa qua cửa kiểm chứng nào. Ưu tiên các công trình mà cây dựa vào nhiều nhất: IPPE, phân tích hai nghiệm của target phẳng, AprilTag 2, bias tâm ellipse, hand-eye, và định lý Brockett.

Câu hỏi về nguyên lý và toán học

Điểm chính sai Δ pixel gây thiên lệch pose bao nhiêu? [A/01](#) mục 5 dẫn ra rằng nó dịch mọi tia nhìn đi Δ/f radian — với $\Delta = 10$ px và $f = 600$ px thì gần một độ — nhưng tôi chưa đo xem bao nhiêu phần bị hấp thụ vào các tham số pose khác. Đây là dòng lớn nhất của bảng ngân sách ví dụ ở [C/15](#), nên nó đáng kiểm soát. Motion blur gây thiên lệch hay chỉ tăng phương sai? [A/06](#) mục 3 lập luận rằng profile độ sáng mất đối xứng nên phép khớp cạnh lệch; nếu hoá ra blur chỉ gây nhiễu thì lập luận cho stop-and-go mất một chân, tuy rolling shutter vẫn còn. Méo làm cong cạnh khuếch đại sai số góc bao nhiêu — [A/01](#) mục 4 dẫn ra cơ chế ba tầng và có ví dụ số cho độ lớn méo, nhưng phần khuếch đại tại giao điểm chưa được đo.

Hai câu hỏi về các con số khuyến nghị chưa kiểm ở [B/09](#): ngưỡng độ lệch khỏi mặt phẳng (khuyến nghị 20% kích thước tag) và tỉ lệ baseline đứng trên ngang (khuyến nghị không dưới một phần ba). Cả hai gắn với mục 1 của sổ này, và [C/16](#) mục 3D đã cho thấy pitch và yaw thật sự tách biệt nên câu hỏi thứ hai giờ đo được.

Câu hỏi về phương pháp và triển khai

Ngưỡng ρ nên đặt ở đâu? [B/10](#) mục 3B nói rõ không có hằng số phổ quát và phải dẫn ra bằng Monte Carlo. [C/16](#) mục 3B cho dữ liệu đầu tiên — ρ biến thiên từ 1,33 tới 66,6 theo góc nghiêng — nhưng chưa có đường cong *tỉ lệ chọn sai theo ρ* , và đường cong ấy mới là thứ cho ngưỡng. IMU loại được nghiệm sai trong trường hợp nào — [B/10](#) mục 5 dẫn ra rằng nó phụ thuộc hướng của trục lật so với trọng lực, chưa kiểm. Saddle point có thật sự chính xác hơn giao hai đường không, và bao nhiêu — [B/08](#) mục 3 lập luận bằng số hướng ràng buộc và được ủng hộ gián tiếp bởi thực hành của cộng đồng hiệu chuẩn, nhưng chưa có so sánh định lượng; đây là mệnh đề nền của đề xuất target lai.

Ba câu hỏi cần phần cứng: teach-by-showing mất hiệu lực ở khoảng cách nào ([B/13](#) mục 3C), chiếu sáng chủ động cải thiện tính ổn định của σ bao nhiêu ([B/14](#) mục 4, hiện chỉ phát biểu định tính), và thứ tự hiệu chuẩn sai gây sai số bao nhiêu ([B/12](#) mục 2 — câu này thực ra kiểm được bằng mô phỏng và nên làm sớm).

Câu hỏi nổi sang nhánh khác

Chương [B/11](#) của cây này chồng lấn đáng kể với Phần B của cây `.. /IMU_Fusion/`. Câu hỏi cụ thể: nguyên tắc “giữ phép đo ở dạng nguyên thuỷ nhất mà mô hình nhiễu còn đúng” ở [B/11](#) mục 4 có tương đương với nguyên tắc tiền tích phân ở cây kia không, hay chúng là hai ý khác nhau? Nếu tương đương thì nên trở sang thay vì viết lại. Và một quan sát đáng theo đuổi: bài toán chọn nghiệm ở [B/10](#) có cấu trúc giống bài toán chọn nghiệm trong khởi tạo thị giác-quán tính — cả hai đều là chọn giữa các nghiệm rời rạc dựa trên một chỉ báo liên tục — nên có thể học được gì đó từ bên kia.

Việc đã làm xong

2026-09-14 — chạy script Monte Carlo. Cửa kiểm chứng (b) cho hai quan hệ tịnh tiến của [A/05](#) và cho toàn bộ luận điểm ambiguity của [A/04](#) đã qua; kết quả ở [C/16](#) mục 3. Đáng ghi lại: trong quá trình chạy tôi phát hiện *hai lỗi của chính script* — một lỗi quy ước hệ quy chiếu, và một lỗi

báo cáo góc bằng con số vô hướng — và cả hai đã được viết vào C/16 mục 2 vì chúng là cạm bẫy chung của mọi mô phỏng loại này, không phải chuyện riêng của tôi.

Nguồn

- [1] JCGM, *Evaluation of Measurement Data — Guide to the Expression of Uncertainty in Measurement (GUM)*. JCGM 100:2008, 2008, mức tin A; chuẩn quốc tế cho ngân sách sai số và phân biệt bias với độ lặp lại.
- [2] B. Triggs, P. F. McLauchlan, R. I. Hartley, et al., “Bundle Adjustment — A Modern Synthesis,” in *Vision Algorithms: Theory and Practice*, mức tin A; tài liệu tham chiếu chuẩn cho BA, gauge freedom và hiệp phương sai, Springer, 2000, pp. 298–372.
- [3] ISO, “ISO 9283:1998 — Manipulating Industrial Robots: Performance Criteria and Related Test Methods,” 1998, mức tin A; định nghĩa chuẩn của độ chính xác (accuracy) và độ lặp lại (repeatability) — nền của mục tách bias khỏi precision.
- [4] D. E. Whitney, “Quasi-Static Assembly of Compliantly Supported Rigid Parts,” *Journal of Dynamic Systems, Measurement, and Control*, vol. 104, no. 1, pp. 65–77, 1982, mức tin A; lý thuyết chamfer và remote center compliance — nguồn của luận điểm dung sai cơ khí thay cho độ chính xác thị giác.
- [5] D. C. Brown, “Close-Range Camera Calibration,” *Photogrammetric Engineering*, vol. 37, no. 8, pp. 855–866, 1971, mức tin A; nguồn gốc của mô hình méo radial-tangential (Brown–Conrady).
- [6] Z. Zhang, “A Flexible New Technique for Camera Calibration,” *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 22, no. 11, pp. 1330–1334, 2000, mức tin A; phương pháp hiệu chuẩn từ target phẳng mà mọi hộp công cụ đang dùng.
- [7] J. Wang and E. Olson, “AprilTag 2: Efficient and Robust Fiducial Detection,” in *IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, mức tin A; quad fitting bằng weighted line fit trên gradient, 2016.
- [8] J. Kannala and S. S. Brandt, “A Generic Camera Model and Calibration Method for Conventional, Wide-Angle, and Fish-Eye Lenses,” in *IEEE Transactions on Pattern Analysis and Machine Intelligence*, mức tin A; mô hình KB cho lens góc rộng, vol. 28, 2006, pp. 1335–1340.
- [9] V. Usenko, N. Demmel, and D. Cremers, “The Double Sphere Camera Model,” in *International Conference on 3D Vision (3DV)*, mức tin B, 2018.
- [10] H. A. Beyer, “Accurate Calibration of CCD-Cameras,” *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 1992, mức tin B; sai số có cấu trúc trong residual hiệu chuẩn.
- [11] P. Furgale, J. Rehder, and R. Siegwart, “Unified Temporal and Spatial Calibration for Multi-Sensor Systems,” in *IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, mức tin A; nền của Kalibr, 2013.
- [12] H. Ha, M. Perdoch, H. Alismail, et al., “Deltile Grids for Geometric Camera Calibration,” in *IEEE International Conference on Computer Vision (ICCV)*, mức tin A; lưới tam giác và tính chỉnh saddle point bậc hai, chính xác hơn checkerboard, 2017.
- [13] R. Hartley and A. Zisserman, *Multiple View Geometry in Computer Vision*, 2nd ed. Cambridge University Press, 2004, mức tin A; nguồn chuẩn cho homography, DLT, phân rã pose, conic.

- [14] S. Garrido-Jurado, R. Muñoz-Salinas, F. J. Madrid-Cuevas, et al., “Automatic Generation and Detection of Highly Reliable Fiducial Markers under Occlusion,” *Pattern Recognition*, vol. 47, no. 6, pp. 2280–2292, 2014, mức tin A; ArUco, sinh từ điển bằng tối ưu khoảng cách Hamming.
- [15] J. Heikkilä, “Geometric Camera Calibration Using Circular Control Points,” *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 22, no. 10, pp. 1066–1077, 2000, mức tin A; nguồn chuẩn cho bias tâm ellipse và cách hiệu chỉnh bằng conic.
- [16] J. Mallon and P. F. Whelan, “Which Pattern? Biasing Aspects of Planar Calibration Patterns and Detection Methods,” *Pattern Recognition Letters*, vol. 28, no. 8, pp. 921–930, 2007, mức tin B; so sánh bias giữa chessboard và target tròn — chỗ dựa cho luận điểm chessboard ít bias hơn.
- [17] T. Collins and A. Bartoli, “Infinitesimal Plane-Based Pose Estimation,” *International Journal of Computer Vision*, vol. 109, no. 3, pp. 252–286, 2014, mức tin A; IPPE — hai nghiệm cho target phẳng, là nền của SOLVEPNP_IPPE trong OpenCV.
- [18] L. Calvet, P. Gurdjos, C. Griwodz, et al., “Detection and Accurate Localization of Circular Fiducials under Highly Challenging Conditions,” in *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, mức tin A; CCTag, vòng tròn đồng tâm định vị chính xác, 2016.
- [19] T. Krajník, M. Nitsche, J. Faigl, et al., “A Practical Multirobot Localization System,” *Journal of Intelligent & Robotic Systems*, vol. 76, pp. 539–562, 2014, mức tin B; WhyCon, marker tròn đồng tâm.
- [20] M. A. Fischler and R. C. Bolles, “Random Sample Consensus: A Paradigm for Model Fitting with Applications to Image Analysis and Automated Cartography,” *Communications of the ACM*, vol. 24, no. 6, pp. 381–395, 1981, mức tin A.
- [21] R. M. Haralick, C.-N. Lee, K. Ottenberg, et al., “Review and Analysis of Solutions of the Three Point Perspective Pose Estimation Problem,” *International Journal of Computer Vision*, vol. 13, no. 3, pp. 331–356, 1994, mức tin A; lịch sử P3P từ Grunert 1841, và phân tích số nghiệm.
- [22] L. Kneip, D. Scaramuzza, and R. Siegwart, “A Novel Parametrization of the Perspective-Three-Point Problem for a Direct Computation of Absolute Camera Position and Orientation,” in *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, mức tin A; P3P dạng đóng, bốn nghiệm, 2011.
- [23] V. Lepetit, F. Moreno-Noguer, and P. Fua, “EPnP: An Accurate $O(n)$ Solution to the PnP Problem,” *International Journal of Computer Vision*, vol. 81, no. 2, pp. 155–166, 2009, mức tin A.
- [24] G. Terzakis and M. Lourakis, “A Consistently Fast and Globally Optimal Solution to the Perspective-n-Point Problem,” in *European Conference on Computer Vision (ECCV)*, mức tin B; SQPnP, có trong OpenCV, 2020.
- [25] J. Solà, J. Deray, and D. Atchuthan, “A Micro Lie Theory for State Estimation in Robotics,” *arXiv:1812.01537*, 2018, mức tin B; tiền ẩn nhưng được dùng rộng rãi làm tài liệu tham chiếu.
- [26] S. Urban, J. Leitloff, and S. Hinz, “MLPnP — A Real-Time Maximum Likelihood Solution to the Perspective-n-Point Problem,” in *ISPRS Annals of the Photogrammetry, Remote Sensing and Spatial Information Sciences*, mức tin B; PnP có trọng số theo hiệp phương sai từng điểm, 2016.
- [27] S. Agarwal, K. Mierle, et al., “Ceres Solver,” <http://ceres-solver.org>, 2012, mức tin A; phần mềm, không phải bài báo.

- [28] D. Oberkampf, D. F. DeMenthon, and L. S. Davis, "Iterative Pose Estimation Using Coplanar Feature Points," *Computer Vision and Image Understanding*, vol. 63, no. 3, pp. 495–511, 1996, mức tin A; hai nghiệm cho điểm đồng phẳng, trước Schweighofer.
- [29] G. Schweighofer and A. Pinz, "Robust Pose Estimation from a Planar Target," *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 28, no. 12, pp. 2024–2030, 2006, mức tin A; phân tích hai cực tiểu của hàm mục tiêu với target phẳng — nguồn gốc của pose ambiguity.
- [30] B. Benligiray, C. Topal, and C. Akinlar, "STag: A Stable Fiducial Marker System," in *Image and Vision Computing*, mức tin B; dùng đường tròn ngoại tiếp để ổn định pose, vol. 89, 2019, pp. 158–169.
- [31] B. Pfrommer and K. Daniilidis, "TagSLAM: Robust SLAM with Fiducial Markers," in *arXiv preprint arXiv:1910.00679*, mức tin C; tiền án nhưng có mã nguồn công khai và được dùng rộng rãi — dùng làm gợi ý về kiến trúc, không làm bằng chứng số, 2019.
- [32] S. M. Kay, *Fundamentals of Statistical Signal Processing: Estimation Theory*. Prentice Hall, 1993, mức tin A; cận Cramér–Rao và ma trận thông tin Fisher.
- [33] R. M. Haralick, "Propagating Covariance in Computer Vision," *International Journal of Pattern Recognition and Artificial Intelligence*, vol. 10, no. 5, pp. 561–572, 1996, mức tin B; công thức Jacobian chuẩn cho lan truyền hiệp phương sai qua bài toán bình phương tối thiểu.
- [34] K. Kanatani, "Statistical Optimization for Geometric Computation: Theory and Practice," 1996, mức tin A; lý thuyết sai số hình học, cận lý thuyết cho ước lượng từ điểm ảnh.
- [35] M. Moakher, "Means and Averaging in the Group of Rotations," *SIAM Journal on Matrix Analysis and Applications*, vol. 24, no. 1, pp. 1–16, 2002, mức tin A; trung bình Karcher trên $SO(3)$, và vì sao trung bình quaternion thô là sai.
- [36] Z. Zhang and D. Scaramuzza, "A Tutorial on Quantitative Trajectory Evaluation for Visual(-Inertial) Odometry," in *IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, mức tin A; căn chỉnh quỹ đạo trước khi tính sai số, và vì sao chọn sai phép căn chỉnh làm con số vô nghĩa, 2018.
- [37] O. Ait-Aider, N. Andreff, J. M. Lavest, et al., "Simultaneous Object Pose and Velocity Computation Using a Single View from a Rolling Shutter Camera," in *European Conference on Computer Vision (ECCV)*, mức tin A; mô hình rolling shutter cho ước lượng pose, và cách biến nó thành thông tin thay vì nhiễu, 2006.
- [38] L. Oth, P. Furgale, L. Kneip, et al., "Rolling Shutter Camera Calibration," in *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, mức tin A; đo line delay bằng target đã biết, 2013.
- [39] J. W. Goodman, "Introduction to Fourier Optics," 2005, mức tin A; cơ sở của MTF và giới hạn nhiễu xạ.
- [40] E. Olson, "AprilTag: A Robust and Flexible Visual Fiducial System," in *IEEE International Conference on Robotics and Automation (ICRA)*, mức tin A; bài gốc, lexicode và quy trình phát hiện, 2011.
- [41] M. Krogus, A. Haggemiller, and E. Olson, "Flexible Layouts for Fiducial Tags," in *IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, mức tin B; April-Tag 3, các family không vuông, 2019.
- [42] S. Garrido-Jurado, R. Muñoz-Salinas, F. J. Madrid-Cuevas, et al., "Generation of Fiducial Marker Dictionaries Using Mixed Integer Linear Programming," *Pattern Recognition*, vol. 51, pp. 481–491, 2016, mức tin B.
- [43] Z. Zhang, Y. Hu, G. Yu, et al., "DeepTag: A General Framework for Fiducial Marker Design and Detection," in *IEEE Transactions on Pattern Analysis and Machine Intelligence*, mức tin B, 2023.

- [44] R. Berral-Soler, F. J. Madrid-Cuevas, R. Muñoz-Salinas, et al., "DeepArUco: Marker Detection and Classification in Challenging Lighting Conditions," *IEEE/CVF Conference on Computer Vision and Pattern Recognition Workshops (CVPRW)*, 2023, mức tin C; tiền ấn/-workshop, dùng làm gợi ý.
- [45] M. Kalaitzakis, B. Cain, S. Carroll, et al., "Fiducial Markers for Pose Estimation: Overview, Applications and Experimental Comparison of the ARTag, AprilTag, ArUco and STag Markers," in *Journal of Intelligent & Robotic Systems*, mức tin B; so sánh thực nghiệm nhiều họ tag — nguồn duy nhất trong cây có số đo trực tiếp so sánh giữa các họ, vol. 101, 2021.
- [46] W. Förstner and E. Gülch, "A Fast Operator for Detection and Precise Location of Distinct Points, Corners and Centres of Circular Features," *ISPRS Intercommission Workshop*, 1987, mức tin A; nguồn của phép tính chỉnh góc bằng tính trực giao gradient, tức cornerSubPix.
- [47] L. Lucchese and S. K. Mitra, "Using Saddle Points for Subpixel Feature Detection in Camera Calibration Targets," *Asia-Pacific Conference on Circuits and Systems*, 2002, mức tin B; khớp bậc hai quanh điểm yên ngựa.
- [48] S. Ghosal and R. Mehrotra, "Orthogonal Moment Operators for Subpixel Edge Detection," *Pattern Recognition*, vol. 26, no. 2, pp. 295–306, 1993, mức tin B; định vị cạnh dưới pixel bằng moment Zernike.
- [49] G. D. Evangelidis and E. Z. Psarakis, "Parametric Image Alignment Using Enhanced Correlation Coefficient Maximization," *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 30, no. 10, pp. 1858–1865, 2008, mức tin A; ECC, căn chỉnh quang trắc bất biến với thay đổi sáng.
- [50] S. Baker and I. Matthews, "Lucas-Kanade 20 Years On: A Unifying Framework," *International Journal of Computer Vision*, vol. 56, no. 3, pp. 221–255, 2004, mức tin A; inverse compositional alignment.
- [51] J. L. Schönberger and J.-M. Frahm, "Structure-from-Motion Revisited," in *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, mức tin A; COLMAP, 2016.
- [52] R. Hartley, J. Trumpf, Y. Dai, et al., "Rotation Averaging," *International Journal of Computer Vision*, vol. 103, no. 3, pp. 267–305, 2013, mức tin A.
- [53] T. D. Barfoot, *State Estimation for Robotics*. Cambridge University Press, 2017, mức tin A; ước lượng trên đa tạp, trung bình trên SO(3).
- [54] F. Dellaert and M. Kaess, "Factor Graphs for Robot Perception," *Foundations and Trends in Robotics*, vol. 6, no. 1-2, pp. 1–139, 2017, mức tin A.
- [55] R. Y. Tsai and R. K. Lenz, "A New Technique for Fully Autonomous and Efficient 3D Robotics Hand/Eye Calibration," *IEEE Transactions on Robotics and Automation*, vol. 5, no. 3, pp. 345–358, 1989, mức tin A; lời giải kinh điển cho $AX=XB$.
- [56] F. C. Park and B. J. Martin, "Robot Sensor Calibration: Solving $AX=XB$ on the Euclidean Group," *IEEE Transactions on Robotics and Automation*, vol. 10, no. 5, pp. 717–721, 1994, mức tin A.
- [57] K. Daniilidis, "Hand-Eye Calibration Using Dual Quaternions," *The International Journal of Robotics Research*, vol. 18, no. 3, pp. 286–298, 1999, mức tin A; giải đồng thời xoay và tịnh tiến.
- [58] K. H. Strobl and G. Hirzinger, "Optimal Hand-Eye Calibration," *IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, 2006, mức tin B; trọng số theo bất định thay vì bình phương tối thiểu thuần.
- [59] F. Chaumette and S. Hutchinson, "Visual Servo Control, Part I: Basic Approaches," *IEEE Robotics & Automation Magazine*, vol. 13, no. 4, pp. 82–90, 2006, mức tin A; tài liệu chuẩn cho IBVS và PBVS, ma trận tương tác.

- [60] B. Espiau, F. Chaumette, and P. Rives, “A New Approach to Visual Servoing in Robotics,” *IEEE Transactions on Robotics and Automation*, vol. 8, no. 3, pp. 313–326, 1992, mức tin A; bài gốc của IBVS, và nguồn của luận điểm IBVS bền với sai số hiệu chuẩn.
- [61] E. Malis, F. Chaumette, and S. Boudet, “2-1/2-D Visual Servoing,” *IEEE Transactions on Robotics and Automation*, vol. 15, no. 2, pp. 238–250, 1999, mức tin A.
- [62] F. Chaumette and S. Hutchinson, “Visual Servo Control, Part II: Advanced Approaches,” *IEEE Robotics & Automation Magazine*, vol. 14, no. 1, pp. 109–118, 2007, mức tin A; hybrid 2.5D và các trường hợp hỏng.
- [63] R. W. Brockett, “Asymptotic Stability and Feedback Stabilization,” *Differential Geometric Control Theory*, pp. 181–191, 1983, mức tin A; định lý Brockett — không có luật phản hồi trơn dừng nào ổn định hoá hệ nonholonomic.
- [64] C. Samson, “Control of Chained Systems: Application to Path Following and Time-Varying Point-Stabilization of Mobile Robots,” *IEEE Transactions on Automatic Control*, vol. 40, no. 1, pp. 64–77, 1995, mức tin A.
- [65] É. Marchand, F. Spindler, and F. Chaumette, “ViSP for Visual Servoing: A Generic Software Platform with a Wide Class of Robot Control Skills,” *IEEE Robotics & Automation Magazine*, vol. 12, no. 4, pp. 40–52, 2005, mức tin A.
- [66] M. Grupp, “evo: Python Package for the Evaluation of Odometry and SLAM,” <https://github.com/MichaelGrupp/evo>, 2017, mức tin B; phần mềm.
- [67] G. Yu, Y. Hu, and J. Dai, “TopoTag: A Robust and Scalable Topological Fiducial Marker System,” in *IEEE Transactions on Visualization and Computer Graphics*, mức tin B, 2021.
- [68] J. DeGol, T. Bretl, and D. Hoiem, “ChromaTag: A Colored Marker and Fast Detection Algorithm,” in *IEEE International Conference on Computer Vision (ICCV)*, mức tin B, 2017.